

从 C++ 到 AI 计算：第三册实践与代码卷

本地量化推理引擎、完整源码、KV Cache、后端优化与验收

CPU Performance Study

本卷是《从 C++ 到 AI 计算：第三册》的配套实践与代码卷，围绕 `linux_cpu_inference` 贯穿项目展开：模型资产、tokenizer、张量布局、int8 kernel、reference decoder、KV cache、trace、7B 账本、服务调度、CPU/GPU 后端边界和工程验收都要落到可构建代码、完整源码、测试和报告。

前言

第三册原理卷把 AI 推理系统拆成模型资产、typed graph、张量布局、量化、KV cache、后端、服务运行时和验收体系。本实践与代码卷只做一件事：把这些概念压到同一个可运行项目里，并把完整源码阅读路径并入同一目录，让读者知道每个概念对应哪段代码、哪条命令、哪种输入、哪份报告和哪条失败路径。

贯穿项目是 books/cpu-volume-3/labs/linux_cpu_inference。它不是玩具目录，而是第三册已经持续维护的本地 CPU 量化推理切片：包含教学模型格式、tokenizer、safetensors manifest gate、int8 linear、packed/AVX2 路径、tiny reference decoder、GPT-2 reference path、KV cache trace、7B compute ledger、serving SLO probe、自研 GPT-2 smoke、真实 small 开源模型 smoke、CLI、benchmark 和 CTest。本卷不维护第二份实现；源码章节通过 [lstinputlisting](#) 直接读取同一份工程文件，所有练习、讲解和完整代码都回到同一份源码。

本卷的学习顺序是从小证据到大系统：先让一个 tiny MLP 算对，再解释张量地址流和 int8 累加；再让 reference decoder、sampler 和 KV cache trace 固定语义；然后接模型导入、shape verifier、memory planner 和 IR lowering；最后用 CPU/GPU 后端边界、工业专项和回归门禁收束。代码走读不再被放到最后当附录，而是插入到相同主题部分中：讲完项目入口就读构建文件和调用链，讲完量化就读公共合同、tiny MLP 和 int8 kernel，讲到真实模型时读 tokenizer、safetensors 和 GPT-2，收束验收时读工具、benchmark、smoke 和测试。每章都要求读者交付代码运行结果或报告字段，而不是只抄概念定义。

核心思想

第三册实践的目标不是“写一个看起来像大模型的 demo”，而是建立一条能被复查的推理证据链：资产能校验，shape 能验证，kernel 能对照，trace 能定位，性能能降级，服务指标能解释，回归门禁能长期维护。

常见缩写和术语先导

缩写	全称	本卷中的含义
AI	Artificial Intelligence	这里特指本地推理工程，不讨论泛泛应用口号。
LLM	Large Language Model	decoder-only 文本生成模型是本卷主线。
KV Cache	Key/Value Cache	attention 历史 K/V 状态，属于请求生命周期。
GEMM	General Matrix Multiply	矩阵乘，是预填充和训练常见核心算子。
GEMV	General Matrix Vector Multiply	矩阵向量乘，是单 token decode 的核心形态。
SIMD	Single Instruction Multiple Data	一条指令处理多个 lane 的 CPU 执行方式。
AVX2	Advanced Vector Extensions 2	x86-64 上的 256-bit SIMD 指令集之一。
FMA	Fused Multiply-Add	乘加融合指令，常出现在浮点 kernel 中。
GGUF	GGML Universal File	llama.cpp 生态常见模型文件，保存权重、量化格式、tokenizer 和模型元数据。
GGML	Georgi Gerganov Machine Learning	llama.cpp 生态底层张量、量化 block 和 CPU kernel 体系；本卷按它的 block 布局读取 Q4_K/Q5_0/Q6_K/Q8_0 。
Q4_K/Q5_0 Q6_K/Q8_0	Quantized GGML formats	GGML 量化权重格式名。数字表示主要位宽，后缀表示 block 布局族；它们不是 C++ 类型名，进入代码前必须先经过格式 verifier。
RMSNorm	Root Mean Square Normalization	Transformer 中按 hidden 维做均方根归一化的层。
RoPE	Rotary Position Embedding	旋转位置编码，把位置信息注入 Q/K 向量。
GQA	Grouped-Query Attention	多个 query head 共享一组 KV head 的 attention 形状。
SwiGLU	Swish-Gated Linear Unit	LLaMA-like MLP 常用的门控激活结构。
SLO TTFT TPOT	Service Level Objective Time To First Token Time Per Output Token	服务目标，例如 TTFT、TPOT、p95/p99。从请求进入到首 token 输出的时间。decode 阶段每个输出 token 的平均或尾部时间。
IR	Intermediate Representation	编译器内部表示，用于 pass、lowering 和计划生成。
RAG	Retrieval-Augmented Generation	检索增强生成，需要把外部文档和 prompt 编排起来。
MoE	Mixture of Experts	多专家模型，会改变路由、负载和内存账本。
LoRA	Low-Rank Adaptation	低秩适配权重，常用于微调和多租户加载。

这些缩写不要死记。每个缩写在正文里都要落到一个代码对象或报告字段：KV cache 对应 trace 里的 slot 和 position，SLO 对应 serving probe 的 CSV，IR 对应 executable plan，SIMD/AVX2 对应 kernel 反汇编和 benchmark，GGUF/GGML

对应 manifest、tensor bytes、direct/fallback tensor count 和 caw A/B 报告。

本卷结构

本卷按第三册原书的四个大块组织，但目录不再采用“前面做实践、后面贴源码”的割裂方式。每个大块都把实践任务、代码走读和验收证据放在同一条主线里：[linux_cpu_inference](#)。

第一部分从模型资产到量化 kernel。你会运行 tiny MLP，先读构建入口、项目边界和调用链总图，再检查 tokenizer 和 safetensors manifest，手算张量地址，比较 scalar、packed 和 AVX2 路径，并在公共合同、tiny MLP 与 int8 kernel 源码里解释 scale、accumulator 和 requantize 的边界。

第二部分进入 Transformer、KV cache、sampler 和服务运行时。你会跑 reference decoder trace，解释每个 checkpoint 的 shape、stride、dtype 和 layout，构造 KV cache 状态表，分析 admission、queue wait、TTFT、TPOT、取消和拒绝。

第三部分处理模型导入、真实模型加载闭环、memory planner 和 IR lowering。你会把 manifest、shape verifier、tensor role、state edge、workspace、liveness 和 executable plan 写成可检查报告，并在同一部分中读 reference decoder、tokenizer、safetensors 和 GPT-2 reference path 的完整实现，显式运行 LCQI 自研 GPT-2 safetensors smoke 和一个 small 级别开源 GGUF 模型 smoke，而不是只停留在 tiny fixture。

第四部分收束到后端优化和验收。你会比较 CPU kernel 证据、GPU 后端边界、工业专项能力和回归门禁，并直接读工具、smoke、benchmark、ledger、serving probe 和测试源码。最终交付不是“跑通一次”，而是可复查的工程档案：命令、输入、输出、trace、benchmark、不可验证边界和回归规则。

目录

前言	ii
常见缩写和术语先导	iii
本卷结构	v
第一部分 推理链路、张量账本与量化	
第 1 章实践：端到端推理链路和项目总入口	2
一、对应原书问题：概念必须落到对象和命令	2
二、从特例推到规律：先抓一个能手算的切片	2
三、实践任务：把观察固定成报告字段	2
四、验收和递进大作业	2
代码走读：构建入口、项目边界与调用链总图	4
源码阅读覆盖范围	4
根构建入口	4
LCQI 目标定义	7
项目说明与模型资产	11
为什么要先讲调用链	20
一、全工程入口：哪些文件决定能不能复现	20
二、Tiny MLP 调用链：最短闭环怎样跑通	21
三、Int8 Kernel 调用链：reference、packed、AVX2 和 benchmark 怎样对照	22
四、Tokenizer、Safetensors 与 Reference Decoder：真实模型前的三道门	25
五、GPT-2 调用链：从 HuggingFace 资产到生成 token	26
六、报告、服务探针、外部对标和测试	27
第 2 章实践：张量布局、地址流与第一个 MatVec	28
一、对应原书问题：概念必须落到对象和命令	28
二、从特例推到规律：先抓一个能手算的切片	28
三、实践任务：把观察固定成报告字段	28
四、验收和递进大作业	28
第 3 章实践：GEMM、packing 与 SIMD 证据	30
一、对应原书问题：概念必须落到对象和命令	30
二、从特例推到规律：先抓一个能手算的切片	30
三、实践任务：把观察固定成报告字段	30
四、验收和递进大作业	30
第 4 章实践：int8 量化、累加器与重新量化	31
一、对应原书问题：概念必须落到对象和命令	31
二、从特例推到规律：先抓一个能手算的切片	31
三、实践任务：把观察固定成报告字段	31
四、验收和递进大作业	31
代码走读：公共合同、Tiny MLP 与 Int8 Kernel	32
为什么先读头文件	32
Tiny MLP 与推理入口	32
Int8 Kernel、Tokenizer 与 Safetensors	33
GGUF 与 Q4_K 合同	39
Reference Decoder 与 GPT-2 合同	48
最短闭环：从模型文件到 logits	57
模型加载与基础推理	57
Int8 Kernel 基础实现	63
第二部分 Transformer、KV Cache 与服务运行时	
第 5 章实践：Transformer 切片与 KV Cache 状态	75
一、对应原书问题：概念必须落到对象和命令	75
二、从特例推到规律：先抓一个能手算的切片	75
三、实践任务：把观察固定成报告字段	75
四、验收和递进大作业	75

第 6 章实践：Reference Decoder、logits 与采样	77
一、对应原书问题：概念必须落到对象和命令	77
二、从特例推到规律：先抓一个能手算的切片	77
三、实践任务：把观察固定成报告字段	77
四、验收和递进大作业	77
第 7 章实践：服务调度、SLO 与资源预算	79
一、对应原书问题：概念必须落到对象和命令	79
二、从特例推到规律：先抓一个能手算的切片	79
三、实践任务：把观察固定成报告字段	79
四、验收和递进大作业	79
第 8 章实践：RAG、Agent 与状态图边界	81
一、对应原书问题：概念必须落到对象和命令	81
二、从特例推到规律：先抓一个能手算的切片	81
三、实践任务：把观察固定成报告字段	81
四、验收和递进大作业	81
第三部分 模型导入、AI 编译器与内存计划	
第 9 章实践：模型导入、manifest 与 Shape Verifier	83
一、对应原书问题：概念必须落到对象和命令	83
二、从特例推到规律：先抓一个能手算的切片	83
三、实践任务：把观察固定成报告字段	83
四、验收和递进大作业	83
第 10 章实践：真实模型加载闭环与首 Token Trace	85
一、对应原书问题：概念必须落到对象和命令	85
二、从特例推到规律：先抓一个能手算的切片	85
三、自研 GPT-2 reference smoke：先把标准 safetensors 跑通	85
四、真实 small 模型 smoke：不用 tiny，先让开源模型实际跑起来	86
五、实践任务：把观察固定成报告字段	87
六、验收和递进大作业	87
代码走读：Reference Decoder、Tokenizer、Safetensors 与 GPT-2	89
从 tiny MLP 走向 decoder-only 模型	89
Tiny Reference Decoder	89
Tokenizer 与 Safetensors 实现	106
GPT-2 Reference Path	122
一次可审查的 cached-KV 优化	123
第 11 章实践：运行时内存计划、liveness 与 Workspace	201
一、对应原书问题：概念必须落到对象和命令	201
二、从特例推到规律：先抓一个能手算的切片	201
三、实践任务：把观察固定成报告字段	201
四、验收和递进大作业	201
第 12 章实践：IR、Pass、Lowering 与 Executable Plan	203
一、对应原书问题：概念必须落到对象和命令	203
二、从特例推到规律：先抓一个能手算的切片	203
三、实践任务：把观察固定成报告字段	203
四、验收和递进大作业	203
第四部分 后端优化、工业专项与验收	
第 13 章实践：CPU 后端、微内核与性能证据	206
一、对应原书问题：概念必须落到对象和命令	206
二、从特例推到规律：先抓一个能手算的切片	206
三、实践任务：把观察固定成报告字段	206
四、验收和递进大作业	207
第 14 章实践：GPU 后端边界、copy 与同步成本	208
一、对应原书问题：概念必须落到对象和命令	208
二、从特例推到规律：先抓一个能手算的切片	208
三、实践任务：把观察固定成报告字段	208
四、验收和递进大作业	208
第 15 章实践：工业 LLM 专项能力对照	210
一、对应原书问题：概念必须落到对象和命令	210
二、从特例推到规律：先抓一个能手算的切片	210
三、实践任务：把观察固定成报告字段	210
四、验收和递进大作业	210

第 16 章实践：工程验收、回归门禁与最终项目	211
一、对应原书问题：概念必须落到对象和命令	211
二、从特例推到规律：先抓一个能手算的切片	211
三、实践任务：把观察固定成报告字段	211
四、验收和递进大作业	211
代码走读：工具、Smoke、Benchmark 与回归测试	213
工具不是附属品	213
Benchmark、Trace、Ledger 与 Serving Probe	213
外部模型 Smoke 与对比脚本	342
完整测试	424
实践与代码总索引	479
一、实践主题到代码走读的合并位置	479
二、最小复现命令	479
术语表	480

第零部分

推理链路、张量账本与量化

第 1 章实践：端到端推理链路与项目总入口

本章对应原书中的第三册“端到端链路：从模型文件到下一个 token”。不要把它当作概念复述，本章要把模型目录、manifest、tokenizer、typed graph、backend plan、request session、KV cache、oracle 和 profiler 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `tiny_mlp_i8.txt`，核心命令或测试入口是 `lcqi_cli`。

Listing 1.1 第 1 章实践：端到端推理链路与项目总入口的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  <- DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / tiny_mlp_i8.txt
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：只跑一个 tiny MLP 推理请求。读者要先手写预期结果，再运行项目观察输出。动作是：把 `readfiles->tokenize->run_inference->logits->predicted_class` 串起来。如果输出里没有 `predicted_class` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型文件存在不等于推理链路可信；只有 CLI 输出、测试断言、输入合同和失败路径同时成立，才算完成入口闭环这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：最终大作业的总目录：每章报告都放入 `reports/volume3-practice/`，第一章提交项目地图、命令记录和一次成功/失败输入对

照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：构建入口、项目边界与调用链总图

源码阅读覆盖范围

LCQI 的源码边界由三个层次组成。第一层是仓库构建入口，它决定 CMake 怎样把第三册的实验代码纳入构建和测试。第二层是 `labs/linux_cpu_inference`，它包含真正的库、CLI、benchmark、trace、ledger、serving probe 和测试目标。第三层是工具脚本，它们下载或复用外部模型资产，生成 smoke 和 benchmark 报告。

源码阅读任务

先读本章的 CMake 文件，再读 README。读者要能回答三个问题：一是 `lcqi` 静态库包含哪些实现文件；二是哪些可执行文件只依赖自研代码，哪些依赖可选外部库；三是真实 GPT-2 和 SmolLM2 smoke 为什么不进入默认 CTest。

根构建入口

第三册根目录的 CMake 文件只做项目级选择：是否构建第三册 labs，以及如何把 `labs/linux_cpu_inference` 接入。它应该保持薄层，避免把具体目标定义分散到多个地方。

Listing 1.2 `cpu-volume-3/README.md`

```
1 # 从 C++ 到 AI 计算：第三册
2
3 第三册用于承接 AI、AI 编译器、算子开发和本地大模型推理引擎。第二册只讨论硬件原
  ↳ 理、多核、高性能计算、并发、锁、无锁数据结构、并行算法和分布式计算，不再
  ↳ 直接进入 AI。
4
5 本册贯穿项目是一台能推理开源 7B 左右 decoder-only 大模型的本地引擎，并把 AI 编
  ↳ 译器作为引擎内部主线：模型图导入、typed tensor IR、算子融合、内存规划、
  ↳ CPU/GPU lowering、kernel selection 和 runtime dispatch 都要能落回编译原理
  ↳ 与底层机器账本。工程路线从 CPU 上可解释的 reference 和量化切片开始，逐步
  ↳ 扩展到 tokenizer、模型加载、Transformer 层、KV cache、CPU SIMD/多线程后
  ↳ 端、GPU 后端、正确性报告和性能对标。
6
7 正式书稿工程位于：
8
9 ```text
10 source/latex/
11 ```
12
13 构建命令：
14
15 ```bash
16 make -C source/latex check
17 make -C source/latex pdf
18 ```
19
20 生成文件：
21
```

```

22 ```text
23 source/latex/main.pdf
24 ```
25
26 当前保留的第一阶段切片：
27
28 ```text
29 labs/linux_cpu_inference/
30 ```
31
32 该目录目前包含最小 MLP/int8 权重教学切片、tiny reference decoder、GPT-2 ↵
    ↵ reference path、int8 scalar baseline、packed layout、AVX2 SIMD 路径和 ↵
    ↵ shape sweep benchmark，用来验证张量、线性层、量化权重、tokenizer、↵
    ↵ safetensors、GPT-2 forward、KV cache、oracle 和 benchmark 的基本边界。当↵
    ↵ 前 x86-64 AVX2 结果见：
33
34 ```text
35 results/lcqi-x86-avx2-benchmark-2026-06-25.csv
36 results/lcqi-x86-avx2-decode4096-2026-06-25.csv
37 ```
38
39 自研 GPT-2 smoke 入口如下：
40
41 ```bash
42 make cpu3-gpt2-smoke
43 make cpu3-gpt2-benchmark-compare
44 ```
45
46 第一条命令下载 `openai-community/gpt2` 的标准 safetensors 资产到用户缓存目录，↵
    ↵ 用 LCQI C++ reference path 生成 1 个 token，并把报告写入：
47
48 ```text
49 results/lcqi-gpt2-smoke.txt
50 ```
51
52 第二条命令用同一个 GPT-2 F32 模型对比 LCQI full-prefix 和 cached-KV 生成路径，↵
    ↵ 并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考，报告写入：
53
54 ```text
55 results/lcqi-gpt2-benchmark-compare.txt
56 ```
57
58 它仍然不是生产级 AI Infra。生产级门禁见：
59
60 ```text
61 reports/ai-infra-maturity-audit.md
62 ```
63
64 第三册后续必须把 lab 推进到“小型 CPU 推理 runtime + 手写高性能算子 + 严格 ↵
    ↵ benchmark 报告 + 真实系统对标”，否则只能证明系统学习，不能证明生产级能↵
    ↵ 力。

```

Listing 1.3 cpu-volume-3/CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.31)
2 project(CPUVolume3 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9 option(CPU_VOLUME_3_BUILD_LABS "Build CPU Volume 3 labs" ON)
10
11 if(CPU_VOLUME_3_BUILD_LABS)
12     enable_testing()
13     add_subdirectory(labs/linux_cpu_inference)
14 endif()

```

实践与代码卷也提供一个薄 CMake 入口，它把同一份 LCQI 源码作为可测试对象接入。这样本卷不会复制实现，只复用第三册主工程。

Listing 1.4 cpu-volume-3-practice/CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.31)
2 project(CPUVolume3Practice LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9 enable_testing()
10 add_subdirectory(..cpu-volume-3/labs/linux_cpu_inference linux_cpu_inference)

```

本卷自身的 Makefile 负责构建 PDF，并通过 `test` 目标回到第三册真实 LCQI 工程做 CMake/CTest 验证。实践与代码卷如果只排版、不测试，就不能宣称“完整源码可运行”。

Listing 1.5 cpu-volume-3-practice/Makefile

```

1 LATEX_DIR := source/latex
2
3 .PHONY: all check pdf test clean
4
5 all: pdf
6
7 check:
8     $(MAKE) -C $(LATEX_DIR) check
9
10 pdf:
11     $(MAKE) -C $(LATEX_DIR) pdf
12
13 test:
14     cmake -S . -B build/lcqi-debug -DCMAKE_BUILD_TYPE=Debug

```

```

15     cmake --build build/lcqi-debug
16     ctest --test-dir build/lcqi-debug --output-on-failure
17
18 clean:
19     $(MAKE) -C $(LATEX_DIR) clean

```

Listing 1.6 cpu-volume-3-practice/source/latex/Makefile

```

1 LATEXMK ?= latexmk
2 ENGINE ?= -xelatex
3 MAIN ?= main.tex
4
5 .PHONY: all pdf clean check
6
7 all: pdf
8
9 pdf:
10     $(LATEXMK) $(ENGINE) -interaction=nonstopmode -halt-on-error $(MAIN)
11
12 clean:
13     $(LATEXMK) -C
14
15 check:
16     @python3 scripts/check_inputs.py
17     @command -v xelatex >/dev/null 2>&1 || { echo "missing xelatex"; exit 1; }
18     @command -v latexmk >/dev/null 2>&1 || { echo "missing latexmk"; exit 1; }

```

LCQI 目标定义

linux_cpu_inference/CMakeLists.txt 是代码走读中的第一份核心文件。这里能看到 lcqi 库、lcqi_cli、lcqi_bench、lcqi_trace、lcqi_gpt2、lcqi_ledger、lcqi_serving_probe、可选 lcqi_onednn_bench 和 lcqi_tests 的完整关系。

Listing 1.7 linux_cpu_inference/CMakeLists.txt

```

1 find_package(Threads REQUIRED)
2
3 add_library(lcqi STATIC
4     src/f32_kernels.cpp
5     src/ggml_matvec.cpp
6     src/ggml_tensors.cpp
7     src/gguf.cpp
8     src/gpt2_reference.cpp
9     src/inference.cpp
10    src/int8_kernels.cpp
11    src/llama_gguf.cpp
12    src/model.cpp
13    src/q4_k.cpp
14    src/q5_0_packed.cpp
15    src/reference_decoder.cpp
16    src/safetensors.cpp
17    src/tokenizer.cpp

```

```

18 )
19
20 if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|i[3-6]86")
21     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
22         target_sources(lcqi PRIVATE src/f32_kernels_avx2.cpp)
23         target_sources(lcqi PRIVATE src/ggml_matvec_avx2.cpp)
24         target_sources(lcqi PRIVATE src/int8_kernels_avx2.cpp)
25         target_sources(lcqi PRIVATE src/q4_k_avx2.cpp)
26         target_sources(lcqi PRIVATE src/q5_0_packed_avx2.cpp)
27         set_source_files_properties(src/f32_kernels_avx2.cpp PROPERTIES ↵
↵ COMPILE_OPTIONS "-mavx2;-mfma")
28         set_source_files_properties(src/ggml_matvec_avx2.cpp PROPERTIES ↵
↵ COMPILE_OPTIONS "-mavx2")
29         set_source_files_properties(src/int8_kernels_avx2.cpp PROPERTIES ↵
↵ COMPILE_OPTIONS "-mavx2;-mfma")
30         set_source_files_properties(src/q4_k_avx2.cpp PROPERTIES ↵
↵ COMPILE_OPTIONS "-mavx2")
31         set_source_files_properties(src/q5_0_packed_avx2.cpp PROPERTIES ↵
↵ COMPILE_OPTIONS "-mavx2")
32         target_compile_definitions(lcqi PRIVATE LCQI_ENABLE_AVX2=1)
33     endif()
34 endif()
35
36 if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
37     message(WARNING "LCQI is written for the book's local Linux CPU target; ↵
↵ current system is ${CMAKE_SYSTEM_NAME}.")
38 endif()
39
40 target_include_directories(lcqi
41     PUBLIC
42     ${CMAKE_CURRENT_SOURCE_DIR}/include
43 )
44
45 target_compile_features(lcqi PUBLIC cxx_std_20)
46 target_link_libraries(lcqi PUBLIC Threads::Threads)
47
48 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
49     target_compile_options(lcqi PRIVATE -Wall -Wextra -Wpedantic)
50 endif()
51
52 add_executable(lcqi_cli
53     src/main.cpp
54 )
55
56 target_link_libraries(lcqi_cli PRIVATE lcqi)
57
58 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
59     target_compile_options(lcqi_cli PRIVATE -Wall -Wextra -Wpedantic)
60 endif()
61
62 add_executable(lcqi_bench
63     src/bench.cpp

```



```
64 )
65
66 target_link_libraries(lcqi_bench PRIVATE lcqi)
67
68 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
69     target_compile_options(lcqi_bench PRIVATE -Wall -Wextra -Wpedantic)
70 endif()
71
72 add_executable(lcqi_gguf_q4_bench
73     src/gguf_q4_bench.cpp
74 )
75
76 target_link_libraries(lcqi_gguf_q4_bench PRIVATE lcqi)
77
78 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
79     target_compile_options(lcqi_gguf_q4_bench PRIVATE -Wall -Wextra -Wpedantic)
80 endif()
81
82 add_executable(lcqi_trace
83     src/trace.cpp
84 )
85
86 target_link_libraries(lcqi_trace PRIVATE lcqi)
87
88 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
89     target_compile_options(lcqi_trace PRIVATE -Wall -Wextra -Wpedantic)
90 endif()
91
92 add_executable(lcqi_gpt2
93     src/gpt2_cli.cpp
94 )
95
96 target_link_libraries(lcqi_gpt2 PRIVATE lcqi)
97
98 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
99     target_compile_options(lcqi_gpt2 PRIVATE -Wall -Wextra -Wpedantic)
100 endif()
101
102 add_executable(lcqi_llama_gguf
103     src/llama_cli.cpp
104 )
105
106 target_link_libraries(lcqi_llama_gguf PRIVATE lcqi)
107
108 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
109     target_compile_options(lcqi_llama_gguf PRIVATE -Wall -Wextra -Wpedantic)
110 endif()
111
112 add_executable(lcqi_ledger
113     src/ledger.cpp
114 )
115
```

```

116 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
117     target_compile_options(lcqi_ledger PRIVATE -Wall -Wextra -Wpedantic)
118 endif()
119
120 add_executable(lcqi_serving_probe
121     src/serving_probe.cpp
122 )
123
124 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
125     target_compile_options(lcqi_serving_probe PRIVATE -Wall -Wextra -Wpedantic)
126 endif()
127
128 find_package(dnnl QUIET)
129 if(dnnl_FOUND)
130     add_executable(lcqi_onednn_bench
131         src/onednn_bench.cpp
132     )
133     target_link_libraries(lcqi_onednn_bench PRIVATE DNNL::dnnl)
134     target_compile_features(lcqi_onednn_bench PRIVATE cxx_std_20)
135     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
136         target_compile_options(lcqi_onednn_bench PRIVATE -Wall -Wextra -↵↵
137             ↵↵ Wpedantic)
138     endif()
139 endif()
140
141 add_executable(lcqi_tests
142     tests/lcqi_tests.cpp
143 )
144 target_link_libraries(lcqi_tests PRIVATE lcqi)
145
146 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
147     target_compile_options(lcqi_tests PRIVATE -Wall -Wextra -Wpedantic)
148 endif()
149
150 add_test(NAME lcqi_tests COMMAND ${CMAKE_TARGET_FILE:lcqi_tests})
151 add_test(NAME lcqi_gpt2_tiny COMMAND ${CMAKE_TARGET_FILE:lcqi_gpt2})
152 set_tests_properties(lcqi_gpt2_tiny PROPERTIES
153     PASS_REGULAR_EXPRESSION "generated_ids 1 2 3 3 3"
154 )
155 add_test(NAME lcqi_trace COMMAND ${CMAKE_TARGET_FILE:lcqi_trace})
156 set_tests_properties(lcqi_trace PROPERTIES
157     PASS_REGULAR_EXPRESSION "tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4"
158 )
159 add_test(NAME lcqi_ledger COMMAND ${CMAKE_TARGET_FILE:lcqi_ledger})
160 set_tests_properties(lcqi_ledger PROPERTIES
161     PASS_REGULAR_EXPRESSION "bandwidth_limit,int4_at_100_gbps,23.602,tokens/s"
162 )
163 add_test(NAME lcqi_serving_probe COMMAND ${CMAKE_TARGET_FILE:lcqi_serving_probe})
164 set_tests_properties(lcqi_serving_probe PROPERTIES
165     PASS_REGULAR_EXPRESSION "request,req_too_long,0,memory_budget_exceeded"
166 )

```

项目说明与模型资产

README 是复现实验的入口。它写清楚当前能力、后续边界、构建命令、tiny 推理命令、benchmark 字段、reference trace 字段、GPT-2 smoke、7B ledger、serving SLO probe 和 SmolLM2 small smoke。读源码前先读 README，可以避免把 tiny fixture、GPT-2 reference path 和外部 llama.cpp smoke 混成同一个能力等级。

Listing 1.8 linux_cpu_inference/README.md

```

1 # Linux CPU Quantized Inference
2
3 这是第三册贯穿项目的第一阶段切片：Linux 本地 CPU 量化线性层和最小推理流程。第三
  ↳ 册的贯穿目标不是这个 MLP 本身，而是一台能推理开源 7B 左右 decoder-only 大
  ↳ 模型的本地引擎，并逐步覆盖 tokenizer、模型加载、Transformer 层、KV cache
  ↳ 、CPU/GPU 后端、正确性报告和性能对标。
4
5 目标平台是 x86-64 Linux 实验环境。完整性能报告应记录 CPU 型号、内核版本、编译器
  ↳ 版本、是否能使用 PMU、NUMA 和线程亲和能力。若运行环境缺少 perf、NUMA、线
  ↳ 程亲和性或硬件性能计数器权限，报告必须把相关结论降级为“未验证”，不能把源
  ↳ 码路径存在当作实测性能证据。后续 GPU 后端会作为独立 backend 接入，不改变
  ↳ 当前切片的语义合同。
6
7 当前版本支持：
8
9 - 教学文本模型格式 `LCQI_MODEL_V1`
10 - 最小 tokenizer 合同格式 `LCQI_TOKENIZER_V1`
11 - safetensors header manifest 解析：metadata、tensor name、dtype、shape、data
  ↳ offsets、offset 边界和 dtype/shape 字节数校验
12 - 两层 MLP 教学切片
13 - int8 权重加 per-layer scale
14 - float 输入和激活
15 - 量化线性层、ReLU、分类 argmax
16 - int8 linear scalar baseline、packed layout、AVX2 路径和 shape sweep benchmark
17 - GPT-2 F32 dot kernel：cached generation 热路径使用可替换 dot 边界，x86-64/GNU
  ↳ /Clang 构建接入 AVX2/FMA 实现，其他平台保留标量 fallback
18 - tiny reference decoder：RMSNorm、float linear、RoPE、GQA KV cache、decode
  ↳ attention、SwiGLU、lm head
19 - GPT-2 reference path：byte-level BPE tokenizer、`config.json`、F32/F16/BF16
  ↳ safetensors 张量读取、HuggingFace GPT-2 name mapping、LayerNorm、绝对位置
  ↳ 嵌入、packed QKV、causal attention、GELU、tied lm head、full-prefix
  ↳ greedy generation、KV cache greedy generation 和分阶段 benchmark
20 - reference trace CSV：从 prompt/tokenizer 合同到 logits/sampler/token，逐
  ↳ checkpoint 输出 shape、stride、dtype、layout、checksum、max_abs、values
  ↳ 摘要
21 - 7B ledger CSV：输出典型 7B shape 的 FLOPs、KV 容量、权重/KV 字节和 bandwidth
  ↳ only tokens/s 上限
22 - serving SLO probe CSV：输出 admission、batch state、KV 预算、queue wait、TTFT
  ↳ 、TPOT、取消回收和拒绝率
23 - 自研 GPT-2 冒烟：加载 `openai-community/gpt2` 的 `config.json`、`vocab.json`
  ↳ 、`merges.txt`、`model.safetensors`，在 LCQI C++ reference path 上生成 1
  ↳ 个 token，并把文件 hash、prompt ids、generated ids 和文本输出写入报告
24 - 自研 GPT-2 benchmark 对比：同一个 GPT-2 F32 safetensors 模型分别跑 full
  ↳ prefix 和 cached-KV，并在可用时附带 llama.cpp/SmolLM2 Q4_K_M 作为成熟引擎

```

```

    ↪ 参考
25 - 自研 GPT-2 优化 A/B: 从指定 baseline commit 建临时 worktree, 当前实现和 ↪
    ↪ baseline 都通过 `books/cpu-volume-3-practice` 的 Release CMake 入口构建, ↪
    ↪ 再用同一个 GPT-2 F32 模型多轮测量 full-prefix 与 cached-KV
26 - 真实 small 开源模型冒烟: 通过 llama.cpp 加载 SmolLM2-135M-Instruct 的 GGUF 量↪
    ↪ 化文件, 在本地 CPU 上生成一段 assistant 文本, 并把模型 hash、命令、输出和↪
    ↪ 性能行写入报告
27 - 自研 SmolLM2/GGUF reference decode: 读取同一个 GGUF 内的 tokenizer tokens/↪
    ↪ merges 和 `F32/Q8_0/Q5_0/Q6_K/Q4_K` 混合权重, 加载 30 层 SmolLM2, 执行 ↪
    ↪ RMSNorm、normal RoPE、GQA KV cache、SwiGLU 和 tied lm head; 默认把 shape ↪
    ↪ 兼容的 `Q4_K` linear tensor 留在 GGUF block bytes 中直接执行, 其余格式走 ↪
    ↪ f32 fallback; `LCQI_LLAMA_GGML_DIRECT=1` 可打开 `Q5_0/Q6_K/Q8_0` 实验直算↪
    ↪ 路径用于分析
28 - CLI 推理和简单 benchmark
29 - CTest 正确性测试
30
31 后续扩展方向:
32
33 - 真实 7B 级模型 metadata、tokenizer 和量化权重导入
34 - SentencePiece tokenizer、safetensors 分片加载和更多模型方言 name mapping
35 - GPT-2 reference path 的 Transformers/PyTorch golden logits 对齐报告
36 - KV cache 分页、内存规划和可选量化
37 - CPU packed weight、SIMD、NUMA 和线程亲和优化
38 - GPU backend adapter 和 device kernel
39 - 与现有框架的 prefill/decode/内存/误差对标报告
40
41 构建:
42
43 ```bash
44 cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↪
    ↪ DCMMAKE_BUILD_TYPE=Debug
45 cmake --build books/cpu-volume-3/build/lcqi-debug
46 ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
47 ```
48
49 运行:
50
51 ```bash
52 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_cli \
53   books/cpu-volume-3/labs/linux_cpu_inference/models/tiny_mlp_i8.txt \
54   1.0 2.0 -1.0 0.5 1000
55 ```
56
57 kernel benchmark:
58
59 ```bash
60 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200
61 ```
62
63 输出 CSV 字段:
64
65 ```text

```

```

66 target_arch,input_size,output_size,output_block,repeat,backend,available,↵
    ↪ average_us,max_abs_diff,checksum
67 ```
68
69 当前 benchmark 按 backend 逐行输出: `scalar`、`packed_scalar`、`avx2`。x86-64 ↵
    ↪ 构建会编译 AVX2 路径; 不支持 AVX2/FMA 的机器会把该 backend 标为 `↵
    ↪ available=0`, 避免把源码存在误报成当前环境实测性能。
70
71 这还不是生产级高性能 kernel。当前 benchmark 建立 scalar baseline、packed layout↵
    ↪ 、AVX2 基线正确性和 shape sweep 口径。要达到完整 AI Infra 证据, 还必须继↵
    ↪ 续补反汇编分析、AVX-512、perf counter、oneDNN/OpenBLAS/llama.cpp/ggml 对↵
    ↪ 照、sanitizer、fuzz 和 CI。
72
73 safetensors manifest gate:
74
75 `lcqi_tests` 会运行时生成一个最小 safetensors fixture, 验证 header 长度、`↵
    ↪ __metadata__`、tensor name、dtype、shape、data offsets、payload 边界和 ↵
    ↪ dtype/shape 推导出的字节数。它还会生成 offset 超出 payload、dtype/shape ↵
    ↪ 字节数不匹配的坏文件, 确认加载器会拒绝。这个 gate 只覆盖模型资产目录表, ↵
    ↪ 不等于已经支持完整 LLaMA/Qwen 权重映射、分片合并或 mmap 执行; 后续真实模↵
    ↪ 型加载必须在这个 manifest 上继续接 name mapping、shape verifier、quant ↵
    ↪ descriptor 和 first token trace。
76
77 reference decoder trace:
78
79 ```bash
80 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
81 ```
82
83 输出 CSV 字段:
84
85 ```text
86 name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
87 ```
88
89 这条 trace 固定 prompt fixture `tok1 tok2 tok3`, tokenizer 输出完整 `tokens↵
    ↪ =[0,1,2,3,4]`, 其中 `0/4` 是 BOS/EOS; decoder trace 会剥离 BOS/EOS 后进入↵
    ↪ tiny decoder。这样报告能同时固定 tokenizer 合同和 decoder 输入合同, 避免↵
    ↪ 把二者混成一个不可解释的 token 数组。随后 trace 把 embedding、RMSNorm、Q/↵
    ↪ K/V、RoPE、KV cache slot、attention、SwiGLU、layer output、final norm、↵
    ↪ logits、argmax sampler 和 generated token 串成可回归检查的硬链路。`↵
    ↪ lcqi_tests` 会检查 tokenizer contract hash、关键 checkpoint 的 shape、↵
    ↪ stride、dtype、layout 和 logits golden, CTest 也会直接运行 `lcqi_trace`。
90
91 当前 tokenizer fixture 的关键输出:
92
93 ```text
94 tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
95 tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-↵
    ↪ chat-template-v1
96 ```
97

```

```

98 GPT-2 自研 reference smoke:
99
100 ```bash
101 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
102 make cpu3-gpt2-smoke
103 make cpu3-gpt2-benchmark-compare
104 make cpu3-gpt2-hotspot-profile
105 python3 books/cpu-volume-3/tools/run_gpt2_optimization_ab.py --rounds 3
106 python3 books/cpu-volume-3/tools/run_gpt2_hotspot_profile.py --token-counts ↵
    ↵ 4,16 --rounds 3
107 ```
108
109 第一条命令不需要下载模型，直接运行仓库内 tiny GPT-2 fixture，输出固定 prompt ↵
    ↵ ids、logits、predicted token 和 greedy generated ids。`lcqi_tests` 会覆盖 ↵
    ↵ tiny GPT-2 forward/generation、byte-level BPE 编码解码、F32/F16/BF16 ↵
    ↵ safetensors 读取、HuggingFace 风格 tiny GPT-2 checkpoint 加载和坏 shape ↵
    ↵ 拒绝。
110
111 第二条命令显式依赖网络，只下载 `openai-community/gpt2` 的 `config.json`、`vocab` ↵
    ↵ `.json`、`merges.txt`、`model.safetensors` 到 `~/cache/lcqi-gpt2-smoke/` ↵
    ↵ `openai-community--gpt2`，模型文件不进入 Git。脚本会校验四个文件的 bytes ↵
    ↵ 和 SHA256，构建 `lcqi_gpt2`，运行：
112
113 ```text
114 Hello, my name is
115 ```
116
117 当前验证报告写到：
118
119 ```text
120 books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
121 ```
122
123 最近一次本机报告的关键行是：
124
125 ```text
126 prompt_ids 15496 11 616 1438 318
127 generated_ids 15496 11 616 1438 318 1757
128 generated_text Hello, my name is John
129 validation=PASS
130 ```
131
132 这说明 LCQI 自研 C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer ↵
    ↵ 、权重导入、forward 和 greedy generation。默认真实模型路径现在使用 ` ↵
    ↵ `cached_kv`，也可以用 `--engine full` 复现旧的 full-prefix 重算路径：
133
134 ```bash
135 books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_gpt2 \
136 --benchmark --engine cached --threads 0 \
137 ~/cache/lcqi-gpt2-smoke/openai-community--gpt2 \
138 "Hello, my name is" 4
139 ```

```



```

140
141 `--threads 0` 表示自动选择 cached decoder worker 数；`--threads 1` 强制串行；↵
    ↪ `--threads N` 固定 worker 数。当前 auto 上限是 16，避免在短 decode 上把线↵
    ↪ 程调度开销放大到超过收益。
142
143 `make cpu3-gpt2-benchmark-compare` 会生成：
144
145 ```text
146 books/cpu-volume-3/results/lcqi-gpt2-benchmark-compare.txt
147 ```
148
149 `run_gpt2_optimization_ab.py` 会生成：
150
151 ```text
152 books/cpu-volume-3/results/lcqi-gpt2-optimization-ab.txt
153 ```
154
155 这个 A/B 报告专门回答“当前 LCQI 代码相对指定 baseline 改快了吗”。脚本默认 ↵
    ↪ baseline 是 `4feb3f`，会临时创建 baseline worktree，分别构建 baseline 和↵
    ↪ 当前工作区的 Release `lcqi_gpt2`，再多轮运行同一条 prompt。上一轮工作区复↵
    ↪ 用优化的 2 轮 smoke 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-↵
    ↪ optimization-ab-summary.md`；本轮线程优化的正式 raw 报告保存在 `books/cpu-↵
    ↪ -volume-3/reports/lcqi-gpt2-threaded-manual-ab-caw.txt`，汇总保存在 `↵
    ↪ books/cpu-volume-3/reports/lcqi-gpt2-threaded-optimization-summary.md`。↵
    ↪ 以 `73f40c7` 作为 baseline，在 `caw` 上 5 轮中位数显示：4 token cached-KV↵
    ↪ 生成阶段从 `395.397 ms` 降到 `76.4141 ms`，16 token 从 `989.060 ms` 降到↵
    ↪ `185.479 ms`，生成吞吐分别提升到 `52.3464 token/s` 和 `86.2633 token/s`↵
    ↪ 。随后的 F32 dot kernel 优化以 `cb4d89f` 为 baseline，raw 报告保存在 `↵
    ↪ books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-ab-caw.txt`，汇总保存在 `↵
    ↪ books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-optimization-summary.md`。在↵
    ↪ 同一台 `caw`、同一 GPT-2 F32 模型、同样 `--threads 0` 自动 16 workers ↵
    ↪ 下，4 token 生成阶段中位数从 `74.4247 ms` 到 `62.3628 ms`，16 token 从 ↵
    ↪ `181.257 ms` 到 `157.230 ms`；decode token/s 分别提升到 `130.073` 和 ↵
    ↪ `127.405`。再往后，row-range kernel 和去 `std::function` 调度开销的 raw ↵
    ↪ 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-ab-caw.txt`↵
    ↪ ，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-↵
    ↪ optimization-summary.md`。这一轮相对 `ab72ccb` 的端到端提升很小，16 token↵
    ↪ 生成中位数约 `153.354 ms` 到 `152.574 ms`；但直接比较 row-range 与 4-row↵
    ↪ AVX2 row-range 时，4 token 从 `62.0243 ms` 到 `60.3965 ms`，16 token 从 ↵
    ↪ `154.087 ms` 到 `151.222 ms`。本轮 prefill-skip 优化的 raw 报告保存在 `↵
    ↪ books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-ab-caw.txt`，汇总保存在↵
    ↪ `books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-optimization-summary.↵
    ↪ md`。它跳过 prompt 前缀 token 上不会被使用的 `lm_head` 预测：4 token 生成↵
    ↪ 中位数从 `60.5154 ms` 到 `52.1949 ms`，16 token 从 `153.574 ms` 到 ↵
    ↪ `147.424 ms`，生成文本和 token id 全部一致。端到端 GPT-2 数字受调度、温度↵
    ↪ 和共享机器状态影响很大，因此正式结论必须看多轮中位数、min/max 和生成文本↵
    ↪ 一致性，不能只挑一次最快结果。
156
157 `run_gpt2_hotspot_profile.py` 专门回答“现在热点到底在哪”。它通过 `--profile-↵
    ↪ hotspots` 打开 cached decode 内部计时，字段包括 `hotspot_lm_head_pct`、`↵
    ↪ hotspot_mlp_fc_pct`、`hotspot_mlp_projection_pct`、`↵
    ↪ hotspot_qkv_projection_pct` 和 `hotspot_attention_pct`。`caw` 上的 3 轮 ↵

```

↪ Release profile 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-hotspot-
 ↪ profile-summary.md`，原始报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-
 ↪ -hotspot-profile-caw.txt`。关键结论是：4 token 与 16 token 两个口径下，`
 ↪ lm_head` 中位数约 `30.2%`，MLP 两个线性投影合计约 `46%`，QKV 投影约 `17%`
 ↪ ，cached attention 本体只有 `0.06%` 到 `0.11%`。所以当前短上下文 GPT-2 的
 ↪ 慢点不是 KV attention，而是标量 F32 matvec 和 `50257 x 768` vocab 扫描。

158

159 当前优化点集中在 cached-KV generation: `Gpt2CachedGreedyDecoder` 把模型级 shape
 ↪ validation、KV cache、临时 workspace 和 worker pool 的生命周期提升到整段
 ↪ 生成；`Gpt2ForwardWorkspace` 复用 hidden、normed、Q/K/V、packed QKV、
 ↪ attention、MLP、scores 和 logits 缓冲区；生成路径只需要 greedy token 时走
 ↪ logits-free argmax，避免把完整 logits vector 作为 API 结果反复分配；
 ↪ prompt prefill 前缀通过 `advance\_without\_prediction()` 只更新 KV cache，
 ↪ 不跑 final norm 和 `lm\_head`，因为这些预测结果不会被用作输出；QKV、
 ↪ attention output projection、MLP 两个 projection 和 tied `lm\_head` 都在
 ↪ cached session 内按输出行并行；每个线程 chunk 再通过
 ↪ linear_f32_rows_unchecked 或 `max\_dot\_f32\_rows\_unchecked` 进入 F32 row-
 ↪ range kernel，x86-64 可用时使用 AVX2/FMA，其他平台走标量 fallback。KV
 ↪ cache 的 checked public API 仍保留，内部 cached attention 在一次边界校验
 ↪ 后使用私有 unchecked span 扫描热路径。它不改变 GPT-2 数学语义，只改变会话
 ↪ 生命周期、任务分片、无用工作剔除和热循环调度。

160

161 `make cpu3-gpt2-benchmark-compare` 仍用于和外部成熟引擎放在同一报告里看工程差
 ↪ 距。同机 llama.cpp/SmolLM2 Q4_K_M 是成熟引擎参照，不是同模型质量排名：
 ↪ LCQI 跑的是 GPT-2 F32 reference path，llama.cpp 跑的是 GGUF 量化 small
 ↪ instruct 模型。它揭示的是工程差距：LCQI 已经消除了“每步重算前缀”的主要算
 ↪ 法错误，补上 cached F32 row parallelism，并把热路径接到 AVX2/FMA row-
 ↪ range kernel；但还缺 packed/quantized weight、GGUF 量化 block、mmap/lazy
 ↪ load、采样器、分页 KV cache、NUMA/亲和和成熟调度。

162

163 它仍然不是生产级推理引擎：当前 GPT-2 路径还没有把 logits 和 Transformers/
 ↪ PyTorch 逐数值对齐成外部 golden 报告，也没有 GGUF 自研导入、分页 KV cache
 ↪ 、量化权重执行和高性能 SIMD/packed kernel。

164

165 7B ledger:

166

```
167 ```bash
168 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger \
169 > books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
170 ```
```

171

172 输出 CSV 字段:

173

```
174 ```text
175 section,item,value,unit,notes
176 ```
```

177

178 这份账本固定 `L=32,H=4096,n\_q=32,n\_kv=8,head\_dim=128,context=4096`，报告 decode
 ↪ FLOPs、KV 容量、fp16/int8/int4 每 token 字节和不同有效带宽下的 tokens/s
 ↪ 上限。CTest 会检查 `int4\_at\_100\_gbps=23.602 tokens/s` 这一行，防止账本公
 ↪ 式漂移。

179


```

180 serving SLO probe:
181
182 ```bash
183 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe↵
    ↵ \
184 > books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
185 ```
186
187 输出 CSV 字段:
188
189 ```text
190 row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,↵
    ↵ kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,↵
    ↵ tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,↵
    ↵ metric_value
191 ```
192
193 probe 固定四个请求：短请求、RAG 请求、取消请求和超长请求。它演示 admission 先按↵
    ↵ `prompt_tokens + max_new_tokens` 预留 KV，取消请求释放预算，超长请求返回↵
    ↵ `memory_budget_exceeded`，并输出 `ttft_p95_ms`、`queue_wait_p95_ms`、↵
    ↵ `rejection_rate` 等服务化指标。
194
195 真实 small 开源模型 smoke:
196
197 ```bash
198 make cpu3-smolllm2-smoke
199 ```
200
201 这条命令显式依赖网络和外部构建，因此不放进默认 CTest。脚本会把 llama.cpp 和模型↵
    ↵ 文件缓存到 `~/.cache/lcqi-smolllm2-small-smoke`，模型不进入 Git。固定模型↵
    ↵ 是 `HuggingFaceTB/SmolLM2-135M-Instruct` 的 GGUF 量化版本，来源仓库为 `↵
    ↵ bartowski/SmolLM2-135M-Instruct-GGUF`，文件为 `SmolLM2-135M-Instruct-↵
    ↵ Q4_K_M.gguf`。脚本会校验文件大小 `105454432` 字节和 SHA256:
202
203 ```text
204 2e8040ceae7815abe0dcb3540b9995ead1fa0d2ca9e797d0a635ae4433c68c2d
205 ```
206
207 脚本构建固定 commit 的 `llama-simple-chat`，用 `-ngl 0` 强制 CPU 路径，输入短↵
    ↵ prompt，检查 assistant response 非空，并生成:
208
209 ```text
210 books/cpu-volume-3/results/lcqi-smolllm2-small-model-smoke.txt
211 ```
212
213 这份报告只能证明 llama.cpp 成熟外部引擎上的真实 GGUF small 模型已经完成加载、↵
    ↵ tokenizer/chat template、prefill、decode 和文本输出。它不能证明 LCQI 自研↵
    ↵ C++ 引擎已经达到 llama.cpp 的性能，也不能替代 tiny fixture 的逐层 golden↵
    ↵ trace。
214
215 LCQI 自研 SmolLM2/GGUF reference decode:
216

```

```

217 ```bash
218 books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_llama_gguf ↵
    ↵ \
219 ~/.cache/lcqi-smolllm2-small-smoke/models/SmolLM2-135M-Instruct-Q4_K_M.gguf \
220 --prompt "Hello" --max-new 2 --benchmark --decode-text
221 ```
222
223 caw 上的最新报告写到:
224
225 ```text
226 books/cpu-volume-3/results/lcqi-smolllm2-gguf-reference-decode-caw.txt
227 ```
228
229 关键行是:
230
231 ```text
232 weight_execution gguf_mixed_q4_k_direct
233 generated_text ... <|im_start|>assistant
234 Hello!
235 benchmark_prompt_tokens 31
236 benchmark_generated_tokens 2
237 benchmark_decode_tokens_per_second 64.8586
238 benchmark_quantized_weight_bytes 103668480
239 benchmark_f32_weight_bytes 481436928
240 benchmark_direct_quantized_weight_bytes 7962624
241 benchmark_fallback_dequantized_weight_bytes 481436928
242 benchmark_q4_k_direct_tensors 16
243 benchmark_ggml_direct_tensors 0
244 benchmark_f32_fallback_tensors 256
245 benchmark_hotspot_w_down_ms 63.5371
246 benchmark_hotspot_q4_k_direct_calls 512
247 benchmark_hotspot_ggml_direct_calls 0
248 benchmark_hotspot_f32_fallback_calls 6208
249 ```
250
251 这条路径证明 LCQI 自研代码已经能读取同一个 SmolLM2 GGUF、解析 tokenizer arrays↵
    ↵ 、处理混合量化权重并端到端生成文本。默认执行路径不再只是“加载时全部解成 ↵
    ↵ f32”: shape 兼容的 `Q4_K` linear tensor 会保留 GGUF block bytes, 并在真实↵
    ↵ decoder 里通过 `matvec_q4_k_q8` 直接执行; `Q5_0/Q6_K/Q8_0/F32` tensor 仍↵
    ↵ 然 materialize 成 f32 fallback。当前只有约 `7.68%` 的加载权重字节进入默认↵
    ↵ direct Q4_K 路径, prefill 仍逐 token 执行; 与 llama.cpp 的差距主要来自更↵
    ↵ 完整的低比特权重覆盖、prompt batching、packed kernel、线程调度、prefetch↵
    ↵ 、KV cache 和整图执行。
252
253 实验路径 `LCQI_LLAMA_GGML_DIRECT=1` 会尝试让 shape 兼容的 `Q5_0/Q6_K/Q8_0` ↵
    ↵ linear tensor 也保留 GGUF block bytes, 并通过 `matvec_ggml_quantized_q8_0`↵
    ↵ 执行。第一版单线程/整矩阵实验曾把 direct 量化字节覆盖率从 `0.076809` 提↵
    ↵ 高到 `0.708479`, 但 prefill 从默认 `370.847 ms` 退化到 `1211.710 ms`, 热↵
    ↵ 点 `benchmark_hotspot_ggml_direct_ms` 达到 `1207.13 ms`。当前版本已经给 ↵
    ↵ GGML direct 补上 row-range unchecked 入口, 并接入同一个 `↵
    ↵ LlamaParallelWorkerPool`: caw 同输入 5 轮中位数显示, 实验路径降到 prefill↵
    ↵ `242.149 ms`、decode step `8.98937 ms`、`↵

```

```

    ↪ benchmark_hotspot_ggml_direct_ms=228.300`。这说明 row-range 解决了“所有行↪
    ↪ 串行扫”的一大块问题，但它仍慢于默认 Q4_K+f32 fallback 的 `162.492 ms`、↪
    ↪ `6.41487 ms`，所以继续保留为 opt-in，不默认启用。
254
255 LCQI 与 llama.cpp 同输入对比：
256
257 ```bash
258 python3 books/cpu-volume-3/tools/run_smollm2_same_input_compare.py \
259 --cache-dir ~/.cache/lcqi-smollm2-same-input-compare \
260 --model-path ~/.cache/lcqi-smollm2-small-smoke/models/SmolLM2-135M-Instruct-↪
    ↪ Q4_K_M.gguf \
261 --build-dir books/cpu-volume-3/build/lcqi-release \
262 --rounds 5 --max-new 2
263 ```
264
265 这个脚本会构建固定 commit 的 llama.cpp，运行 `llama-tokenize`、`llama-simple` ↪
    ↪ 和 `llama-bench`，并确认 LCQI 和 llama.cpp 看到的 prompt token ids 完全↪
    ↪ 致。caw 上的同输入报告保存在：
266
267 ```text
268 books/cpu-volume-3/reports/lcqi-smollm2-ggml-threaded-compare-caw.txt
269 ```
270
271 关键结论：同一个模型、同一个展开 chat prompt、同样 31 个 token id、同样 `max-↪
    ↪ new=2` 下，当前默认 LCQI 使用 `8` 个 worker，对大行数 f32 fallback、Q4_K ↪
    ↪ direct row-range 和实验 GGML direct row-range 都能做输出行切分。默认 Q4_K↪
    ↪ direct prefill 中位数从串行 `364.282 ms` 降到 `162.492 ms`，decode step ↪
    ↪ 从串行 `14.9794 ms` 降到 `6.41487 ms`；f32 fallback hotspot 从 `338.779 ↪
    ↪ ms` 降到 `148.132 ms`。相对同输入 llama.cpp，LCQI prefill 仍慢 `7.089529x`↪
    ↪ `，decode step 仍慢 `2.250832x`。同一二进制里关闭 `LCQI_LLAMA_Q4_DIRECT`↪
    ↪ `=0` 后，LCQI prefill 为 `174.378 ms`，decode step 为 `6.78061 ms`，`↪
    ↪ w_down` 热点为 `36.4924 ms`；默认 Q4_K direct 后分别为 `162.492 ms`、↪
    ↪ `6.41487 ms`、`22.2228 ms`。所以这轮默认优化的 A/B 证据是：线程池带来 ↪
    ↪ prefill `2.241846x`、decode step `2.335106x`、f32 hotspot `2.287008x`；↪
    ↪ Q4_K direct 在并行后带来 prefill `1.073148x`、decode step `1.057014x`、↪
    ↪ `w_down` `1.642115x`，并减少 `56,623,104` 字节 f32 materialized 权重。实验↪
    ↪ GGML direct row-range 相比最初 `1211.710 ms` 已明显改善到 `242.149 ms`，↪
    ↪ 但仍慢于默认路径，下一步需要 packed multi-row layout、更完整 SIMD 覆盖和 ↪
    ↪ prefetch，而不是只扩大格式覆盖。

```

tiny MLP 模型文件是最短闭环的资产。它决定输入维度、hidden 维度、输出维度、权重量化值、scale 和 bias；测试中的 logits golden 依赖这份文件。

Listing 1.9 models/tiny_mlp_i8.txt

```

1 LCQI_MODEL_V1
2 input_size 4
3 hidden_size 3
4 output_size 2
5 hidden_scale 0.1
6 hidden_weights
7 5 -3 2 1
8 -4 6 1 -2

```

```
9 3 2 -5 4
10 hidden_bias
11 0.1 -0.2 0.0
12 output_scale 0.05
13 output_weights
14 4 -6 2
15 -3 5 8
16 output_bias
17 -0.5 0.4
```

tiny tokenizer fixture 固定 BOS、EOS、UNK、词表和模板 hash。tokenizer 合同如果漂移，后面的 decoder trace 和测试都会被污染。

Listing 1.10 models/tiny_tokenizer.txt

```
1 LCQI_TOKENIZER_V1
2 bos_id 0
3 eos_id 4
4 unk_id -1
5 chat_template_hash tiny-chat-template-v1
6 vocab_size 5
7 vocab
8 0 <bos>
9 1 tok1
10 2 tok2
11 3 tok3
12 4 <eos>
```

为什么要先讲调用链

实践与代码卷如果只把 *.cpp 和 *.hpp 全部贴出来，读者仍然不知道从哪里下手。LCQI 的代码同时包含 tiny MLP、int8 kernel、tokenizer、safetensors、reference decoder、GPT-2、benchmark、trace、ledger、serving probe 和外部模型脚本；这些文件不是平铺关系，而是几条调用链。先讲调用链，再给完整源码，读者才能把每个文件放到正确位置。

核心思想

读 LCQI 的顺序不是“哪个文件长先读哪个”，而是“入口、合同、资产、执行、证据”。入口说明目标怎样构建，合同说明类型怎样连接，资产说明模型从哪里来，执行说明 token 或张量怎样流动，证据说明测试和报告守住了哪些边界。

一、全工程入口：哪些文件决定能不能复现

文件	负责什么	读源码时要确认什么
第三册README	第三册 AI 计算工程路线、当前 LCQI 能力、GPT-2 smoke 和结果报告位置。	能力声明必须能在 labs/linux_cpu_inference、tools 和 results 中找到证据。

第三册CMake	第三册根 CMake，负责把 <code>linux_cpu_inference</code> 接入。	根 CMake 应保持薄层；真正 target 定义必须在 lab 内部，避免入口文件承担实现细节。
本卷CMake	本卷复用同一份 LCQI lab。	本卷不复制源码，只把同一工程接入自己的测试构建，防止两份代码漂移。
本卷Makefile	本卷的书籍构建入口。	它构建 PDF，也提供 <code>test</code> 目标去构建第三册 LCQI；实践与代码卷不能只排版不验证。
LCQICMake	<code>lcqi</code> 库、CLI、benchmark、trace、GPT-2、ledger、serving probe、CTest 目标。	读这里能看到哪些文件进入库，哪些是可执行程序，哪些依赖可选外部库。
LCQIREADME	本地构建、tiny 推理、benchmark、trace、GPT-2、SmolLM2 smoke 的命令。	先按 README 跑命令，再读源码；否则不知道某个函数是否真的能从命令行触达。

上表里的短标签对应下面这些真实路径：

- `books/cpu-volume-3/README.md`
- `books/cpu-volume-3/CMakeLists.txt`
- `books/cpu-volume-3-practice/CMakeLists.txt`
- `books/cpu-volume-3-practice/Makefile`
- `books/cpu-volume-3/labs/linux_cpu_inference/CMakeLists.txt`
- `books/cpu-volume-3/labs/linux_cpu_inference/README.md`

这些入口文件回答“代码在哪里”和“怎样运行”。读者进入本卷代码走读章时，先不要直接跳到 `gpt2_reference.cpp`。先确认 `lcqi` 静态库包含 `model.cpp`、`inference.cpp`、`int8_kernels.cpp`、`f32_kernels.cpp`、`gguf.cpp`、`ggml_tensors.cpp`、`q4_k.cpp`、`llama_gguf.cpp`、`reference_decoder.cpp`、`tokenizer.cpp`、`safetensors.cpp` 和 `gpt2_reference.cpp`；再确认 `lcqi_cli`、`lcqi_bench`、`lcqi_gguf_q4_bench`、`lcqi_llama_gguf`、`lcqi_trace`、`lcqi_gpt2`、`lcqi_ledger`、`lcqi_serving_probe` 和 `lcqi_tests` 分别从哪条路径进入。

二、Tiny MLP 调用链：最短闭环怎样跑通

Tiny MLP 是最小推理闭环，用来证明“模型资产、shape 检查、int8 linear、激活函数、logits、argmax、CLI 输出”可以完整连起来。它不是大模型能力本身，但它是后面所有复杂路径的可手算底座。

文件	关键类型或函数	这段代码的意思
<code>models/tiny_mlp_i8.txt</code>	输入维度、hidden 维度、输出维度、int8 权重、scale、bias。	这是模型资产。测试中的 golden logits 来自这里，资产改动必须带着测试改动。
<code>include/lcqi/model.hpp</code>	<code>QuantizedLinearLayer</code> 、 <code>TinyMlpModel</code> 、 <code>load_model</code> 。	定义 tiny MLP 如何在内存中表示；只负责模型结构和加载入口，不做推理。
<code>src/model.cpp</code>	<code>read_i8_vector</code> 、 <code>validate_layer</code> 、 <code>load_model</code> 。	解析文本模型，拒绝坏 key、坏维度、权重数量不匹配和非法 scale，把错误挡在执行前。

<code>include/lcqi/inference.hpp</code>	<code>InferenceResult</code> 、 <code>linear_i8</code> 、 <code>relu_inplace</code> 、 <code>run_inference</code> 。	定义推理输出要包含 <code>hidden</code> 、 <code>logits</code> 和 <code>predicted class</code> ，方便测试定位错误层。
<code>src/inference.cpp</code>	输入 <code>shape</code> 检查、两层 <code>int8 linear</code> 、 <code>ReLU</code> 、 <code>argmax</code> 。	最短执行路径：模型加输入变成 <code>logits</code> 。这里的代码应保持清楚，不能混入 <code>CLI</code> 或文件解析。
<code>src/main.cpp</code>	解析命令行输入并打印结果。	<code>CLI</code> 是薄层；它只接模型路径和输入，不复制 <code>run_inference</code> 逻辑。

这条链路按下面顺序读：`main()` 取得模型路径和输入向量，调用 `load_model()` 把 `tiny_mlp_i8.txt` 变成 `TinyMlpModel`，再调用 `run_inference()`。在 `run_inference()` 内部，第一层 `linear_i8()` 把 `float` 输入和 `int8` 权重相乘，乘以 `scale` 后加 `bias`；`relu_inplace()` 把负 `hidden` 截断；第二层 `linear_i8()` 得到 `logits`；最后 `argmax()` 给出 `predicted class`。

读这个路径时要抓住边界条件。输入长度不等于 `input_size` 要立即拒绝；权重数量不是 `rows*columns` 要在加载时拒绝；`logits` 为空不能取 `argmax`。这些错误如果拖到 `SIMD kernel` 或 `benchmark` 里才暴露，定位成本会高很多。

三、Int8 Kernel 调用链：reference、packed、AVX2 和 benchmark 怎样对照

文件	关键代码	这段代码的意思
<code>include/lcqi/int8_kernels.hpp</code>	<code>PackedLinearI8</code> 、 <code>KernelBenchmarkCase</code> 、 <code>benchmark_linear_i8_case</code> 。	定义 <code>kernel</code> 对标口径： <code>shape</code> 、 <code>repeat</code> 、 <code>backend</code> 、平均耗时、 <code>checksum</code> 、 <code>max diff</code> 。
<code>src/int8_kernels.cpp</code>	<code>pack_linear_i8</code> 、 <code>linear_i8_packed</code> 、 <code>linear_i8_packed_with_workspace</code> 、 <code>deterministic fixture</code> 。	<code>scalar</code> 和 <code>packed</code> 路径是正确性基线， <code>workspace</code> 路径减少重复分配， <code>fixture</code> 让 <code>benchmark</code> 和测试可复现。
<code>src/int8_kernels_avx2.cpp</code>	<code>linear_i8_packed_avx2_available</code> 、 <code>linear_i8_packed_avx2</code> 。	平台优化路径。必须先判断机器能力，再执行 <code>AVX2/FMA</code> ；不可用时报告 <code>unavailable</code> ，而不是假装测过。
<code>include/lcqi/f32_kernels.hpp</code> 与 <code>src/f32_kernels.cpp</code>	<code>dot_f32</code> 、 <code>linear_f32_rows_unchecked</code> 、 <code>linear_f32_batch_rows_unchecked</code> 、 <code>max_dot_f32_rows_unchecked</code> 。	<code>F32 fallback</code> 的 <code>kernel</code> 边界。单行 <code>dot</code> 是基础， <code>row-range linear</code> 一次写一段输出行， <code>batch-row linear</code> 在 <code>prompt prefill</code> 中复用同一行权重给多个 <code>token</code> ， <code>row-range max</code> 给 <code>tied lm_head</code> 用。

<code>src/f32_kernels_avx2.cpp</code>	<code>dot_f32_avx2_available</code> 、8-row row-range fast path、8-token batch-row fast path。	x86 平台优化路径。decode path 在一段输出行内部把同一份 input load 复用到 8 个输出行；prefill batch path 把同一行权重复用到 8 个 prompt token，并处理 4/3/2/1 token tail；不改变 greedy tie-breaking。
<code>include/lcqi/gguf.hpp</code> 与 <code>src/gguf.cpp</code>	<code>GgufManifest</code> 、 <code>GgufTensorInfo</code> 、 <code>load_gguf_manifest</code> 、 <code>read_gguf_tensor_bytes</code> 。	读取 GGUF header、metadata、tensor 表、alignment 和真实 tensor payload；它只负责格式边界，不做推理数学。
<code>include/lcqi/ggml_tensors.hpp</code> 与 <code>src/ggml_tensors.cpp</code>	<code>dequantize_ggml_tensor</code> 、 <code>ggml_f16_to_f32</code> 。	把 GGUF 中的 <code>F32</code> 、 <code>Q8_0</code> 、 <code>Q5_0</code> 、 <code>Q6_K</code> 和 <code>Q4_K</code> 张量按 GGML block 布局解释出来。它是格式 oracle，不决定某个 tensor 最终走 f32 fallback 还是量化直算。
<code>include/lcqi/q4_k.hpp</code> 与 <code>src/q4_k.cpp</code>	<code>dequantize_q4_k</code> 、 <code>quantize_q8_k_input</code> 、 <code>matvec_q4_k_q8</code> 。	实现 llama.cpp 同格式的 <code>Q4_K</code> block 解码和 <code>Q4_KxQ8_K</code> 单算子路径；单测用手写 block 固定 nibble 顺序、scale/min 解包和误差边界。
<code>include/lcqi/q5_0_packed.hpp</code> 、 <code>src/q5_0_packed.cpp</code> 与 <code>src/q5_0_packed_avx2.cpp</code>	<code>pack_q5_0_blocks</code> 、 <code>matvec_q5_0_packed_q8_0</code> 、batch group path。	默认 <code>Q5_0</code> decode 路径。loader 先把 GGML 的 high-bit+nibble 布局打包成 32 个 unsigned lane，decode/single-token 前向时用 <code>Q8_0InputBlock</code> 和 AVX2 <code>maddubs</code> 做点积，避免每次 matvec 现场拆 high-bit。低层 batch group path 仍保留并测试，但 caw A/B 证明它不适合作为当前 prompt prefill 默认路径；模型默认 batch prefill 改用同一 Q5 tensor 的 F32 sidecar，通过 <code>q5_0_batch_f32</code> 字段单独记账。

```
include/lcqi/
ggml_matvec.hpp、
src/ggml_matvec.
cpp 与 src/ggml_
matvec_avx2.cpp
matvec_ggml_quantized_
q8_0、matvec_ggml_
quantized_q8_0_batch_
rows_unchecked、max_
dot_ggml_quantized_q8_
0。
```

```
include/lcqi/
llama_gguf.hpp 与
src/llama_gguf.
cpp
load_llama_gguf_
reference_model、
load_gpt2_tokenizer_
from_gguf、llama_gguf_
generate_greedy。
```

```
src/llama_cli.cpp  lcqi_llama_gguf。
```

普通 Q8_0 direct、独立 Q6_K direct 实验和完整 GGML direct 的共享内核。默认只让 Q8_0 linear 和 tied lm head 使用 AVX2 Q8_0xQ8_0; tied lm head 只需要最大 token, 所以现在可以用 fused max-dot 直接在 row-range 内归约, 不必先写完整 vocab logits 再 argmax。LCQI_LLAMA_Q6_K_DIRECT=1 只打开 Q6_K A/B; 完整 Q5_0/Q6_K/Q8_0 direct 仍需 LCQI_LLAMA_GGML_DIRECT=1 显式打开。

自研 SmolLM2/GGUF decoder path。它从同一个 GGUF 读取 tokenizer tokens/merges、混合量化权重、RMSNorm、normal RoPE、GQA KV cache、SwiGLU 和 tied lm head; 默认执行 Q4_K direct、Q5_0 decode packed、Q5 prompt-batch F32 sidecar、普通 Q8_0 direct、tied lm head fused max-dot 与 F32 fallback, 并在 prompt 阶段用 LlamaBatchWorkspace 批量推进 prefill。Q6_K direct 现在是单独实验存储类型和热点字段, caw A/B 已证明当前实现变慢, 所以仍不进入默认路径。

命令行加载真实 SmolLM2 GGUF, 支持 token id prompt 和文本 prompt, 输出生成 token、文本、load/prefill/decode 时间、KV 字节、量化权重字节、Q5_0 packed 字节、Q5_0 batch F32 sidecar 字节、Q6_K direct 实验字节、Q8_0 direct 字节和 fallback f32 字节。

src/q4_k_avx2.cpp	q4_k_q8_avx2_available、matvec_q4_k_q8_avx2_unchecked。	x86 AVX2 路径。它用 maddubs 把 32 个 unsigned nibble 与 32 个 signed int8 激活先归约成整数和，再套用 Q4_K 的 scale/min；报告必须同时给 scalar 与 auto 后端，防止把机器波动写成优化收益。
src/gguf_q4_bench.cpp	lcqi_gguf_q4_bench。	从真实 SmolLM2 GGUF 选择 Q4_K 矩阵，比较 f32 解码基线、Q4_Kxf32 和 Q4_KxQ8_K，输出速度、误差、checksum 和字节账本。
src/bench.cpp	shape 列表、repeat 参数、backend 输出。	benchmark 程序只负责测 kernel 入口。性能结论必须和 max_abs_diff、CPU、编译器、构建类型一起报告。

kernel 文件要按“可信基线到优化路径”的顺序读。先看 deterministic layer 和 input 怎么生成，保证每次运行输入相同；再看 linear_i8_packed() 怎样把 int8 权重、scale、bias 和 float 输入算成输出；再看 workspace 路径怎样复用临时缓冲；最后看 AVX2 路径怎样在支持平台上替换内层循环。Q4_K 路径也遵守同一顺序：先读 scalar 解码，再读 AVX2 maddubs 子块归约，最后看 benchmark 怎样强制 scalar 和 auto 后端并列输出。测试用 packed 与 scalar 的 max_abs_diff 对照，benchmark 再比较耗时。

四、Tokenizer、Safetensors 与 Reference Decoder：真实模型前的三道门

文件	关键类型或函数	这段代码的意思
models/tiny_tokenizer.txt	BOS、EOS、UNK、词表、模板 hash。	固定 tokenizer 合同，避免 prompt 到 token id 的规则漂移。
include/lcqi/tokenizer.hpp 与 src/tokenizer.cpp	TokenizerModel、load_tokenizer、encode_prompt、tokenizer_contract_hash。	把 prompt 切成稳定 token id；未知 token 是否允许，必须由 UNK 合同决定。
include/lcqi/safetensors.hpp 与 src/safetensors.cpp	SafeTensorManifest、SafeTensorFile、manifest parser、dtype 转换。	解析模型字节之前先验证 header、dtype、shape、offset 和 payload 边界，防止坏资产进入数学层。
include/lcqi/reference_decoder.hpp 与 src/reference_decoder.cpp	embedding、RMSNORM、RoPE、KV cache、attention、SwiGLU、trace checkpoint。	Tiny decoder-only reference path。它用 checkpoint 解释每一步张量状态，不只返回最终 token。
src/trace.cpp	trace CSV 输出。	把 reference decoder 的 checkpoint 打印成可复查表格，定位错误发生在哪个阶段。

这三道门分别解决三个问题。Tokenizer 解决“人写的 prompt 如何变成 token id”；Safetensors 解决“磁盘里的模型字节如何变成带 shape 的张量”；Reference Decoder 解决“token 和权重如何经过 decoder-only 层生成 logits”。任何一层含糊，后面的 GPT-2 路径都会变成看似能跑、实则无法定位的黑盒。

五、GPT-2 调用链：从 HuggingFace 资产到生成 token

GPT-2 路径是本卷代码走读里最长的一条链，因为它要处理真实模型格式。读 `gpt2_reference.cpp` 时按模块读，不要从第一行一路滚到最后。

文件	关键代码	这段代码的意思
<code>include/lcqi/gpt2_reference.hpp</code>	配置、模型、KV cache、cached decoder、tokenizer 与 forward/generate API。	公开 GPT-2 reference path 的合同：配置、权重、KV cache、tokenizer、full-prefix、cached generation，以及 prompt prefill 中只推进 cache、不做无用预测的入口。
<code>src/gpt2_reference.cpp</code>	JSON parser、byte-level BPE、safetensors loader、LayerNorm、QKV、causal attention、GELU、lm head、KV cache。	正确性优先实现。它把 HuggingFace GPT-2 权重名、Conv1D 转置、tokenizer 和 forward 规则全部显式写出来。
<code>src/gpt2_cli.cpp</code>	tiny mode、真实模型目录模式、generation、benchmark 输出。	命令行入口。无参数跑 tiny fixture；传模型目录时加载真实 GPT-2 资产；benchmark 对比 full-prefix 和 cached-KV。
<code>tools/run_gpt2_smoke.py</code>	下载标准 GPT-2 资产，调用 <code>lcqi_gpt2</code> 。	证明自研 safetensors、BPE、forward、KV cache 和 greedy generation 能跑真实开源模型。
<code>tools/run_gpt2_benchmark_compare.py</code>	运行 LCQI full-prefix、cached-KV，并可附带成熟引擎参考。	用同一模型口径比较算法差距和 kernel 差距，不把不同模型的速度混成一个结论。

GPT-2 代码按五段读。第一段是配置和权重加载：`load_gpt2_config()` 读 `config.json`，`load_gpt2_reference_model()` 从 safetensors 中取 embedding、每层 LayerNorm、QKV、MLP 和 final norm。HuggingFace GPT-2 的 Conv1D 权重形状和普通线性层方向不同，所以代码显式调用转置函数，不能跳过。

第二段是 tokenizer：`load_gpt2_tokenizer()` 读 `vocab.json` 和 `merges.txt`，`gpt2_encode()` 执行 byte-level BPE，`gpt2_decode()` 把 id 还原为文本。这里的重点是字节到 Unicode 映射、BPE merge rank 和 unknown token 边界。

第三段是 full-prefix forward：`run_gpt2_forward()` 每生成一个新 token 都用完整 token 前缀重新跑所有层。它慢，但容易验证。流程是 token embedding 加 position embedding，逐层执行 LayerNorm、packed QKV、causal attention、残差、MLP、残差，最后 final LayerNorm 和 tied lm head 得到 logits。

第四段是 cached-KV forward：`Gpt2KvCache` 保存每层每个 position 的 key/value，`run_gpt2_forward_cached()` 每次只处理最新 token，并让 attention 读取过去 cache。这里要特别看 `validate_written()`：读取未写入 cache slot 必须报错，否则生成错误会很难定位。

第五段是 greedy generation：`gpt2_generate_greedy()` 用 full-prefix 路径，`gpt2_generate_greedy_cached()` 用 KV cache 路径。两个函数都要检查

prompt 非空、max_new_tokens 非负、不会超过 max_positions，并在生成 EOS 时停止。

六、报告、服务探针、外部对标和测试

文件	关键代码	这段代码的意思
src/ledger.cpp	7B 参 数 量、KV cache、bandwidth-only 上限。	输出的是工程账本，不是实测速度；它帮助读者估算权重和 KV 内存压力。
src/serving_probe.cpp	短请求、RAG 请求、取消请求、超长请求，TTFT/TPOT 字段。	把服务层边界变成字段：哪些请求接受，哪些拒绝，取消后资源是否回收。
src/onednn_bench.cpp	oneDNN matmul benchmark。	可选外部库对标入口。没有 oneDNN 环境时不能声称已验证。
tools/run_smollm2_small_smoke.py	下 载 SmolLM2-135M-Instruct GGUF，构建固定 commit 的 llama.cpp smoke。	这是成熟外部引擎上的真实 small 模型 smoke，用来建立外部参照，不代表 LCQI 已具备同等能力。
tools/run_lcqi_benchmark.sh	收 集 LCQI 本 地 benchmark。	在固定机器上重复运行同一组 shape，形成可复查 CSV 或文本报告。
tests/lcqi_tests.cpp	tiny MLP、tokenizer、safetensors、GPT-2 tiny、HF-style checkpoint、bad shape、decoder、trace、kernel。	这是本卷最重要的代码证据。新增功能如果没有进入这里，就不能写成已经守住的能力。

读测试文件时，不要只看最后是否返回 0。它里面的 fixture 本身就是源码讲解的一部分：测试会手写 safetensors header，构造 F16/BF16/F32 payload，生成 tiny GPT-2 权重，验证 bad shape 会被拒绝，比较 packed int8 kernel 和 scalar 路径。这些测试告诉读者“代码真正承诺了什么”。如果一个能力只出现在 README 或 CLI 输出里，却没有进入测试或 smoke 报告，就只能算实验入口，不能算稳定能力。

源码阅读任务

读完本章后，再去读相邻主题中的完整源码。每看到一个文件，都把它归到五类之一：入口、合同、资产解析、执行路径、证据。归不进去的文件，就说明工程边界需要重新解释或重构。

第 2 章实践：张量布局、地址流与第一个 MatVec

本章对应原书中的第三册“张量布局、地址流与第一个矩阵向量内核”。不要把它当作概念复述，本章要把 TensorView、dtype、shape、stride、row-major、地址公式、bias、ReLU、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `linear_i8`，核心命令或测试入口是 `lcqi_tests`。

Listing 2.1 第 2 章实践：张量布局、地址流与第一个 MatVec 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  < DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / linear_i8
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：手算一个 2 行 4 列权重矩阵的地址偏移。读者要先手写预期结果，再运行项目观察输出。动作是：把 `weight[out,k]` 展开成 `base+(out*K+k)*sizeof(i8)`，再对应到 `linear_i8` 的循环。如果输出里没有 `logit` 或对应字段无法解释，就不要继续优化；先回到 `reference`、输入合同和 `trace` 字段。

从这个特例推出的规律是：张量不是裸数组；每个 kernel 入口都要能解释 dtype、shape、stride、scale 和输出缓冲这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一页地址流账本：输入向量、hidden 权重、输出 logits 各自的 shape、连续性、读写次数和错误边界。这个作业会被后续章节继续引用，命

名、目录和字段不要随意变化。

第 3 章实践：GEMM、packing 与 SIMD 证据

本章对应原书中的第三册“从矩阵向量乘走向 GEMM、packing 与 SIMD”。不要把它当作概念复述，本章要把 packed weight、scalar baseline、AVX2 backend、output block、shape sweep、反汇编和降级放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `output_block`，核心命令或测试入口是 `lcqi_bench`。

Listing 3.1 第 3 章实践：GEMM、packing 与 SIMD 证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / output_block
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：同一组输入同时跑 `scalar`、`packed_scalar` 和 `avx2`。读者要先手写预期结果，再运行项目观察输出。动作是：比较 CSV 中 `backend`、`available`、`average_us`、`max_abs_diff` 和 `checksum`。如果输出里没有 `max_abs_diff` 或对应字段无法解释，就不要继续优化；先回到 `reference`、输入合同和 `trace` 字段。

从这个特例推出的规律是：SIMD 路径只有在结果和 `baseline` 对齐后才有资格谈速度；`available=0` 也是证据，不是失败这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、`perf`、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、`fallback` 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 `shape sweep` 表，至少解释一个 SIMD 有收益、一个收益不明显、一个当前环境不可验证的场景。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 4 章实践：int8 量化、累加器与重新量化

本章对应原书中的第三册“量化系统总览”和“int8 量化、累加器与重新量化”。不要把它当作概念复述，本章要把 int8 weight、float activation、scale、accumulator、requantize、oracle tolerance 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `TinyMlpModel`，核心命令或测试入口是 `lcqi_tests`。

Listing 4.1 第 4 章实践：int8 量化、累加器与重新量化的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / TinyMlpModel
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：构造一行 int8 权重 `[2,-1,3,0]` 和输入 `[1,2,-1,0.5]`。读者要先手写预期结果，再运行项目观察输出。动作是：先手算 int32/float accumulator，再和 `linear_i8` 输出比较。如果输出里没有 `scale` 或对应字段无法解释，就不要继续优化；先回到 `reference`、输入合同和 `trace` 字段。

从这个特例推出的规律是：量化不是把 float 强转成 int8；必须记录 `scale` 作用位置、累加宽度、误差阈值和 `golden` 输出这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交量化合同卡：每个 tensor 的 dtype、scale 粒度、accumulator 类型、误差门限和拒绝条件。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：公共合同、Tiny MLP 与 Int8 Kernel

为什么先读头文件

头文件是系统对外承诺的最小合同。实现可以重写，kernel 可以替换，benchmark 可以补充平台路径，但头文件里暴露的数据结构、函数入口和返回字段决定了测试、CLI、工具脚本和章节讲解怎样调用这套工程。

核心理想

读 LCQI 时不要从最长的 `gpt2_reference.cpp` 开始。先读公共头文件，明确“模型如何表示”“推理返回什么”“trace 记录什么”“KV cache 怎样寻址”“tokenizer 的坏输入怎样处理”，再进入实现。这样读到长循环和 shape 检查时，知道它们在维护哪份合同。

Tiny MLP 与推理入口

`model.hpp` 定义 tiny MLP 的模型对象、权重、bias、scale 和加载入口。它的责任是把文本模型资产解释成 C++ 对象，不负责执行推理。

Listing 4.2 `include/lcqi/model.hpp`

```

1 #pragma once
2
3 #include <cstdint>
4 #include <filesystem>
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct QuantizedLinearLayer {
11     std::int32_t input_size = 0;
12     std::int32_t output_size = 0;
13     float scale = 1.0F;
14     std::vector<std::int8_t> weights;
15     std::vector<float> bias;
16 };
17
18 struct TinyMlpModel {
19     std::int32_t input_size = 0;
20     std::int32_t hidden_size = 0;
21     std::int32_t output_size = 0;
22     QuantizedLinearLayer hidden;
23     QuantizedLinearLayer output;
24 };
25
26 TinyMlpModel load_model(const std::filesystem::path& path);
27
28 } // namespace lcqi

```


`inference.hpp` 定义推理结果和 `run_inference` 入口。返回值包含 `hidden`、`logits` 和 `predicted class`，测试可以用这些字段定位是加载错、kernel 错，还是 `argmax` 错。

Listing 4.3 `include/lcqi/inference.hpp`

```

1 #pragma once
2
3 #include <stdint>
4 #include <span>
5 #include <vector>
6
7 #include <lcqi/model.hpp>
8
9 namespace lcqi {
10
11 struct InferenceResult {
12     std::vector<float> logits;
13     std::int32_t predicted_class = 0;
14 };
15
16 void linear_i8(const QuantizedLinearLayer& layer,
17               std::span<const float> input,
18               std::span<float> output);
19
20 void relu_inplace(std::span<float> values);
21
22 InferenceResult run_inference(const TinyMlpModel& model,
23                               std::span<const float> input);
24
25 } // namespace lcqi

```

Int8 Kernel、Tokenizer 与 Safetensors

`int8 kernel` 头文件定义 `packed linear`、`deterministic fixture`、`scalar/workspace/AVX2` 路径和误差比较。注意 `AVX2` 是运行时能力，不是源码存在就等于当前机器已验证。

Listing 4.4 `include/lcqi/int8_kernels.hpp`

```

1 #pragma once
2
3 #include <stdint>
4 #include <span>
5 #include <vector>
6
7 #include <lcqi/model.hpp>
8
9 namespace lcqi {
10
11 struct PackedLinearI8 {

```

```

12     std::int32_t input_size = 0;
13     std::int32_t output_size = 0;
14     std::int32_t output_block_size = 0;
15     float scale = 1.0F;
16     std::vector<std::int8_t> packed_weights;
17     std::vector<float> bias;
18 };
19
20 struct KernelBenchmarkCase {
21     std::int32_t input_size = 0;
22     std::int32_t output_size = 0;
23     std::int32_t output_block_size = 0;
24     std::int32_t repeat = 0;
25 };
26
27 struct KernelBackendBenchmark {
28     const char* name = "";
29     bool available = false;
30     double average_us = 0.0;
31     float max_abs_diff = 0.0F;
32     float checksum = 0.0F;
33 };
34
35 struct KernelBenchmarkResult {
36     KernelBenchmarkCase benchmark_case;
37     std::vector<KernelBackendBenchmark> backends;
38 };
39
40 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
41                               std::int32_t output_block_size);
42
43 void linear_i8_packed(const PackedLinearI8& layer,
44                      std::span<const float> input,
45                      std::span<float> output);
46
47 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
48                                     std::span<const float> input,
49                                     std::span<float> output,
50                                     std::span<float> accumulator_workspace);
51
52 bool linear_i8_packed_avx2_available() noexcept;
53
54 void linear_i8_packed_avx2(const PackedLinearI8& layer,
55                            std::span<const float> input,
56                            std::span<float> output);
57
58 QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
59                                                  std::int32_t output_size,
60                                                  float scale);
61
62 std::vector<float> make_deterministic_input(std::int32_t input_size);
63

```


[illegible]

```

86                                     std::size_t row_begin←
      ↪ ,
87                                     std::size_t row_end) ←
      ↪ noexcept;
88
89 [[nodiscard]] F32RowMax max_dot_f32_rows_unchecked(const float* weights,
90                                                     const float* input,
91                                                     std::size_t input_size,
92                                                     std::size_t row_begin,
93                                                     std::size_t row_end) ←
      ↪ noexcept;
94
95 [[nodiscard]] F32RowMax max_dot_f32_rows_avx2_unchecked(const float* weights,
96                                                         const float* input,
97                                                         std::size_t input_size,
98                                                         std::size_t row_begin,
99                                                         std::size_t row_end) ←
      ↪ noexcept;
100
101 } // namespace lcqi

```

tokenizer 头文件固定 BOS/EOS/UNK、词表和 template hash。prompt 进入模型前，必须先变成稳定 token id；未知 token 行为也必须被写进合同。

Listing 4.6 include/lcqi/tokenizer.hpp

```

1 #pragma once
2
3 #include <stdint>
4 #include <filesystem>
5 #include <string>
6 #include <string_view>
7 #include <vector>
8
9 namespace lcqi {
10
11 struct TokenEntry {
12     std::string text;
13     std::int32_t id = 0;
14 };
15
16 struct TokenizerModel {
17     std::int32_t bos_id = -1;
18     std::int32_t eos_id = -1;
19     std::int32_t unk_id = -1;
20     std::string chat_template_hash;
21     std::vector<TokenEntry> vocab;
22 };
23
24 [[nodiscard]] TokenizerModel load_tokenizer(const std::filesystem::path& path);
25
26 [[nodiscard]] std::vector<std::int32_t> encode_prompt(
27     const TokenizerModel& tokenizer,

```

```

28     std::string_view prompt);
29
30 [[nodiscard]] std::uint64_t tokenizer_contract_hash(const TokenizerModel& ↵
    ↵ tokenizer);
31
32 } // namespace lcqi

```

safetensors 头文件固定 dtype、shape、offset 和 payload 边界。它负责模型字节怎样被解释，不应该和数学推理逻辑混在一起。

Listing 4.7 include/lcqi/safetensors.hpp

```

1 #pragma once
2
3 #include <stdint>
4 #include <filesystem>
5 #include <span>
6 #include <string>
7 #include <string_view>
8 #include <utility>
9 #include <vector>
10
11 namespace lcqi {
12
13 struct SafeTensorEntry {
14     std::string name;
15     std::string dtype;
16     std::vector<std::int64_t> shape;
17     std::uint64_t data_begin = 0;
18     std::uint64_t data_end = 0;
19
20     [[nodiscard]] std::uint64_t byte_size() const;
21 };
22
23 struct SafeTensorManifest {
24     std::uint64_t header_size = 0;
25     std::uint64_t data_start_offset = 0;
26     std::vector<std::pair<std::string, std::string>> metadata;
27     std::vector<SafeTensorEntry> tensors;
28
29     [[nodiscard]] const SafeTensorEntry* find_tensor(std::string_view name) ↵
    ↵ const;
30 };
31
32 struct SafeTensorFile {
33     std::filesystem::path path;
34     SafeTensorManifest manifest;
35     std::vector<std::uint8_t> bytes;
36 };
37
38 [[nodiscard]] SafeTensorManifest load_safetensors_manifest(
39     const std::filesystem::path& path);
40

```

```

41 [[nodiscard]] SafeTensorFile load_safetensors_file(const std::filesystem::path&←
    ↪ path);
42
43 [[nodiscard]] std::vector<float> read_safetensor_f32_tensor(
44     const SafeTensorFile& file,
45     std::string_view name);
46
47 [[nodiscard]] std::span<const std::uint8_t> read_safetensor_tensor_bytes(
48     const SafeTensorFile& file,
49     std::string_view name);
50
51 } // namespace lcqi

```

GGUF 与 Q4_K 合同

GGUF 头文件固定 llama.cpp 生态模型文件的读取边界：版本、metadata、alignment、tensor 名、shape、ggml type 和 tensor payload offset。它也保留 tokenizer 需要的字符串数组 metadata，例如 `tokenizer.ggml.tokens` 和 `tokenizer.ggml.merges`。它不负责模型图执行，只把“真实 GGUF 文件里有哪些 tensor、每个 tensor 在哪里、哪些 metadata 能支撑 tokenizer”变成可检查对象。

Listing 4.8 include/lcqi/gguf.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <filesystem>
5 #include <span>
6 #include <string>
7 #include <string_view>
8 #include <vector>
9
10 namespace lcqi {
11
12 enum class GgmlType : std::int32_t {
13     f32 = 0,
14     f16 = 1,
15     q4_0 = 2,
16     q4_1 = 3,
17     q5_0 = 6,
18     q5_1 = 7,
19     q8_0 = 8,
20     q8_1 = 9,
21     q2_k = 10,
22     q3_k = 11,
23     q4_k = 12,
24     q5_k = 13,
25     q6_k = 14,
26     q8_k = 15,
27     i8 = 24,
28     i16 = 25,

```



```

29     i32 = 26,
30     i64 = 27,
31     f64 = 28,
32     bf16 = 30,
33     q1_0 = 41,
34 };
35
36 struct GgmlTypeLayout {
37     std::int64_t block_size = 1;
38     std::uint64_t type_size = 0;
39     const char* name = "";
40     bool quantized = false;
41 };
42
43 struct GgufMetadataEntry {
44     std::string key;
45     std::string type;
46     std::string value_preview;
47     std::vector<std::string> string_values;
48 };
49
50 struct GgufTensorInfo {
51     std::string name;
52     std::vector<std::int64_t> shape;
53     GgmlType type = GgmlType::f32;
54     std::uint64_t relative_offset = 0;
55     std::uint64_t absolute_offset = 0;
56     std::uint64_t byte_size = 0;
57
58     [[nodiscard]] std::uint64_t element_count() const;
59 };
60
61 struct GgufManifest {
62     std::uint32_t version = 0;
63     std::uint64_t tensor_count = 0;
64     std::uint64_t metadata_count = 0;
65     std::uint64_t alignment = 32;
66     std::uint64_t data_offset = 0;
67     std::vector<GgufMetadataEntry> metadata;
68     std::vector<GgufTensorInfo> tensors;
69
70     [[nodiscard]] const GgufTensorInfo* find_tensor(std::string_view name) ←
    ↪ const;
71     [[nodiscard]] const GgufMetadataEntry* find_metadata(std::string_view key) ←
    ↪ const;
72 };
73
74 [[nodiscard]] GgmlTypeLayout ggml_type_layout(GgmlType type);
75
76 [[nodiscard]] const char* ggml_type_name(GgmlType type);
77
78 [[nodiscard]] std::uint64_t ggml_tensor_byte_size(GgmlType type,

```

```

79                                     std::span<const std::int64_t><←
    ↪ shape);
80
81 [[nodiscard]] GgufManifest load_gguf_manifest(const std::filesystem::path& path<←
    ↪ );
82
83 [[nodiscard]] std::vector<std::uint8_t> read_gguf_tensor_bytes(
84     const std::filesystem::path& path,
85     const GgufTensorInfo& tensor);
86
87 } // namespace lcqi

```

GGML tensor 头文件是 GGUF 与模型数学之间的格式层。SmolLM2 的 Q4_K_M 文件内部并不是所有矩阵都是 Q4_K：远端 caw 扫描显示它包含 F32、Q8_0、Q5_0、Q6_K 和 Q4_K。因此完整模型加载不能只写一个 Q4_K 解码器，必须把这些格式统一变成后续执行层能消费的张量。

Listing 4.9 include/lcqi/ggml_tensors.hpp

```

1 #pragma once
2
3 #include <lcqi/gguf.hpp>
4
5 #include <stdint>
6 #include <span>
7 #include <vector>
8
9 namespace lcqi {
10
11 constexpr std::int64_t LCQI_Q8_0_BLOCK_VALUES = 32;
12 constexpr std::int64_t LCQI_Q8_0_BLOCK_BYTES = 34;
13 constexpr std::int64_t LCQI_Q5_0_BLOCK_VALUES = 32;
14 constexpr std::int64_t LCQI_Q5_0_BLOCK_BYTES = 22;
15 constexpr std::int64_t LCQI_Q6_K_BLOCK_VALUES = 256;
16 constexpr std::int64_t LCQI_Q6_K_BLOCK_BYTES = 210;
17
18 [[nodiscard]] float ggml_f16_to_f32(std::uint16_t bits);
19
20 [[nodiscard]] std::vector<float> dequantize_ggml_tensor(
21     GgmlType type,
22     std::span<const std::uint8_t> bytes,
23     std::int64_t element_count);
24
25 void dequantize_ggml_tensor_to(GgmlType type,
26                                std::span<const std::uint8_t> bytes,
27                                std::span<float> output);
28
29 } // namespace lcqi

```

Q4_K 头文件固定本轮量化切片的数学合同。一个 Q4_K block 解码出 256 个权重，Q8_K input 是每 256 个激活临时量化一次。这里暴露的 `matvec_q4_k_q8` 是单算子入口，不是完整 LLaMA-like decoder；完整 decoder 还要接 RMSNorm、RoPE、

attention、SwiGLU、KV cache 和 sampler。

Listing 4.10 `include/lcqi/q4_k.hpp`

```

1 #pragma once
2
3 #include <array>
4 #include <cstdint>
5 #include <span>
6 #include <vector>
7
8 namespace lcqi {
9
10 constexpr std::int64_t LCQI_QK_K_BLOCK_VALUES = 256;
11 constexpr std::int64_t LCQI_Q4_K_BLOCK_BYTES = 144;
12 constexpr std::int64_t LCQI_Q8_K_BLOCK_BYTES = 292;
13 constexpr std::int64_t LCQI_Q4_K_SUBBLOCK_VALUES = 32;
14 constexpr std::int64_t LCQI_Q4_K_SUBBLOCKS = 8;
15 constexpr std::int64_t LCQI_Q8_K_BSUM_GROUPS = 16;
16
17 struct Q8KBlock {
18     float d = 0.0F;
19     std::array<std::int8_t, LCQI_QK_K_BLOCK_VALUES> qs{};
20     std::array<std::int16_t, LCQI_Q8_K_BSUM_GROUPS> bsums{};
21 };
22
23 [[nodiscard]] std::vector<float> dequantize_q4_k(std::span<const std::uint8_t> ↵
    ↵ bytes,
    std::int64_t element_count);
24
25
26 void dequantize_q4_k_to(std::span<const std::uint8_t> bytes,
27     std::span<float> output);
28
29 [[nodiscard]] std::vector<Q8KBlock> quantize_q8_k_input(std::span<const float> ↵
    ↵ input);
30
31 void quantize_q8_k_input_to(std::span<const float> input,
32     std::span<Q8KBlock> output);
33
34 [[nodiscard]] bool q4_k_q8_avx2_available() noexcept;
35
36 [[nodiscard]] const char* q4_k_q8_active_backend() noexcept;
37
38 float dot_q4_k_f32(std::span<const std::uint8_t> q4_blocks,
39     std::span<const float> input);
40
41 float dot_q4_k_q8(std::span<const std::uint8_t> q4_blocks,
42     std::span<const Q8KBlock> input);
43
44 void matvec_q4_k_f32(std::span<const std::uint8_t> q4_rows,
45     std::int64_t row_count,
46     std::int64_t column_count,
47     std::span<const float> input,

```

```

48         std::span<float> output);
49
50 void matvec_q4_k_q8(std::span<const std::uint8_t> q4_rows,
51                    std::int64_t row_count,
52                    std::int64_t column_count,
53                    std::span<const Q8KBlock> input,
54                    std::span<float> output);
55
56 void matvec_q4_k_q8_rows_unchecked(std::span<const std::uint8_t> q4_rows,
57                                    std::int64_t row_count,
58                                    std::int64_t column_count,
59                                    std::span<const Q8KBlock> input,
60                                    std::span<float> output,
61                                    std::int64_t row_begin,
62                                    std::int64_t row_end) noexcept;
63
64 } // namespace lcqi

```

GGML matvec 头文件固定 `Q5_0/Q6_K/Q8_0` direct 实验路径的合同。`matvec_ggml_quantized_f32` 是 exact oracle: 它不先 materialize 整个矩阵, 而是直接从 GGUF block bytes 和 f32 input 做点积, 用来对照解码后的 f32 权重。`quantize_q8_0_input` 把 activation 每 32 个元素临时量化一次, `matvec_ggml_quantized_q8_0` 再执行低比特权重乘 `Q8_0` activation。这个头文件只承诺单算子数学和边界检查, 不承诺端到端更快; 是否能进入默认路径必须看 caw 同输入 A/B。

Listing 4.11 include/lcqi/ggml_matvec.hpp

```

1 #pragma once
2
3 #include <lcqi/f32_kernels.hpp>
4 #include <lcqi/gguf.hpp>
5
6 #include <array>
7 #include <cstdint>
8 #include <span>
9 #include <vector>
10
11 namespace lcqi {
12
13 constexpr std::int64_t LCQI_Q8_0_INPUT_BLOCK_VALUES = 32;
14
15 struct Q8_0InputBlock {
16     float d = 0.0F;
17     std::array<std::int8_t, LCQI_Q8_0_INPUT_BLOCK_VALUES> qs{};
18     std::int16_t sum = 0;
19 };
20
21 [[nodiscard]] bool ggml_type_has_f32_direct_matvec(GgmlType type) noexcept;
22
23 [[nodiscard]] bool ggml_type_has_q8_0_direct_matvec(GgmlType type) noexcept;
24

```

```

25 [[nodiscard]] std::vector<Q8_0InputBlock> quantize_q8_0_input(std::span<const float> ←
    ↪ float> input);
26
27 void quantize_q8_0_input_to(std::span<const float> input,
28                             std::span<Q8_0InputBlock> output);
29
30 void matvec_ggml_quantized_f32(GgmlType type,
31                                std::span<const std::uint8_t> rows,
32                                std::int64_t row_count,
33                                std::int64_t column_count,
34                                std::span<const float> input,
35                                std::span<float> output);
36
37 void matvec_ggml_quantized_q8_0(GgmlType type,
38                                 std::span<const std::uint8_t> rows,
39                                 std::int64_t row_count,
40                                 std::int64_t column_count,
41                                 std::span<const Q8_0InputBlock> input,
42                                 std::span<float> output);
43
44 void matvec_ggml_quantized_q8_0_rows_unchecked(GgmlType type,
45                                                  std::span<const std::uint8_t> ←
    ↪ rows,
46                                                  std::int64_t row_count,
47                                                  std::int64_t column_count,
48                                                  std::span<const Q8_0InputBlock> ←
    ↪ input,
49                                                  std::span<float> output,
50                                                  std::int64_t row_begin,
51                                                  std::int64_t row_end);
52
53 void matvec_ggml_quantized_q8_0_batch_rows_unchecked(
54     GgmlType type,
55     std::span<const std::uint8_t> rows,
56     std::int64_t row_count,
57     std::int64_t column_count,
58     std::span<const Q8_0InputBlock> input,
59     std::int64_t batch_size,
60     std::span<float> output,
61     std::int64_t output_stride,
62     std::int64_t row_begin,
63     std::int64_t row_end);
64
65 [[nodiscard]] F32RowMax max_dot_ggml_quantized_q8_0(
66     GgmlType type,
67     std::span<const std::uint8_t> rows,
68     std::int64_t row_count,
69     std::int64_t column_count,
70     std::span<const Q8_0InputBlock> input);
71
72 [[nodiscard]] F32RowMax max_dot_ggml_quantized_q8_0_rows_unchecked(
73     GgmlType type,

```

```

74     std::span<const std::uint8_t> rows,
75     std::int64_t row_count,
76     std::int64_t column_count,
77     std::span<const Q8_0InputBlock> input,
78     std::int64_t row_begin,
79     std::int64_t row_end);
80
81 } // namespace lcqi

```

LLaMA GGUF 头文件把真实 GGUF 文件接到 decoder 数学层。当前实现有三种权重执行状态：关闭 direct 时是 `f32_dequantized_reference`；默认是 `gguf_mixed_q4_k_direct`，只让 shape 兼容的 Q4_K linear tensor 保留原始 block bytes；显式设置 `LCQI_LLAMA_GGML_DIRECT=1` 时才进入 `gguf_mixed_quantized_direct_experimental`，尝试把 Q5_0/Q6_K/Q8_0 也接入 direct。这个边界很重要，因为它防止把“能跑真实模型”误写成“已经拥有 llama.cpp 同等级的低比特执行引擎”。

Listing 4.12 include/lcqi/llama_gguf.hpp

```

1  #pragma once
2
3  #include <lcqi/gpt2_reference.hpp>
4  #include <lcqi/gguf.hpp>
5  #include <lcqi/q5_0_packed.hpp>
6  #include <lcqi/reference_decoder.hpp>
7
8  #include <cstdint>
9  #include <cstdint>
10 #include <filesystem>
11 #include <span>
12 #include <string>
13 #include <vector>
14
15 namespace lcqi {
16
17 enum class LlamaGgufLinearStorage : std::uint8_t {
18     f32,
19     q4_k,
20     q5_0_packed,
21     q6_k_direct,
22     q8_0_direct,
23     ggml_quantized,
24 };
25
26 struct LlamaGgufLinearWeights {
27     std::int32_t input_size = 0;
28     std::int32_t output_size = 0;
29     GgmlType source_type = GgmlType::f32;
30     LlamaGgufLinearStorage storage = LlamaGgufLinearStorage::f32;
31     std::vector<float> f32_weights;
32     std::vector<std::uint8_t> quantized_bytes;
33     std::vector<Q5_0PackedBlock> q5_0_packed_blocks;

```

```

34 };
35
36 struct LlamaGgufDecoderLayerWeights {
37     std::vector<float> rms_attention_weight;
38     LlamaGgufLinearWeights wq;
39     LlamaGgufLinearWeights wk;
40     LlamaGgufLinearWeights wv;
41     LlamaGgufLinearWeights wo;
42     std::vector<float> rms_mlp_weight;
43     LlamaGgufLinearWeights w_gate;
44     LlamaGgufLinearWeights w_up;
45     LlamaGgufLinearWeights w_down;
46 };
47
48 struct LlamaGgufDecoderModel {
49     DecoderConfig config;
50     std::vector<float> token_embedding;
51     LlamaGgufLinearWeights tied_lm_head;
52     std::vector<LlamaGgufDecoderLayerWeights> layers;
53     std::vector<float> final_norm_weight;
54     LlamaGgufLinearWeights lm_head;
55 };
56
57 struct LlamaGgufModel {
58     LlamaGgufDecoderModel decoder;
59     std::string architecture;
60     std::string name;
61     std::int32_t bos_token_id = -1;
62     std::int32_t eos_token_id = -1;
63     std::int32_t unknown_token_id = -1;
64     bool lm_head_tied_to_embedding = true;
65 };
66
67 struct LlamaGgufLoadReport {
68     double manifest_ms = 0.0;
69     double weights_ms = 0.0;
70     std::uint64_t quantized_weight_bytes = 0;
71     std::uint64_t f32_weight_bytes = 0;
72     std::uint64_t direct_quantized_weight_bytes = 0;
73     std::uint64_t fallback_dequantized_weight_bytes = 0;
74     std::uint64_t q5_0_packed_weight_bytes = 0;
75     std::uint64_t q5_0_batch_f32_weight_bytes = 0;
76     std::uint64_t q6_k_direct_weight_bytes = 0;
77     std::uint64_t q8_0_direct_weight_bytes = 0;
78     std::int64_t tensors_loaded = 0;
79     std::int64_t q4_k_direct_tensors = 0;
80     std::int64_t q5_0_packed_tensors = 0;
81     std::int64_t q6_k_direct_tensors = 0;
82     std::int64_t q8_0_direct_tensors = 0;
83     std::int64_t ggml_direct_tensors = 0;
84     std::int64_t f32_fallback_tensors = 0;
85 };

```



```

86
87 struct LlamaGgufLoadedModel {
88     LlamaGgufModel model;
89     LlamaGgufLoadReport report;
90 };
91
92 struct LlamaGgufHotspotReport {
93     double rms_norm_ms = 0.0;
94     double attention_ms = 0.0;
95     double rope_ms = 0.0;
96     double wq_ms = 0.0;
97     double wk_ms = 0.0;
98     double wv_ms = 0.0;
99     double wo_ms = 0.0;
100    double w_gate_ms = 0.0;
101    double w_up_ms = 0.0;
102    double w_down_ms = 0.0;
103    double lm_head_ms = 0.0;
104    double q4_k_direct_ms = 0.0;
105    double q5_0_packed_ms = 0.0;
106    double q5_0_batch_f32_ms = 0.0;
107    double q6_k_direct_ms = 0.0;
108    double q8_0_direct_ms = 0.0;
109    double ggml_direct_ms = 0.0;
110    double f32_fallback_ms = 0.0;
111    std::uint64_t q4_k_direct_calls = 0;
112    std::uint64_t q5_0_packed_calls = 0;
113    std::uint64_t q5_0_batch_f32_calls = 0;
114    std::uint64_t q6_k_direct_calls = 0;
115    std::uint64_t q8_0_direct_calls = 0;
116    std::uint64_t ggml_direct_calls = 0;
117    std::uint64_t f32_fallback_calls = 0;
118 };
119
120 struct LlamaGgufExecutionOptions {
121     std::int32_t worker_count = 0;
122     std::int32_t parallel_min_rows = 512;
123 };
124
125 struct LlamaGgufGenerationResult {
126     std::vector<std::int32_t> prompt_ids;
127     std::vector<std::int32_t> generated_ids;
128     LlamaGgufHotspotReport hotspots;
129     double load_ms = 0.0;
130     double prefill_ms = 0.0;
131     double decode_ms = 0.0;
132     double total_ms = 0.0;
133     std::size_t kv_cache_bytes = 0;
134     std::size_t prefill_steps = 0;
135     std::size_t decode_steps = 0;
136     std::int32_t worker_count = 1;
137     std::int32_t predicted_first_token = -1;

```

```

138 };
139
140 [[nodiscard]] LlamaGgufLoadedModel load_llama_gguf_reference_model(
141     const std::filesystem::path& gguf_path);
142
143 [[nodiscard]] Gpt2Tokenizer load_gpt2_tokenizer_from_gguf(
144     const std::filesystem::path& gguf_path);
145
146 [[nodiscard]] std::vector<std::int32_t> llama_gguf_chat_prompt_ids(
147     const Gpt2Tokenizer& tokenizer,
148     std::string_view user_prompt,
149     std::int32_t bos_token_id);
150
151 [[nodiscard]] LlamaGgufGenerationResult llama_gguf_generate_greedy(
152     const LlamaGgufModel& model,
153     std::span<const std::int32_t> prompt_ids,
154     std::int32_t max_new_tokens);
155
156 [[nodiscard]] LlamaGgufGenerationResult llama_gguf_generate_greedy(
157     const LlamaGgufModel& model,
158     std::span<const std::int32_t> prompt_ids,
159     std::int32_t max_new_tokens,
160     const LlamaGgufExecutionOptions& options);
161
162 } // namespace lcqi

```

Reference Decoder 与 GPT-2 合同

reference decoder 头文件定义张量、层配置、trace checkpoint、采样结果和 decode 函数。它是定位推理错误的主要合同，不只返回最终 token。

Listing 4.13 include/lcqi/reference_decoder.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <span>
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct LinearWeightsF32 {
11     std::int32_t input_size = 0;
12     std::int32_t output_size = 0;
13     std::vector<float> weights;
14     std::vector<float> bias;
15 };
16
17 struct DecoderConfig {
18     std::int32_t hidden_size = 0;
19     std::int32_t query_heads = 0;

```

```

20     std::int32_t kv_heads = 0;
21     std::int32_t head_dim = 0;
22     std::int32_t intermediate_size = 0;
23     std::int32_t vocab_size = 0;
24     std::int32_t max_context = 0;
25     float rms_epsilon = 1.0e-5F;
26     float rope_base = 10000.0F;
27 };
28
29 struct DecoderLayerWeightsF32 {
30     std::vector<float> rms_attention_weight;
31     LinearWeightsF32 wq;
32     LinearWeightsF32 wk;
33     LinearWeightsF32 wv;
34     LinearWeightsF32 wo;
35     std::vector<float> rms_mlp_weight;
36     LinearWeightsF32 w_gate;
37     LinearWeightsF32 w_up;
38     LinearWeightsF32 w_down;
39 };
40
41 struct ReferenceDecoderModel {
42     DecoderConfig config;
43     std::vector<float> token_embedding;
44     std::vector<DecoderLayerWeightsF32> layers;
45     std::vector<float> final_norm_weight;
46     LinearWeightsF32 lm_head;
47 };
48
49 struct ReferenceDecodeResult {
50     std::vector<float> logits;
51     std::int32_t predicted_token = 0;
52 };
53
54 struct ReferenceTensorCheckpoint {
55     std::string name;
56     std::int32_t token_position = 0;
57     std::int32_t layer_id = 0;
58     std::vector<std::int32_t> shape;
59     std::vector<std::int32_t> stride;
60     std::string dtype;
61     std::string layout;
62     float checksum = 0.0F;
63     float max_abs = 0.0F;
64     std::vector<float> values;
65 };
66
67 struct ReferenceDecodeTraceResult {
68     ReferenceDecodeResult result;
69     std::vector<ReferenceTensorCheckpoint> checkpoints;
70 };
71

```

```

72 class ReferenceKVCache {
73 public:
74     ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count);
75
76     void append(std::int32_t layer_id,
77                 std::int32_t model_position,
78                 std::span<const float> key,
79                 std::span<const float> value);
80
81     [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
82                                              std::int32_t model_position,
83                                              std::int32_t kv_head) const;
84
85     [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
86                                              std::int32_t model_position,
87                                              std::int32_t kv_head) const;
88
89     [[nodiscard]] std::int32_t layer_count() const noexcept;
90     [[nodiscard]] std::int32_t filled_tokens() const noexcept;
91
92 private:
93     [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
94                                           std::int32_t model_position,
95                                           std::int32_t kv_head) const;
96
97     void validate_address(std::int32_t layer_id,
98                          std::int32_t model_position,
99                          std::int32_t kv_head) const;
100
101     DecoderConfig config_;
102     std::int32_t layer_count_ = 0;
103     std::int32_t filled_tokens_ = 0;
104     std::vector<float> keys_;
105     std::vector<float> values_;
106     std::vector<std::uint8_t> written_;
107 };
108
109 void rms_norm(std::span<const float> input,
110              std::span<const float> weight,
111              float epsilon,
112              std::span<float> output);
113
114 void linear_f32(const LinearWeightsF32& layer,
115               std::span<const float> input,
116               std::span<float> output);
117
118 void apply_rope(std::span<float> heads,
119                std::int32_t head_count,
120                std::int32_t head_dim,
121                std::int32_t model_position,
122                float rope_base);
123
124 void attention_decode(const DecoderConfig& config,

```

```

124         const ReferenceKVCache& cache,
125         std::int32_t layer_id,
126         std::int32_t model_position,
127         std::span<const float> query,
128         std::span<float> output);
129
130 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
131                                         std::span<const std::int32_t> ↵
132                                         ↵ token_ids);
133
134 ReferenceDecodeTraceResult run_reference_decode_with_trace(
135     const ReferenceDecoderModel& model,
136     std::span<const std::int32_t> token_ids);
137
138 ReferenceDecoderModel make_tiny_reference_decoder_model();
139 } // namespace lcqi

```

GPT-2 reference 头文件定义 GPT-2 配置、byte-level BPE tokenizer、HuggingFace safetensors 加载、forward、cached forward 和 greedy generation。它单独存在，是因为 GPT-2 使用 LayerNorm、绝对位置嵌入、packed QKV、GELU 和 tied lm head，不能直接塞进 LLaMA-like reference decoder。读 `Gpt2CachedGreedyDecoder` 时要区分三个入口：`step()` 推进一个 token 并返回 greedy 预测；`step_with_logits()` 保留完整 logits 给测试和对照；`advance_without_prediction()` 只推进 embedding、Transformer layers 和 KV cache，用在 prompt prefill 的前缀 token 上。第三个入口不是“少算一点但可能改变结果”，而是删掉不会被任何 caller 使用的预测结果。

Listing 4.14 include/lcqi/gpt2_reference.hpp

```

1 #pragma once
2
3 #include <stdint>
4 #include <filesystem>
5 #include <memory>
6 #include <span>
7 #include <string>
8 #include <string_view>
9 #include <unordered_map>
10 #include <vector>
11
12 namespace lcqi {
13
14 struct Gpt2Config {
15     std::int32_t hidden_size = 0;
16     std::int32_t head_count = 0;
17     std::int32_t layer_count = 0;
18     std::int32_t max_positions = 0;
19     std::int32_t vocab_size = 0;
20     std::int32_t intermediate_size = 0;
21     std::int32_t bos_token_id = -1;
22     std::int32_t eos_token_id = -1;

```

```

23     float layer_norm_epsilon = 1.0e-5F;
24     std::string activation_function = "gelu_new";
25 };
26
27 struct Gpt2LinearF32 {
28     std::int32_t input_size = 0;
29     std::int32_t output_size = 0;
30     std::vector<float> weights;
31     std::vector<float> bias;
32 };
33
34 struct Gpt2LayerWeightsF32 {
35     std::vector<float> ln_1_weight;
36     std::vector<float> ln_1_bias;
37     Gpt2LinearF32 c_attn;
38     Gpt2LinearF32 c_proj;
39     std::vector<float> ln_2_weight;
40     std::vector<float> ln_2_bias;
41     Gpt2LinearF32 c_fc;
42     Gpt2LinearF32 mlp_c_proj;
43 };
44
45 struct Gpt2ReferenceModel {
46     Gpt2Config config;
47     std::vector<float> token_embedding;
48     std::vector<float> position_embedding;
49     std::vector<Gpt2LayerWeightsF32> layers;
50     std::vector<float> final_ln_weight;
51     std::vector<float> final_ln_bias;
52     std::vector<float> lm_head_weight;
53     bool tie_lm_head_to_embedding = true;
54 };
55
56 struct Gpt2ForwardResult {
57     std::vector<float> logits;
58     std::int32_t predicted_token = 0;
59 };
60
61 struct Gpt2HotspotProfile {
62     std::int64_t decoder_steps = 0;
63     std::int64_t layer_steps = 0;
64     double total_step_ms = 0.0;
65     double embedding_ms = 0.0;
66     double layer_norm_ms = 0.0;
67     double final_norm_ms = 0.0;
68     double qkv_projection_ms = 0.0;
69     double kv_append_ms = 0.0;
70     double attention_ms = 0.0;
71     double attention_projection_ms = 0.0;
72     double residual_add_ms = 0.0;
73     double mlp_fc_ms = 0.0;
74     double gelu_ms = 0.0;

```

```

75     double mlp_projection_ms = 0.0;
76     double lm_head_ms = 0.0;
77     double logits_result_ms = 0.0;
78 };
79
80 struct Gpt2ExecutionOptions {
81     std::int32_t worker_count = 0;
82     std::int32_t parallel_min_rows = 512;
83 };
84
85 class Gpt2KvCache;
86
87 namespace detail {
88 class Gpt2ParallelWorkerPool;
89
90 void attend_cached_position(const Gpt2KvCache& cache,
91                             std::int32_t layer_id,
92                             std::int32_t model_position,
93                             std::span<const float> query,
94                             std::span<float> scores,
95                             std::span<float> output);
96 } // namespace detail
97
98 class Gpt2ForwardWorkspace {
99 public:
100     explicit Gpt2ForwardWorkspace(const Gpt2Config& config);
101
102     void reset_for_config(const Gpt2Config& config);
103
104     [[nodiscard]] std::span<float> hidden() noexcept;
105     [[nodiscard]] std::span<float> normed() noexcept;
106     [[nodiscard]] std::span<float> query() noexcept;
107     [[nodiscard]] std::span<float> key() noexcept;
108     [[nodiscard]] std::span<float> value() noexcept;
109     [[nodiscard]] std::span<float> attention() noexcept;
110     [[nodiscard]] std::span<float> projected() noexcept;
111     [[nodiscard]] std::span<float> qkv_packed() noexcept;
112     [[nodiscard]] std::span<float> mlp_fc() noexcept;
113     [[nodiscard]] std::span<float> mlp_out() noexcept;
114     [[nodiscard]] std::span<float> logits() noexcept;
115     [[nodiscard]] std::span<float> scores_prefix(std::int32_t count);
116
117 private:
118     Gpt2Config config_;
119     std::vector<float> hidden_;
120     std::vector<float> normed_;
121     std::vector<float> query_;
122     std::vector<float> key_;
123     std::vector<float> value_;
124     std::vector<float> attention_;
125     std::vector<float> projected_;
126     std::vector<float> qkv_packed_;

```

```

127     std::vector<float> mlp_fc_;
128     std::vector<float> mlp_out_;
129     std::vector<float> logits_;
130     std::vector<float> scores_;
131 };
132
133 class Gpt2KvCache {
134 public:
135     explicit Gpt2KvCache(const Gpt2Config& config);
136
137     void append(std::int32_t layer_id,
138                std::int32_t model_position,
139                std::span<const float> key,
140                std::span<const float> value);
141
142     [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
143                                              std::int32_t model_position,
144                                              std::int32_t head) const;
145
146     [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
147                                                std::int32_t model_position,
148                                                std::int32_t head) const;
149
150     [[nodiscard]] std::int32_t filled_tokens() const noexcept;
151     [[nodiscard]] std::size_t byte_size() const noexcept;
152
153 private:
154     friend void detail::attend_cached_position(const Gpt2KvCache& cache,
155                                                std::int32_t layer_id,
156                                                std::int32_t model_position,
157                                                std::span<const float> query,
158                                                std::span<float> scores,
159                                                std::span<float> output);
160
161     [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
162                                           std::int32_t model_position,
163                                           std::int32_t head) const;
164     [[nodiscard]] std::size_t base_offset_unchecked(std::int32_t layer_id,
165                                                     std::int32_t model_position↵
166     ↵ ,
167                                                     std::int32_t head) const ↵
168     ↵ noexcept;
169     [[nodiscard]] std::span<const float> key_unchecked(std::int32_t layer_id,
170                                                         std::int32_t ↵
171     ↵ model_position,
172                                                         std::int32_t head) const↵
173     ↵ noexcept;
174     [[nodiscard]] std::span<const float> value_unchecked(std::int32_t layer_id,
175                                                           std::int32_t ↵
176     ↵ model_position,
177                                                           std::int32_t head) ↵
178     ↵ const noexcept;

```



```

173     void validate_address(std::int32_t layer_id,
174                           std::int32_t model_position,
175                           std::int32_t head) const;
176     void validate_written(std::int32_t layer_id,
177                           std::int32_t model_position) const;
178
179     Gpt2Config config_;
180     std::int32_t filled_tokens_ = 0;
181     std::vector<float> keys_;
182     std::vector<float> values_;
183     std::vector<std::uint8_t> written_;
184 };
185
186 class Gpt2CachedGreedyDecoder {
187 public:
188     explicit Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model);
189     Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
190                             Gpt2HotspotProfile* hotspot_profile);
191     Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
192                             Gpt2HotspotProfile* hotspot_profile,
193                             const Gpt2ExecutionOptions& execution_options);
194     ~Gpt2CachedGreedyDecoder();
195
196     Gpt2CachedGreedyDecoder(const Gpt2CachedGreedyDecoder&) = delete;
197     Gpt2CachedGreedyDecoder& operator=(const Gpt2CachedGreedyDecoder&) = delete;
198     Gpt2CachedGreedyDecoder(Gpt2CachedGreedyDecoder&&) noexcept;
199     Gpt2CachedGreedyDecoder& operator=(Gpt2CachedGreedyDecoder&&) noexcept;
200
201     void advance_without_prediction(std::int32_t token_id);
202     [[nodiscard]] std::int32_t step(std::int32_t token_id);
203     [[nodiscard]] Gpt2ForwardResult step_with_logits(std::int32_t token_id);
204     [[nodiscard]] const Gpt2KvCache& cache() const noexcept;
205     [[nodiscard]] std::int32_t filled_tokens() const noexcept;
206     [[nodiscard]] std::size_t kv_cache_bytes() const noexcept;
207     [[nodiscard]] std::int32_t worker_count() const noexcept;
208
209 private:
210     const Gpt2ReferenceModel* model_;
211     Gpt2HotspotProfile* hotspot_profile_;
212     Gpt2ExecutionOptions execution_options_;
213     Gpt2KvCache cache_;
214     Gpt2ForwardWorkspace workspace_;
215     std::unique_ptr<detail::Gpt2ParallelWorkerPool> worker_pool_;
216 };
217
218 struct Gpt2Tokenizer {
219     std::int32_t bos_token_id = -1;
220     std::int32_t eos_token_id = -1;
221     std::unordered_map<std::string, std::int32_t> vocab;
222     std::vector<std::string> id_to_token;
223     std::unordered_map<std::string, std::int32_t> bpe_ranks;

```

```

224 };
225
226 [[nodiscard]] Gpt2Tokenizer load_gpt2_tokenizer(
227     const std::filesystem::path& vocab_json,
228     const std::filesystem::path& merges_txt,
229     std::int32_t bos_token_id,
230     std::int32_t eos_token_id);
231
232 [[nodiscard]] std::vector<std::int32_t> gpt2_encode(
233     const Gpt2Tokenizer& tokenizer,
234     std::string_view text);
235
236 [[nodiscard]] std::string gpt2_decode(
237     const Gpt2Tokenizer& tokenizer,
238     std::span<const std::int32_t> token_ids);
239
240 [[nodiscard]] Gpt2Config load_gpt2_config(const std::filesystem::path& ↵
    ↵ config_json);
241
242 [[nodiscard]] Gpt2ReferenceModel load_gpt2_reference_model(
243     const std::filesystem::path& config_json,
244     const std::filesystem::path& safetensors_path);
245
246 [[nodiscard]] Gpt2ReferenceModel load_gpt2_from_directory(
247     const std::filesystem::path& model_directory);
248
249 [[nodiscard]] Gpt2ForwardResult run_gpt2_forward(
250     const Gpt2ReferenceModel& model,
251     std::span<const std::int32_t> token_ids);
252
253 [[nodiscard]] Gpt2ForwardResult run_gpt2_forward_cached(
254     const Gpt2ReferenceModel& model,
255     Gpt2KvCache& cache,
256     std::int32_t token_id);
257
258 [[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy(
259     const Gpt2ReferenceModel& model,
260     std::span<const std::int32_t> prompt_token_ids,
261     std::int32_t max_new_tokens);
262
263 [[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy_cached(
264     const Gpt2ReferenceModel& model,
265     std::span<const std::int32_t> prompt_token_ids,
266     std::int32_t max_new_tokens);
267
268 [[nodiscard]] Gpt2ReferenceModel make_tiny_gpt2_reference_model();
269
270 } // namespace lcqi

```

最短闭环：从模型文件到 logits

tiny MLP 路径不是为了证明 LCQI 已经是完整大模型引擎，而是为了建立最短的、可手算的、可测试的推理闭环。读者要把模型加载、输入检查、int8 linear、ReLU、第二层 logits、argmax 和 CLI 输出连起来看。

模型加载与基础推理

`model.cpp` 负责解析 tiny 文本模型、权重、scale、bias 和 shape。坏格式、坏维度和坏数值都应该在这里被拒绝，不能拖到 kernel 内部才崩溃。

Listing 4.15 `src/model.cpp`

```

1 #include <lcqi/model.hpp>
2
3 #include <fstream>
4 #include <limits>
5 #include <stdexcept>
6 #include <string>
7
8 namespace lcqi {
9 namespace {
10
11 constexpr std::int32_t LCQI_INT8_MIN_VALUE = -128;
12 constexpr std::int32_t LCQI_INT8_MAX_VALUE = 127;
13
14 std::string read_key(std::istream& input) {
15     std::string key;
16     if (!(input >> key)) {
17         throw std::runtime_error("unexpected end of model file");
18     }
19     return key;
20 }
21
22 void expect_key(std::istream& input, const std::string& expected) {
23     const std::string key = read_key(input);
24     if (key != expected) {
25         throw std::runtime_error("expected key '" + expected + "', got '" + key +
        ↪ + "'");
26     }
27 }
28
29 std::int32_t read_i32(std::istream& input, const std::string& key) {
30     expect_key(input, key);
31     std::int32_t value = 0;
32     if (!(input >> value)) {
33         throw std::runtime_error("invalid int32 value for key '" + key + "'");
34     }
35     return value;
36 }
37
38 float read_float(std::istream& input, const std::string& key) {
39     expect_key(input, key);
40     float value = 0.0F;

```

```

41     if (!(input >> value)) {
42         throw std::runtime_error("invalid float value for key '" + key + "'");
43     }
44     return value;
45 }
46
47 std::vector<std::int8_t> read_i8_vector(std::istream& input,
48                                       const std::string& key,
49                                       std::int32_t count) {
50     expect_key(input, key);
51     if (count < 0) {
52         throw std::runtime_error("negative vector size for key '" + key + "'");
53     }
54     std::vector<std::int8_t> values(static_cast<std::size_t>(count));
55     for (std::int32_t i = 0; i < count; ++i) {
56         std::int32_t raw = 0;
57         if (!(input >> raw)) {
58             throw std::runtime_error("invalid int8 payload for key '" + key + "↵
↵ '");
59         }
60         if (raw < LCQI_INT8_MIN_VALUE || raw > LCQI_INT8_MAX_VALUE) {
61             throw std::runtime_error("int8 payload out of range for key '" + ↵
↵ key + "'");
62         }
63         values[static_cast<std::size_t>(i)] = static_cast<std::int8_t>(raw);
64     }
65     return values;
66 }
67
68 std::vector<float> read_float_vector(std::istream& input,
69                                       const std::string& key,
70                                       std::int32_t count) {
71     expect_key(input, key);
72     if (count < 0) {
73         throw std::runtime_error("negative vector size for key '" + key + "'");
74     }
75     std::vector<float> values(static_cast<std::size_t>(count));
76     for (std::int32_t i = 0; i < count; ++i) {
77         if (!(input >> values[static_cast<std::size_t>(i)])) {
78             throw std::runtime_error("invalid float payload for key '" + key + ↵
↵ '");
79         }
80     }
81     return values;
82 }
83
84 void validate_layer(const QuantizedLinearLayer& layer, const std::string& name)↵
↵ {
85     if (layer.input_size <= 0 || layer.output_size <= 0) {
86         throw std::runtime_error(name + " layer has invalid shape");
87     }
88     const std::size_t expected_weights =

```

```

89     static_cast<std::size_t>(layer.input_size) *
90     static_cast<std::size_t>(layer.output_size);
91     if (layer.weights.size() != expected_weights) {
92         throw std::runtime_error(name + " layer weight size mismatch");
93     }
94     if (layer.bias.size() != static_cast<std::size_t>(layer.output_size)) {
95         throw std::runtime_error(name + " layer bias size mismatch");
96     }
97 }
98
99 std::int32_t checked_element_count(std::int32_t rows,
100                                   std::int32_t columns,
101                                   const std::string& name) {
102     if (rows <= 0 || columns <= 0) {
103         throw std::runtime_error(name + " layer has invalid shape");
104     }
105     const std::int64_t count =
106         static_cast<std::int64_t>(rows) * static_cast<std::int64_t>(columns);
107     if (count > static_cast<std::int64_t>(std::numeric_limits<std::int32_t>::max() ←
108         throw std::runtime_error(name + " layer element count is too large");
109     }
110     return static_cast<std::int32_t>(count);
111 }
112
113 } // namespace
114
115 TinyMlpModel load_model(const std::filesystem::path& path) {
116     std::ifstream input(path);
117     if (!input) {
118         throw std::runtime_error("cannot open model file: " + path.string());
119     }
120
121     expect_key(input, "LCQI_MODEL_V1");
122
123     TinyMlpModel model;
124     model.input_size = read_i32(input, "input_size");
125     model.hidden_size = read_i32(input, "hidden_size");
126     model.output_size = read_i32(input, "output_size");
127
128     const std::int32_t hidden_weight_count =
129         checked_element_count(model.input_size, model.hidden_size, "hidden");
130     const std::int32_t output_weight_count =
131         checked_element_count(model.hidden_size, model.output_size, "output");
132
133     model.hidden.input_size = model.input_size;
134     model.hidden.output_size = model.hidden_size;
135     model.hidden.scale = read_float(input, "hidden_scale");
136     model.hidden.weights = read_i8_vector(
137         input,
138         "hidden_weights",
139         hidden_weight_count);

```

```

140     model.hidden.bias = read_float_vector(input, "hidden_bias", model.↵
↵ hidden_size);
141
142     model.output.input_size = model.hidden_size;
143     model.output.output_size = model.output_size;
144     model.output.scale = read_float(input, "output_scale");
145     model.output.weights = read_i8_vector(
146         input,
147         "output_weights",
148         output_weight_count);
149     model.output.bias = read_float_vector(input, "output_bias", model.↵
↵ output_size);
150
151     validate_layer(model.hidden, "hidden");
152     validate_layer(model.output, "output");
153     return model;
154 }
155
156 } // namespace lcqi

```

`inference.cpp` 执行 int8 linear、ReLU、第二层 logits 和 predicted class。它是系统中最短的“模型资产到输出”路径。

Listing 4.16 src/inference.cpp

```

1 #include <lcqi/inference.hpp>
2
3 #include <algorithm>
4 #include <cmath>
5 #include <stdexcept>
6
7 namespace lcqi {
8 namespace {
9
10 void validate_input_size(std::span<const float> input, std::int32_t expected) {
11     if (input.size() != static_cast<std::size_t>(expected)) {
12         throw std::runtime_error("input size does not match model shape");
13     }
14 }
15
16 std::int32_t argmax(std::span<const float> values) {
17     if (values.empty()) {
18         throw std::runtime_error("cannot take argmax of empty logits");
19     }
20     std::int32_t best_index = 0;
21     float best_value = values[0];
22     for (std::int32_t i = 1; i < static_cast<std::int32_t>(values.size()); ++i)↵
↵ {
23         const float candidate = values[static_cast<std::size_t>(i)];
24         if (candidate > best_value) {
25             best_value = candidate;
26             best_index = i;
27         }

```

```

28     }
29     return best_index;
30 }
31
32 } // namespace
33
34 void linear_i8(const QuantizedLinearLayer& layer,
35               std::span<const float> input,
36               std::span<float> output) {
37     if (input.size() != static_cast<std::size_t>(layer.input_size)) {
38         throw std::runtime_error("linear_i8 input size mismatch");
39     }
40     if (output.size() != static_cast<std::size_t>(layer.output_size)) {
41         throw std::runtime_error("linear_i8 output size mismatch");
42     }
43
44     for (std::int32_t out = 0; out < layer.output_size; ++out) {
45         const std::size_t row_base =
46             static_cast<std::size_t>(out) * static_cast<std::size_t>(layer.↵
↵ input_size);
47         float acc = layer.bias[static_cast<std::size_t>(out)];
48         for (std::int32_t in = 0; in < layer.input_size; ++in) {
49             const std::int8_t weight =
50                 layer.weights[row_base + static_cast<std::size_t>(in)];
51             acc += static_cast<float>(weight) * layer.scale *
52                 input[static_cast<std::size_t>(in)];
53         }
54         output[static_cast<std::size_t>(out)] = acc;
55     }
56 }
57
58 void relu_inplace(std::span<float> values) {
59     for (float& value : values) {
60         value = std::max(0.0F, value);
61     }
62 }
63
64 InferenceResult run_inference(const TinyMlpModel& model,
65                               std::span<const float> input) {
66     validate_input_size(input, model.input_size);
67
68     std::vector<float> hidden(static_cast<std::size_t>(model.hidden_size), 0.0F↵
↵ );
69     std::vector<float> logits(static_cast<std::size_t>(model.output_size), 0.0F↵
↵ );
70
71     linear_i8(model.hidden, input, hidden);
72     relu_inplace(hidden);
73     linear_i8(model.output, hidden, logits);
74
75     InferenceResult result;
76     result.predicted_class = argmax(logits);

```

```

77     result.logits = std::move(logits);
78     return result;
79 }
80
81 } // namespace lcqi

```

`main.cpp` 把模型路径和输入接到 `run_inference`，并打印结果。CLI 应保持薄层，不复制模型解析和推理逻辑。

Listing 4.17 `src/main.cpp`

```

1  #include <lcqi/inference.hpp>
2  #include <lcqi/model.hpp>
3
4  #include <chrono>
5  #include <cstdint>
6  #include <cstdlib>
7  #include <filesystem>
8  #include <iostream>
9  #include <span>
10 #include <stdexcept>
11 #include <string>
12 #include <vector>
13
14 namespace {
15
16 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 1000;
17 constexpr int LCQI_EXPECTED_ARGC_MIN = 2;
18 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
19
20 std::vector<float> parse_input(int argc, char** argv, std::int32_t ↵
    ↵ expected_size) {
21     if (argc < LCQI_EXPECTED_ARGC_MIN + expected_size) {
22         throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...>↵
    ↵ [repeat]");
23     }
24     std::vector<float> input(static_cast<std::size_t>(expected_size));
25     for (std::int32_t i = 0; i < expected_size; ++i) {
26         input[static_cast<std::size_t>(i)] =
27             std::stof(argv[LCQI_EXPECTED_ARGC_MIN + i]);
28     }
29     return input;
30 }
31
32 std::int32_t parse_repeat(int argc, char** argv, std::int32_t input_size) {
33     const int repeat_index = LCQI_EXPECTED_ARGC_MIN + input_size;
34     if (argc <= repeat_index) {
35         return LCQI_DEFAULT_REPEAT;
36     }
37     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
    ↵ repeat_index]));
38     if (repeat <= 0) {
39         throw std::runtime_error("repeat must be positive");

```



```

40     }
41     return repeat;
42 }
43
44 } // namespace
45
46 int main(int argc, char** argv) {
47     try {
48         if (argc < LCQI_EXPECTED_ARGC_MIN) {
49             throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats>↵
↵ ...> [repeat]");
50         }
51
52         const std::filesystem::path model_path = argv[1];
53         const lcqi::TinyMlpModel model = lcqi::load_model(model_path);
54         const std::vector<float> input = parse_input(argc, argv, model.↵
↵ input_size);
55         const std::int32_t repeat = parse_repeat(argc, argv, model.input_size);
56
57         const lcqi::InferenceResult result = lcqi::run_inference(model, input);
58         std::cout << "predicted_class " << result.predicted_class << "\n";
59         std::cout << "logits";
60         for (const float value : result.logits) {
61             std::cout << ' ' << value;
62         }
63         std::cout << "\n";
64
65         const auto begin = std::chrono::steady_clock::now();
66         std::int32_t checksum = 0;
67         for (std::int32_t i = 0; i < repeat; ++i) {
68             checksum += lcqi::run_inference(model, input).predicted_class;
69         }
70         const auto end = std::chrono::steady_clock::now();
71         const std::chrono::duration<double> elapsed = end - begin;
72         const double average_us =
73             elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>↵
↵ >(repeat);
74         std::cout << "repeat " << repeat << "\n";
75         std::cout << "average_us " << average_us << "\n";
76         std::cout << "checksum " << checksum << "\n";
77         return EXIT_SUCCESS;
78     } catch (const std::exception& error) {
79         std::cerr << "error: " << error.what() << "\n";
80         return EXIT_FAILURE;
81     }
82 }

```

Int8 Kernel 基础实现

`int8_kernels.cpp` 包含 `scalar`、`packed`、`workspace`、`deterministic fixture` 和误差比较。它是 AVX2 路径的 `reference` 对照，也是 `benchmark` 的正确性底座。

Listing 4.18 src/int8_kernels.cpp

```

1  #include <lcqi/int8_kernels.hpp>
2
3  #include <lcqi/inference.hpp>
4
5  #include <algorithm>
6  #include <chrono>
7  #include <cmath>
8  #include <stdexcept>
9  #include <string>
10
11 namespace lcqi {
12 namespace {
13
14 constexpr std::int32_t LCQI_I8_PATTERN_MODULUS = 23;
15 constexpr std::int32_t LCQI_I8_PATTERN_CENTER = 11;
16 constexpr std::int32_t LCQI_INPUT_PATTERN_MODULUS = 17;
17 constexpr std::int32_t LCQI_INPUT_PATTERN_CENTER = 8;
18 constexpr float LCQI_INPUT_PATTERN_SCALE = 0.125F;
19 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
20 constexpr std::int32_t LCQI_SIMD_OUTPUT_BLOCK = 8;
21 constexpr std::size_t LCQI_BENCHMARK_BACKEND_COUNT = 3;
22 constexpr const char* LCQI_BACKEND_SCALAR = "scalar";
23 constexpr const char* LCQI_BACKEND_PACKED_SCALAR = "packed_scalar";
24 constexpr const char* LCQI_BACKEND_AVX2 = "avx2";
25
26 void require_positive(std::int32_t value, const char* name) {
27     if (value <= 0) {
28         throw std::runtime_error(std::string(name) + " must be positive");
29     }
30 }
31
32 std::size_t checked_size(std::int32_t value, const char* name) {
33     if (value < 0) {
34         throw std::runtime_error(std::string(name) + " cannot be negative");
35     }
36     return static_cast<std::size_t>(value);
37 }
38
39 void validate_quantized_layer(const QuantizedLinearLayer& layer) {
40     require_positive(layer.input_size, "input_size");
41     require_positive(layer.output_size, "output_size");
42     const std::size_t expected_weights =
43         checked_size(layer.input_size, "input_size") *
44         checked_size(layer.output_size, "output_size");
45     if (layer.weights.size() != expected_weights) {
46         throw std::runtime_error("quantized layer weight size mismatch");
47     }
48     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
49         throw std::runtime_error("quantized layer bias size mismatch");
50     }
51 }

```

```

52
53 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
54     require_positive(value, "value");
55     require_positive(block_size, "block_size");
56     return ((value + block_size - 1) / block_size) * block_size;
57 }
58
59 float checksum(std::span<const float> values) {
60     float sum = 0.0F;
61     for (const float value : values) {
62         sum += value;
63     }
64     return sum;
65 }
66
67 void add_unavailable_backend(KernelBenchmarkResult& result, const char* name) {
68     result.backends.push_back(KernelBackendBenchmark{
69         .name = name,
70         .available = false,
71         .average_us = 0.0,
72         .max_abs_diff = 0.0F,
73         .checksum = 0.0F,
74     });
75 }
76
77 template <typename Callable>
78 double measure_average_us(std::int32_t repeat, Callable&& callable) {
79     require_positive(repeat, "repeat");
80     const auto begin = std::chrono::steady_clock::now();
81     for (std::int32_t index = 0; index < repeat; ++index) {
82         callable();
83     }
84     const auto end = std::chrono::steady_clock::now();
85     const std::chrono::duration<double> elapsed = end - begin;
86     return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>
↵ ↵ >(repeat);
87 }
88
89 } // namespace
90
91 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
92                               std::int32_t output_block_size) {
93     validate_quantized_layer(layer);
94     require_positive(output_block_size, "output_block_size");
95
96     PackedLinearI8 packed;
97     packed.input_size = layer.input_size;
98     packed.output_size = layer.output_size;
99     packed.output_block_size = output_block_size;
100     packed.scale = layer.scale;
101     packed.bias = layer.bias;
102

```

```

103     const std::int32_t padded_outputs =
104         round_up_to_block(layer.output_size, output_block_size);
105     packed.packed_weights.assign(
106         checked_size(padded_outputs, "padded_outputs") *
107         checked_size(layer.input_size, "input_size"),
108         0);
109
110     std::size_t packed_index = 0;
111     for (std::int32_t output_block = 0; output_block < padded_outputs;
112         output_block += output_block_size) {
113         for (std::int32_t input_index = 0; input_index < layer.input_size; ++
114             ↪ input_index) {
115             for (std::int32_t offset = 0; offset < output_block_size; ++offset) ↪
116             ↪ {
117                 const std::int32_t output_index = output_block + offset;
118                 if (output_index < layer.output_size) {
119                     const std::size_t source =
120                         checked_size(output_index, "output_index") *
121                         checked_size(layer.input_size, "input_size") +
122                         checked_size(input_index, "input_index");
123                     packed.packed_weights[packed_index] = layer.weights[source ↪
124                     ↪ ];
125                 }
126                 ++packed_index;
127             }
128         }
129     }
130     return packed;
131 }
132
133 void linear_i8_packed(const PackedLinearI8& layer,
134     std::span<const float> input,
135     std::span<float> output) {
136     std::vector<float> accumulators(
137         checked_size(layer.output_block_size, "output_block_size"),
138         0.0F);
139     linear_i8_packed_with_workspace(layer, input, output, accumulators);
140 }
141
142 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
143     std::span<const float> input,
144     std::span<float> output,
145     std::span<float> accumulator_workspace) {
146     require_positive(layer.input_size, "packed input_size");
147     require_positive(layer.output_size, "packed output_size");
148     require_positive(layer.output_block_size, "packed output_block_size");
149     if (input.size() != checked_size(layer.input_size, "input_size")) {
150         throw std::runtime_error("linear_i8_packed input size mismatch");
151     }
152     if (output.size() != checked_size(layer.output_size, "output_size")) {
153         throw std::runtime_error("linear_i8_packed output size mismatch");
154     }
155 }

```

```

152     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
153         throw std::runtime_error("linear_i8_packed bias size mismatch");
154     }
155
156     const std::int32_t padded_outputs =
157         round_up_to_block(layer.output_size, layer.output_block_size);
158     const std::size_t expected_packed_size =
159         checked_size(padded_outputs, "padded_outputs") *
160         checked_size(layer.input_size, "input_size");
161     if (layer.packed_weights.size() != expected_packed_size) {
162         throw std::runtime_error("linear_i8_packed packed weight size mismatch"
163     ↪ );
164     }
165     if (accumulator_workspace.size() != checked_size(layer.output_block_size, "
166     ↪ output_block_size")) {
167         throw std::runtime_error("linear_i8_packed accumulator workspace size
168     ↪ mismatch");
169     }
170
171     std::size_t packed_index = 0;
172     for (std::int32_t output_block = 0; output_block < padded_outputs;
173         output_block += layer.output_block_size) {
174         std::fill(accumulator_workspace.begin(), accumulator_workspace.end(),
175     ↪ 0.0F);
176         for (std::int32_t offset = 0; offset < layer.output_block_size; ++
177     ↪ offset) {
178             const std::int32_t output_index = output_block + offset;
179             if (output_index < layer.output_size) {
180                 accumulator_workspace[checked_size(offset, "offset")] =
181                 layer.bias[checked_size(output_index, "output_index")];
182             }
183         }
184
185         for (std::int32_t input_index = 0; input_index < layer.input_size; ++
186     ↪ input_index) {
187             const float input_value = input[checked_size(input_index, "
188     ↪ input_index")];
189             for (std::int32_t offset = 0; offset < layer.output_block_size; ++
190     ↪ offset) {
191                 const std::int32_t output_index = output_block + offset;
192                 const std::int8_t weight = layer.packed_weights[packed_index
193     ↪ ++];
194                 if (output_index < layer.output_size) {
195                     accumulator_workspace[checked_size(offset, "offset")] +=
196                     static_cast<float>(weight) * layer.scale * input_value;
197                 }
198             }
199         }
200
201         for (std::int32_t offset = 0; offset < layer.output_block_size; ++
202     ↪ offset) {
203             const std::int32_t output_index = output_block + offset;

```

```

194         if (output_index < layer.output_size) {
195             output[checked_size(output_index, "output_index")] =
196                 accumulator_workspace[checked_size(offset, "offset")];
197         }
198     }
199 }
200 }
201
202 QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
203                                                  std::int32_t output_size,
204                                                  float scale) {
205     require_positive(input_size, "input_size");
206     require_positive(output_size, "output_size");
207     QuantizedLinearLayer layer;
208     layer.input_size = input_size;
209     layer.output_size = output_size;
210     layer.scale = scale;
211     const std::size_t weight_count =
212         checked_size(input_size, "input_size") * checked_size(output_size, "↵
↵ output_size");
213     layer.weights.resize(weight_count);
214     layer.bias.resize(checked_size(output_size, "output_size"));
215     for (std::int32_t output_index = 0; output_index < output_size; ++↵
↵ output_index) {
216         layer.bias[checked_size(output_index, "output_index")] =
217             static_cast<float>((output_index % LCQI_INPUT_PATTERN_MODULUS) -
218                               LCQI_INPUT_PATTERN_CENTER) *
219             LCQI_INPUT_PATTERN_SCALE;
220         for (std::int32_t input_index = 0; input_index < input_size; ++↵
↵ input_index) {
221             const std::int32_t raw =
222                 ((output_index * input_size + input_index) % ↵
↵ LCQI_I8_PATTERN_MODULUS) -
223                 LCQI_I8_PATTERN_CENTER;
224             layer.weights[checked_size(output_index, "output_index") *
225                           checked_size(input_size, "input_size") +
226                           checked_size(input_index, "input_index")] =
227                 static_cast<std::int8_t>(raw);
228         }
229     }
230     return layer;
231 }
232
233 std::vector<float> make_deterministic_input(std::int32_t input_size) {
234     require_positive(input_size, "input_size");
235     std::vector<float> input(checked_size(input_size, "input_size"));
236     for (std::int32_t input_index = 0; input_index < input_size; ++input_index)↵
↵ {
237         input[checked_size(input_index, "input_index")] =
238             static_cast<float>((input_index % LCQI_INPUT_PATTERN_MODULUS) -
239                               LCQI_INPUT_PATTERN_CENTER) *
240             LCQI_INPUT_PATTERN_SCALE;

```

```

241     }
242     return input;
243 }
244
245 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
246     if (lhs.size() != rhs.size()) {
247         throw std::runtime_error("max_abs_diff size mismatch");
248     }
249     float diff = 0.0F;
250     for (std::size_t index = 0; index < lhs.size(); ++index) {
251         diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
252     }
253     return diff;
254 }
255
256 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ←
    ↪ benchmark_case) {
257     require_positive(benchmark_case.input_size, "benchmark input_size");
258     require_positive(benchmark_case.output_size, "benchmark output_size");
259     require_positive(benchmark_case.output_block_size, "benchmark ←
    ↪ output_block_size");
260     require_positive(benchmark_case.repeat, "benchmark repeat");
261
262     const QuantizedLinearLayer layer =
263         make_deterministic_i8_layer(benchmark_case.input_size,
264                                     benchmark_case.output_size,
265                                     0.015625F);
266     const PackedLinearI8 packed = pack_linear_i8(layer, benchmark_case.←
    ↪ output_block_size);
267     const PackedLinearI8 packed_simd = pack_linear_i8(layer, ←
    ↪ LCQI_SIMD_OUTPUT_BLOCK);
268     const std::vector<float> input = make_deterministic_input(benchmark_case.←
    ↪ input_size);
269     std::vector<float> scalar_output(checked_size(benchmark_case.output_size, "←
    ↪ output_size"),
270                                     0.0F);
271     std::vector<float> packed_output(scalar_output.size(), 0.0F);
272     std::vector<float> simd_output(scalar_output.size(), 0.0F);
273     std::vector<float> packed_workspace(
274         checked_size(benchmark_case.output_block_size, "output_block_size"),
275         0.0F);
276
277     linear_i8(layer, input, scalar_output);
278     linear_i8_packed_with_workspace(packed, input, packed_output, ←
    ↪ packed_workspace);
279
280     KernelBenchmarkResult result;
281     result.benchmark_case = benchmark_case;
282     result.backends.reserve(LCQI_BENCHMARK_BACKEND_COUNT);
283     result.backends.push_back(KernelBackendBenchmark{
284         .name = LCQI_BACKEND_SCALAR,
285         .available = true,

```

```

286         .average_us = measure_average_us(benchmark_case.repeat,
287                                           [&]() { linear_i8(layer, input, ↵
↵ scalar_output); }},
288         .max_abs_diff = 0.0F,
289         .checksum = checksum(scalar_output),
290     });
291     result.backends.push_back(KernelBackendBenchmark{
292         .name = LCQI_BACKEND_PACKED_SCALAR,
293         .available = true,
294         .average_us = measure_average_us(
295             benchmark_case.repeat,
296             [&]() { linear_i8_packed_with_workspace(packed,
297                                                         input,
298                                                         packed_output,
299                                                         packed_workspace); }),
300         .max_abs_diff = max_abs_diff(scalar_output, packed_output),
301         .checksum = checksum(packed_output),
302     });
303     if (linear_i8_packed_avx2_available()) {
304         linear_i8_packed_avx2(packed_simd, input, simd_output);
305         result.backends.push_back(KernelBackendBenchmark{
306             .name = LCQI_BACKEND_AVX2,
307             .available = true,
308             .average_us = measure_average_us(
309                 benchmark_case.repeat,
310                 [&]() { linear_i8_packed_avx2(packed_simd, input, simd_output); ↵
↵ }),
311             .max_abs_diff = max_abs_diff(scalar_output, simd_output),
312             .checksum = checksum(simd_output),
313         });
314     } else {
315         add_unavailable_backend(result, LCQI_BACKEND_AVX2);
316     }
317     return result;
318 }
319
320 } // namespace lcqi
321
322 #if !defined(LCQI_ENABLE_AVX2)
323 namespace lcqi {
324
325 bool linear_i8_packed_avx2_available() noexcept {
326     return false;
327 }
328
329 void linear_i8_packed_avx2(const PackedLinearI8&, std::span<const float>, std::span<float>) {
330     throw std::runtime_error("AVX2 packed kernel is not enabled in this build"); ↵
↵ ;
331 }
332
333 } // namespace lcqi

```


334 **#endif**

`int8_kernels_avx2.cpp` 是平台优化路径。读者要先看可用性判断，再看向量化实现；如果机器不支持 AVX2/FMA，报告必须写 `unavailable`。

Listing 4.19 `src/int8_kernels_avx2.cpp`

```

1 #include <lcqi/int8_kernels.hpp>
2
3 #if defined(LCQI_ENABLE_AVX2)
4
5 #include <immintrin.h>
6
7 #include <algorithm>
8 #include <stdint>
9 #include <cstring>
10 #include <stdexcept>
11 #include <string>
12
13 namespace lcqi {
14 namespace {
15
16 constexpr std::int32_t LCQI_AVX2_OUTPUT_BLOCK = 8;
17
18 void require_positive(std::int32_t value, const char* name) {
19     if (value <= 0) {
20         throw std::runtime_error(std::string(name) + " must be positive");
21     }
22 }
23
24 std::size_t checked_size(std::int32_t value, const char* name) {
25     if (value < 0) {
26         throw std::runtime_error(std::string(name) + " cannot be negative");
27     }
28     return static_cast<std::size_t>(value);
29 }
30
31 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
32     require_positive(value, "value");
33     require_positive(block_size, "block_size");
34     return ((value + block_size - 1) / block_size) * block_size;
35 }
36
37 void validate_avx2_contract(const PackedLinearI8& layer,
38                           std::span<const float> input,
39                           std::span<float> output) {
40     require_positive(layer.input_size, "packed input_size");
41     require_positive(layer.output_size, "packed output_size");
42     if (layer.output_block_size != LCQI_AVX2_OUTPUT_BLOCK) {
43         throw std::runtime_error("AVX2 packed kernel requires output_block_size↵
↵ == 8");
44     }
45     if (input.size() != checked_size(layer.input_size, "input_size")) {

```

```

46     throw std::runtime_error("AVX2 packed kernel input size mismatch");
47 }
48 if (output.size() != checked_size(layer.output_size, "output_size")) {
49     throw std::runtime_error("AVX2 packed kernel output size mismatch");
50 }
51 if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
52     throw std::runtime_error("AVX2 packed kernel bias size mismatch");
53 }
54 const std::int32_t padded_outputs =
55     round_up_to_block(layer.output_size, layer.output_block_size);
56 const std::size_t expected_packed_size =
57     checked_size(padded_outputs, "padded_outputs") *
58     checked_size(layer.input_size, "input_size");
59 if (layer.packed_weights.size() != expected_packed_size) {
60     throw std::runtime_error("AVX2 packed kernel packed weight size ↵
↵ mismatch");
61 }
62 }
63
64 } // namespace
65
66 bool linear_i8_packed_avx2_available() noexcept {
67     #if defined(__GNUC__) || defined(__clang__)
68         __builtin_cpu_init();
69         return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
70     #else
71         return true;
72     #endif
73 }
74
75 void linear_i8_packed_avx2(const PackedLinearI8& layer,
76                             std::span<const float> input,
77                             std::span<float> output) {
78     if (!linear_i8_packed_avx2_available()) {
79         throw std::runtime_error("AVX2/FMA is not available on this CPU");
80     }
81     validate_avx2_contract(layer, input, output);
82
83     const std::int32_t padded_outputs =
84         round_up_to_block(layer.output_size, layer.output_block_size);
85     std::size_t packed_index = 0;
86     alignas(32) float block_output[LCQI_AVX2_OUTPUT_BLOCK] = {};
87
88     for (std::int32_t output_block = 0; output_block < padded_outputs;
89         output_block += LCQI_AVX2_OUTPUT_BLOCK) {
90         _mm256 accumulator = _mm256_setzero_ps();
91
92         for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset↵
↵ ) {
93             const std::int32_t output_index = output_block + offset;
94             block_output[checked_size(offset, "offset")] =
95                 output_index < layer.output_size

```

```

96         ? layer.bias[checked_size(output_index, "output_index")]
97         : 0.0F;
98     }
99     accumulator = _mm256_load_ps(block_output);
100
101     for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
102         std::uint64_t packed_bytes = 0;
103         std::memcpy(&packed_bytes,
104             layer.packed_weights.data() + packed_index,
105             sizeof(packed_bytes));
106         const __m128i packed_i8 =
107             _mm_cvtsi64_si128(static_cast<long long>(packed_bytes));
108         packed_index += LCQI_AVX2_OUTPUT_BLOCK;
109         const __m256i weights_i32 = _mm256_cvtepi8_epi32(packed_i8);
110         const __m256 weights_f32 = _mm256_cvtepi32_ps(weights_i32);
111         const __m256 input_scaled =
112             _mm256_set1_ps(input[checked_size(input_index, "input_index")] * layer.scale);
113         accumulator = _mm256_fmadd_ps(weights_f32, input_scaled, accumulator);
114     }
115
116     _mm256_store_ps(block_output, accumulator);
117     for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {
118         const std::int32_t output_index = output_block + offset;
119         if (output_index < layer.output_size) {
120             output[checked_size(output_index, "output_index")] =
121                 block_output[checked_size(offset, "offset")];
122         }
123     }
124 }
125 }
126
127 } // namespace lcqi
128
129 #endif

```

第零部分

Transformer、KV Cache 与服务运行时

第 5 章实践：Transformer 切片与 KV Cache 状态

本章对应原书中的第三册“AI 推理原理、KV cache 与计算账本”和“KV cache 实现细节”。不要把它当作概念复述，本章要把 RMSNorm、Q/K/V、RoPE、GQA、KV slot、position、state edge 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `kv_cache`，核心命令或测试入口是 `lcqi_trace`。

Listing 5.1 第 5 章实践：Transformer 切片与 KV Cache 状态的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  < DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / kv_cache
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：固定 prompt `tok1tok2tok3`，观察第一个 decode step。读者要先手写预期结果，再运行项目观察输出。动作是：把 trace 中 `tokenizer`、`q/k/v`、`kv_cache_slot`、`attention` 和 `logits` 串成状态表。如果输出里没有 `token_position` 或对应字段无法解释，就不要继续优化；先回到 `reference`、输入合同和 `trace` 字段。

从这个特例推出的规律是：KV cache 是请求状态，不是临时 `activation`；`position`、`layer`、`head`、`layout` 和生命周期必须进入 `trace` 这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、`perf`、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、`fallback` 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 KV cache 状态表：每层每个 token 写入哪里、何时读取、何

时禁止复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 6 章实践：Reference Decoder、logits 与采样

本章对应原书中的第三册“Reference decoder 实现”和“推理算法：logits、softmax、采样”。不要把它当作概念复述，本章要把 reference decoder、logits golden、argmax sampler、tokenizer contract hash、BOS/EOS 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `argmax`，核心命令或测试入口是 `lcqi_trace`。

Listing 6.1 第 6 章实践：Reference Decoder、logits 与采样的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / argmax
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：让 trace 固定输出 tokenizer contract 和 logits golden。读者要先手写预期结果，再运行项目观察输出。动作是：比较 generated token、logits 数组和 tokenizer hash。如果输出里没有 `tokenizer_contract` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：采样前必须先证明 logits 可复查；sampler 的随机性、温度和 top-k 只能建立在 reference 通过之后这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交采样报告：argmax、top-k、temperature 三种策略各自需要

哪些额外字段才能复现。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 7 章实践：服务调度、SLO 与资源预算

本章对应原书中的第三册“业务服务实战：请求、调度、队列、取消与资源预算”。不要把它当作概念复述，本章要把 admission、KV budget、queue wait、TTFT、TPOT、cancel、reject、p95 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `ttft_ms`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 7.1 第 7 章实践：服务调度、SLO 与资源预算的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / ttft_ms
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：运行 `serving probe` 中短请求、RAG 请求、取消请求和超长请求。读者要先手写预期结果，再运行项目观察输出。动作是：解释 `memory_budget_exceeded`、`cancel reclaim`、`queue_wait_p95_ms` 和 `rejection_rate`。如果输出里没有 `rejection_rate` 或对应字段无法解释，就不要继续优化；先回到 `reference`、输入合同和 `trace` 字段。

从这个特例推出的规律是：推理服务不是只看 `tokens/s`；是否接收请求、如何预留 KV、取消后是否回收，决定系统是否可运营这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、`perf`、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、`fallback` 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 SLO 表：每类请求的 `prompt/new token`、预算、是否接受、TTFT、TPOT、拒绝原因和未验证边界。这个作业会被后续章节继续引用，命名、目

录和字段不要随意变化。

第 8 章实践：RAG、Agent 与状态图边界

本章对应原书中的第三册“上层 AI 应用编排：RAG、Agent、工具和状态图”。不要把它当作概念复述，本章要把 retrieval、tool call、prompt assembly、state graph、budget、trace id 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `req_rag`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 8.1 第 8 章实践：RAG、Agent 与状态图边界的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / req_rag
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 RAG 请求拆成 retrieve、rerank、prompt build、prefill、decode 五段。读者要先手写预期结果，再运行项目观察输出。动作是：为每段写输入输出和失败后能否重试。如果输出里没有 `prompt_tokens` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：上层编排不能把外部工具结果和模型状态混成字符串；每个边界都要有身份、预算和可重放记录这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 RAG 状态图：节点、边、超时、重试、缓存键、prompt 版本和审计字段。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

模型导入、AI 编译器与内存计划

第 9 章实践：模型导入、manifest 与 Shape Verifier

本章对应原书中的第三册“模型导入、manifest 与 shape verifier”。不要把它当作概念复述，本章要把 `safetensors header`、`metadata`、`tensor name`、`dtype`、`shape`、`data_offsets`、`payload boundary` 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `load_safetensors_manifest`，核心命令或测试入口是 `lcqi_tests`。

Listing 9.1 第 9 章实践：模型导入、manifest 与 Shape Verifier 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / load_safetensors_manifest
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：加载测试临时 `safetensors fixture`。读者要先手写预期结果，再运行项目观察输出。动作是：故意构造 `offset` 越界和 `dtype/shape` 字节数不匹配。如果输出里没有 `data_offsets` 或对应字段无法解释，就不要继续优化；先回到 `reference`、输入合同和 `trace` 字段。

从这个特例推出的规律是：模型导入的第一职责是早拒绝：名字、`shape`、`dtype`、`offset` 和 `payload` 都必须在 `kernel` 运行前被验证这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 `AVX2`、`perf`、`GPU`、`NUMA` 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、`fallback` 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 `manifest gate` 报告：合法 `fixture` 和两个坏 `fixture` 的

字段、错误原因和拒绝位置。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 10 章实践：真实模型加载闭环与首 Token Trace

本章对应原书中的第三册“真实模型加载闭环：Tokenizer、权重格式与首 token trace”。不要把它当作概念复述，本章要把 tokenizer、weight mapping、chat template、first token trace、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

本章新增一个明确边界：tiny fixture 负责手算和逐层 oracle；LCQI GPT-2 reference path 负责证明自研代码已经能加载标准 GPT-2 safetensors、执行 byte-level BPE、跑 GPT-2 forward 并 greedy 生成；SmolLM2 small smoke 负责证明外部 llama.cpp 路径能加载 GGUF small 模型。三者不能互相冒充。tiny MLP 算对，不等于 GPT-2 完成；GPT-2 reference 能出一个 token，不等于高性能 KV cache runtime 完成；SmolLM2 能输出文本，也不等于 LCQI 自研 GGUF 后端完成。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里有三组核心对象。第一组是 [tiny_tokenizer.txt](#)、reference decoder 和 [lcqi_trace](#)，用于手算 token 合同和逐 checkpoint 追踪。第二组是 [gpt2_reference.hpp/.cpp](#)、[lcqi_gpt2](#) 和 [tools/run_gpt2_smoke.py](#)，用于自研 GPT-2 safetensors 路径。第三组是 [tools/run_smolllm2_small_smoke.py](#)，用于下载并校验 SmolLM2-135M-Instruct 的 GGUF 量化文件，再通过 llama.cpp 在 CPU 上实际生成 assistant 文本。

Listing 10.1 第 10 章实践：真实模型加载闭环与首 Token Trace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / tiny_tokenizer.txt
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
make cpu3-gpt2-smoke
make cpu3-smolllm2-smoke
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：从 tokenizer fixture 开始，不跳到真实 7B。读者要先手写预期结果，再运行项目观察输出。动作是：解释 BOS/EOS、UNK、template hash 和 decoder 输入 tokens 的边界。如果输出里没有 `tokens=[0,1,2,3,4]` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：真实模型加载不是下载权重；必须先让 tokenizer 合同、权重 role、shape verifier 和首 token trace 形成闭环。这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、自研 GPT-2 reference smoke：先把标准 safetensors 跑通

GPT-2 不是 LLaMA。它使用 LayerNorm, 不是 RMSNorm; 使用 learned absolute position embedding, 不是 RoPE; attention 里的 Q/K/V 在 HuggingFace 权重中是 packed Conv1D 权重; MLP 使用 GELU, 不是 SwiGLU; `lm_head` 通常和 token embedding 共享权重。把 GPT-2 硬塞进原来的 LLaMA-like tiny decoder, 会让概念混乱, 所以 LCQI 新增了独立的 `gpt2_reference` 模块。

仓库内最小验收分两层。第一层不下载模型, 运行 tiny GPT-2 fixture:

Listing 10.2 仓库内 tiny GPT-2 数学闭环

```
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
```

它固定 prompt ids 为 123, 检查 logits、predicted token 和 greedy generated ids。对应测试还会临时写出一个 HuggingFace 风格 `model.safetensors`, 再用 loader 读回来, 覆盖 tensor name mapping、shape verifier、Conv1D 转置和 tied lm head。坏 shape 必须在 loader 阶段被拒绝。

第二层下载真实 GPT-2 资产:

Listing 10.3 LCQI 自研 GPT-2 smoke

```
make cpu3-gpt2-smoke
```

脚本下载 `openai-community/gpt2` 的 `config.json`、`vocab.json`、`merges.txt` 和 `model.safetensors` 到用户缓存目录, 不提交模型文件。最近一次报告写到:

Listing 10.4 GPT-2 smoke 报告关键字段

```
books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
prompt_ids 15496 11 616 1438 318
generated_ids 15496 11 616 1438 318 1757
generated_text Hello, my name is John
validation=PASS
```

这说明自研 LCQI C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer、权重导入、forward 和 greedy generation。边界也要写清: 当前实现为了可读和可验证, 会重跑完整前缀, 没有 KV cache、没有 packed weight、没有多线程, 也没有把 logits 与 PyTorch/Transformers 的逐数值 golden 报告纳入仓库。因此它是正确性闭环, 不是性能结论。

四、真实 small 模型 smoke: 不用 tiny, 先让开源模型实际跑起来

本章的真实模型 smoke 固定为 `SmolLM2-135M-Instruct`, 规模是 135M 参数, 不是 tiny 随机夹具。脚本使用 GGUF 文件 `SmolLM2-135M-Instruct-Q4_K_M.gguf`, 来源仓库是 `bartowski/SmolLM2-135M-Instruct-GGUF`, 原模型组织是 `HuggingFaceTB`, license 字段为 `apache-2.0`。脚本不会把模型文件提交到仓库, 而是缓存到用户目录; 验收时检查文件大小 105454432 字节和 SHA256:

Listing 10.5 SmolLM2 small GGUF 文件身份

```
model_source_id=HuggingFaceTB/SmolLM2-135M-Instruct
model_gguf_repo_id=bartowski/SmolLM2-135M-Instruct-GGUF
model_file=SmolLM2-135M-Instruct-Q4_K_M.gguf
model_expected_bytes=105454432
model_expected_sha256=2↵
↵ e8040ceae7815abe0dc3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
```


运行命令是：

Listing 10.6 真实 small 模型 CPU 冒烟

```
make cpu3-smolllm2-smoke
```

这条命令会做四步。第一，下载或复用 GGUF 文件，并校验 SHA256，防止“文件名对了但内容不对”。第二，拉取固定 commit 的 `llama.cpp` 并构建 `llama-simple-chat`，避免依赖本机临时安装。第三，用 `-ngl0` 强制 CPU 路径，输入一个短 prompt，确认程序能走过 tokenizer、chat template、prefill、decode 和 sampler。第四，把报告写到：

Listing 10.7 真实模型 smoke 报告路径

```
books/cpu-volume-3/results/lcqi-smolllm2-small-model-smoke.txt
```

报告里必须至少出现这些字段：`model_expected_sha256`、`llama_cpp_commit`、`command`、`exit_code=0`、`validation=PASS` 和 `assistant_response`。如果只有下载日志，没有 assistant response，不能算通过；如果 assistant response 内容答错了，也不能改写成“质量通过”。本章只验证真实模型资产可加载、可 tokenize、可 decode、可生成非空文本。

五、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：运行 GPT-2 smoke。报告不许只贴“生成了 John”，必须解释 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors` 各自对应哪段代码，为什么 GPT-2 需要 byte-level BPE，为什么 HuggingFace Conv1D 权重要转置，为什么 `lm_head` 可以 tied 到 embedding。

任务四：运行真实 small 模型 smoke。报告不许只贴“命令执行成功”，必须解释脚本为什么校验模型 hash、为什么固定 `llama.cpp` commit、为什么使用 `-ngl0`，以及为什么 assistant response 非空只是 smoke，不是质量评测。

任务五：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。GPT-2 反例可以是：tensor name 缺失、shape 和 config 不一致、vocab 中缺少 BPE merge 后 token、`max_new_tokens` 超过 context。真实模型 smoke 的反例可以是：模型 SHA256 不匹配、网络不可用、`cmake` 缺失、assistant response 为空、命令超时。每个反例都要写清期望处理方式。

六、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。新增 GPT-2 smoke 后，最低验收还包括：真实 GPT-2 文件身份有 hash，prompt ids 可解释，generated ids 可解释，文本反解码可解释，并明确声明这不是高性能推理结论。新增 SmolLM2 smoke 后，最低验收还包括：模型身份有 hash，模型规模不是 tiny，CPU 路径实际运行，报告里有非空 assistant response，并明确声明这不是自研

LCQI 完整 GGUF 支持。

递进大作业：提交 GPT-2 reference path 的下一阶段迁移清单：KV cache、prefill/decode 分离、packed weight、线程并行、Transformers golden logits、更多模型方言 name mapping、分片 safetensors 和性能报告。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：Reference Decoder、Tokenizer、Safetensors 与 GPT-2

从 tiny MLP 走向 decoder-only 模型

Reference decoder 和 GPT-2 reference path 是 LCQI 从教学闭环走向真实模型导入的关键层。这里的代码更长，是因为真实模型不只有矩阵乘法：它还需要 tokenizer、位置、归一化、QKV 拆分、attention mask、KV cache、激活函数、lm head、权重命名映射、dtype 转换和坏资产拒绝。

Tiny Reference Decoder

reference_decoder.cpp 实现 embedding、RMSNorm、Q/K/V、RoPE、KV cache、attention、SwiGLU、final norm、logits 和 sampler，并在关键位置生成 trace checkpoint。读者应按 checkpoint 顺序读，而不是从最终 logits 倒推。

Listing 10.8 src/reference_decoder.cpp

```

1 #include <lcqi/reference_decoder.hpp>
2
3 #include <algorithm>
4 #include <array>
5 #include <cmath>
6 #include <limits>
7 #include <stdexcept>
8 #include <string>
9
10 namespace lcqi {
11 namespace {
12
13 constexpr float LCQI_SOFTMAX_NEGATIVE_INFINITY = -std::numeric_limits<float>::infinity();
14 constexpr std::int32_t LCQI_ROPE_PAIR_WIDTH = 2;
15 constexpr const char* LCQI_TRACE_DTYPE_F32 = "f32";
16 constexpr const char* LCQI_TRACE_LAYOUT_CONTIGUOUS = "contiguous";
17 constexpr const char* LCQI_TRACE_LAYOUT_ROW_MAJOR = "row_major";
18 constexpr const char* LCQI_TRACE_LAYOUT_KV_CONTIGUOUS = "kv_contiguous";
19
20 void require_positive(std::int32_t value, const char* name) {
21     if (value <= 0) {
22         throw std::runtime_error(std::string(name) + " must be positive");
23     }
24 }
25
26 void validate_config(const DecoderConfig& config) {
27     require_positive(config.hidden_size, "hidden_size");
28     require_positive(config.query_heads, "query_heads");
29     require_positive(config.kv_heads, "kv_heads");
30     require_positive(config.head_dim, "head_dim");
31     require_positive(config.intermediate_size, "intermediate_size");

```

```

32     require_positive(config.vocab_size, "vocab_size");
33     require_positive(config.max_context, "max_context");
34     if (config.query_heads % config.kv_heads != 0) {
35         throw std::runtime_error("query_heads must be divisible by kv_heads");
36     }
37     if (config.hidden_size != config.query_heads * config.head_dim) {
38         throw std::runtime_error("hidden_size must equal query_heads * head_dim↵
↵");
39     }
40     if (config.head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
41         throw std::runtime_error("head_dim must be even for RoPE");
42     }
43     if (config.rope_base <= 1.0F) {
44         throw std::runtime_error("rope_base must be greater than 1");
45     }
46 }
47
48 std::size_t checked_size(std::int32_t value, const char* name) {
49     if (value < 0) {
50         throw std::runtime_error(std::string(name) + " cannot be negative");
51     }
52     return static_cast<std::size_t>(value);
53 }
54
55 std::int32_t kv_width(const DecoderConfig& config) {
56     return config.kv_heads * config.head_dim;
57 }
58
59 void validate_linear(const LinearWeightsF32& layer, const char* name) {
60     require_positive(layer.input_size, "linear input_size");
61     require_positive(layer.output_size, "linear output_size");
62     const std::size_t expected_weights =
63         checked_size(layer.input_size, name) * checked_size(layer.output_size, ↵
↵ name);
64     if (layer.weights.size() != expected_weights) {
65         throw std::runtime_error(std::string(name) + " weight size mismatch");
66     }
67     if (!layer.bias.empty() &&
68         layer.bias.size() != checked_size(layer.output_size, name)) {
69         throw std::runtime_error(std::string(name) + " bias size mismatch");
70     }
71 }
72
73 void validate_span_size(std::size_t actual, std::int32_t expected, const char* ↵
↵ name) {
74     if (actual != checked_size(expected, name)) {
75         throw std::runtime_error(std::string(name) + " size mismatch");
76     }
77 }
78
79 std::int32_t argmax(std::span<const float> values) {
80     if (values.empty()) {

```

```

81     throw std::runtime_error("cannot take argmax of empty values");
82 }
83 std::int32_t best_index = 0;
84 float best_value = values[0];
85 for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()
↵  )); ++index) {
86     const float candidate = values[static_cast<std::size_t>(index)];
87     if (candidate > best_value) {
88         best_value = candidate;
89         best_index = index;
90     }
91 }
92 return best_index;
93 }
94
95 float silu(float value) {
96     return value / (1.0F + std::exp(-value));
97 }
98
99 std::span<const float> row_span(std::span<const float> values,
100                                std::int32_t row,
101                                std::int32_t row_width) {
102     const std::size_t begin = checked_size(row, "row") * checked_size(row_width,
↵  , "row_width");
103     return values.subspan(begin, checked_size(row_width, "row_width"));
104 }
105
106 void add_inplace(std::span<float> target, std::span<const float> source) {
107     if (target.size() != source.size()) {
108         throw std::runtime_error("residual add size mismatch");
109     }
110     for (std::size_t index = 0; index < target.size(); ++index) {
111         target[index] += source[index];
112     }
113 }
114
115 void swiglu_mlp(const DecoderLayerWeightsF32& layer,
116                std::span<const float> input,
117                std::span<float> output) {
118     std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F
↵  );
119     std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
120     std::vector<float> hidden(gate.size(), 0.0F);
121     linear_f32(layer.w_gate, input, gate);
122     linear_f32(layer.w_up, input, up);
123     if (gate.size() != up.size()) {
124         throw std::runtime_error("SwiGLU gate/up size mismatch");
125     }
126     for (std::size_t index = 0; index < hidden.size(); ++index) {
127         hidden[index] = silu(gate[index]) * up[index];
128     }
129     linear_f32(layer.w_down, hidden, output);

```

```

130 }
131
132 float checksum(std::span<const float> values) {
133     float sum = 0.0F;
134     for (const float value : values) {
135         sum += value;
136     }
137     return sum;
138 }
139
140 float max_abs(std::span<const float> values) {
141     float result = 0.0F;
142     for (const float value : values) {
143         result = std::max(result, std::fabs(value));
144     }
145     return result;
146 }
147
148 std::vector<std::int32_t> contiguous_stride(std::span<const std::int32_t> shape ←
    ↪ ) {
149     std::vector<std::int32_t> stride(shape.size(), 1);
150     std::int32_t running = 1;
151     for (std::int32_t index = static_cast<std::int32_t>(shape.size()) - 1; ↪
    ↪ index >= 0; --index) {
152         stride[checked_size(index, "stride index")] = running;
153         running *= shape[checked_size(index, "shape index")];
154     }
155     return stride;
156 }
157
158 void add_checkpoint(std::vector<ReferenceTensorCheckpoint>* checkpoints,
159                    const char* name,
160                    std::int32_t token_position,
161                    std::int32_t layer_id,
162                    std::span<const std::int32_t> shape,
163                    const char* layout,
164                    std::span<const float> values) {
165     if (checkpoints == nullptr) {
166         return;
167     }
168     ReferenceTensorCheckpoint checkpoint;
169     checkpoint.name = name;
170     checkpoint.token_position = token_position;
171     checkpoint.layer_id = layer_id;
172     checkpoint.shape.assign(shape.begin(), shape.end());
173     checkpoint.stride = contiguous_stride(shape);
174     checkpoint.dtype = LCQI_TRACE_DTYPE_F32;
175     checkpoint.layout = layout;
176     checkpoint.checksum = checksum(values);
177     checkpoint.max_abs = max_abs(values);
178     checkpoint.values.assign(values.begin(), values.end());
179     checkpoints->push_back(std::move(checkpoint));

```

```

180 }
181
182 void swiglu_mlp_with_trace(const DecoderLayerWeightsF32& layer,
183                          std::span<const float> input,
184                          std::span<float> output,
185                          std::int32_t token_position,
186                          std::int32_t layer_id,
187                          std::vector<ReferenceTensorCheckpoint>* checkpoints)↵
    ↵ {
188     std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0↵
    ↵ F);
189     std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
190     std::vector<float> hidden(gate.size(), 0.0F);
191     linear_f32(layer.w_gate, input, gate);
192     linear_f32(layer.w_up, input, up);
193     if (gate.size() != up.size()) {
194         throw std::runtime_error("SwiGLU gate/up size mismatch");
195     }
196     const std::array<std::int32_t, 1> intermediate_shape{layer.w_gate.↵
    ↵ output_size};
197     add_checkpoint(checkpoints,
198                   "mlp_gate",
199                   token_position,
200                   layer_id,
201                   intermediate_shape,
202                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
203                   gate);
204     add_checkpoint(checkpoints,
205                   "mlp_up",
206                   token_position,
207                   layer_id,
208                   intermediate_shape,
209                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
210                   up);
211     for (std::size_t index = 0; index < hidden.size(); ++index) {
212         hidden[index] = silu(gate[index]) * up[index];
213     }
214     add_checkpoint(checkpoints,
215                   "mlp_hidden",
216                   token_position,
217                   layer_id,
218                   intermediate_shape,
219                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
220                   hidden);
221     linear_f32(layer.w_down, hidden, output);
222 }
223
224 void forward_layer(const DecoderConfig& config,
225                  const DecoderLayerWeightsF32& layer,
226                  std::int32_t layer_id,
227                  std::int32_t model_position,
228                  ReferenceKVCache& cache,

```

```

229         std::span<float> hidden,
230         std::vector<ReferenceTensorCheckpoint*> checkpoints = ↵
↵ nullptr) {
231     validate_span_size(hidden.size(), config.hidden_size, "hidden");
232
233     std::vector<float> normed(check_size(config.hidden_size, "hidden_size"), ↵
↵ 0.0F);
234     std::vector<float> query(check_size(config.hidden_size, "hidden_size"), ↵
↵ 0.0F);
235     std::vector<float> key(check_size(kv_width(config), "kv_width"), 0.0F);
236     std::vector<float> value(check_size(kv_width(config), "kv_width"), 0.0F);
237     std::vector<float> attention(check_size(config.hidden_size, "hidden_size" ↵
↵ ), 0.0F);
238     std::vector<float> attention_projected(check_size(config.hidden_size, " ↵
↵ hidden_size"), 0.0F);
239     std::vector<float> mlp(check_size(config.hidden_size, "hidden_size"), 0.0 ↵
↵ F);
240
241     const std::array<std::int32_t, 1> hidden_shape{config.hidden_size};
242     const std::array<std::int32_t, 2> query_shape{config.query_heads, config. ↵
↵ head_dim};
243     const std::array<std::int32_t, 2> kv_shape{config.kv_heads, config.head_dim ↵
↵ };
244     const std::array<std::int32_t, 4> kv_slot_shape{
245         1, 1, config.kv_heads, config.head_dim};
246
247     rms_norm(hidden, layer.rms_attention_weight, config.rms_epsilon, normed);
248     add_checkpoint(checkpoints,
249         "attn_norm",
250         model_position,
251         layer_id,
252         hidden_shape,
253         LCQI_TRACE_LAYOUT_CONTIGUOUS,
254         normed);
255     linear_f32(layer.wq, normed, query);
256     linear_f32(layer.wk, normed, key);
257     linear_f32(layer.wv, normed, value);
258     add_checkpoint(checkpoints,
259         "q_before_rope",
260         model_position,
261         layer_id,
262         query_shape,
263         LCQI_TRACE_LAYOUT_CONTIGUOUS,
264         query);
265     add_checkpoint(checkpoints,
266         "k_before_rope",
267         model_position,
268         layer_id,
269         kv_shape,
270         LCQI_TRACE_LAYOUT_CONTIGUOUS,
271         key);
272     add_checkpoint(checkpoints,

```



```

273         "v",
274         model_position,
275         layer_id,
276         kv_shape,
277         LCQI_TRACE_LAYOUT_CONTIGUOUS,
278         value);
279     apply_rope(query, config.query_heads, config.head_dim, model_position, ←
    ↪ config.rope_base);
280     apply_rope(key, config.kv_heads, config.head_dim, model_position, config.←
    ↪ rope_base);
281     add_checkpoint(checkpoints,
282         "q_after_rope",
283         model_position,
284         layer_id,
285         query_shape,
286         LCQI_TRACE_LAYOUT_CONTIGUOUS,
287         query);
288     add_checkpoint(checkpoints,
289         "k_after_rope",
290         model_position,
291         layer_id,
292         kv_shape,
293         LCQI_TRACE_LAYOUT_CONTIGUOUS,
294         key);
295
296     cache.append(layer_id, model_position, key, value);
297     add_checkpoint(checkpoints,
298         "kv_cache_key_slot",
299         model_position,
300         layer_id,
301         kv_slot_shape,
302         LCQI_TRACE_LAYOUT_KV_CONTIGUOUS,
303         key);
304     attention_decode(config, cache, layer_id, model_position, query, attention)←
    ↪ ;
305     add_checkpoint(checkpoints,
306         "attention_out",
307         model_position,
308         layer_id,
309         hidden_shape,
310         LCQI_TRACE_LAYOUT_CONTIGUOUS,
311         attention);
312     linear_f32(layer.wo, attention, attention_projected);
313     add_inplace(hidden, attention_projected);
314     add_checkpoint(checkpoints,
315         "post_attention_residual",
316         model_position,
317         layer_id,
318         hidden_shape,
319         LCQI_TRACE_LAYOUT_CONTIGUOUS,
320         hidden);
321

```

```

322     rms_norm(hidden, layer.rms_mlp_weight, config.rms_epsilon, normed);
323     add_checkpoint(checkpoints,
324                   "mlp_norm",
325                   model_position,
326                   layer_id,
327                   hidden_shape,
328                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
329                   normed);
330     if (checkpoints == nullptr) {
331         swiglu_mlp(layer, normed, mlp);
332     } else {
333         swiglu_mlp_with_trace(layer, normed, mlp, model_position, layer_id, ↵
↵ checkpoints);
334     }
335     add_inplace(hidden, mlp);
336     add_checkpoint(checkpoints,
337                   "layer_out",
338                   model_position,
339                   layer_id,
340                   hidden_shape,
341                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
342                   hidden);
343 }
344
345 LinearWeightsF32 make_linear(std::int32_t input_size,
346                             std::int32_t output_size,
347                             std::vector<float> weights,
348                             std::vector<float> bias = {}) {
349     LinearWeightsF32 layer;
350     layer.input_size = input_size;
351     layer.output_size = output_size;
352     layer.weights = std::move(weights);
353     layer.bias = std::move(bias);
354     validate_linear(layer, "tiny linear");
355     return layer;
356 }
357
358 std::vector<float> zeros(std::int32_t count) {
359     return std::vector<float>(checked_size(count, "zero count"), 0.0F);
360 }
361
362 } // namespace
363
364 ReferenceDecodeTraceResult run_reference_decode_with_trace(
365     const ReferenceDecoderModel& model,
366     std::span<const std::int32_t> token_ids) {
367     validate_config(model.config);
368     if (token_ids.empty()) {
369         throw std::runtime_error("reference decode needs at least one token");
370     }
371     if (token_ids.size() > checked_size(model.config.max_context, "max_context" ↵
↵ )) {

```

```

372     throw std::runtime_error("token_ids exceed max_context");
373 }
374 if (model.token_embedding.size() !=
375     checked_size(model.config.vocab_size, "vocab_size") *
376     checked_size(model.config.hidden_size, "hidden_size")) {
377     throw std::runtime_error("token embedding size mismatch");
378 }
379 if (model.final_norm_weight.size() != checked_size(model.config.hidden_size,
380 ↪ , "hidden_size")) {
381     throw std::runtime_error("final norm weight size mismatch");
382 }
383 validate_linear(model.lm_head, "lm_head");
384
385 ReferenceDecodeTraceResult trace;
386 ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers.
387 ↪ .size()));
388 std::vector<float> hidden(checked_size(model.config.hidden_size, "
389 ↪ hidden_size"), 0.0F);
390 const std::array<std::int32_t, 1> hidden_shape{model.config.hidden_size};
391 for (std::int32_t position = 0; position < static_cast<std::int32_t>(
392 ↪ token_ids.size());
393     ++position) {
394     const std::int32_t token_id = token_ids[checked_size(position, "
395 ↪ position)];
396     if (token_id < 0 || token_id >= model.config.vocab_size) {
397         throw std::runtime_error("token id out of vocabulary");
398     }
399     const std::span<const float> embedding =
400     row_span(model.token_embedding, token_id, model.config.hidden_size)
401     ↪ ;
402     std::copy(embedding.begin(), embedding.end(), hidden.begin());
403     add_checkpoint(&trace.checkpoints,
404         "embedding",
405         position,
406         -1,
407         hidden_shape,
408         LCQI_TRACE_LAYOUT_ROW_MAJOR,
409         hidden);
410     for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(
411     ↪ model.layers.size());
412         ++layer_id) {
413         forward_layer(model.config,
414             model.layers[checked_size(layer_id, "layer_id")],
415             layer_id,
416             position,
417             cache,
418             hidden,
419             &trace.checkpoints);
420     }
421 }
422
423 std::vector<float> normed(checked_size(model.config.hidden_size, "

```

```

    ↪ hidden_size"), 0.0F);
417 std::vector<float> logits(check_size(model.config.vocab_size, "vocab_size",
    ↪ "), 0.0F);
418 const std::array<std::int32_t, 1> logits_shape{model.config.vocab_size};
419 const std::int32_t last_position = static_cast<std::int32_t>(token_ids.size()
    ↪ ()) - 1;
420 rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)
    ↪ ;
421 add_checkpoint(&trace.checkpoints,
422               "final_norm",
423               last_position,
424               -1,
425               hidden_shape,
426               LCQI_TRACE_LAYOUT_CONTIGUOUS,
427               normed);
428 linear_f32(model.lm_head, normed, logits);
429 add_checkpoint(&trace.checkpoints,
430               "logits",
431               last_position,
432               -1,
433               logits_shape,
434               LCQI_TRACE_LAYOUT_CONTIGUOUS,
435               logits);
436
437 trace.result.predicted_token = argmax(logits);
438 trace.result.logits = std::move(logits);
439 return trace;
440 }
441
442 ReferenceKVCache::ReferenceKVCache(const DecoderConfig& config, std::int32_t
    ↪ layer_count)
443 : config_(config),
444   layer_count_(layer_count) {
445   validate_config(this->config_);
446   require_positive(this->layer_count_, "layer_count");
447   const std::size_t element_count =
448     checked_size(this->layer_count_, "layer_count") *
449     checked_size(this->config_.max_context, "max_context") *
450     checked_size(this->config_.kv_heads, "kv_heads") *
451     checked_size(this->config_.head_dim, "head_dim");
452   this->keys_.assign(element_count, 0.0F);
453   this->values_.assign(element_count, 0.0F);
454   this->written_.assign(
455     checked_size(this->layer_count_, "layer_count") *
456     checked_size(this->config_.max_context, "max_context"),
457     0);
458 }
459
460 void ReferenceKVCache::append(std::int32_t layer_id,
461                               std::int32_t model_position,
462                               std::span<const float> key,
463                               std::span<const float> value) {

```

```

464 validate_span_size(key.size(), kv_width(this->config_), "KV key");
465 validate_span_size(value.size(), kv_width(this->config_), "KV value");
466 this->validate_address(layer_id, model_position, 0);
467 for (std::int32_t kv_head = 0; kv_head < this->config_.kv_heads; ++kv_head)
    ↪ {
468     const std::size_t target = this->base_offset(layer_id, model_position,
    ↪ kv_head);
469     const std::size_t source =
470         checked_size(kv_head, "kv_head") * checked_size(this->config_.
    ↪ head_dim, "head_dim");
471     std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
472                 this->config_.head_dim,
473                 this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
474     std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
475                 this->config_.head_dim,
476                 this->values_.begin() + static_cast<std::ptrdiff_t>(target)
    ↪ );
477 }
478 const std::size_t written_index =
479     checked_size(layer_id, "layer_id") * checked_size(this->config_.
    ↪ max_context, "max_context") +
480     checked_size(model_position, "model_position");
481 this->written_[written_index] = 1;
482 this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
483 }
484
485 std::span<const float> ReferenceKVCache::key(std::int32_t layer_id,
486                                             std::int32_t model_position,
487                                             std::int32_t kv_head) const {
488     this->validate_address(layer_id, model_position, kv_head);
489     const std::size_t offset = this->base_offset(layer_id, model_position,
    ↪ kv_head);
490     return std::span<const float>(this->keys_.data() + offset,
491                                   checked_size(this->config_.head_dim, "
    ↪ head_dim"));
492 }
493
494 std::span<const float> ReferenceKVCache::value(std::int32_t layer_id,
495                                             std::int32_t model_position,
496                                             std::int32_t kv_head) const {
497     this->validate_address(layer_id, model_position, kv_head);
498     const std::size_t offset = this->base_offset(layer_id, model_position,
    ↪ kv_head);
499     return std::span<const float>(this->values_.data() + offset,
500                                   checked_size(this->config_.head_dim, "
    ↪ head_dim"));
501 }
502
503 std::int32_t ReferenceKVCache::layer_count() const noexcept {
504     return this->layer_count_;
505 }
506

```

```

507 std::int32_t ReferenceKVCache::filled_tokens() const noexcept {
508     return this->filled_tokens_;
509 }
510
511 std::size_t ReferenceKVCache::base_offset(std::int32_t layer_id,
512                                           std::int32_t model_position,
513                                           std::int32_t kv_head) const {
514     return (((checked_size(layer_id, "layer_id") *
515                  checked_size(this->config_.max_context, "max_context")) +
516              checked_size(model_position, "model_position")) *
517             checked_size(this->config_.kv_heads, "kv_heads") +
518             checked_size(kv_head, "kv_head")) *
519             checked_size(this->config_.head_dim, "head_dim"));
520 }
521
522 void ReferenceKVCache::validate_address(std::int32_t layer_id,
523                                         std::int32_t model_position,
524                                         std::int32_t kv_head) const {
525     if (layer_id < 0 || layer_id >= this->layer_count_) {
526         throw std::runtime_error("KV layer_id out of range");
527     }
528     if (model_position < 0 || model_position >= this->config_.max_context) {
529         throw std::runtime_error("KV model_position out of range");
530     }
531     if (kv_head < 0 || kv_head >= this->config_.kv_heads) {
532         throw std::runtime_error("KV head out of range");
533     }
534 }
535
536 void rms_norm(std::span<const float> input,
537              std::span<const float> weight,
538              float epsilon,
539              std::span<float> output) {
540     if (input.size() != weight.size() || input.size() != output.size()) {
541         throw std::runtime_error("RMSNorm size mismatch");
542     }
543     if (input.empty()) {
544         throw std::runtime_error("RMSNorm input is empty");
545     }
546     float sum_square = 0.0F;
547     for (const float value : input) {
548         sum_square += value * value;
549     }
550     const float mean_square = sum_square / static_cast<float>(input.size());
551     const float scale = 1.0F / std::sqrt(mean_square + epsilon);
552     for (std::size_t index = 0; index < input.size(); ++index) {
553         output[index] = input[index] * scale * weight[index];
554     }
555 }
556
557 void linear_f32(const LinearWeightsF32& layer,
558                std::span<const float> input,

```

```

559         std::span<float> output) {
560     validate_linear(layer, "linear_f32");
561     validate_span_size(input.size(), layer.input_size, "linear input");
562     validate_span_size(output.size(), layer.output_size, "linear output");
563     for (std::int32_t out = 0; out < layer.output_size; ++out) {
564         const std::size_t row_base =
565             checked_size(out, "out") * checked_size(layer.input_size, "↵
↵ input_size");
566         float accumulator = layer.bias.empty() ? 0.0F : layer.bias[checked_size↵
↵ (out, "out")];
567         for (std::int32_t in = 0; in < layer.input_size; ++in) {
568             accumulator += layer.weights[row_base + checked_size(in, "in")] *
569                 input[checked_size(in, "in")];
570         }
571         output[checked_size(out, "out")] = accumulator;
572     }
573 }
574
575 void apply_rope(std::span<float> heads,
576                std::int32_t head_count,
577                std::int32_t head_dim,
578                std::int32_t model_position,
579                float rope_base) {
580     require_positive(head_count, "head_count");
581     require_positive(head_dim, "head_dim");
582     validate_span_size(heads.size(), head_count * head_dim, "RoPE heads");
583     if (head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
584         throw std::runtime_error("RoPE head_dim must be even");
585     }
586     if (model_position < 0) {
587         throw std::runtime_error("RoPE model_position cannot be negative");
588     }
589     if (rope_base <= 1.0F) {
590         throw std::runtime_error("RoPE base must be greater than 1");
591     }
592     for (std::int32_t head = 0; head < head_count; ++head) {
593         for (std::int32_t offset = 0; offset < head_dim; offset += ↵
↵ LCQI_ROPE_PAIR_WIDTH) {
594             const float exponent = static_cast<float>(offset) / static_cast<↵
↵ float>(head_dim);
595             const float theta = 1.0F / std::pow(rope_base, exponent);
596             const float angle = static_cast<float>(model_position) * theta;
597             const float cosine = std::cos(angle);
598             const float sine = std::sin(angle);
599             const std::size_t first =
600                 checked_size(head, "head") * checked_size(head_dim, "head_dim")↵
↵ +
601                 checked_size(offset, "offset");
602             const float x0 = heads[first];
603             const float x1 = heads[first + 1];
604             heads[first] = x0 * cosine - x1 * sine;
605             heads[first + 1] = x0 * sine + x1 * cosine;

```

```

606     }
607 }
608 }
609
610 void attention_decode(const DecoderConfig& config,
611                     const ReferenceKVCache& cache,
612                     std::int32_t layer_id,
613                     std::int32_t model_position,
614                     std::span<const float> query,
615                     std::span<float> output) {
616     validate_config(config);
617     validate_span_size(query.size(), config.hidden_size, "attention query");
618     validate_span_size(output.size(), config.hidden_size, "attention output");
619     if (model_position < 0 || model_position >= cache.filled_tokens()) {
620         throw std::runtime_error("attention model_position has not been written
621 ↪ ");
622     }
623     std::fill(output.begin(), output.end(), 0.0F);
624
625     const std::int32_t group_size = config.query_heads / config.kv_heads;
626     const float score_scale = 1.0F / std::sqrt(static_cast<float>(config.
627 ↪ head_dim));
628     std::vector<float> scores(check_size(model_position + 1, "score count"),
629                               LCQI_SOFTMAX_NEGATIVE_INFINITY);
630     std::vector<float> probabilities(scores.size(), 0.0F);
631
632     for (std::int32_t query_head = 0; query_head < config.query_heads; ++
633 ↪ query_head) {
634         const std::int32_t kv_head = query_head / group_size;
635         const std::size_t query_base =
636             checked_size(query_head, "query_head") * checked_size(config.
637 ↪ head_dim, "head_dim");
638         float max_score = LCQI_SOFTMAX_NEGATIVE_INFINITY;
639         for (std::int32_t position = 0; position <= model_position; ++position)
640 ↪ {
641             const std::span<const float> key = cache.key(layer_id, position,
642 ↪ kv_head);
643             float dot = 0.0F;
644             for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
645                 dot += query[query_base + checked_size(dim, "dim")] *
646                     key[checked_size(dim, "dim")];
647             }
648             const float score = dot * score_scale;
649             scores[checked_size(position, "position")] = score;
650             max_score = std::max(max_score, score);
651         }
652
653         float probability_sum = 0.0F;
654         for (std::int32_t position = 0; position <= model_position; ++position)
655 ↪ {
656             const std::size_t index = checked_size(position, "position");
657             const float unnormalized = std::exp(scores[index] - max_score);

```



```

651         probabilities[index] = unnormalized;
652         probability_sum += unnormalized;
653     }
654     if (probability_sum <= 0.0F) {
655         throw std::runtime_error("attention probability sum is invalid");
656     }
657     for (std::int32_t position = 0; position <= model_position; ++position)↵
↵ {
658         const float probability =
659             probabilities[checked_size(position, "position")] / ↵
↵ probability_sum;
660         const std::span<const float> value = cache.value(layer_id, position↵
↵ , kv_head);
661         for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
662             output[query_base + checked_size(dim, "dim")] +=
663                 probability * value[checked_size(dim, "dim")];
664         }
665     }
666 }
667 }
668
669 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
670     std::span<const std::int32_t> ↵
↵ token_ids) {
671     validate_config(model.config);
672     if (token_ids.empty()) {
673         throw std::runtime_error("reference decode needs at least one token");
674     }
675     if (token_ids.size() > checked_size(model.config.max_context, "max_context"↵
↵ )) {
676         throw std::runtime_error("token_ids exceed max_context");
677     }
678     if (model.token_embedding.size() !=
679         checked_size(model.config.vocab_size, "vocab_size") *
680         checked_size(model.config.hidden_size, "hidden_size")) {
681         throw std::runtime_error("token embedding size mismatch");
682     }
683     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size↵
↵ , "hidden_size")) {
684         throw std::runtime_error("final norm weight size mismatch");
685     }
686     validate_linear(model.lm_head, "lm_head");
687
688     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers↵
↵ .size()));
689     std::vector<float> hidden(checked_size(model.config.hidden_size, "↵
↵ hidden_size"), 0.0F);
690     for (std::int32_t position = 0; position < static_cast<std::int32_t>(↵
↵ token_ids.size());
691         ++position) {
692         const std::int32_t token_id = token_ids[checked_size(position, "↵
↵ position")];

```

```

693     if (token_id < 0 || token_id >= model.config.vocab_size) {
694         throw std::runtime_error("token id out of vocabulary");
695     }
696     const std::span<const float> embedding =
697         row_span(model.token_embedding, token_id, model.config.hidden_size)↵
↵ ;
698     std::copy(embedding.begin(), embedding.end(), hidden.begin());
699     for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(↵
↵ model.layers.size());
700         ++layer_id) {
701         forward_layer(model.config,
702             model.layers[checked_size(layer_id, "layer_id")],
703             layer_id,
704             position,
705             cache,
706             hidden);
707     }
708 }
709
710 std::vector<float> normed(checked_size(model.config.hidden_size, "↵
↵ hidden_size"), 0.0F);
711 std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size↵
↵ "), 0.0F);
712 rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)↵
↵ ;
713 linear_f32(model.lm_head, normed, logits);
714
715 ReferenceDecodeResult result;
716 result.predicted_token = argmax(logits);
717 result.logits = std::move(logits);
718 return result;
719 }
720
721 ReferenceDecoderModel make_tiny_reference_decoder_model() {
722     ReferenceDecoderModel model;
723     model.config.hidden_size = 4;
724     model.config.query_heads = 2;
725     model.config.kv_heads = 1;
726     model.config.head_dim = 2;
727     model.config.intermediate_size = 6;
728     model.config.vocab_size = 5;
729     model.config.max_context = 8;
730     model.config.rms_epsilon = 1.0e-5F;
731     model.config.rope_base = 10000.0F;
732
733     model.token_embedding = {
734         0.20F, -0.10F, 0.05F, 0.30F,
735         -0.40F, 0.10F, 0.25F, -0.20F,
736         0.15F, 0.35F, -0.30F, 0.05F,
737         0.50F, -0.25F, 0.10F, -0.15F,
738         -0.05F, 0.20F, 0.40F, -0.35F,
739     };

```

```

740
741 DecoderLayerWeightsF32 layer;
742 layer.rms_attention_weight = {1.0F, 0.9F, 1.1F, 1.0F};
743 layer.wq = make_linear(4,
744                        4,
745                        {
746                            0.20F, -0.10F, 0.05F, 0.30F,
747                            -0.15F, 0.25F, 0.10F, -0.05F,
748                            0.05F, 0.10F, -0.20F, 0.15F,
749                            0.30F, -0.05F, 0.20F, 0.10F,
750                        });
751 layer.wk = make_linear(4,
752                        2,
753                        {
754                            0.10F, 0.20F, -0.10F, 0.05F,
755                            -0.05F, 0.15F, 0.25F, -0.20F,
756                        });
757 layer.wv = make_linear(4,
758                        2,
759                        {
760                            0.25F, -0.05F, 0.10F, 0.15F,
761                            -0.10F, 0.30F, 0.05F, 0.20F,
762                        });
763 layer.wo = make_linear(4,
764                        4,
765                        {
766                            0.20F, 0.10F, -0.05F, 0.15F,
767                            -0.10F, 0.25F, 0.20F, -0.05F,
768                            0.05F, -0.20F, 0.30F, 0.10F,
769                            0.15F, 0.05F, -0.10F, 0.25F,
770                        });
771 layer.rms_mlp_weight = {0.95F, 1.05F, 1.0F, 0.9F};
772 layer.w_gate = make_linear(4,
773                            6,
774                            {
775                                0.20F, -0.10F, 0.05F, 0.10F,
776                                -0.05F, 0.15F, 0.20F, -0.10F,
777                                0.10F, 0.05F, -0.15F, 0.25F,
778                                -0.20F, 0.10F, 0.15F, 0.05F,
779                                0.05F, -0.25F, 0.10F, 0.20F,
780                                0.15F, 0.05F, 0.05F, -0.15F,
781                            });
782 layer.w_up = make_linear(4,
783                            6,
784                            {
785                                0.10F, 0.20F, -0.05F, 0.05F,
786                                -0.15F, 0.05F, 0.25F, 0.10F,
787                                0.20F, -0.10F, 0.05F, 0.15F,
788                                0.05F, 0.10F, -0.20F, 0.25F,
789                                -0.05F, 0.15F, 0.10F, -0.10F,
790                                0.25F, -0.05F, 0.05F, 0.10F,
791                            });

```

```

792     layer.w_down = make_linear(6,
793                               4,
794                               {
795                                   0.10F, -0.05F, 0.15F, 0.20F, -0.10F, 0.05F,
796                                   -0.20F, 0.10F, 0.05F, -0.05F, 0.15F, 0.20F,
797                                   0.05F, 0.15F, -0.10F, 0.10F, 0.20F, -0.05F,
798                                   0.20F, 0.05F, 0.10F, -0.15F, -0.05F, 0.10F,
799                               });
800     model.layers.push_back(std::move(layer));
801     model.final_norm_weight = {1.0F, 0.95F, 1.05F, 1.0F};
802     model.lm_head = make_linear(4,
803                                5,
804                                {
805                                    0.20F, -0.10F, 0.05F, 0.15F,
806                                    -0.05F, 0.25F, 0.10F, -0.20F,
807                                    0.15F, 0.05F, -0.25F, 0.10F,
808                                    -0.10F, 0.20F, 0.15F, 0.05F,
809                                    0.05F, -0.15F, 0.20F, 0.25F,
810                                },
811                                zeros(5));
812     return model;
813 }
814
815 } // namespace lcqi

```

Tokenizer 与 Safetensors 实现

`tokenizer.cpp` 把 prompt 变成 token id, 并固定 unknown token 行为。读者应检查 BOS/EOS 是否稳定插入, UNK 是否按合同处理。

Listing 10.9 src/tokenizer.cpp

```

1  #include <lcqi/tokenizer.hpp>
2
3  #include <cstdint>
4  #include <fstream>
5  #include <stdexcept>
6  #include <string>
7  #include <string_view>
8
9  namespace lcqi {
10 namespace {
11
12 constexpr std::uint64_t LCQI_FNV_OFFSET_BASIS = 1469598103934665603ULL;
13 constexpr std::uint64_t LCQI_FNV_PRIME = 1099511628211ULL;
14 constexpr std::int32_t LCQI_INVALID_TOKEN_ID = -1;
15
16 std::string read_key(std::istream& input) {
17     std::string key;
18     if (!(input >> key)) {
19         throw std::runtime_error("unexpected end of tokenizer file");
20     }

```

```

21     return key;
22 }
23
24 void expect_key(std::istream& input, const std::string& expected) {
25     const std::string key = read_key(input);
26     if (key != expected) {
27         throw std::runtime_error("expected tokenizer key '" + expected + "', ↵
↵ got '" + key + "'");
28     }
29 }
30
31 std::int32_t read_i32(std::istream& input, const std::string& key) {
32     expect_key(input, key);
33     std::int32_t value = 0;
34     if (!(input >> value)) {
35         throw std::runtime_error("invalid tokenizer int32 value for key '" + ↵
↵ key + "'");
36     }
37     return value;
38 }
39
40 std::string read_string(std::istream& input, const std::string& key) {
41     expect_key(input, key);
42     std::string value;
43     if (!(input >> value)) {
44         throw std::runtime_error("invalid tokenizer string value for key '" + ↵
↵ key + "'");
45     }
46     return value;
47 }
48
49 std::int32_t lookup_token_id(const TokenizerModel& tokenizer, std::string_view ↵
↵ text) {
50     for (const TokenEntry& entry : tokenizer.vocab) {
51         if (entry.text == text) {
52             return entry.id;
53         }
54     }
55     return LCQI_INVALID_TOKEN_ID;
56 }
57
58 void validate_tokenizer(const TokenizerModel& tokenizer) {
59     if (tokenizer.vocab.empty()) {
60         throw std::runtime_error("tokenizer vocab is empty");
61     }
62     for (std::size_t i = 0; i < tokenizer.vocab.size(); ++i) {
63         const TokenEntry& left = tokenizer.vocab[i];
64         if (left.id < 0) {
65             throw std::runtime_error("tokenizer token id must be non-negative")↵
↵ ;
66         }
67         for (std::size_t j = i + 1; j < tokenizer.vocab.size(); ++j) {

```

```

68         const TokenEntry& right = tokenizer.vocab[j];
69         if (left.id == right.id) {
70             throw std::runtime_error("duplicate tokenizer token id");
71         }
72         if (left.text == right.text) {
73             throw std::runtime_error("duplicate tokenizer token text");
74         }
75     }
76 }
77 if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID &&
78     lookup_token_id(tokenizer, "<bos>") != tokenizer.bos_id) {
79     throw std::runtime_error("tokenizer BOS id does not match vocab");
80 }
81 if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID &&
82     lookup_token_id(tokenizer, "<eos>") != tokenizer.eos_id) {
83     throw std::runtime_error("tokenizer EOS id does not match vocab");
84 }
85 }
86
87 std::uint64_t hash_byte(std::uint64_t hash, unsigned char value) {
88     hash ^= static_cast<std::uint64_t>(value);
89     hash *= LCQI_FNV_PRIME;
90     return hash;
91 }
92
93 std::uint64_t hash_string(std::uint64_t hash, std::string_view text) {
94     for (const unsigned char value : text) {
95         hash = hash_byte(hash, value);
96     }
97     return hash;
98 }
99
100 std::uint64_t hash_i32(std::uint64_t hash, std::int32_t value) {
101     const std::uint32_t bits = static_cast<std::uint32_t>(value);
102     for (std::int32_t shift = 0; shift < 32; shift += 8) {
103         hash = hash_byte(hash, static_cast<unsigned char>((bits >> shift) & 0x
104             ↪ xFFU));
105     }
106     return hash;
107 }
108 } // namespace
109
110 TokenizerModel load_tokenizer(const std::filesystem::path& path) {
111     std::ifstream input(path);
112     if (!input) {
113         throw std::runtime_error("cannot open tokenizer file: " + path.string()
114             ↪ );
115     }
116     expect_key(input, "LCQI_TOKENIZER_V1");
117

```

```

118     TokenizerModel tokenizer;
119     tokenizer.bos_id = read_i32(input, "bos_id");
120     tokenizer.eos_id = read_i32(input, "eos_id");
121     tokenizer.unk_id = read_i32(input, "unk_id");
122     tokenizer.chat_template_hash = read_string(input, "chat_template_hash");
123     const std::int32_t vocab_size = read_i32(input, "vocab_size");
124     if (vocab_size <= 0) {
125         throw std::runtime_error("tokenizer vocab_size must be positive");
126     }
127
128     tokenizer.vocab.reserve(static_cast<std::size_t>(vocab_size));
129     expect_key(input, "vocab");
130     for (std::int32_t i = 0; i < vocab_size; ++i) {
131         TokenEntry entry;
132         if (!(input >> entry.id >> entry.text)) {
133             throw std::runtime_error("invalid tokenizer vocab entry");
134         }
135         tokenizer.vocab.push_back(entry);
136     }
137
138     validate_tokenizer(tokenizer);
139     return tokenizer;
140 }
141
142 std::vector<std::int32_t> encode_prompt(const TokenizerModel& tokenizer,
143                                         std::string_view prompt) {
144     std::vector<std::int32_t> ids;
145     if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID) {
146         ids.push_back(tokenizer.bos_id);
147     }
148
149     std::size_t begin = 0;
150     while (begin < prompt.size()) {
151         while (begin < prompt.size() && prompt[begin] == ' ') {
152             ++begin;
153         }
154         if (begin >= prompt.size()) {
155             break;
156         }
157         std::size_t end = begin;
158         while (end < prompt.size() && prompt[end] != ' ') {
159             ++end;
160         }
161         const std::string_view token = prompt.substr(begin, end - begin);
162         const std::int32_t token_id = lookup_token_id(tokenizer, token);
163         if (token_id == LCQI_INVALID_TOKEN_ID) {
164             if (tokenizer.unk_id == LCQI_INVALID_TOKEN_ID) {
165                 throw std::runtime_error("tokenizer encountered unknown token")↵
↵ ;
166             }
167             ids.push_back(tokenizer.unk_id);
168         } else {

```

```

169         ids.push_back(token_id);
170     }
171     begin = end;
172 }
173
174 if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID) {
175     ids.push_back(tokenizer.eos_id);
176 }
177 return ids;
178 }
179
180 std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer) {
181     std::uint64_t hash = LCQI_FNV_OFFSET_BASIS;
182     hash = hash_string(hash, tokenizer.chat_template_hash);
183     hash = hash_i32(hash, tokenizer.bos_id);
184     hash = hash_i32(hash, tokenizer.eos_id);
185     hash = hash_i32(hash, tokenizer.unk_id);
186     for (const TokenEntry& entry : tokenizer.vocab) {
187         hash = hash_string(hash, entry.text);
188         hash = hash_i32(hash, entry.id);
189     }
190     return hash;
191 }
192
193 } // namespace lcqi

```

`safetensors.cpp` 解析 header、metadata、dtype、shape 和 data offsets。坏 dtype、坏 shape、坏 offset 和 payload 太短都应该被拒绝。

Listing 10.10 src/safetensors.cpp

```

1 #include <lcqi/safetensors.hpp>
2
3 #include <algorithm>
4 #include <cctype>
5 #include <cstddef>
6 #include <cmath>
7 #include <cstring>
8 #include <fstream>
9 #include <limits>
10 #include <stdexcept>
11 #include <string>
12 #include <string_view>
13 #include <vector>
14
15 namespace lcqi {
16 namespace {
17
18 constexpr std::size_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
19 constexpr std::uint64_t LCQI_SAFETENSORS_MAX_HEADER_BYTES = 16U * 1024U * 1024U↵
    ↵ ;
20 constexpr std::size_t LCQI_SAFETENSORS_OFFSET_COUNT = 2;
21 constexpr std::uint64_t LCQI_BYTE_BITS = 8;

```



```

22 constexpr std::uint64_t LCQI_DECIMAL_BASE = 10;
23 constexpr std::uint64_t LCQI_BYTE_MASK = 0xFFU;
24 constexpr unsigned char LCQI_JSON_CONTROL_CHAR_LIMIT = 0x20U;
25 constexpr std::uint64_t LCQI_DTYPE_BYTES_F64 = 8;
26 constexpr std::uint64_t LCQI_DTYPE_BYTES_F32 = 4;
27 constexpr std::uint64_t LCQI_DTYPE_BYTES_F16 = 2;
28 constexpr std::uint64_t LCQI_DTYPE_BYTES_I64 = 8;
29 constexpr std::uint64_t LCQI_DTYPE_BYTES_I32 = 4;
30 constexpr std::uint64_t LCQI_DTYPE_BYTES_I16 = 2;
31 constexpr std::uint64_t LCQI_DTYPE_BYTES_I8 = 1;
32 constexpr std::uint64_t LCQI_DTYPE_BYTES_U8 = 1;
33 constexpr std::uint32_t LCQI_F32_EXPONENT_MASK = 0x7F800000U;
34 constexpr std::uint32_t LCQI_F16_SIGN_MASK = 0x8000U;
35 constexpr std::uint32_t LCQI_F16_EXPONENT_MASK = 0x7C00U;
36 constexpr std::uint32_t LCQI_F16_MANTISSA_MASK = 0x03FFU;
37 constexpr std::uint32_t LCQI_F16_EXPONENT_BIAS = 15U;
38 constexpr std::uint32_t LCQI_F32_EXPONENT_BIAS = 127U;
39 constexpr std::uint32_t LCQI_F32_MANTISSA_BITS = 23U;
40 constexpr std::uint32_t LCQI_F16_MANTISSA_BITS = 10U;
41 constexpr std::uint32_t LCQI_F16_TO_F32_SIGN_SHIFT = 16U;
42
43 class HeaderCursor {
44 public:
45     explicit HeaderCursor(std::string_view text) : text_(text) {}
46
47     void expect(char expected) {
48         this->skip_ws();
49         if (this->eof() || this->text_[this->position_] != expected) {
50             throw std::runtime_error("invalid safetensors header syntax");
51         }
52         ++this->position_;
53     }
54
55     [[nodiscard]] bool consume(char expected) {
56         this->skip_ws();
57         if (!this->eof() && this->text_[this->position_] == expected) {
58             ++this->position_;
59             return true;
60         }
61         return false;
62     }
63
64     [[nodiscard]] std::string parse_string() {
65         this->skip_ws();
66         this->expect('\"');
67         std::string result;
68         while (!this->eof()) {
69             const char ch = this->text_[this->position_++];
70             if (ch == '\"') {
71                 return result;
72             }
73             if (static_cast<unsigned char>(ch) < LCQI_JSON_CONTROL_CHAR_LIMIT) ←

```

```

↪ {
74         throw std::runtime_error("control character in safetensors ↪
↪ string");
75     }
76     if (ch == '\\') {
77         result.push_back(this->parse_escape());
78     } else {
79         result.push_back(ch);
80     }
81 }
82 throw std::runtime_error("unterminated safetensors string");
83 }
84
85 [[nodiscard]] std::uint64_t parse_u64() {
86     this->skip_ws();
87     if (this->eof() ||
88         !std::isdigit(static_cast<unsigned char>(this->text_[this->↪
↪ position_]))) {
89         throw std::runtime_error("expected unsigned integer in safetensors ↪
↪ header");
90     }
91     std::uint64_t value = 0;
92     while (!this->eof() &&
93         std::isdigit(static_cast<unsigned char>(this->text_[this->↪
↪ position_]))) {
94         const std::uint64_t digit =
95             static_cast<std::uint64_t>(this->text_[this->position_] - '0');
96         if (value >
97             (std::numeric_limits<std::uint64_t>::max() - digit) / ↪
↪ LCQI_DECIMAL_BASE) {
98             throw std::runtime_error("safetensors integer overflow");
99         }
100         value = value * LCQI_DECIMAL_BASE + digit;
101         ++this->position_;
102     }
103     return value;
104 }
105
106 [[nodiscard]] std::vector<std::int64_t> parse_i64_array() {
107     std::vector<std::int64_t> values;
108     this->expect('[');
109     if (this->consume(' ')) {
110         return values;
111     }
112     while (true) {
113         const std::uint64_t raw = this->parse_u64();
114         if (raw > static_cast<std::uint64_t>(std::numeric_limits<std::↪
↪ int64_t>::max())) {
115             throw std::runtime_error("safetensors shape value is too large"↪
↪ );
116         }
117         values.push_back(static_cast<std::int64_t>(raw));

```

```

118         if (this->consume(' ')) {
119             return values;
120         }
121         this->expect(',');
122     }
123 }
124
125 [[nodiscard]] std::vector<std::uint64_t> parse_u64_array() {
126     std::vector<std::uint64_t> values;
127     this->expect('[');
128     if (this->consume(' ')) {
129         return values;
130     }
131     while (true) {
132         values.push_back(this->parse_u64());
133         if (this->consume(' ')) {
134             return values;
135         }
136         this->expect(',');
137     }
138 }
139
140 [[nodiscard]] bool eof_after_ws() {
141     this->skip_ws();
142     return this->eof();
143 }
144
145 private:
146     std::string_view text_;
147     std::size_t position_ = 0;
148
149     [[nodiscard]] bool eof() const {
150         return this->position_ >= this->text_.size();
151     }
152
153     void skip_ws() {
154         while (!this->eof() &&
155             std::isspace(static_cast<unsigned char>(this->text_[this->↵
156             ↵ position_]))) {
157             ++this->position_;
158         }
159     }
160
161     [[nodiscard]] char parse_escape() {
162         if (this->eof()) {
163             throw std::runtime_error("unterminated safetensors escape");
164         }
165         const char escaped = this->text_[this->position_++];
166         switch (escaped) {
167             case '"':
168             case '\\':
169             case '/':

```

```

169         return escaped;
170     case 'b':
171         return '\\b';
172     case 'f':
173         return '\\f';
174     case 'n':
175         return '\\n';
176     case 'r':
177         return '\\r';
178     case 't':
179         return '\\t';
180     default:
181         throw std::runtime_error("unsupported safetensors string escape↵
↵ ");
182     }
183 }
184 };
185
186 std::uint64_t read_le_u64(std::span<const std::uint8_t> bytes) {
187     if (bytes.size() < LCQI_SAFETENSORS_PREFIX_BYTES) {
188         throw std::runtime_error("safetensors file is too small");
189     }
190     std::uint64_t value = 0;
191     for (std::size_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
192         value |= static_cast<std::uint64_t>(bytes[i]) << (LCQI_BYTE_BITS * i);
193     }
194     return value;
195 }
196
197 std::string read_file_bytes(const std::filesystem::path& path) {
198     std::ifstream input(path, std::ios::binary);
199     if (!input) {
200         throw std::runtime_error("cannot open safetensors file: " + path.string↵
↵ ());
201     }
202     input.seekg(0, std::ios::end);
203     const std::streamoff size = input.tellg();
204     if (size < 0) {
205         throw std::runtime_error("cannot determine safetensors file size");
206     }
207     input.seekg(0, std::ios::beg);
208     std::string bytes(static_cast<std::size_t>(size), '\\0');
209     if (!bytes.empty() && !input.read(bytes.data(), static_cast<std::streamsize>↵
↵ >(bytes.size())) {
210         throw std::runtime_error("cannot read safetensors file");
211     }
212     return bytes;
213 }
214
215 std::vector<std::uint8_t> read_file_u8(const std::filesystem::path& path) {
216     std::ifstream input(path, std::ios::binary);
217     if (!input) {

```

```

218     throw std::runtime_error("cannot open safetensors file: " + path.string() ←
    ↪ ());
219 }
220 input.seekg(0, std::ios::end);
221 const std::streamoff size = input.tellg();
222 if (size < 0) {
223     throw std::runtime_error("cannot determine safetensors file size");
224 }
225 input.seekg(0, std::ios::beg);
226 std::vector<std::uint8_t> bytes(static_cast<std::size_t>(size), 0);
227 if (!bytes.empty() &&
228     !input.read(reinterpret_cast<char*>(bytes.data()),
229                static_cast<std::streamsize>(bytes.size()))) {
230     throw std::runtime_error("cannot read safetensors file");
231 }
232 return bytes;
233 }
234
235 void parse_metadata_value(HeaderCursor& cursor,
236                           SafeTensorManifest& manifest,
237                           const std::string& key) {
238     if (key == "__metadata__") {
239         cursor.expect('{');
240         if (cursor.consume('}')) {
241             return;
242         }
243         while (true) {
244             const std::string metadata_key = cursor.parse_string();
245             cursor.expect(':');
246             const std::string metadata_value = cursor.parse_string();
247             manifest.metadata.emplace_back(metadata_key, metadata_value);
248             if (cursor.consume('}')) {
249                 return;
250             }
251             cursor.expect(',');
252         }
253     }
254     throw std::runtime_error("unexpected safetensors metadata value");
255 }
256
257 SafeTensorEntry parse_tensor_entry(HeaderCursor& cursor, const std::string& ←
    ↪ name) {
258     SafeTensorEntry entry;
259     entry.name = name;
260     bool saw_dtype = false;
261     bool saw_shape = false;
262     bool saw_offsets = false;
263
264     cursor.expect('{');
265     if (cursor.consume('}')) {
266         throw std::runtime_error("empty safetensors tensor entry");
267     }

```

```

268     while (true) {
269         const std::string field = cursor.parse_string();
270         cursor.expect(':');
271         if (field == "dtype") {
272             entry.dtype = cursor.parse_string();
273             saw_dtype = true;
274         } else if (field == "shape") {
275             entry.shape = cursor.parse_i64_array();
276             saw_shape = true;
277         } else if (field == "data_offsets") {
278             const std::vector<std::uint64_t> offsets = cursor.parse_u64_array();
279             ↵ ;
280             if (offsets.size() != LCQI_SAFETENSORS_OFFSET_COUNT) {
281                 throw std::runtime_error("safetensors data_offsets must contain ↵
282                 ↵ two values");
283             }
284             entry.data_begin = offsets[0];
285             entry.data_end = offsets[1];
286             saw_offsets = true;
287         } else {
288             throw std::runtime_error("unsupported safetensors tensor field: " + ↵
289             ↵ field);
290         }
291         if (cursor.consume('}')) {
292             break;
293         }
294         cursor.expect(',');
295     }
296     if (!saw_dtype || !saw_shape || !saw_offsets) {
297         throw std::runtime_error("safetensors tensor entry is missing required ↵
298         ↵ fields");
299     }
300     if (entry.data_end < entry.data_begin) {
301         throw std::runtime_error("safetensors tensor offset range is invalid");
302     }
303     return entry;
304 }
305
306 void parse_header(std::string_view header, SafeTensorManifest& manifest) {
307     HeaderCursor cursor(header);
308     cursor.expect('{');
309     if (cursor.consume('}')) {
310         throw std::runtime_error("safetensors header is empty");
311     }
312     while (true) {
313         const std::string key = cursor.parse_string();
314         cursor.expect(':');
315         if (key == "__metadata__") {
316             parse_metadata_value(cursor, manifest, key);
317         } else {

```

```

316         manifest.tensors.push_back(parse_tensor_entry(cursor, key));
317     }
318     if (cursor.consume('}')) {
319         break;
320     }
321     cursor.expect(',');
322 }
323
324 if (!cursor.eof_after_ws()) {
325     throw std::runtime_error("trailing data after safetensors header");
326 }
327 }
328
329 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry);
330
331 void validate_manifest(const SafeTensorManifest& manifest,
332                       std::uint64_t file_size) {
333     const std::uint64_t payload_size = file_size - manifest.data_start_offset;
334     for (std::size_t i = 0; i < manifest.tensors.size(); ++i) {
335         const SafeTensorEntry& left = manifest.tensors[i];
336         if (left.name.empty() || left.dtype.empty()) {
337             throw std::runtime_error("safetensors tensor has empty name or ↵
↵ dtype");
338         }
339         if (left.data_end > payload_size) {
340             throw std::runtime_error("safetensors tensor data_offsets exceed ↵
↵ payload size");
341         }
342         const std::uint64_t expected_bytes = expected_tensor_bytes(left);
343         if (left.byte_size() != expected_bytes) {
344             throw std::runtime_error("safetensors tensor byte size does not ↵
↵ match dtype and shape");
345         }
346         for (const std::int64_t dim : left.shape) {
347             if (dim < 0) {
348                 throw std::runtime_error("safetensors shape dimension is ↵
↵ negative");
349             }
350         }
351         for (std::size_t j = i + 1; j < manifest.tensors.size(); ++j) {
352             const SafeTensorEntry& right = manifest.tensors[j];
353             if (left.name == right.name) {
354                 throw std::runtime_error("duplicate safetensors tensor name");
355             }
356             const bool overlaps =
357                 left.data_begin < right.data_end && right.data_begin < left.↵
↵ data_end;
358             if (overlaps) {
359                 throw std::runtime_error("overlapping safetensors tensor data ↵
↵ range");
360             }
361         }

```

```

362     }
363 }
364
365 std::uint64_t dtype_byte_size(std::string_view dtype) {
366     if (dtype == "F64") {
367         return LCQI_DTYPE_BYTES_F64;
368     }
369     if (dtype == "F32") {
370         return LCQI_DTYPE_BYTES_F32;
371     }
372     if (dtype == "F16" || dtype == "BF16") {
373         return LCQI_DTYPE_BYTES_F16;
374     }
375     if (dtype == "I64" || dtype == "U64") {
376         return LCQI_DTYPE_BYTES_I64;
377     }
378     if (dtype == "I32" || dtype == "U32") {
379         return LCQI_DTYPE_BYTES_I32;
380     }
381     if (dtype == "I16" || dtype == "U16") {
382         return LCQI_DTYPE_BYTES_I16;
383     }
384     if (dtype == "I8") {
385         return LCQI_DTYPE_BYTES_I8;
386     }
387     if (dtype == "U8" || dtype == "BOOL") {
388         return LCQI_DTYPE_BYTES_U8;
389     }
390     throw std::runtime_error("unsupported safetensors dtype: " + std::string(
391         ↪ dtype));
392 }
393
394 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry) {
395     std::uint64_t element_count = 1;
396     for (const std::int64_t dim : entry.shape) {
397         if (dim < 0) {
398             throw std::runtime_error("safetensors shape dimension is negative")
399             ↪ ;
400         }
401         const std::uint64_t extent = static_cast<std::uint64_t>(dim);
402         if (extent != 0 &&
403             ↪ element_count > std::numeric_limits<std::uint64_t>::max() / extent)
404             ↪ {
405             throw std::runtime_error("safetensors shape element count overflow"
406             ↪ );
407         }
408         element_count *= extent;
409     }
410     const std::uint64_t dtype_bytes = dtype_byte_size(entry.dtype);
411     if (dtype_bytes != 0 &&
412         ↪ element_count > std::numeric_limits<std::uint64_t>::max() / dtype_bytes)
413         ↪ {

```



```

409         throw std::runtime_error("safetensors tensor byte size overflow");
410     }
411     return element_count * dtype_bytes;
412 }
413
414 SafeTensorManifest parse_manifest_from_bytes(std::span<const std::uint8_t> ↵
    ↵ bytes) {
415     const std::uint64_t header_size = read_le_u64(bytes);
416     if (header_size == 0 || header_size > LCQI_SAFETENSORS_MAX_HEADER_BYTES) {
417         throw std::runtime_error("safetensors header size is invalid");
418     }
419     const std::uint64_t data_start_offset =
420         static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) + header_size↵
    ↵ ;
421     if (data_start_offset > bytes.size()) {
422         throw std::runtime_error("safetensors header extends beyond file size")↵
    ↵ ;
423     }
424
425     SafeTensorManifest manifest;
426     manifest.header_size = header_size;
427     manifest.data_start_offset = data_start_offset;
428     const char* header_begin =
429         reinterpret_cast<const char*>(bytes.data() + ↵
    ↵ LCQI_SAFETENSORS_PREFIX_BYTES);
430     const std::string_view header(header_begin, static_cast<std::size_t>(↵
    ↵ header_size));
431     parse_header(header, manifest);
432     validate_manifest(manifest, static_cast<std::uint64_t>(bytes.size()));
433     return manifest;
434 }
435
436 std::uint16_t read_le_u16(std::span<const std::uint8_t> bytes, std::size_t ↵
    ↵ offset) {
437     return static_cast<std::uint16_t>(
438         static_cast<std::uint16_t>(bytes[offset]) |
439         (static_cast<std::uint16_t>(bytes[offset + 1]) << LCQI_BYTE_BITS));
440 }
441
442 std::uint32_t read_le_u32(std::span<const std::uint8_t> bytes, std::size_t ↵
    ↵ offset) {
443     std::uint32_t value = 0;
444     for (std::size_t i = 0; i < LCQI_DTYPE_BYTES_F32; ++i) {
445         value |= static_cast<std::uint32_t>(bytes[offset + i]) << (↵
    ↵ LCQI_BYTE_BITS * i);
446     }
447     return value;
448 }
449
450 float bits_to_float(std::uint32_t bits) {
451     float value = 0.0F;
452     std::memcpy(&value, &bits, sizeof(value));

```

```

453     return value;
454 }
455
456 float f16_to_f32(std::uint16_t bits) {
457     const std::uint32_t sign = (static_cast<std::uint32_t>(bits) & ↵
↵ LCQI_F16_SIGN_MASK)
458         << LCQI_F16_TO_F32_SIGN_SHIFT;
459     const std::uint32_t exponent = static_cast<std::uint32_t>(bits) & ↵
↵ LCQI_F16_EXPONENT_MASK;
460     const std::uint32_t mantissa = static_cast<std::uint32_t>(bits) & ↵
↵ LCQI_F16_MANTISSA_MASK;
461
462     if (exponent == 0U) {
463         if (mantissa == 0U) {
464             return bits_to_float(sign);
465         }
466         float value = static_cast<float>(mantissa) /
467             static_cast<float>(1U << LCQI_F16_MANTISSA_BITS);
468         value = std::ldexp(value, 1 - static_cast<int>(LCQI_F16_EXPONENT_BIAS))↵
↵ ;
469         return (sign == 0U) ? value : -value;
470     }
471
472     if (exponent == LCQI_F16_EXPONENT_MASK) {
473         const std::uint32_t f32_mantissa =
474             mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
475         return bits_to_float(sign | LCQI_F32_EXPONENT_MASK | f32_mantissa);
476     }
477
478     const std::uint32_t f32_exponent =
479         ((exponent >> LCQI_F16_MANTISSA_BITS) - LCQI_F16_EXPONENT_BIAS +
480             LCQI_F32_EXPONENT_BIAS)
481         << LCQI_F32_MANTISSA_BITS;
482     const std::uint32_t f32_mantissa =
483         mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
484     return bits_to_float(sign | f32_exponent | f32_mantissa);
485 }
486
487 float bf16_to_f32(std::uint16_t bits) {
488     return bits_to_float(static_cast<std::uint32_t>(bits) << 16U);
489 }
490
491 std::span<const std::uint8_t> tensor_payload_span(const SafeTensorFile& file,
492     const SafeTensorEntry& entry)↵
↵ {
493     const std::uint64_t begin = file.manifest.data_start_offset + entry.↵
↵ data_begin;
494     const std::uint64_t end = file.manifest.data_start_offset + entry.data_end;
495     if (begin > end || end > file.bytes.size()) {
496         throw std::runtime_error("safetensors tensor byte range is invalid");
497     }
498     return std::span<const std::uint8_t>(

```

```

499     file.bytes.data() + static_cast<std::size_t>(begin),
500     static_cast<std::size_t>(end - begin));
501 }
502
503 } // namespace
504
505 std::uint64_t SafeTensorEntry::byte_size() const {
506     return this->data_end - this->data_begin;
507 }
508
509 const SafeTensorEntry* SafeTensorManifest::find_tensor(std::string_view name) ←
    ↪ const {
510     const auto found = std::find_if(
511         this->tensors.begin(),
512         this->tensors.end(),
513         [name](const SafeTensorEntry& entry) {
514             return entry.name == name;
515         });
516     if (found == this->tensors.end()) {
517         return nullptr;
518     }
519     return &(*found);
520 }
521
522 SafeTensorManifest load_safetensors_manifest(const std::filesystem::path& path) ←
    ↪ {
523     const std::string bytes = read_file_bytes(path);
524     return parse_manifest_from_bytes(
525         std::span<const std::uint8_t>(
526             reinterpret_cast<const std::uint8_t*>(bytes.data()),
527             bytes.size()));
528 }
529
530 SafeTensorFile load_safetensors_file(const std::filesystem::path& path) {
531     SafeTensorFile file;
532     file.path = path;
533     file.bytes = read_file_u8(path);
534     file.manifest = parse_manifest_from_bytes(file.bytes);
535     return file;
536 }
537
538 std::span<const std::uint8_t> read_safetensor_tensor_bytes(
539     const SafeTensorFile& file,
540     std::string_view name) {
541     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
542     if (entry == nullptr) {
543         throw std::runtime_error("missing safetensors tensor: " + std::string(←
    ↪ name));
544     }
545     return tensor_payload_span(file, *entry);
546 }
547

```

```

548 std::vector<float> read_safetensor_f32_tensor(const SafeTensorFile& file,
549                                             std::string_view name) {
550     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
551     if (entry == nullptr) {
552         throw std::runtime_error("missing safetensors tensor: " + std::string(↵
↵ name));
553     }
554     const std::span<const std::uint8_t> bytes = tensor_payload_span(file, *↵
↵ entry);
555     std::vector<float> values;
556     if (entry->dtype == "F32") {
557         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F32);
558         for (std::size_t index = 0; index < values.size(); ++index) {
559             values[index] = bits_to_float(
560                 read_le_u32(bytes, index * LCQI_DTYPE_BYTES_F32));
561         }
562         return values;
563     }
564     if (entry->dtype == "F16") {
565         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
566         for (std::size_t index = 0; index < values.size(); ++index) {
567             values[index] = f16_to_f32(
568                 read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
569         }
570         return values;
571     }
572     if (entry->dtype == "BF16") {
573         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
574         for (std::size_t index = 0; index < values.size(); ++index) {
575             values[index] = bf16_to_f32(
576                 read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
577         }
578         return values;
579     }
580     throw std::runtime_error("safetensors tensor is not float-compatible: " + ↵
↵ entry->name);
581 }
582
583 } // namespace lcqi

```

GPT-2 Reference Path

`gpt2_reference.cpp` 包括 JSON 配置读取、byte-level BPE、safetensors F32/F16/BF16 张量读取、HuggingFace 权重名前缀解析、LayerNorm、packed QKV、causal attention、GELU、tied lm head、full-prefix greedy generation 和 cached-KV generation。它仍然是正确性优先的 reference path，但现在已经开始承接前三册的优化方法：先建立 baseline，再改 hot path，再用测试和 A/B benchmark 决定改动是否能进入书。

一次可审查的 cached-KV 优化

优化不能从“感觉慢”开始。LCQI 先用 `makecpu3-gpt2-benchmark-compare` 证明 full-prefix 每生成一个 token 都重算前缀，而 cached-KV 已经把前缀复用下来；随后再看 cached path 内部。旧实现每处理一个 token、每经过一层 Transformer，都会重新分配 `normed、query、key、value、attention、projected、mlp_fc、mlp_out` 和 `attention scores`。同时，生成路径只需要下一个 greedy token，却仍然每步构造完整 `Gpt2ForwardResult.logits`。这些问题不是算法语义错误，但会在真实 GPT-2 的 decode loop 里形成额外分配、重复 shape validation、重复边界转换和不必要的结果写回。

本轮优化对应三册里的三条原则。第一册要求从热循环和二进制可观察行为出发，所以先量 `lm_head`、MLP、QKV、attention projection 和 attention 本体各占多少，再决定改哪里。第二册要求减少重复分配和内存扰动，所以新增 `Gpt2ForwardWorkspace`，把 hidden、QKV、MLP、scores 和 logits 缓冲区的生命周期提升到整段 cached generation。第三册要求按 decoder runtime 的状态组织代码，所以新增 `Gpt2CachedGreedyDecoder`，让 KV cache、workspace、worker pool 和模型验证跟随一次生成会话，而不是散落在每个 token 的临时对象里。

代码上有五处关键边界。第一，旧的 `run_gpt2_forward_cached` 仍然保留完整 logits API，测试和源码阅读可以继续用它和 full-prefix logits 做逐项对照。第二，CLI 的 `cached_kv` 生成路径改用 `Gpt2CachedGreedyDecoder::step`，只返回 greedy token；需要完整 logits 的测试使用 `step_with_logits`。第三，prompt prefill 的前缀 token 使用 `advance_without_prediction`，只更新 KV cache，不做 final norm 和 `lm_head`，因为这些中间预测会被后一个 prompt token 立即覆盖。第四，KV cache 对外仍保留 checked `key/value` API，内部 attention 在一次地址和 workspace 校验后使用私有 unchecked span 扫描热路径，避免把“不检查”的能力暴露给普通调用者。第五，线程池只属于 cached decoder 会话；`--threads0` 自动选择 worker 数，`--threads1` 强制串行，`--threadsN` 固定线程数，避免把全局可变线程服务塞进 reference model。

正确性证据在 `tests/lcqi_tests.cpp`。tiny GPT-2 同一个 prompt 同时覆盖 full-prefix logits、旧 cached logits、优化 decoder logits、强制两 worker 的并行 decoder logits 和 logits-free greedy token。它要求 predicted token、logits size、逐项 logit 误差、KV cache filled token 数和 greedy generated ids 全部一致。这样优化后的代码不是“跑得快但不知道还对不对”，而是先通过 tiny golden，再进入真实 GPT-2 benchmark。

性能证据不只看一次运行。新增 `tools/run_gpt2_optimization_ab.py` 会从指定 baseline commit 建临时 worktree，baseline 和当前工作区都通过 `books/cpu-volume-3-practice` 的 Release CMake 入口构建，再用同一份 GPT-2 F32 safetensors 模型跑多轮 full-prefix 与 cached-KV。后来远端 `caw` 不能直接复用本机 SSH remote 别名时，本轮采用等价的源码包 A/B：本机用 `gitarchive73f40c7` 导出 baseline，用当前工作区导出 current，二者在 `caw` 上分别 Release 构建并运行同一模型。raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-manual-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-optimization-summary.md`。5 轮中位数显示，4 token cached-KV 生成阶段从 395.397ms 到 76.4141ms，16 token 从 989.060ms 到 185.479ms；生成吞吐分别到 52.3464token/s 和 86.2633token/s。

用户看到上一轮提升幅度后，正确反应不是继续猜，而是重新量热点。新增 `--profile-hotspots` 和 `tools/run_gpt2_hotspot_profile.py` 后，可以把 cached decode 的时间拆成 embedding、LayerNorm、QKV projection、KV append、attention、attention projection、MLP fc、GELU、MLP projection 和 lm head。远端 `caw` 使用 Intel Core Ultra 7 265KF、GCC 15.2.1、Release 构建、GPT-2 F32 safetensors，跑 `Hello,mynameis` 这个 prompt，优化前 3 轮中位数保存在

`books/cpu-volume-3/reports/lcqi-gpt2-hotspot-profile-summary.md`：生成 4 个 token 时，`lm_head` 约占 30.2423%，MLP 的 `c_fc` 和 `c_proj` 合计约 46.4228%，QKV projection 约 17.2127%，cached attention 本体只有 0.0558254%。这说明慢点不是 attention，而是大量互相独立的行点积。

这一轮优化据此把 `lm_head` greedy argmax 和 GPT-2 F32 linear 的输出行切给持久 worker pool。每一行仍然是同一个 dot product；不同线程只负责不同输出行。完整 logits 路径把 logits buffer 分片写入，logits-free greedy 路径每个分片先找本地 best token，再按分片顺序合并，用严格大于比较保留较小 token id 的 tie-breaking。这样并行改变调度，不改变数学定义。

还有一个细节说明为什么优化必须带着 profiler 走。第一次 auto 阈值只按“一百万次乘加”判断是否并行，结果 `768\times768` 的 attention output projection 仍走串行。中间 profile 立即显示 attention projection 从约 6% 反弹到约 25%，成为新瓶颈。最终阈值改成：输出行数至少 512，且输入列数至少 512 时也允许并行。最终 profile 保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-hotspot-caw.txt`：4 token 生成中位数 75.9807ms，16 token 184.780ms；`lm_head` 约 20%，MLP 两个投影合计约 45%，QKV 约 19%，attention projection 约 12%，attention 本体仍低于 1%。

用户继续问和 llama.cpp 的差距时，下一步就不能再停留在“行并行”。LCQI 新增 `include/lcqi/f32_kernels.hpp`、`src/f32_kernels.cpp` 和 `src/f32_kernels_avx2.cpp`，把 GPT-2 cached 热路径的单行点积抽成 `dot_f32_unchecked()`。checked `dot_f32()` 负责普通 span 长度检查，unchecked 入口给已经完成 shape validation 的 linear、logits 和 logits-free `lm_head` 使用；x86-64/GNU/Clang 构建会编译 AVX2/FMA 文件，运行时再用 `dot_f32_avx2_available()` 判断是否能执行。这样 AVX intrinsics 不进入 GPT-2 层逻辑，后续替换 AVX-512、int8 或 int4 block kernel 时，边界仍然清楚。

`tests/lcqi_tests.cpp` 也随之增加 F32 dot kernel 对照：同一组固定输入先跑 scalar，再跑 dispatch；如果当前 CPU 支持 AVX2/FMA，再显式跑 AVX2 实现，并要求误差落在 $1.0e-4$ 内。测试还会传入长度不一致的 span，确认 checked API 会拒绝。这样优化路径不是“编译过就算”，而是有 scalar oracle、运行时能力判断和错误输入边界。

这轮 F32 dot 优化以 `cb4d89f` 为 baseline，在 `caw` 上用同一 GPT-2 F32 模型、Release 构建和 `--threads0` 自动 16 workers 跑 5 轮。raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-optimization-summary.md`。4 token 生成中位数从 74.4247ms 到 62.3628ms，约 1.19x；16 token 从 181.257ms 到 157.230ms，约 1.15x。decode 中位数分别从 26.1887ms 到 23.0640ms，从 132.201ms 到 117.734ms。这个提升比线程化小，因为它只缩短每一行内部 dot，而没有减少行数、层数、vocab 扫描或内存流量。

继续顺着热点走，下一步不是在 GPT-2 层里散落更多 intrinsics，而是把 kernel 边界从“单行 dot”扩成“row range”。新增的 `linear_f32_rows_unchecked()` 一次计算一段输出行，`max_dot_f32_rows_unchecked()` 一次扫描一段 vocab 行并返回局部最大值；`Gpt2ParallelWorkerPool::parallel_for_rows()` 改成模板入口，去掉每个热路径任务里的 `std::function` 构造和间接调用。AVX2 文件里还增加 4-row row-range kernel：同一组 input 向量 load 后，同时喂给 4 个输出行累加器。这个写法已经接近真实 matvec kernel 的形状：不是每一行各自重新走完整控制流，而是在一个小 tile 里复用输入。

这轮 row-range 优化的 raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-optimization-summary.md`。相对 `ab72ccb` 这个 F32 dot baseline，7 轮中位数显示 4 token 端到端生成从 60.6291ms 到 61.1045ms，属于噪声内波动；16 token 从 153.354ms 到 152.574ms，约

1.01x。但把 row-range 版本和 4-row AVX2 row-range 版本直接对比，4 token 从 62.0243ms 到 60.3965ms，16 token 从 154.087ms 到 151.222ms。所以这段代码可以保留，但结论要诚实：它证明 tile 化 row-range 有正收益，却不是数量级变化。

最终 hotspot 仍然说明差距在哪里。row-range 后，16 token profile 中 `lm_head` 约 22.3075%，MLP `c_fc` 约 22.4147%，MLP projection 约 22.448%，QKV projection 约 18.6294%，attention projection 约 11.1687%，cached attention 本体只有 0.653274%。也就是说，小 kernel 和调度优化已经把 reference path 又往前推了一点，但主要时间仍在 F32 row-major matvec。真实 llama.cpp/ggml 的大优势来自 GGUF、block quantization、packed layout、cache-friendly matmul/matvec、mmap 和成熟线程调度，不是只把裸 F32 行循环写得更精巧。

这个结论必须保守阅读。真实端到端 GPT-2 benchmark 受调度、频率、温度和共享机器状态影响明显，单次结果可能反向波动。因此报告里必须同时看 median/min/max、生成文本是否一致、baseline 和 current 是否同一构建入口。更大的差距仍然不在这轮 F32 row-range 里：LCQI 还没有 GGUF 量化导入、packed/quantized lm head、量化 block 权重、分页 KV cache、NUMA/亲和和成熟调度。也就是说，本轮把 reference path 从“行点积有 SIMD 内核边界”推进到“输出行段有可替换 tile kernel”，但还没有把它变成 llama.cpp 级推理引擎。

继续看热点时，有一类优化不在 kernel 内部，而在调用链上。cached generation 的 prompt prefill 要把 prompt 的每个 token 写入 KV cache；但如果 prompt 长度是 N ，前 $N-1$ 个 prompt token 的 next-token prediction 根本不会被输出，也不会作为后续输入。只有最后一个 prompt token 的 prediction 会成为第一个新生成 token。旧代码每个 prompt token 都跑 final norm 和 `lm_head`，等于把前缀 token 的预测算出来后马上丢掉。

因此新增 `Gpt2CachedGreedyDecoder::advance_without_prediction()`。它复用同一条 cached layer path，照样执行 embedding、每层 QKV、attention、MLP 和 KV append，只是在层结束后直接返回，不进入 final norm 和 `lm_head`。CLI 和 `gpt2_generate_greedy_cached()` 的 prefill 改成：前缀 token 调 `advance_without_prediction()`，最后一个 prompt token 调 `step()`。这不是近似优化，因为被跳过的预测值在旧流程中没有任何 caller 读取。

远端 `caw` 上 7 轮 A/B 证明这条优化有效，raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-optimization-summary.md`。同一 GPT-2 F32 模型、同一 prompt、同一 `--threads0` 下，4 token 生成中位数从 60.5154ms 到 52.1949ms，约快 13.75%；16 token 从 153.574ms 到 147.424ms，约快 4.00%。收益在短生成里更明显，因为 prompt prefill 占总时间比例更大；decode 段几乎不变，因为真正生成阶段仍然必须预测下一个 token。所有轮次生成文本和 token id 都一致。

这一轮还做过一个没有进入主线的 packed F32 `lm_head` 实验：把 `lm_head` 权重按 8 个输出一组重排，让 AVX2 一次累计 8 个 token score。它能让 `lm_head_ms` 小幅下降，但 `caw` 多轮 A/B 没有稳定端到端收益，甚至会让部分 decode 口径略慢。这个结果本身也要写进工程判断：不是每个看起来更接近 SIMD 的布局都值得进入教材主线；没有稳定证据的优化只能留作实验记录，不能伪装成成功案例。

继续追问“为什么提升还是不够大”时，下一步不是再猜一个更复杂的内核，而是把已经接受的 profile 再拆开看。row-range 之后，GPT-2 cached path 每个 token 会在 12 层里反复调用 QKV projection、attention projection、MLP `c_fc`、MLP projection；再加上需要预测 token 时的 `lm_head`。这些矩阵向量乘每次本身不算巨型 GEMM，却会频繁进入 `parallel_for_rows()`。也就是说，热点不只有“每行点积慢”，还有“每个小并行区唤醒 worker 的成本”。如果 worker pool 每次都拿 mutex、发 condition variable、再等完成信号，那么短生成里会把调度成本支付

几百次。

因此这一轮接受的优化没有改数学公式，而是改 worker pool 的交接方式。旧版本用 `std::mutex`、`std::condition_variable` 和 `finished_` 条件变量发布任务；新版本把任务元数据写入原子字段，再用 `generation` 作为发布点。后台 worker 看到新的 `generation` 后按固定 worker index 取自己的行块，做完后递减 `active_workers`；主线程自己处理最后一个行块，然后用 `bounded spin/yield` 等后台 worker 归零。代码里特意把这个池限制在 `Gpt2CachedGreedyDecoder` 会话内，没有变成全局忙等线程服务；当 `--threads1` 或模型规模不够并行时，路径仍然直接串行执行。

这个设计有三个边界必须看懂。第一，任务上下文仍然是栈上的 `ParallelTaskContext`，所以所有任务元数据必须先写完，再递增 `generation`；worker 对 `generation` 做 `acquire load` 后才允许读上下文。第二，后台 worker 不再争抢全局 `next_worker_index`，而是启动时固定自己的 worker index；这样不会出现旧任务里空闲 worker 和下一轮任务元数据交叉的问题。第三，`spin` 不是无限空转：短时间使用 `x86 pause`，超过阈值后让出调度；这适合当前 `cached decode` 的短小并行区，但不应该被复制到长时间等待的 I/O 或服务端队列里。

`caw` 上 9 轮 A/B 证明这条优化是本轮真正站得住的提升。`raw` 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-spinpool-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-spinpool-optimization-summary.md`。相对旧 `auto 16 worker baseline`，4 token 生成中位数从 `53.7887ms` 到 `37.5403ms`，约快 `30.21%`；16 token 从 `146.930ms` 到 `104.107ms`，约快 `29.15%`；64 token 从 `510.650ms` 到 `377.233ms`，约快 `26.13%`。只和 `auto8 condition-variable` 版本比较，也能看到 4/16/64 token 分别约 `25.67%`、`29.56%`、`20.44%` 的生成阶段下降；固定 `--threads8` 下也有约 `18%` 到 `23%` 的下降。这说明收益不是只来自 worker 数从 16 改成 8，而是高频并行区交接成本确实降下来了。

热点证据也吻合原因。`auto` 口径下，4 token 的 QKV projection 中位数约快 `33.58%`，attention projection 约快 `63.30%`，MLP `c_fc` 约快 `30.36%`，MLP projection 约快 `29.08%`；而 `lm_head` 只约快 `8.57%`。原因是 QKV 和 MLP 这些线性层在每层每 token 都调用小并行区，调度成本占比高；`lm_head` 是一次更大的 vocab 扫描，仍然主要受内存流量和点积内核影响。这个差异比一句“线程池更快”更重要，因为它告诉读者优化到底打中了哪个成本。

本轮还试过一个 8-row AVX2 row-range 候选：把现有一次处理 4 个输出行的 row-range kernel 扩成一次处理 8 行。它没有进入主线，因为 `caw` 数据没有稳定收益：`auto` 口径下 4 token 从 `37.4937ms` 到 `37.7082ms`，16 token 从 `103.714ms` 到 `103.178ms`，64 token 从 `374.439ms` 到 `374.457ms`；固定 `--threads8` 下 64 token 还从 `371.807ms` 退到 `377.053ms`。这说明寄存器压力、load/store 形状和调度噪声抵消了多行复用 input 的设想，不能因为“8 行看起来比 4 行更充分”就合并。

至此可以重新回答和 `llama.cpp` 的差距。LCQI 现在已经有 `cached KV`、`workspace 复用`、`prefill 跳过无用 prediction`、`F32 row-range AVX2`、`行并行和轻量 worker hand-off`；这些能把 `reference path` 推进很多，但它仍然主要在 `F32 row-major matvec` 上花时间。`llama.cpp/ggml` 的优势不只是“也用了线程”，而是 `GGUF 模型格式`、`block quantization`、`量化权重布局`、`成熟 matvec microkernel`、`mmap 加载`、`NUMA/亲和策略`和大量 `shape-specific 调参`一起工作。LCQI 这轮优化的价值是把调度证据链讲清楚，并把失败候选挡在书外；下一阶段如果还要继续接近 `llama.cpp`，应优先做 `GGUF/量化权重路径`和`量化 matvec`，而不是继续在裸 `F32 row-major` 上堆小修小补。

Listing 10.11 `src/f32_kernels.cpp`

```
1 #include <lcqi/f32_kernels.hpp>
```



```

2
3 #include <limits>
4 #include <stdexcept>
5
6 namespace lcqi {
7 namespace {
8
9 constexpr std::size_t LCQI_F32_AVX2_MIN_DOT_SIZE = 32;
10 constexpr std::size_t LCQI_F32_AVX2_MIN_ROW_COUNT = 4;
11 constexpr std::size_t LCQI_F32_AVX2_MIN_BATCH_SIZE = 2;
12
13 } // namespace
14
15 float dot_f32(std::span<const float> lhs, std::span<const float> rhs) {
16     if (lhs.size() != rhs.size()) {
17         throw std::runtime_error("dot_f32 size mismatch");
18     }
19     return dot_f32_unchecked(lhs.data(), rhs.data(), lhs.size());
20 }
21
22 float dot_f32_scalar_unchecked(const float* lhs,
23                               const float* rhs,
24                               std::size_t size) noexcept {
25     float sum = 0.0F;
26     for (std::size_t index = 0; index < size; ++index) {
27         sum += lhs[index] * rhs[index];
28     }
29     return sum;
30 }
31
32 float dot_f32_unchecked(const float* lhs,
33                        const float* rhs,
34                        std::size_t size) noexcept {
35     static const bool AVX2_AVAILABLE = dot_f32_avx2_available();
36     if (AVX2_AVAILABLE && size >= LCQI_F32_AVX2_MIN_DOT_SIZE) {
37         return dot_f32_avx2_unchecked(lhs, rhs, size);
38     }
39     return dot_f32_scalar_unchecked(lhs, rhs, size);
40 }
41
42 void linear_f32_rows_scalar_unchecked(const float* weights,
43                                       const float* input,
44                                       const float* bias,
45                                       std::size_t input_size,
46                                       std::size_t row_begin,
47                                       std::size_t row_end,
48                                       float* output) noexcept {
49     for (std::size_t row = row_begin; row < row_end; ++row) {
50         const float* weight_row = weights + row * input_size;
51         output[row] = dot_f32_scalar_unchecked(weight_row, input, input_size) +
52             (bias == nullptr ? 0.0F : bias[row]);
53     }

```



```

104         std::size_t output_stride,
105         float* output) noexcept {
106     static const bool AVX2_AVAILABLE = dot_f32_avx2_available();
107     if (AVX2_AVAILABLE && input_size >= LCQI_F32_AVX2_MIN_DOT_SIZE &&
108         batch_size >= LCQI_F32_AVX2_MIN_BATCH_SIZE &&
109         row_end - row_begin >= LCQI_F32_AVX2_MIN_ROW_COUNT) {
110         linear_f32_batch_rows_avx2_unchecked(weights,
111             input,
112             bias,
113             input_size,
114             batch_size,
115             row_begin,
116             row_end,
117             output_stride,
118             output);
119     }
120     return;
121 }
122 linear_f32_batch_rows_scalar_unchecked(weights,
123     input,
124     bias,
125     input_size,
126     batch_size,
127     row_begin,
128     row_end,
129     output_stride,
130     output);
131 }
132 F32RowMax max_dot_f32_rows_scalar_unchecked(const float* weights,
133     const float* input,
134     std::size_t input_size,
135     std::size_t row_begin,
136     std::size_t row_end) noexcept {
137     F32RowMax best;
138     best.row = row_begin;
139     best.value = -std::numeric_limits<float>::infinity();
140     for (std::size_t row = row_begin; row < row_end; ++row) {
141         const float* weight_row = weights + row * input_size;
142         const float value = dot_f32_scalar_unchecked(weight_row, input, ↵
↵ input_size);
143         if (row == row_begin || value > best.value) {
144             best.row = row;
145             best.value = value;
146         }
147     }
148     return best;
149 }
150
151 F32RowMax max_dot_f32_rows_unchecked(const float* weights,
152     const float* input,
153     std::size_t input_size,
154     std::size_t row_begin,

```

```

155         std::size_t row_end) noexcept {
156     static const bool AVX2_AVAILABLE = dot_f32_avx2_available();
157     if (AVX2_AVAILABLE && input_size >= LCQI_F32_AVX2_MIN_DOT_SIZE &&
158         row_end - row_begin >= LCQI_F32_AVX2_MIN_ROW_COUNT) {
159         return max_dot_f32_rows_avx2_unchecked(weights,
160             input,
161             input_size,
162             row_begin,
163             row_end);
164     }
165     return max_dot_f32_rows_scalar_unchecked(weights, input, input_size, ↵
        ↵ row_begin, row_end);
166 }
167
168 } // namespace lcqi
169
170 #if !defined(LCQI_ENABLE_AVX2)
171 namespace lcqi {
172
173 bool dot_f32_avx2_available() noexcept {
174     return false;
175 }
176
177 float dot_f32_avx2_unchecked(const float* lhs,
178     const float* rhs,
179     std::size_t size) noexcept {
180     return dot_f32_scalar_unchecked(lhs, rhs, size);
181 }
182
183 void linear_f32_rows_avx2_unchecked(const float* weights,
184     const float* input,
185     const float* bias,
186     std::size_t input_size,
187     std::size_t row_begin,
188     std::size_t row_end,
189     float* output) noexcept {
190     linear_f32_rows_scalar_unchecked(weights,
191         input,
192         bias,
193         input_size,
194         row_begin,
195         row_end,
196         output);
197 }
198
199 F32RowMax max_dot_f32_rows_avx2_unchecked(const float* weights,
200     const float* input,
201     std::size_t input_size,
202     std::size_t row_begin,
203     std::size_t row_end) noexcept {
204     return max_dot_f32_rows_scalar_unchecked(weights, input, input_size, ↵
        ↵ row_begin, row_end);

```

```

205 }
206
207 void linear_f32_batch_rows_avx2_unchecked(const float* weights,
208                                           const float* input,
209                                           const float* bias,
210                                           std::size_t input_size,
211                                           std::size_t batch_size,
212                                           std::size_t row_begin,
213                                           std::size_t row_end,
214                                           std::size_t output_stride,
215                                           float* output) noexcept {
216     linear_f32_batch_rows_scalar_unchecked(weights,
217                                           input,
218                                           bias,
219                                           input_size,
220                                           batch_size,
221                                           row_begin,
222                                           row_end,
223                                           output_stride,
224                                           output);
225 }
226
227 } // namespace lcqi
228 #endif

```

Listing 10.12 src/f32_kernels_avx2.cpp

```

1 #include <lcqi/f32_kernels.hpp>
2
3 #if defined(LCQI_ENABLE_AVX2)
4
5 #include <immintrin.h>
6
7 #include <array>
8 #include <cstdint>
9 #include <limits>
10
11 namespace lcqi {
12 namespace {
13
14 constexpr std::size_t LCQI_F32_AVX2_LANE_COUNT = 8;
15 constexpr std::size_t LCQI_F32_AVX2_UNROLL_COUNT = 4;
16 constexpr std::size_t LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT = 8;
17 constexpr std::size_t LCQI_F32_AVX2_BATCH_UNROLL_COUNT = 4;
18 constexpr std::size_t LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT = 8;
19 constexpr std::size_t LCQI_F32_AVX2_UNROLLED_FLOATS =
20     LCQI_F32_AVX2_LANE_COUNT * LCQI_F32_AVX2_UNROLL_COUNT;
21
22 [[nodiscard]] float horizontal_sum(__m256 values) noexcept {
23     const __m128 low = _mm256_castps256_ps128(values);
24     const __m128 high = _mm256_extractf128_ps(values, 1);
25     const __m128 pair_sum = _mm_add_ps(low, high);

```

```

26     const __m128 first_fold = _mm_hadd_ps(pair_sum, pair_sum);
27     const __m128 second_fold = _mm_hadd_ps(first_fold, first_fold);
28     return _mm_cvtss_f32(second_fold);
29 }
30
31 void compute_four_row_dot(const float* row0,
32                          const float* row1,
33                          const float* row2,
34                          const float* row3,
35                          const float* input,
36                          std::size_t input_size,
37                          float* sums) noexcept {
38     std::size_t index = 0;
39     __m256 accumulator0 = _mm256_setzero_ps();
40     __m256 accumulator1 = _mm256_setzero_ps();
41     __m256 accumulator2 = _mm256_setzero_ps();
42     __m256 accumulator3 = _mm256_setzero_ps();
43
44     for (; index + LCQI_F32_AVX2_LANE_COUNT <= input_size;
45          index += LCQI_F32_AVX2_LANE_COUNT) {
46         const __m256 input_values = _mm256_loadu_ps(input + index);
47         accumulator0 =
48             _mm256_fmadd_ps(_mm256_loadu_ps(row0 + index), input_values, ↵
↵ accumulator0);
49         accumulator1 =
50             _mm256_fmadd_ps(_mm256_loadu_ps(row1 + index), input_values, ↵
↵ accumulator1);
51         accumulator2 =
52             _mm256_fmadd_ps(_mm256_loadu_ps(row2 + index), input_values, ↵
↵ accumulator2);
53         accumulator3 =
54             _mm256_fmadd_ps(_mm256_loadu_ps(row3 + index), input_values, ↵
↵ accumulator3);
55     }
56
57     sums[0] = horizontal_sum(accumulator0);
58     sums[1] = horizontal_sum(accumulator1);
59     sums[2] = horizontal_sum(accumulator2);
60     sums[3] = horizontal_sum(accumulator3);
61     for (; index < input_size; ++index) {
62         const float input_value = input[index];
63         sums[0] += row0[index] * input_value;
64         sums[1] += row1[index] * input_value;
65         sums[2] += row2[index] * input_value;
66         sums[3] += row3[index] * input_value;
67     }
68 }
69
70 void compute_eight_row_dot(const float* const* rows,
71                           const float* input,
72                           std::size_t input_size,
73                           float* sums) noexcept {

```

```

74     std::size_t index = 0;
75     __m256 accumulator0 = _mm256_setzero_ps();
76     __m256 accumulator1 = _mm256_setzero_ps();
77     __m256 accumulator2 = _mm256_setzero_ps();
78     __m256 accumulator3 = _mm256_setzero_ps();
79     __m256 accumulator4 = _mm256_setzero_ps();
80     __m256 accumulator5 = _mm256_setzero_ps();
81     __m256 accumulator6 = _mm256_setzero_ps();
82     __m256 accumulator7 = _mm256_setzero_ps();
83
84     for (; index + LCQI_F32_AVX2_LANE_COUNT <= input_size;
85          index += LCQI_F32_AVX2_LANE_COUNT) {
86         const __m256 input_values = _mm256_loadu_ps(input + index);
87         accumulator0 =
88             _mm256_fmadd_ps(_mm256_loadu_ps(rows[0] + index), input_values, ↵
↵ accumulator0);
89         accumulator1 =
90             _mm256_fmadd_ps(_mm256_loadu_ps(rows[1] + index), input_values, ↵
↵ accumulator1);
91         accumulator2 =
92             _mm256_fmadd_ps(_mm256_loadu_ps(rows[2] + index), input_values, ↵
↵ accumulator2);
93         accumulator3 =
94             _mm256_fmadd_ps(_mm256_loadu_ps(rows[3] + index), input_values, ↵
↵ accumulator3);
95         accumulator4 =
96             _mm256_fmadd_ps(_mm256_loadu_ps(rows[4] + index), input_values, ↵
↵ accumulator4);
97         accumulator5 =
98             _mm256_fmadd_ps(_mm256_loadu_ps(rows[5] + index), input_values, ↵
↵ accumulator5);
99         accumulator6 =
100             _mm256_fmadd_ps(_mm256_loadu_ps(rows[6] + index), input_values, ↵
↵ accumulator6);
101         accumulator7 =
102             _mm256_fmadd_ps(_mm256_loadu_ps(rows[7] + index), input_values, ↵
↵ accumulator7);
103     }
104
105     sums[0] = horizontal_sum(accumulator0);
106     sums[1] = horizontal_sum(accumulator1);
107     sums[2] = horizontal_sum(accumulator2);
108     sums[3] = horizontal_sum(accumulator3);
109     sums[4] = horizontal_sum(accumulator4);
110     sums[5] = horizontal_sum(accumulator5);
111     sums[6] = horizontal_sum(accumulator6);
112     sums[7] = horizontal_sum(accumulator7);
113     for (; index < input_size; ++index) {
114         const float input_value = input[index];
115         for (std::size_t row = 0; row < LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT; ++↵
↵ row) {
116             sums[row] += rows[row][index] * input_value;

```

```

117     }
118 }
119 }
120
121 template <std::size_t BATCH_SIZE>
122 void compute_batch_dot(const float* weights,
123                       const float* input,
124                       std::size_t input_size,
125                       float* sums) noexcept {
126     std::size_t index = 0;
127     __m256 accumulators[BATCH_SIZE];
128     std::array<const float*, BATCH_SIZE> inputs{};
129     for (std::size_t batch = 0; batch < BATCH_SIZE; ++batch) {
130         accumulators[batch] = _mm256_setzero_ps();
131         inputs[batch] = input + batch * input_size;
132     }
133
134     for (; index + LCQI_F32_AVX2_LANE_COUNT <= input_size;
135          index += LCQI_F32_AVX2_LANE_COUNT) {
136         const __m256 weight_values = _mm256_loadu_ps(weights + index);
137         for (std::size_t batch = 0; batch < BATCH_SIZE; ++batch) {
138             accumulators[batch] =
139                 _mm256_fmadd_ps(weight_values,
140                                _mm256_loadu_ps(inputs[batch] + index),
141                                accumulators[batch]);
142         }
143     }
144
145     for (std::size_t batch = 0; batch < BATCH_SIZE; ++batch) {
146         sums[batch] = horizontal_sum(accumulators[batch]);
147     }
148     for (; index < input_size; ++index) {
149         const float weight_value = weights[index];
150         for (std::size_t batch = 0; batch < BATCH_SIZE; ++batch) {
151             sums[batch] += weight_value * inputs[batch][index];
152         }
153     }
154 }
155
156 } // namespace
157
158 bool dot_f32_avx2_available() noexcept {
159     #if defined(__GNUC__) || defined(__clang__)
160         __builtin_cpu_init();
161         return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
162     #else
163         return true;
164     #endif
165 }
166
167 float dot_f32_avx2_unchecked(const float* lhs,
168                              const float* rhs,

```



```

215         std::size_t row_end,
216         float* output) noexcept {
217     std::size_t row = row_begin;
218     for (; row + LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT <= row_end;
219         row += LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT) {
220         std::array<const float*, LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT> rows{};
221         rows[0] = weights + row * input_size;
222         for (std::size_t offset = 1; offset < rows.size(); ++offset) {
223             rows[offset] = rows[offset - 1] + input_size;
224         }
225         std::array<float, LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT> sums{};
226         compute_eight_row_dot(rows.data(), input, input_size, sums.data());
227         for (std::size_t offset = 0; offset < ↵
↵ LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT; ++offset) {
228             output[row + offset] =
229                 sums[offset] + (bias == nullptr ? 0.0F : bias[row + offset]);
230         }
231     }
232     for (; row + LCQI_F32_AVX2_UNROLL_COUNT <= row_end;
233         row += LCQI_F32_AVX2_UNROLL_COUNT) {
234         const float* row0 = weights + row * input_size;
235         const float* row1 = row0 + input_size;
236         const float* row2 = row1 + input_size;
237         const float* row3 = row2 + input_size;
238         std::array<float, LCQI_F32_AVX2_UNROLL_COUNT> sums{};
239         compute_four_row_dot(row0, row1, row2, row3, input, input_size, sums.↵
↵ data());
240         for (std::size_t offset = 0; offset < LCQI_F32_AVX2_UNROLL_COUNT; ++↵
↵ offset) {
241             output[row + offset] =
242                 sums[offset] + (bias == nullptr ? 0.0F : bias[row + offset]);
243         }
244     }
245     for (; row < row_end; ++row) {
246         const float* weight_row = weights + row * input_size;
247         output[row] = dot_f32_avx2_unchecked(weight_row, input, input_size) +
248             (bias == nullptr ? 0.0F : bias[row]);
249     }
250 }
251
252 void linear_f32_batch_rows_avx2_unchecked(const float* weights,
253     const float* input,
254     const float* bias,
255     std::size_t input_size,
256     std::size_t batch_size,
257     std::size_t row_begin,
258     std::size_t row_end,
259     std::size_t output_stride,
260     float* output) noexcept {
261     for (std::size_t row = row_begin; row < row_end; ++row) {
262         const float* weight_row = weights + row * input_size;
263         std::size_t batch = 0;

```

```

264     for (; batch + LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT <= batch_size;
265           batch += LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT) {
266         std::array<float, LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT> sums{};
267         compute_batch_dot<LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT>(
268             weight_row,
269             input + batch * input_size,
270             input_size,
271             sums.data());
272         for (std::size_t offset = 0;
273               offset < LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT;
274               ++offset) {
275             output[(batch + offset) * output_stride + row] =
276                 sums[offset] + (bias == nullptr ? 0.0F : bias[row]);
277         }
278     }
279     for (; batch + LCQI_F32_AVX2_BATCH_UNROLL_COUNT <= batch_size;
280           batch += LCQI_F32_AVX2_BATCH_UNROLL_COUNT) {
281         std::array<float, LCQI_F32_AVX2_BATCH_UNROLL_COUNT> sums{};
282         compute_batch_dot<LCQI_F32_AVX2_BATCH_UNROLL_COUNT>(weight_row,
283                                                             input + batch * ↵
↵ input_size,
                                                             input_size,
                                                             sums.data());
284
285         for (std::size_t offset = 0; offset < ↵
↵ LCQI_F32_AVX2_BATCH_UNROLL_COUNT; ++offset) {
286             output[(batch + offset) * output_stride + row] =
287                 sums[offset] + (bias == nullptr ? 0.0F : bias[row]);
288         }
289     }
290 }
291 for (; batch < batch_size; ++batch) {
292     const std::size_t remaining = batch_size - batch;
293     if (remaining == 3U) {
294         std::array<float, 3> sums{};
295         compute_batch_dot<3>(weight_row,
296                               input + batch * input_size,
297                               input_size,
298                               sums.data());
299         for (std::size_t offset = 0; offset < sums.size(); ++offset) {
300             output[(batch + offset) * output_stride + row] =
301                 sums[offset] + (bias == nullptr ? 0.0F : bias[row]);
302         }
303         break;
304     }
305     if (remaining == 2U) {
306         std::array<float, 2> sums{};
307         compute_batch_dot<2>(weight_row,
308                               input + batch * input_size,
309                               input_size,
310                               sums.data());
311         for (std::size_t offset = 0; offset < sums.size(); ++offset) {
312             output[(batch + offset) * output_stride + row] =
313                 sums[offset] + (bias == nullptr ? 0.0F : bias[row]);

```

```

314         }
315         break;
316     }
317     output[batch * output_stride + row] =
318         dot_f32_avx2_unchecked(weight_row,
319                                input + batch * input_size,
320                                input_size) +
321         (bias == nullptr ? 0.0F : bias[row]);
322     }
323 }
324 }
325
326 F32RowMax max_dot_f32_rows_avx2_unchecked(const float* weights,
327                                           const float* input,
328                                           std::size_t input_size,
329                                           std::size_t row_begin,
330                                           std::size_t row_end) noexcept {
331     F32RowMax best;
332     best.row = row_begin;
333     best.value = -std::numeric_limits<float>::infinity();
334     std::size_t row = row_begin;
335     for (; row + LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT <= row_end;
336          row += LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT) {
337         std::array<const float*, LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT> rows{};
338         rows[0] = weights + row * input_size;
339         for (std::size_t offset = 1; offset < rows.size(); ++offset) {
340             rows[offset] = rows[offset - 1] + input_size;
341         }
342         std::array<float, LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT> sums{};
343         compute_eight_row_dot(rows.data(), input, input_size, sums.data());
344         for (std::size_t offset = 0; offset < ↵
↵ LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT; ++offset) {
345             if (row + offset == row_begin || sums[offset] > best.value) {
346                 best.row = row + offset;
347                 best.value = sums[offset];
348             }
349         }
350     }
351     for (; row + LCQI_F32_AVX2_UNROLL_COUNT <= row_end;
352          row += LCQI_F32_AVX2_UNROLL_COUNT) {
353         const float* row0 = weights + row * input_size;
354         const float* row1 = row0 + input_size;
355         const float* row2 = row1 + input_size;
356         const float* row3 = row2 + input_size;
357         std::array<float, LCQI_F32_AVX2_UNROLL_COUNT> sums{};
358         compute_four_row_dot(row0, row1, row2, row3, input, input_size, sums.↵
↵ data());
359         for (std::size_t offset = 0; offset < LCQI_F32_AVX2_UNROLL_COUNT; ++↵
↵ offset) {
360             if (row + offset == row_begin || sums[offset] > best.value) {
361                 best.row = row + offset;
362                 best.value = sums[offset];

```

```

363     }
364 }
365 }
366 for (; row < row_end; ++row) {
367     const float* weight_row = weights + row * input_size;
368     const float value = dot_f32_avx2_unchecked(weight_row, input, ↵
↵ input_size);
369     if (row == row_begin || value > best.value) {
370         best.row = row;
371         best.value = value;
372     }
373 }
374 return best;
375 }
376
377 } // namespace lcqi
378
379 #endif

```

Listing 10.13 src/gpt2_reference.cpp

```

1 #include <lcqi/gpt2_reference.hpp>
2
3 #include <lcqi/f32_kernels.hpp>
4 #include <lcqi/safetensors.hpp>
5
6 #include <algorithm>
7 #include <array>
8 #include <atomic>
9 #include <chrono>
10 #include <cmath>
11 #include <cstdint>
12 #include <fstream>
13 #include <limits>
14 #include <optional>
15 #include <sstream>
16 #include <stdexcept>
17 #include <string>
18 #include <string_view>
19 #include <thread>
20 #include <unordered_set>
21 #include <utility>
22
23 namespace lcqi {
24 namespace {
25
26 constexpr std::int32_t LCQI_GPT2_DEFAULT_BOS_EOS = 50256;
27 constexpr std::int32_t LCQI_GPT2_ATTENTION_PROJECTIONS = 3;
28 constexpr std::int32_t LCQI_GPT2_PAIR_TOKEN_COUNT = 2;
29 constexpr std::int32_t LCQI_GPT2_BYTE_COUNT = 256;
30 constexpr std::int32_t LCQI_GPT2_UNICODE_PRIVATE_BASE = 256;
31 constexpr std::int32_t LCQI_GPT2_ASCII_EXCLAMATION = 33;

```

```

32 constexpr std::int32_t LCQI_GPT2_ASCII_TILDE = 126;
33 constexpr std::int32_t LCQI_GPT2_LATIN_INVERTED_EXCLAMATION = 161;
34 constexpr std::int32_t LCQI_GPT2_LATIN_NOT_SIGN = 172;
35 constexpr std::int32_t LCQI_GPT2_LATIN_REGISTERED = 174;
36 constexpr std::int32_t LCQI_GPT2_LATIN_Y_DIAERESIS = 255;
37 constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_MASK = 0x3F;
38 constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_BITS = 0x80;
39 constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_HEAD = 0xC0;
40 constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_HEAD = 0xE0;
41 constexpr std::int32_t LCQI_GPT2_UTF8_FOUR_BYTE_HEAD = 0xF0;
42 constexpr std::int32_t LCQI_GPT2_UTF8_ONE_BYTE_LIMIT = 0x80;
43 constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_LIMIT = 0x800;
44 constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_LIMIT = 0x10000;
45 constexpr unsigned char LCQI_GPT2_ASCII_APOSTROPHE = 0x27U;
46 constexpr float LCQI_GPT2_GELU_SQRT_2_OVER_PI = 0.7978845608028654F;
47 constexpr float LCQI_GPT2_GELU_CUBIC_COEFF = 0.044715F;
48 constexpr float LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY =
49     -std::numeric_limits<float>::infinity();
50 constexpr std::int32_t LCQI_GPT2_MAX_AUTO_WORKERS = 8;
51 constexpr std::int32_t LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS = 512;
52 constexpr std::int32_t LCQI_GPT2_PARALLEL_MIN_COLUMNS = 512;
53 constexpr std::int64_t LCQI_GPT2_PARALLEL_MIN_OPS = 1'000'000;
54 constexpr std::size_t LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER = 4;
55 constexpr std::size_t LCQI_GPT2_WORKER_SPIN_PAUSES_BEFORE_YIELD = 4096;
56
57 using Gpt2ProfileClock = std::chrono::steady_clock;
58
59 void pause_worker_spin(std::size_t& pause_count) noexcept {
60     #if defined(__GNUC__) && (defined(__x86_64__) || defined(__i386__))
61         __builtin_ia32_pause();
62     #endif
63     ++pause_count;
64     if (pause_count >= LCQI_GPT2_WORKER_SPIN_PAUSES_BEFORE_YIELD) {
65         pause_count = 0;
66         std::this_thread::yield();
67     }
68 }
69
70 } // namespace
71
72 namespace detail {
73
74 class Gpt2ParallelWorkerPool {
75 public:
76     explicit Gpt2ParallelWorkerPool(std::int32_t requested_workers)
77         : worker_count_(Gpt2ParallelWorkerPool::normalize_worker_count(↵
↵ requested_workers)) {
78         if (this->worker_count_ <= 1) {
79             return;
80         }
81         const std::size_t background_count =
82             static_cast<std::size_t>(this->worker_count_ - 1);

```

```

83     this->threads_.reserve(background_count);
84     for (std::size_t index = 0; index < background_count; ++index) {
85         this->threads_.emplace_back([this, index]() { this->worker_loop(↵
↵ index); });
86     }
87 }
88
89 ~Gpt2ParallelWorkerPool() {
90     this->stopping_.store(true, std::memory_order_release);
91     this->generation_.fetch_add(1, std::memory_order_release);
92     for (std::thread& thread : this->threads_) {
93         if (thread.joinable()) {
94             thread.join();
95         }
96     }
97 }
98
99 Gpt2ParallelWorkerPool(const Gpt2ParallelWorkerPool&) = delete;
100 Gpt2ParallelWorkerPool& operator=(const Gpt2ParallelWorkerPool&) = delete;
101
102 [[nodiscard]] std::int32_t worker_count() const noexcept {
103     return this->worker_count_;
104 }
105
106 template <typename Function>
107 void parallel_for_rows(std::size_t begin,
108                       std::size_t end,
109                       std::size_t grain,
110                       Function&& function) {
111     if (end <= begin) {
112         return;
113     }
114     const std::size_t row_count = end - begin;
115     if (this->worker_count_ <= 1 || row_count <= grain) {
116         function(begin, end);
117         return;
118     }
119
120     const std::size_t task_count =
121         std::min<std::size_t>(static_cast<std::size_t>(this->worker_count_)↵
↵ ,
122                               (row_count + grain - 1) / grain);
123     if (task_count <= 1) {
124         function(begin, end);
125         return;
126     }
127
128     auto task_context = ParallelTaskContext<Function>{function};
129     // Publish all task metadata before the generation bump; workers use ↵
↵ the
130     // generation acquire load as the hand-off point for this stack context↵
↵ .

```

```

131     this->task_begin_.store(begin, std::memory_order_release);
132     this->task_end_.store(end, std::memory_order_release);
133     this->task_count_.store(task_count, std::memory_order_release);
134     this->task_context_.store(&task_context, std::memory_order_release);
135     this->task_invoke_.store(&ParallelTaskContext<Function>::invoke,
136                             std::memory_order_release);
137     this->active_workers_.store(task_count - 1, std::memory_order_release);
138     this->generation_.fetch_add(1, std::memory_order_release);
139
140     const std::size_t main_worker = task_count - 1;
141     const auto [main_begin, main_end] = this->chunk_bounds(begin, end, ↵
↵ task_count, main_worker);
142     function(main_begin, main_end);
143
144     std::size_t pause_count = 0;
145     while (this->active_workers_.load(std::memory_order_acquire) != 0) {
146         pause_worker_spin(pause_count);
147     }
148     this->task_context_.store(nullptr, std::memory_order_release);
149     this->task_invoke_.store(nullptr, std::memory_order_release);
150 }
151
152 private:
153     template <typename Function>
154     struct ParallelTaskContext {
155         Function& function;
156
157         static void invoke(void* context, std::size_t begin, std::size_t end) {
158             auto* typed_context = static_cast<ParallelTaskContext*>(context);
159             typed_context->function(begin, end);
160         }
161     };
162
163     std::int32_t worker_count_ = 1;
164     std::vector<std::thread> threads_;
165     std::atomic<bool> stopping_ = false;
166     std::atomic<std::uint64_t> generation_ = 0;
167     std::atomic<std::size_t> task_begin_ = 0;
168     std::atomic<std::size_t> task_end_ = 0;
169     std::atomic<std::size_t> task_count_ = 0;
170     std::atomic<std::size_t> active_workers_ = 0;
171     std::atomic<void*> task_context_ = nullptr;
172     std::atomic<void (*) (void*, std::size_t, std::size_t)> task_invoke_ = ↵
↵ nullptr;
173
174     [[nodiscard]] static std::int32_t normalize_worker_count(std::int32_t ↵
↵ requested_workers) {
175         if (requested_workers > 0) {
176             return requested_workers;
177         }
178         const unsigned int hardware_workers = std::thread::hardware_concurrency↵
↵ ();

```



```

179     if (hardware_workers == 0) {
180         return 1;
181     }
182     const std::int32_t capped =
183         std::min<std::int32_t>(static_cast<std::int32_t>(hardware_workers),
184                               LCQI_GPT2_MAX_AUTO_WORKERS);
185     return std::max(capped, 1);
186 }
187
188 [[nodiscard]] std::pair<std::size_t, std::size_t> chunk_bounds(
189     std::size_t begin,
190     std::size_t end,
191     std::size_t task_count,
192     std::size_t worker_index) const {
193     const std::size_t row_count = end - begin;
194     const std::size_t chunk_begin = begin + row_count * worker_index / ↵
↵ task_count;
195     const std::size_t chunk_end = begin + row_count * (worker_index + 1) / ↵
↵ task_count;
196     return {chunk_begin, chunk_end};
197 }
198
199 void worker_loop(std::size_t worker_index) {
200     std::uint64_t observed_generation = 0;
201     std::size_t pause_count = 0;
202     while (true) {
203         const std::uint64_t current_generation =
204             this->generation_.load(std::memory_order_acquire);
205         if (current_generation == observed_generation) {
206             if (this->stopping_.load(std::memory_order_acquire)) {
207                 return;
208             }
209             pause_worker_spin(pause_count);
210             continue;
211         }
212
213         observed_generation = current_generation;
214         pause_count = 0;
215         if (this->stopping_.load(std::memory_order_acquire)) {
216             return;
217         }
218
219         const std::size_t task_count = this->task_count_.load(std::↵
↵ memory_order_acquire);
220         if (this->generation_.load(std::memory_order_acquire) != ↵
↵ current_generation) {
221             continue;
222         }
223         const std::size_t background_task_count = task_count - 1;
224         if (worker_index >= background_task_count) {
225             continue;
226         }

```

```

227         void* task_context = this->task_context_.load(std::memory_order_acquire);
228         void (*task_invoke)(void*, std::size_t, std::size_t) =
229             this->task_invoke_.load(std::memory_order_acquire);
230         if (task_context == nullptr || task_invoke == nullptr) {
231             continue;
232         }
233         const std::size_t task_begin = this->task_begin_.load(std::memory_order_acquire);
234         const std::size_t task_end = this->task_end_.load(std::memory_order_acquire);
235         if (this->generation_.load(std::memory_order_acquire) != current_generation) {
236             continue;
237         }
238         const auto [begin, end] =
239             this->chunk_bounds(task_begin,
240                               task_end,
241                               task_count,
242                               worker_index);
243         task_invoke(task_context, begin, end);
244
245         this->active_workers_.fetch_sub(1, std::memory_order_acq_rel);
246     }
247 }
248 };
249
250 } // namespace detail
251
252 namespace {
253
254 class JsonCursor {
255 public:
256     explicit JsonCursor(std::string_view text) : text_(text) {}
257
258     void expect(char expected) {
259         this->skip_ws();
260         if (this->eof() || this->text_[this->position_] != expected) {
261             throw std::runtime_error("invalid JSON syntax");
262         }
263         ++this->position_;
264     }
265
266     [[nodiscard]] bool consume(char expected) {
267         this->skip_ws();
268         if (!this->eof() && this->text_[this->position_] == expected) {
269             ++this->position_;
270             return true;
271         }
272         return false;
273     }
274 }

```

```

275 [[nodiscard]] std::string parse_string() {
276     this->skip_ws();
277     this->expect('\"');
278     std::string result;
279     while (!this->eof()) {
280         const char ch = this->text_[this->position_++];
281         if (ch == '\"') {
282             return result;
283         }
284         if (ch == '\\') {
285             result += this->parse_escape();
286         } else {
287             result.push_back(ch);
288         }
289     }
290     throw std::runtime_error("unterminated JSON string");
291 }
292
293 [[nodiscard]] std::int64_t parse_i64() {
294     this->skip_ws();
295     bool negative = false;
296     if (this->consume('-')) {
297         negative = true;
298     }
299     if (this->eof() || this->text_[this->position_] < '0' ||
300         this->text_[this->position_] > '9') {
301         throw std::runtime_error("expected JSON integer");
302     }
303     std::int64_t value = 0;
304     while (!this->eof() && this->text_[this->position_] >= '0' &&
305         this->text_[this->position_] <= '9') {
306         const std::int64_t digit = this->text_[this->position_] - '0';
307         if (value >
308             (std::numeric_limits<std::int64_t>::max() - digit) / 10) {
309             throw std::runtime_error("JSON integer overflow");
310         }
311         value = value * 10 + digit;
312         ++this->position_;
313     }
314     return negative ? -value : value;
315 }
316
317 [[nodiscard]] double parse_number() {
318     this->skip_ws();
319     const std::size_t begin = this->position_;
320     if (!this->eof() && this->text_[this->position_] == '-') {
321         ++this->position_;
322     }
323     while (!this->eof() && this->text_[this->position_] >= '0' &&
324         this->text_[this->position_] <= '9') {
325         ++this->position_;
326     }

```

```

327     if (!this->eof() && this->text_[this->position_] == '.') {
328         ++this->position_;
329         while (!this->eof() && this->text_[this->position_] >= '0' &&
330             this->text_[this->position_] <= '9') {
331             ++this->position_;
332         }
333     }
334     if (!this->eof() &&
335         (this->text_[this->position_] == 'e' || this->text_[this->position_] == 'E')) {
336         ++this->position_;
337         if (!this->eof() &&
338             (this->text_[this->position_] == '+' || this->text_[this->position_] == '-')) {
339             ++this->position_;
340         }
341         while (!this->eof() && this->text_[this->position_] >= '0' &&
342             this->text_[this->position_] <= '9') {
343             ++this->position_;
344         }
345     }
346     if (begin == this->position_) {
347         throw std::runtime_error("expected JSON number");
348     }
349     return std::stod(std::string(this->text_.substr(begin, this->position_ - begin)));
350 }
351
352 void skip_value() {
353     this->skip_ws();
354     if (this->eof()) {
355         throw std::runtime_error("unexpected end of JSON");
356     }
357     const char ch = this->text_[this->position_];
358     if (ch == '"') {
359         static_cast<void>(this->parse_string());
360         return;
361     }
362     if (ch == '{') {
363         this->expect('{');
364         if (this->consume('}')) {
365             return;
366         }
367         while (true) {
368             static_cast<void>(this->parse_string());
369             this->expect(':');
370             this->skip_value();
371             if (this->consume('}')) {
372                 return;
373             }
374             this->expect(',');
375         }

```

```

376     }
377     if (ch == '[') {
378         this->expect('[');
379         if (this->consume(' ')) {
380             return;
381         }
382         while (true) {
383             this->skip_value();
384             if (this->consume(' ')) {
385                 return;
386             }
387             this->expect(',');
388         }
389     }
390     if (ch == 't') {
391         this->expect_literal("true");
392         return;
393     }
394     if (ch == 'f') {
395         this->expect_literal("false");
396         return;
397     }
398     if (ch == 'n') {
399         this->expect_literal("null");
400         return;
401     }
402     static_cast<void*>(this->parse_number());
403 }
404
405 [[nodiscard]] bool eof_after_ws() {
406     this->skip_ws();
407     return this->eof();
408 }
409
410 private:
411     std::string_view text_;
412     std::size_t position_ = 0;
413
414     [[nodiscard]] bool eof() const {
415         return this->position_ >= this->text_.size();
416     }
417
418     void skip_ws() {
419         while (!this->eof() &&
420             (this->text_[this->position_] == ' ' ||
421              this->text_[this->position_] == '\n' ||
422              this->text_[this->position_] == '\r' ||
423              this->text_[this->position_] == '\t')) {
424             ++this->position_;
425         }
426     }
427

```

```

428 void expect_literal(std::string_view literal) {
429     for (const char expected : literal) {
430         if (this->eof() || this->text_[this->position_] != expected) {
431             throw std::runtime_error("invalid JSON literal");
432         }
433         ++this->position_;
434     }
435 }
436
437 [[nodiscard]] std::string parse_escape() {
438     if (this->eof()) {
439         throw std::runtime_error("unterminated JSON escape");
440     }
441     const char escaped = this->text_[this->position_++];
442     switch (escaped) {
443         case '"':
444         case '\\':
445         case '/':
446             return std::string(1, escaped);
447         case 'b':
448             return "\b";
449         case 'f':
450             return "\f";
451         case 'n':
452             return "\n";
453         case 'r':
454             return "\r";
455         case 't':
456             return "\t";
457         case 'u':
458             return this->parse_unicode_escape();
459         default:
460             throw std::runtime_error("unsupported JSON escape");
461     }
462 }
463
464 [[nodiscard]] std::string parse_unicode_escape() {
465     std::uint32_t value = 0;
466     for (std::int32_t i = 0; i < 4; ++i) {
467         if (this->eof()) {
468             throw std::runtime_error("unterminated JSON unicode escape");
469         }
470         const char ch = this->text_[this->position_++];
471         value <= 4U;
472         if (ch >= '0' && ch <= '9') {
473             value += static_cast<std::uint32_t>(ch - '0');
474         } else if (ch >= 'a' && ch <= 'f') {
475             value += static_cast<std::uint32_t>(10 + ch - 'a');
476         } else if (ch >= 'A' && ch <= 'F') {
477             value += static_cast<std::uint32_t>(10 + ch - 'A');
478         } else {
479             throw std::runtime_error("invalid JSON unicode escape");

```

```

480     }
481 }
482 std::string encoded;
483 if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
484     encoded.push_back(static_cast<char>(value));
485 } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
486     encoded.push_back(static_cast<char>(
487         LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
488     encoded.push_back(static_cast<char>(
489         LCQI_GPT2_UTF8_CONTINUATION_BITS |
490         (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
491 } else {
492     encoded.push_back(static_cast<char>(
493         LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
494     encoded.push_back(static_cast<char>(
495         LCQI_GPT2_UTF8_CONTINUATION_BITS |
496         ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
497     encoded.push_back(static_cast<char>(
498         LCQI_GPT2_UTF8_CONTINUATION_BITS |
499         (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
500 }
501 return encoded;
502 }
503 };
504
505 std::string read_text_file(const std::filesystem::path& path) {
506     std::ifstream input(path);
507     if (!input) {
508         throw std::runtime_error("cannot open text file: " + path.string());
509     }
510     std::ostringstream buffer;
511     buffer << input.rdbuf();
512     return buffer.str();
513 }
514
515 std::size_t checked_size(std::int64_t value, const char* name) {
516     if (value < 0) {
517         throw std::runtime_error(std::string(name) + " cannot be negative");
518     }
519     return static_cast<std::size_t>(value);
520 }
521
522 std::string encode_codepoint(std::uint32_t value) {
523     std::string encoded;
524     if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
525         encoded.push_back(static_cast<char>(value));
526     } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
527         encoded.push_back(static_cast<char>(
528             LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
529         encoded.push_back(static_cast<char>(
530             LCQI_GPT2_UTF8_CONTINUATION_BITS |
531             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));

```

```

532     } else if (value < LCQI_GPT2_UTF8_THREE_BYTE_LIMIT) {
533         encoded.push_back(static_cast<char>(
534             LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
535         encoded.push_back(static_cast<char>(
536             LCQI_GPT2_UTF8_CONTINUATION_BITS |
537             ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
538         encoded.push_back(static_cast<char>(
539             LCQI_GPT2_UTF8_CONTINUATION_BITS |
540             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
541     } else {
542         encoded.push_back(static_cast<char>(
543             LCQI_GPT2_UTF8_FOUR_BYTE_HEAD | (value >> 18U)));
544         encoded.push_back(static_cast<char>(
545             LCQI_GPT2_UTF8_CONTINUATION_BITS |
546             ((value >> 12U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
547         encoded.push_back(static_cast<char>(
548             LCQI_GPT2_UTF8_CONTINUATION_BITS |
549             ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
550         encoded.push_back(static_cast<char>(
551             LCQI_GPT2_UTF8_CONTINUATION_BITS |
552             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
553     }
554     return encoded;
555 }
556
557 void require_positive(std::int32_t value, const char* name) {
558     if (value <= 0) {
559         throw std::runtime_error(std::string(name) + " must be positive");
560     }
561 }
562
563 void validate_config(const Gpt2Config& config) {
564     require_positive(config.hidden_size, "hidden_size");
565     require_positive(config.head_count, "head_count");
566     require_positive(config.layer_count, "layer_count");
567     require_positive(config.max_positions, "max_positions");
568     require_positive(config.vocab_size, "vocab_size");
569     require_positive(config.intermediate_size, "intermediate_size");
570     if (config.hidden_size % config.head_count != 0) {
571         throw std::runtime_error("hidden_size must be divisible by head_count")↵
↵ ;
572     }
573     if (config.layer_norm_epsilon <= 0.0F) {
574         throw std::runtime_error("layer_norm_epsilon must be positive");
575     }
576 }
577
578 std::int32_t head_dim(const Gpt2Config& config) {
579     return config.hidden_size / config.head_count;
580 }
581
582 std::size_t matrix_size(std::int32_t rows, std::int32_t columns) {

```



```

583     return checked_size(rows, "rows") * checked_size(columns, "columns");
584 }
585
586 Gpt2ProfileClock::time_point profile_start(const Gpt2HotspotProfile* profile) {
587     if (profile == nullptr) {
588         return Gpt2ProfileClock::time_point{};
589     }
590     return Gpt2ProfileClock::now();
591 }
592
593 void add_profile_ms(Gpt2HotspotProfile* profile,
594                   double Gpt2HotspotProfile::*field,
595                   Gpt2ProfileClock::time_point start) {
596     if (profile == nullptr) {
597         return;
598     }
599     (*profile).*field +=
600         std::chrono::duration<double, std::milli>(Gpt2ProfileClock::now() -
601 ↪ start).count());
602 }
603
604 void validate_vector_size(std::size_t actual, std::int32_t expected, const char↪
605 ↪ * name) {
606     if (actual != checked_size(expected, name)) {
607         throw std::runtime_error(std::string(name) + " size mismatch");
608     }
609 }
610
611 void validate_linear(const Gpt2LinearF32& linear, const char* name) {
612     require_positive(linear.input_size, "linear input_size");
613     require_positive(linear.output_size, "linear output_size");
614     if (linear.weights.size() != matrix_size(linear.output_size, linear.↪
615 ↪ input_size)) {
616         throw std::runtime_error(std::string(name) + " weight size mismatch");
617     }
618     if (!linear.bias.empty() &&
619         linear.bias.size() != checked_size(linear.output_size, "linear ↪
620 ↪ output_size")) {
621         throw std::runtime_error(std::string(name) + " bias size mismatch");
622     }
623 }
624
625 void validate_model(const Gpt2ReferenceModel& model) {
626     validate_config(model.config);
627     if (model.layers.size() != checked_size(model.config.layer_count, "↪
628 ↪ layer_count")) {
629         throw std::runtime_error("GPT-2 layer count mismatch");
630     }
631     if (model.token_embedding.size() !=
632         matrix_size(model.config.vocab_size, model.config.hidden_size)) {
633         throw std::runtime_error("GPT-2 token embedding size mismatch");
634     }
635 }

```

```

630     if (model.position_embedding.size() !=
631         matrix_size(model.config.max_positions, model.config.hidden_size)) {
632         throw std::runtime_error("GPT-2 position embedding size mismatch");
633     }
634     validate_vector_size(model.final_ln_weight.size(), model.config.hidden_size,
635 ↪ , "ln_f weight");
636     validate_vector_size(model.final_ln_bias.size(), model.config.hidden_size,
637 ↪ "ln_f bias");
638     if (model.tie_lm_head_to_embedding) {
639         if (!model.lm_head_weight.empty()) {
640             throw std::runtime_error("tied GPT-2 lm_head should not carry ↪
641 ↪ separate weights");
642         }
643     } else if (model.lm_head_weight.size() !=
644         matrix_size(model.config.vocab_size, model.config.hidden_size)) ↪
645 ↪ {
646         throw std::runtime_error("GPT-2 lm_head size mismatch");
647     }
648     for (const Gpt2LayerWeightsF32& layer : model.layers) {
649         validate_vector_size(layer.ln_1_weight.size(), model.config.hidden_size,
650 ↪ , "ln_1 weight");
651         validate_vector_size(layer.ln_1_bias.size(), model.config.hidden_size,
652 ↪ "ln_1 bias");
653         validate_linear(layer.c_attn, "c_attn");
654         validate_linear(layer.c_proj, "c_proj");
655         validate_vector_size(layer.ln_2_weight.size(), model.config.hidden_size,
656 ↪ , "ln_2 weight");
657         validate_vector_size(layer.ln_2_bias.size(), model.config.hidden_size,
658 ↪ "ln_2 bias");
659         validate_linear(layer.c_fc, "c_fc");
660         validate_linear(layer.mlp_c_proj, "mlp_c_proj");
661     }
662 }
663
664 std::span<const float> row_span(std::span<const float> values,
665     std::int32_t row,
666     std::int32_t row_width) {
667     return values.subspan(check_size(row, "row") * checked_size(row_width, "↪
668 ↪ row_width"),
669     checked_size(row_width, "row_width"));
670 }
671
672 void add_inplace(std::span<float> target, std::span<const float> source) {
673     if (target.size() != source.size()) {
674         throw std::runtime_error("GPT-2 residual add size mismatch");
675     }
676     for (std::size_t index = 0; index < target.size(); ++index) {
677         target[index] += source[index];
678     }
679 }
680
681 void layer_norm(std::span<const float> input,

```

```

673         std::span<const float> weight,
674         std::span<const float> bias,
675         float epsilon,
676         std::span<float> output) {
677     if (input.empty() || input.size() != weight.size() || input.size() != bias.size() ||
        ↪ size() ||
678         input.size() != output.size()) {
679         throw std::runtime_error("GPT-2 LayerNorm size mismatch");
680     }
681     float mean = 0.0F;
682     for (const float value : input) {
683         mean += value;
684     }
685     mean /= static_cast<float>(input.size());
686
687     float variance = 0.0F;
688     for (const float value : input) {
689         const float centered = value - mean;
690         variance += centered * centered;
691     }
692     variance /= static_cast<float>(input.size());
693     const float scale = 1.0F / std::sqrt(variance + epsilon);
694
695     for (std::size_t index = 0; index < input.size(); ++index) {
696         output[index] = (input[index] - mean) * scale * weight[index] + bias[
        ↪ index];
697     }
698 }
699
700 void linear_f32(const Gpt2LinearF32& linear,
701                std::span<const float> input,
702                std::span<float> output) {
703     validate_linear(linear, "GPT-2 linear");
704     if (input.size() != checked_size(linear.input_size, "linear input") ||
705         output.size() != checked_size(linear.output_size, "linear output")) {
706         throw std::runtime_error("GPT-2 linear input/output size mismatch");
707     }
708     const std::size_t input_size = checked_size(linear.input_size, "linear
        ↪ input");
709     const std::size_t output_size = checked_size(linear.output_size, "linear
        ↪ output");
710     for (std::size_t out = 0; out < output_size; ++out) {
711         float sum = linear.bias.empty() ? 0.0F : linear.bias[out];
712         const std::size_t row_base = out * input_size;
713         for (std::size_t in = 0; in < input_size; ++in) {
714             sum += linear.weights[row_base + in] * input[in];
715         }
716         output[out] = sum;
717     }
718 }
719
720 void linear_f32_unchecked(const Gpt2LinearF32& linear,

```

```

721         std::span<const float> input,
722         std::span<float> output) {
723     const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
724     const std::size_t output_size = static_cast<std::size_t>(linear.output_size)↵
↵ );
725     const float* weights = linear.weights.data();
726     const float* bias = linear.bias.empty() ? nullptr : linear.bias.data();
727     const float* input_data = input.data();
728     linear_f32_rows_unchecked(weights, input_data, bias, input_size, 0, ↵
↵ output_size, output.data());
729 }
730
731 [[nodiscard]] bool should_parallelize_rows(const Gpt2ExecutionOptions& options,
732                                           std::size_t rows,
733                                           std::size_t columns,
734                                           const detail::Gpt2ParallelWorkerPool↵
↵ * worker_pool) {
735     if (worker_pool == nullptr || worker_pool->worker_count() <= 1) {
736         return false;
737     }
738     const std::int32_t min_rows =
739         options.parallel_min_rows > 0 ? options.parallel_min_rows
740         : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;
741     if (rows < checked_size(min_rows, "parallel_min_rows")) {
742         return false;
743     }
744     if (options.worker_count > 1) {
745         return true;
746     }
747     return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS, "↵
↵ parallel_min_columns") ||
748        rows * columns >= static_cast<std::size_t>(↵
↵ LCQI_GPT2_PARALLEL_MIN_OPS);
749 }
750
751 [[nodiscard]] std::size_t parallel_row_grain(std::size_t rows,
752                                              const detail::↵
↵ Gpt2ParallelWorkerPool& worker_pool) {
753     const std::size_t workers = checked_size(worker_pool.worker_count(), "↵
↵ worker_count");
754     const std::size_t target_tasks = workers * ↵
↵ LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER;
755     return std::max<std::size_t>(1, (rows + target_tasks - 1) / target_tasks);
756 }
757
758 [[nodiscard]] bool model_has_parallel_work(const Gpt2ReferenceModel& model,
759                                             const Gpt2ExecutionOptions& options)↵
↵ {
760     const std::size_t hidden_size = checked_size(model.config.hidden_size, "↵
↵ hidden_size");
761     const std::size_t vocab_size = checked_size(model.config.vocab_size, "↵
↵ vocab_size");

```

```

762     const std::size_t intermediate_size =
763         checked_size(model.config.intermediate_size, "intermediate_size");
764     const std::size_t qkv_rows =
765         checked_size(model.config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS
↵ ,
766                     "qkv output rows");
767     const std::int32_t min_rows =
768         options.parallel_min_rows > 0 ? options.parallel_min_rows
769             : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;
770     const auto large_enough = [min_rows](std::size_t rows, std::size_t columns)
↵ {
771         if (rows < checked_size(min_rows, "parallel_min_rows")) {
772             return false;
773         }
774         return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS,
775                                         "parallel_min_columns") ||
776             rows * columns >= static_cast<std::size_t>(
↵ LCQI_GPT2_PARALLEL_MIN_OPS);
777     };
778     if (options.worker_count > 1) {
779         return vocab_size >= checked_size(min_rows, "parallel_min_rows") ||
780             qkv_rows >= checked_size(min_rows, "parallel_min_rows") ||
781             intermediate_size >= checked_size(min_rows, "parallel_min_rows")
↵ ||
782             hidden_size >= checked_size(min_rows, "parallel_min_rows");
783     }
784     if (options.worker_count == 1) {
785         return false;
786     }
787     return large_enough(vocab_size, hidden_size) ||
788         large_enough(qkv_rows, hidden_size) ||
789         large_enough(hidden_size, hidden_size) ||
790         large_enough(intermediate_size, hidden_size) ||
791         large_enough(hidden_size, intermediate_size);
792 }
793
794 [[nodiscard]] std::unique_ptr<detail::Gpt2ParallelWorkerPool> make_worker_pool(
795     const Gpt2ReferenceModel& model,
796     const Gpt2ExecutionOptions& options) {
797     if (!model_has_parallel_work(model, options)) {
798         return nullptr;
799     }
800     return std::make_unique<detail::Gpt2ParallelWorkerPool>(options.
↵ worker_count);
801 }
802
803 void linear_f32_unchecked_parallel(const Gpt2LinearF32& linear,
804                                     std::span<const float> input,
805                                     std::span<float> output,
806                                     const Gpt2ExecutionOptions& options,
807                                     detail::Gpt2ParallelWorkerPool* worker_pool)
↵ {

```

```

808     const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
809     const std::size_t output_size = static_cast<std::size_t>(linear.output_size);
810     if (!should_parallelize_rows(options, output_size, input_size, worker_pool)) {
811         linear_f32_unchecked(linear, input, output);
812         return;
813     }
814
815     const float* weights = linear.weights.data();
816     const float* bias = linear.bias.empty() ? nullptr : linear.bias.data();
817     const float* input_data = input.data();
818     float* output_data = output.data();
819     const auto rows_fn =
820         [input_size, weights, bias, input_data, output_data](std::size_t begin,
821                                                                std::size_t end) {
822         linear_f32_rows_unchecked(weights,
823                                   input_data,
824                                   bias,
825                                   input_size,
826                                   begin,
827                                   end,
828                                   output_data);
829     };
830     worker_pool->parallel_for_rows(0, output_size, parallel_row_grain(
831     ↪ output_size, *worker_pool), rows_fn);
832 }
833
834 float gelu_new(float value) {
835     const float cubic = value * value * value;
836     return 0.5F * value *
837         (1.0F + std::tanh(LCQI_GPT2_GELU_SQRT_2_OVER_PI *
838                          (value + LCQI_GPT2_GELU_CUBIC_COEFF * cubic)));
839 }
840
841 std::int32_t argmax(std::span<const float> values) {
842     if (values.empty()) {
843         throw std::runtime_error("cannot take argmax of empty logits");
844     }
845     std::int32_t best_index = 0;
846     float best_value = values.front();
847     for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size(
848     ↪ )); ++index) {
849         const float value = values[checked_size(index, "argmax index")];
850         if (value > best_value) {
851             best_value = value;
852             best_index = index;
853         }
854     }
855     return best_index;
856 }

```

```

856 void project_qkv(const Gpt2Config& config,
857                 const Gpt2LayerWeightsF32& layer,
858                 std::span<const float> hidden,
859                 std::span<float> q,
860                 std::span<float> k,
861                 std::span<float> v) {
862     std::vector<float> packed(matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS,
863                                           config.hidden_size),
864                               0.0F);
865     linear_f32(layer.c_attn, hidden, packed);
866     const std::size_t width = checked_size(config.hidden_size, "hidden_size");
867     std::copy_n(packed.begin(), config.hidden_size, q.begin());
868     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
869               config.hidden_size,
870               k.begin());
871     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
872               config.hidden_size,
873               v.begin());
874 }
875
876 void project_qkv_reuse_workspace(const Gpt2Config& config,
877                                 const Gpt2LayerWeightsF32& layer,
878                                 std::span<const float> hidden,
879                                 std::span<float> packed,
880                                 std::span<float> q,
881                                 std::span<float> k,
882                                 std::span<float> v,
883                                 const Gpt2ExecutionOptions& options,
884                                 detail::Gpt2ParallelWorkerPool* worker_pool) {
885     linear_f32_unchecked_parallel(layer.c_attn, hidden, packed, options, ↵
886     ↵ worker_pool);
887     const std::size_t width = checked_size(config.hidden_size, "hidden_size");
888     std::copy_n(packed.begin(), config.hidden_size, q.begin());
889     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
890               config.hidden_size,
891               k.begin());
892     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
893               config.hidden_size,
894               v.begin());
895 }
896
897 void attention_full_sequence(const Gpt2Config& config,
898                             std::span<const float> q_by_pos,
899                             std::span<const float> k_by_pos,
900                             std::span<const float> v_by_pos,
901                             std::int32_t sequence_length,
902                             std::span<float> output_by_pos) {
903     const std::int32_t local_head_dim = head_dim(config);
904     const float score_scale = 1.0F / std::sqrt(static_cast<float>(↵
905     ↵ local_head_dim));
906     std::fill(output_by_pos.begin(), output_by_pos.end(), 0.0F);
907     std::vector<float> scores(checked_size(sequence_length, "sequence_length"),

```

```

906         LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY);
907
908     for (std::int32_t position = 0; position < sequence_length; ++position) {
909         for (std::int32_t head = 0; head < config.head_count; ++head) {
910             float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
911             for (std::int32_t past = 0; past <= position; ++past) {
912                 float dot = 0.0F;
913                 for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
914                     const std::size_t q_index =
915                         matrix_size(position, config.hidden_size) +
916                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
917                     const std::size_t k_index =
918                         matrix_size(past, config.hidden_size) +
919                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
920                     dot += q_by_pos[q_index] * k_by_pos[k_index];
921                 }
922                 const float score = dot * score_scale;
923                 scores[checked_size(past, "past")] = score;
924                 max_score = std::max(max_score, score);
925             }
926
927             float denominator = 0.0F;
928             for (std::int32_t past = 0; past <= position; ++past) {
929                 const std::size_t index = checked_size(past, "past");
930                 scores[index] = std::exp(scores[index] - max_score);
931                 denominator += scores[index];
932             }
933             if (denominator <= 0.0F) {
934                 throw std::runtime_error("GPT-2 attention denominator is ↵
↵ invalid");
935             }
936             for (std::int32_t past = 0; past <= position; ++past) {
937                 const float probability = scores[checked_size(past, "past")] / ↵
↵ denominator;
938                 for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
939                     const std::size_t output_index =
940                         matrix_size(position, config.hidden_size) +
941                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
942                     const std::size_t value_index =
943                         matrix_size(past, config.hidden_size) +
944                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
945                     output_by_pos[output_index] += probability * v_by_pos[↵
↵ value_index];
946                 }
947             }
948         }
949     }
950 }

```



```

951
952 void forward_gpt2_layer(const Gpt2Config& config,
953                         const Gpt2LayerWeightsF32& layer,
954                         std::int32_t sequence_length,
955                         std::vector<float>& hidden_by_pos) {
956     std::vector<float> normed(checkered_size(config.hidden_size, "hidden_size"), ←
    ↪ 0.0F);
957     std::vector<float> q_by_pos(matrix_size(sequence_length, config.hidden_size, ←
    ↪ ), 0.0F);
958     std::vector<float> k_by_pos(q_by_pos.size(), 0.0F);
959     std::vector<float> v_by_pos(q_by_pos.size(), 0.0F);
960     std::vector<float> attention_by_pos(q_by_pos.size(), 0.0F);
961     std::vector<float> projected(checkered_size(config.hidden_size, "hidden_size" ←
    ↪ ), 0.0F);
962     std::vector<float> mlp_fc(checkered_size(config.intermediate_size, " ←
    ↪ intermediate_size"), 0.0F);
963     std::vector<float> mlp_out(checkered_size(config.hidden_size, "hidden_size"), ←
    ↪ 0.0F);
964
965     for (std::int32_t position = 0; position < sequence_length; ++position) {
966         const std::span<float> hidden = std::span<float>(
967             hidden_by_pos.data() + matrix_size(position, config.hidden_size),
968             checked_size(config.hidden_size, "hidden_size"));
969         layer_norm(hidden,
970                 layer.ln_1_weight,
971                 layer.ln_1_bias,
972                 config.layer_norm_epsilon,
973                 normed);
974         std::span<float> q = std::span<float>(
975             q_by_pos.data() + matrix_size(position, config.hidden_size),
976             checked_size(config.hidden_size, "hidden_size"));
977         std::span<float> k = std::span<float>(
978             k_by_pos.data() + matrix_size(position, config.hidden_size),
979             checked_size(config.hidden_size, "hidden_size"));
980         std::span<float> v = std::span<float>(
981             v_by_pos.data() + matrix_size(position, config.hidden_size),
982             checked_size(config.hidden_size, "hidden_size"));
983         project_qkv(config, layer, normed, q, k, v);
984     }
985
986     attention_full_sequence(config, q_by_pos, k_by_pos, v_by_pos, ←
    ↪ sequence_length, attention_by_pos);
987
988     for (std::int32_t position = 0; position < sequence_length; ++position) {
989         const std::span<float> hidden = std::span<float>(
990             hidden_by_pos.data() + matrix_size(position, config.hidden_size),
991             checked_size(config.hidden_size, "hidden_size"));
992         const std::span<const float> attention = std::span<const float>(
993             attention_by_pos.data() + matrix_size(position, config.hidden_size) ←
    ↪ ,
994             checked_size(config.hidden_size, "hidden_size"));
995         linear_f32(layer.c_proj, attention, projected);

```

```

996         add_inplace(hidden, projected);
997
998         layer_norm(hidden,
999                     layer.ln_2_weight,
1000                     layer.ln_2_bias,
1001                     config.layer_norm_epsilon,
1002                     normed);
1003         linear_f32(layer.c_fc, normed, mlp_fc);
1004         for (float& value : mlp_fc) {
1005             value = gelu_new(value);
1006         }
1007         linear_f32(layer.mlp_c_proj, mlp_fc, mlp_out);
1008         add_inplace(hidden, mlp_out);
1009     }
1010 }
1011
1012 void forward_gpt2_layer_cached(const Gpt2Config& config,
1013                               const Gpt2LayerWeightsF32& layer,
1014                               std::int32_t layer_id,
1015                               std::int32_t model_position,
1016                               Gpt2KvCache& cache,
1017                               Gpt2ForwardWorkspace& workspace,
1018                               std::span<float> hidden,
1019                               Gpt2HotspotProfile* hotspot_profile,
1020                               const Gpt2ExecutionOptions& options,
1021                               detail::Gpt2ParallelWorkerPool* worker_pool) {
1022     if (hotspot_profile != nullptr) {
1023         ++hotspot_profile->layer_steps;
1024     }
1025     std::span<float> normed = workspace.normed();
1026     std::span<float> query = workspace.query();
1027     std::span<float> key = workspace.key();
1028     std::span<float> value = workspace.value();
1029     std::span<float> attention = workspace.attention();
1030     std::span<float> projected = workspace.projected();
1031     std::span<float> qkv_packed = workspace.qkv_packed();
1032     std::span<float> mlp_fc = workspace.mlp_fc();
1033     std::span<float> mlp_out = workspace.mlp_out();
1034     std::span<float> scores = workspace.scores_prefix(model_position + 1);
1035
1036     Gpt2ProfileClock::time_point start =
1037         profile_start(hotspot_profile);
1038     layer_norm(hidden,
1039               layer.ln_1_weight,
1040               layer.ln_1_bias,
1041               config.layer_norm_epsilon,
1042               normed);
1043     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);
1044
1045     start = profile_start(hotspot_profile);
1046     project_qkv_reuse_workspace(config,
1047                                layer,

```

```

1048         normed,
1049         qkv_packed,
1050         query,
1051         key,
1052         value,
1053         options,
1054         worker_pool);
1055 add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::qkv_projection_ms, ↵
↵ start);
1056
1057 start = profile_start(hotspot_profile);
1058 cache.append(layer_id, model_position, key, value);
1059 add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::kv_append_ms, start);
1060
1061 start = profile_start(hotspot_profile);
1062 detail::attend_cached_position(cache,
1063                                layer_id,
1064                                model_position,
1065                                query,
1066                                scores,
1067                                attention);
1068 add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::attention_ms, start);
1069
1070 start = profile_start(hotspot_profile);
1071 linear_f32_unchecked_parallel(layer.c_proj, attention, projected, options, ↵
↵ worker_pool);
1072 add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::↵
↵ attention_projection_ms, start);
1073
1074 start = profile_start(hotspot_profile);
1075 add_inplace(hidden, projected);
1076 add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::residual_add_ms, start↵
↵ );
1077
1078 start = profile_start(hotspot_profile);
1079 layer_norm(hidden,
1080            layer.ln_2_weight,
1081            layer.ln_2_bias,
1082            config.layer_norm_epsilon,
1083            normed);
1084 add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);
1085
1086 start = profile_start(hotspot_profile);
1087 linear_f32_unchecked_parallel(layer.c_fc, normed, mlp_fc, options, ↵
↵ worker_pool);
1088 add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::mlp_fc_ms, start);
1089
1090 start = profile_start(hotspot_profile);
1091 for (float& value_ref : mlp_fc) {
1092     value_ref = gelu_new(value_ref);
1093 }
1094 add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::gelu_ms, start);

```

[illegible]

```

1138     rows_fn(0, vocab_size);
1139 }
1140 }
1141
1142 void compute_logits(const Gpt2ReferenceModel& model,
1143                    std::span<const float> hidden,
1144                    std::span<float> logits) {
1145     Gpt2ExecutionOptions options;
1146     options.worker_count = 1;
1147     compute_logits(model, hidden, logits, options, nullptr);
1148 }
1149
1150 struct Gpt2TokenScore {
1151     std::int32_t token = 0;
1152     float value = -std::numeric_limits<float>::infinity();
1153 };
1154
1155 enum class Gpt2CachedStepMode {
1156     advance_only,
1157     predict_token,
1158     logits,
1159 };
1160
1161 [[nodiscard]] Gpt2TokenScore compute_predicted_token_range(const float* ↵
1162     ↵ weight_data,
1163                                     const float* ↵
1164     ↵ hidden_data,
1165                                     std::size_t ↵
1166     ↵ hidden_size,
1167                                     std::size_t begin,
1168                                     std::size_t end) {
1169     Gpt2TokenScore best;
1170     const F32RowMax row_max =
1171         max_dot_f32_rows_unchecked(weight_data, hidden_data, hidden_size, begin↵
1172     ↵ , end);
1173     best.token = static_cast<std::int32_t>(row_max.row);
1174     best.value = row_max.value;
1175     return best;
1176 }
1177
1178 std::int32_t compute_predicted_token(const Gpt2ReferenceModel& model,
1179                                     std::span<const float> hidden,
1180                                     const Gpt2ExecutionOptions& options,
1181                                     detail::Gpt2ParallelWorkerPool* ↵
1182     ↵ worker_pool) {
1183     const std::span<const float> weight =
1184         model.tie_lm_head_to_embedding ? std::span<const float>(model.↵
1185     ↵ token_embedding)
1186                                     : std::span<const float>(model.↵
1187     ↵ lm_head_weight);
1188     const std::size_t hidden_size = checked_size(model.config.hidden_size, "↵
1189     ↵ hidden_size");

```

```

1182     const std::size_t vocab_size = checked_size(model.config.vocab_size, "vocab_size");
1183     const float* weight_data = weight.data();
1184     const float* hidden_data = hidden.data();
1185     if (!should_parallelize_rows(options, vocab_size, hidden_size, worker_pool)) {
1186         return compute_predicted_token_range(weight_data, hidden_data,
1187             hidden_size, 0, vocab_size)
1188             .token;
1189     }
1190     const std::size_t worker_count = checked_size(worker_pool->worker_count(), "worker_count");
1191     const std::size_t chunk_count = std::min(worker_count, vocab_size);
1192     std::vector<Gpt2TokenScore> partials(chunk_count);
1193     const auto rows_fn =
1194         [chunk_count, hidden_size, vocab_size, weight_data, hidden_data, &partials](
1195             std::size_t begin,
1196             std::size_t end) {
1197         for (std::size_t chunk = begin; chunk < end; ++chunk) {
1198             const std::size_t token_begin = vocab_size * chunk /
1199                 chunk_count;
1200             const std::size_t token_end = vocab_size * (chunk + 1) /
1201                 chunk_count;
1202             partials[chunk] = compute_predicted_token_range(
1203                 weight_data,
1204                 hidden_data,
1205                 hidden_size,
1206                 token_begin,
1207                 token_end);
1208         }
1209     };
1210     worker_pool->parallel_for_rows(0, chunk_count, 1, rows_fn);
1211     Gpt2TokenScore best = partials.front();
1212     for (std::size_t index = 1; index < partials.size(); ++index) {
1213         if (partials[index].value > best.value) {
1214             best = partials[index];
1215         }
1216     }
1217     return best.token;
1218 }
1219 Gpt2ForwardResult run_gpt2_cached_step_unchecked(const Gpt2ReferenceModel&
1220     model,
1221     Gpt2KvCache& cache,
1222     Gpt2ForwardWorkspace&
1223     workspace,
1224     std::int32_t token_id,
1225     Gpt2CachedStepMode mode,
1226     Gpt2HotspotProfile*

```

```

    ↪ hotspot_profile,
1225                                     const Gpt2ExecutionOptions& ↪
    ↪ options,
1226                                     detail::Gpt2ParallelWorkerPool↪
    ↪ * worker_pool) {
1227     const Gpt2ProfileClock::time_point step_start = profile_start(↪
    ↪ hotspot_profile);
1228     if (hotspot_profile != nullptr) {
1229         ++hotspot_profile->decoder_steps;
1230     }
1231     if (token_id < 0 || token_id >= model.config.vocab_size) {
1232         throw std::runtime_error("GPT-2 token id out of range");
1233     }
1234     const std::int32_t model_position = cache.filled_tokens();
1235     if (model_position >= model.config.max_positions) {
1236         throw std::runtime_error("GPT-2 cached forward would exceed ↪
    ↪ max_positions");
1237     }
1238
1239     std::span<float> hidden = workspace.hidden();
1240     const std::span<const float> token =
1241         row_span(model.token_embedding, token_id, model.config.hidden_size);
1242     const std::span<const float> position_embedding =
1243         row_span(model.position_embedding, model_position, model.config.↪
    ↪ hidden_size);
1244     Gpt2ProfileClock::time_point start = profile_start(hotspot_profile);
1245     for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
1246         hidden[checked_size(dim, "dim")] =
1247             token[checked_size(dim, "dim")] + position_embedding[checked_size(↪
    ↪ dim, "dim")];
1248     }
1249     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::embedding_ms, start);
1250
1251     for (std::int32_t layer_id = 0;
1252         layer_id < static_cast<std::int32_t>(model.layers.size());
1253         ++layer_id) {
1254         forward_gpt2_layer_cached(model.config,
1255             model.layers[checked_size(layer_id, "layer_id"↪
    ↪ ")],
1256             layer_id,
1257             model_position,
1258             cache,
1259             workspace,
1260             hidden,
1261             hotspot_profile,
1262             options,
1263             worker_pool);
1264     }
1265
1266     Gpt2ForwardResult result;
1267     if (mode == Gpt2CachedStepMode::advance_only) {
1268         add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::total_step_ms, ↪

```

```

1269     ↪ step_start);
1270     return result;
1271 }
1272
1273 std::span<float> normed = workspace.normed();
1274 start = profile_start(hotspot_profile);
1275 layer_norm(hidden,
1276             model.final_ln_weight,
1277             model.final_ln_bias,
1278             model.config.layer_norm_epsilon,
1279             normed);
1280 add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::final_norm_ms, start);
1281
1282 if (mode == Gpt2CachedStepMode::logits) {
1283     std::span<float> logits = workspace.logits();
1284     start = profile_start(hotspot_profile);
1285     compute_logits(model, normed, logits, options, worker_pool);
1286     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start)↪
1287     ↪ ;
1288     start = profile_start(hotspot_profile);
1289     result.logits.assign(logits.begin(), logits.end());
1290     result.predicted_token = argmax(result.logits);
1291     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::logits_result_ms, ↪
1292     ↪ start);
1293 } else {
1294     start = profile_start(hotspot_profile);
1295     result.predicted_token = compute_predicted_token(model, normed, options↪
1296     ↪ , worker_pool);
1297     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start)↪
1298     ↪ ;
1299 }
1300 add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::total_step_ms, ↪
1301 ↪ step_start);
1302 return result;
1303 }
1304
1305 std::vector<std::string> gpt2_bytes_to_unicode() {
1306     std::vector<std::int32_t> bytes;
1307     for (std::int32_t value = LCQI_GPT2_ASCII_EXCLAMATION;
1308          value <= LCQI_GPT2_ASCII_TILDE;
1309          ++value) {
1310         bytes.push_back(value);
1311     }
1312     for (std::int32_t value = LCQI_GPT2_LATIN_INVERTED_EXCLAMATION;
1313          value <= LCQI_GPT2_LATIN_NOT_SIGN;
1314          ++value) {
1315         bytes.push_back(value);
1316     }
1317     for (std::int32_t value = LCQI_GPT2_LATIN_REGISTERED;
1318          value <= LCQI_GPT2_LATIN_Y_DIAERESIS;
1319          ++value) {
1320         bytes.push_back(value);
1321     }
1322 }

```



```

1315     }
1316
1317     std::unordered_set<std::int32_t> byte_set(bytes.begin(), bytes.end());
1318     std::vector<std::int32_t> chars = bytes;
1319     std::int32_t private_offset = 0;
1320     for (std::int32_t byte = 0; byte < LCQI_GPT2_BYTE_COUNT; ++byte) {
1321         if (!byte_set.contains(byte)) {
1322             bytes.push_back(byte);
1323             chars.push_back(LCQI_GPT2_UNICODE_PRIVATE_BASE + private_offset);
1324             ++private_offset;
1325         }
1326     }
1327
1328     std::vector<std::string> mapping(LCQI_GPT2_BYTE_COUNT);
1329     for (std::size_t index = 0; index < bytes.size(); ++index) {
1330         mapping[checked_size(bytes[index], "byte unicode index")] =
1331             encode_codepoint(static_cast<std::uint32_t>(chars[index]));
1332     }
1333     return mapping;
1334 }
1335
1336 std::unordered_map<std::string, unsigned char> gpt2_unicode_to_bytes(
1337     const std::vector<std::string>& byte_encoder) {
1338     std::unordered_map<std::string, unsigned char> result;
1339     for (std::size_t byte = 0; byte < byte_encoder.size(); ++byte) {
1340         result.emplace(byte_encoder[byte], static_cast<unsigned char>(byte));
1341     }
1342     return result;
1343 }
1344
1345 std::string bytes_to_bpe_alphabet(std::string_view text) {
1346     static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode(
1347         ↵ ↵ ();
1348     std::string result;
1349     for (const unsigned char byte : text) {
1350         result += BYTE_ENCODER[byte];
1351     }
1352     return result;
1353 }
1354
1355 std::vector<std::string> utf8_symbols(std::string_view text) {
1356     std::vector<std::string> symbols;
1357     std::size_t offset = 0;
1358     while (offset < text.size()) {
1359         const unsigned char lead = static_cast<unsigned char>(text[offset]);
1360         std::size_t width = 1;
1361         if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
1362             width = 2;
1363         } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
1364             width = 3;
1365         } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
1366             width = 4;

```

```

1366     }
1367     if (offset + width > text.size()) {
1368         throw std::runtime_error("invalid UTF-8/BPE symbol boundary");
1369     }
1370     symbols.emplace_back(text.substr(offset, width));
1371     offset += width;
1372 }
1373 return symbols;
1374 }
1375
1376 std::string pair_key(std::string_view left, std::string_view right) {
1377     std::string key(left);
1378     key.push_back('\n');
1379     key += right;
1380     return key;
1381 }
1382
1383 std::vector<std::string> bpe_apply(const Gpt2Tokenizer& tokenizer, std::string_view token) {
1384     std::vector<std::string> word = utf8_symbols(token);
1385     if (word.size() <= 1) {
1386         return word;
1387     }
1388
1389     while (true) {
1390         std::optional<std::int32_t> best_rank;
1391         std::string best_left;
1392         std::string best_right;
1393         for (std::size_t index = 0; index + 1 < word.size(); ++index) {
1394             const auto rank = tokenizer.bpe_ranks.find(pair_key(word[index],
1395                 word[index + 1]));
1396             if (rank != tokenizer.bpe_ranks.end() &&
1397                 (!best_rank.has_value() || rank->second < *best_rank)) {
1398                 best_rank = rank->second;
1399                 best_left = word[index];
1400                 best_right = word[index + 1];
1401             }
1402         }
1403         if (!best_rank.has_value()) {
1404             return word;
1405         }
1406
1407         std::vector<std::string> merged;
1408         std::size_t index = 0;
1409         while (index < word.size()) {
1410             if (index + 1 < word.size() && word[index] == best_left &&
1411                 word[index + 1] == best_right) {
1412                 merged.push_back(best_left + best_right);
1413                 index += LCQI_GPT2_PAIR_TOKEN_COUNT;
1414             } else {
1415                 merged.push_back(word[index]);
1416                 ++index;
1417             }
1418         }
1419     }

```

```

1416     }
1417 }
1418 word = std::move(merged);
1419 if (word.size() == 1) {
1420     return word;
1421 }
1422 }
1423 }
1424
1425 bool is_ascii_letter(unsigned char value) {
1426     return (value >= 'A' && value <= 'Z') || (value >= 'a' && value <= 'z');
1427 }
1428
1429 bool is_ascii_digit(unsigned char value) {
1430     return value >= '0' && value <= '9';
1431 }
1432
1433 bool is_ascii_space(unsigned char value) {
1434     return value == ' ' || value == '\t' || value == '\n' ||
1435            value == '\r' || value == '\f' || value == '\v';
1436 }
1437
1438 bool is_gpt2_punctuation(unsigned char value) {
1439     return !is_ascii_letter(value) && !is_ascii_digit(value) &&
1440            !is_ascii_space(value);
1441 }
1442
1443 std::size_t consume_utf8_codepoint(std::string_view text, std::size_t offset) {
1444     const unsigned char lead = static_cast<unsigned char>(text[offset]);
1445     std::size_t width = 1;
1446     if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
1447         width = 2;
1448     } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
1449         width = 3;
1450     } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
1451         width = 4;
1452     }
1453     if (offset + width > text.size()) {
1454         throw std::runtime_error("invalid UTF-8 text for GPT-2 tokenizer");
1455     }
1456     return offset + width;
1457 }
1458
1459 bool starts_with_contraction(std::string_view text,
1460                             std::size_t offset,
1461                             std::string_view contraction) {
1462     if (offset + contraction.size() > text.size()) {
1463         return false;
1464     }
1465     for (std::size_t index = 0; index < contraction.size(); ++index) {
1466         char left = text[offset + index];
1467         const char right = contraction[index];

```

```

1468     if (left >= 'A' && left <= 'Z') {
1469         left = static_cast<char>(left - 'A' + 'a');
1470     }
1471     if (left != right) {
1472         return false;
1473     }
1474 }
1475 return true;
1476 }
1477
1478 std::vector<std::string> gpt2_pretokenize(std::string_view text) {
1479     std::vector<std::string> pieces;
1480     std::size_t offset = 0;
1481     const std::array<std::string_view, 7> contractions{
1482         "s", "t", "re", "ve", "m", "ll", "d",
1483     };
1484     while (offset < text.size()) {
1485         bool emitted_contraction = false;
1486         for (const std::string_view contraction : contractions) {
1487             if (starts_with_contraction(text, offset, contraction)) {
1488                 pieces.emplace_back(text.substr(offset, contraction.size()));
1489                 offset += contraction.size();
1490                 emitted_contraction = true;
1491                 break;
1492             }
1493         }
1494         if (emitted_contraction) {
1495             continue;
1496         }
1497
1498         const std::size_t begin = offset;
1499         bool has_prefix_space = false;
1500         if (text[offset] == ' ' && offset + 1 < text.size() &&
1501             !is_ascii_space(static_cast<unsigned char>(text[offset + 1]))) {
1502             has_prefix_space = true;
1503             ++offset;
1504         }
1505
1506         if (offset >= text.size()) {
1507             pieces.emplace_back(text.substr(begin, offset - begin));
1508             continue;
1509         }
1510
1511         const unsigned char ch = static_cast<unsigned char>(text[offset]);
1512         if (is_ascii_letter(ch)) {
1513             while (offset < text.size() &&
1514                 is_ascii_letter(static_cast<unsigned char>(text[offset]))) {
1515                 ++offset;
1516             }
1517         } else if (is_ascii_digit(ch)) {
1518             while (offset < text.size() &&
1519                 is_ascii_digit(static_cast<unsigned char>(text[offset]))) {

```

```

1520         ++offset;
1521     }
1522     } else if (is_gpt2_punctuation(ch) && ch != LCQI_GPT2_ASCII_APOSTROPHE) {
1523         while (offset < text.size() &&
1524             is_gpt2_punctuation(static_cast<unsigned char>(text[offset]))
1525             ) &&
1526             static_cast<unsigned char>(text[offset]) !=
1527             LCQI_GPT2_ASCII_APOSTROPHE) {
1528             offset = consume_utf8_codepoint(text, offset);
1529         }
1530     } else if (is_ascii_space(ch)) {
1531         while (offset < text.size() &&
1532             is_ascii_space(static_cast<unsigned char>(text[offset]))) {
1533             ++offset;
1534         }
1535     } else {
1536         offset = consume_utf8_codepoint(text, offset);
1537     }
1538     if (has_prefix_space || offset > begin) {
1539         pieces.emplace_back(text.substr(begin, offset - begin));
1540     } else {
1541         ++offset;
1542     }
1543     return pieces;
1544 }
1545
1546 std::int32_t parse_i32_field(JsonCursor& cursor) {
1547     const std::int64_t value = cursor.parse_i64();
1548     if (value < std::numeric_limits<std::int32_t>::min() ||
1549         value > std::numeric_limits<std::int32_t>::max()) {
1550         throw std::runtime_error("JSON int32 field out of range");
1551     }
1552     return static_cast<std::int32_t>(value);
1553 }
1554
1555 std::unordered_map<std::string, std::int32_t> parse_vocab_json(std::string_view
1556     text) {
1557     JsonCursor cursor(text);
1558     std::unordered_map<std::string, std::int32_t> vocab;
1559     cursor.expect('{');
1560     if (cursor.consume('}')) {
1561         return vocab;
1562     }
1563     while (true) {
1564         const std::string key = cursor.parse_string();
1565         cursor.expect(':');
1566         const std::int32_t id = parse_i32_field(cursor);
1567         vocab.emplace(key, id);
1568         if (cursor.consume('}')) {

```

```

1568         break;
1569     }
1570     cursor.expect(',');
1571 }
1572 if (!cursor.eof_after_ws()) {
1573     throw std::runtime_error("trailing data after vocab JSON");
1574 }
1575 return vocab;
1576 }
1577
1578 std::vector<std::string> build_id_to_token(
1579     const std::unordered_map<std::string, std::int32_t>& vocab) {
1580     std::int32_t max_id = -1;
1581     for (const auto& [token, id] : vocab) {
1582         if (id < 0) {
1583             throw std::runtime_error("GPT-2 vocab id cannot be negative");
1584         }
1585         max_id = std::max(max_id, id);
1586     }
1587     std::vector<std::string> result(check_size(max_id + 1, "max vocab id"));
1588     for (const auto& [token, id] : vocab) {
1589         std::string& slot = result[checked_size(id, "vocab id")];
1590         if (!slot.empty()) {
1591             throw std::runtime_error("duplicate GPT-2 vocab id");
1592         }
1593         slot = token;
1594     }
1595     return result;
1596 }
1597
1598 std::unordered_map<std::string, std::int32_t> parse_merges(std::string_view ↵
    ↵ text) {
1599     std::unordered_map<std::string, std::int32_t> ranks;
1600     std::istringstream input{std::string(text)};
1601     std::string line;
1602     std::int32_t rank = 0;
1603     while (std::getline(input, line)) {
1604         if (line.empty() || line[0] == '#') {
1605             continue;
1606         }
1607         std::istringstream line_stream(line);
1608         std::string left;
1609         std::string right;
1610         if (!(line_stream >> left >> right)) {
1611             throw std::runtime_error("invalid GPT-2 merges line");
1612         }
1613         ranks.emplace(pair_key(left, right), rank);
1614         ++rank;
1615     }
1616     return ranks;
1617 }
1618

```

```

1619 void transpose_hf_conv1d(std::vector<float>& values,
1620                          std::int32_t input_size,
1621                          std::int32_t output_size) {
1622     if (values.size() != matrix_size(input_size, output_size)) {
1623         throw std::runtime_error("GPT-2 Conv1D tensor size mismatch");
1624     }
1625     std::vector<float> transposed(matrix_size(output_size, input_size), 0.0F);
1626     for (std::int32_t input = 0; input < input_size; ++input) {
1627         for (std::int32_t output = 0; output < output_size; ++output) {
1628             transposed[matrix_size(output, input_size) + checked_size(input, "↵
↵ input")] =
1629                 values[matrix_size(input, output_size) + checked_size(output, "↵
↵ output")];
1630         }
1631     }
1632     values = std::move(transposed);
1633 }
1634
1635 std::vector<float> require_tensor(const SafeTensorFile& file,
1636                                  std::string_view name,
1637                                  std::span<const std::int64_t> expected_shape)↵
↵ {
1638     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
1639     if (entry == nullptr) {
1640         throw std::runtime_error("missing GPT-2 tensor: " + std::string(name));
1641     }
1642     if (entry->shape.size() != expected_shape.size()) {
1643         throw std::runtime_error("GPT-2 tensor rank mismatch: " + std::string(↵
↵ name));
1644     }
1645     for (std::size_t index = 0; index < expected_shape.size(); ++index) {
1646         if (entry->shape[index] != expected_shape[index]) {
1647             throw std::runtime_error("GPT-2 tensor shape mismatch: " + std::↵
↵ string(name));
1648         }
1649     }
1650     return read_safetensor_f32_tensor(file, name);
1651 }
1652
1653 std::vector<float> require_first_tensor(const SafeTensorFile& file,
1654                                         std::span<const std::string_view> names↵
↵ ,
1655                                         std::span<const std::int64_t> ↵
↵ expected_shape) {
1656     for (const std::string_view name : names) {
1657         if (file.manifest.find_tensor(name) != nullptr) {
1658             return require_tensor(file, name, expected_shape);
1659         }
1660     }
1661     throw std::runtime_error("missing GPT-2 tensor: " + std::string(names.front↵
↵ ()));
1662 }

```

```

1663
1664 Gpt2LinearF32 make_linear(std::int32_t input_size,
1665                             std::int32_t output_size,
1666                             std::vector<float> weights,
1667                             std::vector<float> bias = {}) {
1668     Gpt2LinearF32 linear;
1669     linear.input_size = input_size;
1670     linear.output_size = output_size;
1671     linear.weights = std::move(weights);
1672     linear.bias = std::move(bias);
1673     validate_linear(linear, "make GPT-2 linear");
1674     return linear;
1675 }
1676
1677 std::vector<float> zeros(std::int32_t count) {
1678     return std::vector<float>(checked_size(count, "zero count"), 0.0F);
1679 }
1680
1681 std::vector<float> ones(std::int32_t count) {
1682     return std::vector<float>(checked_size(count, "one count"), 1.0F);
1683 }
1684
1685 std::string tensor_name(std::int32_t layer, std::string_view suffix) {
1686     return "transformer.h." + std::to_string(layer) + "." + std::string(suffix) ←
    ↪ ;
1687 }
1688
1689 std::string bare_tensor_name(std::int32_t layer, std::string_view suffix) {
1690     return "h." + std::to_string(layer) + "." + std::string(suffix);
1691 }
1692
1693 std::array<std::string_view, 2> embedding_names(std::string_view bare) {
1694     if (bare == "wte.weight") {
1695         return {"transformer.wte.weight", "wte.weight"};
1696     }
1697     if (bare == "wpe.weight") {
1698         return {"transformer.wpe.weight", "wpe.weight"};
1699     }
1700     if (bare == "ln_f.weight") {
1701         return {"transformer.ln_f.weight", "ln_f.weight"};
1702     }
1703     if (bare == "ln_f.bias") {
1704         return {"transformer.ln_f.bias", "ln_f.bias"};
1705     }
1706     throw std::runtime_error("unsupported GPT-2 embedding tensor alias");
1707 }
1708
1709 std::array<std::string, 2> layer_names(std::int32_t layer, std::string_view ←
    ↪ suffix) {
1710     return {tensor_name(layer, suffix), bare_tensor_name(layer, suffix)};
1711 }
1712

```



```

1713 std::vector<float> require_layer_tensor(const SafeTensorFile& file,
1714                                         std::int32_t layer,
1715                                         std::string_view suffix,
1716                                         std::span<const std::int64_t> ↵
        ↵ expected_shape) {
1717     const std::array<std::string, 2> names = layer_names(layer, suffix);
1718     const std::array<std::string_view, 2> views{names[0], names[1]};
1719     return require_first_tensor(file, views, expected_shape);
1720 }
1721
1722 std::filesystem::path find_gpt2_weight_file(const std::filesystem::path& ↵
        ↵ model_directory) {
1723     const std::array<const char*, 2> candidates{
1724         "model.safetensors",
1725         "pytorch_model.safetensors",
1726     };
1727     for (const char* candidate : candidates) {
1728         const std::filesystem::path path = model_directory / candidate;
1729         if (std::filesystem::exists(path)) {
1730             return path;
1731         }
1732     }
1733     throw std::runtime_error("cannot find model.safetensors in GPT-2 directory" ↵
        ↵ );
1734 }
1735
1736 } // namespace
1737
1738 Gpt2Tokenizer load_gpt2_tokenizer(const std::filesystem::path& vocab_json,
1739                                   const std::filesystem::path& merges_txt,
1740                                   std::int32_t bos_token_id,
1741                                   std::int32_t eos_token_id) {
1742     Gpt2Tokenizer tokenizer;
1743     tokenizer.bos_token_id = bos_token_id;
1744     tokenizer.eos_token_id = eos_token_id;
1745     tokenizer.vocab = parse_vocab_json(read_text_file(vocab_json));
1746     tokenizer.id_to_token = build_id_to_token(tokenizer.vocab);
1747     tokenizer.bpe_ranks = parse_merges(read_text_file(merges_txt));
1748     if (tokenizer.vocab.empty() || tokenizer.bpe_ranks.empty()) {
1749         throw std::runtime_error("GPT-2 tokenizer assets are empty");
1750     }
1751     return tokenizer;
1752 }
1753
1754 std::vector<std::int32_t> gpt2_encode(const Gpt2Tokenizer& tokenizer,
1755                                       std::string_view text) {
1756     std::vector<std::int32_t> ids;
1757     for (const std::string& piece : gpt2_pretokenize(text)) {
1758         const std::string byte_piece = bytes_to_bpe_alphabet(piece);
1759         const std::vector<std::string> bpe_tokens = bpe_apply(tokenizer, ↵
        ↵ byte_piece);
1760         for (const std::string& token : bpe_tokens) {

```

```

1761         const auto found = tokenizer.vocab.find(token);
1762         if (found == tokenizer.vocab.end()) {
1763             throw std::runtime_error("GPT-2 BPE token is missing from vocab");
1764         }
1765         ids.push_back(found->second);
1766     }
1767 }
1768 return ids;
1769 }
1770
1771 std::string gpt2_decode(const Gpt2Tokenizer& tokenizer,
1772                        std::span<const std::int32_t> token_ids) {
1773     static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode();
1774     static const std::unordered_map<std::string, unsigned char> BYTE_DECODER =
1775         gpt2_unicode_to_bytes(BYTE_ENCODER);
1776
1777     std::string token_text;
1778     for (const std::int32_t token_id : token_ids) {
1779         if (token_id < 0 || token_id >= static_cast<std::int32_t>(tokenizer.id_to_token.size())) {
1780             throw std::runtime_error("GPT-2 token id out of range");
1781         }
1782         token_text += tokenizer.id_to_token[checked_size(token_id, "token id")];
1783     }
1784
1785     std::string result;
1786     const std::vector<std::string> symbols = utf8_symbols(token_text);
1787     for (const std::string& symbol : symbols) {
1788         const auto found = BYTE_DECODER.find(symbol);
1789         if (found == BYTE_DECODER.end()) {
1790             throw std::runtime_error("GPT-2 token cannot be decoded to byte");
1791         }
1792         result.push_back(static_cast<char>(found->second));
1793     }
1794     return result;
1795 }
1796
1797 Gpt2Config load_gpt2_config(const std::filesystem::path& config_json) {
1798     const std::string config_text = read_text_file(config_json);
1799     JsonCursor cursor(config_text);
1800     Gpt2Config config;
1801     config.bos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
1802     config.eos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
1803
1804     cursor.expect('{');
1805     if (cursor.consume('}')) {
1806         throw std::runtime_error("GPT-2 config is empty");
1807     }
1808     while (true) {

```

```

1809     const std::string key = cursor.parse_string();
1810     cursor.expect(':');
1811     if (key == "n_embd") {
1812         config.hidden_size = parse_i32_field(cursor);
1813     } else if (key == "n_head") {
1814         config.head_count = parse_i32_field(cursor);
1815     } else if (key == "n_layer") {
1816         config.layer_count = parse_i32_field(cursor);
1817     } else if (key == "n_positions" || key == "n_ctx") {
1818         const std::int32_t value = parse_i32_field(cursor);
1819         config.max_positions = std::max(config.max_positions, value);
1820     } else if (key == "vocab_size") {
1821         config.vocab_size = parse_i32_field(cursor);
1822     } else if (key == "n_inner") {
1823         config.intermediate_size = parse_i32_field(cursor);
1824     } else if (key == "layer_norm_epsilon") {
1825         config.layer_norm_epsilon = static_cast<float>(cursor.parse_number()
↪ ());
1826     } else if (key == "activation_function") {
1827         config.activation_function = cursor.parse_string();
1828     } else if (key == "bos_token_id") {
1829         config.bos_token_id = parse_i32_field(cursor);
1830     } else if (key == "eos_token_id") {
1831         config.eos_token_id = parse_i32_field(cursor);
1832     } else {
1833         cursor.skip_value();
1834     }
1835     if (cursor.consume('}')) {
1836         break;
1837     }
1838     cursor.expect(',');
1839 }
1840 if (!cursor.eof_after_ws()) {
1841     throw std::runtime_error("trailing data after GPT-2 config JSON");
1842 }
1843 if (config.intermediate_size == 0 && config.hidden_size > 0) {
1844     config.intermediate_size = config.hidden_size * 4;
1845 }
1846 validate_config(config);
1847 return config;
1848 }
1849
1850 Gpt2ReferenceModel load_gpt2_reference_model(const std::filesystem::path& ↪
↪ config_json,
1851                                             const std::filesystem::path& ↪
↪ safetensors_path) {
1852     Gpt2ReferenceModel model;
1853     model.config = load_gpt2_config(config_json);
1854     const SafeTensorFile file = load_safetensors_file(safetensors_path);
1855     const Gpt2Config& config = model.config;
1856     const std::array<std::int64_t, 2> wte_shape{config.vocab_size, config.↪
↪ hidden_size};

```

```

1857     const std::array<std::int64_t, 2> wpe_shape{config.max_positions, config.↵
↵ hidden_size};
1858     const std::array<std::int64_t, 1> hidden_shape{config.hidden_size};
1859
1860     const std::array<std::string_view, 2> wte_names = embedding_names("wte.↵
↵ weight");
1861     const std::array<std::string_view, 2> wpe_names = embedding_names("wpe.↵
↵ weight");
1862     model.token_embedding = require_first_tensor(file, wte_names, wte_shape);
1863     model.position_embedding = require_first_tensor(file, wpe_names, wpe_shape)↵
↵ ;
1864     model.layers.resize(check_size(config.layer_count, "layer_count"));
1865
1866     for (std::int32_t layer_id = 0; layer_id < config.layer_count; ++layer_id) ↵
↵ {
1867         Gpt2LayerWeightsF32& layer = model.layers[check_size(layer_id, "↵
↵ layer_id")];
1868         layer.ln_1_weight = require_layer_tensor(file, layer_id, "ln_1.weight",↵
↵ hidden_shape);
1869         layer.ln_1_bias = require_layer_tensor(file, layer_id, "ln_1.bias", ↵
↵ hidden_shape);
1870         layer.ln_2_weight = require_layer_tensor(file, layer_id, "ln_2.weight",↵
↵ hidden_shape);
1871         layer.ln_2_bias = require_layer_tensor(file, layer_id, "ln_2.bias", ↵
↵ hidden_shape);
1872
1873         const std::array<std::int64_t, 2> c_attn_shape{
1874             config.hidden_size,
1875             config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
1876         std::vector<float> c_attn_weight =
1877             require_layer_tensor(file, layer_id, "attn.c_attn.weight", ↵
↵ c_attn_shape);
1878         transpose_hf_conv1d(c_attn_weight,
1879                             config.hidden_size,
1880                             config.hidden_size * ↵
↵ LCQI_GPT2_ATTENTION_PROJECTIONS);
1881         const std::array<std::int64_t, 1> c_attn_bias_shape{
1882             config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
1883         layer.c_attn = make_linear(config.hidden_size,
1884                                   config.hidden_size * ↵
↵ LCQI_GPT2_ATTENTION_PROJECTIONS,
1885                                   std::move(c_attn_weight),
1886                                   require_layer_tensor(file,
1887                                                         layer_id,
1888                                                         "attn.c_attn.bias",
1889                                                         c_attn_bias_shape));
1890
1891         const std::array<std::int64_t, 2> c_proj_shape{config.hidden_size, ↵
↵ config.hidden_size};
1892         std::vector<float> c_proj_weight =
1893             require_layer_tensor(file, layer_id, "attn.c_proj.weight", ↵
↵ c_proj_shape);

```

```

1894     transpose_hf_conv1d(c_proj_weight, config.hidden_size, config.↵
↵ hidden_size);
1895     layer.c_proj = make_linear(config.hidden_size,
1896                               config.hidden_size,
1897                               std::move(c_proj_weight),
1898                               require_layer_tensor(file,
1899                                                    layer_id,
1900                                                    "attn.c_proj.bias",
1901                                                    hidden_shape));
1902
1903     const std::array<std::int64_t, 2> c_fc_shape{
1904         config.hidden_size,
1905         config.intermediate_size};
1906     std::vector<float> c_fc_weight =
1907         require_layer_tensor(file, layer_id, "mlp.c_fc.weight", c_fc_shape)↵
↵ ;
1908     transpose_hf_conv1d(c_fc_weight, config.hidden_size, config.↵
↵ intermediate_size);
1909     const std::array<std::int64_t, 1> intermediate_shape{config.↵
↵ intermediate_size};
1910     layer.c_fc = make_linear(config.hidden_size,
1911                              config.intermediate_size,
1912                              std::move(c_fc_weight),
1913                              require_layer_tensor(file,
1914                                                     layer_id,
1915                                                     "mlp.c_fc.bias",
1916                                                     intermediate_shape));
1917
1918     const std::array<std::int64_t, 2> mlp_proj_shape{
1919         config.intermediate_size,
1920         config.hidden_size};
1921     std::vector<float> mlp_proj_weight =
1922         require_layer_tensor(file, layer_id, "mlp.c_proj.weight", ↵
↵ mlp_proj_shape);
1923     transpose_hf_conv1d(mlp_proj_weight, config.intermediate_size, config.↵
↵ hidden_size);
1924     layer.mlp_c_proj = make_linear(config.intermediate_size,
1925                                    config.hidden_size,
1926                                    std::move(mlp_proj_weight),
1927                                    require_layer_tensor(file,
1928                                                         layer_id,
1929                                                         "mlp.c_proj.bias",
1930                                                         hidden_shape));
1931 }
1932
1933 const std::array<std::string_view, 2> ln_weight_names = embedding_names("↵
↵ ln_f.weight");
1934 const std::array<std::string_view, 2> ln_bias_names = embedding_names("ln_f↵
↵ .bias");
1935 model.final_ln_weight = require_first_tensor(file, ln_weight_names, ↵
↵ hidden_shape);
1936 model.final_ln_bias = require_first_tensor(file, ln_bias_names, ↵

```

```

    ↪ hidden_shape);
1937     if (file.manifest.find_tensor("lm_head.weight") != nullptr) {
1938         model.tie_lm_head_to_embedding = false;
1939         model.lm_head_weight = require_tensor(file, "lm_head.weight", wte_shape ↪
    ↪ );
1940     } else {
1941         model.tie_lm_head_to_embedding = true;
1942     }
1943     validate_model(model);
1944     return model;
1945 }
1946
1947 Gpt2ReferenceModel load_gpt2_from_directory(const std::filesystem::path& ↪
    ↪ model_directory) {
1948     return load_gpt2_reference_model(model_directory / "config.json",
1949                                     find_gpt2_weight_file(model_directory));
1950 }
1951
1952 Gpt2ForwardWorkspace::Gpt2ForwardWorkspace(const Gpt2Config& config) : config_( ↪
    ↪ config) {
1953     this->reset_for_config(config);
1954 }
1955
1956 void Gpt2ForwardWorkspace::reset_for_config(const Gpt2Config& config) {
1957     validate_config(config);
1958     this->config_ = config;
1959     const std::size_t hidden_size = checked_size(this->config_.hidden_size, " ↪
    ↪ hidden_size");
1960     const std::size_t intermediate_size =
1961         checked_size(this->config_.intermediate_size, "intermediate_size");
1962     this->hidden_.assign(hidden_size, 0.0F);
1963     this->normed_.assign(hidden_size, 0.0F);
1964     this->query_.assign(hidden_size, 0.0F);
1965     this->key_.assign(hidden_size, 0.0F);
1966     this->value_.assign(hidden_size, 0.0F);
1967     this->attention_.assign(hidden_size, 0.0F);
1968     this->projected_.assign(hidden_size, 0.0F);
1969     this->qkv_packed_.assign(
1970         matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS, this->config_.hidden_size) ↪
    ↪ ,
1971         0.0F);
1972     this->mlp_fc_.assign(intermediate_size, 0.0F);
1973     this->mlp_out_.assign(hidden_size, 0.0F);
1974     this->logits_.assign(checked_size(this->config_.vocab_size, "vocab_size"), ↪
    ↪ 0.0F);
1975     this->scores_.assign(checked_size(this->config_.max_positions, " ↪
    ↪ max_positions"), 0.0F);
1976 }
1977
1978 std::span<float> Gpt2ForwardWorkspace::hidden() noexcept {
1979     return this->hidden_;
1980 }

```

```

1981
1982 std::span<float> Gpt2ForwardWorkspace::normed() noexcept {
1983     return this->normed_;
1984 }
1985
1986 std::span<float> Gpt2ForwardWorkspace::query() noexcept {
1987     return this->query_;
1988 }
1989
1990 std::span<float> Gpt2ForwardWorkspace::key() noexcept {
1991     return this->key_;
1992 }
1993
1994 std::span<float> Gpt2ForwardWorkspace::value() noexcept {
1995     return this->value_;
1996 }
1997
1998 std::span<float> Gpt2ForwardWorkspace::attention() noexcept {
1999     return this->attention_;
2000 }
2001
2002 std::span<float> Gpt2ForwardWorkspace::projected() noexcept {
2003     return this->projected_;
2004 }
2005
2006 std::span<float> Gpt2ForwardWorkspace::qkv_packed() noexcept {
2007     return this->qkv_packed_;
2008 }
2009
2010 std::span<float> Gpt2ForwardWorkspace::mlp_fc() noexcept {
2011     return this->mlp_fc_;
2012 }
2013
2014 std::span<float> Gpt2ForwardWorkspace::mlp_out() noexcept {
2015     return this->mlp_out_;
2016 }
2017
2018 std::span<float> Gpt2ForwardWorkspace::logits() noexcept {
2019     return this->logits_;
2020 }
2021
2022 std::span<float> Gpt2ForwardWorkspace::scores_prefix(std::int32_t count) {
2023     if (count < 0 || count > this->config_.max_positions) {
2024         throw std::runtime_error("GPT-2 attention scores count out of range");
2025     }
2026     return std::span<float>(this->scores_.data(), checked_size(count, "scores ←
↪ count"));
2027 }
2028
2029 Gpt2KvCache::Gpt2KvCache(const Gpt2Config& config) : config_(config) {
2030     validate_config(this->config_);
2031     const std::size_t element_count =

```



```

2032     checked_size(this->config_.layer_count, "layer_count") *
2033     checked_size(this->config_.max_positions, "max_positions") *
2034     checked_size(this->config_.head_count, "head_count") *
2035     checked_size(head_dim(this->config_), "head_dim");
2036     this->keys_.assign(element_count, 0.0F);
2037     this->values_.assign(element_count, 0.0F);
2038     this->written_.assign(
2039         checked_size(this->config_.layer_count, "layer_count") *
2040         checked_size(this->config_.max_positions, "max_positions"),
2041         0);
2042 }
2043
2044 void Gpt2KvCache::append(std::int32_t layer_id,
2045                          std::int32_t model_position,
2046                          std::span<const float> key,
2047                          std::span<const float> value) {
2048     const std::size_t hidden_size = checked_size(this->config_.hidden_size, "↵
↵ hidden_size");
2049     if (key.size() != hidden_size || value.size() != hidden_size) {
2050         throw std::runtime_error("GPT-2 KV key/value size mismatch");
2051     }
2052     this->validate_address(layer_id, model_position, 0);
2053     const std::int32_t local_head_dim = head_dim(this->config_);
2054     for (std::int32_t head = 0; head < this->config_.head_count; ++head) {
2055         const std::size_t source =
2056             checked_size(head, "head") * checked_size(local_head_dim, "head_dim↵
↵ ");
2057         const std::size_t target = this->base_offset(layer_id, model_position, ↵
↵ head);
2058         std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
2059                    local_head_dim,
2060                    this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
2061         std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
2062                    local_head_dim,
2063                    this->values_.begin() + static_cast<std::ptrdiff_t>(target)↵
↵ );
2064     }
2065     const std::size_t written_index =
2066         checked_size(layer_id, "layer_id") *
2067         checked_size(this->config_.max_positions, "max_positions") +
2068         checked_size(model_position, "model_position");
2069     this->written_[written_index] = 1;
2070     this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
2071 }
2072
2073 std::span<const float> Gpt2KvCache::key(std::int32_t layer_id,
2074                                         std::int32_t model_position,
2075                                         std::int32_t head) const {
2076     this->validate_address(layer_id, model_position, head);
2077     this->validate_written(layer_id, model_position);
2078     return this->key_unchecked(layer_id, model_position, head);
2079 }

```



```

2080
2081 std::span<const float> Gpt2KvCache::value(std::int32_t layer_id,
2082                                           std::int32_t model_position,
2083                                           std::int32_t head) const {
2084     this->validate_address(layer_id, model_position, head);
2085     this->validate_written(layer_id, model_position);
2086     return this->value_unchecked(layer_id, model_position, head);
2087 }
2088
2089 void detail::attend_cached_position(const Gpt2KvCache& cache,
2090                                    std::int32_t layer_id,
2091                                    std::int32_t model_position,
2092                                    std::span<const float> query,
2093                                    std::span<float> scores,
2094                                    std::span<float> output) {
2095     cache.validate_address(layer_id, model_position, 0);
2096     cache.validate_written(layer_id, model_position);
2097     const std::int32_t local_head_dim = head_dim(cache.config_);
2098     const std::size_t hidden_size = checked_size(cache.config_.hidden_size, "←
    hidden_size");
2099     const std::size_t local_head_dim_size = checked_size(local_head_dim, "←
    head_dim");
2100     const std::size_t model_position_size =
2101         checked_size(model_position, "model_position");
2102     if (query.size() != hidden_size || output.size() != hidden_size ||
2103         scores.size() < model_position_size + 1) {
2104         throw std::runtime_error("GPT-2 cached attention workspace size ←
    mismatch");
2105     }
2106
2107     const float score_scale = 1.0F / std::sqrt(static_cast<float>(<←
    local_head_dim));
2108     std::fill(output.begin(), output.end(), 0.0F);
2109     for (std::int32_t head = 0; head < cache.config_.head_count; ++head) {
2110         const std::size_t head_base =
2111             checked_size(head, "head") * local_head_dim_size;
2112         float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
2113         for (std::size_t past = 0; past <= model_position_size; ++past) {
2114             const std::int32_t past_i32 = static_cast<std::int32_t>(past);
2115             const std::span<const float> key =
2116                 cache.key_unchecked(layer_id, past_i32, head);
2117             float dot = 0.0F;
2118             for (std::size_t dim = 0; dim < local_head_dim_size; ++dim) {
2119                 dot += query[head_base + dim] * key[dim];
2120             }
2121             const float score = dot * score_scale;
2122             scores[past] = score;
2123             max_score = std::max(max_score, score);
2124         }
2125
2126         float denominator = 0.0F;
2127         for (std::size_t past = 0; past <= model_position_size; ++past) {

```

```

2128         scores[past] = std::exp(scores[past] - max_score);
2129         denominator += scores[past];
2130     }
2131     if (denominator <= 0.0F) {
2132         throw std::runtime_error("GPT-2 cached attention denominator is ↵
↵ invalid");
2133     }
2134
2135     for (std::size_t past = 0; past <= model_position_size; ++past) {
2136         const std::int32_t past_i32 = static_cast<std::int32_t>(past);
2137         const float probability = scores[past] / denominator;
2138         const std::span<const float> value =
2139             cache.value_unchecked(layer_id, past_i32, head);
2140         for (std::size_t dim = 0; dim < local_head_dim_size; ++dim) {
2141             output[head_base + dim] += probability * value[dim];
2142         }
2143     }
2144 }
2145 }
2146
2147 std::span<const float> Gpt2KvCache::key_unchecked(std::int32_t layer_id,
2148                                                    std::int32_t model_position,
2149                                                    std::int32_t head) const ↵
↵ noexcept {
2150     const std::size_t offset = this->base_offset_unchecked(layer_id, ↵
↵ model_position, head);
2151     return std::span<const float>(
2152         this->keys_.data() + offset,
2153         static_cast<std::size_t>(head_dim(this->config_)));
2154 }
2155
2156 std::span<const float> Gpt2KvCache::value_unchecked(std::int32_t layer_id,
2157                                                    std::int32_t model_position↵
↵ ,
2158                                                    std::int32_t head) const ↵
↵ noexcept {
2159     const std::size_t offset = this->base_offset_unchecked(layer_id, ↵
↵ model_position, head);
2160     return std::span<const float>(
2161         this->values_.data() + offset,
2162         static_cast<std::size_t>(head_dim(this->config_)));
2163 }
2164
2165 std::int32_t Gpt2KvCache::filled_tokens() const noexcept {
2166     return this->filled_tokens_;
2167 }
2168
2169 std::size_t Gpt2KvCache::byte_size() const noexcept {
2170     return (this->keys_.size() + this->values_.size()) * sizeof(float) +
2171         this->written_.size() * sizeof(std::uint8_t);
2172 }
2173

```

```

2174 std::size_t Gpt2KvCache::base_offset(std::int32_t layer_id,
2175                                     std::int32_t model_position,
2176                                     std::int32_t head) const {
2177     return (((checked_size(layer_id, "layer_id") *
2178              checked_size(this->config_.max_positions, "max_positions")) +
2179             checked_size(model_position, "model_position")) *
2180            checked_size(this->config_.head_count, "head_count") +
2181            checked_size(head, "head")) *
2182            checked_size(head_dim(this->config_), "head_dim"));
2183 }
2184
2185 std::size_t Gpt2KvCache::base_offset_unchecked(std::int32_t layer_id,
2186                                               std::int32_t model_position,
2187                                               std::int32_t head) const ←
2188     ↪ noexcept {
2189     return (((static_cast<std::size_t>(layer_id) *
2190             static_cast<std::size_t>(this->config_.max_positions)) +
2191            static_cast<std::size_t>(model_position)) *
2192            static_cast<std::size_t>(this->config_.head_count) +
2193            static_cast<std::size_t>(head)) *
2194            static_cast<std::size_t>(head_dim(this->config_)));
2195 }
2196
2197 void Gpt2KvCache::validate_address(std::int32_t layer_id,
2198                                   std::int32_t model_position,
2199                                   std::int32_t head) const {
2200     if (layer_id < 0 || layer_id >= this->config_.layer_count) {
2201         throw std::runtime_error("GPT-2 KV layer_id out of range");
2202     }
2203     if (model_position < 0 || model_position >= this->config_.max_positions) {
2204         throw std::runtime_error("GPT-2 KV model_position out of range");
2205     }
2206     if (head < 0 || head >= this->config_.head_count) {
2207         throw std::runtime_error("GPT-2 KV head out of range");
2208     }
2209 }
2210
2211 void Gpt2KvCache::validate_written(std::int32_t layer_id,
2212                                   std::int32_t model_position) const {
2213     const std::size_t written_index =
2214         checked_size(layer_id, "layer_id") *
2215         checked_size(this->config_.max_positions, "max_positions") +
2216         checked_size(model_position, "model_position");
2217     if (written_index >= this->written_.size() || this->written_[written_index] ←
2218         ↪ == 0) {
2219         throw std::runtime_error("GPT-2 KV slot has not been written");
2220     }
2221 }
2222
2223 Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& ←
2224     ↪ model)
2225     : Gpt2CachedGreedyDecoder(model, nullptr) {}

```

```

2223
2224 Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(
2225     const Gpt2ReferenceModel& model,
2226     Gpt2HotspotProfile* hotspot_profile)
2227     : Gpt2CachedGreedyDecoder(model, hotspot_profile, Gpt2ExecutionOptions{}) ←
    ↪ {}
2228
2229 Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(
2230     const Gpt2ReferenceModel& model,
2231     Gpt2HotspotProfile* hotspot_profile,
2232     const Gpt2ExecutionOptions& execution_options)
2233     : model_(&model),
2234       hotspot_profile_(hotspot_profile),
2235       execution_options_(execution_options),
2236       cache_(model.config),
2237       workspace_(model.config),
2238       worker_pool_(make_worker_pool(model, execution_options)) {
2239     validate_model(*this->model_);
2240 }
2241
2242 Gpt2CachedGreedyDecoder::~Gpt2CachedGreedyDecoder() = default;
2243
2244 Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(Gpt2CachedGreedyDecoder&&) ←
    ↪ noexcept =
2245     default;
2246
2247 Gpt2CachedGreedyDecoder& Gpt2CachedGreedyDecoder::operator=(
2248     Gpt2CachedGreedyDecoder&&) noexcept = default;
2249
2250 void Gpt2CachedGreedyDecoder::advance_without_prediction(std::int32_t token_id) ←
    ↪ {
2251     static_cast<void>(run_gpt2_cached_step_unchecked(*this->model_,
2252                                                       this->cache_,
2253                                                       this->workspace_,
2254                                                       token_id,
2255                                                       Gpt2CachedStepMode::↪
    ↪ advance_only,
2256                                                       this->hotspot_profile_,
2257                                                       this->execution_options_,
2258                                                       this->worker_pool_.get()));
2259 }
2260
2261 std::int32_t Gpt2CachedGreedyDecoder::step(std::int32_t token_id) {
2262     return run_gpt2_cached_step_unchecked(*this->model_,
2263                                           this->cache_,
2264                                           this->workspace_,
2265                                           token_id,
2266                                           Gpt2CachedStepMode::predict_token,
2267                                           this->hotspot_profile_,
2268                                           this->execution_options_,
2269                                           this->worker_pool_.get())
2270         .predicted_token;

```

```

2271 }
2272
2273 Gpt2ForwardResult Gpt2CachedGreedyDecoder::step_with_logits(std::int32_t ↵
    ↵ token_id) {
2274     return run_gpt2_cached_step_unchecked(*this->model_,
2275                                           this->cache_,
2276                                           this->workspace_,
2277                                           token_id,
2278                                           Gpt2CachedStepMode::logits,
2279                                           this->hotspot_profile_,
2280                                           this->execution_options_,
2281                                           this->worker_pool_.get());
2282 }
2283
2284 const Gpt2KvCache& Gpt2CachedGreedyDecoder::cache() const noexcept {
2285     return this->cache_;
2286 }
2287
2288 std::int32_t Gpt2CachedGreedyDecoder::filled_tokens() const noexcept {
2289     return this->cache_.filled_tokens();
2290 }
2291
2292 std::size_t Gpt2CachedGreedyDecoder::kv_cache_bytes() const noexcept {
2293     return this->cache_.byte_size();
2294 }
2295
2296 std::int32_t Gpt2CachedGreedyDecoder::worker_count() const noexcept {
2297     return this->worker_pool_ == nullptr ? 1 : this->worker_pool_->worker_count↵
    ↵ ();
2298 }
2299
2300 Gpt2ForwardResult run_gpt2_forward(const Gpt2ReferenceModel& model,
2301                                    std::span<const std::int32_t> token_ids) {
2302     validate_model(model);
2303     if (token_ids.empty()) {
2304         throw std::runtime_error("GPT-2 forward needs at least one token");
2305     }
2306     if (token_ids.size() > checked_size(model.config.max_positions, "↵
    ↵ max_positions")) {
2307         throw std::runtime_error("GPT-2 token_ids exceed max_positions");
2308     }
2309
2310     const std::int32_t sequence_length = static_cast<std::int32_t>(token_ids.↵
    ↵ size());
2311     std::vector<float> hidden_by_pos(matrix_size(sequence_length, model.config.↵
    ↵ hidden_size), 0.0F);
2312     for (std::int32_t position = 0; position < sequence_length; ++position) {
2313         const std::int32_t token_id = token_ids[checked_size(position, "↵
    ↵ position")];
2314         if (token_id < 0 || token_id >= model.config.vocab_size) {
2315             throw std::runtime_error("GPT-2 token id out of range");
2316         }

```

```

2317     const std::span<const float> token =
2318         row_span(model.token_embedding, token_id, model.config.hidden_size)↵
↵ ;
2319     const std::span<const float> position_embedding =
2320         row_span(model.position_embedding, position, model.config.↵
↵ hidden_size);
2321     std::span<float> hidden = std::span<float>(
2322         hidden_by_pos.data() + matrix_size(position, model.config.↵
↵ hidden_size),
2323         checked_size(model.config.hidden_size, "hidden_size"));
2324     for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
2325         hidden[checked_size(dim, "dim")] =
2326             token[checked_size(dim, "dim")] + position_embedding[↵
↵ checked_size(dim, "dim")];
2327     }
2328 }
2329
2330 for (const Gpt2LayerWeightsF32& layer : model.layers) {
2331     forward_gpt2_layer(model.config, layer, sequence_length, hidden_by_pos)↵
↵ ;
2332 }
2333
2334 std::vector<float> normed(checked_size(model.config.hidden_size, "↵
↵ hidden_size"), 0.0F);
2335 const std::span<const float> final_hidden = std::span<const float>(
2336     hidden_by_pos.data() + matrix_size(sequence_length - 1, model.config.↵
↵ hidden_size),
2337     checked_size(model.config.hidden_size, "hidden_size"));
2338 layer_norm(final_hidden,
2339     model.final_ln_weight,
2340     model.final_ln_bias,
2341     model.config.layer_norm_epsilon,
2342     normed);
2343
2344 Gpt2ForwardResult result;
2345 result.logits.assign(checked_size(model.config.vocab_size, "vocab_size"), ↵
↵ 0.0F);
2346 compute_logits(model, normed, result.logits);
2347 result.predicted_token = argmax(result.logits);
2348 return result;
2349 }
2350
2351 Gpt2ForwardResult run_gpt2_forward_cached(const Gpt2ReferenceModel& model,
2352     Gpt2KvCache& cache,
2353     std::int32_t token_id) {
2354     validate_model(model);
2355     Gpt2ForwardWorkspace workspace(model.config);
2356     Gpt2ExecutionOptions options;
2357     options.worker_count = 1;
2358     return run_gpt2_cached_step_unchecked(
2359         model,
2360         cache,

```

```

2361     workspace,
2362     token_id,
2363     Gpt2CachedStepMode::logits,
2364     nullptr,
2365     options,
2366     nullptr);
2367 }
2368
2369 std::vector<std::int32_t> gpt2_generate_greedy(const Gpt2ReferenceModel& model,
2370                                             std::span<const std::int32_t> ←
2371                                             prompt_token_ids,
2372                                             std::int32_t max_new_tokens) {
2373     if (max_new_tokens < 0) {
2374         throw std::runtime_error("max_new_tokens cannot be negative");
2375     }
2376     std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids ←
2377     .end());
2378     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
2379         if (tokens.size() >= checked_size(model.config.max_positions, " ←
2380         max_positions")) {
2381             throw std::runtime_error("GPT-2 generation would exceed ←
2382             max_positions");
2383         }
2384         const Gpt2ForwardResult result = run_gpt2_forward(model, tokens);
2385         tokens.push_back(result.predicted_token);
2386         if (model.config.eos_token_id >= 0 && result.predicted_token == model. ←
2387         config.eos_token_id) {
2388             break;
2389         }
2390     }
2391     return tokens;
2392 }
2393
2394 std::vector<std::int32_t> gpt2_generate_greedy_cached(
2395     const Gpt2ReferenceModel& model,
2396     std::span<const std::int32_t> prompt_token_ids,
2397     std::int32_t max_new_tokens) {
2398     if (prompt_token_ids.empty()) {
2399         throw std::runtime_error("GPT-2 generation needs at least one prompt ←
2400         token");
2401     }
2402     if (max_new_tokens < 0) {
2403         throw std::runtime_error("max_new_tokens cannot be negative");
2404     }
2405     if (prompt_token_ids.size() > checked_size(model.config.max_positions, " ←
2406     max_positions")) {
2407         throw std::runtime_error("GPT-2 prompt exceeds max_positions");
2408     }
2409
2410     Gpt2CachedGreedyDecoder decoder(model);
2411     std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids ←
2412     .end());

```

```

2405     if (max_new_tokens == 0) {
2406         return tokens;
2407     }
2408     for (std::size_t index = 0; index + 1 < prompt_token_ids.size(); ++index) {
2409         decoder.advance_without_prediction(prompt_token_ids[index]);
2410     }
2411     std::int32_t predicted_token = decoder.step(prompt_token_ids.back());
2412     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
2413         if (tokens.size() >= checked_size(model.config.max_positions, "↵
↵ max_positions")) {
2414             throw std::runtime_error("GPT-2 generation would exceed ↵
↵ max_positions");
2415         }
2416         tokens.push_back(predicted_token);
2417         if (model.config.eos_token_id >= 0 && predicted_token == model.config.↵
↵ eos_token_id) {
2418             break;
2419         }
2420         if (step + 1 < max_new_tokens) {
2421             predicted_token = decoder.step(predicted_token);
2422         }
2423     }
2424     return tokens;
2425 }
2426
2427 Gpt2ReferenceModel make_tiny_gpt2_reference_model() {
2428     Gpt2ReferenceModel model;
2429     model.config.hidden_size = 4;
2430     model.config.head_count = 2;
2431     model.config.layer_count = 1;
2432     model.config.max_positions = 8;
2433     model.config.vocab_size = 6;
2434     model.config.intermediate_size = 8;
2435     model.config.bos_token_id = 0;
2436     model.config.eos_token_id = 5;
2437     model.config.layer_norm_epsilon = 1.0e-5F;
2438     model.config.activation_function = "gelu_new";
2439
2440     model.token_embedding = {
2441         0.20F, -0.10F, 0.00F, 0.10F,
2442         -0.10F, 0.30F, 0.20F, -0.20F,
2443         0.05F, 0.10F, -0.30F, 0.25F,
2444         0.40F, -0.20F, 0.15F, 0.05F,
2445         -0.30F, 0.05F, 0.35F, -0.15F,
2446         0.10F, 0.20F, -0.10F, 0.30F,
2447     };
2448     model.position_embedding = {
2449         0.01F, 0.02F, 0.03F, 0.04F,
2450         -0.02F, 0.01F, 0.00F, 0.03F,
2451         0.03F, -0.01F, 0.02F, 0.00F,
2452         0.00F, 0.02F, -0.02F, 0.01F,
2453         0.02F, 0.00F, 0.01F, -0.01F,

```



```

2454     -0.01F, 0.03F, 0.00F, 0.02F,
2455     0.01F, -0.02F, 0.03F, 0.00F,
2456     0.00F, 0.01F, -0.01F, 0.02F,
2457 };
2458
2459 Gpt2LayerWeightsF32 layer;
2460 layer.ln_1_weight = ones(4);
2461 layer.ln_1_bias = zeros(4);
2462 layer.ln_2_weight = {0.9F, 1.1F, 1.0F, 0.95F};
2463 layer.ln_2_bias = {0.01F, -0.02F, 0.00F, 0.03F};
2464 layer.c_attn = make_linear(
2465     4,
2466     12,
2467     {
2468         0.10F, -0.20F, 0.05F, 0.15F,
2469         -0.05F, 0.12F, 0.20F, -0.10F,
2470         0.18F, 0.02F, -0.08F, 0.11F,
2471         -0.12F, 0.06F, 0.14F, 0.04F,
2472         0.07F, 0.16F, -0.09F, 0.03F,
2473         -0.15F, 0.05F, 0.13F, 0.10F,
2474         0.04F, -0.11F, 0.19F, -0.02F,
2475         0.09F, 0.08F, -0.06F, 0.17F,
2476         0.20F, -0.04F, 0.01F, 0.06F,
2477         -0.08F, 0.15F, 0.07F, -0.03F,
2478         0.05F, 0.10F, -0.14F, 0.12F,
2479         0.11F, -0.07F, 0.16F, 0.02F,
2480     },
2481     {
2482         0.01F, -0.02F, 0.00F, 0.03F,
2483         -0.01F, 0.02F, 0.01F, -0.02F,
2484         0.00F, 0.01F, -0.01F, 0.02F,
2485     });
2486 layer.c_proj = make_linear(
2487     4,
2488     4,
2489     {
2490         0.10F, 0.05F, -0.08F, 0.12F,
2491         -0.04F, 0.11F, 0.09F, -0.02F,
2492         0.07F, -0.10F, 0.13F, 0.05F,
2493         0.02F, 0.14F, -0.06F, 0.08F,
2494     },
2495     {0.01F, 0.00F, -0.01F, 0.02F});
2496 layer.c_fc = make_linear(
2497     4,
2498     8,
2499     {
2500         0.20F, -0.10F, 0.05F, 0.03F,
2501         -0.05F, 0.14F, 0.12F, -0.08F,
2502         0.09F, 0.07F, -0.11F, 0.16F,
2503         -0.12F, 0.05F, 0.18F, 0.02F,
2504         0.04F, -0.09F, 0.13F, 0.10F,
2505         0.11F, 0.03F, -0.07F, 0.15F,

```

```

2506         -0.02F, 0.17F, 0.06F, -0.04F,
2507         0.08F, -0.06F, 0.10F, 0.12F,
2508     },
2509     {0.01F, -0.02F, 0.00F, 0.03F, -0.01F, 0.02F, 0.01F, 0.00F});
2510     layer.mlp_c_proj = make_linear(
2511         8,
2512         4,
2513         {
2514             0.10F, -0.08F, 0.06F, 0.12F, -0.04F, 0.07F, 0.03F, 0.11F,
2515             -0.05F, 0.13F, 0.02F, -0.09F, 0.15F, 0.04F, -0.03F, 0.08F,
2516             0.07F, 0.01F, -0.12F, 0.10F, 0.06F, -0.05F, 0.14F, 0.02F,
2517             0.03F, 0.09F, 0.11F, -0.06F, 0.05F, 0.12F, -0.08F, 0.04F,
2518         },
2519         {0.00F, 0.01F, -0.01F, 0.02F});
2520     model.layers.push_back(std::move(layer));
2521     model.final_ln_weight = {1.0F, 0.95F, 1.05F, 1.0F};
2522     model.final_ln_bias = {0.0F, 0.01F, -0.01F, 0.02F};
2523     model.tie_lm_head_to_embedding = true;
2524     validate_model(model);
2525     return model;
2526 }
2527
2528 } // namespace lcqi

```

`gpt2_cli.cpp` 负责把模型目录、prompt、`max_new_tokens` 和 benchmark 选项接到 GPT-2 reference path。无参数时运行 tiny GPT-2 fixture；传入 HuggingFace 模型目录时加载真实资产。

Listing 10.14 src/gpt2_cli.cpp

```

1  #include <lcqi/gpt2_reference.hpp>
2
3  #include <chrono>
4  #include <cstdlib>
5  #include <exception>
6  #include <filesystem>
7  #include <iostream>
8  #include <span>
9  #include <stdexcept>
10 #include <string>
11 #include <string_view>
12 #include <vector>
13
14 namespace {
15
16 constexpr int LCQI_GPT2_TINY_ARGC = 1;
17 constexpr int LCQI_GPT2_MIN_REAL_ARGC = 3;
18 constexpr std::int32_t LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS = 8;
19 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_0 = 1;
20 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_1 = 2;
21 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_2 = 3;
22
23 enum class Gpt2EngineMode {

```

```

24     full_prefix,
25     cached_kv,
26 };
27
28 struct Gpt2CliOptions {
29     Gpt2EngineMode engine = Gpt2EngineMode::cached_kv;
30     bool benchmark = false;
31     bool profile_hotspots = false;
32     std::filesystem::path model_dir;
33     std::string prompt;
34     std::int32_t max_new_tokens = LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS;
35     std::int32_t worker_count = 0;
36 };
37
38 struct Gpt2BenchmarkTimings {
39     double load_ms = 0.0;
40     double tokenize_ms = 0.0;
41     double prefill_ms = 0.0;
42     double decode_ms = 0.0;
43     double generate_ms = 0.0;
44     double total_ms = 0.0;
45     std::size_t prefill_steps = 0;
46     std::size_t decode_steps = 0;
47     std::size_t kv_cache_bytes = 0;
48     std::int32_t worker_count = 1;
49 };
50
51 using Clock = std::chrono::steady_clock;
52
53 void print_ids(std::span<const std::int32_t> ids) {
54     for (std::size_t index = 0; index < ids.size(); ++index) {
55         if (index != 0) {
56             std::cout << ' ';
57         }
58         std::cout << ids[index];
59     }
60 }
61
62 [[nodiscard]] double elapsed_ms(Clock::time_point start, Clock::time_point end)↵
    ↪ {
63     return std::chrono::duration<double, std::milli>(end - start).count();
64 }
65
66 [[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
67     if (value < 0) {
68         throw std::runtime_error(std::string(name) + " cannot be negative");
69     }
70     return static_cast<std::size_t>(value);
71 }
72
73 [[nodiscard]] const char* engine_name(Gpt2EngineMode engine) {
74     switch (engine) {

```

```

75     case Gpt2EngineMode::full_prefix:
76         return "full_prefix";
77     case Gpt2EngineMode::cached_kv:
78         return "cached_kv";
79 }
80 return "unknown";
81 }
82
83 [[nodiscard]] Gpt2EngineMode parse_engine(std::string_view value) {
84     if (value == "full" || value == "full_prefix") {
85         return Gpt2EngineMode::full_prefix;
86     }
87     if (value == "cached" || value == "cached_kv") {
88         return Gpt2EngineMode::cached_kv;
89     }
90     throw std::runtime_error("unknown --engine value, expected cached or full")↵
    ↵ ;
91 }
92
93 [[nodiscard]] std::int32_t parse_non_negative_i32(const std::string& text,
94                                                    const char* name) {
95     const std::int32_t value = static_cast<std::int32_t>(std::stoi(text));
96     if (value < 0) {
97         throw std::runtime_error(std::string(name) + " cannot be negative");
98     }
99     return value;
100 }
101
102 [[nodiscard]] Gpt2CliOptions parse_options(int argc, char** argv) {
103     Gpt2CliOptions options;
104     std::vector<std::string> positional;
105     for (int index = 1; index < argc; ++index) {
106         const std::string arg = argv[index];
107         if (arg == "--benchmark") {
108             options.benchmark = true;
109         } else if (arg == "--profile-hotspots") {
110             options.profile_hotspots = true;
111             options.benchmark = true;
112         } else if (arg == "--engine") {
113             if (index + 1 >= argc) {
114                 throw std::runtime_error("--engine requires a value");
115             }
116             ++index;
117             options.engine = parse_engine(argv[index]);
118         } else if (arg.rfind("--engine=", 0) == 0) {
119             options.engine = parse_engine(std::string_view(arg).substr(std::string(
    ↵ string("--engine=").size()));
120         } else if (arg == "--threads") {
121             if (index + 1 >= argc) {
122                 throw std::runtime_error("--threads requires a value");
123             }
124             ++index;

```

```

125     options.worker_count = parse_non_negative_i32(argv[index], "--↵
↵ threads");
126     } else if (arg.rfind("--threads=", 0) == 0) {
127         options.worker_count =
128             parse_non_negative_i32(arg.substr(std::string("--threads=").↵
↵ size()), "--threads");
129     } else if (arg == "--full") {
130         options.engine = Gpt2EngineMode::full_prefix;
131     } else if (arg == "--cached") {
132         options.engine = Gpt2EngineMode::cached_kv;
133     } else if (!arg.empty() && arg.front() == '-') {
134         throw std::runtime_error("unknown option: " + arg);
135     } else {
136         positional.push_back(arg);
137     }
138 }
139
140 if (positional.size() < 2 || positional.size() > 3) {
141     throw std::runtime_error(
142         "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark↵
↵ ] "
143         "model_dir prompt [max_new_tokens]");
144 }
145 options.model_dir = positional[0];
146 options.prompt = positional[1];
147 if (positional.size() == 3) {
148     options.max_new_tokens = parse_non_negative_i32(positional[2], "↵
↵ max_new_tokens");
149 }
150 return options;
151 }
152
153 [[nodiscard]] std::vector<std::int32_t> generate_full_prefix(
154     const lcqi::Gpt2ReferenceModel& model,
155     std::span<const std::int32_t> prompt_ids,
156     std::int32_t max_new_tokens,
157     Gpt2BenchmarkTimings& timings) {
158     const Clock::time_point generate_start = Clock::now();
159     std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
160     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
161         if (tokens.size() >= checked_size(model.config.max_positions, "↵
↵ max_positions")) {
162             throw std::runtime_error("GPT-2 generation would exceed ↵
↵ max_positions");
163         }
164         const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, ↵
↵ tokens);
165         ++timings.decode_steps;
166         tokens.push_back(result.predicted_token);
167         if (model.config.eos_token_id >= 0 && result.predicted_token == model.↵
↵ config.eos_token_id) {
168             break;

```

```

169     }
170 }
171 const Clock::time_point generate_end = Clock::now();
172 timings.generate_ms = elapsed_ms(generate_start, generate_end);
173 timings.decode_ms = timings.generate_ms;
174 return tokens;
175 }
176
177 [[nodiscard]] std::vector<std::int32_t> generate_cached(
178     const lcqi::Gpt2ReferenceModel& model,
179     std::span<const std::int32_t> prompt_ids,
180     std::int32_t max_new_tokens,
181     Gpt2BenchmarkTimings& timings,
182     lcqi::Gpt2HotspotProfile* hotspot_profile,
183     std::int32_t worker_count) {
184     if (prompt_ids.empty()) {
185         throw std::runtime_error("GPT-2 generation needs at least one prompt ↵
↵ token");
186     }
187     lcqi::Gpt2ExecutionOptions execution_options;
188     execution_options.worker_count = worker_count;
189     lcqi::Gpt2CachedGreedyDecoder decoder(model, hotspot_profile, ↵
↵ execution_options);
190     timings.kv_cache_bytes = decoder.kv_cache_bytes();
191     timings.worker_count = decoder.worker_count();
192     std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
193     if (max_new_tokens == 0) {
194         return tokens;
195     }
196
197     const Clock::time_point generate_start = Clock::now();
198     const Clock::time_point prefill_start = Clock::now();
199     for (std::size_t index = 0; index + 1 < prompt_ids.size(); ++index) {
200         decoder.advance_without_prediction(prompt_ids[index]);
201         ++timings.prefill_steps;
202     }
203     std::int32_t predicted_token = decoder.step(prompt_ids.back());
204     ++timings.prefill_steps;
205     const Clock::time_point prefill_end = Clock::now();
206     timings.prefill_ms = elapsed_ms(prefill_start, prefill_end);
207
208     const Clock::time_point decode_start = Clock::now();
209     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
210         if (tokens.size() >= checked_size(model.config.max_positions, "↵
↵ max_positions")) {
211             throw std::runtime_error("GPT-2 generation would exceed ↵
↵ max_positions");
212         }
213         tokens.push_back(predicted_token);
214         if (model.config.eos_token_id >= 0 && predicted_token == model.config.↵
↵ eos_token_id) {
215             break;

```

```

216     }
217     if (step + 1 < max_new_tokens) {
218         predicted_token = decoder.step(predicted_token);
219         ++timings.decode_steps;
220     }
221 }
222 const Clock::time_point decode_end = Clock::now();
223 const Clock::time_point generate_end = Clock::now();
224 timings.decode_ms = elapsed_ms(decode_start, decode_end);
225 timings.generate_ms = elapsed_ms(generate_start, generate_end);
226 return tokens;
227 }
228
229 void print_benchmark(const Gpt2BenchmarkTimings& timings,
230                     std::size_t prompt_tokens,
231                     std::size_t generated_tokens) {
232     const double new_tokens = static_cast<double>(generated_tokens);
233     const double generated_tokens_per_second =
234         timings.generate_ms > 0.0 ? new_tokens * 1000.0 / timings.generate_ms : ←
235         ↪ 0.0;
236     const double decode_tokens_per_second =
237         timings.decode_ms > 0.0
238         ? static_cast<double>(timings.decode_steps) * 1000.0 / timings. ←
239         ↪ decode_ms
240         : 0.0;
241
242     std::cout << "benchmark_load_ms " << timings.load_ms << "\n";
243     std::cout << "benchmark_tokenize_ms " << timings.tokenize_ms << "\n";
244     std::cout << "benchmark_prefill_ms " << timings.prefill_ms << "\n";
245     std::cout << "benchmark_decode_ms " << timings.decode_ms << "\n";
246     std::cout << "benchmark_generate_ms " << timings.generate_ms << "\n";
247     std::cout << "benchmark_total_ms " << timings.total_ms << "\n";
248     std::cout << "benchmark_prompt_tokens " << prompt_tokens << "\n";
249     std::cout << "benchmark_generated_tokens " << generated_tokens << "\n";
250     std::cout << "benchmark_prefill_steps " << timings.prefill_steps << "\n";
251     std::cout << "benchmark_decode_steps " << timings.decode_steps << "\n";
252     std::cout << "benchmark_worker_count " << timings.worker_count << "\n";
253     std::cout << "benchmark_generate_tokens_per_second "
254         << generated_tokens_per_second << "\n";
255     std::cout << "benchmark_decode_tokens_per_second "
256         << decode_tokens_per_second << "\n";
257     std::cout << "benchmark_kv_cache_bytes " << timings.kv_cache_bytes << "\n";
258 }
259
260 void print_hotspot_profile(const lcqi::Gpt2HotspotProfile& profile) {
261     const double total = profile.total_step_ms > 0.0 ? profile.total_step_ms : ←
262     ↪ 1.0;
263     const auto print_ms = [total](std::string_view name, double value) {
264         std::cout << "hotspot_" << name << "_ms " << value << "\n";
265         std::cout << "hotspot_" << name << "_pct " << (value * 100.0 / total) ←
266         ↪ << "\n";
267     };

```

```

264     std::cout << "hotspot_decoder_steps " << profile.decoder_steps << "\n";
265     std::cout << "hotspot_layer_steps " << profile.layer_steps << "\n";
266     print_ms("total_step", profile.total_step_ms);
267     print_ms("embedding", profile.embedding_ms);
268     print_ms("layer_norm", profile.layer_norm_ms);
269     print_ms("final_norm", profile.final_norm_ms);
270     print_ms("qkv_projection", profile.qkv_projection_ms);
271     print_ms("kv_append", profile.kv_append_ms);
272     print_ms("attention", profile.attention_ms);
273     print_ms("attention_projection", profile.attention_projection_ms);
274     print_ms("residual_add", profile.residual_add_ms);
275     print_ms("mlp_fc", profile.mlp_fc_ms);
276     print_ms("gelu", profile.gelu_ms);
277     print_ms("mlp_projection", profile.mlp_projection_ms);
278     print_ms("lm_head", profile.lm_head_ms);
279     print_ms("logits_result", profile.logits_result_ms);
280 }
281
282 int run_tiny_mode() {
283     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model(
284         ↪ );
285     const std::vector<std::int32_t> prompt{
286         LCQI_GPT2_TINY_PROMPT_0,
287         LCQI_GPT2_TINY_PROMPT_1,
288         LCQI_GPT2_TINY_PROMPT_2,
289     };
290     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, prompt(
291         ↪ ));
292     const std::vector<std::int32_t> generated =
293         lcqi::gpt2_generate_greedy(model, prompt, 2);
294     std::cout << "mode tiny\n";
295     std::cout << "prompt_ids ";
296     print_ids(prompt);
297     std::cout << "\n";
298     std::cout << "predicted_token " << result.predicted_token << "\n";
299     std::cout << "logits";
300     for (const float value : result.logits) {
301         std::cout << ' ' << value;
302     }
303     std::cout << "\n";
304     std::cout << "generated_ids ";
305     print_ids(generated);
306     std::cout << "\n";
307     return EXIT_SUCCESS;
308 }
309
310 int run_real_model_mode(int argc, char** argv) {
311     if (argc < LCQI_GPT2_MIN_REAL_ARGC) {
312         throw std::runtime_error(
313             ↪ "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark
314             ↪ ] "
315             ↪ "model_dir prompt [max_new_tokens]");

```



```

313     }
314     const Clock::time_point total_start = Clock::now();
315     const Gpt2CliOptions options = parse_options(argc, argv);
316
317     Gpt2BenchmarkTimings timings;
318     const Clock::time_point load_start = Clock::now();
319     const lcqi::Gpt2ReferenceModel model = lcqi::load_gpt2_from_directory(↵
↵ options.model_dir);
320     const lcqi::Gpt2Tokenizer tokenizer = lcqi::load_gpt2_tokenizer(
321         options.model_dir / "vocab.json",
322         options.model_dir / "merges.txt",
323         model.config.bos_token_id,
324         model.config.eos_token_id);
325     timings.load_ms = elapsed_ms(load_start, Clock::now());
326
327     const Clock::time_point tokenize_start = Clock::now();
328     const std::vector<std::int32_t> prompt_ids = lcqi::gpt2_encode(tokenizer, ↵
↵ options.prompt);
329     timings.tokenize_ms = elapsed_ms(tokenize_start, Clock::now());
330
331     std::vector<std::int32_t> generated;
332     lcqi::Gpt2HotspotProfile hotspot_profile;
333     if (options.engine == Gpt2EngineMode::full_prefix) {
334         if (options.profile_hotspots) {
335             throw std::runtime_error("--profile-hotspots currently supports ↵
↵ cached engine only");
336         }
337         generated = generate_full_prefix(model, prompt_ids, options.↵
↵ max_new_tokens, timings);
338     } else {
339         generated = generate_cached(model,
340                                   prompt_ids,
341                                   options.max_new_tokens,
342                                   timings,
343                                   options.profile_hotspots ? &hotspot_profile↵
↵ : nullptr,
344                                   options.worker_count);
345     }
346     timings.total_ms = elapsed_ms(total_start, Clock::now());
347
348     std::cout << "mode gpt2_directory\n";
349     std::cout << "engine " << engine_name(options.engine) << "\n";
350     std::cout << "model_dir " << options.model_dir.string() << "\n";
351     std::cout << "prompt_ids ";
352     print_ids(prompt_ids);
353     std::cout << "\n";
354     std::cout << "generated_ids ";
355     print_ids(generated);
356     std::cout << "\n";
357     std::cout << "generated_text " << lcqi::gpt2_decode(tokenizer, generated) ↵
↵ << "\n";
358     if (options.benchmark) {

```

```
359     const std::size_t generated_tokens = generated.size() - prompt_ids.size();  
360     print_benchmark(timings, prompt_ids.size(), generated_tokens);  
361 }  
362 if (options.profile_hotspots) {  
363     print_hotspot_profile(hotspot_profile);  
364 }  
365 return EXIT_SUCCESS;  
366 }  
367  
368 } // namespace  
369  
370 int main(int argc, char** argv) {  
371     try {  
372         if (argc == LCQI_GPT2_TINY_ARGC) {  
373             return run_tiny_mode();  
374         }  
375         return run_real_model_mode(argc, argv);  
376     } catch (const std::exception& error) {  
377         std::cerr << "error: " << error.what() << "\n";  
378         return EXIT_FAILURE;  
379     }  
380 }
```

第 11 章实践：运行时内存计划、liveness 与 Workspace

本章对应原书中的第三册“推理运行时与内存规划”。不要把它当作概念复述，本章要把 activation arena、workspace、liveness、state edge、debug dump、buffer reuse 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `layout`，核心命令或测试入口是 `lcqi_trace`。

Listing 11.1 第 11 章实践：运行时内存计划、liveness 与 Workspace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / layout
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：比较普通 activation 和 KV cache 的生命周期。读者要先手写预期结果，再运行项目观察输出。动作是：说明为什么 KV cache 不能被 activation planner 当作临时缓冲复用。如果输出里没有 `checksum` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：内存复用只在生命周期不重叠时成立；debug/oracle/materialization 会延长生命周期这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 liveness 表：每个 checkpoint 的 `defined_at`、`last_used_at`、是否 state、是否允许复用。这个作业会被后续章节继续引用，命名、目录和

字段不要随意变化。

第 12 章 实 践：IR、Pass、Lowering 与 Executable Plan

本章对应原书中的第三册“AI 编译器细化：IR、pass、lowering 与 executable plan”。不要把它当作概念复述，本章要把 typed graph、pass、fusion、layout propagation、backend capability、fallback reason 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `run_inference`，核心命令或测试入口是 `lcqi_cli`。

Listing 12.1 第 12 章实践：IR、Pass、Lowering 与 Executable Plan 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / run_inference
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 tiny MLP 写成三节点 IR：linear、relu、linear。读者要先手写预期结果，再运行项目观察输出。动作是：说明哪些 pass 可以融合，哪些必须保留为 oracle checkpoint。如果输出里没有 `backend` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：IR 的价值不是画图，而是让 shape、layout、量化、后端选择和 fallback 诊断有稳定承载物这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 executable plan 草图：节点、输入输出 tensor、layout、

kernel、fallback reason 和 profiler event。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

后端优化、工业专项与验收

第 13 章实践：CPU 后端、微内核与性能证据

本章对应原书中的第三册“CPU 后端优化细讲”和“CPU decode 实战”。不要把它当作概念复述，本章要把 scalar baseline、packed weight、AVX2、objdump、perf boundary、shape sweep 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `avx2`、`GGUF`、`Q4_K` 和 `Q8_K`，核心命令或测试入口是 `lcqi_bench` 与 `lcqi_gguf_q4_bench`。

Listing 13.1 第 13 章实践：CPU 后端、微内核与性能证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / lcqi_gguf_q4_bench / avx2 / Q4_K
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：对同一 kernel 保存 benchmark CSV 和反汇编摘要。读者要先手写预期结果，再运行项目观察输出。动作是：解释速度差来自地址流、SIMD lane、packing、load 或 reduction 哪一项。如果输出里没有 `average_us` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

第二个特例必须对齐真实模型：`lcqi_gguf_q4_bench` 读取 llama.cpp smoke 使用的同一个 `SmolLM2-135M-Instruct-Q4_K_M.gguf`，从 GGUF tensor 表中选出真实 `Q4_K` 矩阵，比较三条路径：预先解成 f32 权重再 matvec、`Q4_K` 权重直接乘 f32 input、`Q4_K` 权重乘临时 `Q8_K` input。报告必须同时写 tensor 名称、shape、原始量化字节、解码后 f32 字节、重复轮次、最大误差和 checksum。

从这个特例推出的规律是：CPU 后端优化必须同时有 oracle、反汇编、shape sweep 和降级边界；只有耗时表不够这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA、GGUF 模型缓存或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。真实模型量化切片用下面命令生成报告：

Listing 13.2 SmolLM2 Q4_K 单算子证据入口


```
python3 books/cpu-volume-3/tools/run_smolm2_q4_k_benchmark.py --no-download --↵  
↵ repeat 20
```

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 CPU kernel evidence：命令、机器、编译器、输入 shape、backend、checksum、反汇编摘要和未验证 PMU 边界。若使用真实 GGUF 模型，还要给出模型 repo、文件名、SHA-256、tensor 名、量化格式、同模型 llama.cpp 参考和“为什么不能把单 tensor us 直接等价为端到端 tokens/s”。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 14 章实践：GPU 后端边界、copy 与同步成本

本章对应原书中的第三册“GPU 硬件基础”和“GPU 后端优化细讲”。不要把它当作概念复述，本章要把 device buffer、stream、event、kernel launch、copy、sync、fallback 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `StepTrace`，核心命令或测试入口是 `lcqi_trace`。

Listing 14.1 第 14 章实践：GPU 后端边界、copy 与同步成本的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  <- DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / StepTrace
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：当前项目没有 GPU kernel，也要写清 adapter 合同。读者要先手写预期结果，再运行项目观察输出。动作是：把 CPU trace 字段扩展成 GPU StepTrace 需要哪些字段。如果输出里没有 `copy_time_ns` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：GPU 快不快不能只看 kernel；copy、sync、sampler、fallback 和 batch wait 必须进入端到端 trace 这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 GPU adapter 设计：buffer owner、stream、event、H2D/D2H 字节、sync 点、fallback 和 oracle 对照。这个作业会被后续章节继续引用，命

名、目录和字段不要随意变化。

第 15 章实践：工业 LLM 专项能力对照

本章对应原书中的第三册“工业 LLM 专项：投机解码、MoE、LoRA、多 GPU 与源码对照”。不要把它当作概念复述，本章要把 speculative decoding、MoE routing、LoRA adapter、multi-GPU、source reading 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `bandwidth_limit`，核心命令或测试入口是 `lcqi_ledger`。

Listing 15.1 第 15 章实践：工业 LLM 专项能力对照的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_ledger / bandwidth_limit
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：选择一个专项，不写泛泛介绍。读者要先手写预期结果，再运行项目观察输出。动作是：把它接到现有 `trace`、`memory`、`backend` 或 `serving` 字段上。如果输出里没有 `tokens/s` 或对应字段无法解释，就不要继续优化；先回到 `reference`、输入合同和 `trace` 字段。

从这个特例推出的规律是：工业专项不是功能清单；必须说明它购买什么能力、增加什么状态、破坏哪些假设这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、`perf`、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、`fallback` 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交专项设计卡：动机、状态新增、`trace` 字段、回归测试、性能账本和不能验证的部分。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 16 章实践：工程验收、回归门禁与最终项目

目

本章对应原书中的第三册“工程验收与回归体系”。不要把它当作概念复述，本章要把 `reference`、`unit test`、`trace golden`、`benchmark gate`、`SLO gate`、`coverage`、`CI` 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `lcqi_tests`，核心命令或测试入口是 `ctest`。

Listing 16.1 第 16 章实践：工程验收、回归门禁与最终项目的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<↵
↵ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: ctest / lcqi_tests
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把所有前面报告合成最终验收包。读者要先手写预期结果，再运行项目观察输出。动作是：运行 `ctest` 并说明每个 `test` 保护哪条合同。如果输出里没有 `100% tests passed` 或对应字段无法解释，就不要继续优化；先回到 `reference`、输入合同和 `trace` 字段。

从这个特例推出的规律是：实践卷最终目标是长期维护：结果正确、`trace` 稳定、性能可解释、服务指标可回归、失败边界不伪装这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 `AVX2`、`perf`、`GPU`、`NUMA` 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、`fallback` 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交最终项目：`README`、命令记录、报告索引、测试结果、`benchmark` 表、`trace` 摘要、风险清单和下一阶段计划。这个作业会被后续章节继续引用，命

名、目录和字段不要随意变化。

代码走读：工具、Smoke、Benchmark 与回归测试

工具不是附属品

LCQI 的工程证据不只在 C++ 源码里。smoke 脚本决定真实模型怎样下载、校验和运行；benchmark 程序决定性能口径；trace、ledger 和 serving probe 决定报告字段；CTest 决定哪些行为能长期回归。读者必须把这些文件当作工程合同的一部分。

Benchmark、Trace、Ledger 与 Serving Probe

bench.cpp 输出 int8 kernel benchmark。它服务性能观察，但必须和 max diff 对照一起看。

Listing 16.2 src/bench.cpp

```

1  #include <lcqi/int8_kernels.hpp>
2
3  #include <algorithm>
4  #include <cstdlib>
5  #include <iostream>
6  #include <stdexcept>
7  #include <string>
8  #include <vector>
9
10 namespace {
11
12 constexpr const char* LCQI_TARGET_ARCH_X86_64 = "x86_64";
13 constexpr const char* LCQI_TARGET_ARCH_I386 = "i386";
14 constexpr const char* LCQI_TARGET_ARCH_UNKNOWN = "unknown";
15 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 200;
16 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
17 constexpr std::int32_t LCQI_LARGE_CASE_REPEAT_DIVISOR = 4;
18 constexpr std::int32_t LCQI_MIN_LARGE_CASE_REPEAT = 1;
19 constexpr std::int32_t LCQI_DECODE_SHAPE_SIZE = 4096;
20
21 const char* target_arch() noexcept {
22     #if defined(__x86_64__) || defined(_M_X64)
23         return LCQI_TARGET_ARCH_X86_64;
24     #elif defined(__i386__) || defined(_M_IX86)
25         return LCQI_TARGET_ARCH_I386;
26     #else
27         return LCQI_TARGET_ARCH_UNKNOWN;
28     #endif
29 }
30
31 std::int32_t parse_repeat(int argc, char** argv) {
32     if (argc <= LCQI_REPEAT_ARG_INDEX) {
33         return LCQI_DEFAULT_REPEAT;

```

```

34     }
35     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_REPEAT_ARG_INDEX]));
36     if (repeat <= 0) {
37         throw std::runtime_error("repeat must be positive");
38     }
39     return repeat;
40 }
41
42 std::vector<lcqi::KernelBenchmarkCase> make_cases(std::int32_t repeat) {
43     return {
44         {128, 128, 16, repeat},
45         {256, 256, 16, repeat},
46         {512, 512, 16, repeat},
47         {513, 257, 16, repeat},
48         {1024, 4096, 16, std::max(LCQI_MIN_LARGE_CASE_REPEAT,
49                                   repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
50         {LCQI_DECODE_SHAPE_SIZE, LCQI_DECODE_SHAPE_SIZE, 16,
51          std::max(LCQI_MIN_LARGE_CASE_REPEAT, repeat / ↵
↵ LCQI_LARGE_CASE_REPEAT_DIVISOR)},
52     };
53 }
54
55 } // namespace
56
57 int main(int argc, char** argv) {
58     try {
59         const std::int32_t repeat = parse_repeat(argc, argv);
60         std::cout
61             << "target_arch,input_size,output_size,output_block,repeat,backend,↵
↵ available,"
62             << "average_us,max_abs_diff,checksum\n";
63         for (const lcqi::KernelBenchmarkCase& benchmark_case : make_cases(↵
↵ repeat)) {
64             const lcqi::KernelBenchmarkResult result =
65                 lcqi::benchmark_linear_i8_case(benchmark_case);
66             for (const lcqi::KernelBackendBenchmark& backend : result.backends)↵
↵ {
67                 std::cout << target_arch() << ','
68                     << result.benchmark_case.input_size << ','
69                     << result.benchmark_case.output_size << ','
70                     << result.benchmark_case.output_block_size << ','
71                     << result.benchmark_case.repeat << ','
72                     << backend.name << ','
73                     << (backend.available ? 1 : 0) << ','
74                     << backend.average_us << ','
75                     << backend.max_abs_diff << ','
76                     << backend.checksum << '\n';
77             }
78         }
79         return EXIT_SUCCESS;
80     } catch (const std::exception& error) {

```



```

81         std::cerr << "error: " << error.what() << '\n';
82         return EXIT_FAILURE;
83     }
84 }

```

`trace.cpp` 输出 reference decoder checkpoint。它用于定位错误边界，不应该被简化成只打印最终 token。

Listing 16.3 src/trace.cpp

```

1  #include <lcqi/reference_decoder.hpp>
2  #include <lcqi/tokenizer.hpp>
3
4  #include <algorithm>
5  #include <array>
6  #include <cstdlib>
7  #include <exception>
8  #include <filesystem>
9  #include <iostream>
10 #include <span>
11 #include <sstream>
12 #include <string>
13 #include <string_view>
14
15 namespace {
16
17 constexpr std::array<std::int32_t, 3> LCQI_TRACE_TOKEN_IDS{1, 2, 3};
18 constexpr std::string_view LCQI_TRACE_PROMPT = "tok1 tok2 tok3";
19 constexpr std::int32_t LCQI_TRACE_METADATA_LAYER = -1;
20 constexpr std::int32_t LCQI_TRACE_BOS_ID = 0;
21 constexpr std::int32_t LCQI_TRACE_EOS_ID = 4;
22 constexpr std::size_t LCQI_TRACE_VALUE_LIMIT = 8;
23
24 std::filesystem::path tokenizer_fixture_path() {
25     return std::filesystem::path(__FILE__).parent_path().parent_path() /
26         "models" / "tiny_tokenizer.txt";
27 }
28
29 std::vector<std::int32_t> decoder_tokens_from_prompt(const lcqi::TokenizerModel& ←
    ↪ & tokenizer,
30                                                     std::string_view prompt) {
31     std::vector<std::int32_t> full_tokens = lcqi::encode_prompt(tokenizer, ←
    ↪ prompt);
32     if (full_tokens.size() != LCQI_TRACE_TOKEN_IDS.size() + 2 ||
33         full_tokens.front() != LCQI_TRACE_BOS_ID ||
34         full_tokens.back() != LCQI_TRACE_EOS_ID) {
35         throw std::runtime_error("unexpected tokenizer fixture ids");
36     }
37
38     std::vector<std::int32_t> decoder_tokens(
39         full_tokens.begin() + 1,
40         full_tokens.end() - 1);
41     if (!std::equal(decoder_tokens.begin(),

```

```

42         decoder_tokens.end(),
43         LCQI_TRACE_TOKEN_IDS.begin(),
44         LCQI_TRACE_TOKEN_IDS.end())) {
45     throw std::runtime_error("unexpected decoder token ids");
46 }
47 return decoder_tokens;
48 }
49
50 std::string join_ints(std::span<const std::int32_t> values) {
51     std::ostringstream stream;
52     for (std::size_t index = 0; index < values.size(); ++index) {
53         if (index != 0) {
54             stream << 'x';
55         }
56         stream << values[index];
57     }
58     return stream.str();
59 }
60
61 std::string join_int_values(std::span<const std::int32_t> values) {
62     std::ostringstream stream;
63     for (std::size_t index = 0; index < values.size(); ++index) {
64         if (index != 0) {
65             stream << ';';
66         }
67         stream << values[index];
68     }
69     return stream.str();
70 }
71
72 std::int32_t checksum_ints(std::span<const std::int32_t> values) {
73     std::int32_t result = 0;
74     for (const std::int32_t value : values) {
75         result += value;
76     }
77     return result;
78 }
79
80 std::int32_t max_abs_ints(std::span<const std::int32_t> values) {
81     std::int32_t result = 0;
82     for (const std::int32_t value : values) {
83         result = std::max(result, std::abs(value));
84     }
85     return result;
86 }
87
88 std::string join_floats(std::span<const float> values) {
89     std::ostringstream stream;
90     const std::size_t count = std::min(values.size(), LCQI_TRACE_VALUE_LIMIT);
91     for (std::size_t index = 0; index < count; ++index) {
92         if (index != 0) {
93             stream << ';';

```

```

94     }
95     stream << values[index];
96 }
97 if (values.size() > LCQI_TRACE_VALUE_LIMIT) {
98     stream << "...";
99 }
100 return stream.str();
101 }
102
103 void print_trace_csv(const lcqi::ReferenceDecodeTraceResult& trace,
104                     const lcqi::TokenizerModel& tokenizer,
105                     std::span<const std::int32_t> full_tokens) {
106     std::cout << "name,token_position,layer_id,shape,stride,dtype,layout,↵
↵ checksum,max_abs,values\n";
107     std::cout << "prompt,0," << LCQI_TRACE_METADATA_LAYER
108         << ",scalar,scalar,utf8,text,0,0," << LCQI_TRACE_PROMPT << '\n';
109     std::cout << "tokenizer,0," << LCQI_TRACE_METADATA_LAYER
110         << ',' << full_tokens.size() << ",1,i32,contiguous,"
111         << checksum_ints(full_tokens) << ','
112         << max_abs_ints(full_tokens) << ','
113         << join_int_values(full_tokens) << '\n';
114     std::cout << "tokenizer_contract,0," << LCQI_TRACE_METADATA_LAYER
115         << ",scalar,scalar,u64,fnv1a,"
116         << tokenizer_contract_hash(tokenizer) << ",0,"
117         << tokenizer.chat_template_hash << '\n';
118     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↵
↵     {
119         std::cout << checkpoint.name << ','
120             << checkpoint.token_position << ','
121             << checkpoint.layer_id << ','
122             << join_ints(checkpoint.shape) << ','
123             << join_ints(checkpoint.stride) << ','
124             << checkpoint.dtype << ','
125             << checkpoint.layout << ','
126             << checkpoint.checksum << ','
127             << checkpoint.max_abs << ','
128             << join_floats(checkpoint.values) << '\n';
129     }
130     std::cout << "sampler_argmax,"
131         << LCQI_TRACE_TOKEN_IDS.size() - 1 << ','
132         << LCQI_TRACE_METADATA_LAYER
133         << ',' << trace.result.logits.size() << ",1,f32,argmax,"
134         << "0,0,token=" << trace.result.predicted_token << '\n';
135     std::cout << "generated_token,"
136         << LCQI_TRACE_TOKEN_IDS.size() << ','
137         << LCQI_TRACE_METADATA_LAYER
138         << ",scalar,scalar,i32,generated,"
139         << trace.result.predicted_token << ','
140         << trace.result.predicted_token << ','
141         << trace.result.predicted_token << '\n';
142 }
143

```

```

144 } // namespace
145
146 int main() {
147     try {
148         const lcqi::TokenizerModel tokenizer =
149             lcqi::load_tokenizer(tokenizer_fixture_path());
150         const std::vector<std::int32_t> full_tokens =
151             lcqi::encode_prompt(tokenizer, LCQI_TRACE_PROMPT);
152         const std::vector<std::int32_t> decoder_tokens =
153             decoder_tokens_from_prompt(tokenizer, LCQI_TRACE_PROMPT);
154         const lcqi::ReferenceDecoderModel model = lcqi::↵
↵ make_tiny_reference_decoder_model();
155         const lcqi::ReferenceDecodeTraceResult trace =
156             lcqi::run_reference_decode_with_trace(model, decoder_tokens);
157         print_trace_csv(trace, tokenizer, full_tokens);
158         return EXIT_SUCCESS;
159     } catch (const std::exception& error) {
160         std::cerr << "error: " << error.what() << '\n';
161         return EXIT_FAILURE;
162     }
163 }

```

`ledger.cpp` 输出 7B 规模账本。它是估算，不是实测吞吐；读者要区分 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限。

Listing 16.4 src/ledger.cpp

```

1 #include <cstdlib>
2 #include <cstdint>
3 #include <exception>
4 #include <iomanip>
5 #include <iostream>
6 #include <string_view>
7
8 namespace {
9
10 constexpr double LCQI_BYTES_PER_GB = 1000.0 * 1000.0 * 1000.0;
11 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
12 constexpr double LCQI_FLOPS_PER_MAC = 2.0;
13 constexpr std::int32_t LCQI_SEVENB_LAYER_COUNT = 32;
14 constexpr std::int32_t LCQI_SEVENB_HIDDEN_SIZE = 4096;
15 constexpr std::int32_t LCQI_SEVENB_QUERY_HEADS = 32;
16 constexpr std::int32_t LCQI_SEVENB_KV_HEADS = 8;
17 constexpr std::int32_t LCQI_SEVENB_HEAD_DIM = 128;
18 constexpr std::int32_t LCQI_SEVENB_INTERMEDIATE_SIZE = 11008;
19 constexpr std::int32_t LCQI_SEVENB_CONTEXT_LENGTH = 4096;
20
21 struct DTypeLedger {
22     std::string_view name;
23     double weight_gb_per_token = 0.0;
24     double kv_element_bytes = 0.0;
25 };
26

```

```

27 constexpr DTypeLedger LCQI_DTYPE_LEDGERS[] = {
28     {"fp16", 14.0, 2.0},
29     {"int8", 7.0, 2.0},
30     {"int4", 3.7, 2.0},
31 };
32
33 constexpr double LCQI_BANDWIDTH_GB_PER_S[] = {
34     50.0,
35     100.0,
36     200.0,
37     600.0,
38     1000.0,
39     1600.0,
40 };
41
42 double linear_macs_per_layer() noexcept {
43     const double hidden = LCQI_SEVENB_HIDDEN_SIZE;
44     const double kv_width = LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM;
45     const double qkv = hidden * (hidden + 2.0 * kv_width);
46     const double output_projection = hidden * hidden;
47     const double mlp = 3.0 * hidden * LCQI_SEVENB_INTERMEDIATE_SIZE;
48     return qkv + output_projection + mlp;
49 }
50
51 double attention_macs_per_layer() noexcept {
52     return 2.0 * LCQI_SEVENB_QUERY_HEADS * LCQI_SEVENB_CONTEXT_LENGTH *
53           LCQI_SEVENB_HEAD_DIM;
54 }
55
56 double kv_bytes_per_token_all_layers(double element_bytes) noexcept {
57     return static_cast<double>(LCQI_SEVENB_LAYER_COUNT) * 2.0 *
58           LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM * element_bytes;
59 }
60
61 double kv_read_gb_per_decode_token(double element_bytes) noexcept {
62     const double bytes = static_cast<double>(LCQI_SEVENB_CONTEXT_LENGTH) *
63           kv_bytes_per_token_all_layers(element_bytes);
64     return bytes / LCQI_BYTES_PER_GB;
65 }
66
67 double kv_capacity_mib(double element_bytes, std::int32_t context_length) ↵
68     ↵ noexcept {
69     const double bytes = static_cast<double>(context_length) *
70           kv_bytes_per_token_all_layers(element_bytes);
71     return bytes / LCQI_BYTES_PER_MIB;
72 }
73
74 void print_model_rows() {
75     const double linear_macs = linear_macs_per_layer();
76     const double attention_macs = attention_macs_per_layer();
77     const double total_flops =
78         (linear_macs + attention_macs) * LCQI_SEVENB_LAYER_COUNT * ↵

```

```

    ↪ LCQI_FLOPS_PER_MAC;
78     std::cout << "section,item,value,unit,notes\n";
79     std::cout << "model,layers," << LCQI_SEVENB_LAYER_COUNT << ",count,7B ↪
    ↪ fixture\n";
80     std::cout << "model,hidden," << LCQI_SEVENB_HIDDEN_SIZE << ",elements,H\n";
81     std::cout << "model,query_heads," << LCQI_SEVENB_QUERY_HEADS << ",count,6QA↪
    ↪ query heads\n";
82     std::cout << "model,kv_heads," << LCQI_SEVENB_KV_HEADS << ",count,6QA KV ↪
    ↪ heads\n";
83     std::cout << "model,head_dim," << LCQI_SEVENB_HEAD_DIM << ",elements,per ↪
    ↪ head\n";
84     std::cout << "compute,linear_macs_per_layer," << linear_macs << ",macs,↪
    ↪ decode token\n";
85     std::cout << "compute,attention_macs_per_layer," << attention_macs
86         << ",macs,context=4096\n";
87     std::cout << "compute,total_decode_flops_per_token," << total_flops
88         << ",flops,linear plus attention\n";
89 }
90
91 void print_kv_rows() {
92     for (const std::int32_t context : {1024, 2048, 4096, 8192}) {
93         std::cout << "kv,fp16_capacity_context_" << context << ','
94             << kv_capacity_mib(2.0, context)
95             << ",MiB,single session\n";
96     }
97 }
98
99 void print_dtype_rows() {
100     for (const DTypeLedger& dtype : LCQI_DTYPE_LEDGERS) {
101         const double kv_read_gb = kv_read_gb_per_decode_token(dtype.↪
    ↪ kv_element_bytes);
102         const double total_gb = dtype.weight_gb_per_token + kv_read_gb;
103         std::cout << "decode_bytes," << dtype.name << "_weight_gb,"
104             << dtype.weight_gb_per_token << ",GB/token,weight stream\n";
105         std::cout << "decode_bytes," << dtype.name << "_kv_read_gb,"
106             << kv_read_gb << ",GB/token,context=4096\n";
107         std::cout << "decode_bytes," << dtype.name << "_total_gb,"
108             << total_gb << ",GB/token,weight plus KV\n";
109         for (const double bandwidth : LCQI_BANDWIDTH_GB_PER_S) {
110             std::cout << "bandwidth_limit," << dtype.name << "_at_"
111                 << static_cast<std::int32_t>(bandwidth) << "_gbps,"
112                 << bandwidth / total_gb
113                 << ",tokens/s,bandwidth-only upper bound\n";
114         }
115     }
116 }
117
118 } // namespace
119
120 int main() {
121     try {
122         std::cout << std::fixed << std::setprecision(3);

```

```

123     print_model_rows();
124     print_kv_rows();
125     print_dtype_rows();
126     return EXIT_SUCCESS;
127 } catch (const std::exception& error) {
128     std::cerr << "error: " << error.what() << '\n';
129     return EXIT_FAILURE;
130 }
131 }

```

`serving_probe.cpp` 固定短请求、RAG 请求、取消请求和超长请求。它把 TTFT、TPOT、拒绝和取消回收变成可观察字段。

Listing 16.5 src/serving_probe.cpp

```

1  #include <algorithm>
2  #include <cstdlib>
3  #include <stdint>
4  #include <exception>
5  #include <cmath>
6  #include <iomanip>
7  #include <iostream>
8  #include <string_view>
9  #include <vector>
10
11 namespace {
12
13 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
14 constexpr std::int32_t LCQI_PROBE_LAYER_COUNT = 32;
15 constexpr std::int32_t LCQI_PROBE_KV_HEADS = 8;
16 constexpr std::int32_t LCQI_PROBE_HEAD_DIM = 128;
17 constexpr double LCQI_PROBE_KV_ELEMENT_BYTES = 2.0;
18 constexpr double LCQI_PROBE_KV_BUDGET_MIB = 512.0;
19 constexpr double LCQI_PROBE_WORKSPACE_MIB = 64.0;
20 constexpr double LCQI_PROBE_ARENA_MIB = 32.0;
21 constexpr double LCQI_PROBE_TRACE_MIB = 4.0;
22 constexpr double LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS = 8.0;
23 constexpr double LCQI_PROBE_PREFILL_MS_PER_TOKEN = 0.08;
24 constexpr double LCQI_PROBE_DECODE_STEP_MS = 14.0;
25 constexpr double LCQI_PROBE_CANCEL_RECLAIM_MS = 2.0;
26 constexpr std::size_t LCQI_PERCENTILE_MIN_INDEX = 1;
27
28 struct ProbeRequest {
29     std::string_view request_id;
30     std::int32_t prompt_tokens = 0;
31     std::int32_t max_new_tokens = 0;
32     bool cancel_after_prefill = false;
33 };
34
35 struct ProbeResult {
36     std::string_view request_id;
37     std::int32_t accepted = 0;
38     std::string_view rejection_reason;

```

```

39     std::int32_t prompt_tokens = 0;
40     std::int32_t reserved_tokens = 0;
41     double kv_reserved_mib = 0.0;
42     double queue_wait_ms = 0.0;
43     double prefill_ms = 0.0;
44     double ttft_ms = 0.0;
45     double decode_step_p95_ms = 0.0;
46     double tpot_ms = 0.0;
47     std::int32_t completion_tokens = 0;
48     std::string_view finish_reason;
49     double cancel_reclaim_ms = 0.0;
50 };
51
52 const std::vector<ProbeRequest> LCQI_PROBE_REQUESTS{
53     {"req_short", 32, 4, false},
54     {"req_rag", 512, 8, false},
55     {"req_cancel", 128, 16, true},
56     {"req_too_long", 4096, 512, false},
57 };
58
59 double kv_mib_for_tokens(std::int32_t tokens) noexcept {
60     const double bytes = static_cast<double>(tokens) * LCQI_PROBE_LAYER_COUNT *
        ↪ 2.0 *
61         LCQI_PROBE_KV_HEADS * LCQI_PROBE_HEAD_DIM *
62         LCQI_PROBE_KV_ELEMENT_BYTES;
63     return bytes / LCQI_BYTES_PER_MIB;
64 }
65
66 double percentile(std::vector<double> values, double fraction) {
67     if (values.empty()) {
68         return 0.0;
69     }
70     std::sort(values.begin(), values.end());
71     const std::size_t max_index = values.size() - LCQI_PERCENTILE_MIN_INDEX;
72     const std::size_t index =
73         std::min(max_index,
74             static_cast<std::size_t>(
75                 std::ceil(fraction * static_cast<double>(max_index))));
76     return values[index];
77 }
78
79 std::vector<ProbeResult> run_probe() {
80     std::vector<ProbeResult> results;
81     double reserved_kv_mib = 0.0;
82     std::int32_t accepted_before = 0;
83     for (const ProbeRequest& request : LCQI_PROBE_REQUESTS) {
84         ProbeResult result;
85         result.request_id = request.request_id;
86         result.prompt_tokens = request.prompt_tokens;
87         result.reserved_tokens = request.prompt_tokens + request.max_new_tokens
        ↪ ;
88         result.kv_reserved_mib = kv_mib_for_tokens(result.reserved_tokens);

```



```

89     const double static_session_mib =
90         result.kv_reserved_mib + LCQI_PROBE_WORKSPACE_MIB +
91         LCQI_PROBE_ARENA_MIB + LCQI_PROBE_TRACE_MIB;
92     if (reserved_kv_mib + result.kv_reserved_mib > LCQI_PROBE_KV_BUDGET_MIB) {
93         result.accepted = 0;
94         result.rejection_reason = "memory_budget_exceeded";
95         result.finish_reason = "rejected";
96         results.push_back(result);
97         continue;
98     }
99
100     reserved_kv_mib += result.kv_reserved_mib;
101     result.accepted = 1;
102     result.rejection_reason = "none";
103     result.queue_wait_ms = static_cast<double>(accepted_before) *
104         LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS;
105     result.prefill_ms = static_cast<double>(request.prompt_tokens) *
106         LCQI_PROBE_PREFILL_MS_PER_TOKEN;
107     result.ttft_ms = result.queue_wait_ms + result.prefill_ms +
108     LCQI_PROBE_DECODE_STEP_MS;
109     result.decode_step_p95_ms = LCQI_PROBE_DECODE_STEP_MS +
110         static_session_mib / 512.0;
111     result.tpot_ms = result.decode_step_p95_ms;
112     if (request.cancel_after_prefill) {
113         result.completion_tokens = 0;
114         result.finish_reason = "cancelled";
115         result.cancel_reclaim_ms = LCQI_PROBE_CANCEL_RECLAIM_MS;
116         reserved_kv_mib -= result.kv_reserved_mib;
117     } else {
118         result.completion_tokens = request.max_new_tokens;
119         result.finish_reason = "max_tokens";
120     }
121     ++accepted_before;
122     results.push_back(result);
123 }
124 return results;
125 }
126
127 void print_request_rows(const std::vector<ProbeResult>& results) {
128     std::cout << "row_type,request_id,accepted,rejection_reason,prompt_tokens,
129     reserved_tokens,"
130     "kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,
131     decode_step_p95_ms,"
132     "tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,
133     metric_name,"
134     "metric_value\n";
135     for (const ProbeResult& result : results) {
136         std::cout << "request," << result.request_id << ',' <<
137             result.accepted << ',' <<
138             result.rejection_reason << ',' <<
139             result.prompt_tokens << ',' <<

```

```

136         << result.reserved_tokens << ','
137         << result.kv_reserved_mib << ','
138         << result.queue_wait_ms << ','
139         << result.prefill_ms << ','
140         << result.ttft_ms << ','
141         << result.decode_step_p95_ms << ','
142         << result.tpot_ms << ','
143         << result.completion_tokens << ','
144         << result.finish_reason << ','
145         << result.cancel_reclaim_ms << ",,\n";
146     }
147 }
148
149 void print_summary_row(const std::vector<ProbeResult>& results) {
150     std::vector<double> ttft_values;
151     std::vector<double> queue_values;
152     double accepted = 0.0;
153     double rejected = 0.0;
154     double cancelled = 0.0;
155     double reserved_peak_mib = 0.0;
156     double running_reserved_mib = 0.0;
157     for (const ProbeResult& result : results) {
158         if (result.accepted == 0) {
159             rejected += 1.0;
160             continue;
161         }
162         accepted += 1.0;
163         ttft_values.push_back(result.ttft_ms);
164         queue_values.push_back(result.queue_wait_ms);
165         running_reserved_mib += result.kv_reserved_mib;
166         reserved_peak_mib = std::max(reserved_peak_mib, running_reserved_mib);
167         if (result.finish_reason == "cancelled") {
168             cancelled += 1.0;
169             running_reserved_mib -= result.kv_reserved_mib;
170         }
171     }
172     const double total = accepted + rejected;
173     const double rejection_rate = total == 0.0 ? 0.0 : rejected / total;
174     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
175         << "accepted_count," << accepted << '\n';
176     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,"
177         << LCQI_PROBE_CANCEL_RECLAIM_MS << ",cancel_count," << cancelled <<
178         << '\n';
179     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
180         << "rejected_count," << rejected << '\n';
181     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
182         << "rejection_rate," << rejection_rate << '\n';
183     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
184         << "kv_reserved_peak_mib," << reserved_peak_mib << '\n';
185     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
186         << "queue_wait_p95_ms," << percentile(queue_values, 0.95) << '\n' <<
187         ;

```

```

186     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
187               << "ttft_p95_ms," << percentile(ttft_values, 0.95) << '\n';
188 }
189
190 } // namespace
191
192 int main() {
193     try {
194         std::cout << std::fixed << std::setprecision(3);
195         const std::vector<ProbeResult> results = run_probe();
196         print_request_rows(results);
197         print_summary_row(results);
198         return EXIT_SUCCESS;
199     } catch (const std::exception& error) {
200         std::cerr << "error: " << error.what() << '\n';
201         return EXIT_FAILURE;
202     }
203 }

```

`onednn_bench.cpp` 是可选外部库对标入口。它只在环境安装 oneDNN 时构建，不能把不可用平台写成已验证性能。

Listing 16.6 src/onednn_bench.cpp

```

1  #include <dnndl.hpp>
2
3  #include <chrono>
4  #include <cstdint>
5  #include <cstdlib>
6  #include <iostream>
7  #include <numeric>
8  #include <stdexcept>
9  #include <string>
10 #include <unordered_map>
11 #include <vector>
12
13 namespace {
14
15 constexpr std::int64_t LCQI_ONEDNN_BATCH = 1;
16 constexpr std::int64_t LCQI_ONEDNN_K = 4096;
17 constexpr std::int64_t LCQI_ONEDNN_O = 4096;
18 constexpr std::int32_t LCQI_ONEDNN_DEFAULT_REPEAT = 20;
19 constexpr std::int32_t LCQI_ONEDNN_WARMUP = 3;
20 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
21 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
22 constexpr float LCQI_INPUT_SCALE = 0.125F;
23 constexpr std::int32_t LCQI_INPUT_MODULUS = 17;
24 constexpr std::int32_t LCQI_INPUT_CENTER = 8;
25 constexpr std::int32_t LCQI_WEIGHT_MODULUS = 23;
26 constexpr std::int32_t LCQI_WEIGHT_CENTER = 11;
27 constexpr float LCQI_WEIGHT_SCALE = 0.03125F;
28
29 std::int32_t parse_repeat(int argc, char** argv) {

```

```

30     if (argc <= LCQI_REPEAT_ARG_INDEX) {
31         return LCQI_ONEDNN_DEFAULT_REPEAT;
32     }
33     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_REPEAT_ARG_INDEX]));
34     if (repeat <= 0) {
35         throw std::runtime_error("repeat must be positive");
36     }
37     return repeat;
38 }
39
40 std::vector<float> make_input() {
41     std::vector<float> input(static_cast<std::size_t>(LCQI_ONEDNN_K));
42     for (std::int64_t index = 0; index < LCQI_ONEDNN_K; ++index) {
43         input[static_cast<std::size_t>(index)] =
44             static_cast<float>((index % LCQI_INPUT_MODULUS) - LCQI_INPUT_CENTER↵
↵ ) *
45             LCQI_INPUT_SCALE;
46     }
47     return input;
48 }
49
50 std::vector<float> make_weights_kn() {
51     std::vector<float> weights(static_cast<std::size_t>(LCQI_ONEDNN_K * ↵
↵ LCQI_ONEDNN_0));
52     for (std::int64_t k = 0; k < LCQI_ONEDNN_K; ++k) {
53         for (std::int64_t out = 0; out < LCQI_ONEDNN_0; ++out) {
54             const std::int64_t row_major_out_k = out * LCQI_ONEDNN_K + k;
55             const float value =
56                 static_cast<float>((row_major_out_k % LCQI_WEIGHT_MODULUS) -
57                                     LCQI_WEIGHT_CENTER) *
58                 LCQI_WEIGHT_SCALE;
59             weights[static_cast<std::size_t>(k * LCQI_ONEDNN_0 + out)] = value;
60         }
61     }
62     return weights;
63 }
64
65 float checksum(const std::vector<float>& values) {
66     return std::accumulate(values.begin(), values.end(), 0.0F);
67 }
68
69 } // namespace
70
71 int main(int argc, char** argv) {
72     try {
73         const std::int32_t repeat = parse_repeat(argc, argv);
74         std::vector<float> input = make_input();
75         std::vector<float> weights = make_weights_kn();
76         std::vector<float> output(static_cast<std::size_t>(LCQI_ONEDNN_BATCH * ↵
↵ LCQI_ONEDNN_0),
77                                 0.0F);

```

```

78
79     dnnl::engine engine(dnnl::engine::kind::cpu, 0);
80     dnnl::stream stream(engine);
81
82     const dnnl::memory::dims source_dims = {LCQI_ONEDNN_BATCH, ↵
↵ LCQI_ONEDNN_K};
83     const dnnl::memory::dims weight_dims = {LCQI_ONEDNN_K, LCQI_ONEDNN_0};
84     const dnnl::memory::dims output_dims = {LCQI_ONEDNN_BATCH, ↵
↵ LCQI_ONEDNN_0};
85
86     const dnnl::memory::desc source_desc(
87         source_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
88     const dnnl::memory::desc weight_desc(
89         weight_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
90     const dnnl::memory::desc output_desc(
91         output_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
92
93     dnnl::memory source_memory(source_desc, engine, input.data());
94     dnnl::memory weight_memory(weight_desc, engine, weights.data());
95     dnnl::memory output_memory(output_desc, engine, output.data());
96
97     dnnl::matmul::primitive_desc matmul_desc(engine, source_desc, ↵
↵ weight_desc, output_desc);
98     dnnl::matmul matmul(matmul_desc);
99     const std::unordered_map<int, dnnl::memory> arguments = {
100         {DNNL_ARG_SRC, source_memory},
101         {DNNL_ARG_WEIGHTS, weight_memory},
102         {DNNL_ARG_DST, output_memory},
103     };
104
105     for (std::int32_t index = 0; index < LCQI_ONEDNN_WARMUP; ++index) {
106         matmul.execute(stream, arguments);
107         stream.wait();
108     }
109
110     const auto begin = std::chrono::steady_clock::now();
111     for (std::int32_t index = 0; index < repeat; ++index) {
112         matmul.execute(stream, arguments);
113         stream.wait();
114     }
115     const auto end = std::chrono::steady_clock::now();
116     const std::chrono::duration<double> elapsed = end - begin;
117     const double average_us =
118         elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>↵
↵ >(repeat);
119
120     std::cout << "library,input_size,output_size,dtype,repeat,average_us,↵
↵ checksum\n";
121     std::cout << "onednn," << LCQI_ONEDNN_K << ',' << LCQI_ONEDNN_0

```

```

122         << ",f32," << repeat << ',' << average_us << ','
123         << checksum(output) << '\n';
124     return EXIT_SUCCESS;
125 } catch (const std::exception& error) {
126     std::cerr << "error: " << error.what() << '\n';
127     return EXIT_FAILURE;
128 }
129 }

```

`gguf_q4_bench.cpp` 是真实 GGUF 量化 tensor 的单算子 benchmark。它读取同一个 `SmolLM2 Q4_K_M` GGUF，选择真实 `Q4_K` 矩阵，分别测 `f32` 解码基线、`Q4_Kxf32` 和 `Q4_KxQ8_K`。它输出的 `speedup` 只适用于这个单 tensor matvec，不能写成完整模型 `tokens/s`。

Listing 16.7 `src/gguf_q4_bench.cpp`

```

1 #include <lcqi/gguf.hpp>
2 #include <lcqi/q4_k.hpp>
3
4 #include <algorithm>
5 #include <chrono>
6 #include <cmath>
7 #include <cstdlib>
8 #include <filesystem>
9 #include <iomanip>
10 #include <iostream>
11 #include <span>
12 #include <stdexcept>
13 #include <string>
14 #include <string_view>
15 #include <vector>
16
17 namespace {
18
19 constexpr int LCQI_ARG_MODEL = 1;
20 constexpr int LCQI_ARG_TENSOR = 2;
21 constexpr int LCQI_ARG_REPEAT = 3;
22 constexpr int LCQI_DEFAULT_REPEAT = 20;
23 constexpr std::int64_t LCQI_MAX_AUTO_ROWS = 4096;
24 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
25 constexpr float LCQI_INPUT_PATTERN_SCALE = 0.015625F;
26 constexpr int LCQI_INPUT_PATTERN_MODULUS = 31;
27 constexpr int LCQI_INPUT_PATTERN_CENTER = 15;
28
29 struct SelectedTensor {
30     const lcqi::GgufTensorInfo* tensor = nullptr;
31     std::int64_t rows = 0;
32     std::int64_t columns = 0;
33 };
34
35 std::int32_t parse_repeat(int argc, char** argv) {
36     if (argc <= LCQI_ARG_REPEAT) {
37         return LCQI_DEFAULT_REPEAT;

```

```

38     }
39     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_ARG_REPEAT]));
40     if (repeat <= 0) {
41         throw std::runtime_error("repeat must be positive");
42     }
43     return repeat;
44 }
45
46 bool is_q4_k_matrix(const lcqi::GgufTensorInfo& tensor) {
47     return tensor.type == lcqi::GgmlType::q4_k && tensor.shape.size() == 2 &&
48         tensor.shape[0] > 0 && tensor.shape[1] > 0 &&
49         tensor.shape[0] % lcqi::LCQI_QK_K_BLOCK_VALUES == 0;
50 }
51
52 bool preferred_tensor_name(std::string_view name) {
53     return name.find("ffn_up.weight") != std::string_view::npos ||
54         name.find("ffn_gate.weight") != std::string_view::npos ||
55         name.find("ffn_down.weight") != std::string_view::npos;
56 }
57
58 SelectedTensor select_tensor(const lcqi::GgufManifest& manifest, std::↵
↵ string_view requested) {
59     if (!requested.empty() && requested != "auto") {
60         const lcqi::GgufTensorInfo* tensor = manifest.find_tensor(requested);
61         if (tensor == nullptr) {
62             throw std::runtime_error("requested GGUF tensor not found");
63         }
64         if (!is_q4_k_matrix(*tensor)) {
65             throw std::runtime_error("requested GGUF tensor is not a rank-2 ↵
↵ Q4_K matrix");
66         }
67         return {tensor, tensor->shape[1], tensor->shape[0]};
68     }
69
70     const lcqi::GgufTensorInfo* best = nullptr;
71     for (const lcqi::GgufTensorInfo& tensor : manifest.tensors) {
72         if (!is_q4_k_matrix(tensor) || tensor.shape[1] > LCQI_MAX_AUTO_ROWS) {
73             continue;
74         }
75         if (best == nullptr) {
76             best = &tensor;
77             continue;
78         }
79         const bool tensor_preferred = preferred_tensor_name(tensor.name);
80         const bool best_preferred = preferred_tensor_name(best->name);
81         if ((tensor_preferred && !best_preferred) ||
82             (tensor_preferred == best_preferred &&
83              tensor.element_count() > best->element_count())) {
84             best = &tensor;
85         }
86     }

```

```

87     if (best == nullptr) {
88         throw std::runtime_error("no suitable rank-2 Q4_K tensor found in GGUF ←
↪ model");
89     }
90     return {best, best->shape[1], best->shape[0]};
91 }
92
93 std::vector<float> make_input(std::int64_t columns) {
94     std::vector<float> input(static_cast<std::size_t>(columns), 0.0F);
95     for (std::int64_t index = 0; index < columns; ++index) {
96         input[static_cast<std::size_t>(index)] =
97             static_cast<float>((index % LCQI_INPUT_PATTERN_MODULUS) -
98                               LCQI_INPUT_PATTERN_CENTER) *
99             LCQI_INPUT_PATTERN_SCALE;
100     }
101     return input;
102 }
103
104 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
105     if (lhs.size() != rhs.size()) {
106         throw std::runtime_error("max_abs_diff size mismatch");
107     }
108     float diff = 0.0F;
109     for (std::size_t index = 0; index < lhs.size(); ++index) {
110         diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
111     }
112     return diff;
113 }
114
115 float checksum(std::span<const float> values) {
116     float sum = 0.0F;
117     for (const float value : values) {
118         sum += value;
119     }
120     return sum;
121 }
122
123 void matvec_f32(std::span<const float> weights,
124                std::int64_t rows,
125                std::int64_t columns,
126                std::span<const float> input,
127                std::span<float> output) {
128     if (weights.size() != static_cast<std::size_t>(rows * columns) ||
129         input.size() != static_cast<std::size_t>(columns) ||
130         output.size() != static_cast<std::size_t>(rows)) {
131         throw std::runtime_error("F32 matvec shape mismatch");
132     }
133     for (std::int64_t row = 0; row < rows; ++row) {
134         const float* row_weights = weights.data() + row * columns;
135         float sum = 0.0F;
136         for (std::int64_t column = 0; column < columns; ++column) {
137             sum += row_weights[column] * input[static_cast<std::size_t>(column)←

```



```

    ↪ };
138     }
139     output[static_cast<std::size_t>(row)] = sum;
140 }
141 }
142
143 template <typename Callable>
144 double measure_average_us(std::int32_t repeat, Callable&& callable) {
145     const auto begin = std::chrono::steady_clock::now();
146     for (std::int32_t index = 0; index < repeat; ++index) {
147         callable();
148     }
149     const auto end = std::chrono::steady_clock::now();
150     const std::chrono::duration<double> elapsed = end - begin;
151     return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>↪
    ↪ >(repeat);
152 }
153
154 void print_usage(const char* program) {
155     std::cerr << "usage: " << program << " <model.gguf> [tensor-name|auto] [↪
    ↪ repeat]\n";
156 }
157
158 } // namespace
159
160 int main(int argc, char** argv) {
161     try {
162         if (argc <= LCQI_ARG_MODEL) {
163             print_usage(argv[0]);
164             return EXIT_FAILURE;
165         }
166         const std::filesystem::path model_path = argv[LCQI_ARG_MODEL];
167         const std::string tensor_request =
168             argc > LCQI_ARG_TENSOR ? argv[LCQI_ARG_TENSOR] : "auto";
169         const std::int32_t repeat = parse_repeat(argc, argv);
170
171         const lcqi::GgufManifest manifest = lcqi::load_gguf_manifest(model_path↪
    ↪ );
172         const SelectedTensor selected = select_tensor(manifest, tensor_request)↪
    ↪ ;
173         const std::vector<std::uint8_t> tensor_bytes =
174             lcqi::read_gguf_tensor_bytes(model_path, *selected.tensor);
175         const std::vector<float> input = make_input(selected.columns);
176         const std::vector<lcqi::Q8KBlock> q8_input = lcqi::quantize_q8_k_input(↪
    ↪ input);
177         std::vector<float> q4_f32_output(static_cast<std::size_t>(selected.rows↪
    ↪ ), 0.0F);
178         std::vector<float> q4_q8_output(q4_f32_output.size(), 0.0F);
179         std::vector<float> f32_output(q4_f32_output.size(), 0.0F);
180
181         lcqi::matvec_q4_k_f32(tensor_bytes,
182                               selected.rows,

```

```

183         selected.columns,
184         input,
185         q4_f32_output);
186     lcqi::matvec_q4_k_q8(tensor_bytes,
187         selected.rows,
188         selected.columns,
189         q8_input,
190         q4_q8_output);
191
192     const std::vector<float> f32_weights =
193         lcqi::dequantize_q4_k(tensor_bytes, selected.rows * selected.↵
↵ columns);
194     matvec_f32(f32_weights, selected.rows, selected.columns, input, ↵
↵ f32_output);
195
196     const double f32_us = measure_average_us(
197         repeat,
198         [&]() { matvec_f32(f32_weights, selected.rows, selected.columns, ↵
↵ input, f32_output); });
199     const double q4_f32_us = measure_average_us(
200         repeat,
201         [&]() {
202             lcqi::matvec_q4_k_f32(tensor_bytes,
203                 selected.rows,
204                 selected.columns,
205                 input,
206                 q4_f32_output);
207         });
208     const double q8_quantize_us = measure_average_us(
209         repeat,
210         [&]() {
211             const std::vector<lcqi::Q8KBlock> temporary =
212                 lcqi::quantize_q8_k_input(input);
213             static_cast<void>(temporary);
214         });
215     const double q4_q8_us = measure_average_us(
216         repeat,
217         [&]() {
218             lcqi::matvec_q4_k_q8(tensor_bytes,
219                 selected.rows,
220                 selected.columns,
221                 q8_input,
222                 q4_q8_output);
223         });
224
225     const double q4_q8_total_us = q8_quantize_us + q4_q8_us;
226     std::cout << std::fixed << std::setprecision(3);
227     std::cout << "lcqi_smollm2_q4_k_benchmark\n";
228     std::cout << "model_path=" << model_path.string() << '\n';
229     std::cout << "gguf_version=" << manifest.version << '\n';
230     std::cout << "gguf_alignment=" << manifest.alignment << '\n';
231     std::cout << "gguf_tensors=" << manifest.tensors.size() << '\n';

```

```

232     std::cout << "tensor_name=" << selected.tensor->name << '\n';
233     std::cout << "tensor_type=" << lcqi::ggml_type_name(selected.tensor->
↪ type) << '\n';
234     std::cout << "tensor_shape_ne0_ne1=" << selected.columns << 'x' << ↪
↪ selected.rows << '\n';
235     std::cout << "tensor_bytes=" << selected.tensor->byte_size << '\n';
236     std::cout << "f32_dequantized_bytes=" << f32_weights.size() * sizeof(↪
↪ float) << '\n';
237     std::cout << "repeat=" << repeat << '\n';
238     std::cout << "f32_matvec_average_us=" << f32_us << '\n';
239     std::cout << "q4_f32_input_average_us=" << q4_f32_us << '\n';
240     std::cout << "q8_quantize_input_average_us=" << q8_quantize_us << '\n';
241     std::cout << "q4_q8_backend=" << lcqi::q4_k_q8_active_backend() << '\n'↪
↪ ;
242     std::cout << "q4_q8_matvec_average_us=" << q4_q8_us << '\n';
243     std::cout << "q4_q8_total_average_us=" << q4_q8_total_us << '\n';
244     std::cout << "speedup_q4_q8_total_vs_f32="
245         << (q4_q8_total_us > 0.0 ? f32_us / q4_q8_total_us : 0.0) << ↪
↪ '\n';
246     std::cout << "speedup_q4_f32_input_vs_f32="
247         << (q4_f32_us > 0.0 ? f32_us / q4_f32_us : 0.0) << '\n';
248     std::cout << "q4_f32_vs_f32_max_abs_diff="
249         << max_abs_diff(q4_f32_output, f32_output) << '\n';
250     std::cout << "q4_q8_vs_q4_f32_max_abs_diff="
251         << max_abs_diff(q4_q8_output, q4_f32_output) << '\n';
252     std::cout << "q4_q8_checksum=" << checksum(q4_q8_output) << '\n';
253     return EXIT_SUCCESS;
254 } catch (const std::exception& error) {
255     std::cerr << "error: " << error.what() << '\n';
256     return EXIT_FAILURE;
257 }
258 }

```

`ggml_tensors.cpp` 是真实 GGUF 混合量化进入 LCQI 的格式闸门。它按 GGML block 布局解码 `Q8_0`、`Q5_0`、`Q6_K` 和已有 `Q4_K`，也读取 `F32` 标量张量。测试用手造 block 固定每个格式的位布局，真实 `SmolLM2 smoke` 再验证同一个解码层能读完 272 个 tensor。

Listing 16.8 `src/ggml_tensors.cpp`

```

1 #include <lcqi/ggml_tensors.hpp>
2
3 #include <lcqi/q4_k.hpp>
4
5 #include <cmath>
6 #include <cstring>
7 #include <limits>
8 #include <stdexcept>
9 #include <string>
10
11 namespace lcqi {
12 namespace {
13

```

```

14 constexpr std::uint32_t LCQI_GGML_F32_EXPONENT_MASK = 0x7F800000U;
15 constexpr std::uint32_t LCQI_GGML_F16_SIGN_MASK = 0x8000U;
16 constexpr std::uint32_t LCQI_GGML_F16_EXPONENT_MASK = 0x7C00U;
17 constexpr std::uint32_t LCQI_GGML_F16_MANTISSA_MASK = 0x03FFU;
18 constexpr std::uint32_t LCQI_GGML_F16_EXPONENT_BIAS = 15U;
19 constexpr std::uint32_t LCQI_GGML_F32_EXPONENT_BIAS = 127U;
20 constexpr std::uint32_t LCQI_GGML_F32_MANTISSA_BITS = 23U;
21 constexpr std::uint32_t LCQI_GGML_F16_MANTISSA_BITS = 10U;
22 constexpr std::uint32_t LCQI_GGML_F16_T0_F32_SIGN_SHIFT = 16U;
23 constexpr std::uint32_t LCQI_GGML_BYTE_BITS = 8U;
24 constexpr std::uint8_t LCQI_GGML_LOW_NIBBLE_MASK = 0x0FU;
25 constexpr std::uint8_t LCQI_GGML_Q5_HIGH_BIT_MASK = 0x10U;
26 constexpr std::uint32_t LCQI_GGML_Q5_HIGH_BIT_SHIFT = 4U;
27 constexpr std::int32_t LCQI_GGML_Q5_SECOND_HALF_BIT_OFFSET = 16;
28 constexpr std::uint8_t LCQI_GGML_Q6_HIGH_TWO_BITS_MASK = 0x03U;
29 constexpr std::int32_t LCQI_GGML_Q5_ZERO_POINT = 16;
30 constexpr std::int32_t LCQI_GGML_Q6_ZERO_POINT = 32;
31 constexpr std::int32_t LCQI_GGML_Q6_HALF_BLOCK_VALUES = 128;
32 constexpr std::int32_t LCQI_GGML_Q6_QUARTER_BLOCK_VALUES = 64;
33 constexpr std::int32_t LCQI_GGML_Q6_SUBBLOCK_VALUES = 32;
34 constexpr std::int32_t LCQI_GGML_Q6_SCALE_STRIDE = 8;
35 constexpr std::int32_t LCQI_GGML_F32_BYTES = 4;
36
37 [[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
38     if (value < 0 ||
39         static_cast<std::uint64_t>(value) >
40         static_cast<std::uint64_t>(std::numeric_limits<std::size_t>::max())) {
41         throw std::runtime_error(std::string(name) + " is outside size_t range");
42     }
43     return static_cast<std::size_t>(value);
44 }
45
46 void require_multiple(std::int64_t value, std::int64_t divisor, const char*
47     name) {
48     if (value <= 0 || value % divisor != 0) {
49         throw std::runtime_error(std::string(name) + " must be a positive
50             multiple of " +
51             std::to_string(divisor));
52     }
53 }
54
55 [[nodiscard]] std::uint16_t read_le_u16(const std::uint8_t* bytes) {
56     return static_cast<std::uint16_t>(
57         static_cast<std::uint16_t>(bytes[0]) |
58         (static_cast<std::uint16_t>(bytes[1]) << LCQI_GGML_BYTE_BITS));
59 }
60
61 [[nodiscard]] std::uint32_t read_le_u32(const std::uint8_t* bytes) {
62     std::uint32_t value = 0;
63     for (std::uint32_t index = 0; index < LCQI_GGML_F32_BYTES; ++index) {

```

```

62     value |= static_cast<std::uint32_t>(bytes[index]) <<
63             (LCQI_GGML_BYTE_BITS * index);
64 }
65 return value;
66 }
67
68 [[nodiscard]] float bits_to_float(std::uint32_t bits) {
69     float value = 0.0F;
70     std::memcpy(&value, &bits, sizeof(value));
71     return value;
72 }
73
74 void validate_quantized_bytes(std::span<const std::uint8_t> bytes,
75                               std::int64_t element_count,
76                               std::int64_t block_values,
77                               std::int64_t block_bytes,
78                               const char* name) {
79     require_multiple(element_count, block_values, name);
80     const std::int64_t expected_bytes = (element_count / block_values) * ↵
↵ block_bytes;
81     if (bytes.size() != checked_size(expected_bytes, name)) {
82         throw std::runtime_error(std::string(name) + " byte count does not ↵
↵ match element count");
83     }
84 }
85
86 void dequantize_f32_to(std::span<const std::uint8_t> bytes, std::span<float> ↵
↵ output) {
87     const std::size_t expected_bytes = output.size() * LCQI_GGML_F32_BYTES;
88     if (bytes.size() != expected_bytes) {
89         throw std::runtime_error("F32 byte count does not match element count")↵
↵ ;
90     }
91     for (std::size_t index = 0; index < output.size(); ++index) {
92         output[index] = bits_to_float(
93             read_le_u32(bytes.data() + index * LCQI_GGML_F32_BYTES));
94     }
95 }
96
97 void dequantize_q8_0_to(std::span<const std::uint8_t> bytes, std::span<float> ↵
↵ output) {
98     validate_quantized_bytes(bytes,
99                               static_cast<std::int64_t>(output.size()),
100                               LCQI_Q8_0_BLOCK_VALUES,
101                               LCQI_Q8_0_BLOCK_BYTES,
102                               "Q8_0");
103     const std::int64_t block_count =
104         static_cast<std::int64_t>(output.size()) / LCQI_Q8_0_BLOCK_VALUES;
105     for (std::int64_t block_index = 0; block_index < block_count; ++block_index↵
↵ ) {
106         const std::uint8_t* block =
107             bytes.data() + checked_size(block_index * LCQI_Q8_0_BLOCK_BYTES, "↵

```

```

    ↪ Q8_0 offset");
108     const float d = ggml_f16_to_f32(read_le_u16(block));
109     const std::uint8_t* qs = block + sizeof(std::uint16_t);
110     float* target =
111         output.data() + checked_size(block_index * LCQI_Q8_0_BLOCK_VALUES, ↪
    ↪ "Q8_0 target");
112     for (std::int32_t index = 0; index < LCQI_Q8_0_BLOCK_VALUES; ++index) {
113         target[index] = static_cast<float>(static_cast<std::int8_t>(qs[↪
    ↪ index])) * d;
114     }
115 }
116 }
117
118 void dequantize_q5_0_to(std::span<const std::uint8_t> bytes, std::span<float> ↪
    ↪ output) {
119     validate_quantized_bytes(bytes,
120                             static_cast<std::int64_t>(output.size()),
121                             LCQI_Q5_0_BLOCK_VALUES,
122                             LCQI_Q5_0_BLOCK_BYTES,
123                             "Q5_0");
124     const std::int64_t block_count =
125         static_cast<std::int64_t>(output.size()) / LCQI_Q5_0_BLOCK_VALUES;
126     for (std::int64_t block_index = 0; block_index < block_count; ++block_index ↪
    ↪ ) {
127         const std::uint8_t* block =
128             bytes.data() + checked_size(block_index * LCQI_Q5_0_BLOCK_BYTES, "↪
    ↪ Q5_0 offset");
129         const float d = ggml_f16_to_f32(read_le_u16(block));
130         const std::uint32_t qh = read_le_u32(block + sizeof(std::uint16_t));
131         const std::uint8_t* qs = block + sizeof(std::uint16_t) + sizeof(std::↪
    ↪ uint32_t);
132         float* target =
133             output.data() + checked_size(block_index * LCQI_Q5_0_BLOCK_VALUES, ↪
    ↪ "Q5_0 target");
134         for (std::int32_t index = 0; index < LCQI_Q5_0_BLOCK_VALUES / 2; ++↪
    ↪ index) {
135             const std::uint8_t high_0 =
136                 static_cast<std::uint8_t>(((qh >> index) & 1U)
137                                         << LCQI_GGML_Q5_HIGH_BIT_SHIFT);
138             const std::uint8_t high_1 =
139                 static_cast<std::uint8_t>(((qh >> (index + ↪
    ↪ LCQI_GGML_Q5_SECOND_HALF_BIT_OFFSET))) &
140                                         1U)
141                                         << LCQI_GGML_Q5_HIGH_BIT_SHIFT);
142             const std::int32_t low =
143                 static_cast<std::int32_t>((qs[index] & ↪
    ↪ LCQI_GGML_LOW_NIBBLE_MASK) | high_0) -
144                 LCQI_GGML_Q5_ZERO_POINT;
145             const std::int32_t high =
146                 static_cast<std::int32_t>((qs[index] >> 4U) | high_1) -
147                 LCQI_GGML_Q5_ZERO_POINT;
148             target[index] = static_cast<float>(low) * d;

```

[illegible]


```

↪ )
189                                     << 4U)) -
190         LCQI_GGML_Q6_ZERO_POINT;
191     const std::int32_t q3 =
192         static_cast<std::int32_t>((ql[lane] >> 4U) |
193                                     (((qh[lane] >> 4U) &
194                                     LCQI_GGML_Q6_HIGH_TWO_BITS_MASK)
↪ )
195                                     << 4U)) -
196         LCQI_GGML_Q6_ZERO_POINT;
197     const std::int32_t q4 =
198         static_cast<std::int32_t>((ql[lane + ↪
↪ LCQI_GGML_Q6_SUBBLOCK_VALUES] >> 4U) |
199                                     (((qh[lane] >> 6U) &
200                                     LCQI_GGML_Q6_HIGH_TWO_BITS_MASK)
↪ )
201                                     << 4U)) -
202         LCQI_GGML_Q6_ZERO_POINT;
203
204     y[lane] = d * static_cast<float>(scales[scale_index + 0]) *
205                 static_cast<float>(q1);
206     y[lane + LCQI_GGML_Q6_SUBBLOCK_VALUES] =
207         d * static_cast<float>(scales[scale_index + 2]) *
208             static_cast<float>(q2);
209     y[lane + LCQI_GGML_Q6_QUARTER_BLOCK_VALUES] =
210         d * static_cast<float>(scales[scale_index + 4]) *
211             static_cast<float>(q3);
212     y[lane + LCQI_GGML_Q6_QUARTER_BLOCK_VALUES +
213         LCQI_GGML_Q6_SUBBLOCK_VALUES] =
214         d * static_cast<float>(scales[scale_index + 6]) *
215             static_cast<float>(q4);
216 }
217 y += LCQI_GGML_Q6_HALF_BLOCK_VALUES;
218 ql += LCQI_GGML_Q6_QUARTER_BLOCK_VALUES;
219 qh += LCQI_GGML_Q6_SUBBLOCK_VALUES;
220 scales += LCQI_GGML_Q6_SCALE_STRIDE;
221 }
222 }
223 }
224
225 } // namespace
226
227 float ggml_f16_to_f32(std::uint16_t bits) {
228     const std::uint32_t sign =
229         (static_cast<std::uint32_t>(bits) & LCQI_GGML_F16_SIGN_MASK)
230         << LCQI_GGML_F16_TO_F32_SIGN_SHIFT;
231     const std::uint32_t exponent =
232         static_cast<std::uint32_t>(bits) & LCQI_GGML_F16_EXPONENT_MASK;
233     const std::uint32_t mantissa =
234         static_cast<std::uint32_t>(bits) & LCQI_GGML_F16_MANTISSA_MASK;
235
236     if (exponent == 0U) {

```



```

237     if (mantissa == 0U) {
238         return bits_to_float(sign);
239     }
240     float value = static_cast<float>(mantissa) /
241                 static_cast<float>(1U << LCQI_GGML_F16_MANTISSA_BITS);
242     value = std::ldexp(value, 1 - static_cast<int>(↵
↵ LCQI_GGML_F16_EXPONENT_BIAS));
243     return sign == 0U ? value : -value;
244 }
245
246 if (exponent == LCQI_GGML_F16_EXPONENT_MASK) {
247     const std::uint32_t f32_mantissa =
248         mantissa << (LCQI_GGML_F32_MANTISSA_BITS - ↵
↵ LCQI_GGML_F16_MANTISSA_BITS);
249     return bits_to_float(sign | LCQI_GGML_F32_EXPONENT_MASK | f32_mantissa)↵
↵ ;
250 }
251
252 const std::uint32_t f32_exponent =
253     ((exponent >> LCQI_GGML_F16_MANTISSA_BITS) - ↵
↵ LCQI_GGML_F16_EXPONENT_BIAS +
254     LCQI_GGML_F32_EXPONENT_BIAS)
255     << LCQI_GGML_F32_MANTISSA_BITS;
256 const std::uint32_t f32_mantissa =
257     mantissa << (LCQI_GGML_F32_MANTISSA_BITS - LCQI_GGML_F16_MANTISSA_BITS)↵
↵ ;
258 return bits_to_float(sign | f32_exponent | f32_mantissa);
259 }
260
261 std::vector<float> dequantize_ggml_tensor(GgmlType type,
262                                         std::span<const std::uint8_t> bytes,
263                                         std::int64_t element_count) {
264     if (element_count < 0) {
265         throw std::runtime_error("GGML element_count cannot be negative");
266     }
267     std::vector<float> output(checked_size(element_count, "GGML element_count")↵
↵ , 0.0F);
268     dequantize_ggml_tensor_to(type, bytes, output);
269     return output;
270 }
271
272 void dequantize_ggml_tensor_to(GgmlType type,
273                               std::span<const std::uint8_t> bytes,
274                               std::span<float> output) {
275     switch (type) {
276     case GgmlType::f32:
277         dequantize_f32_to(bytes, output);
278         return;
279     case GgmlType::q8_0:
280         dequantize_q8_0_to(bytes, output);
281         return;
282     case GgmlType::q5_0:

```

```

283         dequantize_q5_0_to(bytes, output);
284         return;
285     case GgmlType::q6_k:
286         dequantize_q6_k_to(bytes, output);
287         return;
288     case GgmlType::q4_k:
289         dequantize_q4_k_to(bytes, output);
290         return;
291     default:
292         throw std::runtime_error(std::string("LCQI cannot dequantize GGML ↵
↵ type ") +
293                                     ggml_type_name(type));
294     }
295 }
296
297 } // namespace lcqi

```

`ggml_matvec.cpp` 是普通 Q8_0 direct 与 实验 Q5_0/Q6_K/Q8_0 direct matvec 的共享实现。它有两条 oracle：一条是 `matvec_ggml_quantized_f32`，直接从 GGUF block bytes 和 f32 input 做点积，用来对照“先解成 f32 权重再算”；另一条是 `matvec_ggml_quantized_q8_0`，先把 input 临时量化成 `Q8_0InputBlock`，再做低比特权重点积。Q5_0 需要用 input block 的 `sum` 减掉 zero-point 贡献，Q6_K 需要按 256 元素 block 的 scale lane 解包。当前版本还提供 row-range、batch-row 与 fused max-dot 三个 unchecked 入口：row-range 把同一个矩阵的输出行交给 worker pool；batch-row 让 prompt prefill 在一个函数内处理多个 token；max-dot 专门服务 tied lm head 的 greedy argmax，直接返回最大 row 和 value，避免先写完整 vocab logits 再读回扫描。测试会同时比较 exact path、Q8_0 近似 path、拆成两段的 row-range path、batch path 和 max-dot path，避免把 block 布局、输出 row offset、batch stride 或 argmax 行号写错后直接进入真实模型。

Listing 16.9 src/ggml_matvec.cpp

```

1  #include <lcqi/ggml_matvec.hpp>
2
3  #include <lcqi/ggml_tensors.hpp>
4
5  #include <algorithm>
6  #include <cmath>
7  #include <limits>
8  #include <stdexcept>
9  #include <string>
10
11 namespace lcqi {
12 namespace {
13
14 constexpr std::uint32_t LCQI_GGML_MATVEC_BYTE_BITS = 8U;
15 constexpr std::uint8_t LCQI_GGML_MATVEC_LOW_NIBBLE_MASK = 0x0FU;
16 constexpr std::uint8_t LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK = 0x03U;
17 constexpr std::int32_t LCQI_GGML_MATVEC_Q5_ZERO_POINT = 16;
18 constexpr std::int32_t LCQI_GGML_MATVEC_Q6_ZERO_POINT = 32;
19 constexpr std::int32_t LCQI_GGML_MATVEC_Q6_HALF_BLOCK_VALUES = 128;

```

```

20 constexpr std::int32_t LCQI_GGML_MATVEC_Q6_QUARTER_BLOCK_VALUES = 64;
21 constexpr std::int32_t LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES = 32;
22 constexpr std::int32_t LCQI_GGML_MATVEC_Q6_SCALE_STRIDE = 8;
23 constexpr std::int32_t LCQI_GGML_MATVEC_Q8_QUANT_MAX = 127;
24
25 [[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
26     if (value < 0 ||
27         static_cast<std::uint64_t>(value) >
28         static_cast<std::uint64_t>(std::numeric_limits<std::size_t>::max())) ←
29     ↪ ) {
30         throw std::runtime_error(std::string(name) + " is outside size_t range" ←
31     ↪ );
32     }
33     return static_cast<std::size_t>(value);
34 }
35
36 void require_positive(std::int64_t value, const char* name) {
37     if (value <= 0) {
38         throw std::runtime_error(std::string(name) + " must be positive");
39     }
40 }
41
42 void require_multiple(std::int64_t value, std::int64_t divisor, const char* ←
43     ↪ name) {
44     if (value <= 0 || value % divisor != 0) {
45         throw std::runtime_error(std::string(name) + " must be a positive ←
46     ↪ multiple of " +
47         std::to_string(divisor));
48     }
49 }
50
51 [[nodiscard]] std::uint16_t read_le_u16(const std::uint8_t* bytes) {
52     return static_cast<std::uint16_t>(
53         static_cast<std::uint16_t>(bytes[0]) |
54         (static_cast<std::uint16_t>(bytes[1]) << LCQI_GGML_MATVEC_BYTE_BITS));
55 }
56
57 [[nodiscard]] std::uint32_t read_le_u32(const std::uint8_t* bytes) {
58     std::uint32_t value = 0;
59     for (std::uint32_t index = 0; index < sizeof(std::uint32_t); ++index) {
60         value |= static_cast<std::uint32_t>(bytes[index]) <<
61             (LCQI_GGML_MATVEC_BYTE_BITS * index);
62     }
63     return value;
64 }
65
66 [[nodiscard]] float dot_q8_0_block_f32(const std::uint8_t* block, const float* ←
67     ↪ input) {
68     const float d = ggml_f16_to_f32(read_le_u16(block));
69     const auto* qs = reinterpret_cast<const std::int8_t*>(block + sizeof(std::←
70     ↪ uint16_t));
71     float sum = 0.0F;

```

```

66     for (std::int32_t index = 0; index < LCQI_Q8_0_BLOCK_VALUES; ++index) {
67         sum += static_cast<float>(qs[index]) * input[index];
68     }
69     return d * sum;
70 }
71
72 [[nodiscard]] float dot_q8_0_block_q8_0(const std::uint8_t* block,
73                                         const Q8_0InputBlock& input) {
74     const float d = ggml_f16_to_f32(read_le_u16(block));
75     const auto* qs = reinterpret_cast<const std::int8_t*>(block + sizeof(std::uint16_t));
76     std::int32_t sum = 0;
77     for (std::int32_t index = 0; index < LCQI_Q8_0_BLOCK_VALUES; ++index) {
78         sum += static_cast<std::int32_t>(qs[index]) *
79             static_cast<std::int32_t>(input.qs[index]);
80     }
81     return d * input.d * static_cast<float>(sum);
82 }
83
84 [[nodiscard]] float dot_q5_0_block_f32(const std::uint8_t* block, const float* input) {
85     const float d = ggml_f16_to_f32(read_le_u16(block));
86     const std::uint32_t qh = read_le_u32(block + sizeof(std::uint16_t));
87     const std::uint8_t* qs = block + sizeof(std::uint16_t) + sizeof(std::uint32_t);
88     float sum = 0.0F;
89     for (std::int32_t index = 0; index < LCQI_Q5_0_BLOCK_VALUES / 2; ++index) {
90         const std::uint8_t high_0 =
91             static_cast<std::uint8_t>(((qh >> index) & 1U) << 4U);
92         const std::uint8_t high_1 =
93             static_cast<std::uint8_t>(((qh >> (index + LCQI_Q5_0_BLOCK_VALUES / 2)) & 1U)
94                                     << 4U);
95         const std::int32_t first =
96             static_cast<std::int32_t>((qs[index] & LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
97                                     high_0) -
98             LCQI_GGML_MATVEC_Q5_ZERO_POINT;
99         const std::int32_t second =
100             static_cast<std::int32_t>((qs[index] >> 4U) | high_1) -
101             LCQI_GGML_MATVEC_Q5_ZERO_POINT;
102         sum += static_cast<float>(first) * input[index];
103         sum += static_cast<float>(second) * input[index + LCQI_Q5_0_BLOCK_VALUES / 2];
104     }
105     return d * sum;
106 }
107
108 [[nodiscard]] float dot_q5_0_block_q8_0(const std::uint8_t* block,
109                                         const Q8_0InputBlock& input) {
110     const float d = ggml_f16_to_f32(read_le_u16(block));
111     const std::uint32_t qh = read_le_u32(block + sizeof(std::uint16_t));

```

```

112     const std::uint8_t* qs = block + sizeof(std::uint16_t) + sizeof(std::uint32_t);
113     std::int32_t weighted_sum = 0;
114     for (std::int32_t index = 0; index < LCQI_Q5_0_BLOCK_VALUES / 2; ++index) {
115         const std::uint8_t high_0 =
116             static_cast<std::uint8_t>(((qh >> index) & 1U) << 4U);
117         const std::uint8_t high_1 =
118             static_cast<std::uint8_t>(((qh >> (index + LCQI_Q5_0_BLOCK_VALUES / 2)) & 1U)
119                                     << 4U);
120         const std::int32_t first =
121             static_cast<std::int32_t>((qs[index] & LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
122                                     high_0);
123         const std::int32_t second =
124             static_cast<std::int32_t>((qs[index] >> 4U) | high_1);
125         weighted_sum += first * static_cast<std::int32_t>(input.qs[index]);
126         weighted_sum += second *
127             static_cast<std::int32_t>(input.qs[index + LCQI_Q5_0_BLOCK_VALUES / 2]);
128     }
129     weighted_sum -= LCQI_GGML_MATVEC_Q5_ZERO_POINT * static_cast<std::int32_t>(input.sum);
130     return d * input.d * static_cast<float>(weighted_sum);
131 }
132
133 [[nodiscard]] float dot_q6_k_block_f32(const std::uint8_t* block, const float* input) {
134     const std::uint8_t* ql = block;
135     const std::uint8_t* qh = ql + LCQI_Q6_K_BLOCK_VALUES / 2;
136     const auto* scales =
137         reinterpret_cast<const std::int8_t*>(qh + LCQI_Q6_K_BLOCK_VALUES / 4);
138     const float d = ggml_f16_to_f32(
139         read_le_u16(block + LCQI_Q6_K_BLOCK_BYTES - sizeof(std::uint16_t)));
140     float sum = 0.0F;
141
142     for (std::int32_t half = 0; half < LCQI_Q6_K_BLOCK_VALUES;
143         half += LCQI_GGML_MATVEC_Q6_HALF_BLOCK_VALUES) {
144         const float* x = input + half;
145         for (std::int32_t lane = 0; lane < LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES; ++lane) {
146             const std::int32_t scale_index =
147                 lane / (LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES / 2);
148             const std::int32_t q1 =
149                 static_cast<std::int32_t>((ql[lane] & LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
150                                     (((qh[lane] >> 0U) &
151                                     LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
152                                     << 4U)) -
153                 LCQI_GGML_MATVEC_Q6_ZERO_POINT;
154             const std::int32_t q2 =

```

```

155         static_cast<std::int32_t>((ql[lane + ↵
↵ LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES] &
156             LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
157             (((qh[lane] >> 2U) &
158             ↵
↵ LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
159             << 4U)) -
160             LCQI_GGML_MATVEC_Q6_ZERO_POINT;
161         const std::int32_t q3 =
162             static_cast<std::int32_t>((ql[lane] >> 4U) |
163             (((qh[lane] >> 4U) &
164             ↵
↵ LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
165             << 4U)) -
166             LCQI_GGML_MATVEC_Q6_ZERO_POINT;
167         const std::int32_t q4 =
168             static_cast<std::int32_t>((ql[lane + ↵
↵ LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES] >>
169             4U) |
170             (((qh[lane] >> 6U) &
171             ↵
↵ LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
172             << 4U)) -
173             LCQI_GGML_MATVEC_Q6_ZERO_POINT;
174
175         sum += static_cast<float>(scales[scale_index + 0]) *
176             static_cast<float>(q1) * x[lane];
177         sum += static_cast<float>(scales[scale_index + 2]) *
178             static_cast<float>(q2) * x[lane + ↵
↵ LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES];
179         sum += static_cast<float>(scales[scale_index + 4]) *
180             static_cast<float>(q3) * x[lane + ↵
↵ LCQI_GGML_MATVEC_Q6_QUARTER_BLOCK_VALUES];
181         sum += static_cast<float>(scales[scale_index + 6]) *
182             static_cast<float>(q4) *
183             x[lane + LCQI_GGML_MATVEC_Q6_QUARTER_BLOCK_VALUES +
184             LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES];
185     }
186     ql += LCQI_GGML_MATVEC_Q6_QUARTER_BLOCK_VALUES;
187     qh += LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES;
188     scales += LCQI_GGML_MATVEC_Q6_SCALE_STRIDE;
189 }
190 return d * sum;
191 }
192
193 [[nodiscard]] float dot_q6_k_block_q8_0(const std::uint8_t* block,
194                                         const Q8_0InputBlock* input) {
195     const std::uint8_t* ql = block;
196     const std::uint8_t* qh = ql + LCQI_Q6_K_BLOCK_VALUES / 2;
197     const auto* scales =
198         reinterpret_cast<const std::int8_t*>(qh + LCQI_Q6_K_BLOCK_VALUES / 4);
199     const float d = ggml_f16_to_f32(

```

```

200     read_le_u16(block + LCQI_Q6_K_BLOCK_BYTES - sizeof(std::uint16_t)));
201     float sum = 0.0F;
202
203     for (std::int32_t half = 0; half < LCQI_Q6_K_BLOCK_VALUES;
204          half += LCQI_GGML_MATVEC_Q6_HALF_BLOCK_VALUES) {
205         const std::int32_t input_half_block = half / ↵
↵ LCQI_Q8_0_INPUT_BLOCK_VALUES;
206         for (std::int32_t lane = 0; lane < LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES; ↵
↵ ++lane) {
207             const std::int32_t scale_index =
208                 lane / (LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES / 2);
209             const std::int32_t q1 =
210                 static_cast<std::int32_t>((ql[lane] & ↵
↵ LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
211                                         (((qh[lane] >> 0U) &
212                                         ↵
↵ LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
213                                         << 4U)) -
214                 LCQI_GGML_MATVEC_Q6_ZERO_POINT;
215             const std::int32_t q2 =
216                 static_cast<std::int32_t>((ql[lane + ↵
↵ LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES] &
217                                         LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
218                                         (((qh[lane] >> 2U) &
219                                         ↵
↵ LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
220                                         << 4U)) -
221                 LCQI_GGML_MATVEC_Q6_ZERO_POINT;
222             const std::int32_t q3 =
223                 static_cast<std::int32_t>((ql[lane] >> 4U) |
224                                         (((qh[lane] >> 4U) &
225                                         ↵
↵ LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
226                                         << 4U)) -
227                 LCQI_GGML_MATVEC_Q6_ZERO_POINT;
228             const std::int32_t q4 =
229                 static_cast<std::int32_t>((ql[lane + ↵
↵ LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES] >>
230                                         4U) |
231                                         (((qh[lane] >> 6U) &
232                                         ↵
↵ LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
233                                         << 4U)) -
234                 LCQI_GGML_MATVEC_Q6_ZERO_POINT;
235
236             const Q8_0InputBlock& x1 = input[input_half_block + 0];
237             const Q8_0InputBlock& x2 = input[input_half_block + 1];
238             const Q8_0InputBlock& x3 = input[input_half_block + 2];
239             const Q8_0InputBlock& x4 = input[input_half_block + 3];
240             sum += x1.d * static_cast<float>(scales[scale_index + 0]) *
241                 static_cast<float>(q1) * static_cast<float>(x1.qs[lane]);
242             sum += x2.d * static_cast<float>(scales[scale_index + 2]) *

```



```

243         static_cast<float>(q2) * static_cast<float>(x2.qs[lane]);
244         sum += x3.d * static_cast<float>(scales[scale_index + 4]) *
245             static_cast<float>(q3) * static_cast<float>(x3.qs[lane]);
246         sum += x4.d * static_cast<float>(scales[scale_index + 6]) *
247             static_cast<float>(q4) * static_cast<float>(x4.qs[lane]);
248     }
249     ql += LCQI_GGML_MATVEC_Q6_QUARTER_BLOCK_VALUES;
250     qh += LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES;
251     scales += LCQI_GGML_MATVEC_Q6_SCALE_STRIDE;
252 }
253 return d * sum;
254 }
255
256 [[nodiscard]] float dot_quantized_block_f32(GgmlType type,
257                                             const std::uint8_t* block,
258                                             const float* input) {
259     switch (type) {
260         case GgmlType::q8_0:
261             return dot_q8_0_block_f32(block, input);
262         case GgmlType::q5_0:
263             return dot_q5_0_block_f32(block, input);
264         case GgmlType::q6_k:
265             return dot_q6_k_block_f32(block, input);
266         default:
267             throw std::runtime_error(std::string("LCQI has no direct F32 matvec ←
↪ for ") +
268                                     ggml_type_name(type));
269     }
270 }
271
272 [[nodiscard]] float dot_quantized_block_q8_0(GgmlType type,
273                                             const std::uint8_t* block,
274                                             const Q8_0InputBlock* input) {
275     switch (type) {
276         case GgmlType::q8_0:
277             return dot_q8_0_block_q8_0(block, input[0]);
278         case GgmlType::q5_0:
279             return dot_q5_0_block_q8_0(block, input[0]);
280         case GgmlType::q6_k:
281             return dot_q6_k_block_q8_0(block, input);
282         default:
283             throw std::runtime_error(std::string("LCQI has no direct Q8_0 ←
↪ matvec for ") +
284                                     ggml_type_name(type));
285     }
286 }
287
288 void validate_q8_0_blocks(std::span<const Q8_0InputBlock> input, std::int64_t ←
↪ column_count) {
289     require_multiple(column_count, LCQI_Q8_0_INPUT_BLOCK_VALUES, "Q8_0 input ←
↪ column count");
290     if (input.size() !=

```



```

291     checked_size(column_count / LCQI_Q8_0_INPUT_BLOCK_VALUES, "Q8_0 input ↵
↵ block count")) {
292         throw std::runtime_error("Q8_0 input block count mismatch");
293     }
294 }
295
296 } // namespace
297
298 bool ggml_type_has_f32_direct_matvec(GgmlType type) noexcept {
299     return type == GgmlType::q8_0 || type == GgmlType::q5_0 || type == GgmlType↵
↵ ::q6_k;
300 }
301
302 bool ggml_type_has_q8_0_direct_matvec(GgmlType type) noexcept {
303     return type == GgmlType::q8_0 || type == GgmlType::q5_0 || type == GgmlType↵
↵ ::q6_k;
304 }
305
306 #if defined(LCQI_ENABLE_AVX2)
307 bool ggml_q8_0_avx2_available() noexcept;
308
309 void matvec_ggml_quantized_q8_0_avx2_unchecked(GgmlType type,
310                                                 std::span<const std::uint8_t> ↵
↵ rows,
311                                                 std::int64_t row_count,
312                                                 std::int64_t column_count,
313                                                 std::span<const Q8_0InputBlock> ↵
↵ input,
314                                                 std::span<float> output);
315
316 void matvec_ggml_quantized_q8_0_avx2_rows_unchecked(GgmlType type,
317                                                     std::span<const std::↵
↵ uint8_t> rows,
318                                                     std::int64_t row_count,
319                                                     std::int64_t column_count,
320                                                     std::span<const ↵
↵ Q8_0InputBlock> input,
321                                                     std::span<float> output,
322                                                     std::int64_t row_begin,
323                                                     std::int64_t row_end);
324
325 void matvec_ggml_quantized_q8_0_avx2_batch_rows_unchecked(
326     GgmlType type,
327     std::span<const std::uint8_t> rows,
328     std::int64_t row_count,
329     std::int64_t column_count,
330     std::span<const Q8_0InputBlock> input,
331     std::int64_t batch_size,
332     std::span<float> output,
333     std::int64_t output_stride,
334     std::int64_t row_begin,
335     std::int64_t row_end);

```

```

336
337 F32RowMax max_dot_ggml_quantized_q8_0_avx2_rows_unchecked(
338     GgmlType type,
339     std::span<const std::uint8_t> rows,
340     std::int64_t row_count,
341     std::int64_t column_count,
342     std::span<const Q8_0InputBlock> input,
343     std::int64_t row_begin,
344     std::int64_t row_end);
345 #else
346 bool ggml_q8_0_avx2_available() noexcept {
347     return false;
348 }
349
350 void matvec_ggml_quantized_q8_0_avx2_unchecked(GgmlType,
351                                             std::span<const std::uint8_t>,
352                                             std::int64_t,
353                                             std::int64_t,
354                                             std::span<const Q8_0InputBlock>,
355                                             std::span<float>) {}
356
357 void matvec_ggml_quantized_q8_0_avx2_rows_unchecked(GgmlType,
358                                             std::span<const std::uint8_t>,
359                                             std::int64_t,
360                                             std::int64_t,
361                                             std::span<const Q8_0InputBlock>,
362                                             std::span<float>,
363                                             std::int64_t,
364                                             std::int64_t) {}
365
366 void matvec_ggml_quantized_q8_0_avx2_batch_rows_unchecked(
367     GgmlType,
368     std::span<const std::uint8_t>,
369     std::int64_t,
370     std::int64_t,
371     std::span<const Q8_0InputBlock>,
372     std::int64_t,
373     std::span<float>,
374     std::int64_t,
375     std::int64_t,
376     std::int64_t) {}
377
378 F32RowMax max_dot_ggml_quantized_q8_0_avx2_rows_unchecked(
379     GgmlType,
380     std::span<const std::uint8_t>,
381     std::int64_t,
382     std::int64_t,
383     std::span<const Q8_0InputBlock>,
384     std::int64_t,
385     std::int64_t) {

```

```

386     F32RowMax best;
387     best.value = -std::numeric_limits<float>::infinity();
388     return best;
389 }
390 #endif
391
392 std::vector<Q8_0InputBlock> quantize_q8_0_input(std::span<const float> input) {
393     require_multiple(static_cast<std::int64_t>(input.size()),
394                     LCQI_Q8_0_INPUT_BLOCK_VALUES,
395                     "Q8_0 input size");
396     std::vector<Q8_0InputBlock> output(
397         checked_size(static_cast<std::int64_t>(input.size()) / ←
    ↪ LCQI_Q8_0_INPUT_BLOCK_VALUES,
398                     "Q8_0 input block count"));
399     quantize_q8_0_input_to(input, output);
400     return output;
401 }
402
403 void quantize_q8_0_input_to(std::span<const float> input,
404                             std::span<Q8_0InputBlock> output) {
405     require_multiple(static_cast<std::int64_t>(input.size()),
406                     LCQI_Q8_0_INPUT_BLOCK_VALUES,
407                     "Q8_0 input size");
408     if (output.size() != checked_size(static_cast<std::int64_t>(input.size()) /
409                                     LCQI_Q8_0_INPUT_BLOCK_VALUES,
410                                     "Q8_0 input block count")) {
411         throw std::runtime_error("Q8_0 output block count mismatch");
412     }
413     for (std::size_t block_index = 0; block_index < output.size(); ++↵
    ↪ block_index) {
414         Q8_0InputBlock& block = output[block_index];
415         const float* values =
416             input.data() + block_index * checked_size(↵
    ↪ LCQI_Q8_0_INPUT_BLOCK_VALUES,
417                                                         "Q8_0 input block values"↵
    ↪ );
418         float max_value = 0.0F;
419         float max_abs = 0.0F;
420         for (std::int32_t index = 0; index < LCQI_Q8_0_INPUT_BLOCK_VALUES; ++↵
    ↪ index) {
421             const float abs_value = std::fabs(values[index]);
422             if (abs_value > max_abs) {
423                 max_abs = abs_value;
424                 max_value = values[index];
425             }
426         }
427         if (max_abs == 0.0F) {
428             block.d = 0.0F;
429             block.qs.fill(0);
430             block.sum = 0;
431             continue;
432         }

```

```

433     const float inverse_scale =
434         -static_cast<float>(LCQI_GGML_MATVEC_Q8_QUANT_MAX) / max_value;
435     std::int32_t sum = 0;
436     for (std::int32_t index = 0; index < LCQI_Q8_0_INPUT_BLOCK_VALUES; ++
↪ index) {
437         const int quantized = static_cast<int>(std::nearbyint(inverse_scale
↪ * values[index]));
438         const int clamped = std::clamp(quantized,
439             -LCQI_GGML_MATVEC_Q8_QUANT_MAX,
440             LCQI_GGML_MATVEC_Q8_QUANT_MAX);
441         block.qs[index] = static_cast<std::int8_t>(clamped);
442         sum += clamped;
443     }
444     block.sum = static_cast<std::int16_t>(sum);
445     block.d = 1.0F / inverse_scale;
446 }
447 }
448
449 void matvec_ggml_quantized_f32(GgmlType type,
450     std::span<const std::uint8_t> rows,
451     std::int64_t row_count,
452     std::int64_t column_count,
453     std::span<const float> input,
454     std::span<float> output) {
455     require_positive(row_count, "GGML matvec row count");
456     const GgmlTypeLayout layout = ggml_type_layout(type);
457     if (!ggml_type_has_f32_direct_matvec(type)) {
458         throw std::runtime_error(std::string("LCQI has no direct F32 matvec for
↪ ") +
↪ ggml_type_name(type));
459     }
460 }
461 require_multiple(column_count, layout.block_size, "GGML matvec column count
↪ ");
462 if (input.size() != checked_size(column_count, "GGML matvec column count"))
↪ {
463     throw std::runtime_error("GGML matvec input size mismatch");
464 }
465 if (output.size() != checked_size(row_count, "GGML matvec row count")) {
466     throw std::runtime_error("GGML matvec output size mismatch");
467 }
468 const std::int64_t row_bytes =
469     (column_count / layout.block_size) * static_cast<std::int64_t>(layout.
↪ type_size);
470 if (rows.size() != checked_size(row_bytes * row_count, "GGML matvec matrix
↪ bytes")) {
471     throw std::runtime_error("GGML matvec matrix byte size mismatch");
472 }
473
474 for (std::int64_t row = 0; row < row_count; ++row) {
475     const std::uint8_t* row_bytes_begin =
476         rows.data() + checked_size(row * row_bytes, "GGML matvec row offset
↪ ");

```

```

477     float sum = 0.0F;
478     for (std::int64_t block = 0; block < column_count / layout.block_size; ←
↪ ++block) {
479         sum += dot_quantized_block_f32(
480             type,
481             row_bytes_begin +
482             checked_size(block * static_cast<std::int64_t>(layout.↪
↪ type_size),
483                 "GGML matvec block offset"),
484             input.data() + checked_size(block * layout.block_size,
485                 "GGML matvec input block offset"));
486     }
487     output[checked_size(row, "GGML matvec row")] = sum;
488 }
489 }
490
491 void matvec_ggml_quantized_q8_0(GgmlType type,
492     std::span<const std::uint8_t> rows,
493     std::int64_t row_count,
494     std::int64_t column_count,
495     std::span<const Q8_0InputBlock> input,
496     std::span<float> output) {
497     require_positive(row_count, "GGML Q8_0 matvec row count");
498     const GgmlTypeLayout layout = ggml_type_layout(type);
499     if (!ggml_type_has_q8_0_direct_matvec(type)) {
500         throw std::runtime_error(std::string("LCQI has no direct Q8_0 matvec ↪
↪ for ") +
501             ggml_type_name(type));
502     }
503     require_multiple(column_count, layout.block_size, "GGML Q8_0 matvec column ↪
↪ count");
504     validate_q8_0_blocks(input, column_count);
505     if (output.size() != checked_size(row_count, "GGML Q8_0 matvec row count"))↪
↪ {
506         throw std::runtime_error("GGML Q8_0 matvec output size mismatch");
507     }
508     const std::int64_t row_bytes =
509         (column_count / layout.block_size) * static_cast<std::int64_t>(layout.↪
↪ type_size);
510     if (rows.size() != checked_size(row_bytes * row_count, "GGML Q8_0 matvec ↪
↪ matrix bytes")) {
511         throw std::runtime_error("GGML Q8_0 matvec matrix byte size mismatch");
512     }
513
514     matvec_ggml_quantized_q8_0_rows_unchecked(type,
515         rows,
516         row_count,
517         column_count,
518         input,
519         output,
520         0,
521         row_count);

```

```

522 }
523
524 void matvec_ggml_quantized_q8_0_rows_unchecked(GgmlType type,
525                                             std::span<const std::uint8_t> ↵
526         ↵ rows,
527                                             std::int64_t row_count,
528                                             std::int64_t column_count,
529                                             std::span<const Q8_0InputBlock> ↵
530         ↵ input,
531                                             std::span<float> output,
532                                             std::int64_t row_begin,
533                                             std::int64_t row_end) {
534     const GgmlTypeLayout layout = ggml_type_layout(type);
535     #if defined(LCQI_ENABLE_AVX2)
536     if ((type == GgmlType::q8_0 || type == GgmlType::q5_0) && ↵
537         ↵ ggml_q8_0_avx2_available()) {
538         matvec_ggml_quantized_q8_0_avx2_rows_unchecked(type,
539                                                         rows,
540                                                         row_count,
541                                                         column_count,
542                                                         input,
543                                                         output,
544                                                         row_begin,
545                                                         row_end);
546     }
547     return;
548 }
549 #endif
550 (void)row_count;
551 const std::int64_t input_blocks_per_weight_block =
552     layout.block_size / LCQI_Q8_0_INPUT_BLOCK_VALUES;
553 const std::int64_t row_bytes =
554     (column_count / layout.block_size) * static_cast<std::int64_t>(layout.↵
555     ↵ type_size);
556
557 for (std::int64_t row = row_begin; row < row_end; ++row) {
558     const std::uint8_t* row_bytes_begin =
559         rows.data() + static_cast<std::size_t>(row * row_bytes);
560     float sum = 0.0F;
561     for (std::int64_t block = 0; block < column_count / layout.block_size; ↵
562         ↵ ++block) {
563         sum += dot_quantized_block_q8_0(
564             type,
565             row_bytes_begin +
566                 static_cast<std::size_t>(block * static_cast<std::int64_t>(↵
567             ↵ layout.type_size)),
568             input.data() +
569                 static_cast<std::size_t>(block * ↵
570             ↵ input_blocks_per_weight_block));
571     }
572     output[static_cast<std::size_t>(row)] = sum;
573 }
574 }
575 }

```

```

567
568 void matvec_ggml_quantized_q8_0_batch_rows_unchecked(
569     GgmlType type,
570     std::span<const std::uint8_t> rows,
571     std::int64_t row_count,
572     std::int64_t column_count,
573     std::span<const Q8_0InputBlock> input,
574     std::int64_t batch_size,
575     std::span<float> output,
576     std::int64_t output_stride,
577     std::int64_t row_begin,
578     std::int64_t row_end) {
579     const GgmlTypeLayout layout = ggml_type_layout(type);
580     #if defined(LCQI_ENABLE_AVX2)
581     if ((type == GgmlType::q8_0 || type == GgmlType::q5_0) && ↵
↵ ggml_q8_0_avx2_available()) {
582         matvec_ggml_quantized_q8_0_avx2_batch_rows_unchecked(type,
583                                                                 rows,
584                                                                 row_count,
585                                                                 column_count,
586                                                                 input,
587                                                                 batch_size,
588                                                                 output,
589                                                                 output_stride,
590                                                                 row_begin,
591                                                                 row_end);
592         return;
593     }
594     #endif
595     (void)row_count;
596     const std::int64_t input_blocks_per_weight_block =
597         layout.block_size / LCQI_Q8_0_INPUT_BLOCK_VALUES;
598     const std::int64_t input_blocks_per_batch =
599         column_count / LCQI_Q8_0_INPUT_BLOCK_VALUES;
600     const std::int64_t row_bytes =
601         (column_count / layout.block_size) * static_cast<std::int64_t>(layout.↵
↵ type_size);
602
603     for (std::int64_t row = row_begin; row < row_end; ++row) {
604         const std::uint8_t* row_bytes_begin =
605             rows.data() + static_cast<std::size_t>(row * row_bytes);
606         for (std::int64_t batch = 0; batch < batch_size; ++batch) {
607             const Q8_0InputBlock* batch_input =
608                 input.data() + static_cast<std::size_t>(batch * ↵
↵ input_blocks_per_batch);
609             float sum = 0.0F;
610             for (std::int64_t block = 0; block < column_count / layout.↵
↵ block_size; ++block) {
611                 sum += dot_quantized_block_q8_0(
612                     type,
613                     row_bytes_begin +
614                     static_cast<std::size_t>(

```

```

615         block * static_cast<std::int64_t>(layout.type_size) ←
        ↪ ),
616         batch_input +
617         static_cast<std::size_t>(block * ↪
        ↪ input_blocks_per_weight_block));
618     }
619     output[static_cast<std::size_t>(batch * output_stride + row)] = sum ↪
        ↪ ;
620 }
621 }
622 }
623
624 F32RowMax max_dot_ggml_quantized_q8_0(
625     GgmlType type,
626     std::span<const std::uint8_t> rows,
627     std::int64_t row_count,
628     std::int64_t column_count,
629     std::span<const Q8_0InputBlock> input) {
630     require_positive(row_count, "GGML Q8_0 max-dot row count");
631     const GgmlTypeLayout layout = ggml_type_layout(type);
632     if (!ggml_type_has_q8_0_direct_matvec(type)) {
633         throw std::runtime_error(std::string("LCQI has no direct Q8_0 max-dot ↪
        ↪ for ") +
634                                 ggml_type_name(type));
635     }
636     require_multiple(column_count, layout.block_size, "GGML Q8_0 max-dot column ↪
        ↪ count");
637     validate_q8_0_blocks(input, column_count);
638     const std::int64_t row_bytes =
639         (column_count / layout.block_size) * static_cast<std::int64_t>(layout. ↪
        ↪ type_size);
640     if (rows.size() != checked_size(row_bytes * row_count, "GGML Q8_0 max-dot ↪
        ↪ matrix bytes")) {
641         throw std::runtime_error("GGML Q8_0 max-dot matrix byte size mismatch") ↪
        ↪ ;
642     }
643     return max_dot_ggml_quantized_q8_0_rows_unchecked(type,
644                                                         rows,
645                                                         row_count,
646                                                         column_count,
647                                                         input,
648                                                         0,
649                                                         row_count);
650 }
651
652 F32RowMax max_dot_ggml_quantized_q8_0_rows_unchecked(
653     GgmlType type,
654     std::span<const std::uint8_t> rows,
655     std::int64_t row_count,
656     std::int64_t column_count,
657     std::span<const Q8_0InputBlock> input,
658     std::int64_t row_begin,

```



```

659     std::int64_t row_end) {
660         const GgmlTypeLayout layout = ggml_type_layout(type);
661         #if defined(LCQI_ENABLE_AVX2)
662         if ((type == GgmlType::q8_0 || type == GgmlType::q5_0) && ↵
        ↵ ggml_q8_0_avx2_available()) {
663             return max_dot_ggml_quantized_q8_0_avx2_rows_unchecked(type,
664                                                         rows,
665                                                         row_count,
666                                                         column_count,
667                                                         input,
668                                                         row_begin,
669                                                         row_end);
670         }
671         #endif
672         (void)row_count;
673         const std::int64_t input_blocks_per_weight_block =
674             layout.block_size / LCQI_Q8_0_INPUT_BLOCK_VALUES;
675         const std::int64_t row_bytes =
676             (column_count / layout.block_size) * static_cast<std::int64_t>(layout.↵
        ↵ type_size);
677         F32RowMax best;
678         best.row = checked_size(row_begin, "GGML Q8_0 max-dot row_begin");
679         best.value = -std::numeric_limits<float>::infinity();
680         for (std::int64_t row = row_begin; row < row_end; ++row) {
681             const std::uint8_t* row_bytes_begin =
682                 rows.data() + static_cast<std::size_t>(row * row_bytes);
683             float sum = 0.0F;
684             for (std::int64_t block = 0; block < column_count / layout.block_size; ↵
        ↵ ++block) {
685                 sum += dot_quantized_block_q8_0(
686                     type,
687                     row_bytes_begin +
688                         static_cast<std::size_t>(block * static_cast<std::int64_t>(↵
        ↵ layout.type_size)),
689                     input.data() +
690                         static_cast<std::size_t>(block * ↵
        ↵ input_blocks_per_weight_block));
691             }
692             if (row == row_begin || sum > best.value) {
693                 best.row = static_cast<std::size_t>(row);
694                 best.value = sum;
695             }
696         }
697         return best;
698     }
699
700 } // namespace lcqi

```

`ggml_matvec_avx2.cpp` 是一次负结果驱动的优化尝试，也是当前普通 Q8_0 direct 的默认内核。它给 Q8_0xQ8_0 与实验 GGML direct 的 32 元素块点积加了 AVX2 `maddubs` 归约，并提供 row-range、batch-row 与 max-dot 入口，供 worker pool、实验 prompt batch 和 tied lm head 使用。caw 历史端到端报告

显示，全 Q5_0/Q6_K/Q8_0 direct row-range 后从第一版 1211.710ms prefill 降到约 242ms，但仍慢于当时默认路径；因此当前默认只启用普通 Q8_0 direct 和 tied lm head fused max-dot，完整 GGML direct 仍作为实验。源码卷保留这条历史，是为了说明内核优化必须接受端到端证据约束：覆盖率提高不等于性能提高；线程分片能去掉一层瓶颈，但 block layout、Q6_K SIMD 覆盖和 packed multi-row kernel 仍要继续做。

Listing 16.10 src/ggml_matvec_avx2.cpp

```

1 #include <lcqi/ggml_matvec.hpp>
2
3 #if defined(LCQI_ENABLE_AVX2)
4
5 #include <lcqi/ggml_tensors.hpp>
6
7 #include <immintrin.h>
8
9 #include <array>
10 #include <cstdint>
11 #include <limits>
12 #include <span>
13
14 namespace lcqi {
15 namespace {
16
17 constexpr std::uint32_t LCQI_GGML_AVX2_BYTE_BITS = 8U;
18 constexpr std::uint8_t LCQI_GGML_AVX2_LOW_NIBBLE_MASK = 0x0FU;
19 constexpr std::int32_t LCQI_GGML_AVX2_Q5_ZERO_POINT = 16;
20 constexpr std::int64_t LCQI_GGML_AVX2_Q8_0_BLOCK_BYTES = 34;
21 constexpr std::int64_t LCQI_GGML_AVX2_Q5_0_BLOCK_BYTES = 22;
22
23 [[nodiscard]] std::uint16_t read_le_u16(const std::uint8_t* bytes) noexcept {
24     return static_cast<std::uint16_t>((
25         static_cast<std::uint16_t>(bytes[0]) |
26         (static_cast<std::uint16_t>(bytes[1]) << LCQI_GGML_AVX2_BYTE_BITS));
27 }
28
29 [[nodiscard]] std::uint32_t read_le_u32(const std::uint8_t* bytes) noexcept {
30     std::uint32_t value = 0;
31     for (std::uint32_t index = 0; index < sizeof(std::uint32_t); ++index) {
32         value |= static_cast<std::uint32_t>(bytes[index]) <<
33             (LCQI_GGML_AVX2_BYTE_BITS * index);
34     }
35     return value;
36 }
37
38 [[nodiscard]] std::int32_t horizontal_sum_i32(__m256i values) noexcept {
39     const __m128i low = _mm256_castsi256_si128(values);
40     const __m128i high = _mm256_extracti128_si256(values, 1);
41     __m128i sum = _mm_add_epi32(low, high);
42     sum = _mm_add_epi32(sum, _mm_shuffle_epi32(sum, _MM_SHUFFLE(1, 0, 3, 2)));
43     sum = _mm_add_epi32(sum, _mm_shuffle_epi32(sum, _MM_SHUFFLE(2, 3, 0, 1)));
44     return _mm_cvtsi128_si32(sum);

```

```

45 }
46
47 [[nodiscard]] std::int32_t dot_32_i8_i8_avx2(const std::int8_t* lhs,
48                                             const std::int8_t* rhs) noexcept {
49     const __m256i lhs_values = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(lhs));
50     const __m256i rhs_values = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(rhs));
51     const __m256i absolute_lhs = _mm256_sign_epi8(lhs_values, lhs_values);
52     const __m256i signed_rhs = _mm256_sign_epi8(rhs_values, lhs_values);
53     const __m256i products_i16 = _mm256_maddubs_epi16(absolute_lhs, signed_rhs);
54     const __m256i ones = _mm256_set1_epi16(1);
55     const __m256i partial_i32 = _mm256_madd_epi16(products_i16, ones);
56     return horizontal_sum_i32(partial_i32);
57 }
58
59 [[nodiscard]] std::int32_t dot_32_u8_i8_avx2(const std::uint8_t* lhs,
60                                             const std::int8_t* rhs) noexcept {
61     const __m256i lhs_values = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(lhs));
62     const __m256i rhs_values = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(rhs));
63     const __m256i products_i16 = _mm256_maddubs_epi16(lhs_values, rhs_values);
64     const __m256i ones = _mm256_set1_epi16(1);
65     const __m256i partial_i32 = _mm256_madd_epi16(products_i16, ones);
66     return horizontal_sum_i32(partial_i32);
67 }
68
69 [[nodiscard]] float dot_q8_0_block_q8_0_avx2(const std::uint8_t* block,
70                                             const Q8_0InputBlock& input) noexcept {
71     const float d = ggml_f16_to_f32(read_le_u16(block));
72     const auto* qs = reinterpret_cast<const std::int8_t*>(block + sizeof(std::uint16_t));
73     return d * input.d * static_cast<float>(dot_32_i8_i8_avx2(qs, input.qs.data()));
74 }
75
76 [[nodiscard]] float dot_q5_0_block_q8_0_avx2(const std::uint8_t* block,
77                                             const Q8_0InputBlock& input) noexcept {
78     const float d = ggml_f16_to_f32(read_le_u16(block));
79     const std::uint32_t qh = read_le_u32(block + sizeof(std::uint16_t));
80     const std::uint8_t* qs = block + sizeof(std::uint16_t) + sizeof(std::uint32_t);
81
82     alignas(32) std::array<std::uint8_t, LCQI_Q8_0_INPUT_BLOCK_VALUES> values{};
83     for (std::int32_t index = 0; index < LCQI_Q8_0_INPUT_BLOCK_VALUES / 2; ++index) {
84         const std::uint8_t high_0 =

```

```

85         static_cast<std::uint8_t>(((qh >> index) & 1U) << 4U);
86         const std::uint8_t high_1 = static_cast<std::uint8_t>(
87             ((qh >> (index + LCQI_Q8_0_INPUT_BLOCK_VALUES / 2)) & 1U) << 4U);
88         values[static_cast<std::size_t>(index)] =
89             static_cast<std::uint8_t>((qs[index] & ←
↵ LCQI_GGML_AVX2_LOW_NIBBLE_MASK) | high_0);
90         values[static_cast<std::size_t>(index + LCQI_Q8_0_INPUT_BLOCK_VALUES / ←
↵ 2)] =
91             static_cast<std::uint8_t>((qs[index] >> 4U) | high_1);
92     }
93
94     const std::int32_t weighted_sum =
95         dot_32_u8_i8_avx2(values.data(), input.qs.data()) -
96         LCQI_GGML_AVX2_Q5_ZERO_POINT * static_cast<std::int32_t>(input.sum);
97     return d * input.d * static_cast<float>(weighted_sum);
98 }
99
100 void matvec_q8_0_q8_0_avx2_unchecked(std::span<const std::uint8_t> rows,
101                                     std::int64_t row_count,
102                                     std::int64_t column_count,
103                                     std::span<const Q8_0InputBlock> input,
104                                     std::span<float> output,
105                                     std::int64_t row_begin,
106                                     std::int64_t row_end) {
107     (void)row_count;
108     const std::int64_t row_blocks = column_count / LCQI_Q8_0_INPUT_BLOCK_VALUES↵
↵ ;
109     const std::int64_t row_bytes = row_blocks * LCQI_GGML_AVX2_Q8_0_BLOCK_BYTES↵
↵ ;
110     for (std::int64_t row = row_begin; row < row_end; ++row) {
111         float sum = 0.0F;
112         const std::uint8_t* row_data = rows.data() + row * row_bytes;
113         for (std::int64_t block = 0; block < row_blocks; ++block) {
114             sum += dot_q8_0_block_q8_0_avx2(
115                 row_data + block * LCQI_GGML_AVX2_Q8_0_BLOCK_BYTES,
116                 input[static_cast<std::size_t>(block)]);
117         }
118         output[static_cast<std::size_t>(row)] = sum;
119     }
120 }
121
122 void matvec_q8_0_q8_0_avx2_batch_unchecked(std::span<const std::uint8_t> rows,
123                                             std::int64_t row_count,
124                                             std::int64_t column_count,
125                                             std::span<const Q8_0InputBlock> ↵
↵ input,
126                                             std::int64_t batch_size,
127                                             std::span<float> output,
128                                             std::int64_t output_stride,
129                                             std::int64_t row_begin,
130                                             std::int64_t row_end) {
131     (void)row_count;

```

[illegible]

```

178         std::int64_t output_stride,
179         std::int64_t row_begin,
180         std::int64_t row_end) {
181     (void)row_count;
182     const std::int64_t row_blocks = column_count / LCQI_Q8_0_INPUT_BLOCK_VALUES;
183     const std::int64_t row_bytes = row_blocks * LCQI_GGML_AVX2_Q5_0_BLOCK_BYTES;
184     for (std::int64_t row = row_begin; row < row_end; ++row) {
185         const std::uint8_t* row_data = rows.data() + row * row_bytes;
186         for (std::int64_t batch = 0; batch < batch_size; ++batch) {
187             const Q8_0InputBlock* batch_input =
188                 input.data() + static_cast<std::size_t>(batch * row_blocks);
189             float sum = 0.0F;
190             for (std::int64_t block = 0; block < row_blocks; ++block) {
191                 sum += dot_q5_0_block_q8_0_avx2(
192                     row_data + block * LCQI_GGML_AVX2_Q5_0_BLOCK_BYTES,
193                     batch_input[static_cast<std::size_t>(block)]);
194             }
195             output[static_cast<std::size_t>(batch * output_stride + row)] = sum;
196         }
197     }
198 }
199
200 F32RowMax max_dot_q8_0_q8_0_avx2_rows_unchecked(std::span<const std::uint8_t> rows,
201     std::int64_t row_count,
202     std::int64_t column_count,
203     std::span<const Q8_0InputBlock> input,
204     std::int64_t row_begin,
205     std::int64_t row_end) {
206     (void)row_count;
207     const std::int64_t row_blocks = column_count / LCQI_Q8_0_INPUT_BLOCK_VALUES;
208     const std::int64_t row_bytes = row_blocks * LCQI_GGML_AVX2_Q8_0_BLOCK_BYTES;
209     F32RowMax best;
210     best.row = static_cast<std::size_t>(row_begin);
211     best.value = -std::numeric_limits<float>::infinity();
212     for (std::int64_t row = row_begin; row < row_end; ++row) {
213         const std::uint8_t* row_data = rows.data() + row * row_bytes;
214         float sum = 0.0F;
215         for (std::int64_t block = 0; block < row_blocks; ++block) {
216             sum += dot_q8_0_block_q8_0_avx2(
217                 row_data + block * LCQI_GGML_AVX2_Q8_0_BLOCK_BYTES,
218                 input[static_cast<std::size_t>(block)]);
219         }
220         if (row == row_begin || sum > best.value) {
221             best.row = static_cast<std::size_t>(row);
222             best.value = sum;

```

[illegible]

```

270                                     std::int64_t column_count,
271                                     std::span<const float> output,
↳ Q8_0InputBlock> input,
272                                     std::int64_t row_begin,
273                                     std::int64_t row_end);
274
275
276 void matvec_ggml_quantized_q8_0_avx2_unchecked(GgmlType type,
277                                     std::span<const float> output,
↳ rows,
278                                     std::int64_t row_count,
279                                     std::int64_t column_count,
280                                     std::span<const Q8_0InputBlock> input,
281                                     std::span<float> output) {
282     matvec_ggml_quantized_q8_0_avx2_rows_unchecked(type,
283                                                     rows,
284                                                     row_count,
285                                                     column_count,
286                                                     input,
287                                                     output,
288                                                     0,
289                                                     row_count);
290 }
291
292 void matvec_ggml_quantized_q8_0_avx2_rows_unchecked(GgmlType type,
293                                     std::span<const float> output,
↳ uint8_t> rows,
294                                     std::int64_t row_count,
295                                     std::int64_t column_count,
296                                     std::span<const Q8_0InputBlock> input,
297                                     std::span<float> output,
298                                     std::int64_t row_begin,
299                                     std::int64_t row_end) {
300     if (type == GgmlType::q8_0) {
301         matvec_q8_0_q8_0_avx2_unchecked(rows,
302                                           row_count,
303                                           column_count,
304                                           input,
305                                           output,
306                                           row_begin,
307                                           row_end);
308         return;
309     }
310     if (type == GgmlType::q5_0) {
311         matvec_q5_0_q8_0_avx2_unchecked(rows,
312                                           row_count,
313                                           column_count,
314                                           input,
315                                           output,
316                                           row_begin,

```



```

317         row_end);
318     }
319 }
320
321 void matvec_ggml_quantized_q8_0_avx2_batch_rows_unchecked(
322     GgmlType type,
323     std::span<const std::uint8_t> rows,
324     std::int64_t row_count,
325     std::int64_t column_count,
326     std::span<const Q8_0InputBlock> input,
327     std::int64_t batch_size,
328     std::span<float> output,
329     std::int64_t output_stride,
330     std::int64_t row_begin,
331     std::int64_t row_end) {
332     if (type == GgmlType::q8_0) {
333         matvec_q8_0_q8_0_avx2_batch_unchecked(rows,
334             row_count,
335             column_count,
336             input,
337             batch_size,
338             output,
339             output_stride,
340             row_begin,
341             row_end);
342         return;
343     }
344     if (type == GgmlType::q5_0) {
345         matvec_q5_0_q8_0_avx2_batch_unchecked(rows,
346             row_count,
347             column_count,
348             input,
349             batch_size,
350             output,
351             output_stride,
352             row_begin,
353             row_end);
354     }
355 }
356
357 F32RowMax max_dot_ggml_quantized_q8_0_avx2_rows_unchecked(
358     GgmlType type,
359     std::span<const std::uint8_t> rows,
360     std::int64_t row_count,
361     std::int64_t column_count,
362     std::span<const Q8_0InputBlock> input,
363     std::int64_t row_begin,
364     std::int64_t row_end) {
365     if (type == GgmlType::q8_0) {
366         return max_dot_q8_0_q8_0_avx2_rows_unchecked(rows,
367             row_count,
368             column_count,

```

```

369                                     input,
370                                     row_begin,
371                                     row_end);
372     }
373     if (type == GgmLType::q5_0) {
374         return max_dot_q5_0_q8_0_avx2_rows_unchecked(rows,
375                                                         row_count,
376                                                         column_count,
377                                                         input,
378                                                         row_begin,
379                                                         row_end);
380     }
381     F32RowMax best;
382     best.row = static_cast<std::size_t>(row_begin);
383     best.value = -std::numeric_limits<float>::infinity();
384     return best;
385 }
386
387 } // namespace lcqi
388
389 #endif

```

`f32_kernels_avx2.cpp` 是默认路径里 F32 fallback 的 x86 微内核。SmolLM2 Q4_K_M 文件里只有 16 个 shape 兼容 tensor 走默认 Q4_K direct，其余 256 个 linear tensor 仍会 materialize 成 F32 权重，所以这个文件不是“GPT-2 辅助代码”，而是当前 LCQI 默认 SmolLM2 热点之一。当前版本有两条快路径：单 token decode 使用 8-row row-range fast path，一次加载同一段 input，分别和 8 个输出行权重做 FMA；prompt prefill 使用 batch-row path，一次扫描同一行权重，给 8 个 prompt token 同时累加，尾部 4/3/2/1 个 token 分别处理。测试用完整 17 行输出覆盖 8-row path，用 batch fixture 覆盖 batch-row path，并用 batch prefill 对照 token path，防止输出 token 漂移。caw 5 轮报告显示，batch prefill 加 8-token/tail 内核后，prefill 中位数为 **33.807600ms**，F32 fallback hotspot 为 **24.326800ms**，相对关闭 batch prefill 的 prefill 提升 **4.760941x**，F32 hotspot 提升 **6.050693x**，F32 call 数减少 **16x**。这说明 batch prefill 是结构性优化，8-token/tail 是局部强化；二者都不等于 decode 已追平。

Listing 16.11 src/f32_kernels_avx2.cpp

```

1  #include <lcqi/f32_kernels.hpp>
2
3  #if defined(LCQI_ENABLE_AVX2)
4
5  #include <immintrin.h>
6
7  #include <array>
8  #include <cstdint>
9  #include <limits>
10
11 namespace lcqi {
12 namespace {
13

```

```

14 constexpr std::size_t LCQI_F32_AVX2_LANE_COUNT = 8;
15 constexpr std::size_t LCQI_F32_AVX2_UNROLL_COUNT = 4;
16 constexpr std::size_t LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT = 8;
17 constexpr std::size_t LCQI_F32_AVX2_BATCH_UNROLL_COUNT = 4;
18 constexpr std::size_t LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT = 8;
19 constexpr std::size_t LCQI_F32_AVX2_UNROLLED_FLOATS =
20     LCQI_F32_AVX2_LANE_COUNT * LCQI_F32_AVX2_UNROLL_COUNT;
21
22 [[nodiscard]] float horizontal_sum(__m256 values) noexcept {
23     const __m128 low = _mm256_castps256_ps128(values);
24     const __m128 high = _mm256_extractf128_ps(values, 1);
25     const __m128 pair_sum = _mm_add_ps(low, high);
26     const __m128 first_fold = _mm_hadd_ps(pair_sum, pair_sum);
27     const __m128 second_fold = _mm_hadd_ps(first_fold, first_fold);
28     return _mm_cvtss_f32(second_fold);
29 }
30
31 void compute_four_row_dot(const float* row0,
32                          const float* row1,
33                          const float* row2,
34                          const float* row3,
35                          const float* input,
36                          std::size_t input_size,
37                          float* sums) noexcept {
38     std::size_t index = 0;
39     __m256 accumulator0 = _mm256_setzero_ps();
40     __m256 accumulator1 = _mm256_setzero_ps();
41     __m256 accumulator2 = _mm256_setzero_ps();
42     __m256 accumulator3 = _mm256_setzero_ps();
43
44     for (; index + LCQI_F32_AVX2_LANE_COUNT <= input_size;
45          index += LCQI_F32_AVX2_LANE_COUNT) {
46         const __m256 input_values = _mm256_loadu_ps(input + index);
47         accumulator0 =
48             _mm256_fmadd_ps(_mm256_loadu_ps(row0 + index), input_values, ↵
↵ accumulator0);
49         accumulator1 =
50             _mm256_fmadd_ps(_mm256_loadu_ps(row1 + index), input_values, ↵
↵ accumulator1);
51         accumulator2 =
52             _mm256_fmadd_ps(_mm256_loadu_ps(row2 + index), input_values, ↵
↵ accumulator2);
53         accumulator3 =
54             _mm256_fmadd_ps(_mm256_loadu_ps(row3 + index), input_values, ↵
↵ accumulator3);
55     }
56
57     sums[0] = horizontal_sum(accumulator0);
58     sums[1] = horizontal_sum(accumulator1);
59     sums[2] = horizontal_sum(accumulator2);
60     sums[3] = horizontal_sum(accumulator3);
61     for (; index < input_size; ++index) {

```

```

62     const float input_value = input[index];
63     sums[0] += row0[index] * input_value;
64     sums[1] += row1[index] * input_value;
65     sums[2] += row2[index] * input_value;
66     sums[3] += row3[index] * input_value;
67 }
68 }
69
70 void compute_eight_row_dot(const float* const* rows,
71     const float* input,
72     std::size_t input_size,
73     float* sums) noexcept {
74     std::size_t index = 0;
75     __m256 accumulator0 = _mm256_setzero_ps();
76     __m256 accumulator1 = _mm256_setzero_ps();
77     __m256 accumulator2 = _mm256_setzero_ps();
78     __m256 accumulator3 = _mm256_setzero_ps();
79     __m256 accumulator4 = _mm256_setzero_ps();
80     __m256 accumulator5 = _mm256_setzero_ps();
81     __m256 accumulator6 = _mm256_setzero_ps();
82     __m256 accumulator7 = _mm256_setzero_ps();
83
84     for (; index + LCQI_F32_AVX2_LANE_COUNT <= input_size;
85         index += LCQI_F32_AVX2_LANE_COUNT) {
86         const __m256 input_values = _mm256_loadu_ps(input + index);
87         accumulator0 =
88             _mm256_fmadd_ps(_mm256_loadu_ps(rows[0] + index), input_values, ↵
↵ accumulator0);
89         accumulator1 =
90             _mm256_fmadd_ps(_mm256_loadu_ps(rows[1] + index), input_values, ↵
↵ accumulator1);
91         accumulator2 =
92             _mm256_fmadd_ps(_mm256_loadu_ps(rows[2] + index), input_values, ↵
↵ accumulator2);
93         accumulator3 =
94             _mm256_fmadd_ps(_mm256_loadu_ps(rows[3] + index), input_values, ↵
↵ accumulator3);
95         accumulator4 =
96             _mm256_fmadd_ps(_mm256_loadu_ps(rows[4] + index), input_values, ↵
↵ accumulator4);
97         accumulator5 =
98             _mm256_fmadd_ps(_mm256_loadu_ps(rows[5] + index), input_values, ↵
↵ accumulator5);
99         accumulator6 =
100             _mm256_fmadd_ps(_mm256_loadu_ps(rows[6] + index), input_values, ↵
↵ accumulator6);
101         accumulator7 =
102             _mm256_fmadd_ps(_mm256_loadu_ps(rows[7] + index), input_values, ↵
↵ accumulator7);
103     }
104
105     sums[0] = horizontal_sum(accumulator0);

```

```

106     sums[1] = horizontal_sum(accumulator1);
107     sums[2] = horizontal_sum(accumulator2);
108     sums[3] = horizontal_sum(accumulator3);
109     sums[4] = horizontal_sum(accumulator4);
110     sums[5] = horizontal_sum(accumulator5);
111     sums[6] = horizontal_sum(accumulator6);
112     sums[7] = horizontal_sum(accumulator7);
113     for (; index < input_size; ++index) {
114         const float input_value = input[index];
115         for (std::size_t row = 0; row < LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT; ++row) {
116             sums[row] += rows[row][index] * input_value;
117         }
118     }
119 }
120
121 template <std::size_t BATCH_SIZE>
122 void compute_batch_dot(const float* weights,
123                       const float* input,
124                       std::size_t input_size,
125                       float* sums) noexcept {
126     std::size_t index = 0;
127     __m256 accumulators[BATCH_SIZE];
128     std::array<const float*, BATCH_SIZE> inputs{};
129     for (std::size_t batch = 0; batch < BATCH_SIZE; ++batch) {
130         accumulators[batch] = _mm256_setzero_ps();
131         inputs[batch] = input + batch * input_size;
132     }
133
134     for (; index + LCQI_F32_AVX2_LANE_COUNT <= input_size;
135          index += LCQI_F32_AVX2_LANE_COUNT) {
136         const __m256 weight_values = _mm256_loadu_ps(weights + index);
137         for (std::size_t batch = 0; batch < BATCH_SIZE; ++batch) {
138             accumulators[batch] =
139                 _mm256_fmadd_ps(weight_values,
140                                 _mm256_loadu_ps(inputs[batch] + index),
141                                 accumulators[batch]);
142         }
143     }
144
145     for (std::size_t batch = 0; batch < BATCH_SIZE; ++batch) {
146         sums[batch] = horizontal_sum(accumulators[batch]);
147     }
148     for (; index < input_size; ++index) {
149         const float weight_value = weights[index];
150         for (std::size_t batch = 0; batch < BATCH_SIZE; ++batch) {
151             sums[batch] += weight_value * inputs[batch][index];
152         }
153     }
154 }
155
156 } // namespace

```

```

157
158 bool dot_f32_avx2_available() noexcept {
159     #if defined(__GNUC__) || defined(__clang__)
160         __builtin_cpu_init();
161         return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
162     #else
163         return true;
164     #endif
165 }
166
167 float dot_f32_avx2_unchecked(const float* lhs,
168                             const float* rhs,
169                             std::size_t size) noexcept {
170     std::size_t index = 0;
171     __m256 accumulator0 = _mm256_setzero_ps();
172     __m256 accumulator1 = _mm256_setzero_ps();
173     __m256 accumulator2 = _mm256_setzero_ps();
174     __m256 accumulator3 = _mm256_setzero_ps();
175
176     for (; index + LCQI_F32_AVX2_UNROLLED_FLOATS <= size;
177          index += LCQI_F32_AVX2_UNROLLED_FLOATS) {
178         const __m256 lhs0 = _mm256_loadu_ps(lhs + index);
179         const __m256 rhs0 = _mm256_loadu_ps(rhs + index);
180         const __m256 lhs1 = _mm256_loadu_ps(lhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT);
181         const __m256 rhs1 = _mm256_loadu_ps(rhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT);
182         const __m256 lhs2 = _mm256_loadu_ps(lhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT * 2);
183         const __m256 rhs2 = _mm256_loadu_ps(rhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT * 2);
184         const __m256 lhs3 = _mm256_loadu_ps(lhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT * 3);
185         const __m256 rhs3 = _mm256_loadu_ps(rhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT * 3);
186
187         accumulator0 = _mm256_fmadd_ps(lhs0, rhs0, accumulator0);
188         accumulator1 = _mm256_fmadd_ps(lhs1, rhs1, accumulator1);
189         accumulator2 = _mm256_fmadd_ps(lhs2, rhs2, accumulator2);
190         accumulator3 = _mm256_fmadd_ps(lhs3, rhs3, accumulator3);
191     }
192
193     __m256 accumulator = _mm256_add_ps(
194         _mm256_add_ps(accumulator0, accumulator1),
195         _mm256_add_ps(accumulator2, accumulator3));
196     for (; index + LCQI_F32_AVX2_LANE_COUNT <= size;
197          index += LCQI_F32_AVX2_LANE_COUNT) {
198         const __m256 lhs_values = _mm256_loadu_ps(lhs + index);
199         const __m256 rhs_values = _mm256_loadu_ps(rhs + index);
200         accumulator = _mm256_fmadd_ps(lhs_values, rhs_values, accumulator);
201     }
202

```

```

203     float sum = horizontal_sum(accumulator);
204     for (; index < size; ++index) {
205         sum += lhs[index] * rhs[index];
206     }
207     return sum;
208 }
209
210 void linear_f32_rows_avx2_unchecked(const float* weights,
211                                     const float* input,
212                                     const float* bias,
213                                     std::size_t input_size,
214                                     std::size_t row_begin,
215                                     std::size_t row_end,
216                                     float* output) noexcept {
217     std::size_t row = row_begin;
218     for (; row + LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT <= row_end;
219          row += LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT) {
220         std::array<const float*, LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT> rows{};
221         rows[0] = weights + row * input_size;
222         for (std::size_t offset = 1; offset < rows.size(); ++offset) {
223             rows[offset] = rows[offset - 1] + input_size;
224         }
225         std::array<float, LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT> sums{};
226         compute_eight_row_dot(rows.data(), input, input_size, sums.data());
227         for (std::size_t offset = 0; offset < ↵
↵ LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT; ++offset) {
228             output[row + offset] =
229                 sums[offset] + (bias == nullptr ? 0.0F : bias[row + offset]);
230         }
231     }
232     for (; row + LCQI_F32_AVX2_UNROLL_COUNT <= row_end;
233          row += LCQI_F32_AVX2_UNROLL_COUNT) {
234         const float* row0 = weights + row * input_size;
235         const float* row1 = row0 + input_size;
236         const float* row2 = row1 + input_size;
237         const float* row3 = row2 + input_size;
238         std::array<float, LCQI_F32_AVX2_UNROLL_COUNT> sums{};
239         compute_four_row_dot(row0, row1, row2, row3, input, input_size, sums.↵
↵ data());
240         for (std::size_t offset = 0; offset < LCQI_F32_AVX2_UNROLL_COUNT; ++↵
↵ offset) {
241             output[row + offset] =
242                 sums[offset] + (bias == nullptr ? 0.0F : bias[row + offset]);
243         }
244     }
245     for (; row < row_end; ++row) {
246         const float* weight_row = weights + row * input_size;
247         output[row] = dot_f32_avx2_unchecked(weight_row, input, input_size) +
248             (bias == nullptr ? 0.0F : bias[row]);
249     }
250 }
251

```

```

252 void linear_f32_batch_rows_avx2_unchecked(const float* weights,
253                                           const float* input,
254                                           const float* bias,
255                                           std::size_t input_size,
256                                           std::size_t batch_size,
257                                           std::size_t row_begin,
258                                           std::size_t row_end,
259                                           std::size_t output_stride,
260                                           float* output) noexcept {
261     for (std::size_t row = row_begin; row < row_end; ++row) {
262         const float* weight_row = weights + row * input_size;
263         std::size_t batch = 0;
264         for (; batch + LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT <= batch_size;
265             batch += LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT) {
266             std::array<float, LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT> sums{};
267             compute_batch_dot<LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT>(
268                 weight_row,
269                 input + batch * input_size,
270                 input_size,
271                 sums.data());
272             for (std::size_t offset = 0;
273                 offset < LCQI_F32_AVX2_WIDE_BATCH_UNROLL_COUNT;
274                 ++offset) {
275                 output[(batch + offset) * output_stride + row] =
276                     sums[offset] + (bias == nullptr ? 0.0F : bias[row]);
277             }
278         }
279         for (; batch + LCQI_F32_AVX2_BATCH_UNROLL_COUNT <= batch_size;
280             batch += LCQI_F32_AVX2_BATCH_UNROLL_COUNT) {
281             std::array<float, LCQI_F32_AVX2_BATCH_UNROLL_COUNT> sums{};
282             compute_batch_dot<LCQI_F32_AVX2_BATCH_UNROLL_COUNT>(weight_row,
283                 input + batch * ←
284                 ↪ input_size,
285                 ↪ input_size,
286                 ↪ sums.data());
287             for (std::size_t offset = 0; offset < ↪
288                 ↪ LCQI_F32_AVX2_BATCH_UNROLL_COUNT; ++offset) {
289                 output[(batch + offset) * output_stride + row] =
290                     sums[offset] + (bias == nullptr ? 0.0F : bias[row]);
291             }
292         }
293         for (; batch < batch_size; ++batch) {
294             const std::size_t remaining = batch_size - batch;
295             if (remaining == 3U) {
296                 std::array<float, 3> sums{};
297                 compute_batch_dot<3>(weight_row,
298                     input + batch * input_size,
299                     input_size,
300                     sums.data());
301                 for (std::size_t offset = 0; offset < sums.size(); ++offset) {
302                     output[(batch + offset) * output_stride + row] =
303                         sums[offset] + (bias == nullptr ? 0.0F : bias[row]);

```



```

302         }
303         break;
304     }
305     if (remaining == 2U) {
306         std::array<float, 2> sums{};
307         compute_batch_dot<2>(weight_row,
308                             input + batch * input_size,
309                             input_size,
310                             sums.data());
311         for (std::size_t offset = 0; offset < sums.size(); ++offset) {
312             output[(batch + offset) * output_stride + row] =
313                 sums[offset] + (bias == nullptr ? 0.0F : bias[row]);
314         }
315         break;
316     }
317     output[batch * output_stride + row] =
318         dot_f32_avx2_unchecked(weight_row,
319                               input + batch * input_size,
320                               input_size) +
321         (bias == nullptr ? 0.0F : bias[row]);
322 }
323 }
324 }
325
326 F32RowMax max_dot_f32_rows_avx2_unchecked(const float* weights,
327                                           const float* input,
328                                           std::size_t input_size,
329                                           std::size_t row_begin,
330                                           std::size_t row_end) noexcept {
331     F32RowMax best;
332     best.row = row_begin;
333     best.value = -std::numeric_limits<float>::infinity();
334     std::size_t row = row_begin;
335     for (; row + LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT <= row_end;
336          row += LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT) {
337         std::array<const float*, LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT> rows{};
338         rows[0] = weights + row * input_size;
339         for (std::size_t offset = 1; offset < rows.size(); ++offset) {
340             rows[offset] = rows[offset - 1] + input_size;
341         }
342         std::array<float, LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT> sums{};
343         compute_eight_row_dot(rows.data(), input, input_size, sums.data());
344         for (std::size_t offset = 0; offset < ↵
↵ LCQI_F32_AVX2_WIDE_ROW_UNROLL_COUNT; ++offset) {
345             if (row + offset == row_begin || sums[offset] > best.value) {
346                 best.row = row + offset;
347                 best.value = sums[offset];
348             }
349         }
350     }
351     for (; row + LCQI_F32_AVX2_UNROLL_COUNT <= row_end;
352          row += LCQI_F32_AVX2_UNROLL_COUNT) {

```

```

353     const float* row0 = weights + row * input_size;
354     const float* row1 = row0 + input_size;
355     const float* row2 = row1 + input_size;
356     const float* row3 = row2 + input_size;
357     std::array<float, LCQI_F32_AVX2_UNROLL_COUNT> sums{};
358     compute_four_row_dot(row0, row1, row2, row3, input, input_size, sums.↵
↵ data());
359     for (std::size_t offset = 0; offset < LCQI_F32_AVX2_UNROLL_COUNT; ++↵
↵ offset) {
360         if (row + offset == row_begin || sums[offset] > best.value) {
361             best.row = row + offset;
362             best.value = sums[offset];
363         }
364     }
365 }
366 for (; row < row_end; ++row) {
367     const float* weight_row = weights + row * input_size;
368     const float value = dot_f32_avx2_unchecked(weight_row, input, ↵
↵ input_size);
369     if (row == row_begin || value > best.value) {
370         best.row = row;
371         best.value = value;
372     }
373 }
374 return best;
375 }
376
377 } // namespace lcqi
378
379 #endif

```

`q5_0_packed_avx2.cpp` 是默认 `Q5_0` decode packed 路径的 x86 微内核。它和 `ggml_matvec_avx2.cpp` 的区别在于：`Q5_0` 的 high bit 与 low nibble 已经在加载模型时整理进 `Q5_0PackedBlock`，decode 前向不再反复从 GGML 原始 bytes 中拆 bit。单 token decode path 仍是一个 packed block 对一个 `Q8_0InputBlock`；低层 prompt prefill batch group path 也保留，用来验证“同一 packed weight block 服务多个 batch token”的想法。caw A/B 后，默认模型路径没有启用这个 batch group path，而是在加载 `Q5` tensor 时同时保存一份 F32 sidecar，prompt batch 通过 `linear_f32_batch_rows_unchecked` 执行。测试会比较 full path、row-range path、batch path，以及模型级 batch prefill 是否只计入 `q5_0_batch_f32_calls`；这样源码既保留后续 packed batch 优化入口，又不会把当前端到端负收益路径写进默认执行。

Listing 16.12 src/q5_0_packed_avx2.cpp

```

1 #include <lcqi/q5_0_packed.hpp>
2
3 #if defined(LCQI_ENABLE_AVX2)
4
5 #include <immintrin.h>
6
7 #include <cstdint>

```

```

8 #include <cstdint>
9 #include <cstdlib>
10 #include <cstring>
11
12 namespace lcqi {
13 namespace {
14
15 constexpr std::int32_t LCQI_Q5_0_PACKED_ZERO_POINT = 16;
16 constexpr const char* LCQI_Q5_0_PACKED_BACKEND_ENV = "LCQI_Q5_0_PACKED_BACKEND" ←
    ↪ ;
17 constexpr const char* LCQI_Q5_0_PACKED_FORCE_SCALAR_ENV =
18     "LCQI_Q5_0_PACKED_FORCE_SCALAR";
19 constexpr const char* LCQI_Q5_0_PACKED_BACKEND_SCALAR = "scalar";
20 constexpr const char* LCQI_Q5_0_PACKED_FORCE_TRUE = "1";
21 constexpr std::int64_t LCQI_Q5_0_PACKED_BATCH_WIDE = 8;
22 constexpr std::int64_t LCQI_Q5_0_PACKED_BATCH_NARROW = 4;
23
24 [[nodiscard]] float horizontal_sum_ps(__m256 values) noexcept {
25     const __m128 low = _mm256_castps256_ps128(values);
26     const __m128 high = _mm256_extractf128_ps(values, 1);
27     const __m128 pair_sum = _mm_add_ps(low, high);
28     const __m128 first_fold = _mm_hadd_ps(pair_sum, pair_sum);
29     const __m128 second_fold = _mm_hadd_ps(first_fold, first_fold);
30     return _mm_cvtss_f32(second_fold);
31 }
32
33 [[nodiscard]] __m256i dot_lanes_loaded_32_u8_i8_avx2(__m256i lhs_values,
34                                                     const std::int8_t* rhs) ←
    ↪ noexcept {
35     const __m256i rhs_values = _mm256_loadu_si256(reinterpret_cast<const ←
    ↪ __m256i*>(rhs));
36     const __m256i products_i16 = _mm256_maddubs_epi16(lhs_values, rhs_values);
37     const __m256i ones = _mm256_set1_epi16(1);
38     return _mm256_madd_epi16(products_i16, ones);
39 }
40
41 [[nodiscard]] float zero_point_correction(const Q5_0PackedBlock& block,
42                                           const Q8_0InputBlock& input) noexcept ←
    ↪ {
43     return -static_cast<float>(LCQI_Q5_0_PACKED_ZERO_POINT) *
44           block.d *
45           input.d *
46           static_cast<float>(input.sum);
47 }
48
49 void accumulate_q5_0_packed_q8_0_lanes_avx2(__m256i weight_values,
50                                             const Q5_0PackedBlock& block,
51                                             const Q8_0InputBlock& input,
52                                             __m256& lane_sum,
53                                             float& correction_sum) noexcept {
54     const __m256i weighted_lanes =
55         dot_lanes_loaded_32_u8_i8_avx2(weight_values, input.qs.data());

```

```

56     const __m256 scale = _mm256_set1_ps(block.d * input.d);
57     lane_sum =
58         _mm256_add_ps(lane_sum, _mm256_mul_ps(_mm256_cvtepi32_ps(weighted_lanes ←
    ↪ ), scale));
59     correction_sum += zero_point_correction(block, input);
60 }
61
62 template <std::int64_t BATCH_COUNT>
63 void accumulate_q5_0_packed_q8_0_batch_block_avx2(const Q5_0PackedBlock& block,
64                                                     const Q8_0InputBlock* ←
    ↪ batch_input,
65                                                     std::int64_t input_stride,
66                                                     __m256* lane_sums,
67                                                     float* correction_sums) ←
    ↪ noexcept {
68     const __m256i weight_values =
69         _mm256_loadu_si256(reinterpret_cast<const __m256i*>(block.qs.data()));
70     for (std::int64_t batch = 0; batch < BATCH_COUNT; ++batch) {
71         const Q8_0InputBlock& input = batch_input[batch * input_stride];
72         accumulate_q5_0_packed_q8_0_lanes_avx2(
73             weight_values,
74             block,
75             input,
76             lane_sums[batch],
77             correction_sums[batch]);
78     }
79 }
80
81 template <std::int64_t BATCH_COUNT>
82 void matvec_q5_0_packed_q8_0_avx2_batch_group_unchecked(
83     const Q5_0PackedBlock* row_blocks_begin,
84     std::int64_t row_blocks,
85     const Q8_0InputBlock* batch_input,
86     std::int64_t input_stride,
87     float* output,
88     std::int64_t output_stride,
89     std::int64_t row) noexcept {
90     __m256 lane_sums[static_cast<std::size_t>(BATCH_COUNT)];
91     float correction_sums[static_cast<std::size_t>(BATCH_COUNT)]{};
92     for (std::int64_t batch = 0; batch < BATCH_COUNT; ++batch) {
93         lane_sums[batch] = _mm256_setzero_ps();
94     }
95     for (std::int64_t block = 0; block < row_blocks; ++block) {
96         accumulate_q5_0_packed_q8_0_batch_block_avx2<BATCH_COUNT>(
97             row_blocks_begin[static_cast<std::size_t>(block)],
98             batch_input + block,
99             input_stride,
100             lane_sums,
101             correction_sums);
102     }
103     for (std::int64_t batch = 0; batch < BATCH_COUNT; ++batch) {
104         output[batch * output_stride + row] =

```

```

105         horizontal_sum_ps(lane_sums[batch]) + correction_sums[batch];
106     }
107 }
108
109 [[nodiscard]] bool scalar_backend_forced() noexcept {
110     const char* backend = std::getenv(LCQI_Q5_0_PACKED_BACKEND_ENV);
111     if (backend != nullptr && std::strcmp(backend, ↵
↵ LCQI_Q5_0_PACKED_BACKEND_SCALAR) == 0) {
112         return true;
113     }
114     const char* force_scalar = std::getenv(LCQI_Q5_0_PACKED_FORCE_SCALAR_ENV);
115     return force_scalar != nullptr &&
116            std::strcmp(force_scalar, LCQI_Q5_0_PACKED_FORCE_TRUE) == 0;
117 }
118
119 } // namespace
120
121 bool q5_0_packed_q8_0_avx2_available() noexcept {
122     if (scalar_backend_forced()) {
123         return false;
124     }
125     #if defined(__GNUC__) || defined(__clang__)
126     __builtin_cpu_init();
127     return __builtin_cpu_supports("avx2");
128     #else
129     return true;
130     #endif
131 }
132
133 const char* q5_0_packed_q8_0_active_backend() noexcept {
134     return q5_0_packed_q8_0_avx2_available() ? "avx2_maddubs" : "scalar";
135 }
136
137 void matvec_q5_0_packed_q8_0_avx2_rows_unchecked(
138     std::span<const Q5_0PackedBlock> rows,
139     std::int64_t row_count,
140     std::int64_t column_count,
141     std::span<const Q8_0InputBlock> input,
142     std::span<float> output,
143     std::int64_t row_begin,
144     std::int64_t row_end) noexcept {
145     (void)row_count;
146     const std::int64_t row_blocks = column_count / LCQI_Q5_0_BLOCK_VALUES;
147     for (std::int64_t row = row_begin; row < row_end; ++row) {
148         __m256 lane_sum = _mm256_setzero_ps();
149         float correction_sum = 0.0F;
150         const Q5_0PackedBlock* row_blocks_begin =
151             rows.data() + static_cast<std::size_t>(row * row_blocks);
152         for (std::int64_t block = 0; block < row_blocks; ++block) {
153             const Q5_0PackedBlock& packed_block =
154                 row_blocks_begin[static_cast<std::size_t>(block)];
155             const __m256i weight_values =

```

[illegible]

```

205         batch_input,
206         row_blocks,
207         output.data() +
208             static_cast<std::size_t>(
209             batch * output_stride),
210         output_stride,
211         row);
212     }
213     const std::int64_t remaining = batch_size - batch;
214     if (remaining == 3) {
215         const Q8_0InputBlock* batch_input =
216             input.data() + static_cast<std::size_t>(batch * row_blocks);
217         matvec_q5_0_packed_q8_0_avx2_batch_group_unchecked<3>(↵
↵ row_blocks_begin,
218
219             row_blocks,
220             batch_input,
221             row_blocks,
222             output.data()↵
↵ +
223
224             ↵
↵ static_cast<std::size_t>(
225
226             batch↵
↵ * output_stride),
227
228             output_stride↵
↵ ,
229
230             row);
231         continue;
232     }
233     if (remaining == 2) {
234         const Q8_0InputBlock* batch_input =
235             input.data() + static_cast<std::size_t>(batch * row_blocks);
236         matvec_q5_0_packed_q8_0_avx2_batch_group_unchecked<2>(↵
↵ row_blocks_begin,
237
238             row_blocks,
239             batch_input,
240             row_blocks,
241             output.data()↵
↵ +
242
243             ↵
↵ static_cast<std::size_t>(
244
245             batch↵
↵ * output_stride),
246
247             output_stride↵
↵ ,
248
249             row);
250         continue;
251     }
252     if (remaining == 1) {
253         const Q8_0InputBlock* batch_input =
254             input.data() + static_cast<std::size_t>(batch * row_blocks);
255         matvec_q5_0_packed_q8_0_avx2_batch_group_unchecked<1>(↵
↵ row_blocks_begin,

```



```

246                                     row_blocks,
247                                     batch_input,
248                                     row_blocks,
249                                     output.data()↵
↵ +
250                                     ↵
↵ static_cast<std::size_t>(
251                                     batch↵
↵ * output_stride),
252                                     output_stride↵
↵ ,
253                                     row);
254     }
255 }
256 }
257
258 } // namespace lcqi
259
260 #endif

```

`llama_gguf.cpp` 是自研 SmolLM2/GGUF decode path。它读取 GGUF metadata 得到 hidden size、层数、GQA 头数、RoPE base、RMSNorm epsilon 和 tokenizer id；再读取 `token_embd.weight`、每层 attention/FFN 权重和 `output_norm.weight`。最新实现把权重表示分成默认可用和实验可用两层：shape 兼容 Q4_K linear tensor 保留 GGUF 原始 block bytes，通过 Q8_K scratch 调用 `matvec_q4_k_q8`；shape 兼容 Q5_0 tensor 在 load 阶段同时生成 Q5_0PackedBlock 和 prompt batch F32 sidecar；普通 Q8_0 linear tensor 和 tied `token_embd.weight` lm head 保留原始 direct bytes；默认 Q6_K 和小标量 tensor materialize 成 F32 fallback。`max_dot_ggml_quantized_q8_0_parallel` 是 tied lm head 的专用分派：每个 worker 在自己的 vocab row 段里直接找最大 dot，主线程合并局部最大值，因此 decode 不再为 tied Q8_0 lm head 分配并扫描完整 logits。`LCQI_LLAMA_Q6_K_DIRECT=1` 是独立 Q6_K A/B 开关，会把 Q6_K 记录到 `q6_k_direct` 存储类型和单独 hotspot；完整 `LCQI_LLAMA_GGML_DIRECT=1` 才把 Q5_0/Q6_K/Q8_0 一起放进实验 direct。读这份源码时还要看两条前向路径：decode 仍由 `step_llama_decoder` 单 token 推进，Q5_0 走 packed direct；prompt prefill 默认走 `prefill_llama_decoder_batch`，用 `LlamaBatchWorkspace` 同时保存多个 prompt token 的中间张量，其中 Q5 用 F32 sidecar batch rows，Q8 用 direct batch，Q6 direct 只在实验开关下启用。三条主线分别是：loader 如何记录 direct/fallback/sidecar 字节，`linear_gguf` 与 `linear_gguf_batch` 如何在多类路径之间分派，`LlamaGgufHotspotReport` 如何把 worker count、per-op hotspot、direct/fallback/sidecar call 数输出到报告。

Listing 16.13 src/llama_gguf.cpp

```

1 #include <lcqi/llama_gguf.hpp>
2
3 #include <lcqi/f32_kernels.hpp>
4 #include <lcqi/ggml_matvec.hpp>
5 #include <lcqi/ggml_tensors.hpp>
6 #include <lcqi/gguf.hpp>
7 #include <lcqi/q4_k.hpp>
8 #include <lcqi/q5_0_packed.hpp>

```



```

9
10 #include <algorithm>
11 #include <array>
12 #include <atomic>
13 #include <chrono>
14 #include <cmath>
15 #include <cstdlib>
16 #include <limits>
17 #include <memory>
18 #include <sstream>
19 #include <stdexcept>
20 #include <string>
21 #include <thread>
22 #include <unordered_map>
23 #include <utility>
24
25 namespace lcqi {
26 namespace {
27
28 constexpr std::int32_t LCQI_LLAMA_ROPE_PAIR_WIDTH = 2;
29 constexpr std::int32_t LCQI_LLAMA_DEFAULT_BOS_TOKEN = 1;
30 constexpr std::int32_t LCQI_LLAMA_DEFAULT_EOS_TOKEN = 2;
31 constexpr std::int32_t LCQI_LLAMA_DEFAULT_UNK_TOKEN = 0;
32 constexpr const char* LCQI_LLAMA_ARCHITECTURE_KEY = "general.architecture";
33 constexpr const char* LCQI_LLAMA_NAME_KEY = "general.name";
34 constexpr const char* LCQI_LLAMA_EMBEDDING_LENGTH_KEY = "llama.embedding_length"↵
↵ ";
35 constexpr const char* LCQI_LLAMA_BLOCK_COUNT_KEY = "llama.block_count";
36 constexpr const char* LCQI_LLAMA_FEED_FORWARD_LENGTH_KEY = "llama.↵
↵ feed_forward_length";
37 constexpr const char* LCQI_LLAMA_HEAD_COUNT_KEY = "llama.attention.head_count";
38 constexpr const char* LCQI_LLAMA_KV_HEAD_COUNT_KEY = "llama.attention.↵
↵ head_count_kv";
39 constexpr const char* LCQI_LLAMA_ROPE_DIMENSION_KEY = "llama.rope.↵
↵ dimension_count";
40 constexpr const char* LCQI_LLAMA_ROPE_BASE_KEY = "llama.rope.freq_base";
41 constexpr const char* LCQI_LLAMA_CONTEXT_LENGTH_KEY = "llama.context_length";
42 constexpr const char* LCQI_LLAMA_VOCAB_SIZE_KEY = "llama.vocab_size";
43 constexpr const char* LCQI_LLAMA_RMS_EPSILON_KEY = "llama.attention.↵
↵ layer_norm_rms_epsilon";
44 constexpr const char* LCQI_LLAMA_TOKENIZER_TOKENS_KEY = "tokenizer.ggml.tokens"↵
↵ ;
45 constexpr const char* LCQI_LLAMA_TOKENIZER_MERGES_KEY = "tokenizer.ggml.merges"↵
↵ ;
46 constexpr const char* LCQI_LLAMA_BOS_TOKEN_KEY = "tokenizer.ggml.bos_token_id";
47 constexpr const char* LCQI_LLAMA_EOS_TOKEN_KEY = "tokenizer.ggml.eos_token_id";
48 constexpr const char* LCQI_LLAMA_UNKNOWN_TOKEN_KEY = "tokenizer.ggml.↵
↵ unknown_token_id";
49 constexpr const char* LCQI_LLAMA_Q4_DIRECT_ENV = "LCQI_LLAMA_Q4_DIRECT";
50 constexpr const char* LCQI_LLAMA_Q5_0_PACKED_ENV = "LCQI_LLAMA_Q5_0_PACKED";
51 constexpr const char* LCQI_LLAMA_Q6_K_DIRECT_ENV = "LCQI_LLAMA_Q6_K_DIRECT";
52 constexpr const char* LCQI_LLAMA_Q8_0_DIRECT_ENV = "LCQI_LLAMA_Q8_0_DIRECT";

```

```

53 constexpr const char* LCQI_LLAMA_Q8_0_TIED_LM_HEAD_ENV =
54     "LCQI_LLAMA_Q8_0_TIED_LM_HEAD";
55 constexpr const char* LCQI_LLAMA_GGML_DIRECT_ENV = "LCQI_LLAMA_GGML_DIRECT";
56 constexpr const char* LCQI_LLAMA_GGML_SELECTIVE_DIRECT_ENV =
57     "LCQI_LLAMA_GGML_SELECTIVE_DIRECT";
58 constexpr const char* LCQI_LLAMA_BATCH_PREFILL_ENV = "LCQI_LLAMA_BATCH_PREFILL"↵
    ↵ ;
59 constexpr const char* LCQI_LLAMA_FALSE_ENV = "0";
60 constexpr const char* LCQI_LLAMA_TRUE_ENV = "1";
61 constexpr std::int32_t LCQI_LLAMA_MAX_AUTO_WORKERS = 8;
62 constexpr std::int32_t LCQI_LLAMA_DEFAULT_PARALLEL_MIN_ROWS = 512;
63 constexpr std::int32_t LCQI_LLAMA_PARALLEL_MIN_COLUMNS = 512;
64 constexpr std::int64_t LCQI_LLAMA_PARALLEL_MIN_OPS = 1'000'000;
65 constexpr std::size_t LCQI_LLAMA_PARALLEL_ROW_BLOCK_MULTIPLIER = 4;
66 constexpr std::size_t LCQI_LLAMA_WORKER_SPIN_PAUSES_BEFORE_YIELD = 4096;
67
68 using Clock = std::chrono::steady_clock;
69
70 enum class LlamaGgufLinearOp : std::uint8_t {
71     wq,
72     wk,
73     wv,
74     wo,
75     w_gate,
76     w_up,
77     w_down,
78     lm_head,
79 };
80
81 void pause_worker_spin(std::size_t& pause_count) noexcept {
82     #if defined(__GNUC__) && (defined(__x86_64__) || defined(__i386__))
83         __builtin_ia32_pause();
84     #endif
85     ++pause_count;
86     if (pause_count >= LCQI_LLAMA_WORKER_SPIN_PAUSES_BEFORE_YIELD) {
87         pause_count = 0;
88         std::this_thread::yield();
89     }
90 }
91
92 class LlamaParallelWorkerPool {
93 public:
94     explicit LlamaParallelWorkerPool(std::int32_t requested_workers)
95         : worker_count_(LlamaParallelWorkerPool::normalize_worker_count(↵
    ↵ requested_workers)) {
96         if (this->worker_count_ <= 1) {
97             return;
98         }
99         const std::size_t background_count =
100             static_cast<std::size_t>(this->worker_count_ - 1);
101         this->threads_.reserve(background_count);
102         for (std::size_t index = 0; index < background_count; ++index) {

```

```

103         this->threads_.emplace_back([this, index]() { this->worker_loop(↵
↵ index); }));
104     }
105 }
106
107 ~LlamaParallelWorkerPool() {
108     this->stopping_.store(true, std::memory_order_release);
109     this->generation_.fetch_add(1, std::memory_order_release);
110     for (std::thread& thread : this->threads_) {
111         if (thread.joinable()) {
112             thread.join();
113         }
114     }
115 }
116
117 LlamaParallelWorkerPool(const LlamaParallelWorkerPool&) = delete;
118 LlamaParallelWorkerPool& operator=(const LlamaParallelWorkerPool&) = delete;↵
↵ ;
119
120 [[nodiscard]] std::int32_t worker_count() const noexcept {
121     return this->worker_count_;
122 }
123
124 template <typename Function>
125 void parallel_for_rows(std::size_t begin,
126                       std::size_t end,
127                       std::size_t grain,
128                       Function&& function) {
129     if (end <= begin) {
130         return;
131     }
132     const std::size_t row_count = end - begin;
133     if (this->worker_count_ <= 1 || row_count <= grain) {
134         function(begin, end, 0);
135         return;
136     }
137
138     const std::size_t task_count =
139         std::min<std::size_t>(static_cast<std::size_t>(this->worker_count_)↵
↵ ,
140                             (row_count + grain - 1) / grain);
141     if (task_count <= 1) {
142         function(begin, end, 0);
143         return;
144     }
145
146     auto task_context = ParallelTaskContext<Function>{function};
147     this->task_begin_.store(begin, std::memory_order_release);
148     this->task_end_.store(end, std::memory_order_release);
149     this->task_count_.store(task_count, std::memory_order_release);
150     this->task_context_.store(&task_context, std::memory_order_release);
151     this->task_invoke_.store(&ParallelTaskContext<Function>::invoke,

```

```

152         std::memory_order_release);
153     this->active_workers_.store(task_count - 1, std::memory_order_release);
154     this->generation_.fetch_add(1, std::memory_order_release);
155
156     const std::size_t main_worker = task_count - 1;
157     const auto [main_begin, main_end] =
158         this->chunk_bounds(begin, end, task_count, main_worker);
159     function(main_begin, main_end, main_worker);
160
161     std::size_t pause_count = 0;
162     while (this->active_workers_.load(std::memory_order_acquire) != 0) {
163         pause_worker_spin(pause_count);
164     }
165     this->task_context_.store(nullptr, std::memory_order_release);
166     this->task_invoke_.store(nullptr, std::memory_order_release);
167 }
168
169 private:
170     template <typename Function>
171     struct ParallelTaskContext {
172         Function& function;
173
174         static void invoke(void* context,
175                             std::size_t begin,
176                             std::size_t end,
177                             std::size_t worker_index) {
178             auto* typed_context = static_cast<ParallelTaskContext*>(context);
179             typed_context->function(begin, end, worker_index);
180         }
181     };
182
183     std::int32_t worker_count_ = 1;
184     std::vector<std::thread> threads_;
185     std::atomic<bool> stopping_ = false;
186     std::atomic<std::uint64_t> generation_ = 0;
187     std::atomic<std::size_t> task_begin_ = 0;
188     std::atomic<std::size_t> task_end_ = 0;
189     std::atomic<std::size_t> task_count_ = 0;
190     std::atomic<std::size_t> active_workers_ = 0;
191     std::atomic<void*> task_context_ = nullptr;
192     std::atomic<void (*) (void*, std::size_t, std::size_t, std::size_t)> ↵
    ↵ task_invoke_ = nullptr;
193
194     [[nodiscard]] static std::int32_t normalize_worker_count(std::int32_t ↵
    ↵ requested_workers) {
195         if (requested_workers > 0) {
196             return requested_workers;
197         }
198         const unsigned int hardware_workers = std::thread::hardware_concurrency↵
    ↵ ();
199         if (hardware_workers == 0) {
200             return 1;

```

```

201     }
202     const std::int32_t capped =
203         std::min<std::int32_t>(static_cast<std::int32_t>(hardware_workers),
204                               LCQI_LLAMA_MAX_AUTO_WORKERS);
205     return std::max(capped, 1);
206 }
207
208 [[nodiscard]] std::pair<std::size_t, std::size_t> chunk_bounds(
209     std::size_t begin,
210     std::size_t end,
211     std::size_t task_count,
212     std::size_t worker_index) const noexcept {
213     const std::size_t row_count = end - begin;
214     const std::size_t chunk_begin = begin + row_count * worker_index / ↵
↵ task_count;
215     const std::size_t chunk_end = begin + row_count * (worker_index + 1) / ↵
↵ task_count;
216     return {chunk_begin, chunk_end};
217 }
218
219 void worker_loop(std::size_t worker_index) {
220     std::uint64_t observed_generation = 0;
221     std::size_t pause_count = 0;
222     while (true) {
223         const std::uint64_t current_generation =
224             this->generation_.load(std::memory_order_acquire);
225         if (current_generation == observed_generation) {
226             if (this->stopping_.load(std::memory_order_acquire)) {
227                 return;
228             }
229             pause_worker_spin(pause_count);
230             continue;
231         }
232
233         observed_generation = current_generation;
234         pause_count = 0;
235         if (this->stopping_.load(std::memory_order_acquire)) {
236             return;
237         }
238
239         const std::size_t task_count = this->task_count_.load(std::↵
↵ memory_order_acquire);
240         if (this->generation_.load(std::memory_order_acquire) != ↵
↵ current_generation ||
241             task_count == 0) {
242             continue;
243         }
244         const std::size_t background_task_count = task_count - 1;
245         if (worker_index >= background_task_count) {
246             continue;
247         }
248         void* task_context = this->task_context_.load(std::↵

```

```

    ↪ memory_order_acquire);
249     void (*task_invoke)(void*, std::size_t, std::size_t, std::size_t) =
250         this->task_invoke_.load(std::memory_order_acquire);
251     if (task_context == nullptr || task_invoke == nullptr) {
252         continue;
253     }
254     const std::size_t task_begin = this->task_begin_.load(std::memory_order_acquire);
    ↪ memory_order_acquire);
255     const std::size_t task_end = this->task_end_.load(std::memory_order_acquire);
    ↪ memory_order_acquire);
256     if (this->generation_.load(std::memory_order_acquire) != current_generation) {
    ↪ current_generation) {
257         continue;
258     }
259     const auto [begin, end] =
260         this->chunk_bounds(task_begin, task_end, task_count, worker_index);
    ↪ worker_index);
261     task_invoke(task_context, begin, end, worker_index);
262     this->active_workers_.fetch_sub(1, std::memory_order_acq_rel);
263 }
264 }
265 };
266
267 [[nodiscard]] double elapsed_ms(Clock::time_point begin, Clock::time_point end) {
    ↪ {
268     return std::chrono::duration<double, std::milli>(end - begin).count();
269 }
270
271 [[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
272     if (value < 0 ||
273         static_cast<std::uint64_t>(value) >
274         static_cast<std::uint64_t>(std::numeric_limits<std::size_t>::max()))
    ↪ ) {
275         throw std::runtime_error(std::string(name) + " is outside size_t range");
    ↪ );
276     }
277     return static_cast<std::size_t>(value);
278 }
279
280 void require_positive(std::int32_t value, const char* name) {
281     if (value <= 0) {
282         throw std::runtime_error(std::string(name) + " must be positive");
283     }
284 }
285
286 [[nodiscard]] const GgufMetadataEntry& require_metadata(const GgufManifest& manifest,
    ↪ manifest,
287                                                         std::string_view key) {
288     const GgufMetadataEntry* entry = manifest.find_metadata(key);
289     if (entry == nullptr) {
290         throw std::runtime_error("missing GGUF metadata key: " + std::string(key));
    ↪ key));

```

```

291     }
292     return *entry;
293 }
294
295 [[nodiscard]] std::string metadata_string(const GgufManifest& manifest, std::string_view key) {
296     return require_metadata(manifest, key).value_preview;
297 }
298
299 [[nodiscard]] std::int32_t metadata_i32(const GgufManifest& manifest, std::string_view key) {
300     const std::int64_t value = std::stoll(require_metadata(manifest, key).value_preview);
301     if (value < std::numeric_limits<std::int32_t>::min() ||
302         value > std::numeric_limits<std::int32_t>::max()) {
303         throw std::runtime_error("GGUF metadata int32 out of range: " + std::string(key));
304     }
305     return static_cast<std::int32_t>(value);
306 }
307
308 [[nodiscard]] std::int32_t optional_metadata_i32(const GgufManifest& manifest,
309                                                  std::string_view key,
310                                                  std::int32_t fallback) {
311     const GgufMetadataEntry* entry = manifest.find_metadata(key);
312     if (entry == nullptr) {
313         return fallback;
314     }
315     const std::int64_t value = std::stoll(entry->value_preview);
316     if (value < std::numeric_limits<std::int32_t>::min() ||
317         value > std::numeric_limits<std::int32_t>::max()) {
318         throw std::runtime_error("GGUF metadata int32 out of range: " + std::string(key));
319     }
320     return static_cast<std::int32_t>(value);
321 }
322
323 [[nodiscard]] float metadata_f32(const GgufManifest& manifest, std::string_view key) {
324     return std::stof(require_metadata(manifest, key).value_preview);
325 }
326
327 void validate_shape(const GgufTensorInfo& tensor,
328                    std::span<const std::int64_t> expected_shape) {
329     if (tensor.shape.size() != expected_shape.size()) {
330         throw std::runtime_error("GGUF tensor rank mismatch: " + tensor.name);
331     }
332     for (std::size_t index = 0; index < expected_shape.size(); ++index) {
333         if (tensor.shape[index] != expected_shape[index]) {
334             throw std::runtime_error("GGUF tensor shape mismatch: " + tensor.name);
335         }
336     }
337 }

```

```

336     }
337 }
338
339 [[nodiscard]] bool can_use_q4_k_direct(const GgufTensorInfo& tensor) {
340     return tensor.type == GgmlType::q4_k && tensor.shape.size() == 2 &&
341         tensor.shape[0] > 0 && tensor.shape[1] > 0 &&
342         tensor.shape[0] % LCQI_QK_K_BLOCK_VALUES == 0;
343 }
344
345 [[nodiscard]] bool can_use_ggml_direct(const GgufTensorInfo& tensor) {
346     if (tensor.shape.size() != 2 || tensor.shape[0] <= 0 || tensor.shape[1] <= 0 ||
347         !ggml_type_has_q8_0_direct_matvec(tensor.type)) {
348         return false;
349     }
350     const GgmlTypeLayout layout = ggml_type_layout(tensor.type);
351     return tensor.shape[0] % layout.block_size == 0 &&
352         tensor.shape[0] % LCQI_Q8_0_INPUT_BLOCK_VALUES == 0;
353 }
354
355 [[nodiscard]] bool can_use_selective_ggml_direct(const GgufTensorInfo& tensor) {
356     return (tensor.type == GgmlType::q5_0 || tensor.type == GgmlType::q8_0) &&
357         can_use_ggml_direct(tensor);
358 }
359
360 [[nodiscard]] bool q4_k_direct_enabled() noexcept {
361     const char* enabled = std::getenv(LCQI_LLAMA_Q4_DIRECT_ENV);
362     return enabled == nullptr || std::string_view(enabled) != LCQI_LLAMA_FALSE_ENV;
363 }
364
365 [[nodiscard]] bool q5_0_packed_enabled() noexcept {
366     const char* enabled = std::getenv(LCQI_LLAMA_Q5_0_PACKED_ENV);
367     return enabled == nullptr || std::string_view(enabled) != LCQI_LLAMA_FALSE_ENV;
368 }
369
370 [[nodiscard]] bool q6_k_direct_enabled() noexcept {
371     const char* enabled = std::getenv(LCQI_LLAMA_Q6_K_DIRECT_ENV);
372     return enabled != nullptr && std::string_view(enabled) == LCQI_LLAMA_TRUE_ENV;
373 }
374
375 [[nodiscard]] bool q8_0_direct_enabled() noexcept {
376     const char* enabled = std::getenv(LCQI_LLAMA_Q8_0_DIRECT_ENV);
377     return enabled == nullptr || std::string_view(enabled) != LCQI_LLAMA_FALSE_ENV;
378 }
379
380 [[nodiscard]] bool q8_0_tied_lm_head_enabled() noexcept {
381     const char* enabled = std::getenv(LCQI_LLAMA_Q8_0_TIED_LM_HEAD_ENV);

```



```

382     return enabled == nullptr || std::string_view(enabled) != ↵
    ↵ LCQI_LLAMA_FALSE_ENV;
383 }
384
385 [[nodiscard]] bool ggml_direct_enabled() noexcept {
386     const char* enabled = std::getenv(LCQI_LLAMA_GGML_DIRECT_ENV);
387     return enabled != nullptr && std::string_view(enabled) == ↵
    ↵ LCQI_LLAMA_TRUE_ENV;
388 }
389
390 [[nodiscard]] bool ggml_selective_direct_enabled() noexcept {
391     const char* enabled = std::getenv(LCQI_LLAMA_GGML_SELECTIVE_DIRECT_ENV);
392     return enabled != nullptr && std::string_view(enabled) == ↵
    ↵ LCQI_LLAMA_TRUE_ENV;
393 }
394
395 [[nodiscard]] bool batch_prefill_enabled() noexcept {
396     const char* enabled = std::getenv(LCQI_LLAMA_BATCH_PREFILL_ENV);
397     return enabled == nullptr || std::string_view(enabled) != ↵
    ↵ LCQI_LLAMA_FALSE_ENV;
398 }
399
400 void account_f32_tensor_load(const GgufTensorInfo& tensor,
401                             std::span<const float> values,
402                             LlamaGgufLoadReport& report) {
403     report.quantized_weight_bytes += tensor.byte_size;
404     const std::uint64_t f32_bytes =
405         static_cast<std::uint64_t>(values.size()) * static_cast<std::uint64_t>(↵
    ↵ sizeof(float));
406     report.f32_weight_bytes += f32_bytes;
407     report.fallback_dequantized_weight_bytes += f32_bytes;
408     ++report.tensors_loaded;
409     ++report.f32_fallback_tensors;
410 }
411
412 [[nodiscard]] std::vector<float> read_tensor_f32(const std::filesystem::path& ↵
    ↵ gguf_path,
413                                                  const GgufManifest& manifest,
414                                                  std::string_view name,
415                                                  std::span<const std::int64_t> ↵
    ↵ expected_shape,
416                                                  LlamaGgufLoadReport& report) {
417     const GgufTensorInfo* tensor = manifest.find_tensor(name);
418     if (tensor == nullptr) {
419         throw std::runtime_error("missing LLaMA GGUF tensor: " + std::string(↵
    ↵ name));
420     }
421     validate_shape(*tensor, expected_shape);
422     const std::vector<std::uint8_t> bytes = read_gguf_tensor_bytes(gguf_path, *↵
    ↵ tensor);
423     std::vector<float> values =
424         dequantize_ggml_tensor(tensor->type,

```

```

425         bytes,
426         static_cast<std::int64_t>(tensor->element_count()
    ↪ ())),
427     account_f32_tensor_load(*tensor, values, report);
428     return values;
429 }
430
431 void account_quantized_direct_tensor_load(const GgufTensorInfo& tensor,
432     LlamaGgufLoadReport& report) {
433     report.quantized_weight_bytes += tensor.byte_size;
434     report.direct_quantized_weight_bytes += tensor.byte_size;
435     ++report.tensors_loaded;
436     if (tensor.type == GgmlType::q4_k) {
437         ++report.q4_k_direct_tensors;
438     } else {
439         ++report.ggml_direct_tensors;
440     }
441 }
442
443 void account_q5_0_packed_tensor_load(const GgufTensorInfo& tensor,
444     const LlamaGgufLinearWeights& linear,
445     LlamaGgufLoadReport& report) {
446     report.quantized_weight_bytes += tensor.byte_size;
447     report.direct_quantized_weight_bytes += tensor.byte_size;
448     report.q5_0_packed_weight_bytes +=
449         linear.q5_0_packed_blocks.size() * sizeof(Q5_0PackedBlock);
450     const std::uint64_t batch_f32_bytes =
451         static_cast<std::uint64_t>(linear.f32_weights.size()) *
452         static_cast<std::uint64_t>(sizeof(float));
453     report.f32_weight_bytes += batch_f32_bytes;
454     report.q5_0_batch_f32_weight_bytes += batch_f32_bytes;
455     ++report.tensors_loaded;
456     ++report.q5_0_packed_tensors;
457 }
458
459 void account_q6_k_direct_tensor_load(const GgufTensorInfo& tensor,
460     LlamaGgufLoadReport& report) {
461     report.quantized_weight_bytes += tensor.byte_size;
462     report.direct_quantized_weight_bytes += tensor.byte_size;
463     report.q6_k_direct_weight_bytes += tensor.byte_size;
464     ++report.tensors_loaded;
465     ++report.q6_k_direct_tensors;
466 }
467
468 void account_q8_0_direct_tensor_load(const GgufTensorInfo& tensor,
469     LlamaGgufLoadReport& report) {
470     report.direct_quantized_weight_bytes += tensor.byte_size;
471     report.q8_0_direct_weight_bytes += tensor.byte_size;
472     ++report.q8_0_direct_tensors;
473 }
474
475 void account_q8_0_linear_tensor_load(const GgufTensorInfo& tensor,

```

```

476         LlamaGgufLoadReport& report) {
477     report.quantized_weight_bytes += tensor.byte_size;
478     account_q8_0_direct_tensor_load(tensor, report);
479     ++report.tensors_loaded;
480 }
481
482 [[nodiscard]] LlamaGgufLinearWeights make_linear_from_gguf(
483     const std::filesystem::path& gguf_path,
484     const GgufManifest& manifest,
485     std::string_view name,
486     std::int32_t input_size,
487     std::int32_t output_size,
488     LlamaGgufLoadReport& report) {
489     const std::array<std::int64_t, 2> shape{input_size, output_size};
490     const GgufTensorInfo* tensor = manifest.find_tensor(name);
491     if (tensor == nullptr) {
492         throw std::runtime_error("missing LLaMA GGUF tensor: " + std::string(↵
↵ name));
493     }
494     validate_shape(*tensor, shape);
495
496     LlamaGgufLinearWeights linear;
497     linear.input_size = input_size;
498     linear.output_size = output_size;
499     linear.source_type = tensor->type;
500     const std::vector<std::uint8_t> bytes = read_gguf_tensor_bytes(gguf_path, *↵
↵ tensor);
501     if (q4_k_direct_enabled() && can_use_q4_k_direct(*tensor)) {
502         linear.storage = LlamaGgufLinearStorage::q4_k;
503         linear.quantized_bytes = bytes;
504         account_quantized_direct_tensor_load(*tensor, report);
505         return linear;
506     }
507     if (q5_0_packed_enabled() && !ggml_direct_enabled() &&
508         !ggml_selective_direct_enabled() && tensor->type == GgmlType::q5_0 &&
509         can_use_ggml_direct(*tensor)) {
510         linear.storage = LlamaGgufLinearStorage::q5_0_packed;
511         linear.q5_0_packed_blocks = pack_q5_0_blocks(
512             bytes,
513             static_cast<std::int64_t>(tensor->element_count()));
514         linear.f32_weights =
515             dequantize_ggml_tensor(tensor->type,
516                                     bytes,
517                                     static_cast<std::int64_t>(tensor->↵
↵ element_count()));
518         account_q5_0_packed_tensor_load(*tensor, linear, report);
519         return linear;
520     }
521     if (q6_k_direct_enabled() && !ggml_direct_enabled() &&
522         !ggml_selective_direct_enabled() && tensor->type == GgmlType::q6_k &&
523         can_use_ggml_direct(*tensor)) {
524         linear.storage = LlamaGgufLinearStorage::q6_k_direct;

```

```

525     linear.quantized_bytes = bytes;
526     account_q6_k_direct_tensor_load(*tensor, report);
527     return linear;
528 }
529 if (q8_0_direct_enabled() && !ggml_direct_enabled() &&
530     !ggml_selective_direct_enabled() && tensor->type == GgmlType::q8_0 &&
531     can_use_ggml_direct(*tensor)) {
532     linear.storage = LlamaGgufLinearStorage::q8_0_direct;
533     linear.quantized_bytes = bytes;
534     account_q8_0_linear_tensor_load(*tensor, report);
535     return linear;
536 }
537 if (ggml_direct_enabled() && can_use_ggml_direct(*tensor)) {
538     linear.storage = LlamaGgufLinearStorage::ggml_quantized;
539     linear.quantized_bytes = bytes;
540     account_quantized_direct_tensor_load(*tensor, report);
541     return linear;
542 }
543 if (ggml_selective_direct_enabled() && can_use_selective_ggml_direct(*
    ↪ tensor)) {
544     linear.storage = LlamaGgufLinearStorage::ggml_quantized;
545     linear.quantized_bytes = bytes;
546     account_quantized_direct_tensor_load(*tensor, report);
547     return linear;
548 }
549
550 linear.storage = LlamaGgufLinearStorage::f32;
551 linear.f32_weights =
552     dequantize_ggml_tensor(tensor->type,
553                             bytes,
554                             static_cast<std::int64_t>(tensor->element_count
    ↪
    ↪ ())),);
555 account_f32_tensor_load(*tensor, linear.f32_weights, report);
556 return linear;
557 }
558
559 [[nodiscard]] LlamaGgufLinearWeights make_tied_lm_head_from_embedding(
560     const std::filesystem::path& gguf_path,
561     const GgufManifest& manifest,
562     std::int32_t hidden_size,
563     std::int32_t vocab_size,
564     LlamaGgufLoadReport& report) {
565     LlamaGgufLinearWeights linear;
566     linear.input_size = hidden_size;
567     linear.output_size = vocab_size;
568     const GgufTensorInfo* tensor = manifest.find_tensor("token_embd.weight");
569     if (tensor == nullptr || tensor->type != GgmlType::q8_0 ||
570         !q8_0_tied_lm_head_enabled() || !can_use_ggml_direct(*tensor)) {
571         return linear;
572     }
573     validate_shape(*tensor, std::array<std::int64_t, 2>{hidden_size, vocab_size
    ↪
    ↪ });

```

```

574     linear.source_type = tensor->type;
575     linear.storage = LlamaGgufLinearStorage::q8_0_direct;
576     linear.quantized_bytes = read_gguf_tensor_bytes(gguf_path, *tensor);
577     account_q8_0_direct_tensor_load(*tensor, report);
578     return linear;
579 }
580
581 [[nodiscard]] std::string layer_tensor_name(std::int32_t layer_id, std::string_view suffix) {
582     return "blk." + std::to_string(layer_id) + "." + std::string(suffix);
583 }
584
585 void validate_llama_config(const DecoderConfig& config, std::int32_t layer_count) {
586     require_positive(layer_count, "layer_count");
587     require_positive(config.hidden_size, "hidden_size");
588     require_positive(config.query_heads, "query_heads");
589     require_positive(config.kv_heads, "kv_heads");
590     require_positive(config.head_dim, "head_dim");
591     require_positive(config.intermediate_size, "intermediate_size");
592     require_positive(config.vocab_size, "vocab_size");
593     require_positive(config.max_context, "max_context");
594     if (config.query_heads % config.kv_heads != 0) {
595         throw std::runtime_error("LLaMA query_heads must be divisible by kv_heads");
596     }
597     if (config.hidden_size != config.query_heads * config.head_dim) {
598         throw std::runtime_error("LLaMA hidden_size must equal query_heads * head_dim");
599     }
600     if (config.head_dim % LCQI_LLAMA_ROPE_PAIR_WIDTH != 0) {
601         throw std::runtime_error("LLaMA head_dim must be even for RoPE");
602     }
603 }
604
605 [[nodiscard]] std::span<const float> row_span(std::span<const float> values,
606                                               std::int32_t row,
607                                               std::int32_t row_width) {
608     return values.subspan(row, "row") * checked_size(row_width, "row_width"),
609           checked_size(row_width, "row_width"));
610 }
611
612 [[nodiscard]] std::span<float> mutable_row_span(std::span<float> values,
613                                                  std::size_t row,
614                                                  std::size_t row_width) {
615     return values.subspan(row * row_width, row_width);
616 }
617
618 void add_inplace(std::span<float> target, std::span<const float> source) {
619     if (target.size() != source.size()) {
620         throw std::runtime_error("LLaMA residual add size mismatch");

```

```

621     }
622     for (std::size_t index = 0; index < target.size(); ++index) {
623         target[index] += source[index];
624     }
625 }
626
627 [[nodiscard]] float silu(float value) {
628     return value / (1.0F + std::exp(-value));
629 }
630
631 template <typename Func>
632 double measure_ms(Func&& func) {
633     const Clock::time_point begin = Clock::now();
634     std::forward<Func>(func)();
635     return elapsed_ms(begin, Clock::now());
636 }
637
638 [[nodiscard]] double& linear_hotspot_field(LlamaGgufHotspotReport& hotspots,
639                                           LlamaGgufLinearOp operation) {
640     switch (operation) {
641         case LlamaGgufLinearOp::wq:
642             return hotspots.wq_ms;
643         case LlamaGgufLinearOp::wk:
644             return hotspots.wk_ms;
645         case LlamaGgufLinearOp::wv:
646             return hotspots.wv_ms;
647         case LlamaGgufLinearOp::wo:
648             return hotspots.wo_ms;
649         case LlamaGgufLinearOp::w_gate:
650             return hotspots.w_gate_ms;
651         case LlamaGgufLinearOp::w_up:
652             return hotspots.w_up_ms;
653         case LlamaGgufLinearOp::w_down:
654             return hotspots.w_down_ms;
655         case LlamaGgufLinearOp::lm_head:
656             return hotspots.lm_head_ms;
657     }
658     throw std::runtime_error("unknown LLaMA linear hotspot operation");
659 }
660
661 void validate_linear_shape(const LlamaGgufLinearWeights& layer,
662                           std::span<const float> input,
663                           std::span<float> output) {
664     if (input.size() != checked_size(layer.input_size, "linear input size") ||
665         output.size() != checked_size(layer.output_size, "linear output size")) ←
666     {
667         throw std::runtime_error("LLaMA GGUF linear input/output size mismatch" ←
668         );
669     }
670 }
671
672 void linear_f32_fallback(const LlamaGgufLinearWeights& layer,

```

```

671         std::span<const float> input,
672         std::span<float> output) {
673     if (layer.f32_weights.size() !=
674         checked_size(layer.input_size, "linear input size") *
675         checked_size(layer.output_size, "linear output size")) {
676         throw std::runtime_error("LLaMA GGUF F32 linear weight size mismatch");
677     }
678     linear_f32_rows_unchecked(layer.f32_weights.data(),
679                               input.data(),
680                               nullptr,
681                               checked_size(layer.input_size, "linear input size"),
682                               0,
683                               checked_size(layer.output_size, "linear output size"),
684                               output.data());
685 }
686
687 [[nodiscard]] bool should_parallelize_rows(const LlamaGgufExecutionOptions& options,
688                                             std::size_t rows,
689                                             std::size_t columns,
690                                             const LlamaParallelWorkerPool* worker_pool) {
691     if (worker_pool == nullptr || worker_pool->worker_count() <= 1) {
692         return false;
693     }
694     const std::int32_t min_rows =
695         options.parallel_min_rows > 0 ? options.parallel_min_rows
696         : LCQI_LLAMA_DEFAULT_PARALLEL_MIN_ROWS;
697     if (rows < checked_size(min_rows, "parallel_min_rows")) {
698         return false;
699     }
700     if (options.worker_count > 1) {
701         return true;
702     }
703     return columns >= checked_size(LCQI_LLAMA_PARALLEL_MIN_COLUMNS, "parallel_min_columns") ||
704           rows * columns >= static_cast<std::size_t>(LCQI_LLAMA_PARALLEL_MIN_OPS);
705 }
706
707 [[nodiscard]] std::size_t parallel_row_grain(std::size_t rows,
708                                              const LlamaParallelWorkerPool& worker_pool) {
709     const std::size_t workers = checked_size(worker_pool.worker_count(), "worker_count");
710     const std::size_t target_tasks = workers * LCQI_LLAMA_PARALLEL_ROW_BLOCK_MULTIPLIER;
711     return std::max<std::size_t>(1, (rows + target_tasks - 1) / target_tasks);
712 }
713

```



```

714 [[nodiscard]] bool model_has_parallel_work(const LlamaGgufModel& model,
715                                             const LlamaGgufExecutionOptions& ←
    ↪ options) {
716     if (options.worker_count == 1) {
717         return false;
718     }
719     const DecoderConfig& config = model.decoder.config;
720     const std::size_t hidden_size = checked_size(config.hidden_size, "↪
    ↪ hidden_size");
721     const std::size_t kv_width = checked_size(config.kv_heads * config.head_dim↪
    ↪ , "kv_width");
722     const std::size_t intermediate_size =
723         checked_size(config.intermediate_size, "intermediate_size");
724     const std::size_t vocab_size = checked_size(config.vocab_size, "vocab_size"↪
    ↪ );
725     const std::int32_t min_rows =
726         options.parallel_min_rows > 0 ? options.parallel_min_rows
727             : LCQI_LLAMA_DEFAULT_PARALLEL_MIN_ROWS;
728     const auto large_enough = [min_rows](std::size_t rows, std::size_t columns)↪
    ↪ {
729         if (rows < checked_size(min_rows, "parallel_min_rows")) {
730             return false;
731         }
732         return columns >= checked_size(LCQI_LLAMA_PARALLEL_MIN_COLUMNS,
733                                         "parallel_min_columns") ||
734             rows * columns >= static_cast<std::size_t>(↪
    ↪ LCQI_LLAMA_PARALLEL_MIN_OPS);
735     };
736     if (options.worker_count > 1) {
737         return vocab_size >= checked_size(min_rows, "parallel_min_rows") ||
738             hidden_size >= checked_size(min_rows, "parallel_min_rows") ||
739             kv_width >= checked_size(min_rows, "parallel_min_rows") ||
740             intermediate_size >= checked_size(min_rows, "parallel_min_rows")↪
    ↪ ;
741     }
742     return large_enough(vocab_size, hidden_size) ||
743         large_enough(hidden_size, hidden_size) ||
744         large_enough(kv_width, hidden_size) ||
745         large_enough(intermediate_size, hidden_size) ||
746         large_enough(hidden_size, intermediate_size);
747 }
748
749 [[nodiscard]] std::unique_ptr<LlamaParallelWorkerPool> make_worker_pool(
750     const LlamaGgufModel& model,
751     const LlamaGgufExecutionOptions& options) {
752     if (!model_has_parallel_work(model, options)) {
753         return nullptr;
754     }
755     return std::make_unique<LlamaParallelWorkerPool>(options.worker_count);
756 }
757
758 void linear_f32_fallback_parallel(const LlamaGgufLinearWeights& layer,

```



```

759         std::span<const float> input,
760         std::span<float> output,
761         const LlamaGgufExecutionOptions& options,
762         LlamaParallelWorkerPool* worker_pool) {
763     if (layer.f32_weights.size() !=
764         checked_size(layer.input_size, "linear input size") *
765         checked_size(layer.output_size, "linear output size")) {
766         throw std::runtime_error("LLaMA GGUF F32 linear weight size mismatch");
767     }
768     const std::size_t input_size = checked_size(layer.input_size, "linear input ←
↪ size");
769     const std::size_t output_size = checked_size(layer.output_size, "linear ←
↪ output size");
770     if (!should_parallelize_rows(options, output_size, input_size, worker_pool) ←
↪ ) {
771         linear_f32_fallback(layer, input, output);
772         return;
773     }
774
775     const float* weights = layer.f32_weights.data();
776     const float* input_data = input.data();
777     float* output_data = output.data();
778     const auto rows_fn = [weights, input_data, input_size, output_data](
779         std::size_t begin,
780         std::size_t end,
781         std::size_t worker_index) {
782         (void)worker_index;
783         linear_f32_rows_unchecked(weights,
784             input_data,
785             nullptr,
786             input_size,
787             begin,
788             end,
789             output_data);
790     };
791     worker_pool->parallel_for_rows(0,
792         output_size,
793         parallel_row_grain(output_size, *worker_pool ←
↪ ),
794         rows_fn);
795 }
796
797 void linear_f32_batch_parallel(std::span<const float> weights,
798     std::size_t input_size,
799     std::size_t output_size,
800     std::span<const float> input,
801     std::size_t batch_size,
802     std::span<float> output,
803     const LlamaGgufExecutionOptions& options,
804     LlamaParallelWorkerPool* worker_pool) {
805     if (weights.size() != input_size * output_size) {
806         throw std::runtime_error("LLaMA GGUF F32 linear weight size mismatch");

```

```

807     }
808     if (input.size() != batch_size * input_size || output.size() != batch_size ←
↪ * output_size) {
809         throw std::runtime_error("LLaMA GGUF F32 batch linear shape mismatch");
810     }
811     if (!should_parallelize_rows(options, output_size, input_size, worker_pool) ←
↪ ) {
812         linear_f32_batch_rows_unchecked(weights.data(),
813                                         input.data(),
814                                         nullptr,
815                                         input_size,
816                                         batch_size,
817                                         0,
818                                         output_size,
819                                         output_size,
820                                         output.data());
821         return;
822     }
823
824     const float* weights_data = weights.data();
825     const float* input_data = input.data();
826     float* output_data = output.data();
827     const auto rows_fn = [weights_data, input_data, input_size, batch_size, ←
↪ output_size, output_data](
828         std::size_t begin,
829         std::size_t end,
830         std::size_t worker_index) {
831         (void)worker_index;
832         linear_f32_batch_rows_unchecked(weights_data,
833                                         input_data,
834                                         nullptr,
835                                         input_size,
836                                         batch_size,
837                                         begin,
838                                         end,
839                                         output_size,
840                                         output_data);
841     };
842     worker_pool->parallel_for_rows(0,
843                                   output_size,
844                                   parallel_row_grain(output_size, *worker_pool ←
↪ ),
845                                   rows_fn);
846 }
847
848 void linear_f32_fallback_batch_parallel(const LlamaGgufLinearWeights& layer,
849                                         std::span<const float> input,
850                                         std::size_t batch_size,
851                                         std::span<float> output,
852                                         const LlamaGgufExecutionOptions& ←
↪ options,
853                                         LlamaParallelWorkerPool* worker_pool) {

```

```

854     linear_f32_batch_parallel(layer.f32_weights,
855                               checked_size(layer.input_size, "linear input size",
856                               ↪ "),
857                               checked_size(layer.output_size, "linear output ↪
858                               ↪ size"),
859                               input,
860                               batch_size,
861                               output,
862                               options,
863                               worker_pool);
864 }
865
866 [[nodiscard]] F32RowMax max_dot_f32_rows_parallel(const float* weights,
867                                                    const float* input,
868                                                    std::size_t input_size,
869                                                    std::size_t row_count,
870                                                    const ↪
871                                                    ↪ LlamaGgufExecutionOptions& options,
872                                                    LlamaParallelWorkerPool* ↪
873                                                    ↪ worker_pool) {
874     if (!should_parallelize_rows(options, row_count, input_size, worker_pool)) ↪
875     ↪ {
876         return max_dot_f32_rows_unchecked(weights, input, input_size, 0, ↪
877         ↪ row_count);
878     }
879
880     const std::size_t worker_count = checked_size(worker_pool->worker_count(), ↪
881     ↪ "worker_count");
882     std::vector<F32RowMax> local_best(worker_count);
883     std::vector<std::uint8_t> has_result(worker_count, 0U);
884     const auto rows_fn = [weights, input, input_size, &local_best, &has_result↪
885     ↪ ](
886         std::size_t begin,
887         std::size_t end,
888         std::size_t worker_index) {
889         if (begin >= end) {
890             return;
891         }
892         local_best[worker_index] =
893             max_dot_f32_rows_unchecked(weights, input, input_size, begin, end);
894         has_result[worker_index] = 1U;
895     };
896     worker_pool->parallel_for_rows(0,
897                                   row_count,
898                                   parallel_row_grain(row_count, *worker_pool),
899                                   rows_fn);
900
901     F32RowMax best;
902     best.row = 0;
903     best.value = -std::numeric_limits<float>::infinity();
904     bool found = false;
905     for (std::size_t index = 0; index < local_best.size(); ++index) {

```

```

898     if (has_result[index] == 0U) {
899         continue;
900     }
901     const F32RowMax& candidate = local_best[index];
902     if (!found || candidate.value > best.value) {
903         best = candidate;
904         found = true;
905     }
906 }
907 if (!found) {
908     return max_dot_f32_rows_unchecked(weights, input, input_size, 0, ↵
↵ row_count);
909 }
910 return best;
911 }
912
913 [[nodiscard]] F32RowMax max_dot_ggml_quantized_q8_0_parallel(
914     const LlamaGgufLinearWeights& layer,
915     std::span<const Q8_0InputBlock> q8_0_input,
916     const LlamaGgufExecutionOptions& options,
917     LlamaParallelWorkerPool* worker_pool) {
918     if (q8_0_input.size() != checked_size(layer.input_size / ↵
↵ LCQI_Q8_0_INPUT_BLOCK_VALUES,
919                                         "Q8_0 max-dot input block count")) {
920         throw std::runtime_error("LLaMA GGUF Q8_0 max-dot scratch block count ↵
↵ mismatch");
921     }
922     const std::size_t input_size = checked_size(layer.input_size, "linear input ↵
↵ size");
923     const std::size_t output_size = checked_size(layer.output_size, "linear ↵
↵ output size");
924     if (!should_parallelize_rows(options, output_size, input_size, worker_pool) ↵
↵ ) {
925         return max_dot_ggml_quantized_q8_0(layer.source_type,
926                                             layer.quantized_bytes,
927                                             layer.output_size,
928                                             layer.input_size,
929                                             q8_0_input);
930     }
931
932     const std::size_t worker_count = checked_size(worker_pool->worker_count(), ↵
↵ "worker_count");
933     std::vector<F32RowMax> local_best(worker_count);
934     std::vector<std::uint8_t> has_result(worker_count, 0U);
935     const auto rows_fn = [&layer, q8_0_input, &local_best, &has_result](
936         std::size_t begin,
937         std::size_t end,
938         std::size_t worker_index) {
939         if (begin >= end) {
940             return;
941         }
942         local_best[worker_index] =

```

```

943         max_dot_ggml_quantized_q8_0_rows_unchecked(layer.source_type,
944                                                     layer.quantized_bytes,
945                                                     layer.output_size,
946                                                     layer.input_size,
947                                                     q8_0_input,
948                                                     static_cast<std::int64_t>↵
↵ >(begin),
949                                                     static_cast<std::int64_t>↵
↵ >(end));
950         has_result[worker_index] = 1U;
951     };
952     worker_pool->parallel_for_rows(0,
953                                     output_size,
954                                     parallel_row_grain(output_size, *worker_pool↵
↵ ),
955                                     rows_fn);
956
957     F32RowMax best;
958     best.row = 0;
959     best.value = -std::numeric_limits<float>::infinity();
960     bool found = false;
961     for (std::size_t index = 0; index < local_best.size(); ++index) {
962         if (has_result[index] == 0U) {
963             continue;
964         }
965         const F32RowMax& candidate = local_best[index];
966         if (!found || candidate.value > best.value) {
967             best = candidate;
968             found = true;
969         }
970     }
971     if (!found) {
972         return max_dot_ggml_quantized_q8_0(layer.source_type,
973                                             layer.quantized_bytes,
974                                             layer.output_size,
975                                             layer.input_size,
976                                             q8_0_input);
977     }
978     return best;
979 }
980
981 void linear_q4_k_direct(const LlamaGgufLinearWeights& layer,
982                        std::span<const Q8KBlock> q8_input,
983                        std::span<float> output) {
984     if (q8_input.size() !=
985         checked_size(layer.input_size / LCQI_QK_K_BLOCK_VALUES, "Q8_K input ↵
↵ block count")) {
986         throw std::runtime_error("LLaMA GGUF Q4_K scratch block count mismatch"↵
↵ );
987     }
988     matvec_q4_k_q8(layer.quantized_bytes,
989                    layer.output_size,

```

[illegible]

```

1035     throw std::runtime_error("LLaMA GGUF Q5_0 packed scratch block count ←
↪ mismatch");
1036 }
1037 matvec_q5_0_packed_q8_0(layer.q5_0_packed_blocks,
1038                          layer.output_size,
1039                          layer.input_size,
1040                          q8_0_input,
1041                          output);
1042 }
1043
1044 void linear_q5_0_packed_direct_parallel(const LlamaGgufLinearWeights& layer,
1045                                         std::span<const Q8_0InputBlock> ←
↪ q8_0_input,
1046                                         std::span<float> output,
1047                                         const LlamaGgufExecutionOptions& ←
↪ options,
1048                                         LlamaParallelWorkerPool* worker_pool) {
1049     if (q8_0_input.size() != checked_size(layer.input_size / ←
↪ LCQI_Q8_0_INPUT_BLOCK_VALUES,
1050                                         "Q8_0 input block count")) {
1051         throw std::runtime_error("LLaMA GGUF Q5_0 packed scratch block count ←
↪ mismatch");
1052     }
1053     const std::size_t input_size = checked_size(layer.input_size, "linear input←
↪ size");
1054     const std::size_t output_size = checked_size(layer.output_size, "linear ←
↪ output size");
1055     if (!should_parallelize_rows(options, output_size, input_size, worker_pool)←
↪ ) {
1056         linear_q5_0_packed_direct(layer, q8_0_input, output);
1057         return;
1058     }
1059
1060     const auto rows_fn = [&layer, q8_0_input, output](
1061         std::size_t begin,
1062         std::size_t end,
1063         std::size_t worker_index) {
1064         (void)worker_index;
1065         matvec_q5_0_packed_q8_0_rows_unchecked(layer.q5_0_packed_blocks,
1066                                                 layer.output_size,
1067                                                 layer.input_size,
1068                                                 q8_0_input,
1069                                                 output,
1070                                                 static_cast<std::int64_t>(begin)←
↪ ,
1071                                                 static_cast<std::int64_t>(end));
1072     };
1073     worker_pool->parallel_for_rows(0,
1074                                    output_size,
1075                                    parallel_row_grain(output_size, *worker_pool←
↪ ),
1076                                    rows_fn);

```

```

1077 }
1078
1079 void quantize_q8_0_batch_to(std::span<const float> input,
1080                             std::size_t input_size,
1081                             std::size_t batch_size,
1082                             std::span<Q8_0InputBlock> output) {
1083     const std::size_t blocks_per_input =
1084         checked_size(static_cast<std::int64_t>(input_size) / ←
1085         ↪ LCQI_Q8_0_INPUT_BLOCK_VALUES,
1086                     "Q8_0 input block count");
1087     if (input.size() != batch_size * input_size ||
1088         output.size() != batch_size * blocks_per_input) {
1089         throw std::runtime_error("LLaMA GGUF Q8_0 batch quantization shape ←
1090         ↪ mismatch");
1091     }
1092     for (std::size_t batch = 0; batch < batch_size; ++batch) {
1093         quantize_q8_0_input_to(input.subspan(batch * input_size, input_size),
1094                                output.subspan(batch * blocks_per_input, ←
1095                                ↪ blocks_per_input));
1096     }
1097 }
1098
1099 void linear_ggml_quantized_direct(const LlamaGgufLinearWeights& layer,
1100                                   std::span<const Q8_0InputBlock> q8_0_input,
1101                                   std::span<float> output) {
1102     if (q8_0_input.size() != checked_size(layer.input_size / ←
1103     ↪ LCQI_Q8_0_INPUT_BLOCK_VALUES,
1104         "Q8_0 input block count")) {
1105         throw std::runtime_error("LLaMA GGUF Q8_0 scratch block count mismatch" ←
1106         ↪ );
1107     }
1108     matvec_ggml_quantized_q8_0(layer.source_type,
1109                                layer.quantized_bytes,
1110                                layer.output_size,
1111                                layer.input_size,
1112                                q8_0_input,
1113                                output);
1114 }
1115
1116 void linear_ggml_quantized_direct_parallel(const LlamaGgufLinearWeights& layer,
1117                                             std::span<const Q8_0InputBlock> ←
1118                                             ↪ q8_0_input,
1119                                             std::span<float> output,
1120                                             const LlamaGgufExecutionOptions& ←
1121                                             ↪ options,
1122                                             LlamaParallelWorkerPool* worker_pool ←
1123                                             ↪ ) {
1124     if (q8_0_input.size() != checked_size(layer.input_size / ←
1125     ↪ LCQI_Q8_0_INPUT_BLOCK_VALUES,
1126         "Q8_0 input block count")) {
1127         throw std::runtime_error("LLaMA GGUF Q8_0 scratch block count mismatch" ←
1128         ↪ );

```



```

1119     }
1120     const std::size_t input_size = checked_size(layer.input_size, "linear input ←
↪ size");
1121     const std::size_t output_size = checked_size(layer.output_size, "linear ←
↪ output size");
1122     if (!should_parallelize_rows(options, output_size, input_size, worker_pool) ←
↪ ) {
1123         linear_ggml_quantized_direct(layer, q8_0_input, output);
1124         return;
1125     }
1126
1127     const auto rows_fn = [&layer, q8_0_input, output](
1128         std::size_t begin,
1129         std::size_t end,
1130         std::size_t worker_index) {
1131         (void)worker_index;
1132         matvec_ggml_quantized_q8_0_rows_unchecked(layer.source_type,
1133             layer.quantized_bytes,
1134             layer.output_size,
1135             layer.input_size,
1136             q8_0_input,
1137             output,
1138             static_cast<std::int64_t>( ←
↪ begin),
1139             static_cast<std::int64_t>(end ←
↪ ));
1140     };
1141     worker_pool->parallel_for_rows(0,
1142         output_size,
1143         parallel_row_grain(output_size, *worker_pool ←
↪ ),
1144         rows_fn);
1145 }
1146
1147 void linear_ggml_quantized_direct_batch_parallel(const LlamaGgufLinearWeights& ←
↪ layer,
1148     std::span<const float> input,
1149     std::size_t batch_size,
1150     std::vector<Q8_0InputBlock>& ←
↪ q8_0_scratch,
1151     std::span<float> output,
1152     const ←
↪ LlamaGgufExecutionOptions& options,
1153     LlamaParallelWorkerPool* ←
↪ worker_pool) {
1154     const std::size_t input_size = checked_size(layer.input_size, "linear input ←
↪ size");
1155     const std::size_t output_size = checked_size(layer.output_size, "linear ←
↪ output size");
1156     if (input.size() != batch_size * input_size || output.size() != batch_size ←
↪ * output_size) {
1157         throw std::runtime_error("LLaMA GGUF quantized batch linear shape ←

```

```

1158     ↪ mismatch");
1159 }
1160 if (layer.input_size <= 0 || layer.input_size % ↪
    ↪ LCQI_Q8_0_INPUT_BLOCK_VALUES != 0) {
1161     throw std::runtime_error("LLaMA GGUF quantized batch input is not Q8_0 ↪
    ↪ compatible");
1162 }
1163 const std::size_t blocks_per_input =
1164     checked_size(layer.input_size / LCQI_Q8_0_INPUT_BLOCK_VALUES,
1165                 "Q8_0 input block count");
1166 q8_0_scratch.resize(batch_size * blocks_per_input);
1167 quantize_q8_0_batch_to(input, input_size, batch_size, q8_0_scratch);
1168
1169 if (!should_parallelize_rows(options, output_size, input_size, worker_pool)↪
    ↪ ) {
1170     matvec_ggml_quantized_q8_0_batch_rows_unchecked(layer.source_type,
1171                                                     layer.quantized_bytes,
1172                                                     layer.output_size,
1173                                                     layer.input_size,
1174                                                     q8_0_scratch,
1175                                                     ↪ static_cast<std::↪
    ↪ int64_t>(batch_size),
1176                                                     ↪
1177                                                     output,
1178                                                     layer.output_size,
1179                                                     0,
1180                                                     layer.output_size);
1181     return;
1182 }
1183
1184 const auto rows_fn = [&layer, &q8_0_scratch, batch_size, output](
1185     std::size_t begin,
1186     std::size_t end,
1187     std::size_t worker_index) {
1188     (void)worker_index;
1189     matvec_ggml_quantized_q8_0_batch_rows_unchecked(
1190         layer.source_type,
1191         layer.quantized_bytes,
1192         layer.output_size,
1193         layer.input_size,
1194         q8_0_scratch,
1195         ↪ static_cast<std::int64_t>(batch_size),
1196         output,
1197         layer.output_size,
1198         ↪ static_cast<std::int64_t>(begin),
1199         ↪ static_cast<std::int64_t>(end));
1200 };
1201 worker_pool->parallel_for_rows(0,
1202                               output_size,
1203                               ↪ parallel_row_grain(output_size, *worker_pool↪
    ↪ ),
1204                               rows_fn);
1205 }

```

```

1204
1205 void linear_gguf(const LlamaGgufLinearWeights& layer,
1206                 std::span<const float> input,
1207                 std::span<const Q8KBlock> q8_input,
1208                 std::span<const Q8_0InputBlock> q8_0_input,
1209                 std::span<float> output,
1210                 LlamaGgufLinearOp operation,
1211                 LlamaGgufHotspotReport& hotspots,
1212                 const LlamaGgufExecutionOptions& options,
1213                 LlamaParallelWorkerPool* worker_pool) {
1214     validate_linear_shape(layer, input, output);
1215     double elapsed = 0.0;
1216     switch (layer.storage) {
1217         case LlamaGgufLinearStorage::q4_k:
1218             elapsed = measure_ms([&]() {
1219                 linear_q4_k_direct_parallel(layer, q8_input, output, options, ↵
↵ worker_pool);
1220             });
1221             hotspots.q4_k_direct_ms += elapsed;
1222             ++hotspots.q4_k_direct_calls;
1223             break;
1224         case LlamaGgufLinearStorage::q5_0_packed:
1225             elapsed = measure_ms([&]() {
1226                 linear_q5_0_packed_direct_parallel(layer,
1227                                                     q8_0_input,
1228                                                     output,
1229                                                     options,
1230                                                     worker_pool);
1231             });
1232             hotspots.q5_0_packed_ms += elapsed;
1233             ++hotspots.q5_0_packed_calls;
1234             break;
1235         case LlamaGgufLinearStorage::q6_k_direct:
1236             elapsed = measure_ms([&]() {
1237                 linear_ggml_quantized_direct_parallel(layer,
1238                                                         q8_0_input,
1239                                                         output,
1240                                                         options,
1241                                                         worker_pool);
1242             });
1243             hotspots.q6_k_direct_ms += elapsed;
1244             ++hotspots.q6_k_direct_calls;
1245             break;
1246         case LlamaGgufLinearStorage::q8_0_direct:
1247             elapsed = measure_ms([&]() {
1248                 linear_ggml_quantized_direct_parallel(layer,
1249                                                         q8_0_input,
1250                                                         output,
1251                                                         options,
1252                                                         worker_pool);
1253             });
1254             hotspots.q8_0_direct_ms += elapsed;

```

```

1255         ++hotspots.q8_0_direct_calls;
1256         break;
1257     case LlamaGgufLinearStorage::ggml_quantized:
1258         elapsed = measure_ms([&]() {
1259             linear_ggml_quantized_direct_parallel(layer,
1260                                                     q8_0_input,
1261                                                     output,
1262                                                     options,
1263                                                     worker_pool);
1264         });
1265         hotspots.ggml_direct_ms += elapsed;
1266         ++hotspots.ggml_direct_calls;
1267         break;
1268     case LlamaGgufLinearStorage::f32:
1269         elapsed = measure_ms([&]() {
1270             linear_f32_fallback_parallel(layer, input, output, options, ←
1271 ↪ worker_pool);
1272         });
1273         hotspots.f32_fallback_ms += elapsed;
1274         ++hotspots.f32_fallback_calls;
1275         break;
1276     }
1277     linear_hotspot_field(hotspots, operation) += elapsed;
1278 }
1279 [[nodiscard]] bool uses_q4_k_direct(const LlamaGgufLinearWeights& layer) ←
1280 ↪ noexcept;
1281 [[nodiscard]] bool uses_q5_0_packed_direct(const LlamaGgufLinearWeights& layer) ←
1282 ↪ noexcept;
1283 [[nodiscard]] bool uses_ggml_direct(const LlamaGgufLinearWeights& layer) ←
1284 ↪ noexcept;
1285 [[nodiscard]] bool uses_q8_0_activation_direct(const LlamaGgufLinearWeights& ←
1286 ↪ layer) noexcept;
1287 [[nodiscard]] std::size_t q8_scratch_blocks(std::int32_t input_size);
1288 void prepare_q8_if_needed(std::span<const float> input,
1289                             std::span<Q8KBlock> scratch,
1290                             bool needed);
1291 [[nodiscard]] std::size_t q8_0_scratch_blocks(std::int32_t input_size);
1292 void prepare_q8_0_if_needed(std::span<const float> input,
1293                             std::span<Q8_0InputBlock> scratch,
1294                             bool needed);
1295 void linear_gguf_batch(const LlamaGgufLinearWeights& layer,
1296                        std::span<const float> input,
1297                        std::size_t batch_size,
1298                        std::vector<Q8KBlock>& q8_scratch,
1299                        std::vector<Q8_0InputBlock>& q8_0_scratch,
1300                        std::span<float> output,
1301                        LlamaGgufLinearOp operation,
1302                        LlamaGgufHotspotReport& hotspots,
1303                        const LlamaGgufExecutionOptions& options,
1304                        LlamaParallelWorkerPool* worker_pool) {

```

```

1302     const std::size_t input_size = checked_size(layer.input_size, "linear input" ←
1303     ↪ size");
1304     const std::size_t output_size = checked_size(layer.output_size, "linear" ←
1305     ↪ output size");
1306     if (input.size() != batch_size * input_size || output.size() != batch_size ←
1307     ↪ * output_size) {
1308         throw std::runtime_error("LLaMA GGUF batch linear input/output size ←
1309     ↪ mismatch");
1310     }
1311
1312     if (layer.storage == LlamaGgufLinearStorage::f32) {
1313         const double elapsed = measure_ms([&]() {
1314             linear_f32_fallback_batch_parallel(layer,
1315                 input,
1316                 batch_size,
1317                 output,
1318                 options,
1319                 worker_pool);
1320         });
1321         hotspots.f32_fallback_ms += elapsed;
1322         ++hotspots.f32_fallback_calls;
1323         linear_hotspot_field(hotspots, operation) += elapsed;
1324         return;
1325     }
1326
1327     if (layer.storage == LlamaGgufLinearStorage::q5_0_packed) {
1328         const double elapsed = measure_ms([&]() {
1329             linear_f32_fallback_batch_parallel(layer,
1330                 input,
1331                 batch_size,
1332                 output,
1333                 options,
1334                 worker_pool);
1335         });
1336         hotspots.q5_0_batch_f32_ms += elapsed;
1337         ++hotspots.q5_0_batch_f32_calls;
1338         linear_hotspot_field(hotspots, operation) += elapsed;
1339         return;
1340     }
1341
1342     if (layer.storage == LlamaGgufLinearStorage::q6_k_direct ||
1343         layer.storage == LlamaGgufLinearStorage::q8_0_direct ||
1344         layer.storage == LlamaGgufLinearStorage::ggml_quantized) {
1345         const double elapsed = measure_ms([&]() {
1346             linear_ggml_quantized_direct_batch_parallel(layer,
1347                 input,
1348                 batch_size,
1349                 q8_0_scratch,
1350                 output,
1351                 options,
1352                 worker_pool);
1353         });
1354     }

```

```

1350     if (layer.storage == LlamaGgufLinearStorage::q6_k_direct) {
1351         hotspots.q6_k_direct_ms += elapsed;
1352         ++hotspots.q6_k_direct_calls;
1353     } else if (layer.storage == LlamaGgufLinearStorage::q8_0_direct) {
1354         hotspots.q8_0_direct_ms += elapsed;
1355         ++hotspots.q8_0_direct_calls;
1356     } else {
1357         hotspots.ggml_direct_ms += elapsed;
1358         ++hotspots.ggml_direct_calls;
1359     }
1360     linear_hotspot_field(hotspots, operation) += elapsed;
1361     return;
1362 }
1363
1364 q8_scratch.resize(q8_scratch_blocks(layer.input_size));
1365 q8_0_scratch.resize(q8_0_scratch_blocks(layer.input_size));
1366 for (std::size_t batch = 0; batch < batch_size; ++batch) {
1367     const std::span<const float> token_input =
1368         input.subspan(batch * input_size, input_size);
1369     std::span<float> token_output =
1370         output.subspan(batch * output_size, output_size);
1371     prepare_q8_if_needed(token_input, q8_scratch, uses_q4_k_direct(layer));
1372     prepare_q8_0_if_needed(token_input,
1373                             q8_0_scratch,
1374                             uses_q8_0_activation_direct(layer));
1375     linear_gguf(layer,
1376                 token_input,
1377                 q8_scratch,
1378                 q8_0_scratch,
1379                 token_output,
1380                 operation,
1381                 hotspots,
1382                 options,
1383                 worker_pool);
1384 }
1385 }
1386
1387 [[nodiscard]] bool uses_q4_k_direct(const LlamaGgufLinearWeights& layer) ↵
1388     ↪ noexcept {
1389     return layer.storage == LlamaGgufLinearStorage::q4_k;
1390 }
1391
1392 [[nodiscard]] bool uses_q5_0_packed_direct(const LlamaGgufLinearWeights& layer) ↵
1393     ↪ noexcept {
1394     return layer.storage == LlamaGgufLinearStorage::q5_0_packed;
1395 }
1396
1397 [[nodiscard]] bool uses_ggml_direct(const LlamaGgufLinearWeights& layer) ↵
1398     ↪ noexcept {
1399     return layer.storage == LlamaGgufLinearStorage::ggml_quantized;
1400 }

```

```

1399 [[nodiscard]] bool uses_q8_0_activation_direct(const LlamaGgufLinearWeights& ←
    ↪ layer) noexcept {
1400     return uses_q5_0_packed_direct(layer) ||
1401         layer.storage == LlamaGgufLinearStorage::q6_k_direct ||
1402         layer.storage == LlamaGgufLinearStorage::q8_0_direct ||
1403         uses_ggml_direct(layer);
1404 }
1405
1406 [[nodiscard]] std::size_t q8_scratch_blocks(std::int32_t input_size) {
1407     if (input_size <= 0 || input_size % LCQI_QK_K_BLOCK_VALUES != 0) {
1408         return 0;
1409     }
1410     return checked_size(input_size / LCQI_QK_K_BLOCK_VALUES, "Q8_K scratch ←
    ↪ blocks");
1411 }
1412
1413 void prepare_q8_if_needed(std::span<const float> input,
1414                          std::span<Q8KBlock> scratch,
1415                          bool needed) {
1416     if (!needed) {
1417         return;
1418     }
1419     quantize_q8_k_input_to(input, scratch);
1420 }
1421
1422 [[nodiscard]] std::size_t q8_0_scratch_blocks(std::int32_t input_size) {
1423     if (input_size <= 0 || input_size % LCQI_Q8_0_INPUT_BLOCK_VALUES != 0) {
1424         return 0;
1425     }
1426     return checked_size(input_size / LCQI_Q8_0_INPUT_BLOCK_VALUES, "Q8_0 ←
    ↪ scratch blocks");
1427 }
1428
1429 void prepare_q8_0_if_needed(std::span<const float> input,
1430                             std::span<Q8_0InputBlock> scratch,
1431                             bool needed) {
1432     if (!needed) {
1433         return;
1434     }
1435     quantize_q8_0_input_to(input, scratch);
1436 }
1437
1438 struct LlamaWorkspace {
1439     std::vector<float> hidden;
1440     std::vector<float> normed;
1441     std::vector<float> query;
1442     std::vector<float> key;
1443     std::vector<float> value;
1444     std::vector<float> attention;
1445     std::vector<float> projected;
1446     std::vector<float> gate;
1447     std::vector<float> up;

```

```

1448     std::vector<float> mlp_hidden;
1449     std::vector<float> mlp_out;
1450     std::vector<float> final_norm;
1451     std::vector<Q8KBlock> normed_q8;
1452     std::vector<Q8KBlock> attention_q8;
1453     std::vector<Q8KBlock> mlp_hidden_q8;
1454     std::vector<Q8_0InputBlock> normed_q8_0;
1455     std::vector<Q8_0InputBlock> attention_q8_0;
1456     std::vector<Q8_0InputBlock> mlp_hidden_q8_0;
1457
1458     explicit LlamaWorkspace(const DecoderConfig& config)
1459         : hidden(check_size(config.hidden_size, "hidden_size"), 0.0F),
1460           normed(hidden.size(), 0.0F),
1461           query(hidden.size(), 0.0F),
1462           key(check_size(config.kv_heads * config.head_dim, "kv_width"), 0.0F ←
↪ ),
1463           value(key.size(), 0.0F),
1464           attention(hidden.size(), 0.0F),
1465           projected(hidden.size(), 0.0F),
1466           gate(check_size(config.intermediate_size, "intermediate_size"), 0.0 ←
↪ F),
1467           up(gate.size(), 0.0F),
1468           mlp_hidden(gate.size(), 0.0F),
1469           mlp_out(hidden.size(), 0.0F),
1470           final_norm(hidden.size(), 0.0F),
1471           normed_q8(q8_scratch_blocks(config.hidden_size)),
1472           attention_q8(normed_q8.size()),
1473           mlp_hidden_q8(q8_scratch_blocks(config.intermediate_size)),
1474           normed_q8_0(q8_0_scratch_blocks(config.hidden_size)),
1475           attention_q8_0(normed_q8_0.size()),
1476           mlp_hidden_q8_0(q8_0_scratch_blocks(config.intermediate_size)) {}
1477 };
1478
1479 struct LlamaBatchWorkspace {
1480     std::size_t batch_size = 0;
1481     std::vector<float> hidden;
1482     std::vector<float> normed;
1483     std::vector<float> query;
1484     std::vector<float> key;
1485     std::vector<float> value;
1486     std::vector<float> attention;
1487     std::vector<float> projected;
1488     std::vector<float> gate;
1489     std::vector<float> up;
1490     std::vector<float> mlp_hidden;
1491     std::vector<float> mlp_out;
1492     std::vector<float> final_norm;
1493     std::vector<Q8KBlock> q8_scratch;
1494     std::vector<Q8_0InputBlock> q8_0_scratch;
1495
1496     LlamaBatchWorkspace(const DecoderConfig& config, std::size_t ←
↪ requested_batch_size)

```



```

1497         : batch_size(requested_batch_size),
1498         hidden(batch_size * checked_size(config.hidden_size, "hidden_size"), ←
↪ 0.0F),
1499         normed(hidden.size(), 0.0F),
1500         query(hidden.size(), 0.0F),
1501         key(batch_size * checked_size(config.kv_heads * config.head_dim, "↪
↪ kv_width"), 0.0F),
1502         value(key.size(), 0.0F),
1503         attention(hidden.size(), 0.0F),
1504         projected(hidden.size(), 0.0F),
1505         gate(batch_size * checked_size(config.intermediate_size, "↪
↪ intermediate_size"), 0.0F),
1506         up(gate.size(), 0.0F),
1507         mlp_hidden(gate.size(), 0.0F),
1508         mlp_out(hidden.size(), 0.0F),
1509         final_norm(checked_size(config.hidden_size, "hidden_size"), 0.0F) {}
1510 };
1511
1512 void forward_llama_layer(const DecoderConfig& config,
1513                         const LlamaGgufDecoderLayerWeights& layer,
1514                         std::int32_t layer_id,
1515                         std::int32_t model_position,
1516                         ReferenceKVCache& cache,
1517                         LlamaWorkspace& workspace,
1518                         LlamaGgufHotspotReport& hotspots,
1519                         const LlamaGgufExecutionOptions& options,
1520                         LlamaParallelWorkerPool* worker_pool) {
1521     hotspots.rms_norm_ms += measure_ms([&]() {
1522         rms_norm(workspace.hidden,
1523                 layer.rms_attention_weight,
1524                 config.rms_epsilon,
1525                 workspace.normed);
1526     });
1527     prepare_q8_if_needed(workspace.normed,
1528                         workspace.normed_q8,
1529     ↪ wk) ||
↪ wk) ||
1530         uses_q4_k_direct(layer.wq) || uses_q4_k_direct(layer.wk) ||
1531         uses_q4_k_direct(layer.wv));
1532     prepare_q8_0_if_needed(workspace.normed,
1533                         workspace.normed_q8_0,
1534                         uses_q8_0_activation_direct(layer.wq) ||
1535                         uses_q8_0_activation_direct(layer.wk) ||
1536                         uses_q8_0_activation_direct(layer.wv));
1537     linear_gguf(layer.wq,
1538                 workspace.normed,
1539                 workspace.normed_q8,
1540                 workspace.normed_q8_0,
1541                 workspace.query,
1542                 LlamaGgufLinearOp::wq,
1543                 hotspots,
1544                 options,
1545                 worker_pool);

```

```

1545     linear_gguf(layer.wk,
1546                 workspace.normed,
1547                 workspace.normed_q8,
1548                 workspace.normed_q8_0,
1549                 workspace.key,
1550                 LlamaGgufLinearOp::wk,
1551                 hotspots,
1552                 options,
1553                 worker_pool);
1554     linear_gguf(layer.wv,
1555                 workspace.normed,
1556                 workspace.normed_q8,
1557                 workspace.normed_q8_0,
1558                 workspace.value,
1559                 LlamaGgufLinearOp::wv,
1560                 hotspots,
1561                 options,
1562                 worker_pool);
1563     hotspots.rope_ms += measure_ms([&]() {
1564         apply_rope(workspace.query,
1565                   config.query_heads,
1566                   config.head_dim,
1567                   model_position,
1568                   config.rope_base);
1569         apply_rope(workspace.key,
1570                   config.kv_heads,
1571                   config.head_dim,
1572                   model_position,
1573                   config.rope_base);
1574     });
1575     cache.append(layer_id, model_position, workspace.key, workspace.value);
1576     hotspots.attention_ms += measure_ms([&]() {
1577         attention_decode(config,
1578                         cache,
1579                         layer_id,
1580                         model_position,
1581                         workspace.query,
1582                         workspace.attention);
1583     });
1584     prepare_q8_if_needed(workspace.attention,
1585                          workspace.attention_q8,
1586                          uses_q4_k_direct(layer.wo));
1587     prepare_q8_0_if_needed(workspace.attention,
1588                            workspace.attention_q8_0,
1589                            uses_q8_0_activation_direct(layer.wo));
1590     linear_gguf(layer.wo,
1591                 workspace.attention,
1592                 workspace.attention_q8,
1593                 workspace.attention_q8_0,
1594                 workspace.projected,
1595                 LlamaGgufLinearOp::wo,
1596                 hotspots,

```

```

1597         options,
1598         worker_pool);
1599 add_inplace(workspace.hidden, workspace.projected);
1600
1601 hotspots.rms_norm_ms += measure_ms([&]() {
1602     rms_norm(workspace.hidden, layer.rms_mlp_weight, config.rms_epsilon, ←
↪ workspace.normed);
1603 });
1604 prepare_q8_if_needed(workspace.normed,
1605                     workspace.normed_q8,
1606                     uses_q4_k_direct(layer.w_gate) || uses_q4_k_direct(←
↪ layer.w_up));
1607 prepare_q8_0_if_needed(workspace.normed,
1608                       workspace.normed_q8_0,
1609                       uses_q8_0_activation_direct(layer.w_gate) ||
1610                       uses_q8_0_activation_direct(layer.w_up));
1611 linear_gguf(layer.w_gate,
1612            workspace.normed,
1613            workspace.normed_q8,
1614            workspace.normed_q8_0,
1615            workspace.gate,
1616            LlamaGgufLinearOp::w_gate,
1617            hotspots,
1618            options,
1619            worker_pool);
1620 linear_gguf(layer.w_up,
1621            workspace.normed,
1622            workspace.normed_q8,
1623            workspace.normed_q8_0,
1624            workspace.up,
1625            LlamaGgufLinearOp::w_up,
1626            hotspots,
1627            options,
1628            worker_pool);
1629 for (std::size_t index = 0; index < workspace.mlp_hidden.size(); ++index) {
1630     workspace.mlp_hidden[index] = silu(workspace.gate[index]) * workspace.←
↪ up[index];
1631 }
1632 prepare_q8_if_needed(workspace.mlp_hidden,
1633                     workspace.mlp_hidden_q8,
1634                     uses_q4_k_direct(layer.w_down));
1635 prepare_q8_0_if_needed(workspace.mlp_hidden,
1636                       workspace.mlp_hidden_q8_0,
1637                       uses_q8_0_activation_direct(layer.w_down));
1638 linear_gguf(layer.w_down,
1639            workspace.mlp_hidden,
1640            workspace.mlp_hidden_q8,
1641            workspace.mlp_hidden_q8_0,
1642            workspace.mlp_out,
1643            LlamaGgufLinearOp::w_down,
1644            hotspots,
1645            options,

```

```

1646         worker_pool);
1647     add_inplace(workspace.hidden, workspace.mlp_out);
1648 }
1649
1650 void forward_llama_layer_prefill_batch(const DecoderConfig& config,
1651                                       const LlamaGgufDecoderLayerWeights& ←
1652   ↪ layer,
1653                                       std::int32_t layer_id,
1654                                       std::int32_t position_begin,
1655                                       ReferenceKVCache& cache,
1656                                       LlamaBatchWorkspace& workspace,
1657                                       LlamaGgufHotspotReport& hotspots,
1658                                       const LlamaGgufExecutionOptions& options↪
1659   ↪ ,
1660                                       LlamaParallelWorkerPool* worker_pool) {
1661     const std::size_t hidden_size = checked_size(config.hidden_size, "↪
1662     ↪ hidden_size");
1663     const std::size_t kv_width = checked_size(config.kv_heads * config.head_dim↪
1664     ↪ , "kv_width");
1665     const std::size_t intermediate_size =
1666         checked_size(config.intermediate_size, "intermediate_size");
1667
1668     hotspots.rms_norm_ms += measure_ms([&]() {
1669         for (std::size_t batch = 0; batch < workspace.batch_size; ++batch) {
1670             rms_norm.mutable_row_span(workspace.hidden, batch, hidden_size),
1671             layer.rms_attention_weight,
1672             config.rms_epsilon,
1673             mutable_row_span(workspace.normed, batch, hidden_size));
1674         }
1675     });
1676     linear_gguf_batch(layer.wq,
1677                       workspace.normed,
1678                       workspace.batch_size,
1679                       workspace.q8_scratch,
1680                       workspace.q8_0_scratch,
1681                       workspace.query,
1682                       LlamaGgufLinearOp::wq,
1683                       hotspots,
1684                       options,
1685                       worker_pool);
1686     linear_gguf_batch(layer.wk,
1687                       workspace.normed,
1688                       workspace.batch_size,
1689                       workspace.q8_scratch,
1690                       workspace.q8_0_scratch,
1691                       workspace.key,
1692                       LlamaGgufLinearOp::wk,
1693                       hotspots,
1694                       options,
1695                       worker_pool);
1696     linear_gguf_batch(layer.wv,
1697                       workspace.normed,

```

```

1694         workspace.batch_size,
1695         workspace.q8_scratch,
1696         workspace.q8_0_scratch,
1697         workspace.value,
1698         LlamaGgufLinearOp::wv,
1699         hotspots,
1700         options,
1701         worker_pool);
1702     hotspots.ropes_ms += measure_ms([&]() {
1703         for (std::size_t batch = 0; batch < workspace.batch_size; ++batch) {
1704             const std::int32_t model_position =
1705                 position_begin + static_cast<std::int32_t>(batch);
1706             apply_rope(mutable_row_span(workspace.query, batch, hidden_size),
1707                 config.query_heads,
1708                 config.head_dim,
1709                 model_position,
1710                 config.rope_base);
1711             apply_rope(mutable_row_span(workspace.key, batch, kv_width),
1712                 config.kv_heads,
1713                 config.head_dim,
1714                 model_position,
1715                 config.rope_base);
1716         }
1717     });
1718     for (std::size_t batch = 0; batch < workspace.batch_size; ++batch) {
1719         const std::int32_t model_position = position_begin + static_cast<std::int32_t>(batch);
1720         cache.append(layer_id,
1721             model_position,
1722             mutable_row_span(workspace.key, batch, kv_width),
1723             mutable_row_span(workspace.value, batch, kv_width));
1724     }
1725     hotspots.attention_ms += measure_ms([&]() {
1726         for (std::size_t batch = 0; batch < workspace.batch_size; ++batch) {
1727             const std::int32_t model_position =
1728                 position_begin + static_cast<std::int32_t>(batch);
1729             attention_decode(config,
1730                 cache,
1731                 layer_id,
1732                 model_position,
1733                 mutable_row_span(workspace.query, batch,
1734                     ↪ hidden_size),
1735                 mutable_row_span(workspace.attention, batch,
1736                     ↪ hidden_size));
1737         }
1738     });
1739     linear_gguf_batch(layer.wo,
1740         workspace.attention,
1741         workspace.batch_size,
1742         workspace.q8_scratch,
1743         workspace.q8_0_scratch,
1744         workspace.projected,

```

```

1743         LlamaGgufLinearOp::wo,
1744         hotspots,
1745         options,
1746         worker_pool);
1747     for (std::size_t batch = 0; batch < workspace.batch_size; ++batch) {
1748         add_inplace.mutable_row_span(workspace.hidden, batch, hidden_size),
1749         mutable_row_span(workspace.projected, batch, hidden_size));
1750     }
1751
1752     hotspots.rms_norm_ms += measure_ms([&]() {
1753         for (std::size_t batch = 0; batch < workspace.batch_size; ++batch) {
1754             rms_norm.mutable_row_span(workspace.hidden, batch, hidden_size),
1755             layer.rms_mlp_weight,
1756             config.rms_epsilon,
1757             mutable_row_span(workspace.normed, batch, hidden_size));
1758         }
1759     });
1760     linear_gguf_batch(layer.w_gate,
1761         workspace.normed,
1762         workspace.batch_size,
1763         workspace.q8_scratch,
1764         workspace.q8_0_scratch,
1765         workspace.gate,
1766         LlamaGgufLinearOp::w_gate,
1767         hotspots,
1768         options,
1769         worker_pool);
1770     linear_gguf_batch(layer.w_up,
1771         workspace.normed,
1772         workspace.batch_size,
1773         workspace.q8_scratch,
1774         workspace.q8_0_scratch,
1775         workspace.up,
1776         LlamaGgufLinearOp::w_up,
1777         hotspots,
1778         options,
1779         worker_pool);
1780     for (std::size_t batch = 0; batch < workspace.batch_size; ++batch) {
1781         std::span<float> mlp_hidden = mutable_row_span(workspace.mlp_hidden,
1782             batch,
1783             intermediate_size);
1784         std::span<float> gate = mutable_row_span(workspace.gate, batch, ↵
        ↵ intermediate_size);
1785         std::span<float> up = mutable_row_span(workspace.up, batch, ↵
        ↵ intermediate_size);
1786         for (std::size_t index = 0; index < intermediate_size; ++index) {
1787             mlp_hidden[index] = silu(gate[index]) * up[index];
1788         }
1789     }
1790     linear_gguf_batch(layer.w_down,
1791         workspace.mlp_hidden,
1792         workspace.batch_size,

```



```

1838     });
1839     return static_cast<std::int32_t>(best.row);
1840 }
1841
1842 std::vector<float> logits_checked_size(config.vocab_size, "vocab_size"), ←
    ↪ 0.0F);
1843 const std::vector<Q8KBlock> final_hidden_q8 =
1844     uses_q4_k_direct(model.decoder.lm_head)
1845     ? quantize_q8_k_input(final_hidden)
1846     : std::vector<Q8KBlock>{};
1847 const std::vector<Q8_0InputBlock> final_hidden_q8_0 =
1848     uses_q8_0_activation_direct(model.decoder.lm_head)
1849     ? quantize_q8_0_input(final_hidden)
1850     : std::vector<Q8_0InputBlock>{};
1851 linear_gguf(model.decoder.lm_head,
1852             final_hidden,
1853             final_hidden_q8,
1854             final_hidden_q8_0,
1855             logits,
1856             LlamaGgufLinearOp::lm_head,
1857             hotspots,
1858             options,
1859             worker_pool);
1860 best.row = 0;
1861 best.value = logits.front();
1862 for (std::size_t row = 1; row < logits.size(); ++row) {
1863     if (logits[row] > best.value) {
1864         best.row = row;
1865         best.value = logits[row];
1866     }
1867 }
1868 return static_cast<std::int32_t>(best.row);
1869 }
1870
1871 [[nodiscard]] std::int32_t step_llama_decoder(const LlamaGgufModel& model,
1872                                             const DecoderConfig& ←
    ↪ active_config,
1873                                             ReferenceKVCache& cache,
1874                                             LlamaWorkspace& workspace,
1875                                             std::int32_t token_id,
1876                                             bool predict,
1877                                             LlamaGgufHotspotReport& hotspots,
1878                                             const LlamaGgufExecutionOptions& ←
    ↪ options,
1879                                             LlamaParallelWorkerPool* ←
    ↪ worker_pool) {
1880     const DecoderConfig& original_config = model.decoder.config;
1881     if (token_id < 0 || token_id >= original_config.vocab_size) {
1882         throw std::runtime_error("LLaMA token id out of vocabulary");
1883     }
1884     const std::int32_t model_position = cache.filled_tokens();
1885     if (model_position >= active_config.max_context) {

```



```

1886     throw std::runtime_error("LLaMA generation exceeded active context");
1887 }
1888
1889 const std::span<const float> embedding =
1890     row_span(model.decoder.token_embedding, token_id, original_config.↵
↵ hidden_size);
1891 std::copy(embedding.begin(), embedding.end(), workspace.hidden.begin());
1892 for (std::int32_t layer_id = 0;
1893     layer_id < static_cast<std::int32_t>(model.decoder.layers.size());
1894     ++layer_id) {
1895     forward_llama_layer(active_config,
1896         model.decoder.layers[checked_size(layer_id, "↵
↵ layer_id")],
1897         layer_id,
1898         model_position,
1899         cache,
1900         workspace,
1901         hotspots,
1902         options,
1903         worker_pool);
1904 }
1905 if (!predict) {
1906     return -1;
1907 }
1908 hotspots.rms_norm_ms += measure_ms([&]() {
1909     rms_norm(workspace.hidden,
1910         model.decoder.final_norm_weight,
1911         active_config.rms_epsilon,
1912         workspace.final_norm);
1913 });
1914 return predict_next_token(model, workspace.final_norm, hotspots, options, ↵
↵ worker_pool);
1915 }
1916
1917 [[nodiscard]] std::int32_t prefill_llama_decoder_batch(const LlamaGgufModel& ↵
↵ model,
1918                                                         const DecoderConfig& ↵
↵ active_config,
1919                                                         ReferenceKVCache& cache,
1920                                                         std::span<const std::↵
↵ int32_t> prompt_ids,
1921                                                         LlamaGgufHotspotReport& ↵
↵ hotspots,
1922                                                         const ↵
↵ LlamaGgufExecutionOptions& options,
1923                                                         LlamaParallelWorkerPool*&↵
↵ worker_pool) {
1924     const DecoderConfig& original_config = model.decoder.config;
1925     const std::size_t batch_size = prompt_ids.size();
1926     const std::size_t hidden_size = checked_size(original_config.hidden_size, "↵
↵ hidden_size");
1927     LlamaBatchWorkspace workspace(active_config, batch_size);

```

```

1928     for (std::size_t batch = 0; batch < batch_size; ++batch) {
1929         const std::int32_t token_id = prompt_ids[batch];
1930         if (token_id < 0 || token_id >= original_config.vocab_size) {
1931             throw std::runtime_error("LLaMA token id out of vocabulary");
1932         }
1933         const std::span<const float> embedding =
1934             row_span(model.decoder.token_embedding, token_id, original_config.↵
↵ hidden_size);
1935         std::copy(embedding.begin(),
1936                 embedding.end(),
1937                 workspace.hidden.begin() + static_cast<std::ptrdiff_t>(batch ↵
↵ * hidden_size));
1938     }
1939
1940     const std::int32_t position_begin = cache.filled_tokens();
1941     if (position_begin + static_cast<std::int32_t>(batch_size) > active_config.↵
↵ max_context) {
1942         throw std::runtime_error("LLaMA generation exceeded active context");
1943     }
1944     for (std::int32_t layer_id = 0;
1945         layer_id < static_cast<std::int32_t>(model.decoder.layers.size());
1946         ++layer_id) {
1947         forward_llama_layer_prefill_batch(active_config,
1948                                         model.decoder.layers[checked_size(↵
↵ layer_id,
1949                                         "↵
↵ layer_id)],
1950                                         layer_id,
1951                                         position_begin,
1952                                         cache,
1953                                         workspace,
1954                                         hotspots,
1955                                         options,
1956                                         worker_pool);
1957     }
1958     hotspots.rms_norm_ms += measure_ms([&]() {
1959         rms_norm(mutable_row_span(workspace.hidden, batch_size - 1U, ↵
↵ hidden_size),
1960                 model.decoder.final_norm_weight,
1961                 active_config.rms_epsilon,
1962                 workspace.final_norm);
1963     });
1964     return predict_next_token(model, workspace.final_norm, hotspots, options, ↵
↵ worker_pool);
1965 }
1966
1967 [[nodiscard]] std::size_t kv_cache_bytes(const DecoderConfig& config, std::↵
↵ int32_t layer_count) {
1968     return checked_size(layer_count, "layer_count") *
1969            checked_size(config.max_context, "max_context") *
1970            checked_size(config.kv_heads, "kv_heads") *
1971            checked_size(config.head_dim, "head_dim") * sizeof(float) * 2U;

```

```

1972 }
1973
1974 [[nodiscard]] std::unordered_map<std::string, std::int32_t> ↵
    ↵ build_vocab_from_tokens(
1975     std::span<const std::string> tokens) {
1976     std::unordered_map<std::string, std::int32_t> vocab;
1977     for (std::size_t index = 0; index < tokens.size(); ++index) {
1978         vocab.emplace(tokens[index], static_cast<std::int32_t>(index));
1979     }
1980     return vocab;
1981 }
1982
1983 [[nodiscard]] std::unordered_map<std::string, std::int32_t> build_merge_ranks(
1984     std::span<const std::string> merges) {
1985     std::unordered_map<std::string, std::int32_t> ranks;
1986     for (std::size_t index = 0; index < merges.size(); ++index) {
1987         std::istringstream line_stream(merges[index]);
1988         std::string left;
1989         std::string right;
1990         if (!(line_stream >> left >> right)) {
1991             throw std::runtime_error("invalid GGUF tokenizer merge: " + merges[↵
    ↵ index]);
1992         }
1993         ranks.emplace(left + "\n" + right, static_cast<std::int32_t>(index));
1994     }
1995     return ranks;
1996 }
1997
1998 [[nodiscard]] bool starts_with(std::string_view text,
1999                                std::size_t offset,
2000                                std::string_view needle) {
2001     return offset + needle.size() <= text.size() &&
2002         text.substr(offset, needle.size()) == needle;
2003 }
2004
2005 [[nodiscard]] std::vector<std::pair<std::string, std::int32_t>> special_tokens(
2006     const Gpt2Tokenizer& tokenizer) {
2007     std::vector<std::pair<std::string, std::int32_t>> result;
2008     for (const auto& [token, id] : tokenizer.vocab) {
2009         if (token.size() >= 4 && token.front() == '<' && token.find('|') != std↵
    ↵ ::string::npos) {
2010             result.emplace_back(token, id);
2011         }
2012     }
2013     std::sort(result.begin(),
2014               result.end(),
2015               [](const auto& lhs, const auto& rhs) {
2016                   return lhs.first.size() > rhs.first.size();
2017               });
2018     return result;
2019 }
2020

```

```

2021 [[nodiscard]] std::vector<std::int32_t> encode_with_special_tokens(
2022     const Gpt2Tokenizer& tokenizer,
2023     std::string_view text) {
2024     const std::vector<std::pair<std::string, std::int32_t>> specials = ↵
    ↵ special_tokens(tokenizer);
2025     std::vector<std::int32_t> ids;
2026     std::size_t offset = 0;
2027     while (offset < text.size()) {
2028         bool matched_special = false;
2029         for (const auto& [token, id] : specials) {
2030             if (starts_with(text, offset, token)) {
2031                 ids.push_back(id);
2032                 offset += token.size();
2033                 matched_special = true;
2034                 break;
2035             }
2036         }
2037         if (matched_special) {
2038             continue;
2039         }
2040
2041         std::size_t next_special = text.size();
2042         for (const auto& [token, id] : specials) {
2043             (void)id;
2044             const std::size_t found = text.find(token, offset);
2045             if (found != std::string_view::npos) {
2046                 next_special = std::min(next_special, found);
2047             }
2048         }
2049         const std::vector<std::int32_t> chunk =
2050             gpt2_encode(tokenizer, text.substr(offset, next_special - offset));
2051         ids.insert(ids.end(), chunk.begin(), chunk.end());
2052         offset = next_special;
2053     }
2054     return ids;
2055 }
2056
2057 } // namespace
2058
2059 LlamaGgufLoadedModel load_llama_gguf_reference_model(const std::filesystem::↵
    ↵ path& gguf_path) {
2060     LlamaGgufLoadedModel loaded;
2061     const Clock::time_point manifest_begin = Clock::now();
2062     const GgufManifest manifest = load_gguf_manifest(gguf_path);
2063     loaded.report.manifest_ms = elapsed_ms(manifest_begin, Clock::now());
2064
2065     const Clock::time_point weights_begin = Clock::now();
2066     loaded.model.architecture = metadata_string(manifest, ↵
    ↵ LCQI_LLAMA_ARCHITECTURE_KEY);
2067     loaded.model.name = metadata_string(manifest, LCQI_LLAMA_NAME_KEY);
2068     DecoderConfig& config = loaded.model.decoder.config;
2069     config.hidden_size = metadata_i32(manifest, LCQI_LLAMA_EMBEDDING_LENGTH_KEY)↵

```

```

↪ );
2070 const std::int32_t layer_count = metadata_i32(manifest, ↪
↪ LCQI_LLAMA_BLOCK_COUNT_KEY);
2071 config.intermediate_size = metadata_i32(manifest, ↪
↪ LCQI_LLAMA_FEED_FORWARD_LENGTH_KEY);
2072 config.query_heads = metadata_i32(manifest, LCQI_LLAMA_HEAD_COUNT_KEY);
2073 config.kv_heads = metadata_i32(manifest, LCQI_LLAMA_KV_HEAD_COUNT_KEY);
2074 config.head_dim = config.hidden_size / config.query_heads;
2075 const std::int32_t rope_dimension = metadata_i32(manifest, ↪
↪ LCQI_LLAMA_ROPE_DIMENSION_KEY);
2076 if (rope_dimension != config.head_dim) {
2077     throw std::runtime_error("LLaMA rope dimension does not match head_dim"↪
↪ );
2078 }
2079 config.rope_base = metadata_f32(manifest, LCQI_LLAMA_ROPE_BASE_KEY);
2080 config.max_context = metadata_i32(manifest, LCQI_LLAMA_CONTEXT_LENGTH_KEY);
2081 config.vocab_size = metadata_i32(manifest, LCQI_LLAMA_VOCAB_SIZE_KEY);
2082 config.rms_epsilon = metadata_f32(manifest, LCQI_LLAMA_RMS_EPSILON_KEY);
2083 loaded.model.bos_token_id =
2084     optional_metadata_i32(manifest, LCQI_LLAMA_BOS_TOKEN_KEY, ↪
↪ LCQI_LLAMA_DEFAULT_BOS_TOKEN);
2085 loaded.model.eos_token_id =
2086     optional_metadata_i32(manifest, LCQI_LLAMA_EOS_TOKEN_KEY, ↪
↪ LCQI_LLAMA_DEFAULT_EOS_TOKEN);
2087 loaded.model.unknown_token_id = optional_metadata_i32(
2088     manifest,
2089     LCQI_LLAMA_UNKNOWN_TOKEN_KEY,
2090     LCQI_LLAMA_DEFAULT_UNK_TOKEN);
2091 validate_llama_config(config, layer_count);
2092
2093 const std::array<std::int64_t, 2> embedding_shape{config.hidden_size, ↪
↪ config.vocab_size};
2094 loaded.model.decoder.token_embedding = read_tensor_f32(gguf_path,
2095                                                         manifest,
2096                                                         "token_embd.weight",
2097                                                         embedding_shape,
2098                                                         loaded.report);
2099 const std::array<std::int64_t, 1> hidden_shape{config.hidden_size};
2100 loaded.model.decoder.final_norm_weight = read_tensor_f32(gguf_path,
2101                                                         manifest,
2102                                                         "output_norm.↪
↪ weight",
2103                                                         hidden_shape,
2104                                                         loaded.report);
2105 loaded.model.decoder.layers.resize(checkered_size(layer_count, "layer_count")↪
↪ );
2106 const std::int32_t kv_width = config.kv_heads * config.head_dim;
2107 for (std::int32_t layer_id = 0; layer_id < layer_count; ++layer_id) {
2108     LlamaGgufDecoderLayerWeights& layer =
2109         loaded.model.decoder.layers[checked_size(layer_id, "layer_id")];
2110     layer.rms_attention_weight = read_tensor_f32(
2111         gguf_path,

```



```

2157     layer.w_down = make_linear_from_gguf(gguf_path,
2158                                         manifest,
2159                                         layer_tensor_name(layer_id, "↵
↵ ffn_down.weight"),
2160                                         config.intermediate_size,
2161                                         config.hidden_size,
2162                                         loaded.report);
2163 }
2164
2165 if (manifest.find_tensor("output.weight") != nullptr) {
2166     loaded.model.lm_head_tied_to_embedding = false;
2167     loaded.model.decoder.lm_head = make_linear_from_gguf(gguf_path,
2168                                                         manifest,
2169                                                         "output.weight",
2170                                                         config.hidden_size↵
↵ ,
2171                                                         config.vocab_size,
2172                                                         loaded.report);
2173 } else {
2174     loaded.model.lm_head_tied_to_embedding = true;
2175     loaded.model.decoder.lm_head.input_size = config.hidden_size;
2176     loaded.model.decoder.lm_head.output_size = config.vocab_size;
2177     loaded.model.decoder.tied_lm_head =
2178         make_tied_lm_head_from_embedding(gguf_path,
2179                                         manifest,
2180                                         config.hidden_size,
2181                                         config.vocab_size,
2182                                         loaded.report);
2183 }
2184 loaded.report.weights_ms = elapsed_ms(weights_begin, Clock::now());
2185 return loaded;
2186 }
2187
2188 Gpt2Tokenizer load_gpt2_tokenizer_from_gguf(const std::filesystem::path& ↵
↵ gguf_path) {
2189     const GgufManifest manifest = load_gguf_manifest(gguf_path);
2190     const GgufMetadataEntry& tokens_entry = require_metadata(manifest,
2191                                                             ↵
↵ LCQI_LLAMA_TOKENIZER_TOKENS_KEY);
2192     const GgufMetadataEntry& merges_entry = require_metadata(manifest,
2193                                                             ↵
↵ LCQI_LLAMA_TOKENIZER_MERGES_KEY);
2194     if (tokens_entry.string_values.empty() || merges_entry.string_values.empty↵
↵ ()) {
2195         throw std::runtime_error("GGUF tokenizer tokens or merges are empty");
2196     }
2197     Gpt2Tokenizer tokenizer;
2198     tokenizer.bos_token_id =
2199         optional_metadata_i32(manifest, LCQI_LLAMA_BOS_TOKEN_KEY, ↵
↵ LCQI_LLAMA_DEFAULT_BOS_TOKEN);
2200     tokenizer.eos_token_id =
2201         optional_metadata_i32(manifest, LCQI_LLAMA_EOS_TOKEN_KEY, ↵

```

```

    ↪ LCQI_LLAMA_DEFAULT_EOS_TOKEN);
2202 tokenizer.id_to_token = tokens_entry.string_values;
2203 tokenizer.vocab = build_vocab_from_tokens(tokenizer.id_to_token);
2204 tokenizer.bpe_ranks = build_merge_ranks(merges_entry.string_values);
2205 return tokenizer;
2206 }
2207
2208 std::vector<std::int32_t> llama_gguf_chat_prompt_ids(const Gpt2Tokenizer& ↪
    ↪ tokenizer,
2209
    std::string_view ↪
    ↪ user_prompt,
2210
    std::int32_t bos_token_id)↪
    ↪ {
2211     std::string prompt;
2212     prompt += "<|im_start|>system\n";
2213     prompt += "You are a helpful AI assistant named SmolLM, trained by Hugging ↪
    ↪ Face";
2214     prompt += "<|im_end|>\n";
2215     prompt += "<|im_start|>user\n";
2216     prompt += user_prompt;
2217     prompt += "<|im_end|>\n";
2218     prompt += "<|im_start|>assistant\n";
2219     std::vector<std::int32_t> ids = encode_with_special_tokens(tokenizer, ↪
    ↪ prompt);
2220     if (ids.empty() && bos_token_id >= 0) {
2221         ids.push_back(bos_token_id);
2222     }
2223     return ids;
2224 }
2225
2226 LlamaGgufGenerationResult llama_gguf_generate_greedy(const LlamaGgufModel& ↪
    ↪ model,
2227
    std::span<const std::↪
    ↪ int32_t> prompt_ids,
2228
    std::int32_t ↪
    ↪ max_new_tokens) {
2229     return llama_gguf_generate_greedy(model,
2230         prompt_ids,
2231         max_new_tokens,
2232         LlamaGgufExecutionOptions{});
2233 }
2234
2235 LlamaGgufGenerationResult llama_gguf_generate_greedy(
2236     const LlamaGgufModel& model,
2237     std::span<const std::int32_t> prompt_ids,
2238     std::int32_t max_new_tokens,
2239     const LlamaGgufExecutionOptions& options) {
2240     if (prompt_ids.empty()) {
2241         throw std::runtime_error("LLaMA generation requires at least one prompt↪
    ↪ token");
2242     }
2243     if (max_new_tokens < 0) {

```



```

2244     throw std::runtime_error("max_new_tokens cannot be negative");
2245 }
2246 const std::int32_t layer_count = static_cast<std::int32_t>(model.decoder.↵
↵ layers.size());
2247 validate_llama_config(model.decoder.config, layer_count);
2248 const std::int32_t active_context =
2249     static_cast<std::int32_t>(prompt_ids.size()) + std::max(max_new_tokens, ↵
↵ 1);
2250 if (active_context > model.decoder.config.max_context) {
2251     throw std::runtime_error("LLaMA prompt plus generation exceeds model ↵
↵ context");
2252 }
2253
2254 DecoderConfig active_config = model.decoder.config;
2255 active_config.max_context = active_context;
2256 ReferenceKVCache cache(active_config, layer_count);
2257 LlamaWorkspace workspace(active_config);
2258 std::unique_ptr<LlamaParallelWorkerPool> worker_pool = make_worker_pool(↵
↵ model, options);
2259
2260 LlamaGgufGenerationResult result;
2261 result.prompt_ids.assign(prompt_ids.begin(), prompt_ids.end());
2262 result.generated_ids.assign(prompt_ids.begin(), prompt_ids.end());
2263 result.kv_cache_bytes = kv_cache_bytes(active_config, layer_count);
2264 result.worker_count = worker_pool ? worker_pool->worker_count() : 1;
2265 if (max_new_tokens == 0) {
2266     return result;
2267 }
2268
2269 const Clock::time_point prefill_begin = Clock::now();
2270 std::int32_t predicted = -1;
2271 if (batch_prefill_enabled() && prompt_ids.size() > 1U) {
2272     predicted = prefill_llama_decoder_batch(model,
2273                                             active_config,
2274                                             cache,
2275                                             prompt_ids,
2276                                             result.hotspots,
2277                                             options,
2278                                             worker_pool.get());
2279 } else {
2280     for (std::size_t index = 0; index + 1 < prompt_ids.size(); ++index) {
2281         static_cast<void>(step_llama_decoder(model,
2282                                             active_config,
2283                                             cache,
2284                                             workspace,
2285                                             prompt_ids[index],
2286                                             false,
2287                                             result.hotspots,
2288                                             options,
2289                                             worker_pool.get()));
2290     }
2291     predicted = step_llama_decoder(model,

```

```

2292         active_config,
2293         cache,
2294         workspace,
2295         prompt_ids.back(),
2296         true,
2297         result.hotspots,
2298         options,
2299         worker_pool.get());
2300     }
2301     result.predicted_first_token = predicted;
2302     result.prefill_steps = prompt_ids.size();
2303     result.prefill_ms = elapsed_ms(prefill_begin, Clock::now());
2304
2305     const Clock::time_point decode_begin = Clock::now();
2306     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
2307         result.generated_ids.push_back(predicted);
2308         if (predicted == model.eos_token_id) {
2309             break;
2310         }
2311         if (step + 1 < max_new_tokens) {
2312             predicted = step_llama_decoder(model,
2313                 active_config,
2314                 cache,
2315                 workspace,
2316                 predicted,
2317                 true,
2318                 result.hotspots,
2319                 options,
2320                 worker_pool.get());
2321             ++result.decode_steps;
2322         }
2323     }
2324     result.decode_ms = elapsed_ms(decode_begin, Clock::now());
2325     result.total_ms = result.prefill_ms + result.decode_ms;
2326     return result;
2327 }
2328
2329 } // namespace lcqi

```

`llama_cli.cpp` 是 `lcqi_llama_gguf` 的命令行入口。它支持 `--ids` 直接给 token id，也支持 `--prompt` 从 GGUF tokenizer arrays 构造 chat prompt；`--threads0` 自动选择 worker，`--threads1` 强制串行，`--threadsN` 固定 worker 数。`--benchmark` 输出 manifest、权重加载、prefill、decode、KV cache、worker count、量化权重字节、Q5_0 packed 字节、Q5_0 batch F32 sidecar 字节、Q6_K direct 实验字节、Q8_0 direct 字节、fallback f32 字节、direct/fallback tensor 数、per-op hotspot 和 direct/fallback/sidecar call 数。环境变量 `LCQI_LLAMA_Q4_DIRECT=0`、`LCQI_LLAMA_Q5_0_PACKED=0`、`LCQI_LLAMA_Q8_0_DIRECT=0`、`LCQI_LLAMA_Q8_0_TIED_LM_HEAD=0` 和 `LCQI_LLAMA_BATCH_PREFILL=0` 分别关闭默认子优化，用于同二进制 A/B；`LCQI_LLAMA_Q6_K_DIRECT=1` 单独打开 Q6_K direct 实验；`LCQI_LLAMA_GGML_DIRECT=1` 会打开完整 Q5_0/Q6_K/Q8_0 实验 direct path，用来证明覆盖率和速度必须分开验收。报告中 `decode_tokens_per_second` 只统计真正喂入新 token 后再次预测的 decode step，不能把首 token

prefill 的预测误算成绩 decode。

Listing 16.14 src/llama_cli.cpp

```

1  #include <lcqi/llama_gguf.hpp>
2
3  #include <chrono>
4  #include <cstdlib>
5  #include <exception>
6  #include <filesystem>
7  #include <iostream>
8  #include <sstream>
9  #include <stdexcept>
10 #include <string>
11 #include <string_view>
12 #include <vector>
13
14 namespace {
15
16 constexpr std::int32_t LCQI_LLAMA_DEFAULT_MAX_NEW_TOKENS = 1;
17 constexpr std::string_view LCQI_LLAMA_IDS_PREFIX = "--ids=";
18 constexpr std::string_view LCQI_LLAMA_PROMPT_PREFIX = "--prompt=";
19 constexpr std::string_view LCQI_LLAMA_MAX_NEW_PREFIX = "--max-new=";
20 constexpr std::string_view LCQI_LLAMA_THREADS_PREFIX = "--threads=";
21 constexpr const char* LCQI_LLAMA_Q4_DIRECT_ENV = "LCQI_LLAMA_Q4_DIRECT";
22 constexpr const char* LCQI_LLAMA_Q5_0_PACKED_ENV = "LCQI_LLAMA_Q5_0_PACKED";
23 constexpr const char* LCQI_LLAMA_Q6_K_DIRECT_ENV = "LCQI_LLAMA_Q6_K_DIRECT";
24 constexpr const char* LCQI_LLAMA_Q8_0_DIRECT_ENV = "LCQI_LLAMA_Q8_0_DIRECT";
25 constexpr const char* LCQI_LLAMA_Q8_0_TIED_LM_HEAD_ENV =
26     "LCQI_LLAMA_Q8_0_TIED_LM_HEAD";
27 constexpr const char* LCQI_LLAMA_GGML_DIRECT_ENV = "LCQI_LLAMA_GGML_DIRECT";
28 constexpr const char* LCQI_LLAMA_GGML_SELECTIVE_DIRECT_ENV =
29     "LCQI_LLAMA_GGML_SELECTIVE_DIRECT";
30 constexpr std::string_view LCQI_LLAMA_FALSE_ENV = "0";
31 constexpr std::string_view LCQI_LLAMA_TRUE_ENV = "1";
32
33 using Clock = std::chrono::steady_clock;
34
35 struct LlamaCliOptions {
36     std::filesystem::path gguf_path;
37     std::string prompt;
38     std::vector<std::int32_t> ids;
39     std::int32_t max_new_tokens = LCQI_LLAMA_DEFAULT_MAX_NEW_TOKENS;
40     std::int32_t threads = 0;
41     bool use_ids = false;
42     bool benchmark = false;
43     bool decode_text = false;
44 };
45
46 [[nodiscard]] double elapsed_ms(Clock::time_point begin, Clock::time_point end)↵
47     ↵ {
48     return std::chrono::duration<double, std::milli>(end - begin).count();
49 }

```

```

49
50 void print_ids(std::string_view label, const std::vector<std::int32_t>& ids) {
51     std::cout << label;
52     for (const std::int32_t id : ids) {
53         std::cout << ' ' << id;
54     }
55     std::cout << "\n";
56 }
57
58 [[nodiscard]] std::int32_t parse_non_negative_i32(const std::string& text,
59                                                  const char* name) {
60     const int value = std::stoi(text);
61     if (value < 0) {
62         throw std::runtime_error(std::string(name) + " cannot be negative");
63     }
64     return static_cast<std::int32_t>(value);
65 }
66
67 [[nodiscard]] std::vector<std::int32_t> parse_ids(std::string_view text) {
68     std::vector<std::int32_t> ids;
69     std::string normalized(text);
70     for (char& ch : normalized) {
71         if (ch == ',') {
72             ch = ' ';
73         }
74     }
75     std::istringstream input(normalized);
76     std::int32_t id = 0;
77     while (input >> id) {
78         if (id < 0) {
79             throw std::runtime_error("token ids cannot be negative");
80         }
81         ids.push_back(id);
82     }
83     if (ids.empty()) {
84         throw std::runtime_error("--ids requires at least one token id");
85     }
86     return ids;
87 }
88
89 [[nodiscard]] bool consume_prefix(std::string_view arg,
90                                  std::string_view prefix,
91                                  std::string_view& value) {
92     if (arg.substr(0, prefix.size()) != prefix) {
93         return false;
94     }
95     value = arg.substr(prefix.size());
96     return true;
97 }
98
99 [[nodiscard]] LlamaCliOptions parse_options(int argc, char** argv) {
100     if (argc < 2) {

```

```

101     throw std::runtime_error(
102         "usage: lcqi_llama_gguf model.gguf [--prompt TEXT|--ids 1,2,3] "
103         "[--max-new N] [--threads N] [--benchmark] [--decode-text]");
104 }
105 LlamaCliOptions options;
106 options.gguf_path = argv[1];
107 for (int index = 2; index < argc; ++index) {
108     const std::string arg = argv[index];
109     std::string_view value;
110     if (arg == "--benchmark") {
111         options.benchmark = true;
112     } else if (arg == "--decode-text") {
113         options.decode_text = true;
114     } else if (arg == "--ids") {
115         if (index + 1 >= argc) {
116             throw std::runtime_error("--ids requires a value");
117         }
118         ++index;
119         options.ids = parse_ids(argv[index]);
120         options.use_ids = true;
121     } else if (consume_prefix(arg, LCQI_LLAMA_IDS_PREFIX, value)) {
122         options.ids = parse_ids(value);
123         options.use_ids = true;
124     } else if (arg == "--prompt") {
125         if (index + 1 >= argc) {
126             throw std::runtime_error("--prompt requires a value");
127         }
128         ++index;
129         options.prompt = argv[index];
130         options.use_ids = false;
131     } else if (consume_prefix(arg, LCQI_LLAMA_PROMPT_PREFIX, value)) {
132         options.prompt = std::string(value);
133         options.use_ids = false;
134     } else if (arg == "--max-new") {
135         if (index + 1 >= argc) {
136             throw std::runtime_error("--max-new requires a value");
137         }
138         ++index;
139         options.max_new_tokens = parse_non_negative_i32(argv[index], "--max-↵
↵ -new");
140     } else if (consume_prefix(arg, LCQI_LLAMA_MAX_NEW_PREFIX, value)) {
141         options.max_new_tokens =
142             parse_non_negative_i32(std::string(value), "--max-new");
143     } else if (arg == "--threads") {
144         if (index + 1 >= argc) {
145             throw std::runtime_error("--threads requires a value");
146         }
147         ++index;
148         options.threads = parse_non_negative_i32(argv[index], "--threads");
149     } else if (consume_prefix(arg, LCQI_LLAMA_THREADS_PREFIX, value)) {
150         options.threads = parse_non_negative_i32(std::string(value), "--↵
↵ threads");

```

```

151     } else {
152         throw std::runtime_error("unknown option: " + arg);
153     }
154 }
155 if (!options.use_ids && options.prompt.empty()) {
156     options.prompt = "Hello";
157 }
158 return options;
159 }
160
161 [[nodiscard]] std::vector<std::int32_t> make_prompt_ids(const LlamaCliOptions& ←
    ↪ options,
162                                                         lcqi::Gpt2Tokenizer* ←
    ↪ tokenizer,
163                                                         std::int32_t ←
    ↪ bos_token_id) {
164     if (options.use_ids) {
165         return options.ids;
166     }
167     return lcqi::llama_gguf_chat_prompt_ids(*tokenizer, options.prompt, ←
    ↪ bos_token_id);
168 }
169
170 void print_benchmark(const lcqi::LlamaGgufLoadedModel& loaded,
171                     const lcqi::LlamaGgufGenerationResult& result,
172                     std::size_t prompt_size,
173                     std::size_t generated_new_tokens) {
174     const double first_token_tokens_per_second =
175         result.prefill_ms > 0.0 ? 1000.0 / result.prefill_ms : 0.0;
176     const double decode_tokens_per_second =
177         result.decode_ms > 0.0
178         ? static_cast<double>(result.decode_steps) * 1000.0 / result.←
    ↪ decode_ms
179         : 0.0;
180     std::cout << "benchmark_manifest_ms " << loaded.report.manifest_ms << "\n";
181     std::cout << "benchmark_weight_load_ms " << loaded.report.weights_ms << "\n"←
    ↪ ";
182     std::cout << "benchmark_load_ms " << result.load_ms << "\n";
183     std::cout << "benchmark_prefill_ms " << result.prefill_ms << "\n";
184     std::cout << "benchmark_decode_ms " << result.decode_ms << "\n";
185     std::cout << "benchmark_total_ms " << result.total_ms << "\n";
186     std::cout << "benchmark_prompt_tokens " << prompt_size << "\n";
187     std::cout << "benchmark_generated_tokens " << generated_new_tokens << "\n";
188     std::cout << "benchmark_prefill_steps " << result.prefill_steps << "\n";
189     std::cout << "benchmark_decode_steps " << result.decode_steps << "\n";
190     std::cout << "benchmark_worker_count " << result.worker_count << "\n";
191     std::cout << "benchmark_first_token_tokens_per_second "
192         << first_token_tokens_per_second << "\n";
193     std::cout << "benchmark_decode_tokens_per_second " << ←
    ↪ decode_tokens_per_second << "\n";
194     std::cout << "benchmark_kv_cache_bytes " << result.kv_cache_bytes << "\n";
195     std::cout << "benchmark_quantized_weight_bytes "

```

```

196         << loaded.report.quantized_weight_bytes << "\n";
197     std::cout << "benchmark_f32_weight_bytes " << loaded.report.↵
↵ f32_weight_bytes << "\n";
198     std::cout << "benchmark_direct_quantized_weight_bytes "
199         << loaded.report.direct_quantized_weight_bytes << "\n";
200     std::cout << "benchmark_fallback_dequantized_weight_bytes "
201         << loaded.report.fallback_dequantized_weight_bytes << "\n";
202     std::cout << "benchmark_q5_0_packed_weight_bytes "
203         << loaded.report.q5_0_packed_weight_bytes << "\n";
204     std::cout << "benchmark_q5_0_batch_f32_weight_bytes "
205         << loaded.report.q5_0_batch_f32_weight_bytes << "\n";
206     std::cout << "benchmark_q6_k_direct_weight_bytes "
207         << loaded.report.q6_k_direct_weight_bytes << "\n";
208     std::cout << "benchmark_q8_0_direct_weight_bytes "
209         << loaded.report.q8_0_direct_weight_bytes << "\n";
210     std::cout << "benchmark_tensors_loaded " << loaded.report.tensors_loaded <<↵
↵ "\n";
211     std::cout << "benchmark_q4_k_direct_tensors "
212         << loaded.report.q4_k_direct_tensors << "\n";
213     std::cout << "benchmark_q5_0_packed_tensors "
214         << loaded.report.q5_0_packed_tensors << "\n";
215     std::cout << "benchmark_q6_k_direct_tensors "
216         << loaded.report.q6_k_direct_tensors << "\n";
217     std::cout << "benchmark_q8_0_direct_tensors "
218         << loaded.report.q8_0_direct_tensors << "\n";
219     std::cout << "benchmark_ggml_direct_tensors "
220         << loaded.report.ggml_direct_tensors << "\n";
221     std::cout << "benchmark_f32_fallback_tensors "
222         << loaded.report.f32_fallback_tensors << "\n";
223     std::cout << "benchmark_hotspot_rms_norm_ms " << result.hotspots.↵
↵ rms_norm_ms << "\n";
224     std::cout << "benchmark_hotspot_attention_ms " << result.hotspots.↵
↵ attention_ms << "\n";
225     std::cout << "benchmark_hotspot_rope_ms " << result.hotspots.rope_ms << "\n↵
↵ ";
226     std::cout << "benchmark_hotspot_wq_ms " << result.hotspots.wq_ms << "\n";
227     std::cout << "benchmark_hotspot_wk_ms " << result.hotspots.wk_ms << "\n";
228     std::cout << "benchmark_hotspot_wv_ms " << result.hotspots.wv_ms << "\n";
229     std::cout << "benchmark_hotspot_wo_ms " << result.hotspots.wo_ms << "\n";
230     std::cout << "benchmark_hotspot_w_gate_ms " << result.hotspots.w_gate_ms <<↵
↵ "\n";
231     std::cout << "benchmark_hotspot_w_up_ms " << result.hotspots.w_up_ms << "\n↵
↵ ";
232     std::cout << "benchmark_hotspot_w_down_ms " << result.hotspots.w_down_ms <<↵
↵ "\n";
233     std::cout << "benchmark_hotspot_lm_head_ms " << result.hotspots.lm_head_ms ↵
↵ << "\n";
234     std::cout << "benchmark_hotspot_q4_k_direct_ms "
235         << result.hotspots.q4_k_direct_ms << "\n";
236     std::cout << "benchmark_hotspot_q5_0_packed_ms "
237         << result.hotspots.q5_0_packed_ms << "\n";
238     std::cout << "benchmark_hotspot_q5_0_batch_f32_ms "

```



```

239         << result.hotspots.q5_0_batch_f32_ms << "\n";
240     std::cout << "benchmark_hotspot_q6_k_direct_ms "
241         << result.hotspots.q6_k_direct_ms << "\n";
242     std::cout << "benchmark_hotspot_q8_0_direct_ms "
243         << result.hotspots.q8_0_direct_ms << "\n";
244     std::cout << "benchmark_hotspot_ggml_direct_ms "
245         << result.hotspots.ggml_direct_ms << "\n";
246     std::cout << "benchmark_hotspot_f32_fallback_ms "
247         << result.hotspots.f32_fallback_ms << "\n";
248     std::cout << "benchmark_hotspot_q4_k_direct_calls "
249         << result.hotspots.q4_k_direct_calls << "\n";
250     std::cout << "benchmark_hotspot_q5_0_packed_calls "
251         << result.hotspots.q5_0_packed_calls << "\n";
252     std::cout << "benchmark_hotspot_q5_0_batch_f32_calls "
253         << result.hotspots.q5_0_batch_f32_calls << "\n";
254     std::cout << "benchmark_hotspot_q6_k_direct_calls "
255         << result.hotspots.q6_k_direct_calls << "\n";
256     std::cout << "benchmark_hotspot_q8_0_direct_calls "
257         << result.hotspots.q8_0_direct_calls << "\n";
258     std::cout << "benchmark_hotspot_ggml_direct_calls "
259         << result.hotspots.ggml_direct_calls << "\n";
260     std::cout << "benchmark_hotspot_f32_fallback_calls "
261         << result.hotspots.f32_fallback_calls << "\n";
262 }
263
264 [[nodiscard]] bool q4_direct_enabled() noexcept {
265     const char* enabled = std::getenv(LCQI_LLAMA_Q4_DIRECT_ENV);
266     return enabled == nullptr || std::string_view(enabled) != ↵
        ↵ LCQI_LLAMA_FALSE_ENV;
267 }
268
269 [[nodiscard]] bool q5_0_packed_enabled() noexcept {
270     const char* enabled = std::getenv(LCQI_LLAMA_Q5_0_PACKED_ENV);
271     return enabled == nullptr || std::string_view(enabled) != ↵
        ↵ LCQI_LLAMA_FALSE_ENV;
272 }
273
274 [[nodiscard]] bool q6_k_direct_enabled() noexcept {
275     const char* enabled = std::getenv(LCQI_LLAMA_Q6_K_DIRECT_ENV);
276     return enabled != nullptr && std::string_view(enabled) == ↵
        ↵ LCQI_LLAMA_TRUE_ENV;
277 }
278
279 [[nodiscard]] bool q8_0_direct_enabled() noexcept {
280     const char* enabled = std::getenv(LCQI_LLAMA_Q8_0_DIRECT_ENV);
281     return enabled == nullptr || std::string_view(enabled) != ↵
        ↵ LCQI_LLAMA_FALSE_ENV;
282 }
283
284 [[nodiscard]] bool q8_0_tied_lm_head_enabled() noexcept {
285     const char* enabled = std::getenv(LCQI_LLAMA_Q8_0_TIED_LM_HEAD_ENV);
286     return enabled == nullptr || std::string_view(enabled) != ↵

```



```

    ↪ LCQI_LLAMA_FALSE_ENV;
287 }
288
289 [[nodiscard]] bool ggml_direct_enabled() noexcept {
290     const char* enabled = std::getenv(LCQI_LLAMA_GGML_DIRECT_ENV);
291     return enabled != nullptr && std::string_view(enabled) == ↪
    ↪ LCQI_LLAMA_TRUE_ENV;
292 }
293
294 [[nodiscard]] bool ggml_selective_direct_enabled() noexcept {
295     const char* enabled = std::getenv(LCQI_LLAMA_GGML_SELECTIVE_DIRECT_ENV);
296     return enabled != nullptr && std::string_view(enabled) == ↪
    ↪ LCQI_LLAMA_TRUE_ENV;
297 }
298
299 [[nodiscard]] const char* weight_execution_mode() noexcept {
300     if (ggml_direct_enabled()) {
301         return "gguf_mixed_quantized_direct_experimental";
302     }
303     if (ggml_selective_direct_enabled()) {
304         return "gguf_mixed_selective_ggml_direct";
305     }
306     if (q6_k_direct_enabled()) {
307         return "gguf_mixed_q4_k_q5_0_packed_q6_k_q8_0_direct_experimental";
308     }
309     if (!q4_direct_enabled() && !q5_0_packed_enabled() && !q8_0_direct_enabled() ↪
    ↪ () &&
310         !q8_0_tied_lm_head_enabled()) {
311         return "f32_dequantized_reference";
312     }
313     if (q4_direct_enabled() && q5_0_packed_enabled() && q8_0_direct_enabled() ↪
    ↪ &&
314         q8_0_tied_lm_head_enabled()) {
315         return "gguf_mixed_q4_k_q5_0_packed_q8_0_direct";
316     }
317     if (!q4_direct_enabled() && q5_0_packed_enabled()) {
318         return "gguf_mixed_q5_0_packed_direct";
319     }
320     if (q4_direct_enabled() && q5_0_packed_enabled() && ↪
    ↪ q8_0_tied_lm_head_enabled()) {
321         return "gguf_mixed_q4_k_q5_0_packed_q8_0_lm_head_direct";
322     }
323     if (q4_direct_enabled() && q5_0_packed_enabled()) {
324         return "gguf_mixed_q4_k_q5_0_packed_direct";
325     }
326     return "gguf_mixed_q4_k_direct";
327 }
328
329 } // namespace
330
331 int main(int argc, char** argv) {
332     try {

```

```

333     const Clock::time_point total_begin = Clock::now();
334     const LlamaCliOptions options = parse_options(argc, argv);
335     const Clock::time_point load_begin = Clock::now();
336     lcqi::LlamaGgufLoadedModel loaded =
337         lcqi::load_llama_gguf_reference_model(options.gguf_path);
338     const double load_ms = elapsed_ms(load_begin, Clock::now());
339
340     lcqi::Gpt2Tokenizer tokenizer;
341     if (!options.use_ids || options.decode_text) {
342         tokenizer = lcqi::load_gpt2_tokenizer_from_gguf(options.gguf_path);
343     }
344     const std::vector<std::int32_t> prompt_ids =
345         make_prompt_ids(options, &tokenizer, loaded.model.bos_token_id);
346     lcqi::LlamaGgufExecutionOptions execution_options;
347     execution_options.worker_count = options.threads;
348     lcqi::LlamaGgufGenerationResult result =
349         lcqi::llama_gguf_generate_greedy(loaded.model,
350                                         prompt_ids,
351                                         options.max_new_tokens,
352                                         execution_options);
353     result.load_ms = load_ms;
354     result.total_ms = elapsed_ms(total_begin, Clock::now());
355
356     std::cout << "mode llama_gguf_reference\n";
357     std::cout << "weight_execution " << weight_execution_mode() << "\n";
358     std::cout << "model_path " << options.gguf_path.string() << "\n";
359     std::cout << "architecture " << loaded.model.architecture << "\n";
360     std::cout << "model_name " << loaded.model.name << "\n";
361     std::cout << "layers " << loaded.model.decoder.layers.size() << "\n";
362     std::cout << "hidden_size " << loaded.model.decoder.config.hidden_size <<
    ↪ << "\n";
363     std::cout << "query_heads " << loaded.model.decoder.config.query_heads <<
    ↪ << "\n";
364     std::cout << "kv_heads " << loaded.model.decoder.config.kv_heads << "\n"
    ↪ << ";
365     std::cout << "head_dim " << loaded.model.decoder.config.head_dim << "\n"
    ↪ << ";
366     std::cout << "vocab_size " << loaded.model.decoder.config.vocab_size <<
    ↪ << "\n";
367     std::cout << "tie_lm_head_to_embedding "
368         << (loaded.model.lm_head_tied_to_embedding ? 1 : 0) << "\n";
369     print_ids("prompt_ids", result.prompt_ids);
370     print_ids("generated_ids", result.generated_ids);
371     std::cout << "predicted_first_token " << result.predicted_first_token <<
    ↪ << "\n";
372     if (options.decode_text) {
373         std::cout << "generated_text "
374             << lcqi::gpt2_decode(tokenizer, result.generated_ids) <<
    ↪ << "\n";
375     }
376     if (options.benchmark) {
377         const std::size_t generated_new_tokens =

```

```

378         result.generated_ids.size() >= result.prompt_ids.size()
379         ? result.generated_ids.size() - result.prompt_ids.size()
380         : 0;
381     print_benchmark(loaded, result, result.prompt_ids.size(), ↵
↵ generated_new_tokens);
382     }
383     return EXIT_SUCCESS;
384 } catch (const std::exception& error) {
385     std::cerr << "error: " << error.what() << "\n";
386     return EXIT_FAILURE;
387 }
388 }

```

`q4_k_avx2.cpp` 是本轮 Q4_K 优化入口。它不是完整 `llama.cpp` 后端，只把 `Q4_KxQ8_K` 的 32 元素子块点积换成 AVX2 `maddubs` 归约，并提供 `row-range unchecked` 入口，供 LLaMA GGUF decoder 按输出行并行切分。读这份源码时要同时看 `fallback`：环境变量 `LCQI_Q4K_BACKEND=scalar` 可以强制走标量路径，报告脚本用它生成同机同模型的 A/B 证据；而端到端报告会验证 `row-range` 和 `worker pool` 是否真正改善 `prefill/decode`。

Listing 16.15 `src/q4_k_avx2.cpp`

```

1  #include <lcqi/q4_k.hpp>
2
3  #if defined(LCQI_ENABLE_AVX2)
4
5  #include <immintrin.h>
6
7  #include <array>
8  #include <cmath>
9  #include <cstdint>
10 #include <cstdint>
11 #include <cstdlib>
12 #include <cstring>
13
14 namespace lcqi {
15 namespace {
16
17 constexpr std::uint32_t LCQI_Q4K_F32_EXPONENT_MASK = 0x7F800000U;
18 constexpr std::uint32_t LCQI_Q4K_F16_SIGN_MASK = 0x8000U;
19 constexpr std::uint32_t LCQI_Q4K_F16_EXPONENT_MASK = 0x7C00U;
20 constexpr std::uint32_t LCQI_Q4K_F16_MANTISSA_MASK = 0x03FFU;
21 constexpr std::uint32_t LCQI_Q4K_F16_EXPONENT_BIAS = 15U;
22 constexpr std::uint32_t LCQI_Q4K_F32_EXPONENT_BIAS = 127U;
23 constexpr std::uint32_t LCQI_Q4K_F32_MANTISSA_BITS = 23U;
24 constexpr std::uint32_t LCQI_Q4K_F16_MANTISSA_BITS = 10U;
25 constexpr std::uint32_t LCQI_Q4K_F16_TO_F32_SIGN_SHIFT = 16U;
26 constexpr std::uint32_t LCQI_Q4K_BYTE_BITS = 8U;
27 constexpr std::uint8_t LCQI_Q4K_LOW_NIBBLE_MASK = 0x0FU;
28 constexpr std::uint8_t LCQI_Q4K_SIX_BIT_MASK = 63U;
29 constexpr std::int64_t LCQI_Q4K_QS_OFFSET = 16;
30 constexpr std::int64_t LCQI_Q4K_QS_BYTES_PER_PAIR = 32;

```

```

31 constexpr std::int64_t LCQI_Q4K_SUBBLOCK_PAIR_COUNT = 4;
32 constexpr const char* LCQI_Q4K_BACKEND_ENV = "LCQI_Q4K_BACKEND";
33 constexpr const char* LCQI_Q4K_FORCE_SCALAR_ENV = "LCQI_Q4K_FORCE_SCALAR";
34 constexpr const char* LCQI_Q4K_BACKEND_SCALAR = "scalar";
35 constexpr const char* LCQI_Q4K_FORCE_TRUE = "1";
36
37 std::uint16_t read_le_u16(const std::uint8_t* bytes) noexcept {
38     return static_cast<std::uint16_t>(
39         static_cast<std::uint16_t>(bytes[0]) |
40         (static_cast<std::uint16_t>(bytes[1]) << LCQI_Q4K_BYTE_BITS));
41 }
42
43 float bits_to_float(std::uint32_t bits) noexcept {
44     float value = 0.0F;
45     std::memcpy(&value, &bits, sizeof(value));
46     return value;
47 }
48
49 float f16_to_f32(std::uint16_t bits) noexcept {
50     const std::uint32_t sign =
51         (static_cast<std::uint32_t>(bits) & LCQI_Q4K_F16_SIGN_MASK)
52         << LCQI_Q4K_F16_TO_F32_SIGN_SHIFT;
53     const std::uint32_t exponent =
54         static_cast<std::uint32_t>(bits) & LCQI_Q4K_F16_EXPONENT_MASK;
55     const std::uint32_t mantissa =
56         static_cast<std::uint32_t>(bits) & LCQI_Q4K_F16_MANTISSA_MASK;
57
58     if (exponent == 0U) {
59         if (mantissa == 0U) {
60             return bits_to_float(sign);
61         }
62         float value = static_cast<float>(mantissa) /
63             static_cast<float>(1U << LCQI_Q4K_F16_MANTISSA_BITS);
64         value = std::ldexp(value, 1 - static_cast<int>(↵
↵ LCQI_Q4K_F16_EXPONENT_BIAS));
65         return sign == 0U ? value : -value;
66     }
67
68     if (exponent == LCQI_Q4K_F16_EXPONENT_MASK) {
69         const std::uint32_t f32_mantissa =
70             mantissa << (LCQI_Q4K_F32_MANTISSA_BITS - ↵
↵ LCQI_Q4K_F16_MANTISSA_BITS);
71         return bits_to_float(sign | LCQI_Q4K_F32_EXPONENT_MASK | f32_mantissa);
72     }
73
74     const std::uint32_t f32_exponent =
75         ((exponent >> LCQI_Q4K_F16_MANTISSA_BITS) - LCQI_Q4K_F16_EXPONENT_BIAS ↵
↵ +
76         LCQI_Q4K_F32_EXPONENT_BIAS)
77         << LCQI_Q4K_F32_MANTISSA_BITS;
78     const std::uint32_t f32_mantissa =
79         mantissa << (LCQI_Q4K_F32_MANTISSA_BITS - LCQI_Q4K_F16_MANTISSA_BITS);

```

```

80     return bits_to_float(sign | f32_exponent | f32_mantissa);
81 }
82
83 void get_scale_min_k4(std::int32_t index,
84                      const std::uint8_t* packed,
85                      std::uint8_t& scale,
86                      std::uint8_t& min) noexcept {
87     if (index < LCQI_Q4K_SUBBLOCK_PAIR_COUNT) {
88         scale = packed[index] & LCQI_Q4K_SIX_BIT_MASK;
89         min = packed[index + LCQI_Q4K_SUBBLOCK_PAIR_COUNT] & ↵
↵ LCQI_Q4K_SIX_BIT_MASK;
90         return;
91     }
92     scale = static_cast<std::uint8_t>((packed[index + ↵
↵ LCQI_Q4K_SUBBLOCK_PAIR_COUNT] &
93                                         LCQI_Q4K_LOW_NIBBLE_MASK) |
94                                         ((packed[index - ↵
↵ LCQI_Q4K_SUBBLOCK_PAIR_COUNT] >> 6U)
95                                         << 4U));
96     min = static_cast<std::uint8_t>((packed[index + ↵
↵ LCQI_Q4K_SUBBLOCK_PAIR_COUNT] >> 4U) |
97                                     ((packed[index] >> 6U) << 4U));
98 }
99
100 std::int32_t horizontal_sum_i32(__m256i values) noexcept {
101     const __m128i low = _mm256_castsi256_si128(values);
102     const __m128i high = _mm256_extracti128_si256(values, 1);
103     __m128i sum = _mm_add_epi32(low, high);
104     sum = _mm_add_epi32(sum, _mm_shuffle_epi32(sum, _MM_SHUFFLE(1, 0, 3, 2)));
105     sum = _mm_add_epi32(sum, _mm_shuffle_epi32(sum, _MM_SHUFFLE(2, 3, 0, 1)));
106     return _mm_cvtsi128_si32(sum);
107 }
108
109 std::int32_t dot_32_u4_i8_avx2(__m256i q4_values, const std::int8_t* q8_values)↵
↵ noexcept {
110     const __m256i q8 = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(↵
↵ q8_values));
111     const __m256i pair_sums_i16 = _mm256_maddubs_epi16(q4_values, q8);
112     const __m256i ones = _mm256_set1_epi16(1);
113     const __m256i partial_i32 = _mm256_madd_epi16(pair_sums_i16, ones);
114     return horizontal_sum_i32(partial_i32);
115 }
116
117 float dot_block_q4_k_q8_avx2(const std::uint8_t* block, const Q8KBlock& input) ↵
↵ noexcept {
118     const float d = f16_to_f32(read_le_u16(block));
119     const float dmin = f16_to_f32(read_le_u16(block + 2));
120     const std::uint8_t* scales = block + 4;
121     const std::uint8_t* qs = block + LCQI_Q4K_QS_OFFSET;
122     const __m256i nibble_mask = _mm256_set1_epi8(static_cast<char>(↵
↵ LCQI_Q4K_LOW_NIBBLE_MASK));
123     std::int32_t weighted_sum = 0;

```

```

124     std::int32_t min_sum = 0;
125
126     for (std::int32_t pair = 0; pair < LCQI_Q4K_SUBBLOCK_PAIR_COUNT; ++pair) {
127         const __m256i packed =
128             _mm256_loadu_si256(reinterpret_cast<const __m256i*>(
129                 qs + pair * LCQI_Q4K_QS_BYTES_PER_PAIR));
130         const __m256i low_nibbles = _mm256_and_si256(packed, nibble_mask);
131         const __m256i high_nibbles =
132             _mm256_and_si256(_mm256_srli_epi16(packed, 4), nibble_mask);
133
134         const std::int32_t low_subblock = pair * 2;
135         const std::int32_t high_subblock = low_subblock + 1;
136         std::uint8_t low_scale = 0;
137         std::uint8_t low_min = 0;
138         std::uint8_t high_scale = 0;
139         std::uint8_t high_min = 0;
140         get_scale_min_k4(low_subblock, scales, low_scale, low_min);
141         get_scale_min_k4(high_subblock, scales, high_scale, high_min);
142
143         const std::int32_t low_dot =
144             dot_32_u4_i8_avx2(low_nibbles,
145                 input.qs.data() + low_subblock * ←
146                 ↪ LCQI_Q4_K_SUBBLOCK_VALUES);
147         const std::int32_t high_dot =
148             dot_32_u4_i8_avx2(high_nibbles,
149                 input.qs.data() + high_subblock * ←
150                 ↪ LCQI_Q4_K_SUBBLOCK_VALUES);
151         weighted_sum += static_cast<std::int32_t>(low_scale) * low_dot;
152         weighted_sum += static_cast<std::int32_t>(high_scale) * high_dot;
153
154         min_sum += static_cast<std::int32_t>(low_min) *
155             (static_cast<std::int32_t>(input.bsums[low_subblock * 2]) +
156             ↪ static_cast<std::int32_t>(input.bsums[low_subblock * 2 + ←
157             ↪ 1])));
158         min_sum += static_cast<std::int32_t>(high_min) *
159             (static_cast<std::int32_t>(input.bsums[high_subblock * 2]) +
160             ↪ static_cast<std::int32_t>(input.bsums[high_subblock * 2 + ←
161             ↪ 1])));
162     }
163
164     return (d * input.d) * static_cast<float>(weighted_sum) -
165         (dmin * input.d) * static_cast<float>(min_sum);
166 }
167
168 bool scalar_backend_forced() noexcept {
169     const char* backend = std::getenv(LCQI_Q4K_BACKEND_ENV);
170     if (backend != nullptr && std::strcmp(backend, LCQI_Q4K_BACKEND_SCALAR) == ←
171         ↪ 0) {
172         return true;
173     }
174     const char* force_scalar = std::getenv(LCQI_Q4K_FORCE_SCALAR_ENV);
175     return force_scalar != nullptr && std::strcmp(force_scalar, ←

```

```

    ↪ LCQI_Q4K_FORCE_TRUE) == 0;
171 }
172
173 } // namespace
174
175 bool q4_k_q8_avx2_available() noexcept {
176     if (scalar_backend_forced()) {
177         return false;
178     }
179     #if defined(__GNUC__) || defined(__clang__)
180         __builtin_cpu_init();
181         return __builtin_cpu_supports("avx2");
182     #else
183         return true;
184     #endif
185 }
186
187 const char* q4_k_q8_active_backend() noexcept {
188     return q4_k_q8_avx2_available() ? "avx2_maddubs" : "scalar";
189 }
190
191 void matvec_q4_k_q8_avx2_rows_unchecked(std::span<const std::uint8_t> q4_rows,
192                                         std::int64_t row_count,
193                                         std::int64_t column_count,
194                                         std::span<const Q8KBlock> input,
195                                         std::span<float> output,
196                                         std::int64_t row_begin,
197                                         std::int64_t row_end) noexcept;
198
199 void matvec_q4_k_q8_avx2_unchecked(std::span<const std::uint8_t> q4_rows,
200                                     std::int64_t row_count,
201                                     std::int64_t column_count,
202                                     std::span<const Q8KBlock> input,
203                                     std::span<float> output) {
204     matvec_q4_k_q8_avx2_rows_unchecked(q4_rows,
205                                         row_count,
206                                         column_count,
207                                         input,
208                                         output,
209                                         0,
210                                         row_count);
211 }
212
213 void matvec_q4_k_q8_avx2_rows_unchecked(std::span<const std::uint8_t> q4_rows,
214                                         std::int64_t row_count,
215                                         std::int64_t column_count,
216                                         std::span<const Q8KBlock> input,
217                                         std::span<float> output,
218                                         std::int64_t row_begin,
219                                         std::int64_t row_end) noexcept {
220     (void)row_count;
221     const std::int64_t row_blocks = column_count / LCQI_QK_K_BLOCK_VALUES;

```



```

222     const std::int64_t row_bytes = row_blocks * LCQI_Q4_K_BLOCK_BYTES;
223     for (std::int64_t row = row_begin; row < row_end; ++row) {
224         float sum = 0.0F;
225         const std::uint8_t* row_data = q4_rows.data() + row * row_bytes;
226         for (std::int64_t block = 0; block < row_blocks; ++block) {
227             sum += dot_block_q4_k_q8_avx2(
228                 row_data + block * LCQI_Q4_K_BLOCK_BYTES,
229                 input[static_cast<std::size_t>(block)]);
230         }
231         output[static_cast<std::size_t>(row)] = sum;
232     }
233 }
234
235 } // namespace lcqi
236
237 #endif

```

外部模型 Smoke 与对比脚本

`run_smollm2_small_smoke.py` 下载并校验 SmoLLM2-135M-Instruct 的 GGUF 量化文件，构建固定 commit 的 `llama-simple-chat`，在 CPU 上运行一次真实开源 small 模型生成，并输出可复查报告。它是外部真实模型 smoke，不替代 LCQI tiny golden trace。

Listing 16.16 `tools/run_smollm2_small_smoke.py`

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import hashlib
7  import os
8  import re
9  import shlex
10 import shutil
11 import subprocess
12 import sys
13 from dataclasses import dataclass
14 from pathlib import Path
15
16
17 MODEL_SOURCE_ID = "HuggingFaceTB/SmolLM2-135M-Instruct"
18 MODEL_GGUF_REPO_ID = "bartowski/SmolLM2-135M-Instruct-GGUF"
19 MODEL_FILE_NAME = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
20 MODEL_URL = (
21     "https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF"
22     f"/resolve/main/{MODEL_FILE_NAME}"
23 )
24 MODEL_EXPECTED_BYTES = 105454432
25 MODEL_EXPECTED_SHA256 = (
26     "2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d"

```



```

27 )
28 LLAMA_CPP_REPO_URL = "https://github.com/ggml-org/llama.cpp.git"
29 LLAMA_CPP_COMMIT = "b3fed31b99f9bd37725833674252bccb429bb183"
30 LLAMA_TARGET = "llama-simple-chat"
31 DEFAULT_PROMPT = "What is 2+3? Answer in one short sentence."
32 DEFAULT_CONTEXT_TOKENS = 512
33 DEFAULT_TIMEOUT_SECONDS = 180
34 DEFAULT_BUILD_JOBS = 2
35
36
37 @dataclass(frozen=True)
38 class CommandResult:
39     args: list[str]
40     returncode: int
41     stdout: str
42     stderr: str
43
44
45 def script_path() -> Path:
46     return Path(__file__).resolve()
47
48
49 def volume_dir() -> Path:
50     return script_path().parents[1]
51
52
53 def repo_dir() -> Path:
54     return script_path().parents[3]
55
56
57 def default_cache_dir() -> Path:
58     explicit_cache = os.environ.get("LCQI_SMOLLM2_CACHE_DIR")
59     if explicit_cache:
60         return Path(explicit_cache).expanduser()
61     xdg_cache = os.environ.get("XDG_CACHE_HOME")
62     cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "←
    ↪ .cache"
63     return cache_root / "lcqi-smolllm2-small-smoke"
64
65
66 def default_report_path() -> Path:
67     return volume_dir() / "results" / "lcqi-smolllm2-small-model-smoke.txt"
68
69
70 def command_line(args: list[str]) -> str:
71     return " ".join(shlex.quote(arg) for arg in args)
72
73
74 def require_tool(name: str) -> str:
75     path = shutil.which(name)
76     if path is None:
77         raise RuntimeError(f"required tool not found in PATH: {name}")

```

```

78     return path
79
80
81 def run_command(
82     args: list[str],
83     *,
84     cwd: Path | None = None,
85     input_text: str | None = None,
86     timeout_seconds: int | None = None,
87 ) -> CommandResult:
88     try:
89         completed = subprocess.run(
90             args,
91             cwd=str(cwd) if cwd else None,
92             input=input_text,
93             text=True,
94             capture_output=True,
95             timeout=timeout_seconds,
96             check=False,
97         )
98     except subprocess.TimeoutExpired as exc:
99         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
100         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
101         return CommandResult(args=args, returncode=124, stdout=stdout, stderr=
↵ stderr)
102
103     return CommandResult(
104         args=args,
105         returncode=completed.returncode,
106         stdout=completed.stdout,
107         stderr=completed.stderr,
108     )
109
110
111 def ensure_success(result: CommandResult, action: str) -> None:
112     if result.returncode == 0:
113         return
114     raise RuntimeError(
115         f"{action} failed with exit code {result.returncode}\n"
116         f"command: {command_line(result.args)}\n"
117         f"stdout tail:\n{tail(result.stdout)}\n"
118         f"stderr tail:\n{tail(result.stderr)}"
119     )
120
121
122 def tail(text: str, max_chars: int = 4000) -> str:
123     if len(text) <= max_chars:
124         return text
125     return text[-max_chars:]
126
127
128 def sha256_file(path: Path) -> str:

```

```

129     digest = hashlib.sha256()
130     with path.open("rb") as handle:
131         for block in iter(lambda: handle.read(1024 * 1024), b''):
132             digest.update(block)
133     return digest.hexdigest()
134
135
136 def verify_model_file(path: Path) -> tuple[bool, str, int]:
137     if not path.exists():
138         return False, "", 0
139     actual_bytes = path.stat().st_size
140     actual_sha = sha256_file(path)
141     return (
142         actual_bytes == MODEL_EXPECTED_BYTES
143         and actual_sha == MODEL_EXPECTED_SHA256,
144         actual_sha,
145         actual_bytes,
146     )
147
148
149 def download_model(model_path: Path, *, force_download: bool, no_download: bool ↵
    ↵ ) -> None:
150     model_path.parent.mkdir(parents=True, exist_ok=True)
151
152     if force_download and model_path.exists():
153         model_path.unlink()
154
155     is_valid, actual_sha, actual_bytes = verify_model_file(model_path)
156     if is_valid:
157         print(f"[lcqi-smoke] model cache hit: {model_path}")
158         return
159
160     if model_path.exists():
161         print(
162             "[lcqi-smoke] cached model hash mismatch, redownloading: "
163             f"bytes={actual_bytes}, sha256={actual_sha}"
164         )
165         model_path.unlink()
166
167     if no_download:
168         raise RuntimeError(
169             f"model file is missing or invalid and --no-download was set: {↵
    ↵ model_path}"
170         )
171
172     require_tool("curl")
173     part_path = model_path.with_suffix(model_path.suffix + ".part")
174     if part_path.exists():
175         part_path.unlink()
176
177     print(f"[lcqi-smoke] downloading small GGUF model: {MODEL_GGUF_REPO_ID}/{↵
    ↵ MODEL_FILE_NAME}")

```

```

178     result = run_command(
179         [
180             "curl",
181             "-L",
182             "--fail",
183             "--retry",
184             "3",
185             "-o",
186             str(part_path),
187             MODEL_URL,
188         ]
189     )
190     ensure_success(result, "download SmolLM2 GGUF")
191
192     downloaded_ok, downloaded_sha, downloaded_bytes = verify_model_file(↵
↵ part_path)
193     if not downloaded_ok:
194         raise RuntimeError(
195             "downloaded model did not match expected identity: "
196             f"bytes={downloaded_bytes}, sha256={downloaded_sha}"
197         )
198     part_path.replace(model_path)
199     print(f"[lcqi-smoke] model verified: sha256={MODEL_EXPECTED_SHA256}")
200
201
202 def ensure_llama_simple_chat(cache_dir: Path, *, jobs: int, skip_build: bool) ↵
↵ -> Path:
203     cached_binary = cache_dir / "llama.cpp" / "build" / "bin" / LLAMA_TARGET
204     source_dir = cache_dir / "llama.cpp"
205     build_dir = source_dir / "build"
206     if cached_binary.exists() and os.access(cached_binary, os.X_OK) and ↵
↵ is_pinned_llama_checkout(source_dir):
207         print(f"[lcqi-smoke] llama.cpp binary cache hit: {cached_binary}")
208         return cached_binary
209
210     if skip_build:
211         raise RuntimeError(f"llama.cpp binary is missing and --skip-build was ↵
↵ set: {cached_binary}")
212
213     require_tool("git")
214     require_tool("cmake")
215
216     cache_dir.mkdir(parents=True, exist_ok=True)
217
218     if not (source_dir / ".git").exists():
219         print(f"[lcqi-smoke] cloning llama.cpp into cache: {source_dir}")
220         result = run_command(
221             [
222                 "git",
223                 "clone",
224                 "--filter=blob:none",
225                 "--no-checkout",

```

```

226         LLAMA_CPP_REPO_URL,
227         str(source_dir),
228     ]
229 )
230 ensure_success(result, "clone llama.cpp")
231
232 print(f"[lcqi-smoke] checking out pinned llama.cpp commit: {↵
↵ LLAMA_CPP_COMMIT}")
233 fetch = run_command(
234     ["git", "-C", str(source_dir), "fetch", "--depth", "1", "origin", ↵
↵ LLAMA_CPP_COMMIT]
235 )
236 ensure_success(fetch, "fetch pinned llama.cpp commit")
237 checkout = run_command(["git", "-C", str(source_dir), "checkout", "--detach↵
↵ ", LLAMA_CPP_COMMIT])
238 ensure_success(checkout, "checkout pinned llama.cpp commit")
239
240 print(f"[lcqi-smoke] configuring llama.cpp build: {build_dir}")
241 configure = run_command(
242     [
243         "cmake",
244         "-S",
245         str(source_dir),
246         "-B",
247         str(build_dir),
248         "-DCMAKE_BUILD_TYPE=Release",
249         "-DLLAMA_BUILD_TESTS=OFF",
250         "-DLLAMA_BUILD_EXAMPLES=ON",
251         "-DLLAMA_BUILD_SERVER=OFF",
252         "-DGGML_NATIVE=OFF",
253     ]
254 )
255 ensure_success(configure, "configure llama.cpp")
256
257 print(f"[lcqi-smoke] building {LLAMA_TARGET} with -j{jobs}")
258 build = run_command(
259     ["cmake", "--build", str(build_dir), "--target", LLAMA_TARGET, "-j", ↵
↵ str(jobs)]
260 )
261 ensure_success(build, f"build {LLAMA_TARGET}")
262
263 if not cached_binary.exists():
264     raise RuntimeError(f"expected binary was not produced: {cached_binary}"↵
↵ )
265 return cached_binary
266
267
268 def is_pinned_llama_checkout(source_dir: Path) -> bool:
269     if not (source_dir / ".git").exists():
270         return False
271     result = run_command(["git", "-C", str(source_dir), "rev-parse", "HEAD"])
272     if result.returncode != 0:

```

```

273         return False
274     return result.stdout.strip() == LLAMA_CPP_COMMIT
275
276
277 def strip_ansi(text: str) -> str:
278     return re.sub(r"\x1b\[([0-?]*[ -/]*[@-~])", "", text)
279
280
281 def extract_response(stdout: str) -> str:
282     cleaned = strip_ansi(stdout).replace("\r", "")
283     chunks = [chunk.strip() for chunk in re.split(r"(?:^|\n)> ?", cleaned) if ←
        ↪ chunk.strip()]
284     if not chunks:
285         return ""
286     return chunks[0]
287
288
289 def run_model_smoke(
290     llama_binary: Path,
291     model_path: Path,
292     *,
293     prompt: str,
294     context_tokens: int,
295     timeout_seconds: int,
296 ) -> tuple[CommandResult, str]:
297     command = [
298         str(llama_binary),
299         "-m",
300         str(model_path),
301         "-ngl",
302         "0",
303         "-c",
304         str(context_tokens),
305     ]
306     print("[lcqi-smoke] running real small model generation")
307     result = run_command(
308         command,
309         input_text=prompt.rstrip("\n") + "\n\n",
310         timeout_seconds=timeout_seconds,
311     )
312     ensure_success(result, "run small model smoke")
313
314     response = extract_response(result.stdout)
315     if not response:
316         raise RuntimeError(
317             "small model smoke exited successfully but produced no assistant ←
        ↪ text\n"
318             f"stdout:\n{result.stdout}\n"
319             f"stderr:\n{result.stderr}"
320         )
321     return result, response
322

```

```

323
324 def current_repo_commit() -> str:
325     result = run_command(["git", "-C", str(repo_dir()), "rev-parse", "--short",
↵ "HEAD"])
326     if result.returncode != 0:
327         return "unknown"
328     return result.stdout.strip()
329
330
331 def current_uname() -> str:
332     result = run_command(["uname", "-a"])
333     if result.returncode != 0:
334         return "unknown"
335     return result.stdout.strip()
336
337
338 def write_report(
339     report_path: Path,
340     *,
341     cache_dir: Path,
342     llama_binary: Path,
343     model_path: Path,
344     prompt: str,
345     result: CommandResult,
346     response: str,
347 ) -> None:
348     report_path.parent.mkdir(parents=True, exist_ok=True)
349     clean_stdout = strip_ansi(result.stdout).strip()
350     report = "\n".join(
351         [
352             "LCQI SmolLM2 Small Model Smoke",
353             "",
354             f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
355             f"repo_commit={current_repo_commit()}",
356             f"host={current_uname()}",
357             f"python={sys.version.split()[0]}",
358             f"cache_dir={cache_dir}",
359             "",
360             f"model_source_id={MODEL_SOURCE_ID}",
361             f"model_gguf_repo_id={MODEL_GGUF_REPO_ID}",
362             f"model_file={MODEL_FILE_NAME}",
363             f"model_url={MODEL_URL}",
364             f"model_expected_bytes={MODEL_EXPECTED_BYTES}",
365             f"model_expected_sha256={MODEL_EXPECTED_SHA256}",
366             f"model_path={model_path}",
367             "",
368             f"llama_cpp_repo={LLAMA_CPP_REPO_URL}",
369             f"llama_cpp_commit={LLAMA_CPP_COMMIT}",
370             f"llama_target={LLAMA_TARGET}",
371             f"llama_binary={llama_binary}",
372             "",
373             f"prompt={prompt}",

```

```

374         f"command={command_line(result.args)}",
375         f"exit_code={result.returncode}",
376         "",
377         "validation=PASS: model hash matched, llama-simple-chat exited 0, "
378         "assistant response was non-empty",
379         "",
380         "assistant_response:",
381         response,
382         "",
383         "stdout_clean_tail:",
384         tail(clean_stdout, 2000),
385         "",
386         "stderr_tail:",
387         tail(result.stderr.strip(), 2000),
388         "",
389     ]
390 )
391 report_path.write_text(report, encoding="utf-8")
392
393
394 def parse_args() -> argparse.Namespace:
395     parser = argparse.ArgumentParser(
396         description=(
397             "Run a real local small open-source model smoke for CPU Volume 3. "
398             "The script caches llama.cpp and the GGUF model outside the Git ↵
399             ↵ repository."
400         )
401     )
402     parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
403     parser.add_argument("--model-path", type=Path, default=None)
404     parser.add_argument("--llama-bin", type=Path, default=None)
405     parser.add_argument("--report", type=Path, default=default_report_path())
406     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
407     parser.add_argument("--ctx", type=int, default=DEFAULT_CONTEXT_TOKENS)
408     parser.add_argument("--timeout", type=int, default=DEFAULT_TIMEOUT_SECONDS)
409     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
410     parser.add_argument("--force-download", action="store_true")
411     parser.add_argument("--no-download", action="store_true")
412     parser.add_argument("--skip-build", action="store_true")
413     return parser.parse_args()
414
415 def main() -> int:
416     args = parse_args()
417     cache_dir = args.cache_dir.expanduser().resolve()
418     model_path = (
419         args.model_path.expanduser().resolve()
420         if args.model_path
421         else cache_dir / "models" / MODEL_FILE_NAME
422     )
423     report_path = args.report.expanduser().resolve()
424

```



```

425     try:
426         download_model(
427             model_path,
428             force_download=args.force_download,
429             no_download=args.no_download,
430         )
431         if args.llama_bin:
432             llama_binary = args.llama_bin.expanduser().resolve()
433             if not llama_binary.exists() or not os.access(llama_binary, os.X_OK):
434                 raise RuntimeError(f"--llama-bin is not executable: {llama_binary}")
435             else:
436                 llama_binary = ensure_llama_simple_chat(
437                     cache_dir,
438                     jobs=args.jobs,
439                     skip_build=args.skip_build,
440                 )
441
442             result, response = run_model_smoke(
443                 llama_binary,
444                 model_path,
445                 prompt=args.prompt,
446                 context_tokens=args.ctx,
447                 timeout_seconds=args.timeout,
448             )
449             write_report(
450                 report_path,
451                 cache_dir=cache_dir,
452                 llama_binary=llama_binary,
453                 model_path=model_path,
454                 prompt=args.prompt,
455                 result=result,
456                 response=response,
457             )
458         except RuntimeError as exc:
459             print(f"[lcqi-smoke] ERROR: {exc}", file=sys.stderr)
460             return 1
461
462         print(f"report={report_path}")
463         print(f"model_sha256={MODEL_EXPECTED_SHA256}")
464         print(f"assistant_response={response}")
465         return 0
466
467     if __name__ == "__main__":
468         raise SystemExit(main())
469 
```

`run_smollm2_q4_k_benchmark.py` 复用同一个 SmolLM2 GGUF 缓存，构建 LCQI 的 `lcqi_gguf_q4_bench`，并可附带 `llama.cpp llama-bench` 端到端参考。它会先用 `LCQI_Q4K_BACKEND=scalar` 跑标量路径，再跑 `auto` 后端，因此报告里能直接看到 `avx2_maddubs` 相对 `scalar` 的提升。报告要同时说明模型 SHA-256、tensor 名、

shape、量化字节、f32 解码字节、误差、checksum 和成熟引擎参考，避免把不同层级数字混在一起。

Listing 16.17 tools/run_smollm2_q4_k_benchmark.py

```
1 #!/usr/bin/env python3
2 from __future__ import annotations
3
4 import argparse
5 import datetime as dt
6 import os
7 import re
8 import shlex
9 import subprocess
10 import sys
11 from dataclasses import dataclass
12 from pathlib import Path
13
14 import run_smollm2_small_smoke
15
16
17 DEFAULT_BUILD_JOBS = 2
18 DEFAULT_REPEAT = 20
19 DEFAULT_TIMEOUT_SECONDS = 300
20 BENCH_TARGET = "lcqi_gguf_q4_bench"
21
22
23 @dataclass(frozen=True)
24 class CommandResult:
25     name: str
26     args: list[str]
27     returncode: int
28     stdout: str
29     stderr: str
30     metrics: dict[str, str]
31
32
33 def volume_dir() -> Path:
34     return Path(__file__).resolve().parents[1]
35
36
37 def repo_dir() -> Path:
38     return Path(__file__).resolve().parents[3]
39
40
41 def default_build_dir() -> Path:
42     return volume_dir() / "build" / "lcqi-release"
43
44
45 def default_report_path() -> Path:
46     return volume_dir() / "results" / "lcqi-smollm2-q4-k-benchmark.txt"
47
48
```

```

49 def command_line(args: list[str]) -> str:
50     return " ".join(shlex.quote(arg) for arg in args)
51
52
53 def run_command(
54     name: str,
55     args: list[str],
56     *,
57     timeout_seconds: int,
58     cwd: Path | None = None,
59     env: dict[str, str] | None = None,
60 ) -> CommandResult:
61     command_env = os.environ.copy()
62     if env:
63         command_env.update(env)
64     try:
65         completed = subprocess.run(
66             args,
67             cwd=cwd if cwd else str(repo_dir()),
68             env=command_env,
69             text=True,
70             capture_output=True,
71             timeout=timeout_seconds,
72             check=False,
73         )
74     except subprocess.TimeoutExpired as exc:
75         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
76         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
77         return CommandResult(name, args, 124, stdout, stderr, {})
78     return CommandResult(
79         name=name,
80         args=args,
81         returncode=completed.returncode,
82         stdout=completed.stdout,
83         stderr=completed.stderr,
84         metrics=parse_key_value_metrics(completed.stdout),
85     )
86
87
88 def ensure_success(result: CommandResult, action: str) -> None:
89     if result.returncode == 0:
90         return
91     raise RuntimeError(
92         f"{action} failed with exit code {result.returncode}\n"
93         f"command: {command_line(result.args)}\n"
94         f"stdout tail:\n{tail(result.stdout)}\n"
95         f"stderr tail:\n{tail(result.stderr)}"
96     )
97
98
99 def tail(text: str, max_chars: int = 4000) -> str:
100     return text if len(text) <= max_chars else text[-max_chars:]

```

```

101
102
103 def parse_key_value_metrics(stdout: str) -> dict[str, str]:
104     metrics: dict[str, str] = {}
105     for line in stdout.splitlines():
106         key, sep, value = line.partition("=")
107         if sep:
108             metrics[key.strip()] = value.strip()
109     return metrics
110
111
112 def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
113     metrics: dict[str, str] = {}
114     for line in stdout.splitlines():
115         stripped = line.strip()
116         if not stripped.startswith("|"):
117             continue
118         columns = [column.strip() for column in stripped.strip("|").split("|")]
119         if len(columns) != 7 or columns[0] == "model" or columns[0].startswith(←
↪ "-"):
120             continue
121         metrics["llama_model"] = columns[0]
122         metrics["llama_size"] = columns[1]
123         metrics["llama_params"] = columns[2]
124         metrics["llama_backend"] = columns[3]
125         metrics["llama_threads"] = columns[4]
126         tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
127         if columns[5].startswith("pp"):
128             metrics["llama_prefill_test"] = columns[5]
129             metrics["llama_prefill_tps"] = tokens_per_second
130         if columns[5].startswith("tg"):
131             metrics["llama_decode_test"] = columns[5]
132             metrics["llama_decode_tps"] = tokens_per_second
133     return metrics
134
135
136 def build_lcqi(build_dir: Path, jobs: int, timeout_seconds: int) -> Path:
137     configure = run_command(
138         "cmake_configure",
139         [
140             "cmake",
141             "-S",
142             str(volume_dir()),
143             "-B",
144             str(build_dir),
145             "-DCMAKE_BUILD_TYPE=Release",
146         ],
147         timeout_seconds=timeout_seconds,
148     )
149     ensure_success(configure, "configure LCQI")
150     build = run_command(
151         "cmake_build_lcqi_gguf_q4_bench",

```

```

152     [
153         "cmake",
154         "--build",
155         str(build_dir),
156         "--target",
157         BENCH_TARGET,
158         "-j",
159         str(jobs),
160     ],
161     timeout_seconds=timeout_seconds,
162 )
163 ensure_success(build, "build LCQI Q4_K benchmark")
164 binary = build_dir / "labs" / "linux_cpu_inference" / BENCH_TARGET
165 if not binary.exists() or not os.access(binary, os.X_OK):
166     raise RuntimeError(f"LCQI Q4_K benchmark binary not found: {binary}")
167 return binary
168
169
170 def llama_bench_binary(cache_dir: Path) -> Path:
171     return cache_dir / "llama.cpp" / "build" / "bin" / "llama-bench"
172
173
174 def model_path(cache_dir: Path) -> Path:
175     return cache_dir / "models" / run_smollm2_small_smoke.MODEL_FILE_NAME
176
177
178 def run_lcqi_benchmark(
179     binary: Path,
180     model: Path,
181     *,
182     name: str,
183     tensor: str,
184     repeat: int,
185     timeout_seconds: int,
186     env: dict[str, str] | None = None,
187 ) -> CommandResult:
188     return run_command(
189         name,
190         [str(binary), str(model), tensor, str(repeat)],
191         timeout_seconds=timeout_seconds,
192         env=env,
193     )
194
195
196 def run_llama_bench(cache_dir: Path, timeout_seconds: int) -> CommandResult | ←
    ↪ None:
197     binary = llama_bench_binary(cache_dir)
198     model = model_path(cache_dir)
199     if not binary.exists() or not os.access(binary, os.X_OK) or not model.↪
    ↪ exists():
200         return None
201     result = run_command(

```

```

202     "llama_cpp_smolllm2_q4_k_m",
203     [
204         str(binary),
205         "-m",
206         str(model),
207         "-ngl",
208         "0",
209         "-p",
210         "5",
211         "-n",
212         "4",
213         "-r",
214         "1",
215     ],
216     timeout_seconds=timeout_seconds,
217 )
218 return CommandResult(
219     result.name,
220     result.args,
221     result.returncode,
222     result.stdout,
223     result.stderr,
224     parse_llama_bench_metrics(result.stdout),
225 )
226
227
228 def current_commit() -> str:
229     result = subprocess.run(
230         ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
231         text=True,
232         capture_output=True,
233         check=False,
234     )
235     return result.stdout.strip() if result.returncode == 0 else "unknown"
236
237
238 def working_tree_dirty() -> str:
239     result = subprocess.run(
240         ["git", "-C", str(repo_dir()), "status", "--porcelain"],
241         text=True,
242         capture_output=True,
243         check=False,
244     )
245     if result.returncode != 0:
246         return "unknown"
247     return "true" if result.stdout.strip() else "false"
248
249
250 def write_report(path: Path, runs: list[CommandResult], *, model: Path) -> None↵
251     ↵ :
252     path.parent.mkdir(parents=True, exist_ok=True)
253     model_matches, actual_sha256, actual_bytes = run_smolllm2_small_smoke.↵

```

```

↪ verify_model_file(model)
253 lines = [
254     "LCQI SmolLM2 Q4_K Benchmark",
255     "",
256     f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
257     f"repo_commit={current_commit()}",
258     f"working_tree_dirty={working_tree_dirty()}",
259     f"model={model}",
260     f"model_source={run_smolllm2_small_smoke.MODEL_GGUF_REPO_ID}",
261     f"model_file={run_smolllm2_small_smoke.MODEL_FILE_NAME}",
262     f"model_expected_bytes={run_smolllm2_small_smoke.MODEL_EXPECTED_BYTES}",
263     f"model_actual_bytes={actual_bytes}",
264     f"model_expected_sha256={run_smolllm2_small_smoke.MODEL_EXPECTED_SHA256}↪
↪ ",
265     f"model_actual_sha256={actual_sha256}",
266     f"model_identity_match={str(model_matches).lower()}",
267     f"llama_cpp_commit={run_smolllm2_small_smoke.LLAMA_CPP_COMMIT}",
268     "",
269     "scope=LCQI reads the same SmolLM2 Q4_K_M GGUF used by llama.cpp and ↪
↪ runs a "
270     "single real Q4_K tensor matvec slice. llama.cpp numbers are end-to-end↪
↪ mature "
271     "engine context, not same-function microkernel parity.",
272     "",
273 ]
274 for run in runs:
275     lines.extend(
276         [
277             f"[{run.name}]",
278             f"returncode={run.returncode}",
279             f"command={command_line(run.args)}",
280         ]
281     )
282     for key in sorted(run.metrics):
283         lines.append(f"{key}={run.metrics[key]}")
284     lines.extend(
285         [
286             "stdout_tail:",
287             tail(run.stdout.strip(), 2400),
288             "stderr_tail:",
289             tail(run.stderr.strip(), 2400),
290             "",
291         ]
292     )
293     scalar = next((run for run in runs if run.name == "lcqi_smolllm2_q4_k_scalar↪
↪ "), None)
294     auto = next((run for run in runs if run.name == "lcqi_smolllm2_q4_k_auto"), ↪
↪ None)
295     if scalar is not None and auto is not None:
296         scalar_matvec = metric_as_float(scalar.metrics, "↪
↪ q4_q8_matvec_average_us")
297         auto_matvec = metric_as_float(auto.metrics, "q4_q8_matvec_average_us")

```

```

298     scalar_total = metric_as_float(scalar.metrics, "q4_q8_total_average_us" ←
    ↪ )
299     auto_total = metric_as_float(auto.metrics, "q4_q8_total_average_us")
300     if scalar_matvec is not None and auto_matvec is not None and ←
    ↪ auto_matvec > 0.0:
301         lines.append("[lcqi_auto_vs_scalar]")
302         lines.append(f"q4_q8_matvec_speedup_auto_vs_scalar={scalar_matvec / ←
    ↪ auto_matvec:.3f}")
303         if scalar_total is not None and auto_total is not None and ←
    ↪ auto_total > 0.0:
304             lines.append(f"q4_q8_total_speedup_auto_vs_scalar={scalar_total ←
    ↪ / auto_total:.3f}")
305             lines.append("")
306     while lines and lines[-1] == "":
307         lines.pop()
308     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
309
310
311 def metric_as_float(metrics: dict[str, str], key: str) -> float | None:
312     value = metrics.get(key)
313     if value is None:
314         return None
315     try:
316         return float(value)
317     except ValueError:
318         return None
319
320
321 def parse_args() -> argparse.Namespace:
322     parser = argparse.ArgumentParser(
323         description="Benchmark LCQI Q4_K matvec on the same SmolLM2 GGUF used ←
    ↪ by llama.cpp."
324     )
325     parser.add_argument("--cache-dir", type=Path, default=←
    ↪ run_smolllm2_small_smoke.default_cache_dir())
326     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
327     parser.add_argument("--report", type=Path, default=default_report_path())
328     parser.add_argument("--tensor", default="auto")
329     parser.add_argument("--repeat", type=int, default=DEFAULT_REPEAT)
330     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
331     parser.add_argument("--timeout-seconds", type=int, default=←
    ↪ DEFAULT_TIMEOUT_SECONDS)
332     parser.add_argument("--no-download", action="store_true")
333     parser.add_argument("--skip-llama", action="store_true")
334     return parser.parse_args()
335
336
337 def main() -> int:
338     args = parse_args()
339     if args.repeat <= 0:
340         print("[lcqi-smolllm2-q4] --repeat must be positive", file=sys.stderr)
341         return 1

```



```

342     try:
343         cache_dir = args.cache_dir.expanduser().resolve()
344         model = model_path(cache_dir)
345         run_smollm2_small_smoke.download_model(
346             model,
347             force_download=False,
348             no_download=args.no_download,
349         )
350         binary = build_lcqi(args.build_dir, args.jobs, args.timeout_seconds)
351         runs = [
352             run_lcqi_benchmark(
353                 binary,
354                 model,
355                 name="lcqi_smollm2_q4_k_scalar",
356                 tensor=args.tensor,
357                 repeat=args.repeat,
358                 timeout_seconds=args.timeout_seconds,
359                 env={"LCQI_Q4K_BACKEND": "scalar"},
360             ),
361             run_lcqi_benchmark(
362                 binary,
363                 model,
364                 name="lcqi_smollm2_q4_k_auto",
365                 tensor=args.tensor,
366                 repeat=args.repeat,
367                 timeout_seconds=args.timeout_seconds,
368             )
369         ]
370         if not args.skip_llama:
371             llama = run_llama_bench(cache_dir, args.timeout_seconds)
372             if llama is not None:
373                 runs.append(llama)
374         write_report(args.report, runs, model=model)
375     except Exception as error:
376         print(f"[lcqi-smollm2-q4] error: {error}", file=sys.stderr)
377         return 1
378
379     for run in runs:
380         print(f"{run.name}: returncode={run.returncode}")
381         for key in sorted(run.metrics):
382             print(f"    {key}={run.metrics[key]}")
383         print(f"report={args.report}")
384         return 0 if all(run.returncode == 0 for run in runs) else 1
385
386
387 if __name__ == "__main__":
388     raise SystemExit(main())

```

`run_smollm2_same_input_compare.py` 是 SmollM2 同输入对齐脚本。它构建 LCQI 和固定 commit 的 llama.cpp，分别用 `LCQI_LLAMA_Q5_0_PACKED=0`、`LCQI_LLAMA_Q8_0_DIRECT=0`、`LCQI_LLAMA_Q8_0_TIED_LM_HEAD=0`、`LCQI_LLAMA_BATCH_PREFILL=0` 跑关闭子优化的 A/B，再用 `LCQI_LLAMA_Q6_K_DIRECT=1` 跑

Q6_K direct 实验，再用 `--threads1` 跑串行默认路径，再跑自动 worker 默认路径；可选再用 `LCQI_LLAMA_GGML_DIRECT=1` 跑完整 Q5_0/Q6_K/Q8_0 实验 direct 路径，最后跑 `llama-simple`、`llama-tokenize` 和 `llama-bench`。报告必须同时看 `same_input_prompt_token_ids_match`、Q5 packed A/B、Q5 batch F32 sidecar 字节和调用次数、Q6 direct A/B、Q8 direct A/B、tied lm head max-dot A/B、batch prefill A/B、LCQI 线程 A/B、GGML direct A/B、`llama.cpp` 对比和 hotspot 字段。最新 caw 报告 [books/cpu-volume-3/reports/lcqi-smolllm2-q5-sidecar-maxdot-compare-caw.txt](#) 与 [books/cpu-volume-3/reports/lcqi-smolllm2-q5-sidecar-maxdot-decode16-compare-caw.txt](#) 显示：同输入 token id 完全一致；`max-new=16` 时 LCQI prefill 为 `35.351000ms`，`llama.cpp` prompt eval 为 `21.510000ms`，差距 `1.643468x`；LCQI decode step 为 `3.217533ms`，`llama.cpp` 为 `2.710000ms`，差距 `1.187282x`。同一报告还给出 Q5 packed decode 提升 `1.795226x`，但 Q5 packed 对 prefill 是 `0.908339x` 的回退，所以默认采用 decode packed、prompt batch F32 sidecar；tied lm head max-dot 让 decode step 提升 `1.238844x`；Q6 direct 实验让 prefill 和 decode 都变慢，因此继续保持关闭。这正是源码卷要训练的工程判断：覆盖率、内存、局部 kernel、线程调度、prefill 结构和 decode step 速度必须分开证明。

Listing 16.18 `tools/run_smolllm2_same_input_compare.py`

```

1 #!/usr/bin/env python3
2 from __future__ import annotations
3
4 import argparse
5 import datetime as dt
6 import os
7 import re
8 import shlex
9 import statistics
10 import subprocess
11 import sys
12 from dataclasses import dataclass
13 from pathlib import Path
14
15 import run_smolllm2_small_smoke
16
17
18 DEFAULT_PROMPT = "Hello"
19 DEFAULT_MAX_NEW_TOKENS = 2
20 DEFAULT_ROUNDS = 5
21 DEFAULT_BUILD_JOBS = 2
22 DEFAULT_TIMEOUT_SECONDS = 300
23 LCQI_TARGET = "lcqi_llama_gguf"
24 LLAMA_SIMPLE_TARGET = "llama-simple"
25 LLAMA_BENCH_TARGET = "llama-bench"
26 LLAMA_TOKENIZE_TARGET = "llama-tokenize"
27 LCQI_Q4_DIRECT_ENV = "LCQI_LLAMA_Q4_DIRECT"
28 LCQI_Q5_0_PACKED_ENV = "LCQI_LLAMA_Q5_0_PACKED"
29 LCQI_Q6_K_DIRECT_ENV = "LCQI_LLAMA_Q6_K_DIRECT"
30 LCQI_Q8_0_DIRECT_ENV = "LCQI_LLAMA_Q8_0_DIRECT"
31 LCQI_Q8_0_TIED_LM_HEAD_ENV = "LCQI_LLAMA_Q8_0_TIED_LM_HEAD"
32 LCQI_GGML_DIRECT_ENV = "LCQI_LLAMA_GGML_DIRECT"

```

```

33 LCQI_GGML_SELECTIVE_DIRECT_ENV = "LCQI_LLAMA_GGML_SELECTIVE_DIRECT"
34 LCQI_BATCH_PREFILL_ENV = "LCQI_LLAMA_BATCH_PREFILL"
35 LCQI_Q4_DIRECT_OFF = "0"
36 LCQI_Q5_0_PACKED_OFF = "0"
37 LCQI_Q6_K_DIRECT_ON = "1"
38 LCQI_Q8_0_DIRECT_OFF = "0"
39 LCQI_Q8_0_TIED_LM_HEAD_OFF = "0"
40 LCQI_GGML_DIRECT_ON = "1"
41 LCQI_GGML_SELECTIVE_DIRECT_ON = "1"
42 LCQI_BATCH_PREFILL_OFF = "0"
43 LCQI_SUMMARY_KEYS = [
44     "benchmark_load_ms",
45     "benchmark_prefill_ms",
46     "derived_prefill_ms_per_prompt_token",
47     "derived_prefill_prompt_tokens_per_second",
48     "benchmark_decode_ms",
49     "derived_decode_ms_per_step",
50     "benchmark_decode_tokens_per_second",
51     "benchmark_worker_count",
52     "derived_f32_weight_inflation_ratio",
53     "derived_direct_quantized_weight_byte_share",
54     "benchmark_q5_0_packed_weight_bytes",
55     "benchmark_q5_0_batch_f32_weight_bytes",
56     "benchmark_q6_k_direct_weight_bytes",
57     "benchmark_q8_0_direct_weight_bytes",
58     "benchmark_hotspot_rms_norm_ms",
59     "benchmark_hotspot_attention_ms",
60     "benchmark_hotspot_rope_ms",
61     "benchmark_hotspot_wq_ms",
62     "benchmark_hotspot_wk_ms",
63     "benchmark_hotspot_wv_ms",
64     "benchmark_hotspot_wo_ms",
65     "benchmark_hotspot_w_gate_ms",
66     "benchmark_hotspot_w_up_ms",
67     "benchmark_hotspot_w_down_ms",
68     "benchmark_hotspot_lm_head_ms",
69     "benchmark_hotspot_q4_k_direct_ms",
70     "benchmark_hotspot_q5_0_packed_ms",
71     "benchmark_hotspot_q5_0_batch_f32_ms",
72     "benchmark_hotspot_q6_k_direct_ms",
73     "benchmark_hotspot_q8_0_direct_ms",
74     "benchmark_hotspot_ggml_direct_ms",
75     "benchmark_hotspot_f32_fallback_ms",
76     "benchmark_hotspot_q4_k_direct_calls",
77     "benchmark_hotspot_q5_0_packed_calls",
78     "benchmark_hotspot_q5_0_batch_f32_calls",
79     "benchmark_hotspot_q6_k_direct_calls",
80     "benchmark_hotspot_q8_0_direct_calls",
81     "benchmark_hotspot_ggml_direct_calls",
82     "benchmark_hotspot_f32_fallback_calls",
83     "benchmark_q4_k_direct_tensors",
84     "benchmark_q5_0_packed_tensors",

```

```
85     "benchmark_q6_k_direct_tensors",
86     "benchmark_q8_0_direct_tensors",
87     "benchmark_ggml_direct_tensors",
88     "benchmark_f32_fallback_tensors",
89 ]
90
91
92 @dataclass(frozen=True)
93 class CommandResult:
94     name: str
95     args: list[str]
96     returncode: int
97     stdout: str
98     stderr: str
99     metrics: dict[str, str]
100
101
102 def volume_dir() -> Path:
103     return Path(__file__).resolve().parents[1]
104
105
106 def repo_dir() -> Path:
107     return Path(__file__).resolve().parents[3]
108
109
110 def default_build_dir() -> Path:
111     return volume_dir() / "build" / "lcqi-release"
112
113
114 def default_report_path() -> Path:
115     return volume_dir() / "results" / "lcqi-smolllm2-same-input-compare-caw.txt"
116
117
118 def command_line(args: list[str]) -> str:
119     return " ".join(shlex.quote(arg) for arg in args)
120
121
122 def tail(text: str, max_chars: int = 4000) -> str:
123     return text if len(text) <= max_chars else text[-max_chars:]
124
125
126 def run_command(
127     name: str,
128     args: list[str],
129     *,
130     timeout_seconds: int,
131     cwd: Path | None = None,
132     env: dict[str, str] | None = None,
133 ) -> CommandResult:
134     command_env = os.environ.copy()
135     if env:
136         command_env.update(env)
```

```

137     try:
138         completed = subprocess.run(
139             args,
140             cwd=str(cwd) if cwd else str(repo_dir()),
141             env=command_env,
142             text=True,
143             capture_output=True,
144             timeout=timeout_seconds,
145             check=False,
146         )
147     except subprocess.TimeoutExpired as exc:
148         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
149         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
150         return CommandResult(name, args, 124, stdout, stderr, {})
151     return CommandResult(
152         name=name,
153         args=args,
154         returncode=completed.returncode,
155         stdout=completed.stdout,
156         stderr=completed.stderr,
157         metrics={},
158     )
159
160
161 def ensure_success(result: CommandResult, action: str) -> None:
162     if result.returncode == 0:
163         return
164     raise RuntimeError(
165         f"{action} failed with exit code {result.returncode}\n"
166         f"command: {command_line(result.args)}\n"
167         f"stdout tail:\n{tail(result.stdout)}\n"
168         f"stderr tail:\n{tail(result.stderr)}"
169     )
170
171
172 def smollm2_chat_prompt(user_prompt: str) -> str:
173     return (
174         "<|im_start|>system\n"
175         "You are a helpful AI assistant named SmollM, trained by Hugging Face"
176         "<|im_end|>\n"
177         "<|im_start|>user\n"
178         f"{user_prompt}"
179         "<|im_end|>\n"
180         "<|im_start|>assistant\n"
181     )
182
183
184 def escape_newlines(text: str) -> str:
185     return text.replace("\\", "\\\\").replace("\n", "\\n")
186
187
188 def parse_int_list(value: str) -> list[int]:

```

```

189     ids: list[int] = []
190     for item in value.replace(",", " ").split():
191         ids.append(int(item))
192     return ids
193
194
195 def parse_lcqi_metrics(stdout: str) -> dict[str, str]:
196     metrics: dict[str, str] = {}
197     generated_ids: list[int] = []
198     prompt_ids: list[int] = []
199     for line in stdout.splitlines():
200         key, sep, value = line.partition(" ")
201         if not sep:
202             continue
203         if key in {
204             "mode",
205             "weight_execution",
206             "architecture",
207             "model_name",
208             "layers",
209             "hidden_size",
210             "query_heads",
211             "kv_heads",
212             "head_dim",
213             "vocab_size",
214             "tie_lm_head_to_embedding",
215             "predicted_first_token",
216             "benchmark_manifest_ms",
217             "benchmark_weight_load_ms",
218             "benchmark_load_ms",
219             "benchmark_prefill_ms",
220             "benchmark_decode_ms",
221             "benchmark_total_ms",
222             "benchmark_prompt_tokens",
223             "benchmark_generated_tokens",
224             "benchmark_prefill_steps",
225             "benchmark_decode_steps",
226             "benchmark_worker_count",
227             "benchmark_first_token_tokens_per_second",
228             "benchmark_decode_tokens_per_second",
229             "benchmark_kv_cache_bytes",
230             "benchmark_quantized_weight_bytes",
231             "benchmark_f32_weight_bytes",
232             "benchmark_direct_quantized_weight_bytes",
233             "benchmark_fallback_dequantized_weight_bytes",
234             "benchmark_q5_0_packed_weight_bytes",
235             "benchmark_q5_0_batch_f32_weight_bytes",
236             "benchmark_q6_k_direct_weight_bytes",
237             "benchmark_q8_0_direct_weight_bytes",
238             "benchmark_tensors_loaded",
239             "benchmark_q4_k_direct_tensors",
240             "benchmark_q5_0_packed_tensors",

```

```

241         "benchmark_q6_k_direct_tensors",
242         "benchmark_q8_0_direct_tensors",
243         "benchmark_ggml_direct_tensors",
244         "benchmark_f32_fallback_tensors",
245         "benchmark_hotspot_rms_norm_ms",
246         "benchmark_hotspot_attention_ms",
247         "benchmark_hotspot_rope_ms",
248         "benchmark_hotspot_wq_ms",
249         "benchmark_hotspot_wk_ms",
250         "benchmark_hotspot_wv_ms",
251         "benchmark_hotspot_wo_ms",
252         "benchmark_hotspot_w_gate_ms",
253         "benchmark_hotspot_w_up_ms",
254         "benchmark_hotspot_w_down_ms",
255         "benchmark_hotspot_lm_head_ms",
256         "benchmark_hotspot_q4_k_direct_ms",
257         "benchmark_hotspot_q5_0_packed_ms",
258         "benchmark_hotspot_q5_0_batch_f32_ms",
259         "benchmark_hotspot_q6_k_direct_ms",
260         "benchmark_hotspot_q8_0_direct_ms",
261         "benchmark_hotspot_ggml_direct_ms",
262         "benchmark_hotspot_f32_fallback_ms",
263         "benchmark_hotspot_q4_k_direct_calls",
264         "benchmark_hotspot_q5_0_packed_calls",
265         "benchmark_hotspot_q5_0_batch_f32_calls",
266         "benchmark_hotspot_q6_k_direct_calls",
267         "benchmark_hotspot_q8_0_direct_calls",
268         "benchmark_hotspot_ggml_direct_calls",
269         "benchmark_hotspot_f32_fallback_calls",
270     }:
271         metrics[key] = value.strip()
272     elif key == "prompt_ids":
273         prompt_ids = parse_int_list(value)
274         metrics["prompt_ids"] = " ".join(str(item) for item in prompt_ids)
275         metrics["prompt_token_count_from_ids"] = str(len(prompt_ids))
276     elif key == "generated_ids":
277         generated_ids = parse_int_list(value)
278         metrics["generated_ids"] = " ".join(str(item) for item in ↵
↳ generated_ids)
279         metrics["generated_token_count_from_ids"] = str(len(generated_ids))
280     if prompt_ids and generated_ids and len(generated_ids) >= len(prompt_ids):
281         new_ids = generated_ids[len(prompt_ids):]
282         metrics["generated_new_ids"] = " ".join(str(item) for item in new_ids)
283     add_lcqi_derived_metrics(metrics)
284     return metrics
285
286
287 def add_lcqi_derived_metrics(metrics: dict[str, str]) -> None:
288     prefill_ms = metric_as_float(metrics, "benchmark_prefill_ms")
289     prompt_tokens = metric_as_float(metrics, "benchmark_prompt_tokens")
290     decode_ms = metric_as_float(metrics, "benchmark_decode_ms")
291     decode_steps = metric_as_float(metrics, "benchmark_decode_steps")

```

```

292     quantized_bytes = metric_as_float(metrics, "↵
↵ benchmark_quantized_weight_bytes")
293     f32_bytes = metric_as_float(metrics, "benchmark_f32_weight_bytes")
294     direct_bytes = metric_as_float(metrics, "↵
↵ benchmark_direct_quantized_weight_bytes")
295     if prefill_ms is not None and prefill_ms > 0.0 and prompt_tokens is not ↵
↵ None:
296         metrics["derived_prefill_prompt_tokens_per_second"] = (
297             f"{prompt_tokens * 1000.0 / prefill_ms:.6f}"
298         )
299         metrics["derived_prefill_ms_per_prompt_token"] = (
300             f"{prefill_ms / prompt_tokens:.6f}"
301         )
302     if decode_ms is not None and decode_ms > 0.0 and decode_steps is not None:
303         metrics["derived_decode_steps_per_second"] = f"{decode_steps * 1000.0 /↵
↵ decode_ms:.6f}"
304         metrics["derived_decode_ms_per_step"] = f"{decode_ms / max(decode_steps↵
↵ , 1.0):.6f}"
305     if quantized_bytes is not None and quantized_bytes > 0.0 and f32_bytes is ↵
↵ not None:
306         metrics["derived_f32_weight_inflation_ratio"] = f"{f32_bytes / ↵
↵ quantized_bytes:.6f}"
307     if quantized_bytes is not None and quantized_bytes > 0.0 and direct_bytes ↵
↵ is not None:
308         metrics["derived_direct_quantized_weight_byte_share"] = (
309             f"{direct_bytes / quantized_bytes:.6f}"
310         )
311
312
313 def parse_llama_simple_metrics(stderr: str, stdout: str, full_prompt: str) -> ↵
↵ dict[str, str]:
314     metrics: dict[str, str] = {}
315     parse_llama_perf_metrics(stderr, metrics)
316     clean_stdout = strip_ansi(stdout).replace("\r", "")
317     metrics["stdout_prompt_prefix_match"] = str(clean_stdout.startswith(↵
↵ full_prompt)).lower()
318     if clean_stdout.startswith(full_prompt):
319         generated = clean_stdout[len(full_prompt):].strip()
320         metrics["generated_suffix"] = escape_newlines(generated)
321     decoded_match = re.search(
322         r"main:\s+decoded\s+(?P<count>\d+)\s+tokens\s+in\s+(?P<seconds>[0-9.]+)↵
↵ \s+s,"
323         r"\s+speed:\s+(?P<tps>[0-9.]+)\s+t/s",
324         stderr,
325     )
326     if decoded_match:
327         metrics["llama_simple_decoded_tokens"] = decoded_match.group("count")
328         metrics["llama_simple_main_seconds"] = decoded_match.group("seconds")
329         metrics["llama_simple_main_tokens_per_second"] = decoded_match.group("↵
↵ tps")
330     return metrics
331

```



```

332
333 def parse_llama_perf_metrics(text: str, metrics: dict[str, str]) -> None:
334     load_match = re.search(
335         r"llama_(?:perf_context_print|print_timings):\s+load time\s*=\s*"
336         r"(?P<ms>[0-9.]+)\s*ms",
337         text,
338     )
339     if load_match:
340         metrics["llama_load_ms"] = load_match.group("ms")
341
342     prompt_match = re.search(
343         r"llama_(?:perf_context_print|print_timings):\s+prompt eval time\s*=\s*"
344         ↪ "
345         r"(?P<ms>[0-9.]+)\s*ms\s*/\s*(?P<count>\d+)\s+tokens\s*"
346         r"(\s*(?P<ms_per>[0-9.]+)\s*ms per token,\s*"
347         r"(?P<tps>[0-9.]+)\s*tokens per second\s*\)",
348         text,
349     )
350     if prompt_match:
351         metrics["llama_prompt_eval_ms"] = prompt_match.group("ms")
352         metrics["llama_prompt_eval_tokens"] = prompt_match.group("count")
353         metrics["llama_prompt_eval_ms_per_token"] = prompt_match.group("ms_per"
354         ↪ )
355         metrics["llama_prompt_eval_tokens_per_second"] = prompt_match.group("
356         ↪ tps")
357
358     eval_match = re.search(
359         r"llama_(?:perf_context_print|print_timings):\s+eval time\s*=\s*"
360         r"(?P<ms>[0-9.]+)\s*ms\s*/\s*(?P<count>\d+)\s+runs\s*"
361         r"(\s*(?P<ms_per>[0-9.]+)\s*ms per token,\s*"
362         r"(?P<tps>[0-9.]+)\s*tokens per second\s*\)",
363         text,
364     )
365     if eval_match:
366         metrics["llama_eval_ms"] = eval_match.group("ms")
367         metrics["llama_eval_runs"] = eval_match.group("count")
368         metrics["llama_eval_ms_per_run"] = eval_match.group("ms_per")
369         metrics["llama_eval_tokens_per_second"] = eval_match.group("tps")
370
371     total_match = re.search(
372         r"llama_(?:perf_context_print|print_timings):\s+total time\s*=\s*"
373         r"(?P<ms>[0-9.]+)\s*ms(?:\s*/\s*(?P<count>\d+)\s+tokens)?",
374         text,
375     )
376     if total_match:
377         metrics["llama_total_ms"] = total_match.group("ms")
378         if total_match.group("count"):
379             metrics["llama_total_tokens"] = total_match.group("count")
380
381 def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
382     metrics: dict[str, str] = {}

```

```

381     for line in stdout.splitlines():
382         stripped = line.strip()
383         if not stripped.startswith("|"):
384             continue
385         columns = [column.strip() for column in stripped.strip("|").split("|")]
386         if len(columns) != 7 or columns[0] == "model" or columns[0].startswith(↵
↵ "-"):
387             continue
388         metrics["llama_bench_model"] = columns[0]
389         metrics["llama_bench_size"] = columns[1]
390         metrics["llama_bench_params"] = columns[2]
391         metrics["llama_bench_backend"] = columns[3]
392         metrics["llama_bench_threads"] = columns[4]
393         test_name = columns[5]
394         tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
395         if test_name.startswith("pp"):
396             metrics["llama_bench_prefill_test"] = test_name
397             metrics["llama_bench_prefill_tokens_per_second"] = ↵
↵ tokens_per_second
398         elif test_name.startswith("tg"):
399             metrics["llama_bench_decode_test"] = test_name
400             metrics["llama_bench_decode_tokens_per_second"] = tokens_per_second
401     return metrics
402
403
404 def parse_llama_tokenize_metrics(stdout: str) -> dict[str, str]:
405     metrics: dict[str, str] = {}
406     ids_match = re.search(r"\[(?P<ids>[0-9,\s-]+)\]", stdout)
407     if ids_match:
408         ids = parse_int_list(ids_match.group("ids"))
409         metrics["llama_tokenize_ids"] = " ".join(str(item) for item in ids)
410         metrics["llama_tokenize_token_count_from_ids"] = str(len(ids))
411     count_match = re.search(r"Total number of tokens:\s*(?P<count>\d+)", stdout↵
↵ )
412     if count_match:
413         metrics["llama_tokenize_show_count"] = count_match.group("count")
414     return metrics
415
416
417 def strip_ansi(text: str) -> str:
418     return re.sub(r"\x1b\[([0-?]*[ -/]*[@-~])", "", text)
419
420
421 def metric_as_float(metrics: dict[str, str], key: str) -> float | None:
422     value = metrics.get(key)
423     if value is None:
424         return None
425     try:
426         return float(value)
427     except ValueError:
428         return None
429

```

```

430
431 def metric_values(runs: list[CommandResult], key: str) -> list[float]:
432     values: list[float] = []
433     for run in runs:
434         value = metric_as_float(run.metrics, key)
435         if value is not None:
436             values.append(value)
437     return values
438
439
440 def median_metric(runs: list[CommandResult], key: str) -> float | None:
441     values = metric_values(runs, key)
442     return statistics.median(values) if values else None
443
444
445 def format_summary(name: str, runs: list[CommandResult], keys: list[str]) -> ↵
    ↵ list[str]:
446     lines = [f"[summary:{name}]", f"rounds={len(runs)}"]
447     for key in keys:
448         values = metric_values(runs, key)
449         if not values:
450             continue
451         lines.append(
452             f"{key}_median={statistics.median(values):.6f} "
453             f"{key}_min={min(values):.6f} "
454             f"{key}_max={max(values):.6f}"
455         )
456     return lines
457
458
459 def build_lcqi(build_dir: Path, jobs: int, timeout_seconds: int) -> Path:
460     configure = run_command(
461         "cmake_configure_lcqi",
462         [
463             "cmake",
464             "-S",
465             str(volume_dir()),
466             "-B",
467             str(build_dir),
468             "-DCMAKE_BUILD_TYPE=Release",
469         ],
470         timeout_seconds=timeout_seconds,
471     )
472     ensure_success(configure, "configure LCQI")
473     build = run_command(
474         "cmake_build_lcqi_llama_gguf",
475         ["cmake", "--build", str(build_dir), "--target", LCQI_TARGET, "-j", str(↵
    ↵ (jobs))],
476         timeout_seconds=timeout_seconds,
477     )
478     ensure_success(build, "build LCQI llama GGUF target")
479     binary = build_dir / "labs" / "linux_cpu_inference" / LCQI_TARGET

```

```

480     if not binary.exists() or not os.access(binary, os.X_OK):
481         raise RuntimeError(f"LCQI binary not found or not executable: {binary}"↵
↵ )
482     return binary
483
484
485 def ensure_llama_targets(
486     cache_dir: Path,
487     *,
488     jobs: int,
489     skip_build: bool,
490     timeout_seconds: int,
491 ) -> tuple[Path, Path, Path]:
492     build_dir = cache_dir / "llama.cpp" / "build"
493     binaries = {
494         LLAMA_SIMPLE_TARGET: build_dir / "bin" / LLAMA_SIMPLE_TARGET,
495         LLAMA_BENCH_TARGET: build_dir / "bin" / LLAMA_BENCH_TARGET,
496         LLAMA_TOKENIZE_TARGET: build_dir / "bin" / LLAMA_TOKENIZE_TARGET,
497     }
498     if not skip_build:
499         run_smollm2_small_smoke.ensure_llama_simple_chat(
500             cache_dir,
501             jobs=jobs,
502             skip_build=False,
503         )
504     for target, binary in binaries.items():
505         if binary.exists() and os.access(binary, os.X_OK):
506             continue
507         if skip_build:
508             raise RuntimeError(f"{target} is missing and --skip-llama-build was↵
↵ set: {binary}")
509         build = run_command(
510             f"cmake_build_{target}",
511             ["cmake", "--build", str(build_dir), "--target", target, "-j", str(↵
↵ jobs)],
512             timeout_seconds=timeout_seconds,
513         )
514         ensure_success(build, f"build llama.cpp target {target}")
515         if not binary.exists() or not os.access(binary, os.X_OK):
516             raise RuntimeError(f"{target} was not produced: {binary}")
517     return (
518         binaries[LLAMA_SIMPLE_TARGET],
519         binaries[LLAMA_BENCH_TARGET],
520         binaries[LLAMA_TOKENIZE_TARGET],
521     )
522
523
524 def run_lcqi_round(
525     binary: Path,
526     model_path: Path,
527     *,
528     user_prompt: str,

```

```

529     max_new_tokens: int,
530     round_index: int,
531     timeout_seconds: int,
532     name_prefix: str = "lcqi_round",
533     env: dict[str, str] | None = None,
534     threads: int | None = None,
535 ) -> CommandResult:
536     command = [
537         str(binary),
538         str(model_path),
539         "--prompt",
540         user_prompt,
541         "--max-new",
542         str(max_new_tokens),
543         "--benchmark",
544         "--decode-text",
545     ]
546     if threads is not None:
547         command.extend(["--threads", str(threads)])
548     result = run_command(
549         f"{name_prefix}_{round_index}",
550         command,
551         timeout_seconds=timeout_seconds,
552         env=env,
553     )
554     return CommandResult(
555         result.name,
556         result.args,
557         result.returncode,
558         result.stdout,
559         result.stderr,
560         parse_lcqi_metrics(result.stdout),
561     )
562
563
564 def run_llama_simple_round(
565     binary: Path,
566     model_path: Path,
567     *,
568     full_prompt: str,
569     max_new_tokens: int,
570     round_index: int,
571     timeout_seconds: int,
572 ) -> CommandResult:
573     command = [
574         str(binary),
575         "-m",
576         str(model_path),
577         "-n",
578         "0",
579         "-n",
580         str(max_new_tokens),

```

```
581     full_prompt,
582 ]
583 result = run_command(
584     f"llama_simple_round_{round_index}",
585     command,
586     timeout_seconds=timeout_seconds,
587     env={"LC_ALL": "C"},
588 )
589 return CommandResult(
590     result.name,
591     result.args,
592     result.returncode,
593     result.stdout,
594     result.stderr,
595     parse_llama_simple_metrics(result.stderr, result.stdout, full_prompt),
596 )
597
598
599 def run_llama_tokenize(
600     binary: Path,
601     model_path: Path,
602     *,
603     full_prompt: str,
604     timeout_seconds: int,
605 ) -> CommandResult:
606     command = [
607         str(binary),
608         "-m",
609         str(model_path),
610         "--ids",
611         "--show-count",
612         "--log-disable",
613         "-p",
614         full_prompt,
615     ]
616     result = run_command(
617         "llama_tokenize_same_prompt",
618         command,
619         timeout_seconds=timeout_seconds,
620         env={"LC_ALL": "C"},
621     )
622     return CommandResult(
623         result.name,
624         result.args,
625         result.returncode,
626         result.stdout,
627         result.stderr,
628         parse_llama_tokenize_metrics(result.stdout),
629     )
630
631
632 def run_llama_bench(
```

```

633     binary: Path,
634     model_path: Path,
635     *,
636     prompt_tokens: int,
637     max_new_tokens: int,
638     rounds: int,
639     timeout_seconds: int,
640 ) -> CommandResult:
641     command = [
642         str(binary),
643         "-m",
644         str(model_path),
645         "-ngl",
646         "0",
647         "-p",
648         str(prompt_tokens),
649         "-n",
650         str(max_new_tokens),
651         "-r",
652         str(rounds),
653     ]
654     result = run_command(
655         "llama_bench_same_token_count",
656         command,
657         timeout_seconds=timeout_seconds,
658         env={"LC_ALL": "C"},
659     )
660     return CommandResult(
661         result.name,
662         result.args,
663         result.returncode,
664         result.stdout,
665         result.stderr,
666         parse_llama_bench_metrics(result.stdout),
667     )
668
669
670 def current_commit() -> str:
671     result = subprocess.run(
672         ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
673         text=True,
674         capture_output=True,
675         check=False,
676     )
677     return result.stdout.strip() if result.returncode == 0 else "unknown"
678
679
680 def working_tree_dirty() -> str:
681     result = subprocess.run(
682         ["git", "-C", str(repo_dir()), "status", "--porcelain"],
683         text=True,
684         capture_output=True,

```

```

685         check=False,
686     )
687     if result.returncode != 0:
688         return "unknown"
689     return "true" if result.stdout.strip() else "false"
690
691
692 def command_output(args: list[str]) -> str:
693     completed = subprocess.run(args, text=True, capture_output=True, check=↵
↵ False)
694     if completed.returncode != 0:
695         return "unknown"
696     return completed.stdout.strip()
697
698
699 def llama_source_commit(cache_dir: Path) -> str:
700     source_dir = cache_dir / "llama.cpp"
701     if not (source_dir / ".git").exists():
702         return "unavailable"
703     result = subprocess.run(
704         ["git", "-C", str(source_dir), "rev-parse", "HEAD"],
705         text=True,
706         capture_output=True,
707         check=False,
708     )
709     return result.stdout.strip() if result.returncode == 0 else "unknown"
710
711
712 def write_report(
713     report_path: Path,
714     *,
715     cache_dir: Path,
716     model_path: Path,
717     user_prompt: str,
718     full_prompt: str,
719     max_new_tokens: int,
720     lcqi_runs: list[CommandResult],
721     lcqi_batch_prefill_off_runs: list[CommandResult],
722     lcqi_serial_runs: list[CommandResult],
723     lcqi_q4_direct_off_runs: list[CommandResult],
724     lcqi_q5_0_packed_off_runs: list[CommandResult],
725     lcqi_q6_k_direct_runs: list[CommandResult],
726     lcqi_q8_0_direct_off_runs: list[CommandResult],
727     lcqi_q8_0_lm_head_off_runs: list[CommandResult],
728     lcqi_ggml_selective_direct_runs: list[CommandResult],
729     lcqi_ggml_direct_runs: list[CommandResult],
730     llama_tokenize: CommandResult,
731     llama_simple_runs: list[CommandResult],
732     llama_bench: CommandResult,
733     skip_llama_build: bool,
734 ) -> None:
735     report_path.parent.mkdir(parents=True, exist_ok=True)

```



```

736     model_matches, actual_sha256, actual_bytes = run_smolllm2_small_smoke.↵
↵ verify_model_file(
737         model_path
738     )
739     actual_llama_commit = llama_source_commit(cache_dir)
740     expected_llama_commit = run_smolllm2_small_smoke.LLAMA_CPP_COMMIT
741     lines = [
742         "LCQI vs llama.cpp SmolLM2 Same Input Compare",
743         "",
744         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
745         f"repo_commit={current_commit()}",
746         f"working_tree_dirty={working_tree_dirty()}",
747         f"host={command_output(['hostname'])}",
748         f"uname={command_output(['uname', '-a'])}",
749         f"nproc={command_output(['nproc'])}",
750         f"python={sys.version.split()[0]}",
751         f"cache_dir={cache_dir}",
752         "",
753         f"model_source={run_smolllm2_small_smoke.MODEL_GGUF_REPO_ID}",
754         f"model_file={run_smolllm2_small_smoke.MODEL_FILE_NAME}",
755         f"model_path={model_path}",
756         f"model_expected_bytes={run_smolllm2_small_smoke.MODEL_EXPECTED_BYTES}",
757         f"model_actual_bytes={actual_bytes}",
758         f"model_expected_sha256={run_smolllm2_small_smoke.MODEL_EXPECTED_SHA256}↵
↵ ",
759         f"model_actual_sha256={actual_sha256}",
760         f"model_identity_match={str(model_matches).lower()}",
761         f"llama_cpp_expected_commit={expected_llama_commit}",
762         f"llama_cpp_source_git_commit={actual_llama_commit}",
763         "llama_cpp_source_commit_verified="
764         f"{str(actual_llama_commit == expected_llama_commit).lower()}",
765         f"llama_cpp_skip_build={str(skip_llama_build).lower()}",
766         "",
767         f"user_prompt={escape_newlines(user_prompt)}",
768         f"max_new_tokens={max_new_tokens}",
769         f"expanded_chat_prompt={escape_newlines(full_prompt)}",
770         "",
771         "scope=LCQI uses its internal SmolLM2 chat-prompt builder from the same↵
↵ user "
772         "prompt. llama-simple receives the expanded chat prompt string directly↵
↵ . "
773         "The same-input check is prompt-token-count equality plus matching ↵
↵ visible "
774         "generated suffix; llama-bench is reported separately as same-token-↵
↵ count "
775         "synthetic throughput, not same text.",
776         "",
777     ]
778
779     if lcqi_q4_direct_off_runs:
780         lines.extend(format_summary(
781             "lcqi_q4_direct_off_same_input",

```

```
782         lcqi_q4_direct_off_runs,
783         LCQI_SUMMARY_KEYS,
784     ))
785     lines.append("")
786
787     if lcqi_batch_prefill_off_runs:
788         lines.extend(format_summary(
789             "lcqi_batch_prefill_off_same_input",
790             lcqi_batch_prefill_off_runs,
791             LCQI_SUMMARY_KEYS,
792         ))
793     lines.append("")
794
795     if lcqi_q5_0_packed_off_runs:
796         lines.extend(format_summary(
797             "lcqi_q5_0_packed_off_same_input",
798             lcqi_q5_0_packed_off_runs,
799             LCQI_SUMMARY_KEYS,
800         ))
801     lines.append("")
802
803     if lcqi_q6_k_direct_runs:
804         lines.extend(format_summary(
805             "lcqi_q6_k_direct_experimental_same_input",
806             lcqi_q6_k_direct_runs,
807             LCQI_SUMMARY_KEYS,
808         ))
809     lines.append("")
810
811     if lcqi_q8_0_lm_head_off_runs:
812         lines.extend(format_summary(
813             "lcqi_q8_0_lm_head_off_same_input",
814             lcqi_q8_0_lm_head_off_runs,
815             LCQI_SUMMARY_KEYS,
816         ))
817     lines.append("")
818
819     if lcqi_q8_0_direct_off_runs:
820         lines.extend(format_summary(
821             "lcqi_q8_0_direct_off_same_input",
822             lcqi_q8_0_direct_off_runs,
823             LCQI_SUMMARY_KEYS,
824         ))
825     lines.append("")
826
827     if lcqi_serial_runs:
828         lines.extend(format_summary(
829             "lcqi_serial_same_input",
830             lcqi_serial_runs,
831             LCQI_SUMMARY_KEYS,
832         ))
833     lines.append("")
```

```

834
835     lines.extend(format_summary(
836         "lcqi_same_input",
837         lcqi_runs,
838         LCQI_SUMMARY_KEYS,
839     ))
840     lines.append("")
841     if lcqi_ggml_selective_direct_runs:
842         lines.extend(format_summary(
843             "lcqi_ggml_selective_direct_same_input",
844             lcqi_ggml_selective_direct_runs,
845             LCQI_SUMMARY_KEYS,
846         ))
847         lines.append("")
848     if lcqi_ggml_direct_runs:
849         lines.extend(format_summary(
850             "lcqi_ggml_direct_experimental_same_input",
851             lcqi_ggml_direct_runs,
852             LCQI_SUMMARY_KEYS,
853         ))
854         lines.append("")
855     lines.extend(format_summary(
856         "llama_simple_same_input",
857         llama_simple_runs,
858         [
859             "llama_load_ms",
860             "llama_prompt_eval_ms",
861             "llama_prompt_eval_ms_per_token",
862             "llama_prompt_eval_tokens_per_second",
863             "llama_eval_ms",
864             "llama_eval_ms_per_run",
865             "llama_eval_tokens_per_second",
866             "llama_total_ms",
867         ],
868     ))
869     lines.append("")
870
871     add_ratio_summary(
872         lines,
873         lcqi_runs,
874         llama_tokenize,
875         llama_simple_runs,
876         llama_bench,
877         lcqi_serial_runs,
878         lcqi_q4_direct_off_runs,
879         lcqi_q5_0_packed_off_runs,
880         lcqi_q6_k_direct_runs,
881         lcqi_q8_0_direct_off_runs,
882         lcqi_q8_0_lm_head_off_runs,
883         lcqi_batch_prefill_off_runs,
884         lcqi_ggml_selective_direct_runs,
885         lcqi_ggml_direct_runs,

```

```

886     )
887
888     for run in (
889         lcqi_q4_direct_off_runs +
890         lcqi_q5_0_packed_off_runs +
891         lcqi_q6_k_direct_runs +
892         lcqi_q8_0_direct_off_runs +
893         lcqi_q8_0_lm_head_off_runs +
894         lcqi_batch_prefill_off_runs +
895         lcqi_serial_runs +
896         lcqi_runs +
897         lcqi_ggml_selective_direct_runs +
898         lcqi_ggml_direct_runs +
899         [llama_tokenize] +
900         llama_simple_runs +
901         [llama_bench]
902     ):
903         lines.extend(
904             [
905                 "",
906                 f"[{run.name}]",
907                 f"returncode={run.returncode}",
908                 f"command={command_line(run.args)}",
909             ]
910         )
911         for key in sorted(run.metrics):
912             lines.append(f"{key}={run.metrics[key]}")
913         lines.extend(
914             [
915                 "stdout_tail:",
916                 tail(strip_ansi(run.stdout).strip(), 3000),
917                 "stderr_tail:",
918                 tail(strip_ansi(run.stderr).strip(), 3000),
919             ]
920         )
921
922     while lines and lines[-1] == "":
923         lines.pop()
924     report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
925
926
927 def add_ratio_summary(
928     lines: list[str],
929     lcqi_runs: list[CommandResult],
930     llama_tokenize: CommandResult,
931     llama_simple_runs: list[CommandResult],
932     llama_bench: CommandResult,
933     lcqi_serial_runs: list[CommandResult],
934     lcqi_q4_direct_off_runs: list[CommandResult],
935     lcqi_q5_0_packed_off_runs: list[CommandResult],
936     lcqi_q6_k_direct_runs: list[CommandResult],
937     lcqi_q8_0_direct_off_runs: list[CommandResult],

```

```

938     lcqi_q8_0_lm_head_off_runs: list[CommandResult],
939     lcqi_batch_prefill_off_runs: list[CommandResult],
940     lcqi_ggml_selective_direct_runs: list[CommandResult],
941     lcqi_ggml_direct_runs: list[CommandResult],
942 ) -> None:
943     lcqi_prompt_tokens = median_metric(lcqi_runs, "benchmark_prompt_tokens")
944     lcqi_prompt_ids = lcqi_runs[0].metrics.get("prompt_ids") if lcqi_runs else ↵
    ↵ None
945     llama_tokenize_ids = llama_tokenize.metrics.get("llama_tokenize_ids")
946     llama_tokenize_count = metric_as_float(llama_tokenize.metrics, "↵
    ↵ llama_tokenize_show_count")
947     llama_prompt_tokens = median_metric(llama_simple_runs, "↵
    ↵ llama_prompt_eval_tokens")
948     lcqi_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
949     llama_prompt = median_metric(llama_simple_runs, "llama_prompt_eval_ms")
950     lcqi_decode_step = median_metric(lcqi_runs, "derived_decode_ms_per_step")
951     llama_eval_step = median_metric(llama_simple_runs, "llama_eval_ms_per_run")
952     lcqi_weight_ratio = median_metric(lcqi_runs, "↵
    ↵ derived_f32_weight_inflation_ratio")
953     lcqi_direct_share = median_metric(lcqi_runs, "↵
    ↵ derived_direct_quantized_weight_byte_share")
954     bench_pp_tps = metric_as_float(llama_bench.metrics, "↵
    ↵ llama_bench_prefill_tokens_per_second")
955     bench_tg_tps = metric_as_float(llama_bench.metrics, "↵
    ↵ llama_bench_decode_tokens_per_second")
956
957     lines.extend(["[interpretation]", f"prompt_tokens_lcqi_median={↵
    ↵ lcqi_prompt_tokens}"])
958     lines.append(f"prompt_tokens_llama_tokenize={llama_tokenize_count}")
959     lines.append(f"prompt_tokens_llama_simple_median={llama_prompt_tokens}")
960     if lcqi_prompt_tokens == llama_prompt_tokens == llama_tokenize_count:
961         lines.append("same_input_prompt_token_count_match=true")
962     else:
963         lines.append("same_input_prompt_token_count_match=false")
964     lines.append(
965         "same_input_prompt_token_ids_match="
966         f"{str(lcqi_prompt_ids == llama_tokenize_ids).lower()}")
967 )
968     if lcqi_prefill is not None and llama_prompt is not None and llama_prompt >↵
    ↵ 0.0:
969         lines.append(f"prefill_ms_ratio_lcqi_over_llama_simple={lcqi_prefill / ↵
    ↵ llama_prompt:.6f}")
970     if lcqi_decode_step is not None and llama_eval_step is not None and ↵
    ↵ llama_eval_step > 0.0:
971         lines.append(f"decode_step_ms_ratio_lcqi_over_llama_simple={↵
    ↵ lcqi_decode_step / llama_eval_step:.6f}")
972     if lcqi_weight_ratio is not None:
973         lines.append(f"lcqi_f32_dequantized_weight_inflation_ratio={↵
    ↵ lcqi_weight_ratio:.6f}")
974     if lcqi_direct_share is not None:
975         lines.append(f"lcqi_direct_quantized_weight_byte_share={↵
    ↵ lcqi_direct_share:.6f}")

```

```

976 add_lcqi_threaded_ratios(lines, lcqi_serial_runs, lcqi_runs)
977 add_lcqi_ab_ratios(lines, lcqi_q4_direct_off_runs, lcqi_runs)
978 add_lcqi_q5_0_packed_ratios(lines, lcqi_q5_0_packed_off_runs, lcqi_runs)
979 add_lcqi_q6_k_direct_ratios(lines, lcqi_runs, lcqi_q6_k_direct_runs)
980 add_lcqi_q8_0_direct_ratios(lines, lcqi_q8_0_direct_off_runs, lcqi_runs)
981 add_lcqi_q8_0_lm_head_ratios(lines, lcqi_q8_0_lm_head_off_runs, lcqi_runs)
982 add_lcqi_batch_prefill_ratios(lines, lcqi_batch_prefill_off_runs, lcqi_runs)
983 add_lcqi_ggml_selective_ratios(lines, lcqi_runs,
    lcqi_ggml_selective_direct_runs)
984 add_lcqi_ggml_experimental_ratios(lines, lcqi_runs, lcqi_ggml_direct_runs)
985 if bench_pp_tps is not None:
986     lines.append(f"llama_bench_same_token_count_prefill_tps={bench_pp_tps:
    .6f}")
987 if bench_tg_tps is not None:
988     lines.append(f"llama_bench_same_token_count_decode_tps={bench_tg_tps:.6
    f}")
989 lines.append(
990     "root_cause=LCQI keeps shape-compatible Q4_K matrices in the default
    direct "
991     "quantized path, packs shape-compatible Q5_0 matrices into a load-time
    "
992     "AVX2-friendly direct layout for decode while keeping an F32 sidecar
    for "
993     "batched prompt prefill, evaluates the tied Q8_0 embedding/lm_head "
994     "without scanning the dequantized F32 embedding, uses an 8-row AVX2 F32
    "
995     "fallback fast path, and batches prompt prefill F32 fallback
    projections across prompt tokens. Batched "
996     "prefill reduces repeated row-major weight scans and worker dispatches
    "
997     "for prompt evaluation, but decode is still single-token GEMV. The "
998     "selective GGML direct path is reported separately: it only tries Q5_0
    "
999     "and Q8_0 weights that already have AVX2 Q8_0 input kernels, while "
1000     "leaving Q6_K in the F32 fallback. The full experimental "
1001     "Q5_0/Q6_K/Q8_0 direct path "
1002     "uses the same row worker pool and is much faster than the first scalar
    "
1003     "whole-matrix experiment, but it is still slower than the default Q4_K
    "
1004     "plus F32 fallback mix because the GGML block layout, Q6_K scalar "
1005     "unpacking, and per-block scale handling are not yet packed into tuned
    "
1006     "multi-row SIMD kernels. --threads 1 keeps the serial reference. "
1007     "llama.cpp still batches prompt evaluation, uses ggml graph scheduling,
    "
1008     "tuned low-bit CPU kernels, prefetch/thread scheduling, and more
    compact "
1009     "KV/cache execution."
1010 )
1011

```

```

1012
1013 def add_lcqi_threaded_ratios(
1014     lines: list[str],
1015     lcqi_serial_runs: list[CommandResult],
1016     lcqi_runs: list[CommandResult],
1017 ) -> None:
1018     if not lcqi_serial_runs:
1019         return
1020     serial_prefill = median_metric(lcqi_serial_runs, "benchmark_prefill_ms")
1021     threaded_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
1022     serial_decode = median_metric(lcqi_serial_runs, "derived_decode_ms_per_step"
    ↪ ")
1023     threaded_decode = median_metric(lcqi_runs, "derived_decode_ms_per_step")
1024     serial_f32 = median_metric(lcqi_serial_runs, "
    ↪ benchmark_hotspot_f32_fallback_ms")
1025     threaded_f32 = median_metric(lcqi_runs, "benchmark_hotspot_f32_fallback_ms"
    ↪ )
1026     worker_count = median_metric(lcqi_runs, "benchmark_worker_count")
1027     if worker_count is not None:
1028         lines.append(f"lcqi_threaded_worker_count_median={worker_count:.0f}")
1029     if serial_prefill is not None and threaded_prefill is not None and
    ↪ threaded_prefill > 0.0:
1030         lines.append(
1031             "lcqi_threaded_prefill_speedup_serial_over_threaded="
1032             f"{serial_prefill / threaded_prefill:.6f}"
1033         )
1034     if serial_decode is not None and threaded_decode is not None and
    ↪ threaded_decode > 0.0:
1035         lines.append(
1036             "lcqi_threaded_decode_step_speedup_serial_over_threaded="
1037             f"{serial_decode / threaded_decode:.6f}"
1038         )
1039     if serial_f32 is not None and threaded_f32 is not None and threaded_f32 >
    ↪ 0.0:
1040         lines.append(
1041             "lcqi_threaded_f32_hotspot_speedup_serial_over_threaded="
1042             f"{serial_f32 / threaded_f32:.6f}"
1043         )
1044
1045
1046 def add_lcqi_ab_ratios(
1047     lines: list[str],
1048     lcqi_q4_direct_off_runs: list[CommandResult],
1049     lcqi_runs: list[CommandResult],
1050 ) -> None:
1051     if not lcqi_q4_direct_off_runs:
1052         return
1053     off_prefill = median_metric(lcqi_q4_direct_off_runs, "benchmark_prefill_ms"
    ↪ )
1054     on_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
1055     off_decode = median_metric(lcqi_q4_direct_off_runs, "
    ↪ derived_decode_ms_per_step")

```

```

1056 on_decode = median_metric(lcqi_runs, "derived_decode_ms_per_step")
1057 off_down = median_metric(lcqi_q4_direct_off_runs, "↵
↵ benchmark_hotspot_w_down_ms")
1058 on_down = median_metric(lcqi_runs, "benchmark_hotspot_w_down_ms")
1059 off_f32_bytes = median_metric(lcqi_q4_direct_off_runs, "↵
↵ benchmark_f32_weight_bytes")
1060 on_f32_bytes = median_metric(lcqi_runs, "benchmark_f32_weight_bytes")
1061 if off_prefill is not None and on_prefill is not None and on_prefill > 0.0:
1062     lines.append(f"lcqi_q4_direct_prefill_speedup_off_over_on={off_prefill ↵
↵ / on_prefill:.6f}")
1063 if off_decode is not None and on_decode is not None and on_decode > 0.0:
1064     lines.append(f"lcqi_q4_direct_decode_step_speedup_off_over_on={↵
↵ off_decode / on_decode:.6f}")
1065 if off_down is not None and on_down is not None and on_down > 0.0:
1066     lines.append(f"lcqi_q4_direct_w_down_speedup_off_over_on={off_down / ↵
↵ on_down:.6f}")
1067 if off_f32_bytes is not None and on_f32_bytes is not None:
1068     lines.append(f"lcqi_q4_direct_f32_weight_bytes_saved={off_f32_bytes - ↵
↵ on_f32_bytes:.0f}")
1069
1070
1071 def add_lcqi_q5_0_packed_ratios(
1072     lines: list[str],
1073     lcqi_q5_0_packed_off_runs: list[CommandResult],
1074     lcqi_runs: list[CommandResult],
1075 ) -> None:
1076     if not lcqi_q5_0_packed_off_runs:
1077         return
1078     off_prefill = median_metric(lcqi_q5_0_packed_off_runs, "↵
↵ benchmark_prefill_ms")
1079     on_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
1080     off_decode = median_metric(lcqi_q5_0_packed_off_runs, "↵
↵ derived_decode_ms_per_step")
1081     on_decode = median_metric(lcqi_runs, "derived_decode_ms_per_step")
1082     off_f32 = median_metric(lcqi_q5_0_packed_off_runs, "↵
↵ benchmark_hotspot_f32_fallback_ms")
1083     on_f32 = median_metric(lcqi_runs, "benchmark_hotspot_f32_fallback_ms")
1084     on_packed = median_metric(lcqi_runs, "benchmark_hotspot_q5_0_packed_ms")
1085     on_batch_f32 = median_metric(lcqi_runs, "↵
↵ benchmark_hotspot_q5_0_batch_f32_ms")
1086     on_batch_f32_calls = median_metric(lcqi_runs, "↵
↵ benchmark_hotspot_q5_0_batch_f32_calls")
1087     off_f32_bytes = median_metric(lcqi_q5_0_packed_off_runs, "↵
↵ benchmark_f32_weight_bytes")
1088     on_f32_bytes = median_metric(lcqi_runs, "benchmark_f32_weight_bytes")
1089     on_packed_bytes = median_metric(lcqi_runs, "↵
↵ benchmark_q5_0_packed_weight_bytes")
1090     on_batch_f32_bytes = median_metric(lcqi_runs, "↵
↵ benchmark_q5_0_batch_f32_weight_bytes")
1091     on_packed_tensors = median_metric(lcqi_runs, "benchmark_q5_0_packed_tensors↵
↵ ")
1092     if off_prefill is not None and on_prefill is not None and on_prefill > 0.0:

```



```

1093     lines.append(
1094         "lcqi_q5_0_packed_prefill_speedup_off_over_on="
1095         f"{off_prefill / on_prefill:.6f}"
1096     )
1097 if off_decode is not None and on_decode is not None and on_decode > 0.0:
1098     lines.append(
1099         "lcqi_q5_0_packed_decode_step_speedup_off_over_on="
1100         f"{off_decode / on_decode:.6f}"
1101     )
1102 if off_f32 is not None and on_f32 is not None and on_f32 > 0.0:
1103     lines.append(
1104         "lcqi_q5_0_packed_f32_hotspot_speedup_off_over_on="
1105         f"{off_f32 / on_f32:.6f}"
1106     )
1107 if on_packed is not None:
1108     lines.append(f"lcqi_q5_0_packed_hotspot_ms={on_packed:.6f}")
1109 if on_batch_f32 is not None:
1110     lines.append(f"lcqi_q5_0_batch_f32_hotspot_ms={on_batch_f32:.6f}")
1111 if on_batch_f32_calls is not None:
1112     lines.append(f"lcqi_q5_0_batch_f32_calls={on_batch_f32_calls:.0f}")
1113 if off_f32_bytes is not None and on_f32_bytes is not None:
1114     lines.append(
1115         "lcqi_q5_0_packed_f32_weight_bytes_saved="
1116         f"{off_f32_bytes - on_f32_bytes:.0f}"
1117     )
1118 if on_packed_bytes is not None:
1119     lines.append(f"lcqi_q5_0_packed_weight_bytes={on_packed_bytes:.0f}")
1120 if on_batch_f32_bytes is not None:
1121     lines.append(f"lcqi_q5_0_batch_f32_weight_bytes={on_batch_f32_bytes:.0f}↵
↵ }")
1122 if on_packed_tensors is not None:
1123     lines.append(f"lcqi_q5_0_packed_tensors={on_packed_tensors:.0f}")
1124
1125
1126 def add_lcqi_q6_k_direct_ratios(
1127     lines: list[str],
1128     lcqi_runs: list[CommandResult],
1129     lcqi_q6_k_direct_runs: list[CommandResult],
1130 ) -> None:
1131     if not lcqi_q6_k_direct_runs:
1132         return
1133     default_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
1134     q6_prefill = median_metric(lcqi_q6_k_direct_runs, "benchmark_prefill_ms")
1135     default_decode = median_metric(lcqi_runs, "derived_decode_ms_per_step")
1136     q6_decode = median_metric(lcqi_q6_k_direct_runs, "↵
↵ derived_decode_ms_per_step")
1137     default_f32 = median_metric(lcqi_runs, "benchmark_hotspot_f32_fallback_ms")
1138     q6_f32 = median_metric(lcqi_q6_k_direct_runs, "↵
↵ benchmark_hotspot_f32_fallback_ms")
1139     default_f32_bytes = median_metric(lcqi_runs, "benchmark_f32_weight_bytes")
1140     q6_f32_bytes = median_metric(lcqi_q6_k_direct_runs, "↵
↵ benchmark_f32_weight_bytes")

```

```

1141     q6_direct_bytes = median_metric(
1142         lcqi_q6_k_direct_runs,
1143         "benchmark_q6_k_direct_weight_bytes",
1144     )
1145     q6_direct_hotspot = median_metric(
1146         lcqi_q6_k_direct_runs,
1147         "benchmark_hotspot_q6_k_direct_ms",
1148     )
1149     q6_direct_calls = median_metric(
1150         lcqi_q6_k_direct_runs,
1151         "benchmark_hotspot_q6_k_direct_calls",
1152     )
1153     q6_direct_tensors = median_metric(
1154         lcqi_q6_k_direct_runs,
1155         "benchmark_q6_k_direct_tensors",
1156     )
1157     if default_prefill is not None and q6_prefill is not None and q6_prefill > ←
        ↪ 0.0:
1158         lines.append(
1159             "lcqi_q6_k_direct_prefill_speedup_default_over_experimental="
1160             f"{default_prefill / q6_prefill:.6f}"
1161         )
1162     if default_decode is not None and q6_decode is not None and q6_decode > ←
        ↪ 0.0:
1163         lines.append(
1164             "lcqi_q6_k_direct_decode_step_speedup_default_over_experimental="
1165             f"{default_decode / q6_decode:.6f}"
1166         )
1167     if default_f32 is not None and q6_f32 is not None:
1168         lines.append(
1169             "lcqi_q6_k_direct_f32_hotspot_ms_delta_experimental_minus_default="
1170             f"{q6_f32 - default_f32:.6f}"
1171         )
1172     if default_f32_bytes is not None and q6_f32_bytes is not None:
1173         lines.append(
1174             "lcqi_q6_k_direct_f32_weight_bytes_saved="
1175             f"{default_f32_bytes - q6_f32_bytes:.0f}"
1176         )
1177     if q6_direct_bytes is not None:
1178         lines.append(f"lcqi_q6_k_direct_weight_bytes={q6_direct_bytes:.0f}")
1179     if q6_direct_hotspot is not None:
1180         lines.append(f"lcqi_q6_k_direct_hotspot_ms={q6_direct_hotspot:.6f}")
1181     if q6_direct_calls is not None:
1182         lines.append(f"lcqi_q6_k_direct_calls={q6_direct_calls:.0f}")
1183     if q6_direct_tensors is not None:
1184         lines.append(f"lcqi_q6_k_direct_tensors={q6_direct_tensors:.0f}")
1185
1186
1187 def add_lcqi_q8_0_lm_head_ratios(
1188     lines: list[str],
1189     lcqi_q8_0_lm_head_off_runs: list[CommandResult],
1190     lcqi_runs: list[CommandResult],

```

```

1191 ) -> None:
1192     if not lcqi_q8_0_lm_head_off_runs:
1193         return
1194     off_decode = median_metric(lcqi_q8_0_lm_head_off_runs, "↵
↵ derived_decode_ms_per_step")
1195     on_decode = median_metric(lcqi_runs, "derived_decode_ms_per_step")
1196     off_lm_head = median_metric(lcqi_q8_0_lm_head_off_runs, "↵
↵ benchmark_hotspot_lm_head_ms")
1197     on_lm_head = median_metric(lcqi_runs, "benchmark_hotspot_lm_head_ms")
1198     on_q8 = median_metric(lcqi_runs, "benchmark_hotspot_q8_0_direct_ms")
1199     on_q8_bytes = median_metric(lcqi_runs, "benchmark_q8_0_direct_weight_bytes"↵
↵ )
1200     on_q8_tensors = median_metric(lcqi_runs, "benchmark_q8_0_direct_tensors")
1201     if off_decode is not None and on_decode is not None and on_decode > 0.0:
1202         lines.append(
1203             "lcqi_q8_0_lm_head_decode_step_speedup_off_over_on="
1204             f"{off_decode / on_decode:.6f}"
1205         )
1206     if off_lm_head is not None and on_lm_head is not None and on_lm_head > 0.0:
1207         lines.append(
1208             "lcqi_q8_0_lm_head_hotspot_speedup_off_over_on="
1209             f"{off_lm_head / on_lm_head:.6f}"
1210         )
1211     if on_q8 is not None:
1212         lines.append(f"lcqi_q8_0_direct_hotspot_ms={on_q8:.6f}")
1213     if on_q8_bytes is not None:
1214         lines.append(f"lcqi_q8_0_direct_weight_bytes={on_q8_bytes:.0f}")
1215     if on_q8_tensors is not None:
1216         lines.append(f"lcqi_q8_0_direct_tensors={on_q8_tensors:.0f}")
1217
1218
1219 def add_lcqi_q8_0_direct_ratios(
1220     lines: list[str],
1221     lcqi_q8_0_direct_off_runs: list[CommandResult],
1222     lcqi_runs: list[CommandResult],
1223 ) -> None:
1224     if not lcqi_q8_0_direct_off_runs:
1225         return
1226     off_prefill = median_metric(lcqi_q8_0_direct_off_runs, "↵
↵ benchmark_prefill_ms")
1227     on_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
1228     off_decode = median_metric(lcqi_q8_0_direct_off_runs, "↵
↵ derived_decode_ms_per_step")
1229     on_decode = median_metric(lcqi_runs, "derived_decode_ms_per_step")
1230     off_f32 = median_metric(lcqi_q8_0_direct_off_runs, "↵
↵ benchmark_hotspot_f32_fallback_ms")
1231     on_f32 = median_metric(lcqi_runs, "benchmark_hotspot_f32_fallback_ms")
1232     off_calls = median_metric(lcqi_q8_0_direct_off_runs, "↵
↵ benchmark_hotspot_f32_fallback_calls")
1233     on_calls = median_metric(lcqi_runs, "benchmark_hotspot_f32_fallback_calls")
1234     on_q8 = median_metric(lcqi_runs, "benchmark_hotspot_q8_0_direct_ms")
1235     on_q8_calls = median_metric(lcqi_runs, "benchmark_hotspot_q8_0_direct_calls"↵

```

```

↪ ")
1236 on_q8_tensors = median_metric(lcqi_runs, "benchmark_q8_0_direct_tensors")
1237 if off_prefill is not None and on_prefill is not None and on_prefill > 0.0:
1238     lines.append(
1239         "lcqi_q8_0_direct_prefill_speedup_off_over_on="
1240         f"{off_prefill / on_prefill:.6f}"
1241     )
1242 if off_decode is not None and on_decode is not None and on_decode > 0.0:
1243     lines.append(
1244         "lcqi_q8_0_direct_decode_step_speedup_off_over_on="
1245         f"{off_decode / on_decode:.6f}"
1246     )
1247 if off_f32 is not None and on_f32 is not None and on_f32 > 0.0:
1248     lines.append(
1249         "lcqi_q8_0_direct_f32_hotspot_speedup_off_over_on="
1250         f"{off_f32 / on_f32:.6f}"
1251     )
1252 if off_calls is not None and on_calls is not None:
1253     lines.append(
1254         "lcqi_q8_0_direct_f32_fallback_calls_saved="
1255         f"{off_calls - on_calls:.0f}"
1256     )
1257 if on_q8 is not None:
1258     lines.append(f"lcqi_q8_0_direct_all_hotspot_ms={on_q8:.6f}")
1259 if on_q8_calls is not None:
1260     lines.append(f"lcqi_q8_0_direct_all_calls={on_q8_calls:.0f}")
1261 if on_q8_tensors is not None:
1262     lines.append(f"lcqi_q8_0_direct_all_tensors={on_q8_tensors:.0f}")
1263
1264
1265 def add_lcqi_batch_prefill_ratios(
1266     lines: list[str],
1267     lcqi_batch_prefill_off_runs: list[CommandResult],
1268     lcqi_runs: list[CommandResult],
1269 ) -> None:
1270     if not lcqi_batch_prefill_off_runs:
1271         return
1272     off_prefill = median_metric(lcqi_batch_prefill_off_runs, "↪
↪ benchmark_prefill_ms")
1273     on_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
1274     off_f32 = median_metric(lcqi_batch_prefill_off_runs, "↪
↪ benchmark_hotspot_f32_fallback_ms")
1275     on_f32 = median_metric(lcqi_runs, "benchmark_hotspot_f32_fallback_ms")
1276     off_calls = median_metric(lcqi_batch_prefill_off_runs, "↪
↪ benchmark_hotspot_f32_fallback_calls")
1277     on_calls = median_metric(lcqi_runs, "benchmark_hotspot_f32_fallback_calls")
1278     if off_prefill is not None and on_prefill is not None and on_prefill > 0.0:
1279         lines.append(
1280             "lcqi_batch_prefill_speedup_off_over_on="
1281             f"{off_prefill / on_prefill:.6f}"
1282         )
1283     if off_f32 is not None and on_f32 is not None and on_f32 > 0.0:

```

```

1284         lines.append(
1285             "lcqi_batch_prefill_f32_hotspot_speedup_off_over_on="
1286             f"{off_f32 / on_f32:.6f}"
1287         )
1288     if off_calls is not None and on_calls is not None and on_calls > 0.0:
1289         lines.append(
1290             "lcqi_batch_prefill_f32_call_reduction_off_over_on="
1291             f"{off_calls / on_calls:.6f}"
1292         )
1293
1294
1295 def add_lcqi_ggml_selective_ratios(
1296     lines: list[str],
1297     lcqi_runs: list[CommandResult],
1298     lcqi_ggml_selective_direct_runs: list[CommandResult],
1299 ) -> None:
1300     if not lcqi_ggml_selective_direct_runs:
1301         return
1302     default_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
1303     selective_prefill = median_metric(lcqi_ggml_selective_direct_runs, "↵
↵ benchmark_prefill_ms")
1304     default_decode = median_metric(lcqi_runs, "derived_decode_ms_per_step")
1305     selective_decode = median_metric(lcqi_ggml_selective_direct_runs, "↵
↵ derived_decode_ms_per_step")
1306     default_direct_share = median_metric(lcqi_runs, "↵
↵ derived_direct_quantized_weight_byte_share")
1307     selective_direct_share = median_metric(
1308         lcqi_ggml_selective_direct_runs,
1309         "derived_direct_quantized_weight_byte_share",
1310     )
1311     default_f32_bytes = median_metric(lcqi_runs, "benchmark_f32_weight_bytes")
1312     selective_f32_bytes = median_metric(lcqi_ggml_selective_direct_runs, "↵
↵ benchmark_f32_weight_bytes")
1313     selective_hotspot = median_metric(
1314         lcqi_ggml_selective_direct_runs,
1315         "benchmark_hotspot_ggml_direct_ms",
1316     )
1317     if default_prefill is not None and selective_prefill is not None and ↵
↵ selective_prefill > 0.0:
1318         lines.append(
1319             "lcqi_ggml_selective_prefill_speedup_default_over_selective="
1320             f"{default_prefill / selective_prefill:.6f}"
1321         )
1322     if default_decode is not None and selective_decode is not None and ↵
↵ selective_decode > 0.0:
1323         lines.append(
1324             "lcqi_ggml_selective_decode_step_speedup_default_over_selective="
1325             f"{default_decode / selective_decode:.6f}"
1326         )
1327     if default_direct_share is not None and selective_direct_share is not None:
1328         lines.append(
1329             "lcqi_ggml_selective_direct_share_delta="

```

```

1330         f"{selective_direct_share - default_direct_share:.6f}"
1331     )
1332     if default_f32_bytes is not None and selective_f32_bytes is not None:
1333         lines.append(
1334             "lcqi_ggml_selective_f32_weight_bytes_saved="
1335             f"{default_f32_bytes - selective_f32_bytes:.0f}"
1336         )
1337     if selective_hotspot is not None:
1338         lines.append(f"lcqi_ggml_selective_hotspot_ms={selective_hotspot:.6f}")
1339
1340
1341 def add_lcqi_ggml_experimental_ratios(
1342     lines: list[str],
1343     lcqi_runs: list[CommandResult],
1344     lcqi_ggml_direct_runs: list[CommandResult],
1345 ) -> None:
1346     if not lcqi_ggml_direct_runs:
1347         return
1348     default_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
1349     ggml_prefill = median_metric(lcqi_ggml_direct_runs, "benchmark_prefill_ms")
1350     default_decode = median_metric(lcqi_runs, "derived_decode_ms_per_step")
1351     ggml_decode = median_metric(lcqi_ggml_direct_runs, "↵
↵ derived_decode_ms_per_step")
1352     default_direct_share = median_metric(lcqi_runs, "↵
↵ derived_direct_quantized_weight_byte_share")
1353     ggml_direct_share = median_metric(lcqi_ggml_direct_runs, "↵
↵ derived_direct_quantized_weight_byte_share")
1354     ggml_hotspot = median_metric(lcqi_ggml_direct_runs, "↵
↵ benchmark_hotspot_ggml_direct_ms")
1355     if default_prefill is not None and ggml_prefill is not None and ↵
↵ ggml_prefill > 0.0:
1356         lines.append(
1357             "lcqi_ggml_experimental_prefill_speedup_default_over_experimental="
1358             f"{default_prefill / ggml_prefill:.6f}"
1359         )
1360     if default_decode is not None and ggml_decode is not None and ggml_decode >↵
↵ 0.0:
1361         lines.append(
1362             "↵
↵ lcqi_ggml_experimental_decode_step_speedup_default_over_experimental="
1363             f"{default_decode / ggml_decode:.6f}"
1364         )
1365     if default_direct_share is not None and ggml_direct_share is not None:
1366         lines.append(
1367             "lcqi_ggml_experimental_direct_share_delta="
1368             f"{ggml_direct_share - default_direct_share:.6f}"
1369         )
1370     if ggml_hotspot is not None:
1371         lines.append(f"lcqi_ggml_experimental_hotspot_ms={ggml_hotspot:.6f}")
1372
1373
1374 def parse_args() -> argparse.Namespace:

```

```

1375     parser = argparse.ArgumentParser(
1376         description="Run same-input SmolLM2 comparison between LCQI and llama.↵
↵ cpp."
1377     )
1378     parser.add_argument("--cache-dir", type=Path, default=↵
↵ run_smolllm2_small_smoke.default_cache_dir())
1379     parser.add_argument("--model-path", type=Path, default=None)
1380     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
1381     parser.add_argument("--report", type=Path, default=default_report_path())
1382     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
1383     parser.add_argument("--max-new", type=int, default=DEFAULT_MAX_NEW_TOKENS)
1384     parser.add_argument("--rounds", type=int, default=DEFAULT_ROUNDS)
1385     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
1386     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
1387     parser.add_argument("--force-download", action="store_true")
1388     parser.add_argument("--no-download", action="store_true")
1389     parser.add_argument("--skip-llama-build", action="store_true")
1390     parser.add_argument("--include-lcqi-q4-direct-off", action=argparse.↵
↵ BooleanOptionalAction, default=True)
1391     parser.add_argument("--include-lcqi-q5-0-packed-off", action=argparse.↵
↵ BooleanOptionalAction, default=True)
1392     parser.add_argument("--include-lcqi-q6-k-direct", action=argparse.↵
↵ BooleanOptionalAction, default=True)
1393     parser.add_argument("--include-lcqi-q8-0-direct-off", action=argparse.↵
↵ BooleanOptionalAction, default=True)
1394     parser.add_argument("--include-lcqi-q8-0-lm-head-off", action=argparse.↵
↵ BooleanOptionalAction, default=True)
1395     parser.add_argument("--include-lcqi-batch-prefill-off", action=argparse.↵
↵ BooleanOptionalAction, default=True)
1396     parser.add_argument("--include-lcqi-ggml-selective-direct", action=argparse.↵
↵ BooleanOptionalAction, default=True)
1397     parser.add_argument("--include-lcqi-ggml-direct", action=argparse.↵
↵ BooleanOptionalAction, default=True)
1398     return parser.parse_args()
1399
1400
1401 def main() -> int:
1402     args = parse_args()
1403     if args.rounds <= 0:
1404         print("[lcqi-same-input] --rounds must be positive", file=sys.stderr)
1405         return 1
1406     if args.max_new < 0:
1407         print("[lcqi-same-input] --max-new cannot be negative", file=sys.stderr↵
↵ )
1408     return 1
1409
1410 cache_dir = args.cache_dir.expanduser().resolve()
1411 model_path = (
1412     args.model_path.expanduser().resolve()
1413     if args.model_path
1414     else cache_dir / "models" / run_smolllm2_small_smoke.MODEL_FILE_NAME

```



```

1415     )
1416     build_dir = args.build_dir.expanduser().resolve()
1417     report_path = args.report.expanduser().resolve()
1418     full_prompt = smollm2_chat_prompt(args.prompt)
1419
1420     try:
1421         run_smollm2_small_smoke.download_model(
1422             model_path,
1423             force_download=args.force_download,
1424             no_download=args.no_download,
1425         )
1426         lcqi_binary = build_lcqi(build_dir, args.jobs, args.timeout_seconds)
1427         llama_simple_binary, llama_bench_binary, llama_tokenize_binary = ←
    ↪ ensure_llama_targets(
1428             cache_dir,
1429             jobs=args.jobs,
1430             skip_build=args.skip_llama_build,
1431             timeout_seconds=args.timeout_seconds,
1432         )
1433
1434         lcqi_q4_direct_off_runs: list[CommandResult] = []
1435         lcqi_q5_0_packed_off_runs: list[CommandResult] = []
1436         lcqi_q6_k_direct_runs: list[CommandResult] = []
1437         lcqi_q8_0_direct_off_runs: list[CommandResult] = []
1438         lcqi_q8_0_lm_head_off_runs: list[CommandResult] = []
1439         lcqi_batch_prefill_off_runs: list[CommandResult] = []
1440         lcqi_serial_runs: list[CommandResult] = []
1441         lcqi_runs: list[CommandResult] = []
1442         lcqi_ggml_selective_direct_runs: list[CommandResult] = []
1443         lcqi_ggml_direct_runs: list[CommandResult] = []
1444         llama_simple_runs: list[CommandResult] = []
1445         for round_index in range(1, args.rounds + 1):
1446             if args.include_lcqi_q4_direct_off:
1447                 print(f"[lcqi-same-input] LCQI Q4 direct off round {round_index}←
    ↪ }/{args.rounds}")
1448                 lcqi_q4_direct_off_runs.append(
1449                     run_lcqi_round(
1450                         lcqi_binary,
1451                         model_path,
1452                         user_prompt=args.prompt,
1453                         max_new_tokens=args.max_new,
1454                         round_index=round_index,
1455                         timeout_seconds=args.timeout_seconds,
1456                         name_prefix="lcqi_q4_direct_off_round",
1457                         env={LCQI_Q4_DIRECT_ENV: LCQI_Q4_DIRECT_OFF},
1458                     )
1459                 )
1460                 ensure_success(
1461                     lcqi_q4_direct_off_runs[-1],
1462                     "run LCQI Q4 direct off same-input round",
1463                 )
1464             if args.include_lcqi_q5_0_packed_off:

```



```

1465         print(f"[lcqi-same-input] LCQI Q5_0 packed off round {↵
↵ round_index}/{args.rounds}")
1466         lcqi_q5_0_packed_off_runs.append(
1467             run_lcqi_round(
1468                 lcqi_binary,
1469                 model_path,
1470                 user_prompt=args.prompt,
1471                 max_new_tokens=args.max_new,
1472                 round_index=round_index,
1473                 timeout_seconds=args.timeout_seconds,
1474                 name_prefix="lcqi_q5_0_packed_off_round",
1475                 env={LCQI_Q5_0_PACKED_ENV: LCQI_Q5_0_PACKED_OFF},
1476             )
1477         )
1478         ensure_success(
1479             lcqi_q5_0_packed_off_runs[-1],
1480             "run LCQI Q5_0 packed off same-input round",
1481         )
1482     if args.include_lcqi_q8_0_lm_head_off:
1483         print(f"[lcqi-same-input] LCQI Q8_0 lm head off round {↵
↵ round_index}/{args.rounds}")
1484         lcqi_q8_0_lm_head_off_runs.append(
1485             run_lcqi_round(
1486                 lcqi_binary,
1487                 model_path,
1488                 user_prompt=args.prompt,
1489                 max_new_tokens=args.max_new,
1490                 round_index=round_index,
1491                 timeout_seconds=args.timeout_seconds,
1492                 name_prefix="lcqi_q8_0_lm_head_off_round",
1493                 env={LCQI_Q8_0_TIED_LM_HEAD_ENV: ↵
↵ LCQI_Q8_0_TIED_LM_HEAD_OFF},
1494             )
1495         )
1496         ensure_success(
1497             lcqi_q8_0_lm_head_off_runs[-1],
1498             "run LCQI Q8_0 lm head off same-input round",
1499         )
1500     if args.include_lcqi_q6_k_direct:
1501         print(
1502             "[lcqi-same-input] LCQI Q6_K direct experimental round "
1503             f"{round_index}/{args.rounds}"
1504         )
1505         lcqi_q6_k_direct_runs.append(
1506             run_lcqi_round(
1507                 lcqi_binary,
1508                 model_path,
1509                 user_prompt=args.prompt,
1510                 max_new_tokens=args.max_new,
1511                 round_index=round_index,
1512                 timeout_seconds=args.timeout_seconds,
1513                 name_prefix="lcqi_q6_k_direct_experimental_round",

```

```

1514         env={LCQI_Q6_K_DIRECT_ENV: LCQI_Q6_K_DIRECT_ON},
1515     )
1516 )
1517 ensure_success(
1518     lcqi_q6_k_direct_runs[-1],
1519     "run LCQI Q6_K direct experimental same-input round",
1520 )
1521 if args.include_lcqi_q8_0_direct_off:
1522     print(f"[lcqi-same-input] LCQI Q8_0 direct off round {↵
↵ round_index}/{args.rounds}")
1523     lcqi_q8_0_direct_off_runs.append(
1524         run_lcqi_round(
1525             lcqi_binary,
1526             model_path,
1527             user_prompt=args.prompt,
1528             max_new_tokens=args.max_new,
1529             round_index=round_index,
1530             timeout_seconds=args.timeout_seconds,
1531             name_prefix="lcqi_q8_0_direct_off_round",
1532             env={LCQI_Q8_0_DIRECT_ENV: LCQI_Q8_0_DIRECT_OFF},
1533         )
1534     )
1535     ensure_success(
1536         lcqi_q8_0_direct_off_runs[-1],
1537         "run LCQI Q8_0 direct off same-input round",
1538     )
1539 if args.include_lcqi_batch_prefill_off:
1540     print(f"[lcqi-same-input] LCQI batch prefill off round {↵
↵ round_index}/{args.rounds}")
1541     lcqi_batch_prefill_off_runs.append(
1542         run_lcqi_round(
1543             lcqi_binary,
1544             model_path,
1545             user_prompt=args.prompt,
1546             max_new_tokens=args.max_new,
1547             round_index=round_index,
1548             timeout_seconds=args.timeout_seconds,
1549             name_prefix="lcqi_batch_prefill_off_round",
1550             env={LCQI_BATCH_PREFILL_ENV: LCQI_BATCH_PREFILL_OFF},
1551         )
1552     )
1553     ensure_success(
1554         lcqi_batch_prefill_off_runs[-1],
1555         "run LCQI batch prefill off same-input round",
1556     )
1557     print(f"[lcqi-same-input] LCQI serial round {round_index}/{args.↵
↵ rounds}")
1558     lcqi_serial_runs.append(
1559         run_lcqi_round(
1560             lcqi_binary,
1561             model_path,
1562             user_prompt=args.prompt,

```

```

1563         max_new_tokens=args.max_new,
1564         round_index=round_index,
1565         timeout_seconds=args.timeout_seconds,
1566         name_prefix="lcqi_serial_round",
1567         threads=1,
1568     )
1569 )
1570 ensure_success(lcqi_serial_runs[-1], "run LCQI serial same-input ↵
↵ round")
1571 print(f"[lcqi-same-input] LCQI round {round_index}/{args.rounds}")
1572 lcqi_runs.append(
1573     run_lcqi_round(
1574         lcqi_binary,
1575         model_path,
1576         user_prompt=args.prompt,
1577         max_new_tokens=args.max_new,
1578         round_index=round_index,
1579         timeout_seconds=args.timeout_seconds,
1580     )
1581 )
1582 ensure_success(lcqi_runs[-1], "run LCQI same-input round")
1583 if args.include_lcqi_ggml_selective_direct:
1584     print(
1585         "[lcqi-same-input] LCQI GGML selective direct round "
1586         f"{round_index}/{args.rounds}"
1587     )
1588     lcqi_ggml_selective_direct_runs.append(
1589         run_lcqi_round(
1590             lcqi_binary,
1591             model_path,
1592             user_prompt=args.prompt,
1593             max_new_tokens=args.max_new,
1594             round_index=round_index,
1595             timeout_seconds=args.timeout_seconds,
1596             name_prefix="lcqi_ggml_selective_direct_round",
1597             env={
1598                 LCQI_GGML_SELECTIVE_DIRECT_ENV: ↵
↵ LCQI_GGML_SELECTIVE_DIRECT_ON,
1599             },
1600         )
1601     )
1602     ensure_success(
1603         lcqi_ggml_selective_direct_runs[-1],
1604         "run LCQI GGML selective direct same-input round",
1605     )
1606 if args.include_lcqi_ggml_direct:
1607     print(f"[lcqi-same-input] LCQI GGML direct experimental round {↵
↵ round_index}/{args.rounds}")
1608     lcqi_ggml_direct_runs.append(
1609         run_lcqi_round(
1610             lcqi_binary,
1611             model_path,

```

```

1612         user_prompt=args.prompt,
1613         max_new_tokens=args.max_new,
1614         round_index=round_index,
1615         timeout_seconds=args.timeout_seconds,
1616         name_prefix="lcqi_ggml_direct_experimental_round",
1617         env={LCQI_GGML_DIRECT_ENV: LCQI_GGML_DIRECT_ON},
1618     )
1619 )
1620 ensure_success(
1621     lcqi_ggml_direct_runs[-1],
1622     "run LCQI GGML direct experimental same-input round",
1623 )
1624 print(f"[lcqi-same-input] llama-simple round {round_index}/{args.↵
↵ rounds}")
1625 llama_simple_runs.append(
1626     run_llama_simple_round(
1627         llama_simple_binary,
1628         model_path,
1629         full_prompt=full_prompt,
1630         max_new_tokens=args.max_new,
1631         round_index=round_index,
1632         timeout_seconds=args.timeout_seconds,
1633     )
1634 )
1635 ensure_success(llama_simple_runs[-1], "run llama-simple same-input ↵
↵ round")
1636
1637 prompt_tokens = int(lcqi_runs[0].metrics["benchmark_prompt_tokens"])
1638 print("[lcqi-same-input] llama-tokenize prompt identity check")
1639 llama_tokenize = run_llama_tokenize(
1640     llama_tokenize_binary,
1641     model_path,
1642     full_prompt=full_prompt,
1643     timeout_seconds=args.timeout_seconds,
1644 )
1645 ensure_success(llama_tokenize, "run llama-tokenize same-input check")
1646 print("[lcqi-same-input] llama-bench same-token-count reference")
1647 llama_bench = run_llama_bench(
1648     llama_bench_binary,
1649     model_path,
1650     prompt_tokens=prompt_tokens,
1651     max_new_tokens=args.max_new,
1652     rounds=args.rounds,
1653     timeout_seconds=args.timeout_seconds,
1654 )
1655 ensure_success(llama_bench, "run llama-bench same-token-count reference↵
↵ ")
1656
1657 write_report(
1658     report_path,
1659     cache_dir=cache_dir,
1660     model_path=model_path,

```

```

1661         user_prompt=args.prompt,
1662         full_prompt=full_prompt,
1663         max_new_tokens=args.max_new,
1664         lcqi_runs=lcqi_runs,
1665         lcqi_batch_prefill_off_runs=lcqi_batch_prefill_off_runs,
1666         lcqi_serial_runs=lcqi_serial_runs,
1667         lcqi_q4_direct_off_runs=lcqi_q4_direct_off_runs,
1668         lcqi_q5_0_packed_off_runs=lcqi_q5_0_packed_off_runs,
1669         lcqi_q6_k_direct_runs=lcqi_q6_k_direct_runs,
1670         lcqi_q8_0_direct_off_runs=lcqi_q8_0_direct_off_runs,
1671         lcqi_q8_0_lm_head_off_runs=lcqi_q8_0_lm_head_off_runs,
1672         lcqi_ggml_selective_direct_runs=lcqi_ggml_selective_direct_runs,
1673         lcqi_ggml_direct_runs=lcqi_ggml_direct_runs,
1674         llama_tokenize=llama_tokenize,
1675         llama_simple_runs=llama_simple_runs,
1676         llama_bench=llama_bench,
1677         skip_llama_build=args.skip_llama_build,
1678     )
1679 except Exception as error:
1680     print(f"[lcqi-same-input] ERROR: {error}", file=sys.stderr)
1681     return 1
1682
1683 print(f"report={report_path}")
1684 for line in format_summary(
1685     "lcqi_same_input",
1686     lcqi_runs,
1687     ["benchmark_prefill_ms", "derived_prefill_prompt_tokens_per_second", "↔
↪ derived_decode_ms_per_step"],
1688 ):
1689     print(line)
1690 for line in format_summary(
1691     "llama_simple_same_input",
1692     llama_simple_runs,
1693     ["llama_prompt_eval_ms", "llama_prompt_eval_tokens_per_second", "↔
↪ llama_eval_ms_per_run"],
1694 ):
1695     print(line)
1696 return 0
1697
1698
1699 if __name__ == "__main__":
1700     raise SystemExit(main())

```

run_gpt2_smoke.py 下载标准 GPT-2 的 config.json、vocab.json、merges.txt 和 model.safetensors，再调用 LCQI 自研 lcqi_gpt2 reference path 生成 token。它证明自研 safetensors、BPE、GPT-2 forward、KV cache decode 和 greedy generation 已连成闭环。

Listing 16.19 tools/run_gpt2_smoke.py

```

1 #!/usr/bin/env python3
2 from __future__ import annotations
3

```

```
4 import argparse
5 import datetime as dt
6 import hashlib
7 import os
8 import shlex
9 import subprocess
10 import sys
11 import urllib.request
12 from dataclasses import dataclass
13 from pathlib import Path
14
15
16 MODEL_REPO_ID = "openai-community/gpt2"
17 MODEL_REVISION = "main"
18 MODEL_FILE_IDENTITIES = {
19     "config.json": (
20         665,
21         "0daed7749b4f02b8f76240d5444551d7b08712dab4d0adb8239c56ba823bb7b4",
22     ),
23     "vocab.json": (
24         1042301,
25         "196139668be63f3b5d6574427317ae82f612a97c5d1cdaf36ed2256dbf636783",
26     ),
27     "merges.txt": (
28         456318,
29         "1ce1664773c50f3e0cc8842619a93edc4624525b728b188a9e0be33b7726adc5",
30     ),
31     "model.safetensors": (
32         548105171,
33         "248dfc3911869ec493c76e65bf2fcf7f615828b0254c12b473182f0f81d3a707",
34     ),
35 }
36 DEFAULT_PROMPT = "Hello, my name is"
37 DEFAULT_MAX_NEW_TOKENS = 1
38 DEFAULT_TIMEOUT_SECONDS = 300
39 DEFAULT_BUILD_JOBS = 2
40
41
42 @dataclass(frozen=True)
43 class CommandResult:
44     args: list[str]
45     returncode: int
46     stdout: str
47     stderr: str
48
49
50 def script_path() -> Path:
51     return Path(__file__).resolve()
52
53
54 def volume_dir() -> Path:
55     return script_path().parents[1]
```

```

56
57
58 def repo_dir() -> Path:
59     return script_path().parents[3]
60
61
62 def default_cache_dir() -> Path:
63     explicit_cache = os.environ.get("LCQI_GPT2_CACHE_DIR")
64     if explicit_cache:
65         return Path(explicit_cache).expanduser()
66     xdg_cache = os.environ.get("XDG_CACHE_HOME")
67     cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "↵
↵ .cache"
68     return cache_root / "lcqi-gpt2-smoke" / MODEL_REPO_ID.replace("/", "--")
69
70
71 def default_build_dir() -> Path:
72     return volume_dir() / "build" / "lcqi-release"
73
74
75 def default_report_path() -> Path:
76     return volume_dir() / "results" / "lcqi-gpt2-smoke.txt"
77
78
79 def command_line(args: list[str]) -> str:
80     return " ".join(shlex.quote(arg) for arg in args)
81
82
83 def run_command(
84     args: list[str],
85     *,
86     cwd: Path | None = None,
87     timeout_seconds: int | None = None,
88 ) -> CommandResult:
89     try:
90         completed = subprocess.run(
91             args,
92             cwd=cwd if cwd else None,
93             text=True,
94             capture_output=True,
95             timeout=timeout_seconds,
96             check=False,
97         )
98     except subprocess.TimeoutExpired as exc:
99         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
100         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
101         return CommandResult(args=args, returncode=124, stdout=stdout, stderr=↵
↵ stderr)
102     return CommandResult(
103         args=args,
104         returncode=completed.returncode,
105         stdout=completed.stdout,

```

```

106         stderr=completed.stderr,
107     )
108
109
110 def ensure_success(result: CommandResult, action: str) -> None:
111     if result.returncode == 0:
112         return
113     raise RuntimeError(
114         f"{action} failed with exit code {result.returncode}\n"
115         f"command: {command_line(result.args)}\n"
116         f"stdout tail:\n{tail(result.stdout)}\n"
117         f"stderr tail:\n{tail(result.stderr)}"
118     )
119
120
121 def tail(text: str, max_chars: int = 4000) -> str:
122     if len(text) <= max_chars:
123         return text
124     return text[-max_chars:]
125
126
127 def sha256_file(path: Path) -> str:
128     digest = hashlib.sha256()
129     with path.open("rb") as handle:
130         for block in iter(lambda: handle.read(1024 * 1024), b''):
131             digest.update(block)
132     return digest.hexdigest()
133
134
135 def model_url(file_name: str) -> str:
136     return (
137         f"https://huggingface.co/{MODEL_REPO_ID}/resolve/"
138         f"{MODEL_REVISION}/{file_name}"
139     )
140
141
142 def download_file(url: str, target: Path, timeout_seconds: int) -> None:
143     part_path = target.with_suffix(target.suffix + ".part")
144     if part_path.exists():
145         part_path.unlink()
146     request = urllib.request.Request(url, headers={"User-Agent": "lcqi-gpt2-↵
↵ smoke"})
147     with urllib.request.urlopen(request, timeout=timeout_seconds) as response:
148         with part_path.open("wb") as output:
149             while True:
150                 block = response.read(1024 * 1024)
151                 if not block:
152                     break
153                 output.write(block)
154     part_path.replace(target)
155
156

```



```

157 def file_identity_matches(path: Path, expected_bytes: int, expected_sha256: str ←
    ↪ ) -> bool:
158     return (
159         path.exists()
160         and path.stat().st_size == expected_bytes
161         and sha256_file(path) == expected_sha256
162     )
163
164
165 def ensure_model_files(cache_dir: Path, *, no_download: bool, timeout_seconds: ←
    ↪ int) -> None:
166     cache_dir.mkdir(parents=True, exist_ok=True)
167     missing = [
168         name
169         for name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES. ←
    ↪ items()
170         if not file_identity_matches(cache_dir / name, expected_bytes, ←
    ↪ expected_sha256)
171     ]
172     if not missing:
173         print(f"[lcqi-gpt2] model cache hit: {cache_dir}")
174         return
175     if no_download:
176         raise RuntimeError(
177             f"missing GPT-2 model files and --no-download was set: {missing}"
178         )
179     for file_name in missing:
180         target = cache_dir / file_name
181         if target.exists():
182             target.unlink()
183         print(f"[lcqi-gpt2] downloading {MODEL_REPO_ID}/{file_name}")
184         download_file(model_url(file_name), target, timeout_seconds)
185         expected_bytes, expected_sha256 = MODEL_FILE_IDENTITIES[file_name]
186         if not file_identity_matches(target, expected_bytes, expected_sha256):
187             raise RuntimeError(
188                 f"downloaded GPT-2 file identity mismatch: {file_name}"
189             )
190
191
192 def build_lcqi_gpt2(build_dir: Path, jobs: int) -> Path:
193     configure = run_command(
194         [
195             "cmake",
196             "-S",
197             str(volume_dir()),
198             "-B",
199             str(build_dir),
200             "-DCMAKE_BUILD_TYPE=Release",
201         ],
202         timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
203     )
204     ensure_success(configure, "configure LCQI")

```

```

205     build = run_command(
206         [
207             "cmake",
208             "--build",
209             str(build_dir),
210             "--target",
211             "lcqi_gpt2",
212             "-j",
213             str(jobs),
214         ],
215         timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
216     )
217     ensure_success(build, "build lcqi_gpt2")
218     binary = build_dir / "labs" / "linux_cpu_inference" / "lcqi_gpt2"
219     if not binary.exists():
220         raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
221     return binary
222
223
224 def write_report(
225     report_path: Path,
226     *,
227     cache_dir: Path,
228     binary: Path,
229     prompt: str,
230     max_new_tokens: int,
231     engine: str,
232     result: CommandResult,
233 ) -> None:
234     report_path.parent.mkdir(parents=True, exist_ok=True)
235     lines = [
236         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
237         f"model_repo_id={MODEL_REPO_ID}",
238         f"model_revision={MODEL_REVISION}",
239         f"model_cache_dir={cache_dir}",
240     ]
241     for file_name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES:
242         path = cache_dir / file_name
243         lines.append(f"file.{file_name}.bytes={path.stat().st_size}")
244         lines.append(f"file.{file_name}.sha256={sha256_file(path)}")
245         lines.append(f"file.{file_name}.expected_bytes={expected_bytes}")
246         lines.append(f"file.{file_name}.expected_sha256={expected_sha256}")
247     lines.extend(
248         [
249             f"binary={binary}",
250             f"prompt={prompt}",
251             f"max_new_tokens={max_new_tokens}",
252             f"engine={engine}",
253             f"command={command_line(result.args)}",
254             f"exit_code={result.returncode}",
255             "stdout_begin",

```

```

256         result.stdout.rstrip(),
257         "stdout_end",
258         "stderr_begin",
259         tail(result.stderr).rstrip(),
260         "stderr_end",
261     ]
262 )
263 passed = result.returncode == 0 and "generated_text " in result.stdout
264 lines.append(f"validation={'PASS' if passed else 'FAIL'}")
265 report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
266
267
268 def parse_args() -> argparse.Namespace:
269     parser = argparse.ArgumentParser(
270         description="Run LCQI self-owned GPT-2 smoke on a HuggingFace ↵
↵ safetensors directory."
271     )
272     parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
273     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
274     parser.add_argument("--report", type=Path, default=default_report_path())
275     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
276     parser.add_argument("--max-new-tokens", type=int, default=↵
↵ DEFAULT_MAX_NEW_TOKENS)
277     parser.add_argument("--engine", choices=("cached", "full"), default="cached↵
↵ ")
278     parser.add_argument("--benchmark", action="store_true")
279     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
280     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
281     parser.add_argument("--no-download", action="store_true")
282     return parser.parse_args()
283
284
285 def main() -> int:
286     args = parse_args()
287     try:
288         if args.max_new_tokens < 0:
289             raise RuntimeError("--max-new-tokens cannot be negative")
290         ensure_model_files(
291             args.cache_dir,
292             no_download=args.no_download,
293             timeout_seconds=args.timeout_seconds,
294         )
295         binary = build_lcqi_gpt2(args.build_dir, args.jobs)
296         command = [
297             str(binary),
298             "--engine",
299             args.engine,
300             str(args.cache_dir),
301             args.prompt,
302             str(args.max_new_tokens),
303         ]

```

```

304     if args.benchmark:
305         command.insert(1, "--benchmark")
306         result = run_command(command, cwd=repo_dir(), timeout_seconds=args.↵
↵ timeout_seconds)
307         write_report(
308             args.report,
309             cache_dir=args.cache_dir,
310             binary=binary,
311             prompt=args.prompt,
312             max_new_tokens=args.max_new_tokens,
313             engine=args.engine,
314             result=result,
315         )
316         print(f"[lcqi-gpt2] report: {args.report}")
317         print(tail(result.stdout, max_chars=2000))
318         return 0 if result.returncode == 0 else result.returncode
319     except Exception as error:
320         print(f"[lcqi-gpt2] error: {error}", file=sys.stderr)
321         return 1
322
323
324 if __name__ == "__main__":
325     raise SystemExit(main())

```

`run_gpt2_benchmark_compare.py` 用同一个 GPT-2 F32 safetensors 模型分别运行 LCQI full-prefix 与 cached-KV 路径,并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考。它的报告说明算法差距、kernel 差距和模型口径差异。

Listing 16.20 tools/run_gpt2_benchmark_compare.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import os
7  import re
8  import shlex
9  import subprocess
10 import sys
11 from dataclasses import dataclass
12 from pathlib import Path
13
14 import run_gpt2_smoke
15
16
17 DEFAULT_PROMPT = "Hello, my name is"
18 DEFAULT_MAX_NEW_TOKENS = 4
19 DEFAULT_BUILD_JOBS = 2
20 DEFAULT_TIMEOUT_SECONDS = 300
21 DEFAULT_SMOLLM2_CACHE_DIR = Path.home() / ".cache" / "lcqi-smolllm2-small-smoke"
22 SMOLLM2_GGUF = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
23

```

```

24
25 @dataclass(frozen=True)
26 class EngineRun:
27     name: str
28     command: list[str]
29     returncode: int
30     stdout: str
31     stderr: str
32     metrics: dict[str, str]
33
34
35 def volume_dir() -> Path:
36     return Path(__file__).resolve().parents[1]
37
38
39 def repo_dir() -> Path:
40     return Path(__file__).resolve().parents[3]
41
42
43 def default_report_path() -> Path:
44     return volume_dir() / "results" / "lcqi-gpt2-benchmark-compare.txt"
45
46
47 def default_build_dir() -> Path:
48     return volume_dir() / "build" / "lcqi-release"
49
50
51 def command_line(args: list[str]) -> str:
52     return " ".join(shlex.quote(arg) for arg in args)
53
54
55 def tail(text: str, max_chars: int = 4000) -> str:
56     return text if len(text) <= max_chars else text[-max_chars:]
57
58
59 def run_command(args: list[str], *, timeout_seconds: int) -> tuple[int, str, ←
    ↪ str]:
60     try:
61         completed = subprocess.run(
62             args,
63             cwd=repo_dir(),
64             text=True,
65             capture_output=True,
66             timeout=timeout_seconds,
67             check=False,
68         )
69     except subprocess.TimeoutExpired as exc:
70         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
71         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
72         return 124, stdout, stderr
73     return completed.returncode, completed.stdout, completed.stderr
74

```

```

75
76 def parse_lcqi_metrics(stdout: str) -> dict[str, str]:
77     metrics: dict[str, str] = {}
78     for line in stdout.splitlines():
79         if not line:
80             continue
81         key, _, value = line.partition(" ")
82         if key in {
83             "engine",
84             "generated_text",
85             "benchmark_load_ms",
86             "benchmark_tokenize_ms",
87             "benchmark_prefill_ms",
88             "benchmark_decode_ms",
89             "benchmark_generate_ms",
90             "benchmark_total_ms",
91             "benchmark_prompt_tokens",
92             "benchmark_generated_tokens",
93             "benchmark_prefill_steps",
94             "benchmark_decode_steps",
95             "benchmark_generate_tokens_per_second",
96             "benchmark_decode_tokens_per_second",
97             "benchmark_kv_cache_bytes",
98         }:
99             metrics[key] = value
100     return metrics
101
102
103 def run_lcqi(
104     binary: Path,
105     model_dir: Path,
106     *,
107     engine: str,
108     prompt: str,
109     max_new_tokens: int,
110     timeout_seconds: int,
111 ) -> EngineRun:
112     command = [
113         str(binary),
114         "--benchmark",
115         "--engine",
116         engine,
117         str(model_dir),
118         prompt,
119         str(max_new_tokens),
120     ]
121     returncode, stdout, stderr = run_command(command, timeout_seconds=↵
↵ timeout_seconds)
122     return EngineRun(
123         name=f"lcqi_{engine}",
124         command=command,
125         returncode=returncode,

```

```

126         stdout=stdout,
127         stderr=stderr,
128         metrics=parse_lcqi_metrics(stdout),
129     )
130
131
132 def llama_bench_binary(cache_dir: Path) -> Path:
133     return cache_dir / "llama.cpp" / "build" / "bin" / "llama-bench"
134
135
136 def smollm2_model_path(cache_dir: Path) -> Path:
137     return cache_dir / "models" / SMOLLM2_GGUF
138
139
140 def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
141     metrics: dict[str, str] = {}
142     lines = [line.strip() for line in stdout.splitlines() if line.strip()]
143     for line in lines:
144         if not line.startswith("|"):
145             continue
146         columns = [column.strip() for column in line.strip("|").split("|")]
147         if len(columns) != 7 or columns[0] == "model" or columns[0].startswith(↵
↵ "-"):
148             continue
149         metrics["llama_model"] = columns[0]
150         metrics["llama_size"] = columns[1]
151         metrics["llama_params"] = columns[2]
152         metrics["llama_backend"] = columns[3]
153         metrics["llama_threads"] = columns[4]
154         test_name = columns[5]
155         tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
156         if test_name.startswith("pp"):
157             metrics["llama_prefill_test"] = test_name
158             metrics["llama_prefill_tps"] = tokens_per_second
159         elif test_name.startswith("tg"):
160             metrics["llama_decode_test"] = test_name
161             metrics["llama_decode_tps"] = tokens_per_second
162     return metrics
163
164
165 def run_llama_bench(cache_dir: Path, *, timeout_seconds: int) -> EngineRun | ↵
↵ None:
166     binary = llama_bench_binary(cache_dir)
167     model = smollm2_model_path(cache_dir)
168     if not binary.exists() or not os.access(binary, os.X_OK) or not model.↵
↵ exists():
169         return None
170     command = [
171         str(binary),
172         "-m",
173         str(model),
174         "-ngl",

```

```

175     "0",
176     "-p",
177     "5",
178     "-n",
179     "4",
180     "-r",
181     "1",
182 ]
183 returncode, stdout, stderr = run_command(command, timeout_seconds=↵
↵ timeout_seconds)
184 return EngineRun(
185     name="llama_cpp_smolLM2_q4_k_m",
186     command=command,
187     returncode=returncode,
188     stdout=stdout,
189     stderr=stderr,
190     metrics=parse_llama_bench_metrics(stdout),
191 )
192
193
194 def write_report(path: Path, runs: list[EngineRun], *, prompt: str, ↵
↵ max_new_tokens: int) -> None:
195     path.parent.mkdir(parents=True, exist_ok=True)
196     lines = [
197         "LCQI GPT-2 Benchmark Compare",
198         "",
199         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
200         f"repo_commit={current_commit()}",
201         f"working_tree_dirty={working_tree_dirty()}",
202         f"python={sys.version.split()[0]}",
203         f"prompt={prompt}",
204         f"max_new_tokens={max_new_tokens}",
205         "",
206         "scope=LCQI full_prefix and cached_kv run the same GPT-2 F32 ↵
↵ safetensors model. "
207         "llama.cpp uses SmolLM2-135M-Instruct Q4_K_M as an external mature↵
↵ engine reference, "
208         "so its numbers are engineering context, not same-model quality ranking↵
↵ .",
209         "",
210     ]
211     for run in runs:
212         lines.extend(
213             [
214                 f"[{run.name}]",
215                 f"returncode={run.returncode}",
216                 f"command={command_line(run.command)}",
217             ]
218         )
219         for key in sorted(run.metrics):
220             lines.append(f"{key}={run.metrics[key]}")
221         lines.extend(

```



```

222         [
223             "stdout_tail:",
224             tail(run.stdout.strip(), 2000),
225             "stderr_tail:",
226             tail(run.stderr.strip(), 2000),
227             "",
228         ]
229     )
230     while lines and lines[-1] == "":
231         lines.pop()
232     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
233
234
235 def current_commit() -> str:
236     result = subprocess.run(
237         ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
238         text=True,
239         capture_output=True,
240         check=False,
241     )
242     return result.stdout.strip() if result.returncode == 0 else "unknown"
243
244
245 def working_tree_dirty() -> str:
246     result = subprocess.run(
247         ["git", "-C", str(repo_dir()), "status", "--porcelain"],
248         text=True,
249         capture_output=True,
250         check=False,
251     )
252     if result.returncode != 0:
253         return "unknown"
254     return "true" if result.stdout.strip() else "false"
255
256
257 def parse_args() -> argparse.Namespace:
258     parser = argparse.ArgumentParser(
259         description="Compare LCQI GPT-2 full-prefix and KV-cache paths, with ↵
↵ optional llama.cpp context."
260     )
261     parser.add_argument("--cache-dir", type=Path, default=run_gpt2_smoke.↵
↵ default_cache_dir())
262     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
263     parser.add_argument("--report", type=Path, default=default_report_path())
264     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
265     parser.add_argument("--max-new-tokens", type=int, default=↵
↵ DEFAULT_MAX_NEW_TOKENS)
266     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
267     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
268     parser.add_argument("--no-download", action="store_true")
269     parser.add_argument("--skip-llama", action="store_true")

```

```

270     parser.add_argument("--smollm2-cache-dir", type=Path, default=↵
↵ DEFAULT_SMOLLM2_CACHE_DIR)
271     return parser.parse_args()
272
273
274 def main() -> int:
275     args = parse_args()
276     if args.max_new_tokens < 0:
277         print("[lcqi-compare] --max-new-tokens cannot be negative", file=sys.↵
↵ stderr)
278         return 1
279     try:
280         run_gpt2_smoke.ensure_model_files(
281             args.cache_dir,
282             no_download=args.no_download,
283             timeout_seconds=args.timeout_seconds,
284         )
285         binary = run_gpt2_smoke.build_lcqi_gpt2(args.build_dir, args.jobs)
286         runs = [
287             run_lcqi(
288                 binary,
289                 args.cache_dir,
290                 engine="full",
291                 prompt=args.prompt,
292                 max_new_tokens=args.max_new_tokens,
293                 timeout_seconds=args.timeout_seconds,
294             ),
295             run_lcqi(
296                 binary,
297                 args.cache_dir,
298                 engine="cached",
299                 prompt=args.prompt,
300                 max_new_tokens=args.max_new_tokens,
301                 timeout_seconds=args.timeout_seconds,
302             ),
303         ]
304         if not args.skip_llama:
305             llama_run = run_llama_bench(
306                 args.smollm2_cache_dir.expanduser().resolve(),
307                 timeout_seconds=args.timeout_seconds,
308             )
309             if llama_run is not None:
310                 runs.append(llama_run)
311         write_report(args.report, runs, prompt=args.prompt, max_new_tokens=args.↵
↵ .max_new_tokens)
312     except Exception as error:
313         print(f"[lcqi-compare] error: {error}", file=sys.stderr)
314         return 1
315
316     for run in runs:
317         print(f"{run.name}: returncode={run.returncode}")
318         for key in sorted(run.metrics):

```

```

319         print(f" {key}={run.metrics[key]}")
320     print(f"report={args.report}")
321     return 0 if all(run.returncode == 0 for run in runs) else 1
322
323
324 if __name__ == "__main__":
325     raise SystemExit(main())

```

`run_gpt2_optimization_ab.py` 专门比较 LCQI 自身改动前后。它从 `baseline commit` 建临时 `worktree`, `baseline` 和当前工作区都通过实践卷 `CMake` 入口构建 `Release`, 再多轮运行同一个 `GPT-2 F32` 模型。报告中的 `median/min/max` 用来约束机器波动, `generated_text` 用来确认优化前后输出没有漂移, `benchmark_worker_count` 用来确认 `cached decoder` 到底是串行、自动线程数还是固定线程数。

Listing 16.21 `tools/run_gpt2_optimization_ab.py`

```

1 #!/usr/bin/env python3
2 from __future__ import annotations
3
4 import argparse
5 import datetime as dt
6 import shutil
7 import statistics
8 import subprocess
9 import sys
10 import tempfile
11 from dataclasses import dataclass
12 from pathlib import Path
13
14
15 DEFAULT_BASELINE_REF = "4febf3f"
16 DEFAULT_PROMPT = "Hello, my name is"
17 DEFAULT_MAX_NEW_TOKENS = 4
18 DEFAULT_ROUNDS = 3
19 DEFAULT_BUILD_JOBS = 2
20 DEFAULT_TIMEOUT_SECONDS = 300
21
22 METRIC_KEYS = [
23     "benchmark_prefill_ms",
24     "benchmark_decode_ms",
25     "benchmark_generate_ms",
26     "benchmark_worker_count",
27     "benchmark_generate_tokens_per_second",
28     "benchmark_decode_tokens_per_second",
29 ]
30
31
32 @dataclass(frozen=True)
33 class RunSample:
34     variant: str
35     engine: str
36     round_id: int

```

```

37     metrics: dict[str, float]
38     generated_text: str
39
40
41 def script_path() -> Path:
42     return Path(__file__).resolve()
43
44
45 def repo_dir() -> Path:
46     return script_path().parents[3]
47
48
49 def volume_dir() -> Path:
50     return script_path().parents[1]
51
52
53 def practice_dir(root: Path) -> Path:
54     return root / "books" / "cpu-volume-3-practice"
55
56
57 def default_report_path() -> Path:
58     return volume_dir() / "results" / "lcqi-gpt2-optimization-ab.txt"
59
60
61 def default_model_dir() -> Path:
62     return Path.home() / ".cache" / "lcqi-gpt2-smoke" / "openai-community--gpt2"
63     ↪ "
64
65 def run_command(
66     args: list[str],
67     *,
68     cwd: Path,
69     timeout_seconds: int,
70 ) -> subprocess.CompletedProcess[str]:
71     return subprocess.run(
72         args,
73         cwd=cwd,
74         text=True,
75         capture_output=True,
76         timeout=timeout_seconds,
77         check=False,
78     )
79
80
81 def ensure_success(
82     completed: subprocess.CompletedProcess[str],
83     action: str,
84 ) -> None:
85     if completed.returncode == 0:
86         return
87     command = (

```

```

88     " ".join(completed.args)
89     if isinstance(completed.args, list)
90     else str(completed.args)
91 )
92 raise RuntimeError(
93     f"{action} failed with exit code {completed.returncode}\n"
94     f"command: {command}\n"
95     f"stdout tail:\n{completed.stdout[-2000:]]}\n"
96     f"stderr tail:\n{completed.stderr[-2000:]]}"
97 )
98
99
100 def git_text(args: list[str]) -> str:
101     completed = run_command(["git", *args], cwd=repo_dir(), timeout_seconds=60)
102     ensure_success(completed, "git command")
103     return completed.stdout.strip()
104
105
106 def working_tree_dirty() -> bool:
107     completed = run_command(
108         ["git", "status", "--porcelain"],
109         cwd=repo_dir(),
110         timeout_seconds=60,
111     )
112     ensure_success(completed, "git status")
113     return bool(completed.stdout.strip())
114
115
116 def build_binary(root: Path, build_dir: Path, jobs: int, timeout_seconds: int) ←
    ↪ -> Path:
117     configure = run_command(
118         [
119             "cmake",
120             "-S",
121             str(practice_dir(root)),
122             "-B",
123             str(build_dir),
124             "-DCMAKE_BUILD_TYPE=Release",
125         ],
126         cwd=root,
127         timeout_seconds=timeout_seconds,
128     )
129     ensure_success(configure, f"configure {root}")
130     build = run_command(
131         [
132             "cmake",
133             "--build",
134             str(build_dir),
135             "--target",
136             "lcqi_gpt2",
137             "-j",
138             str(jobs),

```

```

139     ],
140     cwd=root,
141     timeout_seconds=timeout_seconds,
142 )
143 ensure_success(build, f"build {root}")
144 binary = build_dir / "linux_cpu_inference" / "lcqi_gpt2"
145 if not binary.exists():
146     raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
147 return binary
148
149
150 def parse_metrics(stdout: str) -> tuple[dict[str, float], str]:
151     metrics: dict[str, float] = {}
152     generated_text = ""
153     for line in stdout.splitlines():
154         key, _, value = line.partition(" ")
155         if key in METRIC_KEYS:
156             metrics[key] = float(value)
157         elif key == "generated_text":
158             generated_text = value
159     metrics.setdefault("benchmark_worker_count", 1.0)
160     missing = sorted(set(METRIC_KEYS) - metrics.keys())
161     if missing:
162         raise RuntimeError(f"missing benchmark metrics: {missing}")
163     return metrics, generated_text
164
165
166 def run_sample(
167     binary: Path,
168     *,
169     variant: str,
170     engine: str,
171     round_id: int,
172     model_dir: Path,
173     prompt: str,
174     max_new_tokens: int,
175     threads: int,
176     timeout_seconds: int,
177 ) -> RunSample:
178     command = [
179         str(binary),
180         "--benchmark",
181         "--engine",
182         engine,
183     ]
184     if variant == "current":
185         command.extend(["--threads", str(threads)])
186     command.extend([
187         str(model_dir),
188         prompt,
189         str(max_new_tokens),
190     ])

```

```

191     completed = run_command(
192         command,
193         cwd=repo_dir(),
194         timeout_seconds=timeout_seconds,
195     )
196     ensure_success(completed, f"run {variant} {engine}")
197     metrics, generated_text = parse_metrics(completed.stdout)
198     return RunSample(
199         variant=variant,
200         engine=engine,
201         round_id=round_id,
202         metrics=metrics,
203         generated_text=generated_text,
204     )
205
206
207 def summarize(samples: list[RunSample]) -> list[str]:
208     lines: list[str] = []
209     variants = sorted({sample.variant for sample in samples})
210     engines = sorted({sample.engine for sample in samples})
211     for variant in variants:
212         for engine in engines:
213             selected = [
214                 sample for sample in samples
215                 if sample.variant == variant and sample.engine == engine
216             ]
217             if not selected:
218                 continue
219             lines.append(f"[{variant} {engine}]")
220             for key in METRIC_KEYS:
221                 values = [sample.metrics[key] for sample in selected]
222                 lines.append(
223                     f"{key}=median:{statistics.median(values):.6g},"
224                     f"min:{min(values):.6g},max:{max(values):.6g}"
225                 )
226             texts = sorted({sample.generated_text for sample in selected})
227             lines.append(f"generated_text={texts[0] if len(texts) == 1 else ↵
↵ texts}")
228             lines.append("")
229     return lines
230
231
232 def write_report(
233     path: Path,
234     samples: list[RunSample],
235     *,
236     baseline_ref: str,
237     baseline_commit: str,
238     current_commit: str,
239     prompt: str,
240     max_new_tokens: int,
241     rounds: int,

```

```

242     current_threads: int,
243 ) -> None:
244     path.parent.mkdir(parents=True, exist_ok=True)
245     lines = [
246         "LCQI GPT-2 Optimization A/B",
247         "",
248         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
249         f"baseline_ref={baseline_ref}",
250         f"baseline_commit={baseline_commit}",
251         f"current_commit={current_commit}",
252         f"current_working_tree_dirty={str(working_tree_dirty()).lower()}",
253         f"prompt={prompt}",
254         f"max_new_tokens={max_new_tokens}",
255         f"rounds={rounds}",
256         f"baseline_threads=1",
257         f"current_threads={current_threads}",
258         "",
259         "scope=Both variants are built through books/cpu-volume-3-practice with↵
↵ "
260         "CMAKE_BUILD_TYPE=Release and run on the same GPT-2 F32 safetensors ↵
↵ model. "
261         "The llama.cpp comparison remains in lcqi-gpt2-benchmark-compare.txt; ↵
↵ this "
262         "report isolates LCQI before/after code changes.",
263         "",
264         "[samples]",
265     ]
266     for sample in samples:
267         metric_text = ",".join(
268             f"{key}:{sample.metrics[key]:.6g}" for key in METRIC_KEYS
269         )
270         lines.append(
271             f"round={sample.round_id},variant={sample.variant},engine={sample.↵
↵ engine},"
272             f"{metric_text},generated_text={sample.generated_text}"
273         )
274     lines.append("")
275     lines.extend(summarize(samples))
276     while lines and lines[-1] == "":
277         lines.pop()
278     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
279
280
281 def parse_args() -> argparse.Namespace:
282     parser = argparse.ArgumentParser(description="Run LCQI GPT-2 before/after A↵
↵ /B benchmark.")
283     parser.add_argument("--baseline-ref", default=DEFAULT_BASELINE_REF)
284     parser.add_argument("--model-dir", type=Path, default=default_model_dir())
285     parser.add_argument("--report", type=Path, default=default_report_path())
286     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
287     parser.add_argument("--max-new-tokens", type=int, default=↵
↵ DEFAULT_MAX_NEW_TOKENS)

```



```

288     parser.add_argument("--rounds", type=int, default=DEFAULT_ROUNDS)
289     parser.add_argument("--threads", type=int, default=0)
290     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
291     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
292     return parser.parse_args()
293
294
295 def main() -> int:
296     args = parse_args()
297     if args.rounds <= 0:
298         print("[lcqi-ab] --rounds must be positive", file=sys.stderr)
299         return 1
300     if args.max_new_tokens < 0:
301         print("[lcqi-ab] --max-new-tokens cannot be negative", file=sys.stderr)
302         return 1
303     if args.threads < 0:
304         print("[lcqi-ab] --threads cannot be negative", file=sys.stderr)
305         return 1
306     if not args.model_dir.exists():
307         print(f"[lcqi-ab] model directory not found: {args.model_dir}", file=↵
↵ sys.stderr)
308         return 1
309
310     baseline_commit = git_text(["rev-parse", "--short", args.baseline_ref])
311     current_commit = git_text(["rev-parse", "--short", "HEAD"])
312     with tempfile.TemporaryDirectory(prefix="lcqi-gpt2-ab-") as tmp:
313         tmp_path = Path(tmp)
314         baseline_root = tmp_path / "baseline"
315         current_build = tmp_path / "current-build"
316         baseline_build = tmp_path / "baseline-build"
317         ensure_success(
318             run_command(
319                 ↵ ["git", "worktree", "add", "--detach", str(baseline_root), args.↵
↵ .baseline_ref],
320                 cwd=repo_dir(),
321                 timeout_seconds=args.timeout_seconds,
322             ),
323             "create baseline worktree",
324         )
325         try:
326             baseline_binary = build_binary(
327                 baseline_root,
328                 baseline_build,
329                 args.jobs,
330                 args.timeout_seconds,
331             )
332             current_binary = build_binary(
333                 repo_dir(),
334                 current_build,
335                 args.jobs,
336                 args.timeout_seconds,

```

```

337         )
338
339     samples: list[RunSample] = []
340     for round_id in range(1, args.rounds + 1):
341         for variant, binary in [
342             ("baseline", baseline_binary),
343             ("current", current_binary),
344         ]:
345             for engine in ["full", "cached"]:
346                 sample = run_sample(
347                     binary,
348                     variant=variant,
349                     engine=engine,
350                     round_id=round_id,
351                     model_dir=args.model_dir,
352                     prompt=args.prompt,
353                     max_new_tokens=args.max_new_tokens,
354                     threads=1 if variant == "baseline" else args.
↪ threads,
355                     timeout_seconds=args.timeout_seconds,
356                 )
357                 samples.append(sample)
358                 print(
359                     f"round={round_id} variant={variant} engine={engine}
↪ } "
360                     f"generate_ms={sample.metrics['
↪ benchmark_generate_ms']:.3f} "
361                     f"workers={sample.metrics['benchmark_worker_count
↪ ']:.0f} "
362                     f"gen_tps={sample.metrics['
↪ benchmark_generate_tokens_per_second']:.5f}"
363                 )
364             write_report(
365                 args.report,
366                 samples,
367                 baseline_ref=args.baseline_ref,
368                 baseline_commit=baseline_commit,
369                 current_commit=current_commit,
370                 prompt=args.prompt,
371                 max_new_tokens=args.max_new_tokens,
372                 rounds=args.rounds,
373                 current_threads=args.threads,
374             )
375     finally:
376         subprocess.run(
377             ["git", "worktree", "remove", "--force", str(baseline_root)],
378             cwd=repo_dir(),
379             text=True,
380             capture_output=True,
381             check=False,
382         )
383     print(f"report={args.report}")

```

```

384     return 0
385
386
387 if __name__ == "__main__":
388     raise SystemExit(main())

```

`run_gpt2_hotspot_profile.py` 专门收集 `cached decode` 内部热点。它构建 `Release lcqi_gpt2`，用 `--profile-hotspots` 输出 `hotspot_lm_head_pct`、`hotspot_mlp_fc_pct`、`hotspot_mlp_projection_pct`、`hotspot_qkv_projection_pct`、`hotspot_attention_projection_pct`、`hotspot_attention_pct` 和 `benchmark_worker_count`，再按 token 数和轮次计算中位数。这个脚本回答“优化应该打哪里”，不是回答“某次提交快了多少”。本轮线程优化就是靠它发现第一次 `auto` 阈值漏掉了 `attention output projection`，随后把行数和列数阈值一起纳入并行判定。

Listing 16.22 `tools/run_gpt2_hotspot_profile.py`

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import statistics
7  import subprocess
8  from dataclasses import dataclass
9  from pathlib import Path
10
11
12  DEFAULT_PROMPT = "Hello, my name is"
13  DEFAULT_TOKEN_COUNTS = "4,16"
14  DEFAULT_ROUNDS = 3
15  DEFAULT_BUILD_JOBS = 2
16  DEFAULT_TIMEOUT_SECONDS = 900
17
18  METRIC_PREFIXES = ("benchmark_", "hotspot_")
19
20
21  @dataclass(frozen=True)
22  class HotspotSample:
23      max_new_tokens: int
24      round_id: int
25      metrics: dict[str, float]
26      generated_text: str
27
28
29  def script_path() -> Path:
30      return Path(__file__).resolve()
31
32
33  def repo_dir() -> Path:
34      return script_path().parents[3]
35
36

```

```

37 def volume_dir() -> Path:
38     return script_path().parents[1]
39
40
41 def practice_dir() -> Path:
42     return repo_dir() / "books" / "cpu-volume-3-practice"
43
44
45 def default_build_dir() -> Path:
46     return volume_dir() / "build" / "lcqi-hotspot-profile"
47
48
49 def default_model_dir() -> Path:
50     return Path.home() / ".cache" / "lcqi-gpt2-smoke" / "openai-community--gpt2"
51     ↪ "
52
53 def default_report_path() -> Path:
54     return volume_dir() / "results" / "lcqi-gpt2-hotspot-profile.txt"
55
56
57 def run_command(
58     args: list[str],
59     *,
60     cwd: Path,
61     timeout_seconds: int,
62 ) -> subprocess.CompletedProcess[str]:
63     return subprocess.run(
64         args,
65         cwd=cwd,
66         text=True,
67         capture_output=True,
68         timeout=timeout_seconds,
69         check=False,
70     )
71
72
73 def ensure_success(completed: subprocess.CompletedProcess[str], action: str) -> ↪
74     ↪ None:
75     if completed.returncode == 0:
76         return
77     command = (
78         " ".join(completed.args)
79         if isinstance(completed.args, list)
80         else str(completed.args)
81     )
82     raise RuntimeError(
83         f"{action} failed with exit code {completed.returncode}\n"
84         f"command: {command}\n"
85         f"stdout tail:\n{completed.stdout[-2000:]]}\n"
86         f"stderr tail:\n{completed.stderr[-2000:]]}"
87     )

```

```

87
88
89 def build_binary(build_dir: Path, jobs: int, timeout_seconds: int) -> Path:
90     configure = run_command(
91         [
92             "cmake",
93             "-S",
94             str(practice_dir()),
95             "-B",
96             str(build_dir),
97             "-DCMAKE_BUILD_TYPE=Release",
98         ],
99         cwd=repo_dir(),
100         timeout_seconds=timeout_seconds,
101     )
102     ensure_success(configure, "configure hotspot profile build")
103     build = run_command(
104         [
105             "cmake",
106             "--build",
107             str(build_dir),
108             "--target",
109             "lcqi_gpt2",
110             "-j",
111             str(jobs),
112         ],
113         cwd=repo_dir(),
114         timeout_seconds=timeout_seconds,
115     )
116     ensure_success(build, "build lcqi_gpt2")
117     binary = build_dir / "linux_cpu_inference" / "lcqi_gpt2"
118     if not binary.exists():
119         raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
120     return binary
121
122
123 def parse_numeric_list(value: str) -> list[int]:
124     result: list[int] = []
125     for part in value.split(","):
126         stripped = part.strip()
127         if not stripped:
128             continue
129         parsed = int(stripped)
130         if parsed <= 0:
131             raise RuntimeError("--token-counts entries must be positive")
132         result.append(parsed)
133     if not result:
134         raise RuntimeError("--token-counts must contain at least one positive ↵
↵ integer")
135     return result
136
137

```

```

138 def parse_stdout(stdout: str) -> tuple[dict[str, float], str]:
139     metrics: dict[str, float] = {}
140     generated_text = ""
141     for line in stdout.splitlines():
142         key, _, value = line.partition(" ")
143         if key.startswith(METRIC_PREFIXES):
144             metrics[key] = float(value)
145         elif key == "generated_text":
146             generated_text = value
147     required = {
148         "benchmark_generate_ms",
149         "benchmark_worker_count",
150         "hotspot_total_step_ms",
151         "hotspot_lm_head_pct",
152         "hotspot_mlp_fc_pct",
153         "hotspot_mlp_projection_pct",
154         "hotspot_qkv_projection_pct",
155         "hotspot_attention_pct",
156     }
157     missing = sorted(required - metrics.keys())
158     if missing:
159         raise RuntimeError(f"missing hotspot metrics: {missing}")
160     if not generated_text:
161         raise RuntimeError("missing generated_text")
162     return metrics, generated_text
163
164
165 def run_sample(
166     binary: Path,
167     *,
168     model_dir: Path,
169     prompt: str,
170     max_new_tokens: int,
171     round_id: int,
172     threads: int,
173     timeout_seconds: int,
174 ) -> HotspotSample:
175     completed = run_command(
176         [
177             str(binary),
178             "--profile-hotspots",
179             "--engine",
180             "cached",
181             "--threads",
182             str(threads),
183             str(model_dir),
184             prompt,
185             str(max_new_tokens),
186         ],
187         cwd=repo_dir(),
188         timeout_seconds=timeout_seconds,
189     )

```

```

190     ensure_success(completed, f"run hotspot profile tokens={max_new_tokens}")
191     metrics, generated_text = parse_stdout(completed.stdout)
192     return HotspotSample(
193         max_new_tokens=max_new_tokens,
194         round_id=round_id,
195         metrics=metrics,
196         generated_text=generated_text,
197     )
198
199
200 def summarize(samples: list[HotspotSample]) -> list[str]:
201     lines: list[str] = []
202     metric_keys = [
203         "benchmark_load_ms",
204         "benchmark_generate_ms",
205         "benchmark_generate_tokens_per_second",
206         "benchmark_decode_tokens_per_second",
207         "hotspot_total_step_ms",
208         "benchmark_worker_count",
209         "hotspot_lm_head_pct",
210         "hotspot_mlp_fc_pct",
211         "hotspot_mlp_projection_pct",
212         "hotspot_qkv_projection_pct",
213         "hotspot_attention_projection_pct",
214         "hotspot_attention_pct",
215     ]
216     for token_count in sorted({sample.max_new_tokens for sample in samples}):
217         selected = [sample for sample in samples if sample.max_new_tokens == ←
↪ token_count]
218         lines.append(f"[summary max_new_tokens={token_count}]")
219         for key in metric_keys:
220             values = [sample.metrics[key] for sample in selected if key in ←
↪ sample.metrics]
221             if not values:
222                 continue
223             lines.append(
224                 f"{key}=median:{statistics.median(values):.6g},"
225                 f"min:{min(values):.6g},max:{max(values):.6g}"
226             )
227             texts = sorted({sample.generated_text for sample in selected})
228             lines.append(f"generated_text={texts[0] if len(texts) == 1 else texts}" ←
↪ )
229             lines.append("")
230     return lines
231
232
233 def write_report(
234     path: Path,
235     samples: list[HotspotSample],
236     *,
237     model_dir: Path,
238     prompt: str,

```

```

239     token_counts: list[int],
240     rounds: int,
241     threads: int,
242 ) -> None:
243     path.parent.mkdir(parents=True, exist_ok=True)
244     lines = [
245         "LCQI GPT-2 Hotspot Profile",
246         "",
247         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
248         f"model_dir={model_dir}",
249         f"prompt={prompt}",
250         f"token_counts='{','.join(str(value) for value in token_counts)}'",
251         f"rounds={rounds}",
252         f"requested_threads={threads}",
253         "engine=cached",
254         "build=CMAKE_BUILD_TYPE=Release through books/cpu-volume-3-practice",
255         "",
256         "[samples]",
257     ]
258     for sample in samples:
259         metric_text = ",".join(
260             f"{key}:{sample.metrics[key]:.6g}"
261             for key in sorted(sample.metrics)
262             if key.startswith(METRIC_PREFIXES)
263         )
264         lines.append(
265             f"round={sample.round_id},max_new_tokens={sample.max_new_tokens},"
266             f"{metric_text},generated_text={sample.generated_text}"
267         )
268     lines.append("")
269     lines.extend(summarize(samples))
270     while lines and lines[-1] == "":
271         lines.pop()
272     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
273
274
275 def parse_args() -> argparse.Namespace:
276     parser = argparse.ArgumentParser(description="Run LCQI GPT-2 cached hotspot↵
↵ profile.")
277     parser.add_argument("--model-dir", type=Path, default=default_model_dir())
278     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
279     parser.add_argument("--report", type=Path, default=default_report_path())
280     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
281     parser.add_argument("--token-counts", default=DEFAULT_TOKEN_COUNTS)
282     parser.add_argument("--rounds", type=int, default=DEFAULT_ROUNDS)
283     parser.add_argument("--threads", type=int, default=0)
284     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
285     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
286     return parser.parse_args()
287
288

```



```

289 def main() -> int:
290     args = parse_args()
291     if args.rounds <= 0:
292         print("[lcqi-hotspot] --rounds must be positive")
293         return 1
294     if args.threads < 0:
295         print("[lcqi-hotspot] --threads cannot be negative")
296         return 1
297     if not args.model_dir.exists():
298         print(f"[lcqi-hotspot] model directory not found: {args.model_dir}")
299         return 1
300     token_counts = parse_numeric_list(args.token_counts)
301     binary = build_binary(args.build_dir, args.jobs, args.timeout_seconds)
302     samples: List[HotspotSample] = []
303     for round_id in range(1, args.rounds + 1):
304         for token_count in token_counts:
305             sample = run_sample(
306                 binary,
307                 model_dir=args.model_dir,
308                 prompt=args.prompt,
309                 max_new_tokens=token_count,
310                 round_id=round_id,
311                 threads=args.threads,
312                 timeout_seconds=args.timeout_seconds,
313             )
314             samples.append(sample)
315             print(
316                 f"round={round_id} max_new_tokens={token_count} "
317                 f"generate_ms={sample.metrics['benchmark_generate_ms']:.3f} "
318                 f"workers={sample.metrics['benchmark_worker_count']:.0f} "
319                 f"lm_head_pct={sample.metrics['hotspot_lm_head_pct']:.3f} "
320                 f"mlp_pct={sample.metrics['hotspot_mlp_fc_pct']} + sample.↵
↵ metrics['hotspot_mlp_projection_pct']:.3f"
321             )
322     write_report(
323         args.report,
324         samples,
325         model_dir=args.model_dir,
326         prompt=args.prompt,
327         token_counts=token_counts,
328         rounds=args.rounds,
329         threads=args.threads,
330     )
331     print(f"report={args.report}")
332     return 0
333
334
335 if __name__ == "__main__":
336     raise SystemExit(main())

```

`run_lcqi_benchmark.sh` 是本地 benchmark 收集脚本。它适合在固定机器上反复运行，把同一组 shape 和 backend 的结果落到报告文件。

Listing 16.23 tools/run_lcqi_benchmark.sh

```

1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5 VOLUME_DIR="$(cd "${SCRIPT_DIR}/.." && pwd)"
6 REPO_DIR="$(cd "${VOLUME_DIR}/../.." && pwd)"
7 BUILD_DIR="${VOLUME_DIR}/build/lcqi-release"
8 RESULT_DIR="${VOLUME_DIR}/results"
9 REPEAT="${1:-200}"
10 STAMP="$(date -u +%Y%m%dT%H%M%S)"
11 CSV_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.csv"
12 META_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.meta.txt"
13 PERF_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.perf.txt"
14
15 mkdir -p "${RESULT_DIR}"
16
17 cmake -S "${VOLUME_DIR}" -B "${BUILD_DIR}" -DCMAKE_BUILD_TYPE=Release
18 cmake --build "${BUILD_DIR}"
19 ctest --test-dir "${BUILD_DIR}" --output-on-failure
20
21 {
22     echo "timestamp_utc=${STAMP}"
23     echo "repeat=${REPEAT}"
24     echo "commit=$(git -C "${REPO_DIR}" rev-parse --short HEAD)"
25     echo "uname=$(uname -a)"
26     echo "compiler=$((${CXX:-c++} --version | sed -n '1p')"
27     echo "cmake=$(cmake --version | sed -n '1p')"
28     echo "bench=${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench"
29 } > "${META_PATH}"
30
31 "${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" > "${CSV_PATH}"
32
33 if command -v perf >/dev/null 2>&1; then
34     perf stat \
35         -e cycles,instructions,cache-misses,branches,branch-misses \
36         "${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" \
37         > /dev/null 2> "${PERF_PATH}" || true
38 else
39     echo "perf_unavailable=1" > "${PERF_PATH}"
40 fi
41
42 echo "csv=${CSV_PATH}"
43 echo "meta=${META_PATH}"
44 echo "perf=${PERF_PATH}"

```

完整测试

lcqi_tests.cpp 把 safetensors、tokenizer、tiny MLP、int8 kernel、reference decoder、GPT-2 tiny path、trace、ledger 和 serving probe 串成验收网。GPT-2 优化不是只看速度：测试会把 full-prefix logits、旧 cached logits、

优化 decoder logits、强制两 worker 的并行 decoder logits 和 logits-free greedy token 放在同一个 tiny prompt 下对齐，确认 predicted token、logits、KV filled token 数和 generated ids 不漂移。新增优化或 shape 时，必须先扩展这里的 golden 和边界样例。

Listing 16.24 tests/lcqi_tests.cpp

```

1 #include <lcqi/gpt2_reference.hpp>
2 #include <lcqi/f32_kernels.hpp>
3 #include <lcqi/ggml_matvec.hpp>
4 #include <lcqi/ggml_tensors.hpp>
5 #include <lcqi/gguf.hpp>
6 #include <lcqi/inference.hpp>
7 #include <lcqi/int8_kernels.hpp>
8 #include <lcqi/llama_gguf.hpp>
9 #include <lcqi/model.hpp>
10 #include <lcqi/q4_k.hpp>
11 #include <lcqi/q5_0_packed.hpp>
12 #include <lcqi/reference_decoder.hpp>
13 #include <lcqi/safetensors.hpp>
14 #include <lcqi/tokenizer.hpp>
15
16 #include <array>
17 #include <cmath>
18 #include <cstring>
19 #include <cstdlib>
20 #include <filesystem>
21 #include <fstream>
22 #include <iostream>
23 #include <span>
24 #include <sstream>
25 #include <stdexcept>
26 #include <string>
27 #include <utility>
28 #include <vector>
29
30 namespace {
31
32 constexpr float LCQI_TEST_EPSILON = 1.0e-5F;
33 constexpr std::int32_t LCQI_TEST_INPUT_SIZE = 4;
34 constexpr std::int32_t LCQI_TEST_HIDDEN_SIZE = 3;
35 constexpr std::int32_t LCQI_TEST_OUTPUT_SIZE = 2;
36 constexpr std::int32_t LCQI_TEST_PREDICTED_CLASS = 1;
37 constexpr float LCQI_TEST_LOGIT_0 = -0.48F;
38 constexpr float LCQI_TEST_LOGIT_1 = 1.06F;
39 constexpr float LCQI_TEST_HIDDEN_0 = 0.0F;
40 constexpr float LCQI_TEST_HIDDEN_1 = 0.4F;
41 constexpr float LCQI_TEST_HIDDEN_2 = 1.4F;
42 constexpr float LCQI_RMS_EXPECTED_0 = 0.848528F;
43 constexpr float LCQI_RMS_EXPECTED_1 = 0.565685F;
44 constexpr float LCQI_ATTENTION_EXPECTED_0 = 2.0F;
45 constexpr float LCQI_ATTENTION_EXPECTED_1 = 3.0F;
46 constexpr float LCQI_KERNEL_MAX_DIFF = 1.0e-5F;

```

```

47 constexpr float LCQI_F32_KERNEL_MAX_DIFF = 1.0e-4F;
48 constexpr std::int32_t LCQI_SIMD_TEST_BLOCK = 8;
49 constexpr std::size_t LCQI_F32_DOT_TEST_SIZE = 257;
50 constexpr std::size_t LCQI_F32_ROW_TEST_INPUT_SIZE = 65;
51 constexpr std::size_t LCQI_F32_ROW_TEST_OUTPUT_SIZE = 17;
52 constexpr std::size_t LCQI_F32_BATCH_TEST_SIZE = 5;
53 constexpr std::int32_t LCQI_REFERENCE_DECODER_PREDICTED_TOKEN = 0;
54 constexpr std::size_t LCQI_REFERENCE_DECODER_VOCAB_SIZE = 5;
55 constexpr std::array<float, LCQI_REFERENCE_DECODER_VOCAB_SIZE> ↵
    ↵ LCQI_REFERENCE_DECODER_LOGITS{
56     0.352516979F,
57     -0.126884326F,
58     0.0797837451F,
59     -0.29797405F,
60     0.127030417F,
61 };
62 constexpr std::int32_t LCQI_REFERENCE_TRACE_TOKEN_COUNT = 3;
63 constexpr std::int32_t LCQI_REFERENCE_TRACE_HIDDEN_SIZE = 4;
64 constexpr std::int32_t LCQI_REFERENCE_TRACE_FIRST_TOKEN = 0;
65 constexpr std::uint64_t LCQI_TOKENIZER_HASH_EXPECTED = 13452902845388333734ULL;
66 constexpr std::int32_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
67 constexpr std::int32_t LCQI_TEST_BYTE_BITS = 8;
68 constexpr std::uint64_t LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES = 48;
69 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_BEGIN = 0;
70 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_END = 24;
71 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_BEGIN = 24;
72 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_END = 48;
73 constexpr unsigned int LCQI_BYTE_MASK = 0xFFU;
74 constexpr std::int32_t LCQI_GPT2_PREDICTED_TOKEN = 3;
75 constexpr std::size_t LCQI_GPT2_VOCAB_SIZE = 6;
76 constexpr std::array<float, LCQI_GPT2_VOCAB_SIZE> LCQI_GPT2_LOGITS{
77     0.407358F,
78     -0.504049F,
79     -0.107834F,
80     0.840337F,
81     -0.450123F,
82     -0.156763F,
83 };
84 constexpr std::int32_t LCQI_GPT2_GENERATED_LENGTH = 5;
85 constexpr std::int32_t LCQI_GPT2_TOKENIZER_HELLO_ID = 7;
86 constexpr std::uint16_t LCQI_TEST_F16_ONE = 0x3C00U;
87 constexpr std::uint16_t LCQI_TEST_F16_HALF = 0x3800U;
88 constexpr std::uint16_t LCQI_TEST_F16_NEGATIVE_TWO = 0xC000U;
89 constexpr std::uint16_t LCQI_TEST_BF16_ONE_POINT_FIVE = 0x3FC0U;
90 constexpr std::uint16_t LCQI_TEST_BF16_NEGATIVE_HALF = 0xBF00U;
91 constexpr std::uint32_t LCQI_GGUF_TEST_VERSION = 3;
92 constexpr std::uint32_t LCQI_GGUF_TEST_ALIGNMENT = 32;
93 constexpr std::int64_t LCQI_GGUF_TEST_TENSOR_COUNT = 1;
94 constexpr std::int64_t LCQI_GGUF_TEST_METADATA_COUNT = 3;
95 constexpr std::int32_t LCQI_GGUF_TYPE_UINT32 = 4;
96 constexpr std::int32_t LCQI_GGUF_TYPE_FLOAT32 = 6;
97 constexpr std::int32_t LCQI_GGUF_TYPE_STRING = 8;

```

```

98 constexpr std::int32_t LCQI_GGUF_TYPE_ARRAY = 9;
99 constexpr std::int32_t LCQI_GGUF_TENSOR_TYPE_Q4_K = 12;
100 constexpr std::uint64_t LCQI_GGUF_TEST_TENSOR_OFFSET = 0;
101 constexpr float LCQI_Q4K_TEST_BLOCK_SCALE = 0.5F;
102 constexpr float LCQI_Q4K_TEST_BLOCK_MIN = 0.25F;
103 constexpr float LCQI_Q4K_Q8_DIFF_LIMIT = 0.08F;
104 constexpr float LCQI_GGML_MATVEC_MAX_DIFF = 1.0e-4F;
105 constexpr float LCQI_GGML_Q8_MATVEC_MAX_DIFF = 0.2F;
106 constexpr std::int32_t LCQI_GGML_TEST_Q5_HALF_VALUES = 16;
107 constexpr std::int32_t LCQI_GGML_TEST_Q5_ZERO_POINT = 16;
108 constexpr std::int32_t LCQI_GGML_TEST_Q6_LANE = 5;
109 constexpr std::int32_t LCQI_LLAMA_TEST_HIDDEN = 4;
110 constexpr std::int32_t LCQI_LLAMA_TEST_HEADS = 2;
111 constexpr std::int32_t LCQI_LLAMA_TEST_KV_HEADS = 1;
112 constexpr std::int32_t LCQI_LLAMA_TEST_HEAD_DIM = 2;
113 constexpr std::int32_t LCQI_LLAMA_TEST_INTERMEDIATE = 4;
114 constexpr std::int32_t LCQI_LLAMA_TEST_VOCAB = 4;
115 constexpr std::int32_t LCQI_LLAMA_TEST_CONTEXT = 8;
116 constexpr std::int32_t LCQI_LLAMA_TEST_LAYER_COUNT = 1;
117 constexpr std::int32_t LCQI_LLAMA_TEST_EXPECTED_TOKEN = 2;
118 constexpr float LCQI_LLAMA_TEST_SMALL_ATTENTION_VALUE = 0.1F;
119 constexpr float LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE = 0.1F;
120 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HIDDEN = 256;
121 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HEADS = 1;
122 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_KV_HEADS = 1;
123 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HEAD_DIM = 256;
124 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_INTERMEDIATE = 256;
125 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_VOCAB = 2;
126 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_CONTEXT = 4;
127 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_LAYER_COUNT = 1;
128 constexpr const char* LCQI_TEST_LLAMA_BATCH_PREFILL_ENV = "↵
↵ LCQI_LLAMA_BATCH_PREFILL";
129 constexpr const char* LCQI_TEST_LLAMA_GGML_DIRECT_ENV = "LCQI_LLAMA_GGML_DIRECT↵
↵ ";
130 constexpr const char* LCQI_TEST_LLAMA_GGML_SELECTIVE_DIRECT_ENV =
131     "LCQI_LLAMA_GGML_SELECTIVE_DIRECT";
132 constexpr const char* LCQI_TEST_LLAMA_Q5_0_PACKED_ENV = "LCQI_LLAMA_Q5_0_PACKED↵
↵ ";
133 constexpr const char* LCQI_TEST_LLAMA_Q6_K_DIRECT_ENV = "LCQI_LLAMA_Q6_K_DIRECT↵
↵ ";
134 constexpr const char* LCQI_TEST_LLAMA_Q8_0_DIRECT_ENV = "LCQI_LLAMA_Q8_0_DIRECT↵
↵ ";
135 constexpr const char* LCQI_TEST_FALSE_ENV = "0";
136 constexpr const char* LCQI_TEST_TRUE_ENV = "1";
137
138 struct KernelShapeCase {
139     std::int32_t input_size = 0;
140     std::int32_t output_size = 0;
141     std::int32_t output_block_size = 0;
142 };
143
144 struct RawTensorFixture {

```

```

145     std::string name;
146     std::string dtype;
147     std::vector<std::int64_t> shape;
148     std::vector<std::uint8_t> bytes;
149 };
150
151 struct FloatTensorFixture {
152     std::string name;
153     std::vector<std::int64_t> shape;
154     std::vector<float> values;
155 };
156
157 struct GgufTensorFixture {
158     std::string name;
159     std::vector<std::int64_t> shape;
160     lcqi::GgmlType type = lcqi::GgmlType::f32;
161     std::vector<std::uint8_t> bytes;
162     std::uint64_t relative_offset = 0;
163 };
164
165 struct ScopedEnvVar {
166     std::string name;
167     std::string old_value;
168     bool had_old_value = false;
169
170     ScopedEnvVar(const char* env_name, const char* value) : name(env_name) {
171         const char* old = std::getenv(env_name);
172         if (old != nullptr) {
173             this->old_value = old;
174             this->had_old_value = true;
175         }
176         if (setenv(env_name, value, 1) != 0) {
177             throw std::runtime_error("failed to set test environment variable");
178         }
179     }
180
181     ScopedEnvVar(const ScopedEnvVar&) = delete;
182     ScopedEnvVar& operator=(const ScopedEnvVar&) = delete;
183
184     ~ScopedEnvVar() {
185         if (this->had_old_value) {
186             static_cast<void>(setenv(this->name.c_str(), this->old_value.c_str(),
187             ↪ (1), 1));
188         } else {
189             static_cast<void>(unsetenv(this->name.c_str()));
190         }
191     };
192
193     constexpr std::array<KernelShapeCase, 4> LCQI_KERNEL_SHAPE_CASES{{
194         {17, 19, 8},

```

```

195     {33, 7, 8},
196     {64, 32, 16},
197     {65, 31, 16},
198     });
199
200 void require(bool condition, const char* message) {
201     if (!condition) {
202         throw std::runtime_error(message);
203     }
204 }
205
206 void require_close(float actual, float expected, const char* message) {
207     if (std::fabs(actual - expected) > LCQI_TEST_EPSILON) {
208         throw std::runtime_error(message);
209     }
210 }
211
212 std::filesystem::path test_model_path() {
213     return std::filesystem::path(__FILE__).parent_path().parent_path() /
214         "models" / "tiny_mlp_i8.txt";
215 }
216
217 std::filesystem::path test_tokenizer_path() {
218     return std::filesystem::path(__FILE__).parent_path().parent_path() /
219         "models" / "tiny_tokenizer.txt";
220 }
221
222 std::filesystem::path temp_file_path(const std::string& name) {
223     return std::filesystem::temp_directory_path() / name;
224 }
225
226 void write_le_u64(std::ofstream& output, std::uint64_t value) {
227     for (std::int32_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
228         const char byte = static_cast<char>(
229             (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK);
230         output.write(&byte, 1);
231     }
232 }
233
234 void append_le_u16(std::vector<std::uint8_t>& output, std::uint16_t value) {
235     output.push_back(static_cast<std::uint8_t>(value & LCQI_BYTE_MASK));
236     output.push_back(static_cast<std::uint8_t>((value >> LCQI_TEST_BYTE_BITS) &
↵ LCQI_BYTE_MASK));
237 }
238
239 void append_le_u32(std::vector<std::uint8_t>& output, std::uint32_t value) {
240     for (std::int32_t i = 0; i < 4; ++i) {
241         output.push_back(static_cast<std::uint8_t>(
242             (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
243     }
244 }
245

```



```

246 void append_le_i32(std::vector<std::uint8_t>& output, std::int32_t value) {
247     std::uint32_t bits = 0;
248     std::memcpy(&bits, &value, sizeof(bits));
249     append_le_u32(output, bits);
250 }
251
252 void append_le_u64(std::vector<std::uint8_t>& output, std::uint64_t value) {
253     for (std::int32_t i = 0; i < 8; ++i) {
254         output.push_back(static_cast<std::uint8_t>(
255             (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
256     }
257 }
258
259 void append_le_i64(std::vector<std::uint8_t>& output, std::int64_t value) {
260     std::uint64_t bits = 0;
261     std::memcpy(&bits, &value, sizeof(bits));
262     append_le_u64(output, bits);
263 }
264
265 void append_gguf_string(std::vector<std::uint8_t>& output, const std::string& ↵
    ↵ text) {
266     append_le_u64(output, static_cast<std::uint64_t>(text.size()));
267     output.insert(output.end(), text.begin(), text.end());
268 }
269
270 void append_le_f32(std::vector<std::uint8_t>& output, float value) {
271     std::uint32_t bits = 0;
272     std::memcpy(&bits, &value, sizeof(bits));
273     for (std::int32_t i = 0; i < 4; ++i) {
274         output.push_back(static_cast<std::uint8_t>(
275             (bits >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
276     }
277 }
278
279 void write_safetensors_fixture(const std::filesystem::path& path,
280                               const std::string& header,
281                               std::uint64_t payload_bytes) {
282     std::ofstream output(path, std::ios::binary);
283     if (!output) {
284         throw std::runtime_error("cannot create safetensors test fixture");
285     }
286     write_le_u64(output, static_cast<std::uint64_t>(header.size()));
287     output.write(header.data(), static_cast<std::streamsize>(header.size()));
288     for (std::uint64_t i = 0; i < payload_bytes; ++i) {
289         const char byte = static_cast<char>(i & 0xFFU);
290         output.write(&byte, 1);
291     }
292 }
293
294 std::string json_shape(std::span<const std::int64_t> shape) {
295     std::ostringstream stream;
296     stream << '[';

```



```

297     for (std::size_t index = 0; index < shape.size(); ++index) {
298         if (index != 0) {
299             stream << ',';
300         }
301         stream << shape[index];
302     }
303     stream << ']';
304     return stream.str();
305 }
306
307 void write_safetensors_raw_fixture(const std::filesystem::path& path,
308                                   std::span<const RawTensorFixture> tensors) {
309     std::ostringstream header;
310     header << "{\"__metadata__\":{\"format\":\"lcqi-test\"}";
311     std::uint64_t offset = 0;
312     for (const RawTensorFixture& tensor : tensors) {
313         header << ",\"\" << tensor.name << "\":{\"dtype\":\"\" << tensor.dtype
314             << "\",\"shape\":\"\" << json_shape(tensor.shape)
315             << "\",\"data_offsets\":[\" << offset << ', '
316             << offset + tensor.bytes.size() << "]}\"";
317         offset += tensor.bytes.size();
318     }
319     header << '}'';
320
321     std::ofstream output(path, std::ios::binary);
322     if (!output) {
323         throw std::runtime_error("cannot create safetensors raw test fixture");
324     }
325     const std::string header_text = header.str();
326     write_le_u64(output, static_cast<std::uint64_t>(header_text.size()));
327     output.write(header_text.data(), static_cast<std::streamsize>(header_text.↵
↵ size()));
328     for (const RawTensorFixture& tensor : tensors) {
329         output.write(reinterpret_cast<const char*>(tensor.bytes.data()),
330                     static_cast<std::streamsize>(tensor.bytes.size()));
331     }
332 }
333
334 std::vector<std::uint8_t> f32_payload(std::span<const float> values) {
335     std::vector<std::uint8_t> bytes;
336     bytes.reserve(values.size() * 4);
337     for (const float value : values) {
338         append_le_f32(bytes, value);
339     }
340     return bytes;
341 }
342
343 std::vector<float> transpose_linear_to_hf_conv1d(const lcqi::Gpt2LinearF32& ↵
↵ linear) {
344     std::vector<float> transposed(
345         static_cast<std::size_t>(linear.input_size) *
346         static_cast<std::size_t>(linear.output_size),

```

```

347     0.0F);
348     for (std::int32_t out = 0; out < linear.output_size; ++out) {
349         for (std::int32_t in = 0; in < linear.input_size; ++in) {
350             transposed[static_cast<std::size_t>(in) *
351                         static_cast<std::size_t>(linear.output_size) +
352                         static_cast<std::size_t>(out)] =
353                 linear.weights[static_cast<std::size_t>(out) *
354                                static_cast<std::size_t>(linear.input_size) ←
355                                ↪ +
356                                static_cast<std::size_t>(in)];
357         }
358     }
359     return transposed;
360 }
361 void add_f32_tensor(std::vector<RawTensorFixture>& tensors,
362                    std::string name,
363                    std::vector<std::int64_t> shape,
364                    std::span<const float> values) {
365     tensors.push_back(RawTensorFixture{
366         std::move(name),
367         "F32",
368         std::move(shape),
369         f32_payload(values),
370     });
371 }
372
373 void write_text_file(const std::filesystem::path& path, const std::string& text ←
374 ↪ ) {
375     std::ofstream output(path);
376     if (!output) {
377         throw std::runtime_error("cannot create text fixture");
378     }
379     output << text;
380 }
381
382 std::vector<std::uint8_t> make_q4_k_test_block() {
383     std::vector<std::uint8_t> block(
384         static_cast<std::size_t>(lcqi::LCQI_Q4_K_BLOCK_BYTES),
385         0);
386     block[0] = 0x00U;
387     block[1] = 0x38U;
388     block[2] = 0x00U;
389     block[3] = 0x34U;
390
391     std::array<std::uint8_t, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS ←
392 ↪ )> scales{
393         1, 2, 3, 4, 5, 6, 7, 8,
394     };
395     std::array<std::uint8_t, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS ←
396 ↪ )> mins{
397         1, 1, 2, 2, 3, 3, 4, 4,

```

```

395     };
396     for (std::size_t index = 0; index < 4; ++index) {
397         block[4 + index] = static_cast<std::uint8_t>(
398             (scales[index] & 0x3FU) | ((scales[index + 4] >> 4U) << 6U));
399         block[8 + index] = static_cast<std::uint8_t>(
400             (mins[index] & 0x3FU) | ((mins[index + 4] >> 4U) << 6U));
401         block[12 + index] = static_cast<std::uint8_t>(
402             (scales[index + 4] & 0x0FU) | ((mins[index + 4] & 0x0FU) << 4U));
403     }
404
405     for (std::int32_t pair = 0; pair < 4; ++pair) {
406         const std::int32_t low_subblock = pair * 2;
407         const std::int32_t high_subblock = low_subblock + 1;
408         for (std::int32_t index = 0; index < lcqi::LCQI_Q4_K_SUBBLOCK_VALUES; ↵
↵ ++index) {
409             const std::uint8_t low = static_cast<std::uint8_t>(
410                 (low_subblock + index) & 0x0F);
411             const std::uint8_t high = static_cast<std::uint8_t>(
412                 (high_subblock + index + 1) & 0x0F);
413             block[static_cast<std::size_t>(16 + pair * 32 + index)] =
414                 static_cast<std::uint8_t>(low | (high << 4U));
415         }
416     }
417     return block;
418 }
419
420 std::vector<float> expected_q4_k_test_values() {
421     const std::array<float, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS)↵
↵ > scales{
422         1, 2, 3, 4, 5, 6, 7, 8,
423     };
424     const std::array<float, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS)↵
↵ > mins{
425         1, 1, 2, 2, 3, 3, 4, 4,
426     };
427     std::vector<float> values(
428         static_cast<std::size_t>(lcqi::LCQI_QK_K_BLOCK_VALUES),
429         0.0F);
430     for (std::int32_t subblock = 0; subblock < lcqi::LCQI_Q4_K_SUBBLOCKS; ++↵
↵ subblock) {
431         for (std::int32_t index = 0; index < lcqi::LCQI_Q4_K_SUBBLOCK_VALUES; ↵
↵ ++index) {
432             const std::int32_t quantized =
433                 (subblock % 2 == 0)
434                     ? ((subblock + index) & 0x0F)
435                     : ((subblock + index + 1) & 0x0F);
436             values[static_cast<std::size_t>(
437                 subblock * lcqi::LCQI_Q4_K_SUBBLOCK_VALUES + index)] =
438                 LCQI_Q4K_TEST_BLOCK_SCALE * scales[static_cast<std::size_t>(↵
↵ subblock)] *
439                 static_cast<float>(quantized) -
440                 LCQI_Q4K_TEST_BLOCK_MIN * mins[static_cast<std::size_t>(↵

```

```

    ↪ subblock)];
441     }
442 }
443 return values;
444 }
445
446 std::vector<std::uint8_t> make_q8_0_test_block() {
447     std::vector<std::uint8_t> block(
448         static_cast<std::size_t>(lcqi::LCQI_Q8_0_BLOCK_BYTES),
449         0);
450     block[0] = static_cast<std::uint8_t>(LCQI_TEST_F16_HALF & LCQI_BYTE_MASK);
451     block[1] = static_cast<std::uint8_t>((LCQI_TEST_F16_HALF >> ↪
    ↪ LCQI_TEST_BYTE_BITS) &
452                                         LCQI_BYTE_MASK);
453     for (std::int32_t index = 0; index < lcqi::LCQI_Q8_0_BLOCK_VALUES; ++index)↪
    ↪ {
454         block[static_cast<std::size_t>(2 + index)] =
455             static_cast<std::uint8_t>(static_cast<std::int8_t>(index - 16));
456     }
457     return block;
458 }
459
460 std::vector<std::uint8_t> make_q5_0_test_block() {
461     std::vector<std::uint8_t> block(
462         static_cast<std::size_t>(lcqi::LCQI_Q5_0_BLOCK_BYTES),
463         0);
464     block[0] = static_cast<std::uint8_t>(LCQI_TEST_F16_HALF & LCQI_BYTE_MASK);
465     block[1] = static_cast<std::uint8_t>((LCQI_TEST_F16_HALF >> ↪
    ↪ LCQI_TEST_BYTE_BITS) &
466                                         LCQI_BYTE_MASK);
467
468     std::uint32_t qh = 0;
469     for (std::int32_t index = 0; index < lcqi::LCQI_Q5_0_BLOCK_VALUES / 2; ++↪
    ↪ index) {
470         const std::int32_t first_quantized = index - ↪
    ↪ LCQI_GGML_TEST_Q5_ZERO_POINT;
471         const std::int32_t second_quantized = index;
472         const std::uint8_t first_encoded =
473             static_cast<std::uint8_t>(first_quantized + ↪
    ↪ LCQI_GGML_TEST_Q5_ZERO_POINT);
474         const std::uint8_t second_encoded =
475             static_cast<std::uint8_t>(second_quantized + ↪
    ↪ LCQI_GGML_TEST_Q5_ZERO_POINT);
476         if ((first_encoded & 0x10U) != 0U) {
477             qh |= (1U << index);
478         }
479         if ((second_encoded & 0x10U) != 0U) {
480             qh |= (1U << (index + LCQI_GGML_TEST_Q5_HALF_VALUES));
481         }
482         const std::uint8_t low = static_cast<std::uint8_t>(first_encoded & 0↪
    ↪ x0FU);
483         const std::uint8_t high = static_cast<std::uint8_t>(second_encoded & 0↪

```

```

    ↪ x0FU);
484     block[static_cast<std::size_t>(6 + index)] =
485         static_cast<std::uint8_t>(low | (high << 4U));
486 }
487 block[2] = static_cast<std::uint8_t>(qh & LCQI_BYTE_MASK);
488 block[3] = static_cast<std::uint8_t>((qh >> LCQI_TEST_BYTE_BITS) & ↪
    ↪ LCQI_BYTE_MASK);
489 block[4] = static_cast<std::uint8_t>((qh >> (LCQI_TEST_BYTE_BITS * 2)) & ↪
    ↪ LCQI_BYTE_MASK);
490 block[5] = static_cast<std::uint8_t>((qh >> (LCQI_TEST_BYTE_BITS * 3)) & ↪
    ↪ LCQI_BYTE_MASK);
491 return block;
492 }
493
494 std::vector<std::uint8_t> make_q6_k_test_block() {
495     std::vector<std::uint8_t> block(
496         static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES),
497         0);
498     const std::size_t ql_offset = 0;
499     const std::size_t qh_offset = static_cast<std::size_t>(lcqi::↪
    ↪ LCQI_Q6_K_BLOCK_VALUES / 2);
500     const std::size_t scales_offset =
501         qh_offset + static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_VALUES / 4);
502     for (std::int32_t index = 0; index < 16; ++index) {
503         block[scales_offset + static_cast<std::size_t>(index)] = 1;
504     }
505     block[static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES - 2)] =
506         static_cast<std::uint8_t>(LCQI_TEST_F16_ONE & LCQI_BYTE_MASK);
507     block[static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES - 1)] =
508         static_cast<std::uint8_t>((LCQI_TEST_F16_ONE >> LCQI_TEST_BYTE_BITS) &
509             LCQI_BYTE_MASK);
510
511     const auto store_q6 = [&block, ql_offset, qh_offset](std::int32_t position,
512                                                         std::uint8_t encoded) {
513         const std::int32_t half = position / 128;
514         const std::int32_t local = position % 128;
515         const std::size_t ql_base = ql_offset + static_cast<std::size_t>(half *↪
    ↪ 64);
516         const std::size_t qh_base = qh_offset + static_cast<std::size_t>(half *↪
    ↪ 32);
517         const std::uint8_t low = encoded & 0x0FU;
518         const std::uint8_t high = static_cast<std::uint8_t>((encoded >> 4U) & 0↪
    ↪ x03U);
519         if (local < 32) {
520             block[ql_base + static_cast<std::size_t>(local)] =
521                 static_cast<std::uint8_t>(
522                     (block[ql_base + static_cast<std::size_t>(local)] & 0xF0U) ↪
    ↪ | low);
523             block[qh_base + static_cast<std::size_t>(local)] =
524                 static_cast<std::uint8_t>(
525                     (block[qh_base + static_cast<std::size_t>(local)] & 0xFCU) ↪
    ↪ | high);

```

```

526     } else if (local < 64) {
527         const std::size_t lane = static_cast<std::size_t>(local - 32);
528         block[ql_base + static_cast<std::size_t>(32) + lane] =
529             static_cast<std::uint8_t>(
530                 (block[ql_base + static_cast<std::size_t>(32) + lane] & 0x0FU) | low);
531         block[qh_base + lane] =
532             static_cast<std::uint8_t>((block[qh_base + lane] & 0xF3U) | (high << 2U));
533     } else if (local < 96) {
534         const std::size_t lane = static_cast<std::size_t>(local - 64);
535         block[ql_base + lane] =
536             static_cast<std::uint8_t>(
537                 (block[ql_base + lane] & 0x0FU) | (low << 4U));
538         block[qh_base + lane] =
539             static_cast<std::uint8_t>((block[qh_base + lane] & 0xCFU) | (high << 4U));
540     } else {
541         const std::size_t lane = static_cast<std::size_t>(local - 96);
542         block[ql_base + static_cast<std::size_t>(32) + lane] =
543             static_cast<std::uint8_t>(
544                 (block[ql_base + static_cast<std::size_t>(32) + lane] & 0x0FU) |
545                 (low << 4U));
546         block[qh_base + lane] =
547             static_cast<std::uint8_t>((block[qh_base + lane] & 0x3FU) | (high << 6U));
548     }
549 };
550
551 store_q6(LCQI_GGML_TEST_Q6_LANE, 35);
552 store_q6(32 + LCQI_GGML_TEST_Q6_LANE, 31);
553 store_q6(64 + LCQI_GGML_TEST_Q6_LANE, 63);
554 store_q6(96 + LCQI_GGML_TEST_Q6_LANE, 0);
555 store_q6(128 + LCQI_GGML_TEST_Q6_LANE, 36);
556 return block;
557 }
558
559 void write_gguf_q4_k_fixture(const std::filesystem::path& path) {
560     std::vector<std::uint8_t> bytes;
561     bytes.push_back('G');
562     bytes.push_back('G');
563     bytes.push_back('U');
564     bytes.push_back('F');
565     append_le_u32(bytes, LCQI_GGUF_TEST_VERSION);
566     append_le_i64(bytes, LCQI_GGUF_TEST_TENSOR_COUNT);
567     append_le_i64(bytes, LCQI_GGUF_TEST_METADATA_COUNT);
568
569     append_gguf_string(bytes, "general.alignment");
570     append_le_i32(bytes, LCQI_GGUF_TYPE_UINT32);
571     append_le_u32(bytes, LCQI_GGUF_TEST_ALIGNMENT);
572     append_gguf_string(bytes, "general.architecture");

```

```

573     append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
574     append_gguf_string(bytes, "lcqi-test");
575     append_gguf_string(bytes, "tokenizer.ggml.tokens");
576     append_le_i32(bytes, LCQI_GGUF_TYPE_ARRAY);
577     append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
578     append_le_u64(bytes, 2);
579     append_gguf_string(bytes, "<bos>");
580     append_gguf_string(bytes, "hello");
581
582     append_gguf_string(bytes, "blk.0.ffn_up.weight");
583     append_le_u32(bytes, 2);
584     append_le_i64(bytes, lcqi::LCQI_QK_K_BLOCK_VALUES);
585     append_le_i64(bytes, 1);
586     append_le_i32(bytes, LCQI_GGUF_TENSOR_TYPE_Q4_K);
587     append_le_u64(bytes, LCQI_GGUF_TEST_TENSOR_OFFSET);
588     while (bytes.size() % LCQI_GGUF_TEST_ALIGNMENT != 0) {
589         bytes.push_back(0);
590     }
591     const std::vector<std::uint8_t> block = make_q4_k_test_block();
592     bytes.insert(bytes.end(), block.begin(), block.end());
593
594     std::ofstream output(path, std::ios::binary);
595     if (!output) {
596         throw std::runtime_error("cannot create GGUF fixture");
597     }
598     output.write(reinterpret_cast<const char*>(bytes.data()),
599                static_cast<std::streamsize>(bytes.size()));
600 }
601
602 void append_gguf_uint32_metadata(std::vector<std::uint8_t>& bytes,
603                                const std::string& key,
604                                std::uint32_t value) {
605     append_gguf_string(bytes, key);
606     append_le_i32(bytes, LCQI_GGUF_TYPE_UINT32);
607     append_le_u32(bytes, value);
608 }
609
610 void append_gguf_float32_metadata(std::vector<std::uint8_t>& bytes,
611                                  const std::string& key,
612                                  float value) {
613     append_gguf_string(bytes, key);
614     append_le_i32(bytes, LCQI_GGUF_TYPE_FLOAT32);
615     append_le_f32(bytes, value);
616 }
617
618 void append_gguf_string_metadata(std::vector<std::uint8_t>& bytes,
619                                  const std::string& key,
620                                  const std::string& value) {
621     append_gguf_string(bytes, key);
622     append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
623     append_gguf_string(bytes, value);
624 }

```



```

625
626 void write_gguf_fixture(const std::filesystem::path& path,
627                         std::span<GgufTensorFixture> tensors,
628                         const std::vector<std::uint8_t>& metadata_bytes) {
629     std::uint64_t relative_offset = 0;
630     for (GgufTensorFixture& tensor : tensors) {
631         tensor.relative_offset = relative_offset;
632         relative_offset += tensor.bytes.size();
633         while (relative_offset % LCQI_GGUF_TEST_ALIGNMENT != 0) {
634             ++relative_offset;
635         }
636     }
637
638     std::vector<std::uint8_t> bytes;
639     bytes.push_back('G');
640     bytes.push_back('G');
641     bytes.push_back('U');
642     bytes.push_back('F');
643     append_le_u32(bytes, LCQI_GGUF_TEST_VERSION);
644     append_le_u64(bytes, static_cast<std::uint64_t>(tensors.size()));
645     append_le_u64(bytes, 14);
646     bytes.insert(bytes.end(), metadata_bytes.begin(), metadata_bytes.end());
647
648     for (const GgufTensorFixture& tensor : tensors) {
649         append_gguf_string(bytes, tensor.name);
650         append_le_u32(bytes, static_cast<std::uint32_t>(tensor.shape.size()));
651         for (const std::int64_t extent : tensor.shape) {
652             append_le_i64(bytes, extent);
653         }
654         append_le_i32(bytes, static_cast<std::int32_t>(tensor.type));
655         append_le_u64(bytes, tensor.relative_offset);
656     }
657     while (bytes.size() % LCQI_GGUF_TEST_ALIGNMENT != 0) {
658         bytes.push_back(0);
659     }
660     std::uint64_t written_relative = 0;
661     for (const GgufTensorFixture& tensor : tensors) {
662         while (written_relative < tensor.relative_offset) {
663             bytes.push_back(0);
664             ++written_relative;
665         }
666         bytes.insert(bytes.end(), tensor.bytes.begin(), tensor.bytes.end());
667         written_relative += tensor.bytes.size();
668     }
669
670     std::ofstream output(path, std::ios::binary);
671     if (!output) {
672         throw std::runtime_error("cannot create GGUF fixture");
673     }
674     output.write(reinterpret_cast<const char*>(bytes.data()),
675                 static_cast<std::streamsize>(bytes.size()));
676 }

```



```

677
678 void add_gguf_f32_tensor(std::vector<GgufTensorFixture>& tensors,
679                          std::string name,
680                          std::vector<std::int64_t> shape,
681                          std::span<const float> values) {
682     tensors.push_back(GgufTensorFixture{
683         std::move(name),
684         std::move(shape),
685         lcqi::GgmlType::f32,
686         f32_payload(values),
687         0,
688     });
689 }
690
691 void add_gguf_q4_k_tensor(std::vector<GgufTensorFixture>& tensors,
692                          std::string name,
693                          std::vector<std::int64_t> shape,
694                          const std::vector<std::uint8_t>& bytes) {
695     tensors.push_back(GgufTensorFixture{
696         std::move(name),
697         std::move(shape),
698         lcqi::GgmlType::q4_k,
699         bytes,
700         0,
701     });
702 }
703
704 void add_gguf_quantized_tensor(std::vector<GgufTensorFixture>& tensors,
705                                std::string name,
706                                std::vector<std::int64_t> shape,
707                                lcqi::GgmlType type,
708                                const std::vector<std::uint8_t>& bytes) {
709     tensors.push_back(GgufTensorFixture{
710         std::move(name),
711         std::move(shape),
712         type,
713         bytes,
714         0,
715     });
716 }
717
718 std::vector<std::uint8_t> repeated_ggml_block(const std::vector<std::uint8_t>& ←
    ↪ block,
719                                              std::int32_t input_size,
720                                              std::int32_t output_size,
721                                              std::int64_t block_values) {
722     const std::size_t block_count =
723         static_cast<std::size_t>((input_size / block_values) * output_size);
724     std::vector<std::uint8_t> bytes;
725     bytes.reserve(block.size() * block_count);
726     for (std::size_t index = 0; index < block_count; ++index) {
727         bytes.insert(bytes.end(), block.begin(), block.end());

```

```

728     }
729     return bytes;
730 }
731
732 void write_tiny_llama_gguf_fixture(const std::filesystem::path& path) {
733     std::vector<std::uint8_t> metadata;
734     append_gguf_uint32_metadata(metadata, "general.alignment", ←
    ↪ LCQI_GGUF_TEST_ALIGNMENT);
735     append_gguf_string_metadata(metadata, "general.architecture", "llama");
736     append_gguf_string_metadata(metadata, "general.name", "lcqi tiny llama");
737     append_gguf_uint32_metadata(metadata, "llama.embedding_length", ←
    ↪ LCQI_LLAMA_TEST_HIDDEN);
738     append_gguf_uint32_metadata(metadata, "llama.block_count", ←
    ↪ LCQI_LLAMA_TEST_LAYER_COUNT);
739     append_gguf_uint32_metadata(metadata, "llama.feed_forward_length", ←
    ↪ LCQI_LLAMA_TEST_INTERMEDIATE);
740     append_gguf_uint32_metadata(metadata, "llama.attention.head_count", ←
    ↪ LCQI_LLAMA_TEST_HEADS);
741     append_gguf_uint32_metadata(metadata, "llama.attention.head_count_kv", ←
    ↪ LCQI_LLAMA_TEST_KV_HEADS);
742     append_gguf_uint32_metadata(metadata, "llama.rope.dimension_count", ←
    ↪ LCQI_LLAMA_TEST_HEAD_DIM);
743     append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
744     append_gguf_uint32_metadata(metadata, "llama.context_length", ←
    ↪ LCQI_LLAMA_TEST_CONTEXT);
745     append_gguf_uint32_metadata(metadata, "llama.vocab_size", ←
    ↪ LCQI_LLAMA_TEST_VOCAB);
746     append_gguf_float32_metadata(metadata, "llama.attention.←
    ↪ layer_norm_rms_epsilon", 1.0e-5F);
747     append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 3);
748
749     std::vector<GgufTensorFixture> tensors;
750     const std::vector<float> embedding{
751         1.0F, 0.0F, 0.0F, 0.0F,
752         0.0F, 1.0F, 0.0F, 0.0F,
753         0.0F, 2.0F, 0.0F, 0.0F,
754         0.0F, 0.0F, 1.0F, 0.0F,
755     };
756     add_gguf_f32_tensor(tensors,
757         "token_embd.weight",
758         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_VOCAB},
759         embedding);
760     add_gguf_f32_tensor(tensors,
761         "output_norm.weight",
762         {LCQI_LLAMA_TEST_HIDDEN},
763         std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
764     add_gguf_f32_tensor(tensors,
765         "blk.0.attn_norm.weight",
766         {LCQI_LLAMA_TEST_HIDDEN},
767         std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
768     add_gguf_f32_tensor(tensors,
769         "blk.0.ffn_norm.weight",

```

```

770         {LCQI_LLAMA_TEST_HIDDEN},
771         std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
772     const std::vector<float> hidden_to_hidden(
773         static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * ←
↪ LCQI_LLAMA_TEST_HIDDEN),
774         0.0F);
775     std::vector<float> hidden_to_kv(
776         static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * ←
↪ LCQI_LLAMA_TEST_HEAD_DIM),
777         0.0F);
778     hidden_to_kv[1] = LCQI_LLAMA_TEST_SMALL_ATTENTION_VALUE;
779     std::vector<float> attention_output = hidden_to_hidden;
780     attention_output[0] = LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE;
781     attention_output[10] = LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE;
782     const std::vector<float> hidden_to_intermediate(
783         static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * ←
↪ LCQI_LLAMA_TEST_INTERMEDIATE),
784         0.0F);
785     const std::vector<float> intermediate_to_hidden(
786         static_cast<std::size_t>(LCQI_LLAMA_TEST_INTERMEDIATE * ←
↪ LCQI_LLAMA_TEST_HIDDEN),
787         0.0F);
788     add_gguf_f32_tensor(tensors,
789         "blk.0.attn.q.weight",
790         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HIDDEN},
791         hidden_to_hidden);
792     add_gguf_f32_tensor(tensors,
793         "blk.0.attn.k.weight",
794         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HEAD_DIM},
795         hidden_to_kv);
796     add_gguf_f32_tensor(tensors,
797         "blk.0.attn.v.weight",
798         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HEAD_DIM},
799         hidden_to_kv);
800     add_gguf_f32_tensor(tensors,
801         "blk.0.attn_output.weight",
802         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HIDDEN},
803         attention_output);
804     add_gguf_f32_tensor(tensors,
805         "blk.0.ffn_gate.weight",
806         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_INTERMEDIATE},
807         hidden_to_intermediate);
808     add_gguf_f32_tensor(tensors,
809         "blk.0.ffn_up.weight",
810         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_INTERMEDIATE},
811         hidden_to_intermediate);
812     add_gguf_f32_tensor(tensors,
813         "blk.0.ffn_down.weight",
814         {LCQI_LLAMA_TEST_INTERMEDIATE, LCQI_LLAMA_TEST_HIDDEN},
815         intermediate_to_hidden);
816     write_gguf_fixture(path, tensors, metadata);
817 }

```

```

818
819 void write_tiny_llama_ggml_direct_gguf_fixture(const std::filesystem::path& ↵
    ↵ path) {
820     std::vector<std::uint8_t> metadata;
821     append_gguf_uint32_metadata(metadata, "general.alignment", ↵
    ↵ LCQI_GGUF_TEST_ALIGNMENT);
822     append_gguf_string_metadata(metadata, "general.architecture", "llama");
823     append_gguf_string_metadata(metadata, "general.name", "lcqi ggml direct ↵
    ↵ tiny llama");
824     append_gguf_uint32_metadata(metadata, "llama.embedding_length", ↵
    ↵ LCQI_LLAMA_Q4_TEST_HIDDEN);
825     append_gguf_uint32_metadata(metadata, "llama.block_count", ↵
    ↵ LCQI_LLAMA_Q4_TEST_LAYER_COUNT);
826     append_gguf_uint32_metadata(metadata,
827                                 "llama.feed_forward_length",
828                                 LCQI_LLAMA_Q4_TEST_INTERMEDIATE);
829     append_gguf_uint32_metadata(metadata, "llama.attention.head_count", ↵
    ↵ LCQI_LLAMA_Q4_TEST_HEADS);
830     append_gguf_uint32_metadata(metadata,
831                                 "llama.attention.head_count_kv",
832                                 LCQI_LLAMA_Q4_TEST_KV_HEADS);
833     append_gguf_uint32_metadata(metadata,
834                                 "llama.rope.dimension_count",
835                                 LCQI_LLAMA_Q4_TEST_HEAD_DIM);
836     append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
837     append_gguf_uint32_metadata(metadata, "llama.context_length", ↵
    ↵ LCQI_LLAMA_Q4_TEST_CONTEXT);
838     append_gguf_uint32_metadata(metadata, "llama.vocab_size", ↵
    ↵ LCQI_LLAMA_Q4_TEST_VOCAB);
839     append_gguf_float32_metadata(metadata, "llama.attention.↵
    ↵ layer_norm_rms_epsilon", 1.0e-5F);
840     append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 1);
841
842     std::vector<GgufTensorFixture> tensors;
843     std::vector<float> embedding(
844         static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN * ↵
    ↵ LCQI_LLAMA_Q4_TEST_VOCAB),
845         0.0F);
846     embedding[0] = 1.0F;
847     embedding[static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN)] = 0.5F;
848     add_gguf_f32_tensor(tensors,
849                         "token_embd.weight",
850                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_VOCAB},
851                         embedding);
852     add_gguf_f32_tensor(tensors,
853                         "output_norm.weight",
854                         {LCQI_LLAMA_Q4_TEST_HIDDEN},
855                         std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
856     add_gguf_f32_tensor(tensors,
857                         "blk.0.attn_norm.weight",
858                         {LCQI_LLAMA_Q4_TEST_HIDDEN},
859                         std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));

```

```

860 add_gguf_f32_tensor(tensors,
861                     "blk.0.ffn_norm.weight",
862                     {LCQI_LLAMA_Q4_TEST_HIDDEN},
863                     std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
864
865 const std::vector<std::uint8_t> q5_matrix = repeated_ggml_block(
866     std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::←
↪ LCQI_Q5_0_BLOCK_BYTES), 0),
867     LCQI_LLAMA_Q4_TEST_HIDDEN,
868     LCQI_LLAMA_Q4_TEST_HIDDEN,
869     lcqi::LCQI_Q5_0_BLOCK_VALUES);
870 const std::vector<std::uint8_t> q8_matrix = repeated_ggml_block(
871     std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::←
↪ LCQI_Q8_0_BLOCK_BYTES), 0),
872     LCQI_LLAMA_Q4_TEST_HIDDEN,
873     LCQI_LLAMA_Q4_TEST_HIDDEN,
874     lcqi::LCQI_Q8_0_BLOCK_VALUES);
875 const std::vector<std::uint8_t> q6_matrix = repeated_ggml_block(
876     std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::←
↪ LCQI_Q6_K_BLOCK_BYTES), 0),
877     LCQI_LLAMA_Q4_TEST_INTERMEDIATE,
878     LCQI_LLAMA_Q4_TEST_HIDDEN,
879     lcqi::LCQI_Q6_K_BLOCK_VALUES);
880
881 add_gguf_quantized_tensor(tensors,
882                           "blk.0.attn_q.weight",
883                           {LCQI_LLAMA_Q4_TEST_HIDDEN, ←
↪ LCQI_LLAMA_Q4_TEST_HIDDEN},
884                           lcqi::GgmlType::q5_0,
885                           q5_matrix);
886 add_gguf_quantized_tensor(tensors,
887                           "blk.0.attn_k.weight",
888                           {LCQI_LLAMA_Q4_TEST_HIDDEN, ←
↪ LCQI_LLAMA_Q4_TEST_HEAD_DIM},
889                           lcqi::GgmlType::q5_0,
890                           q5_matrix);
891 add_gguf_quantized_tensor(tensors,
892                           "blk.0.attn_v.weight",
893                           {LCQI_LLAMA_Q4_TEST_HIDDEN, ←
↪ LCQI_LLAMA_Q4_TEST_HEAD_DIM},
894                           lcqi::GgmlType::q8_0,
895                           q8_matrix);
896 add_gguf_quantized_tensor(tensors,
897                           "blk.0.attn_output.weight",
898                           {LCQI_LLAMA_Q4_TEST_HIDDEN, ←
↪ LCQI_LLAMA_Q4_TEST_HIDDEN},
899                           lcqi::GgmlType::q5_0,
900                           q5_matrix);
901 add_gguf_quantized_tensor(tensors,
902                           "blk.0.ffn_gate.weight",
903                           {LCQI_LLAMA_Q4_TEST_HIDDEN, ←
↪ LCQI_LLAMA_Q4_TEST_INTERMEDIATE},

```

```

904         lcqi::GgmlType::q5_0,
905         q5_matrix);
906     add_gguf_quantized_tensor(tensors,
907         "blk.0.ffn_up.weight",
908         {LCQI_LLAMA_Q4_TEST_HIDDEN, ←
↪ LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
909         lcqi::GgmlType::q8_0,
910         q8_matrix);
911     add_gguf_quantized_tensor(tensors,
912         "blk.0.ffn_down.weight",
913         {LCQI_LLAMA_Q4_TEST_INTERMEDIATE, ←
↪ LCQI_LLAMA_Q4_TEST_HIDDEN},
914         lcqi::GgmlType::q6_k,
915         q6_matrix);
916     write_gguf_fixture(path, tensors, metadata);
917 }
918
919 void write_tiny_llama_q4_direct_gguf_fixture(const std::filesystem::path& path)↪
↪ {
920     std::vector<std::uint8_t> metadata;
921     append_gguf_uint32_metadata(metadata, "general.alignment", ←
↪ LCQI_GGUF_TEST_ALIGNMENT);
922     append_gguf_string_metadata(metadata, "general.architecture", "llama");
923     append_gguf_string_metadata(metadata, "general.name", "lcqi q4 direct tiny ←
↪ llama");
924     append_gguf_uint32_metadata(metadata, "llama.embedding_length", ←
↪ LCQI_LLAMA_Q4_TEST_HIDDEN);
925     append_gguf_uint32_metadata(metadata, "llama.block_count", ←
↪ LCQI_LLAMA_Q4_TEST_LAYER_COUNT);
926     append_gguf_uint32_metadata(metadata, "llama.feed_forward_length", ←
↪ LCQI_LLAMA_Q4_TEST_INTERMEDIATE);
927     append_gguf_uint32_metadata(metadata, "llama.attention.head_count", ←
↪ LCQI_LLAMA_Q4_TEST_HEADS);
928     append_gguf_uint32_metadata(metadata, "llama.attention.head_count_kv", ←
↪ LCQI_LLAMA_Q4_TEST_KV_HEADS);
929     append_gguf_uint32_metadata(metadata, "llama.rope.dimension_count", ←
↪ LCQI_LLAMA_Q4_TEST_HEAD_DIM);
930     append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
931     append_gguf_uint32_metadata(metadata, "llama.context_length", ←
↪ LCQI_LLAMA_Q4_TEST_CONTEXT);
932     append_gguf_uint32_metadata(metadata, "llama.vocab_size", ←
↪ LCQI_LLAMA_Q4_TEST_VOCAB);
933     append_gguf_float32_metadata(metadata, "llama.attention.↪
↪ layer_norm_rms_epsilon", 1.0e-5F);
934     append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 1);
935
936     std::vector<GgufTensorFixture> tensors;
937     std::vector<float> embedding(
938         static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN * ←
↪ LCQI_LLAMA_Q4_TEST_VOCAB),
939         0.0F);
940     embedding[0] = 1.0F;

```

```

941 embedding[static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN)] = 0.5F;
942 add_gguf_f32_tensor(tensors,
943     "token_embd.weight",
944     {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_VOCAB},
945     embedding);
946 add_gguf_f32_tensor(tensors,
947     "output_norm.weight",
948     {LCQI_LLAMA_Q4_TEST_HIDDEN},
949     std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
950 add_gguf_f32_tensor(tensors,
951     "blk.0.attn_norm.weight",
952     {LCQI_LLAMA_Q4_TEST_HIDDEN},
953     std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
954 add_gguf_f32_tensor(tensors,
955     "blk.0.ffn_norm.weight",
956     {LCQI_LLAMA_Q4_TEST_HIDDEN},
957     std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
958
959 const std::vector<std::uint8_t> zero_q4_matrix(
960     static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN) *
961     static_cast<std::size_t>(lcqi::LCQI_Q4_K_BLOCK_BYTES),
962     0);
963 add_gguf_q4_k_tensor(tensors,
964     "blk.0.attn_q.weight",
965     {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HIDDEN},
966     zero_q4_matrix);
967 add_gguf_q4_k_tensor(tensors,
968     "blk.0.attn_k.weight",
969     {LCQI_LLAMA_Q4_TEST_HIDDEN,
970     LCQI_LLAMA_Q4_TEST_HEAD_DIM},
971     zero_q4_matrix);
972 add_gguf_q4_k_tensor(tensors,
973     "blk.0.attn_v.weight",
974     {LCQI_LLAMA_Q4_TEST_HIDDEN,
975     LCQI_LLAMA_Q4_TEST_HEAD_DIM},
976     zero_q4_matrix);
977 add_gguf_q4_k_tensor(tensors,
978     "blk.0.attn_output.weight",
979     {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HIDDEN},
980     zero_q4_matrix);
981 add_gguf_q4_k_tensor(tensors,
982     "blk.0.ffn_gate.weight",
983     {LCQI_LLAMA_Q4_TEST_HIDDEN,
984     LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
985     zero_q4_matrix);
986 add_gguf_q4_k_tensor(tensors,
987     "blk.0.ffn_up.weight",
988     {LCQI_LLAMA_Q4_TEST_HIDDEN,
989     LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
990     zero_q4_matrix);

```



```

987     add_gguf_q4_k_tensor(tensors,
988                          "blk.0.ffn_down.weight",
989                          {LCQI_LLAMA_Q4_TEST_INTERMEDIATE, ←
↪ LCQI_LLAMA_Q4_TEST_HIDDEN},
990                          zero_q4_matrix);
991     write_gguf_fixture(path, tensors, metadata);
992 }
993
994 void write_tiny_gpt2_safetensors(const std::filesystem::path& path,
995                                 const lcqi::Gpt2ReferenceModel& model) {
996     std::vector<RawTensorFixture> tensors;
997     add_f32_tensor(tensors,
998                   "transformer.wte.weight",
999                   {model.config.vocab_size, model.config.hidden_size},
1000                   model.token_embedding);
1001     add_f32_tensor(tensors,
1002                   "transformer.wpe.weight",
1003                   {model.config.max_positions, model.config.hidden_size},
1004                   model.position_embedding);
1005     for (std::int32_t layer_id = 0; layer_id < model.config.layer_count; ++↪
↪ layer_id) {
1006         const lcqi::Gpt2LayerWeightsF32& layer =
1007             model.layers[static_cast<std::size_t>(layer_id)];
1008         const std::string prefix = "transformer.h." + std::to_string(layer_id) ←
↪ + ".";
1009         add_f32_tensor(tensors, prefix + "ln_1.weight", {model.config.↪
↪ hidden_size}, layer.ln_1_weight);
1010         add_f32_tensor(tensors, prefix + "ln_1.bias", {model.config.hidden_size↪
↪ }, layer.ln_1_bias);
1011         const std::vector<float> c_attn_hf = transpose_linear_to_hf_conv1d(↪
↪ layer.c_attn);
1012         add_f32_tensor(tensors,
1013                       prefix + "attn.c_attn.weight",
1014                       {model.config.hidden_size,
1015                       model.config.hidden_size * 3},
1016                       c_attn_hf);
1017         add_f32_tensor(tensors,
1018                       prefix + "attn.c_attn.bias",
1019                       {model.config.hidden_size * 3},
1020                       layer.c_attn.bias);
1021         const std::vector<float> c_proj_hf = transpose_linear_to_hf_conv1d(↪
↪ layer.c_proj);
1022         add_f32_tensor(tensors,
1023                       prefix + "attn.c_proj.weight",
1024                       {model.config.hidden_size, model.config.hidden_size},
1025                       c_proj_hf);
1026         add_f32_tensor(tensors,
1027                       prefix + "attn.c_proj.bias",
1028                       {model.config.hidden_size},
1029                       layer.c_proj.bias);
1030         add_f32_tensor(tensors, prefix + "ln_2.weight", {model.config.↪
↪ hidden_size}, layer.ln_2_weight);

```



```

1031     add_f32_tensor(tensors, prefix + "ln_2.bias", {model.config.hidden_size,
↪ }, layer.ln_2.bias);
1032     const std::vector<float> c_fc_hf = transpose_linear_to_hf_conv1d(layer.c_fc,
↪ c_fc);
1033     add_f32_tensor(tensors,
1034                     prefix + "mlp.c_fc.weight",
1035                     {model.config.hidden_size, model.config.intermediate_size},
↪ c_fc_hf);
1036     add_f32_tensor(tensors,
1037                     prefix + "mlp.c_fc.bias",
1038                     {model.config.intermediate_size},
1039                     layer.c_fc.bias);
1040     const std::vector<float> mlp_proj_hf =
1041         transpose_linear_to_hf_conv1d(layer.mlp_c_proj);
1042     add_f32_tensor(tensors,
1043                     prefix + "mlp.c_proj.weight",
1044                     {model.config.intermediate_size, model.config.hidden_size},
↪ mlp_proj_hf);
1045     add_f32_tensor(tensors,
1046                     prefix + "mlp.c_proj.bias",
1047                     {model.config.hidden_size},
1048                     layer.mlp_c_proj.bias);
1049 }
1050 add_f32_tensor(tensors,
1051                 "transformer.ln_f.weight",
1052                 {model.config.hidden_size},
1053                 model.final_ln_weight);
1054 add_f32_tensor(tensors,
1055                 "transformer.ln_f.bias",
1056                 {model.config.hidden_size},
1057                 model.final_ln_bias);
1058 write_safetensors_raw_fixture(path, tensors);
1059 }
1060
1061 void test_tiny_mlp() {
1062     const lcqi::TinyMlpModel model = lcqi::load_model(test_model_path());
1063     require(model.input_size == LCQI_TEST_INPUT_SIZE, "input size mismatch");
1064     require(model.hidden_size == LCQI_TEST_HIDDEN_SIZE, "hidden size mismatch")↪
↪ ;
1065     require(model.output_size == LCQI_TEST_OUTPUT_SIZE, "output size mismatch")↪
↪ ;
1066
1067     const std::vector<float> input{1.0F, 2.0F, -1.0F, 0.5F};
1068     const lcqi::InferenceResult result = lcqi::run_inference(model, input);
1069
1070     require(result.predicted_class == LCQI_TEST_PREDICTED_CLASS, "unexpected ↪
↪ predicted class");
1071     require(result.logits.size() == static_cast<std::size_t>(↪
↪ LCQI_TEST_OUTPUT_SIZE),
1072             "unexpected logits size");

```

```

1075     require_close(result.logits[0], LCQI_TEST_LOGIT_0, "logit 0 mismatch");
1076     require_close(result.logits[1], LCQI_TEST_LOGIT_1, "logit 1 mismatch");
1077
1078     std::vector<float> hidden(static_cast<std::size_t>(LCQI_TEST_HIDDEN_SIZE), ←
    ↪ 0.0F);
1079     lcqi::linear_i8(model.hidden, input, hidden);
1080     lcqi::relu_inplace(hidden);
1081     require_close(hidden[0], LCQI_TEST_HIDDEN_0, "hidden 0 mismatch");
1082     require_close(hidden[1], LCQI_TEST_HIDDEN_1, "hidden 1 mismatch");
1083     require_close(hidden[2], LCQI_TEST_HIDDEN_2, "hidden 2 mismatch");
1084 }
1085
1086 void test_tokenizer_contract() {
1087     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(←
    ↪ test_tokenizer_path());
1088     require(tokenizer.bos_id == 0, "tokenizer BOS id mismatch");
1089     require(tokenizer.eos_id == 4, "tokenizer EOS id mismatch");
1090     require(tokenizer.unk_id == -1, "tokenizer UNK id mismatch");
1091     require(tokenizer.chat_template_hash == "tiny-chat-template-v1",
1092             "tokenizer template hash mismatch");
1093
1094     const std::vector<std::int32_t> token_ids =
1095         lcqi::encode_prompt(tokenizer, "tok1 tok2 tok3");
1096     const std::vector<std::int32_t> expected{0, 1, 2, 3, 4};
1097     require(token_ids == expected, "tokenizer prompt ids mismatch");
1098     require(lcqi::tokenizer_contract_hash(tokenizer) == ←
    ↪ LCQI_TOKENIZER_HASH_EXPECTED,
1099             "tokenizer contract hash mismatch");
1100 }
1101
1102 void test_tokenizer_rejects_unknown_without_unk() {
1103     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(←
    ↪ test_tokenizer_path());
1104     bool threw = false;
1105     try {
1106         static_cast<void>(lcqi::encode_prompt(tokenizer, "tok1 missing_token"))←
    ↪ ;
1107     } catch (const std::runtime_error&) {
1108         threw = true;
1109     }
1110     require(threw, "tokenizer should reject unknown token when unk id is absent←
    ↪ ");
1111 }
1112
1113 void test_safetensors_manifest_contract() {
1114     const std::filesystem::path path =
1115         temp_file_path("lcqi_safetensors_manifest_contract.safetensors");
1116     const std::string header =
1117         R"({"__metadata__":{"format":"lcqi-fixture"},"model.embed_tokens.weight←
    ↪ {"dtype":"F32","shape":[2,3],"data_offsets":[0,24]},"model.layers.0.←
    ↪ attn.q_proj.weight":{"dtype":"F16","shape":[4,3],"data_offsets"←
    ↪ :[24,48]}})";

```

```

1118 write_safetensors_fixture(path, header, ←
    ↪ LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES);
1119
1120 const lcqi::SafeTensorManifest manifest = lcqi::load_safetensors_manifest(←
    ↪ path);
1121 std::filesystem::remove(path);
1122
1123 require(manifest.header_size == static_cast<std::uint64_t>(header.size()),
1124         "safetensors header size mismatch");
1125 require(manifest.data_start_offset ==
1126         static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) +
1127         static_cast<std::uint64_t>(header.size()),
1128         "safetensors data start mismatch");
1129 require(manifest.metadata.size() == 1, "safetensors metadata count mismatch"←
    ↪ ");
1130 require(manifest.metadata[0].first == "format" &&
1131         manifest.metadata[0].second == "lcqi-fixture",
1132         "safetensors metadata mismatch");
1133 require(manifest.tensors.size() == 2, "safetensors tensor count mismatch");
1134
1135 const lcqi::SafeTensorEntry* embedding =
1136     manifest.find_tensor("model.embed_tokens.weight");
1137 require(embedding != nullptr, "safetensors embedding tensor missing");
1138 require(embedding->dtype == "F32", "safetensors embedding dtype mismatch");
1139 require(embedding->shape == std::vector<std::int64_t>({2, 3}),
1140         "safetensors embedding shape mismatch");
1141 require(embedding->data_begin == LCQI_SAFETENSORS_EMBED_BEGIN &&
1142         embedding->data_end == LCQI_SAFETENSORS_EMBED_END,
1143         "safetensors embedding offset mismatch");
1144 require(embedding->byte_size() ==
1145         LCQI_SAFETENSORS_EMBED_END - LCQI_SAFETENSORS_EMBED_BEGIN,
1146         "safetensors embedding byte size mismatch");
1147
1148 const lcqi::SafeTensorEntry* q_proj =
1149     manifest.find_tensor("model.layers.0.attn.q_proj.weight");
1150 require(q_proj != nullptr, "safetensors q projection tensor missing");
1151 require(q_proj->dtype == "F16", "safetensors q projection dtype mismatch");
1152 require(q_proj->data_begin == LCQI_SAFETENSORS_QPROJ_BEGIN &&
1153         q_proj->data_end == LCQI_SAFETENSORS_QPROJ_END,
1154         "safetensors q projection offset mismatch");
1155 }
1156
1157 void test_safetensors_rejects_bad_offsets() {
1158     const std::filesystem::path path =
1159         temp_file_path("lcqi_safetensors_bad_offsets.safetensors");
1160     const std::string header =
1161         R("{\"tensor\":{\"dtype\":\"F32\",\"shape\":[4],\"data_offsets\":[0,32]}}");
1162     write_safetensors_fixture(path, header, 4);
1163
1164     bool threw = false;
1165     try {
1166         static_cast<void>(lcqi::load_safetensors_manifest(path));

```

```

1167     } catch (const std::runtime_error&) {
1168         threw = true;
1169     }
1170     std::filesystem::remove(path);
1171     require(threw, "safetensors parser should reject offsets beyond payload");
1172 }
1173
1174 void test_safetensors_rejects_bad_dtype_shape_size() {
1175     const std::filesystem::path path =
1176         temp_file_path("lcqi_safetensors_bad_dtype_shape_size.safetensors");
1177     const std::string header =
1178         R("{\"tensor\":{\"dtype\":\"F32\",\"shape\":[4],\"data_offsets\":[0,8]}}");
1179     write_safetensors_fixture(path, header, 8);
1180
1181     bool threw = false;
1182     try {
1183         static_cast<void>(lcqi::load_safetensors_manifest(path));
1184     } catch (const std::runtime_error&) {
1185         threw = true;
1186     }
1187     std::filesystem::remove(path);
1188     require(threw, "safetensors parser should reject dtype/shape byte mismatch"↵
↵ );
1189 }
1190
1191 void test_safetensors_reads_float_tensors() {
1192     const std::filesystem::path path =
1193         temp_file_path("lcqi_safetensors_float_read.safetensors");
1194     std::vector<std::uint8_t> f16_bytes;
1195     append_le_u16(f16_bytes, LCQI_TEST_F16_ONE);
1196     append_le_u16(f16_bytes, LCQI_TEST_F16_NEGATIVE_TWO);
1197     std::vector<std::uint8_t> bf16_bytes;
1198     append_le_u16(bf16_bytes, LCQI_TEST_BF16_ONE_POINT_FIVE);
1199     append_le_u16(bf16_bytes, LCQI_TEST_BF16_NEGATIVE_HALF);
1200
1201     const std::array<RawTensorFixture, 3> tensors{{
1202         {"F32", "F32", {2}, f32_payload(std::array<float, 2>{1.25F, -2.5F})},
1203         {"f16", "F16", {2}, f16_bytes},
1204         {"bf16", "BF16", {2}, bf16_bytes},
1205     }};
1206     write_safetensors_raw_fixture(path, tensors);
1207
1208     const lcqi::SafeTensorFile file = lcqi::load_safetensors_file(path);
1209     std::filesystem::remove(path);
1210     const std::vector<float> f32 = lcqi::read_safetensor_f32_tensor(file, "f32"↵
↵ );
1211     const std::vector<float> f16 = lcqi::read_safetensor_f32_tensor(file, "f16"↵
↵ );
1212     const std::vector<float> bf16 = lcqi::read_safetensor_f32_tensor(file, "↵
↵ bf16");
1213
1214     require_close(f32[0], 1.25F, "safetensors F32 value 0 mismatch");

```

```

1215     require_close(f32[1], -2.5F, "safetensors F32 value 1 mismatch");
1216     require_close(f16[0], 1.0F, "safetensors F16 value 0 mismatch");
1217     require_close(f16[1], -2.0F, "safetensors F16 value 1 mismatch");
1218     require_close(bf16[0], 1.5F, "safetensors BF16 value 0 mismatch");
1219     require_close(bf16[1], -0.5F, "safetensors BF16 value 1 mismatch");
1220 }
1221
1222 void test_gguf_manifest_reads_q4_k_tensor() {
1223     const std::filesystem::path path = temp_file_path("lcqi_q4_k_fixture.gguf")↵
↵ ;
1224     write_gguf_q4_k_fixture(path);
1225
1226     const lcqi::GgufManifest manifest = lcqi::load_gguf_manifest(path);
1227     const lcqi::GgufTensorInfo* tensor =
1228         manifest.find_tensor("blk.0.ffn_up.weight");
1229     require(tensor != nullptr, "GGUF Q4_K tensor missing");
1230     const std::vector<std::uint8_t> bytes =
1231         lcqi::read_gguf_tensor_bytes(path, *tensor);
1232     std::filesystem::remove(path);
1233
1234     require(manifest.version == LCQI_GGUF_TEST_VERSION, "GGUF version mismatch"↵
↵ );
1235     require(manifest.alignment == LCQI_GGUF_TEST_ALIGNMENT, "GGUF alignment ↵
↵ mismatch");
1236     require(manifest.metadata.size() == LCQI_GGUF_TEST_METADATA_COUNT,
1237         "GGUF metadata count mismatch");
1238     const lcqi::GgufMetadataEntry* tokens = manifest.find_metadata("tokenizer.↵
↵ ggml.tokens");
1239     require(tokens != nullptr, "GGUF tokenizer tokens metadata missing");
1240     require(tokens->string_values == std::vector<std::string>({"<bos>", "hello"↵
↵ })),
1241         "GGUF tokenizer string array mismatch");
1242     require(manifest.tensors.size() == LCQI_GGUF_TEST_TENSOR_COUNT,
1243         "GGUF tensor count mismatch");
1244     require(tensor->type == lcqi::GgmlType::q4_k, "GGUF tensor type mismatch");
1245     require(tensor->shape == std::vector<std::int64_t>({lcqi::↵
↵ LCQI_QK_K_BLOCK_VALUES, 1})),
1246         "GGUF tensor shape mismatch");
1247     require(tensor->byte_size == lcqi::LCQI_Q4_K_BLOCK_BYTES,
1248         "GGUF tensor byte size mismatch");
1249     require(bytes == make_q4_k_test_block(), "GGUF tensor payload mismatch");
1250 }
1251
1252 void test_q4_k_dequant_and_dot_paths() {
1253     const std::vector<std::uint8_t> block = make_q4_k_test_block();
1254     const std::vector<float> expected = expected_q4_k_test_values();
1255     const std::vector<float> decoded =
1256         lcqi::dequantize_q4_k(block, lcqi::LCQI_QK_K_BLOCK_VALUES);
1257
1258     require(decoded.size() == expected.size(), "Q4_K decoded size mismatch");
1259     for (std::size_t index = 0; index < expected.size(); ++index) {
1260         require_close(decoded[index], expected[index], "Q4_K decoded value ↵

```

```

    ↪ mismatch");
1261 }
1262
1263 std::vector<float> input(expected.size(), 0.0F);
1264 for (std::size_t index = 0; index < input.size(); ++index) {
1265     input[index] = static_cast<float>(static_cast<std::int32_t>(index % 11U
    ↪ ) - 5) *
1266         0.125F;
1267 }
1268 float expected_dot = 0.0F;
1269 for (std::size_t index = 0; index < expected.size(); ++index) {
1270     expected_dot += expected[index] * input[index];
1271 }
1272 const float q4_f32_dot = lcqi::dot_q4_k_f32(block, input);
1273 require_close(q4_f32_dot, expected_dot, "Q4_K F32 dot mismatch");
1274
1275 const std::vector<lcqi::Q8KBlock> q8_input = lcqi::quantize_q8_k_input(
    ↪ input);
1276 const float q4_q8_dot = lcqi::dot_q4_k_q8(block, q8_input);
1277 require(std::fabs(q4_q8_dot - q4_f32_dot) <= LCQI_Q4K_Q8_DIFF_LIMIT,
1278     "Q4_K Q8 dot drift is too large");
1279
1280 std::vector<float> matvec_output(1, 0.0F);
1281 lcqi::matvec_q4_k_q8(block, 1, lcqi::LCQI_QK_K_BLOCK_VALUES, q8_input,
    ↪ matvec_output);
1282 require_close(matvec_output[0], q4_q8_dot, "Q4_K Q8 matvec mismatch");
1283 }
1284
1285 void test_ggml_mixed_dequantizers() {
1286     std::vector<std::uint8_t> f32_bytes;
1287     append_le_f32(f32_bytes, 1.25F);
1288     append_le_f32(f32_bytes, -2.5F);
1289     const std::vector<float> f32 =
1290         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::f32, f32_bytes, 2);
1291     require_close(f32[0], 1.25F, "GGML F32 decoded value 0 mismatch");
1292     require_close(f32[1], -2.5F, "GGML F32 decoded value 1 mismatch");
1293
1294     const std::vector<std::uint8_t> q8 = make_q8_0_test_block();
1295     const std::vector<float> q8_values =
1296         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q8_0,
1297             q8,
1298             lcqi::LCQI_Q8_0_BLOCK_VALUES);
1299     require_close(q8_values[0], -8.0F, "Q8_0 decoded value 0 mismatch");
1300     require_close(q8_values[16], 0.0F, "Q8_0 decoded value 16 mismatch");
1301     require_close(q8_values[31], 7.5F, "Q8_0 decoded value 31 mismatch");
1302
1303     const std::vector<std::uint8_t> q5 = make_q5_0_test_block();
1304     const std::vector<float> q5_values =
1305         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q5_0,
1306             q5,
1307             lcqi::LCQI_Q5_0_BLOCK_VALUES);
1308     require_close(q5_values[0], -8.0F, "Q5_0 decoded value 0 mismatch");

```



```

1309     require_close(q5_values[1], -7.5F, "Q5_0 decoded value 1 mismatch");
1310     require_close(q5_values[15], -0.5F, "Q5_0 decoded value 15 mismatch");
1311     require_close(q5_values[16], 0.0F, "Q5_0 decoded value 16 mismatch");
1312     require_close(q5_values[31], 7.5F, "Q5_0 decoded value 31 mismatch");
1313
1314     const std::vector<std::uint8_t> q6 = make_q6_k_test_block();
1315     const std::vector<float> q6_values =
1316         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q6_k,
1317                                     q6,
1318                                     lcqi::LCQI_Q6_K_BLOCK_VALUES);
1319     require_close(q6_values[LCQI_GGML_TEST_Q6_LANE], 3.0F, "Q6_K q1 mismatch");
1320     require_close(q6_values[32 + LCQI_GGML_TEST_Q6_LANE], -1.0F, "Q6_K q2 ↵
↵ mismatch");
1321     require_close(q6_values[64 + LCQI_GGML_TEST_Q6_LANE], 31.0F, "Q6_K q3 ↵
↵ mismatch");
1322     require_close(q6_values[96 + LCQI_GGML_TEST_Q6_LANE], -32.0F, "Q6_K q4 ↵
↵ mismatch");
1323     require_close(q6_values[128 + LCQI_GGML_TEST_Q6_LANE], 4.0F, "Q6_K second ↵
↵ half mismatch");
1324
1325     bool rejected_bad_size = false;
1326     try {
1327         static_cast<void>(lcqi::dequantize_ggml_tensor(
1328             lcqi::GgmlType::q8_0,
1329             std::span<const std::uint8_t>(q8.data(), q8.size() - 1U),
1330             lcqi::LCQI_Q8_0_BLOCK_VALUES));
1331     } catch (const std::runtime_error&) {
1332         rejected_bad_size = true;
1333     }
1334     require(rejected_bad_size, "GGML dequantizer accepted a truncated Q8_0 ↵
↵ block");
1335 }
1336
1337 void test_ggml_quantized_f32_matvec_paths() {
1338     const std::vector<float> input32 = [] {
1339         std::vector<float> values(static_cast<std::size_t>(lcqi::↵
↵ LCQI_Q8_0_BLOCK_VALUES), 0.0F);
1340         for (std::size_t index = 0; index < values.size(); ++index) {
1341             values[index] =
1342                 static_cast<float>(static_cast<std::int32_t>(index % 9U) - 4) *↵
↵ 0.125F;
1343         }
1344         return values;
1345     }();
1346     const std::vector<float> input256 = [] {
1347         std::vector<float> values(static_cast<std::size_t>(lcqi::↵
↵ LCQI_Q6_K_BLOCK_VALUES), 0.0F);
1348         for (std::size_t index = 0; index < values.size(); ++index) {
1349             values[index] =
1350                 static_cast<float>(static_cast<std::int32_t>(index % 13U) - 6) ↵
↵ * 0.0625F;
1351         }

```

```

1352     return values;
1353 }();
1354
1355 const auto assert_matvec_matches_dequantized =
1356     [](lcqi::GgmlType type,
1357        const std::vector<std::uint8_t>& block,
1358        std::span<const float> input,
1359        const char* message) {
1360     const std::int64_t columns = static_cast<std::int64_t>(input.size())↵
↵ );
1361     const std::vector<std::uint8_t> matrix = [&] {
1362         std::vector<std::uint8_t> bytes;
1363         bytes.reserve(block.size() * 2U);
1364         bytes.insert(bytes.end(), block.begin(), block.end());
1365         bytes.insert(bytes.end(), block.begin(), block.end());
1366         return bytes;
1367     }();
1368     std::vector<float> direct(2, 0.0F);
1369     lcqi::matvec_ggml_quantized_f32(type, matrix, 2, columns, input, ↵
↵ direct);
1370     const std::vector<lcqi::Q8_0InputBlock> q8_0_input =
1371         lcqi::quantize_q8_0_input(input);
1372     std::vector<float> q8_0_direct(2, 0.0F);
1373     lcqi::matvec_ggml_quantized_q8_0(type,
1374                                     matrix,
1375                                     2,
1376                                     columns,
1377                                     q8_0_input,
1378                                     q8_0_direct);
1379     std::vector<float> q8_0_row_range_direct(2, 0.0F);
1380     lcqi::matvec_ggml_quantized_q8_0_rows_unchecked(type,
1381                                                     matrix,
1382                                                     2,
1383                                                     columns,
1384                                                     q8_0_input,
1385                                                     ↵
↵ q8_0_row_range_direct,
1386                                                     0,
1387                                                     1);
1388     lcqi::matvec_ggml_quantized_q8_0_rows_unchecked(type,
1389                                                     matrix,
1390                                                     2,
1391                                                     columns,
1392                                                     q8_0_input,
1393                                                     ↵
↵ q8_0_row_range_direct,
1394                                                     1,
1395                                                     2);
1396     constexpr std::size_t LCQI_GGML_DIRECT_BATCH_SIZE = 3;
1397     std::vector<lcqi::Q8_0InputBlock> batch_q8_0_input(
1398         LCQI_GGML_DIRECT_BATCH_SIZE * q8_0_input.size());
1399     std::vector<float> expected_batch(LCQI_GGML_DIRECT_BATCH_SIZE * 2U, ↵

```



```

1400     ↪ 0.0F);
1401     std::vector<float> direct_batch(expected_batch.size(), 0.0F);
1402     ↪ for (std::size_t batch = 0; batch < LCQI_GGML_DIRECT_BATCH_SIZE; ++batch) {
1403         std::vector<float> batch_input(input.begin(), input.end());
1404         ↪ for (std::size_t index = 0; index < batch_input.size(); ++index) {
1405             batch_input[index] +=
1406                 ↪ static_cast<float>>(static_cast<std::int32_t>(batch) -
1407                     ↪ 1) *
1408                     0.015625F;
1409             }
1410             ↪ const std::vector<lcqi::Q8_0InputBlock> batch_quantized =
1411                 ↪ lcqi::quantize_q8_0_input(batch_input);
1412                 ↪ std::copy(batch_quantized.begin(),
1413                     ↪ batch_quantized.end(),
1414                     ↪ batch_q8_0_input.begin() +
1415                     ↪ static_cast<std::ptrdiff_t>(batch *
1416                         ↪ batch_quantized.size()));
1417                     ↪ lcqi::matvec_ggml_quantized_q8_0(
1418                         ↪ type,
1419                         ↪ matrix,
1420                         ↪ 2,
1421                         ↪ columns,
1422                         ↪ batch_quantized,
1423                         ↪ std::span<float>(expected_batch).subspan(batch * 2U, 2U));
1424                     }
1425                     ↪ lcqi::matvec_ggml_quantized_q8_0_batch_rows_unchecked(
1426                         ↪ type,
1427                         ↪ matrix,
1428                         ↪ 2,
1429                         ↪ columns,
1430                         ↪ batch_q8_0_input,
1431                         ↪ static_cast<std::int64_t>(LCQI_GGML_DIRECT_BATCH_SIZE),
1432                         ↪ direct_batch,
1433                         ↪ 2,
1434                         ↪ 0,
1435                         ↪ 2);
1436                     ↪ const lcqi::F32RowMax q8_0_max =
1437                         ↪ lcqi::max_dot_ggml_quantized_q8_0(type,
1438                             ↪ matrix,
1439                             ↪ 2,
1440                             ↪ columns,
1441                             ↪ q8_0_input);
1442                     ↪ const lcqi::F32RowMax q8_0_subrange_max =
1443                         ↪ lcqi::max_dot_ggml_quantized_q8_0_rows_unchecked(type,
1444                             ↪ matrix,
1445                             ↪ 2,
1446                             ↪ columns,
1447                             ↪ q8_0_input,
1448                             ↪ 1,
1449                             ↪ 2);

```

```

1447     const std::vector<float> weights =
1448         lcqi::dequantize_ggml_tensor(type, matrix, columns * 2);
1449     std::array<float, 2> reference{0.0F, 0.0F};
1450     for (std::size_t row = 0; row < reference.size(); ++row) {
1451         for (std::size_t column = 0; column < input.size(); ++column) {
1452             reference[row] +=
1453                 weights[row * input.size() + column] * input[column];
1454         }
1455         if (std::fabs(reference[row] - direct[row]) > ←
↪ LCQI_GGML_MATVEC_MAX_DIFF) {
1456             throw std::runtime_error(message);
1457         }
1458         if (std::fabs(reference[row] - q8_0_direct[row]) >
1459             LCQI_GGML_Q8_MATVEC_MAX_DIFF) {
1460             throw std::runtime_error(message);
1461         }
1462         if (std::fabs(q8_0_direct[row] - q8_0_row_range_direct[row]) >
1463             LCQI_GGML_Q8_MATVEC_MAX_DIFF) {
1464             throw std::runtime_error(message);
1465         }
1466     }
1467     require(q8_0_max.row == 0U, "GGML Q8_0 max-dot selected the wrong ←
↪ row");
1468     require(std::fabs(q8_0_max.value - q8_0_direct[0]) <=
1469         LCQI_GGML_Q8_MATVEC_MAX_DIFF,
1470         "GGML Q8_0 max-dot value differs from matvec output");
1471     require(q8_0_subrange_max.row == 1U,
1472         "GGML Q8_0 row-range max-dot selected the wrong row");
1473     require(std::fabs(q8_0_subrange_max.value - q8_0_direct[1]) <=
1474         LCQI_GGML_Q8_MATVEC_MAX_DIFF,
1475         "GGML Q8_0 row-range max-dot value differs from matvec ←
↪ output");
1476     for (std::size_t index = 0; index < direct_batch.size(); ++index) {
1477         if (std::fabs(expected_batch[index] - direct_batch[index]) >
1478             LCQI_GGML_Q8_MATVEC_MAX_DIFF) {
1479             throw std::runtime_error(message);
1480         }
1481     }
1482 };
1483
1484 require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q5_0),
1485     "Q5_0 direct matvec support flag mismatch");
1486 require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q6_k),
1487     "Q6_K direct matvec support flag mismatch");
1488 require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q8_0),
1489     "Q8_0 direct matvec support flag mismatch");
1490 require(!lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::f32),
1491     "F32 should not be reported as quantized direct matvec");
1492 require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q5_0),
1493     "Q5_0 Q8_0 direct matvec support flag mismatch");
1494 require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q6_k),
1495     "Q6_K Q8_0 direct matvec support flag mismatch");

```

```

1496 require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q8_0),
1497         "Q8_0 Q8_0 direct matvec support flag mismatch");
1498 require(!lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::f32),
1499         "F32 should not be reported as Q8_0 quantized direct matvec");
1500
1501 assert_matvec_matches_dequantized(lcqi::GgmlType::q8_0,
1502                                   make_q8_0_test_block(),
1503                                   input32,
1504                                   "Q8_0 direct matvec mismatch");
1505 assert_matvec_matches_dequantized(lcqi::GgmlType::q5_0,
1506                                   make_q5_0_test_block(),
1507                                   input32,
1508                                   "Q5_0 direct matvec mismatch");
1509 assert_matvec_matches_dequantized(lcqi::GgmlType::q6_k,
1510                                   make_q6_k_test_block(),
1511                                   input256,
1512                                   "Q6_K direct matvec mismatch");
1513 }
1514
1515 void test_q5_0_packed_matvec_matches_ggml_q8_0_path() {
1516     std::vector<float> input(static_cast<std::size_t>(lcqi::↵
↵ LCQI_Q5_0_BLOCK_VALUES), 0.0F);
1517     for (std::size_t index = 0; index < input.size(); ++index) {
1518         input[index] =
1519             static_cast<float>(static_cast<std::int32_t>(index % 11U) - 5) * ↵
↵ 0.0625F;
1520     }
1521     const std::vector<std::uint8_t> block = make_q5_0_test_block();
1522     std::vector<std::uint8_t> matrix;
1523     matrix.reserve(block.size() * 2U);
1524     matrix.insert(matrix.end(), block.begin(), block.end());
1525     matrix.insert(matrix.end(), block.begin(), block.end());
1526
1527     const std::vector<lcqi::Q8_0InputBlock> q8_0_input = lcqi::↵
↵ quantize_q8_0_input(input);
1528     std::vector<float> ggml_direct(2, 0.0F);
1529     lcqi::matvec_ggml_quantized_q8_0(lcqi::GgmlType::q5_0,
1530                                     matrix,
1531                                     2,
1532                                     lcqi::LCQI_Q5_0_BLOCK_VALUES,
1533                                     q8_0_input,
1534                                     ggml_direct);
1535     const std::vector<lcqi::Q5_0PackedBlock> packed =
1536         lcqi::pack_q5_0_blocks(matrix, lcqi::LCQI_Q5_0_BLOCK_VALUES * 2);
1537     std::vector<float> packed_direct(2, 0.0F);
1538     lcqi::matvec_q5_0_packed_q8_0(packed,
1539                                   2,
1540                                   lcqi::LCQI_Q5_0_BLOCK_VALUES,
1541                                   q8_0_input,
1542                                   packed_direct);
1543     std::vector<float> row_range_direct(2, 0.0F);
1544     lcqi::matvec_q5_0_packed_q8_0_rows_unchecked(packed,

```

```

1545         2,
1546         lcqi::LCQI_Q5_0_BLOCK_VALUES,
1547         q8_0_input,
1548         row_range_direct,
1549         0,
1550         1);
1551     lcqi::matvec_q5_0_packed_q8_0_rows_unchecked(packed,
1552         2,
1553         lcqi::LCQI_Q5_0_BLOCK_VALUES,
1554         q8_0_input,
1555         row_range_direct,
1556         1,
1557         2);
1558     const auto assert_packed_batch_matches_per_token =
1559         [&](std::size_t batch_size, const char* message) {
1560         std::vector<lcqi::Q8_0InputBlock> batch_q8_0_input(
1561             batch_size * q8_0_input.size());
1562         std::vector<float> expected_batch(batch_size * ggml_direct.size(), ←
↪ 0.0F);
1563         std::vector<float> packed_batch(expected_batch.size(), 0.0F);
1564         for (std::size_t batch = 0; batch < batch_size; ++batch) {
1565             std::vector<float> batch_input = input;
1566             for (std::size_t index = 0; index < batch_input.size(); ++index↪
↪ ) {
1567                 batch_input[index] +=
1568                     static_cast<float>(static_cast<std::int32_t>(batch) - ←
↪ 1) * 0.03125F;
1569             }
1570             const std::vector<lcqi::Q8_0InputBlock> batch_quantized =
1571                 lcqi::quantize_q8_0_input(batch_input);
1572             std::copy(batch_quantized.begin(),
1573                 batch_quantized.end(),
1574                 batch_q8_0_input.begin() +
1575                     static_cast<std::ptrdiff_t>(batch * ←
↪ batch_quantized.size()));
1576             lcqi::matvec_q5_0_packed_q8_0(
1577                 packed,
1578                 2,
1579                 lcqi::LCQI_Q5_0_BLOCK_VALUES,
1580                 batch_quantized,
1581                 std::span<float>(expected_batch)
1582                     .subspan(batch * ggml_direct.size(), ggml_direct.size())↪
↪ ));
1583         }
1584         lcqi::matvec_q5_0_packed_q8_0_batch_rows_unchecked(
1585             packed,
1586             2,
1587             lcqi::LCQI_Q5_0_BLOCK_VALUES,
1588             batch_q8_0_input,
1589             static_cast<std::int64_t>(batch_size),
1590             packed_batch,
1591             static_cast<std::int64_t>(ggml_direct.size()),

```

```

1592         0,
1593         2);
1594     for (std::size_t index = 0; index < packed_batch.size(); ++index) {
1595         require(std::fabs(expected_batch[index] - packed_batch[index]) <=
↪ <=
1596             LCQI_GGML_Q8_MATVEC_MAX_DIFF,
1597             message);
1598     }
1599 };
1600 assert_packed_batch_matches_per_token(3, "Q5_0 packed batch tail-3 output <=
↪ mismatch");
1601 assert_packed_batch_matches_per_token(5, "Q5_0 packed batch 4+1 output <=
↪ mismatch");
1602 assert_packed_batch_matches_per_token(11, "Q5_0 packed batch 8+3 output <=
↪ mismatch");
1603 for (std::size_t row = 0; row < ggml_direct.size(); ++row) {
1604     require(std::fabs(ggml_direct[row] - packed_direct[row]) <=
1605         LCQI_GGML_Q8_MATVEC_MAX_DIFF,
1606         "Q5_0 packed output differs from GGML Q8_0 direct path");
1607     require(std::fabs(packed_direct[row] - row_range_direct[row]) <=
1608         LCQI_GGML_Q8_MATVEC_MAX_DIFF,
1609         "Q5_0 packed row range output differs from full path");
1610 }
1611 }
1612
1613 void test_llama_gguf_loader_and_generation() {
1614     const std::filesystem::path path = temp_file_path("lcqi_tiny_llama_fixture.<=
↪ gguf");
1615     write_tiny_llama_gguf_fixture(path);
1616
1617     const lcqi::LlamaGgufLoadedModel loaded =
1618         lcqi::load_llama_gguf_reference_model(path);
1619     const std::vector<std::int32_t> prompt{1};
1620     const lcqi::LlamaGgufGenerationResult result =
1621         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1622     std::filesystem::remove(path);
1623
1624     require(loaded.model.architecture == "llama", "LLaMA GGUF architecture <=
↪ mismatch");
1625     require(loaded.model.decoder.config.hidden_size == LCQI_LLAMA_TEST_HIDDEN,
1626         "LLaMA GGUF hidden size mismatch");
1627     require(loaded.model.decoder.layers.size() == LCQI_LLAMA_TEST_LAYER_COUNT,
1628         "LLaMA GGUF layer count mismatch");
1629     require(loaded.model.lm_head_tied_to_embedding,
1630         "LLaMA GGUF tiny fixture should tie lm head to embeddings");
1631     require(result.generated_ids == std::vector<std::int32_t>({1, <=
↪ LCQI_LLAMA_TEST_EXPECTED_TOKEN}),
1632         "LLaMA GGUF generated ids mismatch");
1633     require(result.predicted_first_token == LCQI_LLAMA_TEST_EXPECTED_TOKEN,
1634         "LLaMA GGUF predicted token mismatch");
1635     require(loaded.report.tensors_loaded == 11, "LLaMA GGUF tensor load count <=
↪ mismatch");

```

```

1636     require(loaded.report.f32_weight_bytes == loaded.report.↵
↵ quantized_weight_bytes,
1637         "LLaMA GGUF F32 fixture byte accounting mismatch");
1638     require(loaded.report.q4_k_direct_tensors == 0,
1639         "LLaMA GGUF F32 fixture should not use direct Q4_K tensors");
1640     require(loaded.report.ggml_direct_tensors == 0,
1641         "LLaMA GGUF F32 fixture should not use direct GGML tensors");
1642     require(result.hotspots.f32_fallback_calls == 7,
1643         "LLaMA GGUF F32 fixture linear fallback call count mismatch");
1644     require(result.hotspots.q4_k_direct_calls == 0,
1645         "LLaMA GGUF F32 fixture should not call Q4_K direct matvec");
1646     require(result.hotspots.ggml_direct_calls == 0,
1647         "LLaMA GGUF F32 fixture should not call GGML direct matvec");
1648 }
1649
1650 void test_llama_gguf_default_packs_q5_0_tensors() {
1651     const std::filesystem::path path = temp_file_path("↵
↵ lcqi_tiny_llama_q5_0_packed.gguf");
1652     write_tiny_llama_ggml_direct_gguf_fixture(path);
1653
1654     const ScopedEnvVar disable_ggml_direct(LCQI_TEST_LLAMA_GGML_DIRECT_ENV,
1655                                             LCQI_TEST_FALSE_ENV);
1656     const ScopedEnvVar disable_selective_ggml_direct(↵
↵ LCQI_TEST_LLAMA_GGML_SELECTIVE_DIRECT_ENV,
1657                                             LCQI_TEST_FALSE_ENV);
1658     const ScopedEnvVar enable_q5_0_packed(LCQI_TEST_LLAMA_Q5_0_PACKED_ENV,
1659                                             LCQI_TEST_TRUE_ENV);
1660     const ScopedEnvVar enable_q8_0_direct(LCQI_TEST_LLAMA_Q8_0_DIRECT_ENV,
1661                                             LCQI_TEST_TRUE_ENV);
1662     const lcqi::LlamaGgufLoadedModel loaded =
1663         lcqi::load_llama_gguf_reference_model(path);
1664     const std::vector<std::int32_t> prompt{0};
1665     const lcqi::LlamaGgufGenerationResult result =
1666         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1667     std::filesystem::remove(path);
1668
1669     require(loaded.report.q5_0_packed_tensors == 4,
1670         "default LLaMA GGUF path should pack Q5_0 tensors");
1671     require(loaded.report.ggml_direct_tensors == 0,
1672         "default LLaMA GGUF path should not use experimental GGML direct ↵
↵ tensors");
1673     require(loaded.report.q8_0_direct_tensors == 2,
1674         "default LLaMA GGUF path should keep Q8_0 tensors direct");
1675     require(loaded.report.f32_fallback_tensors == 5,
1676         "default LLaMA GGUF path should leave F32 and Q6_K tensors in ↵
↵ fallback");
1677     require(result.hotspots.q5_0_packed_calls == 4,
1678         "default LLaMA GGUF path Q5_0 packed call count mismatch");
1679     require(result.hotspots.q8_0_direct_calls == 2,
1680         "default LLaMA GGUF path Q8_0 direct call count mismatch");
1681     require(result.hotspots.f32_fallback_calls == 1,
1682         "default LLaMA GGUF path fallback call count mismatch");

```

```

1683     require(result.hotspots.ggml_direct_calls == 0,
1684             "default LLaMA GGUF path should not call experimental GGML direct")↵
1685     ↵ ;
1686     require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
1687             "default LLaMA GGUF Q5_0 packed generated ids mismatch");
1688 }
1689 void test_llama_gguf_batch_prefill_uses_q5_0_f32_sidecar_path() {
1690     const std::filesystem::path path =
1691         temp_file_path("lcqi_tiny_llama_q5_0_batch_prefill.gguf");
1692     write_tiny_llama_ggml_direct_gguf_fixture(path);
1693
1694     const ScopedEnvVar disable_ggml_direct(LCQI_TEST_LLAMA_GGML_DIRECT_ENV,
1695                                             LCQI_TEST_FALSE_ENV);
1696     const ScopedEnvVar disable_selective_ggml_direct(↵
1697     ↵ LCQI_TEST_LLAMA_GGML_SELECTIVE_DIRECT_ENV,
1698                                             LCQI_TEST_FALSE_ENV);
1699     const ScopedEnvVar enable_q5_0_packed(LCQI_TEST_LLAMA_Q5_0_PACKED_ENV,
1700                                             LCQI_TEST_TRUE_ENV);
1701     const ScopedEnvVar enable_q8_0_direct(LCQI_TEST_LLAMA_Q8_0_DIRECT_ENV,
1702                                             LCQI_TEST_TRUE_ENV);
1703     const lcqi::LlamaGgufLoadedModel loaded =
1704         lcqi::load_llama_gguf_reference_model(path);
1705     const std::vector<std::int32_t> prompt{0, 0, 0};
1706     {
1707         const ScopedEnvVar disable_batch_prefill(↵
1708         ↵ LCQI_TEST_LLAMA_BATCH_PREFILL_ENV,
1709                                             LCQI_TEST_FALSE_ENV);
1710         const lcqi::LlamaGgufGenerationResult token_path =
1711             lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1712         const ScopedEnvVar enable_batch_prefill(↵
1713         ↵ LCQI_TEST_LLAMA_BATCH_PREFILL_ENV,
1714                                             LCQI_TEST_TRUE_ENV);
1715         const lcqi::LlamaGgufGenerationResult batch_path =
1716             lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1717         require(batch_path.generated_ids == token_path.generated_ids,
1718                 "Q5_0 packed batch prefill generated ids differ from token path↵
1719         ↵ ");
1720         require(batch_path.predicted_first_token == token_path.↵
1721         ↵ predicted_first_token,
1722                 "Q5_0 packed batch prefill predicted token differs from token ↵
1723         ↵ path");
1724         require(batch_path.hotspots.q5_0_packed_calls == 0,
1725                 "Q5_0 batch prefill should not use packed decode kernels");
1726         require(batch_path.hotspots.q5_0_batch_f32_calls ==
1727                 static_cast<std::uint64_t>(loaded.report.↵
1728         ↵ q5_0_packed_tensors),
1729                 "Q5_0 batch prefill should call each F32 sidecar tensor once");
1730         require(token_path.hotspots.q5_0_packed_calls >
1731                 batch_path.hotspots.q5_0_packed_calls,
1732                 "Q5_0 batch prefill should leave packed matvecs to token/decode↵
1733         ↵ path");

```



```

1726     }
1727     std::filesystem::remove(path);
1728 }
1729
1730 void test_llama_gguf_parallel_generation_matches_serial() {
1731     const std::filesystem::path path = temp_file_path("lcqi_tiny_llama_parallel↵
↵ .gguf");
1732     write_tiny_llama_gguf_fixture(path);
1733
1734     const lcqi::LlamaGgufLoadedModel loaded =
1735         lcqi::load_llama_gguf_reference_model(path);
1736     const std::vector<std::int32_t> prompt{1};
1737     const lcqi::LlamaGgufGenerationResult serial =
1738         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1739     lcqi::LlamaGgufExecutionOptions parallel_options;
1740     parallel_options.worker_count = 2;
1741     parallel_options.parallel_min_rows = 1;
1742     const lcqi::LlamaGgufGenerationResult parallel =
1743         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1, ↵
↵ parallel_options);
1744     std::filesystem::remove(path);
1745
1746     require(parallel.worker_count == parallel_options.worker_count,
1747         "parallel LLaMA worker count mismatch");
1748     require(parallel.generated_ids == serial.generated_ids,
1749         "parallel LLaMA generated ids mismatch");
1750     require(parallel.predicted_first_token == serial.predicted_first_token,
1751         "parallel LLaMA predicted token mismatch");
1752     require(parallel.hotspots.f32_fallback_calls == serial.hotspots.↵
↵ f32_fallback_calls,
1753         "parallel LLaMA fallback call count mismatch");
1754     require(parallel.hotspots.q4_k_direct_calls == serial.hotspots.↵
↵ q4_k_direct_calls,
1755         "parallel LLaMA Q4 direct call count mismatch");
1756     require(parallel.hotspots.ggml_direct_calls == serial.hotspots.↵
↵ ggml_direct_calls,
1757         "parallel LLaMA GGML direct call count mismatch");
1758 }
1759
1760 void test_llama_gguf_q4_direct_generation() {
1761     const std::filesystem::path path = temp_file_path("↵
↵ lcqi_tiny_llama_q4_direct.gguf");
1762     write_tiny_llama_q4_direct_gguf_fixture(path);
1763
1764     const lcqi::LlamaGgufLoadedModel loaded =
1765         lcqi::load_llama_gguf_reference_model(path);
1766     const std::vector<std::int32_t> prompt{0};
1767     const lcqi::LlamaGgufGenerationResult result =
1768         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1769     std::filesystem::remove(path);
1770
1771     require(loaded.model.decoder.config.hidden_size == ↵

```



```

1772     LCQI_LLAMA_Q4_TEST_HIDDEN,
1773     "LLaMA Q4_K fixture hidden size mismatch");
1774 require(loaded.report.tensors_loaded == 11, "LLaMA Q4_K fixture tensor load↵
1775     ↵ count mismatch");
1776 require(loaded.report.q4_k_direct_tensors == 7,
1777     "LLaMA Q4_K fixture direct tensor count mismatch");
1778 require(loaded.report.ggml_direct_tensors == 0,
1779     "LLaMA Q4_K fixture should not use GGML direct tensors");
1780 require(loaded.report.f32_fallback_tensors == 4,
1781     "LLaMA Q4_K fixture fallback tensor count mismatch");
1782 require(loaded.report.direct_quantized_weight_bytes >
1783     loaded.report.fallback_dequantized_weight_bytes,
1784     "LLaMA Q4_K fixture should keep direct quantized matrix bytes");
1785 require(result.hotspots.q4_k_direct_calls == 7,
1786     "LLaMA Q4_K fixture direct matvec call count mismatch");
1787 require(result.hotspots.ggml_direct_calls == 0,
1788     "LLaMA Q4_K fixture should not call GGML direct matvec");
1789 require(result.hotspots.f32_fallback_calls == 0,
1790     "LLaMA Q4_K fixture should not call fallback linear matvec");
1791 require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
1792     "LLaMA Q4_K fixture generated ids mismatch");
1793 }
1794
1795 void test_llama_gguf_ggml_direct_generation() {
1796     const ScopedEnvVar enable_ggml_direct(LCQI_TEST_LLAMA_GGML_DIRECT_ENV,
1797         LCQI_TEST_TRUE_ENV);
1798
1799     const std::filesystem::path path = temp_file_path("↵
1800     ↵ lcqi_tiny_llama_ggml_direct.gguf");
1801     write_tiny_llama_ggml_direct_gguf_fixture(path);
1802
1803     const lcqi::LlamaGgufLoadedModel loaded =
1804         lcqi::load_llama_gguf_reference_model(path);
1805     const std::vector<std::int32_t> prompt{0};
1806     const lcqi::LlamaGgufGenerationResult result =
1807         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1808     std::filesystem::remove(path);
1809
1810     require(loaded.model.decoder.config.hidden_size == ↵
1811     ↵ LCQI_LLAMA_Q4_TEST_HIDDEN,
1812     "LLaMA GGML fixture hidden size mismatch");
1813 require(loaded.report.tensors_loaded == 11, "LLaMA GGML fixture tensor load↵
1814     ↵ count mismatch");
1815 require(loaded.report.q4_k_direct_tensors == 0,
1816     "LLaMA GGML fixture should not use Q4_K direct tensors");
1817 require(loaded.report.ggml_direct_tensors == 7,
1818     "LLaMA GGML fixture direct tensor count mismatch");
1819 require(loaded.report.f32_fallback_tensors == 4,
1820     "LLaMA GGML fixture fallback tensor count mismatch");
1821 require(result.hotspots.ggml_direct_calls == 7,
1822     "LLaMA GGML fixture direct matvec call count mismatch");
1823 require(result.hotspots.q4_k_direct_calls == 0,
1824     "LLaMA GGML fixture should not call Q4_K direct matvec");

```

```

1819     require(result.hotspots.f32_fallback_calls == 0,
1820             "LLaMA GGML fixture should not call fallback linear matvec");
1821     require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
1822             "LLaMA GGML fixture generated ids mismatch");
1823 }
1824
1825 void test_llama_gguf_selective_ggml_direct_keeps_q6_fallback() {
1826     const ScopedEnvVar enable_selective_ggml_direct(
1827         LCQI_TEST_LLAMA_GGML_SELECTIVE_DIRECT_ENV,
1828         LCQI_TEST_TRUE_ENV);
1829     const std::filesystem::path path = temp_file_path("↵
↵ lcqi_tiny_llama_selective_ggml.gguf");
1830     write_tiny_llama_ggml_direct_gguf_fixture(path);
1831
1832     const lcqi::LlamaGgufLoadedModel loaded =
1833         lcqi::load_llama_gguf_reference_model(path);
1834     const std::vector<std::int32_t> prompt{0};
1835     const lcqi::LlamaGgufGenerationResult result =
1836         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1837     std::filesystem::remove(path);
1838
1839     require(loaded.report.q4_k_direct_tensors == 0,
1840             "selective GGML fixture should not use Q4_K direct tensors");
1841     require(loaded.report.ggml_direct_tensors == 6,
1842             "selective GGML fixture should only keep Q5_0/Q8_0 tensors direct")↵
↵ ;
1843     require(loaded.report.f32_fallback_tensors == 5,
1844             "selective GGML fixture should dequantize Q6_K through the F32 ↵
↵ fallback");
1845     require(result.hotspots.ggml_direct_calls == 6,
1846             "selective GGML fixture direct call count mismatch");
1847     require(result.hotspots.f32_fallback_calls == 1,
1848             "selective GGML fixture Q6_K fallback call count mismatch");
1849     require(result.hotspots.q4_k_direct_calls == 0,
1850             "selective GGML fixture should not call Q4_K direct matvec");
1851     require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
1852             "selective GGML fixture generated ids mismatch");
1853 }
1854
1855 void test_llama_gguf_q6_k_direct_experimental_path() {
1856     const ScopedEnvVar disable_ggml_direct(LCQI_TEST_LLAMA_GGML_DIRECT_ENV,
1857         LCQI_TEST_FALSE_ENV);
1858     const ScopedEnvVar disable_selective_ggml_direct(↵
↵ LCQI_TEST_LLAMA_GGML_SELECTIVE_DIRECT_ENV,
1859         LCQI_TEST_FALSE_ENV);
1860     const ScopedEnvVar enable_q5_0_packed(LCQI_TEST_LLAMA_Q5_0_PACKED_ENV,
1861         LCQI_TEST_TRUE_ENV);
1862     const ScopedEnvVar enable_q8_0_direct(LCQI_TEST_LLAMA_Q8_0_DIRECT_ENV,
1863         LCQI_TEST_TRUE_ENV);
1864     const ScopedEnvVar enable_q6_k_direct(LCQI_TEST_LLAMA_Q6_K_DIRECT_ENV,
1865         LCQI_TEST_TRUE_ENV);
1866     const std::filesystem::path path =

```

```

1867     temp_file_path("lcqi_tiny_llama_q6_k_direct.gguf");
1868     write_tiny_llama_ggml_direct_gguf_fixture(path);
1869
1870     const lcqi::LlamaGgufLoadedModel loaded =
1871         lcqi::load_llama_gguf_reference_model(path);
1872     const std::vector<std::int32_t> prompt{0};
1873     const lcqi::LlamaGgufGenerationResult result =
1874         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1875     std::filesystem::remove(path);
1876
1877     require(loaded.report.q5_0_packed_tensors == 4,
1878             "Q6_K direct fixture should still pack Q5_0 tensors");
1879     require(loaded.report.q6_k_direct_tensors == 1,
1880             "Q6_K direct fixture should keep the Q6_K tensor direct");
1881     require(loaded.report.q8_0_direct_tensors == 2,
1882             "Q6_K direct fixture should still keep Q8_0 tensors direct");
1883     require(loaded.report.f32_fallback_tensors == 4,
1884             "Q6_K direct fixture should remove Q6_K from the F32 fallback set")↵
1885     ↵ ;
1886     require(result.hotspots.q6_k_direct_calls == 1,
1887             "Q6_K direct fixture should call the Q6_K direct path once");
1888     require(result.hotspots.f32_fallback_calls == 0,
1889             "Q6_K direct fixture should not call fallback linear matvec");
1890     require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
1891             "Q6_K direct fixture generated ids mismatch");
1892 }
1893
1894 void test_gpt2_tiny_forward_and_generation() {
1895     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model↵
1896     ↵ ();
1897     const std::vector<std::int32_t> tokens{1, 2, 3};
1898     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens↵
1899     ↵ );
1900
1901     require(result.logits.size() == LCQI_GPT2_VOCAB_SIZE, "GPT-2 logits size ↵
1902     ↵ mismatch");
1903     require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
1904             "GPT-2 predicted token mismatch");
1905     for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
1906         require_close(result.logits[index], LCQI_GPT2_LOGITS[index], "GPT-2 ↵
1907         ↵ logit mismatch");
1908     }
1909
1910     const std::vector<std::int32_t> generated =
1911         lcqi::gpt2_generate_greedy(model, tokens, 2);
1912     const std::vector<std::int32_t> expected{1, 2, 3, 3, 3};
1913     require(generated == expected, "GPT-2 greedy generation mismatch");
1914     require(static_cast<std::int32_t>(generated.size()) == ↵
1915     ↵ LCQI_GPT2_GENERATED_LENGTH,
1916             "GPT-2 generated length mismatch");
1917
1918     lcqi::Gpt2KvCache cache(model.config);

```

```

1913     lcqi::Gpt2ForwardResult cached_result;
1914     for (const std::int32_t token : tokens) {
1915         cached_result = lcqi::run_gpt2_forward_cached(model, cache, token);
1916     }
1917     require(cache.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
1918             "GPT-2 KV cache filled token count mismatch");
1919     require(cache.byte_size() > 0, "GPT-2 KV cache byte size should be non-zero↵
↵");
1920     require(cached_result.predicted_token == result.predicted_token,
1921             "cached GPT-2 predicted token mismatch");
1922     require(cached_result.logits.size() == result.logits.size(),
1923             "cached GPT-2 logits size mismatch");
1924     for (std::size_t index = 0; index < result.logits.size(); ++index) {
1925         require_close(cached_result.logits[index],
1926                       result.logits[index],
1927                       "cached GPT-2 logit mismatch");
1928     }
1929
1930     lcqi::Gpt2CachedGreedyDecoder decoder_with_logits(model);
1931     lcqi::Gpt2ForwardResult decoder_result;
1932     for (const std::int32_t token : tokens) {
1933         decoder_result = decoder_with_logits.step_with_logits(token);
1934     }
1935     require(decoder_with_logits.filled_tokens() == static_cast<std::int32_t>(↵
↵ tokens.size()),
1936             "GPT-2 optimized decoder filled token count mismatch");
1937     require(decoder_with_logits.kv_cache_bytes() == cache.byte_size(),
1938             "GPT-2 optimized decoder KV byte size mismatch");
1939     require(decoder_result.predicted_token == result.predicted_token,
1940             "optimized cached GPT-2 predicted token mismatch");
1941     require(decoder_result.logits.size() == result.logits.size(),
1942             "optimized cached GPT-2 logits size mismatch");
1943     for (std::size_t index = 0; index < result.logits.size(); ++index) {
1944         require_close(decoder_result.logits[index],
1945                       result.logits[index],
1946                       "optimized cached GPT-2 logit mismatch");
1947     }
1948
1949     lcqi::Gpt2ExecutionOptions parallel_options;
1950     parallel_options.worker_count = 2;
1951     parallel_options.parallel_min_rows = 1;
1952     lcqi::Gpt2CachedGreedyDecoder parallel_decoder(model, nullptr, ↵
↵ parallel_options);
1953     lcqi::Gpt2ForwardResult parallel_result;
1954     for (const std::int32_t token : tokens) {
1955         parallel_result = parallel_decoder.step_with_logits(token);
1956     }
1957     require(parallel_decoder.worker_count() == parallel_options.worker_count,
1958             "parallel GPT-2 decoder worker count mismatch");
1959     require(parallel_result.predicted_token == result.predicted_token,
1960             "parallel cached GPT-2 predicted token mismatch");
1961     require(parallel_result.logits.size() == result.logits.size(),

```

```

1962     "parallel cached GPT-2 logits size mismatch");
1963     for (std::size_t index = 0; index < result.logits.size(); ++index) {
1964         require_close(parallel_result.logits[index],
1965             result.logits[index],
1966             "parallel cached GPT-2 logit mismatch");
1967     }
1968
1969     lcqi::Gpt2CachedGreedyDecoder greedy_decoder(model);
1970     std::int32_t greedy_predicted = 0;
1971     for (const std::int32_t token : tokens) {
1972         greedy_predicted = greedy_decoder.step(token);
1973     }
1974     require(greedy_predicted == result.predicted_token,
1975         "optimized logits-free GPT-2 predicted token mismatch");
1976
1977     lcqi::Gpt2CachedGreedyDecoder prefill_decoder(model);
1978     for (std::size_t index = 0; index + 1 < tokens.size(); ++index) {
1979         prefill_decoder.advance_without_prediction(tokens[index]);
1980     }
1981     require(prefill_decoder.filled_tokens() ==
1982         static_cast<std::int32_t>(tokens.size() - 1U),
1983         "GPT-2 prefill-only decoder filled token count mismatch");
1984     const std::int32_t prefill_predicted = prefill_decoder.step(tokens.back());
1985     require(prefill_decoder.filled_tokens() == static_cast<std::int32_t>(tokens
1986 ↪ .size()),
1987         "GPT-2 prefill decoder final filled token count mismatch");
1988     require(prefill_predicted == result.predicted_token,
1989         "GPT-2 prefill-skipped predicted token mismatch");
1990
1991     lcqi::Gpt2HotspotProfile hotspot_profile;
1992     lcqi::Gpt2CachedGreedyDecoder profiled_decoder(model, &hotspot_profile);
1993     std::int32_t profiled_predicted = 0;
1994     for (const std::int32_t token : tokens) {
1995         profiled_predicted = profiled_decoder.step(token);
1996     }
1997     require(profiled_predicted == result.predicted_token,
1998         "profiled GPT-2 predicted token mismatch");
1999     require(hotspot_profile.decoder_steps == static_cast<std::int64_t>(tokens
2000 ↪ .size()),
2001         "profiled GPT-2 decoder step count mismatch");
2002     require(hotspot_profile.layer_steps ==
2003         static_cast<std::int64_t>(tokens.size()) * model.config
2004 ↪ .layer_count,
2005         "profiled GPT-2 layer step count mismatch");
2006     require(hotspot_profile.total_step_ms >= hotspot_profile.lm_head_ms,
2007         "profiled GPT-2 total time should include lm head time");
2008
2009     const std::vector<std::int32_t> cached_generated =
2010         lcqi::gpt2_generate_greedy_cached(model, tokens, 2);
2011     require(cached_generated == expected, "cached GPT-2 greedy generation
2012 ↪ mismatch");
2013 }

```

```

2010
2011 void test_gpt2_byte_bpe_tokenizer() {
2012     const std::filesystem::path vocab_path = temp_file_path("lcqi_gpt2_vocab.↵
↵ json");
2013     const std::filesystem::path merges_path = temp_file_path("lcqi_gpt2_merges.↵
↵ txt");
2014     const std::string space_marker = "\xC4\xA0";
2015     const std::string vocab =
2016         "{ \"Hello\":7, \" \":8, \"\" + space_marker + "world\":9, \"!\":10}";
2017     const std::string merges =
2018         "#version: 0.2\n"
2019         "H e\n"
2020         "He l\n"
2021         "Hel l\n"
2022         "Hell o\n" +
2023         space_marker + " w\n" +
2024         space_marker + "w o\n" +
2025         space_marker + "wo r\n" +
2026         space_marker + "wor l\n" +
2027         space_marker + "worl d\n";
2028     write_text_file(vocab_path, vocab);
2029     write_text_file(merges_path, merges);
2030
2031     const lcqi::Gpt2Tokenizer tokenizer =
2032         lcqi::load_gpt2_tokenizer(vocab_path, merges_path, -1, -1);
2033     std::filesystem::remove(vocab_path);
2034     std::filesystem::remove(merges_path);
2035
2036     const std::vector<std::int32_t> ids =
2037         lcqi::gpt2_encode(tokenizer, "Hello, world!");
2038     const std::vector<std::int32_t> expected{
2039         LCQI_GPT2_TOKENIZER_HELLO_ID,
2040         8,
2041         9,
2042         10,
2043     };
2044     require(ids == expected, "GPT-2 BPE ids mismatch");
2045     require(lcqi::gpt2_decode(tokenizer, ids) == "Hello, world!",
2046         "GPT-2 BPE decode mismatch");
2047 }
2048
2049 void test_gpt2_loads_hf_style_tiny_checkpoint() {
2050     const std::filesystem::path directory =
2051         temp_file_path("lcqi_gpt2_tiny_checkpoint_dir");
2052     std::filesystem::remove_all(directory);
2053     std::filesystem::create_directories(directory);
2054
2055     const lcqi::Gpt2ReferenceModel original = lcqi::↵
↵ make_tiny_gpt2_reference_model();
2056     write_text_file(
2057         directory / "config.json",
2058         R("{ \"model_type\": \"gpt2\", \"n_embd\": 4, \"n_head\": 2, \"n_layer\": 1, \"n_positions\" ↵

```



```

    ↪ :8,"n_ctx":8,"vocab_size":6,"n_inner":8,"layer_norm_epsilon":0.00001,"↪
    ↪ activation_function":"gelu_new","bos_token_id":0,"eos_token_id":5}));
2059 write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
2060
2061 const lcqi::Gpt2ReferenceModel loaded = lcqi::load_gpt2_from_directory(↪
    ↪ directory);
2062 const std::vector<std::int32_t> tokens{1, 2, 3};
2063 const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(loaded, ↪
    ↪ tokens);
2064 std::filesystem::remove_all(directory);
2065
2066 require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
2067         "loaded GPT-2 predicted token mismatch");
2068 for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
2069     require_close(result.logits[index],
2070                   LCQI_GPT2_LOGITS[index],
2071                   "loaded GPT-2 logit mismatch");
2072 }
2073 }
2074
2075 void test_gpt2_loader_rejects_bad_shape() {
2076     const std::filesystem::path directory =
2077         temp_file_path("lcqi_gpt2_bad_shape_dir");
2078     std::filesystem::remove_all(directory);
2079     std::filesystem::create_directories(directory);
2080
2081     const lcqi::Gpt2ReferenceModel original = lcqi::↪
    ↪ make_tiny_gpt2_reference_model();
2082     write_text_file(
2083         directory / "config.json",
2084         R("{\"model_type\":\"gpt2\",\"n_embd\":5,\"n_head\":1,\"n_layer\":1,\"n_positions\"↪
    ↪ :8,\"n_ctx\":8,\"vocab_size\":6,\"n_inner\":8,\"layer_norm_epsilon\":0.00001,\"↪
    ↪ activation_function\":\"gelu_new\",\"bos_token_id\":0,\"eos_token_id\":5}));
2085     write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
2086
2087     bool threw = false;
2088     try {
2089         static_cast<void>(lcqi::load_gpt2_from_directory(directory));
2090     } catch (const std::runtime_error&) {
2091         threw = true;
2092     }
2093     std::filesystem::remove_all(directory);
2094     require(threw, "GPT-2 loader should reject bad tensor shape");
2095 }
2096
2097 void test_reference_rms_norm() {
2098     const std::vector<float> input{3.0F, 4.0F};
2099     const std::vector<float> weight{1.0F, 0.5F};
2100     std::vector<float> output(2, 0.0F);
2101     lcqi::rms_norm(input, weight, 0.0F, output);
2102     require_close(output[0], LCQI_RMS_EXPECTED_0, "RMSNorm output 0 mismatch");
2103     require_close(output[1], LCQI_RMS_EXPECTED_1, "RMSNorm output 1 mismatch");

```

```

2104 }
2105
2106 void test_reference_rope() {
2107     std::vector<float> heads{1.0F, 0.0F, 0.0F, 1.0F};
2108     lcqi::apply_rope(heads, 1, 4, 1, 10000.0F);
2109     require_close(heads[0], std::cos(1.0F), "RoPE pair 0 cosine mismatch");
2110     require_close(heads[1], std::sin(1.0F), "RoPE pair 0 sine mismatch");
2111     require_close(heads[2], -std::sin(0.01F), "RoPE pair 1 negative sine ←
    ↪ mismatch");
2112     require_close(heads[3], std::cos(0.01F), "RoPE pair 1 cosine mismatch");
2113 }
2114
2115 void test_reference_kv_attention() {
2116     lcqi::DecoderConfig config;
2117     config.hidden_size = 4;
2118     config.query_heads = 2;
2119     config.kv_heads = 1;
2120     config.head_dim = 2;
2121     config.intermediate_size = 4;
2122     config.vocab_size = 4;
2123     config.max_context = 4;
2124
2125     lcqi::ReferenceKVCache cache(config, 1);
2126     const std::vector<float> key{1.0F, 0.0F};
2127     const std::vector<float> value{LCQI_ATTENTION_EXPECTED_0, ←
    ↪ LCQI_ATTENTION_EXPECTED_1};
2128     cache.append(0, 0, key, value);
2129     require(cache.filled_tokens() == 1, "KV filled token count mismatch");
2130     require_close(cache.key(0, 0, 0)[0], 1.0F, "KV key read mismatch");
2131
2132     const std::vector<float> query{1.0F, 0.0F, 0.0F, 1.0F};
2133     std::vector<float> output(4, 0.0F);
2134     lcqi::attention_decode(config, cache, 0, 0, query, output);
2135     require_close(output[0], LCQI_ATTENTION_EXPECTED_0, "attention head 0 dim 0←
    ↪ mismatch");
2136     require_close(output[1], LCQI_ATTENTION_EXPECTED_1, "attention head 0 dim 1←
    ↪ mismatch");
2137     require_close(output[2], LCQI_ATTENTION_EXPECTED_0, "attention head 1 dim 0←
    ↪ mismatch");
2138     require_close(output[3], LCQI_ATTENTION_EXPECTED_1, "attention head 1 dim 1←
    ↪ mismatch");
2139 }
2140
2141 void test_reference_decoder_end_to_end() {
2142     const lcqi::ReferenceDecoderModel model = lcqi::←
    ↪ make_tiny_reference_decoder_model();
2143     const std::vector<std::int32_t> tokens{1, 2, 3};
2144     const lcqi::ReferenceDecodeResult result = lcqi::run_reference_decode(model←
    ↪ , tokens);
2145     require(result.logits.size() == LCQI_REFERENCE_DECODER_VOCAB_SIZE,
2146             "reference decoder logits size mismatch");
2147     require(result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,

```



```

2148         "reference decoder predicted token mismatch");
2149     for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ↵
↵ ++index) {
2150         require_close(result.logits[index],
2151             LCQI_REFERENCE_DECODER_LOGITS[index],
2152             "reference decoder golden logit mismatch");
2153     }
2154 }
2155
2156 void test_reference_decoder_trace_contract() {
2157     const lcqi::ReferenceDecoderModel model = lcqi::↵
↵ make_tiny_reference_decoder_model();
2158     const std::vector<std::int32_t> tokens{1, 2, 3};
2159     const lcqi::ReferenceDecodeTraceResult trace =
2160         lcqi::run_reference_decode_with_trace(model, tokens);
2161
2162     require(trace.result.predicted_token == ↵
↵ LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
2163         "reference trace predicted token mismatch");
2164     require(!trace.checkpoints.empty(), "reference trace checkpoints missing");
2165
2166     const lcqi::ReferenceTensorCheckpoint& first = trace.checkpoints.front();
2167     require(first.name == "embedding", "first checkpoint should be embedding");
2168     require(first.token_position == LCQI_REFERENCE_TRACE_FIRST_TOKEN,
2169         "first checkpoint token position mismatch");
2170     require(first.dtype == "f32", "first checkpoint dtype mismatch");
2171     require(first.layout == "row_major", "first checkpoint layout mismatch");
2172     require(first.shape.size() == 1 &&
2173         first.shape[0] == LCQI_REFERENCE_TRACE_HIDDEN_SIZE,
2174         "first checkpoint shape mismatch");
2175     require(first.stride.size() == 1 && first.stride[0] == 1,
2176         "first checkpoint stride mismatch");
2177
2178     bool saw_logits = false;
2179     bool saw_kv_slot = false;
2180     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↵
↵ {
2181         if (checkpoint.name == "kv_cache_key_slot") {
2182             saw_kv_slot = true;
2183             require(checkpoint.shape.size() == 4, "KV checkpoint rank mismatch"↵
↵ );
2184             require(checkpoint.layout == "kv_contiguous", "KV checkpoint layout↵
↵ mismatch");
2185         }
2186         if (checkpoint.name == "logits") {
2187             saw_logits = true;
2188             require(checkpoint.token_position == ↵
↵ LCQI_REFERENCE_TRACE_TOKEN_COUNT - 1,
2189                 "logits checkpoint token position mismatch");
2190             require(checkpoint.shape.size() == 1 &&
2191                 checkpoint.shape[0] ==
2192                 static_cast<std::int32_t>(↵

```

```

    ↪ LCQI_REFERENCE_DECODER_VOCAB_SIZE),
2193         "logits checkpoint shape mismatch");
2194     require(checkpoint.values.size() == LCQI_REFERENCE_DECODER_LOGITS.↪
    ↪ size(),
2195         "logits checkpoint value size mismatch");
2196     for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.↪
    ↪ size(); ++index) {
2197         require_close(checkpoint.values[index],
2198             LCQI_REFERENCE_DECODER_LOGITS[index],
2199             "trace logits checkpoint mismatch");
2200     }
2201 }
2202 }
2203 require(saw_kv_slot, "KV checkpoint missing");
2204 require(saw_logits, "logits checkpoint missing");
2205 }
2206
2207 void test_packed_i8_kernel_matches_scalar() {
2208     for (const KernelShapeCase& shape_case : LCQI_KERNEL_SHAPE_CASES) {
2209         const lcqi::QuantizedLinearLayer layer =
2210             lcqi::make_deterministic_i8_layer(shape_case.input_size,
2211                 shape_case.output_size,
2212                 0.03125F);
2213         const lcqi::PackedLinearI8 packed =
2214             lcqi::pack_linear_i8(layer, shape_case.output_block_size);
2215         const std::vector<float> input = lcqi::make_deterministic_input(layer.↪
    ↪ input_size);
2216         std::vector<float> scalar_output(static_cast<std::size_t>(layer.↪
    ↪ output_size), 0.0F);
2217         std::vector<float> packed_output(static_cast<std::size_t>(layer.↪
    ↪ output_size), 0.0F);
2218         lcqi::linear_i8(layer, input, scalar_output);
2219         lcqi::linear_i8_packed(packed, input, packed_output);
2220         require(lcqi::max_abs_diff(scalar_output, packed_output) <= ↪
    ↪ LCQI_KERNEL_MAX_DIFF,
2221             "packed int8 kernel output differs from scalar");
2222
2223         const lcqi::PackedLinearI8 simd_packed =
2224             lcqi::pack_linear_i8(layer, LCQI_SIMD_TEST_BLOCK);
2225         if (lcqi::linear_i8_packed_avx2_available()) {
2226             std::vector<float> avx2_output(static_cast<std::size_t>(layer.↪
    ↪ output_size), 0.0F);
2227             lcqi::linear_i8_packed_avx2(simd_packed, input, avx2_output);
2228             require(lcqi::max_abs_diff(scalar_output, avx2_output) <= ↪
    ↪ LCQI_KERNEL_MAX_DIFF,
2229                 "AVX2 int8 kernel output differs from scalar");
2230         }
2231     }
2232 }
2233
2234 void test_f32_dot_kernel_matches_scalar() {
2235     std::vector<float> lhs(LCQI_F32_DOT_TEST_SIZE, 0.0F);

```

```

2236 std::vector<float> rhs(LCQI_F32_DOT_TEST_SIZE, 0.0F);
2237 for (std::size_t index = 0; index < LCQI_F32_DOT_TEST_SIZE; ++index) {
2238     lhs[index] = static_cast<float>(static_cast<std::int32_t>(index % 19U) ←
↵ + 1) *
2239         0.03125F;
2240     rhs[index] = static_cast<float>(static_cast<std::int32_t>(index % 23U) ←
↵ - 11) *
2241         0.0625F;
2242 }
2243
2244 const float scalar =
2245     lcqi::dot_f32_scalar_unchecked(lhs.data(), rhs.data(), lhs.size());
2246 const float dispatched = lcqi::dot_f32(lhs, rhs);
2247 require(std::fabs(scalar - dispatched) <= LCQI_F32_KERNEL_MAX_DIFF,
2248     "F32 dot kernel output differs from scalar");
2249
2250 if (lcqi::dot_f32_avx2_available()) {
2251     const float avx2 =
2252         lcqi::dot_f32_avx2_unchecked(lhs.data(), rhs.data(), lhs.size());
2253     require(std::fabs(scalar - avx2) <= LCQI_F32_KERNEL_MAX_DIFF,
2254         "AVX2 F32 dot kernel output differs from scalar");
2255 }
2256
2257 bool rejected_mismatch = false;
2258 try {
2259     (void)lcqi::dot_f32(std::span<const float>(lhs.data(), lhs.size() - 1U) ←
↵ , rhs);
2260 } catch (const std::runtime_error&) {
2261     rejected_mismatch = true;
2262 }
2263 require(rejected_mismatch, "F32 dot kernel accepted mismatched sizes");
2264 }
2265
2266 void test_f32_row_kernels_match_scalar() {
2267     std::vector<float> weights(LCQI_F32_ROW_TEST_INPUT_SIZE *
2268         LCQI_F32_ROW_TEST_OUTPUT_SIZE,
2269         0.0F);
2270     std::vector<float> input(LCQI_F32_ROW_TEST_INPUT_SIZE, 0.0F);
2271     std::vector<float> bias(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
2272     for (std::size_t index = 0; index < weights.size(); ++index) {
2273         weights[index] = static_cast<float>(static_cast<std::int32_t>(index % ←
↵ 29U) - 14) *
2274             0.015625F;
2275     }
2276     for (std::size_t index = 0; index < input.size(); ++index) {
2277         input[index] = static_cast<float>(static_cast<std::int32_t>(index % 13U ←
↵ ) - 6) *
2278             0.03125F;
2279     }
2280     for (std::size_t index = 0; index < bias.size(); ++index) {
2281         bias[index] = static_cast<float>(static_cast<std::int32_t>(index % 7U) ←
↵ - 3) *

```

[illegible]

```

2333     for (std::size_t index = 0; index < scalar_output.size(); ++index) {
2334         require(std::fabs(scalar_output[index] - avx2_output[index]) <=
2335             LCQI_F32_KERNEL_MAX_DIFF,
2336             "AVX2 F32 linear row kernel output differs from scalar");
2337     }
2338
2339     std::vector<float> scalar_subrange(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F)↵
↵ ;
2340     std::vector<float> avx2_subrange(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
2341     constexpr std::size_t LCQI_F32_ROW_TEST_SUBRANGE_BEGIN = 1;
2342     constexpr std::size_t LCQI_F32_ROW_TEST_SUBRANGE_END = 7;
2343     lcqi::linear_f32_rows_scalar_unchecked(weights.data(),
2344         input.data(),
2345         bias.data(),
2346         input.size(),
2347         LCQI_F32_ROW_TEST_SUBRANGE_BEGIN↵
↵ ,
2348         LCQI_F32_ROW_TEST_SUBRANGE_END,
2349         scalar_subrange.data());
2350     lcqi::linear_f32_rows_avx2_unchecked(weights.data(),
2351         input.data(),
2352         bias.data(),
2353         input.size(),
2354         LCQI_F32_ROW_TEST_SUBRANGE_BEGIN,
2355         LCQI_F32_ROW_TEST_SUBRANGE_END,
2356         avx2_subrange.data());
2357     for (std::size_t index = LCQI_F32_ROW_TEST_SUBRANGE_BEGIN;
2358         index < LCQI_F32_ROW_TEST_SUBRANGE_END;
2359         ++index) {
2360         require(std::fabs(scalar_subrange[index] - avx2_subrange[index]) <=
2361             LCQI_F32_KERNEL_MAX_DIFF,
2362             "AVX2 F32 subrange row kernel output differs from scalar");
2363     }
2364
2365     const lcqi::F32RowMax avx2_max =
2366         lcqi::max_dot_f32_rows_avx2_unchecked(weights.data(),
2367             input.data(),
2368             input.size(),
2369             0,
2370             LCQI_F32_ROW_TEST_OUTPUT_SIZE↵
↵ );
2371     require(scalar_max.row == avx2_max.row,
2372         "AVX2 F32 row max kernel selected a different row");
2373     require(std::fabs(scalar_max.value - avx2_max.value) <= ↵
↵ LCQI_F32_KERNEL_MAX_DIFF,
2374         "AVX2 F32 row max kernel value differs from scalar");
2375 }
2376 }
2377
2378 void test_f32_batch_row_kernels_match_scalar() {
2379     std::vector<float> weights(LCQI_F32_ROW_TEST_INPUT_SIZE *
2380         LCQI_F32_ROW_TEST_OUTPUT_SIZE,

```

```

2381         0.0F);
2382     std::vector<float> input(LCQI_F32_BATCH_TEST_SIZE * ←
    ↪ LCQI_F32_ROW_TEST_INPUT_SIZE, 0.0F);
2383     std::vector<float> bias(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
2384     for (std::size_t index = 0; index < weights.size(); ++index) {
2385         weights[index] =
2386             static_cast<float>>(static_cast<std::int32_t>(index % 19U) - 9) * ←
    ↪ 0.03125F;
2387     }
2388     for (std::size_t index = 0; index < input.size(); ++index) {
2389         input[index] =
2390             static_cast<float>>(static_cast<std::int32_t>(index % 23U) - 11) * ←
    ↪ 0.046875F;
2391     }
2392     for (std::size_t index = 0; index < bias.size(); ++index) {
2393         bias[index] =
2394             static_cast<float>>(static_cast<std::int32_t>(index % 5U) - 2) * ←
    ↪ 0.125F;
2395     }
2396
2397     constexpr std::size_t LCQI_F32_BATCH_ROW_BEGIN = 1;
2398     constexpr std::size_t LCQI_F32_BATCH_ROW_END = 16;
2399     std::vector<float> scalar(LCQI_F32_BATCH_TEST_SIZE * ←
    ↪ LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
2400     std::vector<float> dispatched(scalar.size(), 0.0F);
2401     for (std::size_t batch = 0; batch < LCQI_F32_BATCH_TEST_SIZE; ++batch) {
2402         lcqi::linear_f32_rows_scalar_unchecked(
2403             weights.data(),
2404             input.data() + batch * LCQI_F32_ROW_TEST_INPUT_SIZE,
2405             bias.data(),
2406             LCQI_F32_ROW_TEST_INPUT_SIZE,
2407             LCQI_F32_BATCH_ROW_BEGIN,
2408             LCQI_F32_BATCH_ROW_END,
2409             scalar.data() + batch * LCQI_F32_ROW_TEST_OUTPUT_SIZE);
2410     }
2411     lcqi::linear_f32_batch_rows_unchecked(weights.data(),
2412                                           input.data(),
2413                                           bias.data(),
2414                                           LCQI_F32_ROW_TEST_INPUT_SIZE,
2415                                           LCQI_F32_BATCH_TEST_SIZE,
2416                                           LCQI_F32_BATCH_ROW_BEGIN,
2417                                           LCQI_F32_BATCH_ROW_END,
2418                                           LCQI_F32_ROW_TEST_OUTPUT_SIZE,
2419                                           dispatched.data());
2420     for (std::size_t batch = 0; batch < LCQI_F32_BATCH_TEST_SIZE; ++batch) {
2421         for (std::size_t row = LCQI_F32_BATCH_ROW_BEGIN; row < ←
    ↪ LCQI_F32_BATCH_ROW_END; ++row) {
2422             const std::size_t index = batch * LCQI_F32_ROW_TEST_OUTPUT_SIZE + ←
    ↪ row;
2423             require(std::fabs(scalar[index] - dispatched[index]) <= ←
    ↪ LCQI_F32_KERNEL_MAX_DIFF,
2424                 "F32 batch row kernel output differs from scalar");

```

```

2425     }
2426 }
2427 }
2428
2429 void test_llama_gguf_batch_prefill_matches_token_path() {
2430     const std::filesystem::path path = temp_file_path("↵
↵ lcqi_tiny_llama_batch_prefill.gguf");
2431     write_tiny_llama_gguf_fixture(path);
2432
2433     const lcqi::LlamaGgufLoadedModel loaded =
2434         lcqi::load_llama_gguf_reference_model(path);
2435     const std::vector<std::int32_t> prompt{1, 2, 3};
2436     {
2437         const ScopedEnvVar disable_batch_prefill(↵
↵ LCQI_TEST_LLAMA_BATCH_PREFILL_ENV,
2438                                                     LCQI_TEST_FALSE_ENV);
2439         const lcqi::LlamaGgufGenerationResult token_path =
2440             lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
2441         const ScopedEnvVar enable_batch_prefill(↵
↵ LCQI_TEST_LLAMA_BATCH_PREFILL_ENV,
2442                                                     LCQI_TEST_TRUE_ENV);
2443         const lcqi::LlamaGgufGenerationResult batch_path =
2444             lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
2445         require(batch_path.generated_ids == token_path.generated_ids,
2446                 "batch prefill generated ids differ from token path");
2447         require(batch_path.predicted_first_token == token_path.↵
↵ predicted_first_token,
2448                 "batch prefill predicted token differs from token path");
2449         require(batch_path.prefill_steps == token_path.prefill_steps,
2450                 "batch prefill step count differs from token path");
2451         require(batch_path.hotspots.f32_fallback_calls < token_path.hotspots.↵
↵ f32_fallback_calls,
2452                 "batch prefill should reduce fallback linear call count");
2453     }
2454     std::filesystem::remove(path);
2455 }
2456
2457 } // namespace
2458
2459 int main() {
2460     try {
2461         test_tiny_mlp();
2462         test_tokenizer_contract();
2463         test_tokenizer_rejects_unknown_without_unk();
2464         test_safetensors_manifest_contract();
2465         test_safetensors_rejects_bad_offsets();
2466         test_safetensors_rejects_bad_dtype_shape_size();
2467         test_safetensors_reads_float_tensors();
2468         test_gguf_manifest_reads_q4_k_tensor();
2469         test_q4_k_dequant_and_dot_paths();
2470         test_ggml_mixed_dequantizers();
2471         test_ggml_quantized_f32_matvec_paths();

```



```
2472     test_q5_0_packed_matvec_matches_ggml_q8_0_path();
2473     test_llama_gguf_loader_and_generation();
2474     test_llama_gguf_default_packs_q5_0_tensors();
2475     test_llama_gguf_batch_prefill_uses_q5_0_f32_sidecar_path();
2476     test_llama_gguf_parallel_generation_matches_serial();
2477     test_llama_gguf_q4_direct_generation();
2478     test_llama_gguf_ggml_direct_generation();
2479     test_llama_gguf_selective_ggml_direct_keeps_q6_fallback();
2480     test_llama_gguf_q6_k_direct_experimental_path();
2481     test_gpt2_tiny_forward_and_generation();
2482     test_gpt2_byte_bpe_tokenizer();
2483     test_gpt2_loads_hf_style_tiny_checkpoint();
2484     test_gpt2_loader_rejects_bad_shape();
2485     test_reference_rms_norm();
2486     test_reference_rope();
2487     test_reference_kv_attention();
2488     test_reference_decoder_end_to_end();
2489     test_reference_decoder_trace_contract();
2490     test_packed_i8_kernel_matches_scalar();
2491     test_f32_dot_kernel_matches_scalar();
2492     test_f32_row_kernels_match_scalar();
2493     test_f32_batch_row_kernels_match_scalar();
2494     test_llama_gguf_batch_prefill_matches_token_path();
2495
2496     std::cout << "lcqi tests passed\n";
2497     return EXIT_SUCCESS;
2498 } catch (const std::exception& error) {
2499     std::cerr << "lcqi test failure: " << error.what() << "\n";
2500     return EXIT_FAILURE;
2501 }
2502 }
```


实践与代码总索引

本卷已经把实践任务、代码走读路线和完整代码清单合并到同一套内容目录。目录中不再存在一个独立的“源码卷部分”：构建入口、调用链、公共合同、tiny MLP、int8 kernel、reference decoder、GPT-2、工具脚本和测试文件都插入到相应主题之后，共同解释同一份 `books/cpu-volume-3/labs/linux_cpu_inference` 源码。

源码阅读任务

读者不需要再在实践卷和源码卷之间来回切换，也不需要先读完 16 个实践章再去末尾翻代码。实践章节提出任务，紧邻的代码走读解释完整实现，测试和 benchmark 章节给出回归证据；三者已经合并到这一本实践与代码卷里，共同构成一条可复查的工程证据链。

一、实践主题到代码走读的合并位置

- 第 1 章之后 插入“代码走读：构建入口、项目边界与调用链总图”，先把 CMake、README、模型资产、target、CLI、benchmark、trace、ledger、serving probe 和测试入口放到一张可复现地图里。
- 第 4 章之后 插入“代码走读：公共合同、Tiny MLP 与 Int8 Kernel”，把模型、推理、tokenizer、safetensors、reference decoder、GPT-2 头文件，以及 tiny MLP、基础 CLI、scalar/packed/AVX2 kernel 连起来。
- 第 10 章之后 插入“代码走读：Reference Decoder、Tokenizer、Safetensors 与 GPT-2”，把真实模型加载、byte-level BPE、safetensors、GPT-2 forward、cached KV 和 greedy generation 放在模型导入主题内部讲清楚。
- 第 16 章之后 插入“代码走读：工具、Smoke、Benchmark 与回归测试”，把 benchmark、trace、ledger、serving probe、SmolLM2 smoke、GPT-2 smoke、benchmark compare 和 CTest 回归证据集中收束。

二、最小复现命令

```
1 cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<↩>
   ↪ DCMMAKE_BUILD_TYPE=Debug
2 cmake --build books/cpu-volume-3/build/lcqi-debug
3 ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
4 make cpu3p-pdf
5 make cpu3p-pdf
```

完成这组命令后，读者应同时检查相邻实践章节的作业结果、对应代码走读、测试断言和报告字段，确认代码、测试、报告字段和章节讲解是一条证据链。

术语表

术语	实践含义
manifest	模型资产清单，记录 tensor name、dtype、shape、offset、checksum 和 tokenizer/配置版本。
shape verifier	在 kernel 运行前检查 shape、dtype、layout、量化 descriptor 和 state edge 是否匹配。
reference decoder	以清楚正确为目标的 decoder 实现，用来生成 trace 和 golden。
oracle	比较候选实现与 reference 的判定器，可使用精确相等或误差界。
trace checkpoint	推理链路中的可复查节点，至少包含 shape、stride、dtype、layout、checksum 和摘要值。
KV cache	attention 历史 K/V 状态，跨 token 存活，不能被普通 activation planner 复用。
packing	把权重改写成更适合热循环和 SIMD 的布局。
backend	执行计划落到机器的实现，例如 scalar、AVX2、AVX-512、GPU 或库调用。
fallback	后端能力不满足时显式降级到可验证路径，并记录原因。
SLO	服务目标，例如 TTFT、TPOT、queue wait、拒绝率和取消回收时间。